

Harald Störrle
Models of Software Architecture

Fakultät für Mathematik und Informatik
Ludwig-Maximilians-Universität
München

MODELS OF SOFTWARE ARCHITECTURE

Design and Analysis with UML and Petri-nets

Harald Störrle

Fakultät für Mathematik und Informatik
Ludwig-Maximilians-Universität München
Oettingenstr. 67
D-80538 München
Germany
Harald.Stoerrle@gmx.de

Dissertation zur Erlangung des akademischen Grades des Doktors der Naturwissenschaften an der Fakultät für Mathematik und Informatik der Ludwig-Maximilians-Universität München.

1. Berichterstatter: Prof. Dr. Martin Wirsing, LMU
2. Berichterstatter: Prof. Dr. Gregor Engels, Uni Paderborn

Eingereicht zur Begutachtung: 22.11.2000
Tag der Prüfung: 20.12.2000

Errant consilia nostra, quia non habent quo derigantur; ignorant
quem portum petat nullus suus ventus est.

L. Annaeus Seneca, Ep. Mor. ad Luc., IV, LXXI, 3

The way to a man's heart
is through his chest

Iain Banks, Use of Weapons

Summary

The architecture of a software system is now widely recognized as the key success factor when it comes to devising and maintaining product families, reusing software components, and evolving large-scale software systems. There are three groups of approaches towards this goal: Architecture Description Languages (ADLs), general purpose software design languages, and componentware/reuse. Each of these has had some success, but not quite as much as had been hoped—more than 30 years after the discipline of software engineering has emerged, the software crisis has not been solved. Productivity increases in the software industry have not kept pace with that of other industries.

We believe that the UML, ADLs, and software reuse experiences have not been properly combined so far, but could complement each other. A comprehensive approach implanting ADL and reuse concepts into the conceptual framework of the UML might amass the critical momentum. Thus, the main goal of the doctoral dissertation underlying this book has been to bridge the gap between the different communities by compiling the best parts of the different approaches, modify them so that they may be integrated, add some missing links, and weld them into a coherent, seamless whole. Pursuing this goal, we chose the following approach.

(1) The prime purpose of software architecture is to support the communication among *all* stakeholders. So, today, the Unified Modeling Language (UML) is the obvious starting point, as it is the most widely understood abstract notation for software: it has become the “*lingua franca of software engineering*”. But the UML still has many deficiencies, in particular the lack of adequate concepts, notations and tools for describing software at the architecture level.

(2) Such concepts, however, have been studied in various ADLs. While introducing a number of interesting concepts, complete with formal semantics and matching formal-methods based tools, classical ADLs paid little attention to practical considerations like concrete syntax, early development phases, and support for the architectural process and reuse.

(3) Reuse of software components, however, has been discussed as a possible solution to the *software crisis* since the 1968 Garmisch conference. While not entirely successful, this approach has yielded many interesting results, and is developed further continuously. In particular, pragmatic questions have been explored in numerous quantitative empirical field studies.

Integrating these aspects, this book first provides a compilation of the most versatile notions of various ADLs and other sources; the novel notion of architectural unit; an architectural modeling notation for them; and an embedding of all these concepts into the UML metamodel so that the other notations of the UML, its tools and methods can, be shared by the new concepts. Second, a formal semantics for the dynamic models of the UML (and some other aspects) is defined, that is by far more precise, complete and truthful than than most other approaches. It allows the definition of formal consistency conditions for architectural models, and thus design debugging. Finally, A language of architectural process patterns, with classification, precise and detailed description, and workflow diagrams to express their contents.

The most important aspect of our work, however, is not any single one of these, but their collective, the overall system of contributions under the overarching vision of software architecture.

Preface

It is now 15 years ago that I first stumbled over structured methods, and in particular GEI's Promod tool. I can vividly remember that, though not understanding much of what it was all about, I was immediately struck by the feeling that this was important stuff—up to then, I had been programming Assembler only. I had not received proper programming training, let alone exposure to theoretical computer science. The idea of visually specifying systems with tool support has never left me.

Then, over the years, I began to understand. Today I would like to think that, in all modesty, I have understood a fair bit indeed, so that what was a feeling then, has become a precise vision now, the vision of developing and maintaining software represented by diagrams, supported by powerful, semantics-based CASE tools. One would need adequate processes, too, of course, and libraries of prefabricated bits to be reused during design. With this ensemble, creating a system would become an exploration of the design space rather than a constant fixing, mending and patching.

Craftsmen and artisans were replaced by engineers and architects—programmers ought be replaced by software engineers and software architects. This is what this book is all about.

Acknowledgments First of all, I would like to thank my two supervisors, Gregor Engels and Martin Wirsing. Then, thanks go to my colleagues Alexander Knapp and Stephan Merz, whom I had the pleasure to work together with. Special thanks go to Alexander for incessant criticism and L^AT_EX-expertise. Then, I would like to thank all of my students, and particularly Christian Hansen for their work. Finally, I thank Serena Buchner for nursing me twice.

There are many more people that deserve mentioning at this point: I cannot list them all, so I apologize to those that have not appeared here.

Part of this work has been funded by the “Deutsche Forschungsgemeinschaft” through a grant of the “Graduiertenkolleg Logik in der Informatik.”

München, 12.1.2001

Contents

1. Introduction	1
1.1. Motivation	1
1.1.1. Software architecture—a vision	1
1.1.2. . . . and the reality	2
1.1.3. Means and ends	3
1.2. Approach	4
1.2.1. Software architecture	4
1.2.2. Architectural units	5
1.2.3. Transparent formality	6
1.2.4. Research objectives	7
1.3. Organization of this book	8
1.3.1. Interpreting the title	8
1.3.2. Straight sequential order	8
1.3.3. A software engineer’s view	9
1.3.4. Chapter dependencies	9
1.4. Reading advice	10
1.4.1. Prerequisites for reading	10
1.4.2. Typographic conventions	11
1.4.3. Using the apparatus	11
2. State of the art	13
2.1. Notions of software architecture	13
2.2. Notions of architectural units	14
2.3. Languages for architectural modeling	18
2.4. UML semantics	22
2.5. Architectural process	24
2.6. Chapter summary	27
I. Concepts and Syntax	29
3. Concepts for architectural modeling	31
3.1. Stakeholders and their stakes	31
3.2. Relationships between stakeholders	32
3.3. Further implications of architecture-based software construction	33
3.3.1. Development process and system life cycle	33
3.3.2. Organizational infrastructure	34
3.3.3. Component markets	34

3.4.	Usage scenarios	35
3.4.1.	Initial design	35
3.4.2.	The reuse cycle	36
3.4.3.	The implementation phase	38
3.4.4.	The maintenance phase	39
3.5.	Main concepts	41
3.5.1.	Views and viewpoints	42
3.5.2.	Architectural units	43
3.5.3.	Architectural glue and points of interaction	44
3.5.4.	Architecture	45
3.5.5.	Architectural style and product line architecture	46
3.6.	Chapter summary	47
4.	Logical metamodel	49
4.1.	Preliminary remarks	49
4.1.1.	A note on terminology	49
4.1.2.	Approach	49
4.1.3.	Synopsis of embedding	50
4.2.	Architectural structures	50
4.2.1.	Capsule	52
4.2.2.	View	56
4.2.3.	Port	60
4.2.4.	Configuration, Architecture	61
4.2.5.	Connector, ConnectorEnd	62
4.2.6.	ConfigurationSpace	63
4.3.	Instance level	66
4.4.	Pattern level	67
4.4.1.	ProtocolRole	67
4.4.2.	Protocol	67
4.4.3.	Abstraction	68
4.5.	Chapter summary	69
5.	Concrete syntax	71
5.1.	Syntactic Elements	71
5.1.1.	Port and PortInstance	71
5.1.2.	Capsule and CapsuleInstance	72
5.1.3.	Connector and ConnectorInstance	72
5.1.4.	Nodes	73
5.1.5.	ProtocolRole, Protocol	73
5.1.6.	Other metaclasses	73
5.2.	Usage of Capsule in diagram types	73
5.2.1.	Structural diagrams	74
5.2.2.	Behavioral diagrams	76
5.2.3.	Context diagrams	76
5.3.	Diagrammatic representations for viewpoints	78
5.3.1.	Viewpoints identical to diagram types	78
5.3.2.	Viewpoints derivable from diagram types	79

5.3.3.	Viewpoints without UML representation	79
5.3.4.	Recursive presentation of viewpoints	81
5.4.	First examples	82
5.4.1.	A simple POS-support system	82
5.4.2.	A call-by-call least cost router	83
5.5.	Styles	89
5.5.1.	Pipeline style	91
5.5.2.	Blackboard style	94
5.5.3.	Layers	94
5.5.4.	Compute/Control style	97
5.5.5.	Using a style	97
5.6.	Chapter summary	98
II.	Semantics	101
6.	Approach	103
6.1.	Overview	103
6.2.	Purposes	104
6.3.	Consistency checking	105
6.4.	Analysis targets	106
6.4.1.	Consistency of individual views	107
6.4.2.	Consistency among views	108
6.4.3.	Architectural refinement	109
6.4.4.	Quantitative analysis	109
6.4.5.	Code generation	109
6.5.	Semantic domains	109
6.6.	Semantic mapping	110
6.7.	Syntactic domain	111
6.8.	Adequacy	112
6.9.	Chapter summary	112
7.	Physical metamodel	113
7.1.	General principles	113
7.1.1.	Transition from logical to physical metamodel	113
7.1.2.	Textual representation	114
7.2.	PresentationElement	115
7.3.	Relationships	116
7.4.	Namespace	116
7.4.1.	ConfigurationSpace	116
7.4.2.	Capsule	117
7.5.	StateMachine	118
7.5.1.	Overview	118
7.5.2.	Differences to the logical metamodel	120
7.6.	Chapter summary	122

8. Semantic domains	125
8.1. General concepts	125
8.2. Low-level Petri-nets	126
8.2.1. Structure	126
8.2.2. Behavior	126
8.2.3. Semantics	127
8.2.4. Behavioral equivalences	129
8.2.5. Low-level net operators	133
8.2.6. Creating thread-like nets	134
8.2.7. Connecting nets asynchronously	134
8.2.8. Synchronizing nets	135
8.3. Chapter summary	136
9. Semantic mappings for models	137
9.1. Preliminaries	137
9.1.1. Notation	137
9.1.2. Elementary rules and rule-schemes	137
9.2. Event, Action, Operation, and Signal	138
9.3. Creating a sample run from Interaction	140
9.3.1. Intuitive semantics of Interaction	141
9.3.2. Mapping	141
9.3.3. Examples	142
9.4. Creating a Petri-net from StateMachine	144
9.4.1. Intuitive semantics of StateMachine	145
9.4.2. Loose semantics for StateMachine	147
9.4.3. Examples	152
9.4.4. Strict semantics	153
9.4.5. Example	159
9.4.6. Initial and terminal states	162
9.5. Semantics of Capsule and Style	163
9.5.1. Auxiliary functions	163
9.5.2. ProtocolRole and its users	164
9.5.3. Executable models from Configurations	165
9.5.4. Dependency	168
9.6. Quantitative models of Capsules	170
9.6.1. Size metrics	170
9.6.2. A simple behavioral complexity metrics	173
9.6.3. Stochastic analysis of buffer capacities	173
9.7. Chapter summary	175
III. Pragmatics	177
10. Consistency conditions	179
10.1. Expressing intuitive properties formally	179
10.1.1. "Is-run-of" relations	180
10.1.2. Well-behavedness of a ProtocolRole	180

10.1.3. "Implements" relation	181
10.1.4. "Is-equivalent" relation	181
10.2. Consistency within views	181
10.2.1. Interfaces	182
10.2.2. Functionality	182
10.2.3. Behavior	182
10.2.4. Structure	182
10.2.5. Protocol	183
10.3. Consistency among views	184
10.3.1. Overview	184
10.3.2. Interfaces vs. structure	184
10.3.3. Interfaces vs. behavior	186
10.3.4. Behavior vs. structure	187
10.3.5. Functionality vs. interfaces	187
10.4. Architectural refinement	188
10.4.1. Correct Abstraction between Capsules	188
10.4.2. Homogenous Abstraction among Views	190
10.4.3. Homogenous Abstraction among Capsules	190
10.4.4. Heterogenous Abstractions among Views	190
10.4.5. Heterogenous Abstraction among Capsules	191
10.4.6. Style vs. Capsule	191
10.5. Adequacy	191
10.6. Compositionality	191
10.7. Chapter summary	192
11. The architectural process	193
11.1. Introduction	193
11.1.1. Characteristics of architectural processes	193
11.1.2. Domains of pattern languages	194
11.1.3. Reconciling rigor and flexibility	195
11.2. Description of process patterns	195
11.2.1. Overall schema	195
11.2.2. Process modeling	197
11.2.3. Classification of process patterns	199
11.3. An architectural process pattern language	202
11.4. Chapter summary	225
12. Extended example	227
12.1. Specification	227
12.1.1. Context	227
12.1.2. Domain	230
12.1.3. Functionality	233
12.2. Design	234
12.2.1. Interfaces	234
12.2.2. Behavior	239
12.2.3. Structure	240
12.3. Implementation	240

12.3.1. Logic	240
12.3.2. Low level Capsules	243
12.4. Process patterns	246
12.5. Design debugging and maintenance	248
12.5.1. Tuning	250
12.5.2. Rearchitecting	250
12.6. Chapter summary	252
IV. Final Remarks	253
13. Results	255
13.1. Summary	255
13.2. Discussion	256
13.2.1. Results vs. goals—are the research questions answered?	257
13.2.2. Relevance—do the answers really matter?	258
13.2.3. Limits—how far does the approach carry?	259
13.3. Computer Aided Software Architecture—a vision revisited	260
A. The Unified Modeling Language	263
A.1. Introduction	263
A.1.1. Historical overview	263
A.1.2. Structural overview	263
A.2. Level 2: metamodel	264
A.2.1. Overview	264
A.2.2. An example: model management	265
A.2.3. OCL	267
A.3. Levels 0 and 1: models	267
A.3.1. Interaction diagrams	268
A.3.2. Activity and state transition diagrams	268
A.3.3. Use case diagrams	269
A.3.4. Static structure diagrams	269
A.3.5. Architectural structure diagram	269
A.3.6. Implementation diagrams	270
A.3.7. Inscriptions and OCL	270
A.4. UML 1.3 versus UML 1.4 and 2.0	270
B. Glossary	271
C. Rules	279
D. Viewpoints	283
References	284
Index	299

1. Introduction

1.1. Motivation

Software is now the backbone of almost all industrial and service processes. We see larger and more complex software systems being put into operation every day. At the same time, they also exhibit increasing rates of change. This has readily absorbed all progress that has been made in the history of software engineering, so that, today, building reliable large software systems on schedule and budget is still a major problem.

Consider e.g. the manufacturers of appliances like mobile phones and manufacturing equipment, or the providers of e.g. financial services. Typically, there is not just a single product or service access channel that is being offered, but a whole range, with varying features but great similarity, i.e., a *product line*. Reducing time to market and development costs over the whole product line is a prime objective. In addressing this issue, *software architecture* has attracted much attention in recent years: it promises component-based development and reuse of large portions of the software of individual products.

1.1.1. Software architecture—a vision

The term “software architecture” suggests an analogy to the classical notion of architecture (“building architecture”). It has been claimed, that this analogy is “*not just convenient or superficial [but] truly profound*”, and that it “*not only raises the right questions, it has the answers to what has been called ‘The Software Crisis’*” (see WWISA [1997]). Looking at building architecture, today we have

- stability:** buildings don’t just break down;
- usability:** buildings serve their purpose;
- planability:** the cost and time of building can be planned precisely;
- evolvability:** buildings can be adapted to changing requirements.

Even if sometimes buildings do break down, even if they sometimes do miss their intended purpose, and they are behind schedule and over budget, and even if the limits to changing a building might be quite strict in many cases—just imagine the software equivalent of the Potsdamer Platz construction site in Berlin after the German reunification.

And this is precisely our vision: being able to design and plan software architectures in much the same way as we design and plan buildings today.

In our vision, the software architect would create the overall design of a software system, focusing on system-level properties, on organizational issues related to the system, on the overall construction process from requirements to deployment rather than on technical details. He would have a deep understanding of the technical implications of his designs, however, though not necessarily expertise in the involved technologies. Thus, his role would be quite different to that of today’s software engineers or programmers.¹ He would have a vast array of materials, methods, and standardized, reusable com-

¹More detailed and vivid portraits of the imagined professional life of software architects are given in Kruchten [1999],

ponents from which he might choose, exploring and extending the design space in understanding, and serving the clients' needs, and enforcing their requirements in the actual realization. He would have the education and the tools to do his job, collaborating with the specialized colleagues of neighboring disciplines. He would be the "front end" of the overall software construction process.

1.1.2. ... and the reality

The reality, alas, is quite different: "*software architecture is now at a point identical to where building architecture was in the mid-1800's*" (see WWISA [1997]). Our vision encounters a verdant state of the art, to say the least.

Software architecture is today mainly used as a buzzword, and even many a seasoned practitioner seems to think that it is nothing but (coarse) system structure. The lessons learned from reuse—invaluable for component oriented software construction Braun [1999]—have been forgotten again, in particular concerning organizational and process support. Going into this in greater detail, the following deficiencies may be identified (see also Chapter 2 for an in-depth survey of the state-of-the-art).

Concepts There is a considerable body of concepts for architectural modeling, but it is scattered over various approaches. For instance, Darwin deals with the instance level while Wright deals with the type level; SDL deals with timing while Use Case Maps focus on causality. The Unified Modeling Language (UML) has a little bit of everything, but only in nascent and still very vague forms: viewpoint is absent altogether, view is just an informal concept, and SubSystem, albeit a proper metaclass, is very scarcely described, and poorly defined. The other classical concepts such as "configuration" and "glue" are missing entirely. Also, a notion of context is missing in UML, while it is featured in HOOD and the P1471 standard of the IEEE Architecture Working Group (AWG).

Notations Most Architecture Description Languages (ADL) only have textual notations, or ad-hoc diagrammatical visualizations. There are some approaches incorporating diagrammatical notations capable of expressing at least some aspects of architecture (e.g. SARA, ROOM, HOOD), but they, too, are scattered over various approaches and not integrated, and they are mostly ad hoc. Also, they lack the expressiveness and intuition of box-and-line diagrams (e.g. no provision for layered architectures). Finally, they lack the widespread acceptance of the UML's notations. Since communication is one of the core purposes of software architecture, obviously, using the UML is a *sine qua non*.

Semantics The meaning of the concepts and notations is not always very well defined. While for ADLs, this is typically not a problem, as they are typically based on formal methods, the industrial approaches are far behind in terms of clarity and precision.

Tools The previous two points are also reflected in the situation concerning tools: on the one hand, there are tools to support ADLs with back-ends for elaborate formal analysis, but weak front-ends for designing an architecture in the first place. On the other hand, there are industrial tools like Rose/RT (née ObjecTime Developer Toolset) and Rhapsody, that come with good front-ends, but functionally rather poor back-ends.²

Mills [1985], WWISA [1997].

²One notable exception here is ObjectGeode, a well-known SDL toolset. It supports both design and analysis of architectural models, but is incompatible with UML, and thus of little use.

Processes For ROOM, SDL and Use Case Maps, as well as for the ADLs, there is no or not enough process support. UML on the other hand does have significant process support (see e.g. D’Souza and Wills [1999], Jacobson et al. [1999], Rational RUP), but it does not address the relevant issues of software architecture, despite vociferous claims of the opposite.³ In particular, all tasks necessary for software reuse (procurement, evaluation etc.) and evolution (abstraction of reusable artifacts, using styles) are completely missing (more details of process requirements of reuse are found in Braun [1999], Foote and Opdyke [1995], Hesse [1997], Moreton [1990]). Also, the early phases are only poorly supported (see “context”, above), but they are of pivotal importance to software architectures.

In a nutshell: the academic community, on the one hand, has created ADLs like Wright, Darwin and Rapide, featuring advanced modeling concepts, precision and formal analysis. On the other hand, industrial users are more interested in understandability, expressiveness, and practical value for concrete applications, and so their focus is on notations and methodologies.

Thus, the two communities pursue complementary approaches, providing, in a way, the “back-end” and the “front-end”, respectively. Unfortunately, there is little interaction between the two communities, they are almost mutually exclusive. This book tries to bridge the gap, by providing both a conceptual framework, embedded into the industry de facto standard UML, and thus making it amenable to practical purposes, with process patterns supporting the pragmatics, while at the same time giving a rigorously defined formal semantics allowing for formal analysis.

1.1.3. Means and ends

Pursuing our vision, a number of questions arise immediately.

- What is software architecture, after all? What should and shouldn’t be considered part of an architecture, and why? Under which circumstances or for which ends?
- Mediating between various stakeholders: which concepts and notations are best suited for describing software architectures, both in planning and after-the-fact documentation?
- How can the use of these concepts and notations be supported by tools and development processes? Are there implications on the concepts and notations, if so, which?

Addressing these questions, the goal of this book is to provide an integrated, finely attuned system of concepts, notations, semantics, and pragmatic guidelines to support the design of architectural models of software systems and their subsequent analysis with computer based tools. Equipped in this way, we hope, a software engineer is empowered to tinker with large software systems on an abstract level, analyzing implications of design decisions before they have propagated to the code. In other words: is empowered to being a software architect in the true sense of the word.

As the means at our disposal, we emphasize the following four. First, the UML is the “*lingua franca of the software engineering community*” (cf. [Selic et al., 2000, p. v]). Thus, irrespective of all the disadvantages it may have, using the UML’s conceptual framework and notations is a pragmatic necessity. Second, there are several ADLs like Wright, Darwin, and SARA, but also the IEEE Standard on software architecture, P1471, on whose concepts we draw directly. Third, there is a body of knowledge from the software reuse community, including ample empirical evidence which must be taken into account also when dealing with software architecture. Fourth, there is a wealth of formal

³Catalysis, for instance, is advertised as “component based”, and the UP is said to be “architecture centric”.

methods that may be used as the formal underpinnings of these concepts. We use Petri-nets, which feature a number of properties that make them particularly amenable to the task at hand. Our approach is not at all dependent on this particular formalism, however.

1.2. Approach

We have stated the research objectives and given a brief outline of our approach. Now we elaborate and justify our approach. It rests on three pillars: (1) a notion of software architecture, (2) the notion of architectural unit as its building blocks, and (3) a novel approach to formal semantics of designs that is geared to automated software engineering and Computer Aided Software Engineering (CASE). Though each of these three elements is sensible and valuable in its own right, the big picture requires all three of them to be exactly the way they are. Actually, the pillars-metaphor is slightly misleading, as there really are three interwoven strands of argumentation, each of which refers to, builds upon, and supports the respective other two strands. The reader will just have to bear with us for a few pages.

1.2.1. Software architecture

There are many different accounts of what is and isn't a software architecture, and what its relevant properties, purposes and parts are. We discuss the spectrum of opinions in Section 2.1, while considering here only our own point of view.

In our understanding, the architecture plays a pivotal role in the construction of software systems: architectural models are the common ground of clients, architects, and software engineers. That is, the prime purpose of architectural models is to allow communication among all stakeholders. As a first consequence, this makes the use of the UML as our starting point inevitable: *“it is the lingua franca of the software engineering community”* (cf. [Selic et al., 2000, p. v]).

As a second consequence, this implies that architecture is also the basis for planning and controlling the construction process, and the construction proper. In fact, it turns out that software architecture is an issue throughout *all* phases of system construction, from the first conception (the “drawing on a napkin”-stage) via the analysis of context and requirements, through to design and implementation, and further into the evolution and maintenance of a system: *“A successful process encourages early generation of an overall system architecture—it is architecture centric”* (cf. [Larman, 1998, p. 437]) (emphasis in original text).

These considerations lead us to our working definition of software architecture, which we adopt and adapt from the IEEE AWG:

“Architecture is defined [...] as the fundamental organization of a system, embodied in its components, their relationships to each other and the environment, and the principles governing its design and evolution.”

[IEEE Architecture Working Group, 1999b, p. 13]

This definition will be refined in Chapter 3. It offers basically the same perspective as that underlying the classic slogan by Perry and Wolf: *“Architecture={Elements, Form, Rationale}”* (cf. [Perry and Wolf, 1992, p. 44]). This latter definition is intentionally abstract, so as to characterize as precisely as possible the set of *all* software architectures, see [IEEE Architecture Working Group, 1998, Q. 5–7, 11, 20–22]. The goal pursued in this book, however, is to provide concepts, notations, and processes to support working with architectural models. Thus, a much more detailed and specialized understanding of software architecture is called for, implying a much more verbose definition. We

will obtain it by specializing the individual terms of our working definition, e.g. by spelling out which *elements*, *forms*, and *rationales* we consider.

1.2.2. Architectural units

There is no concept in UML that can be used for architectural building blocks ("elements"): Component is a piece of code only with no content to the activities of earlier phases. SubSystems are as yet almost undefined, and there are a number of detail problems (see the section on SubSystem in Chapter 2 for a more detailed discussion). We introduce the notion of *architectural units* for this purpose. Architectural units are very similar to Commercial off-the-shelf (COTS) components. Since "*COTS-based development is, in fact, software reuse*" (cf. [Braun, 1999, p. 30]), reusability is the essential quality of architectural units.

The idea of systematic software reuse can be traced back to the 1968 Garmisch conference, the same occasion that is commonly singled out as marking the beginning of software engineering as a discipline.⁴ Software is, of course, not just code, and reuse is by no means limited to code: it extends to *all* kinds of knowledge related to the development of software, and includes requirements, domain models, specifications, designs, proofs, project plans etc. as well as code (cf. e.g. [Prieto-Diaz, 1994, Sec. 2.5.2], [Zendler, 1997, p. 69] or [McClure, 1992, p. 234]). Freeman even states:

"The use of program code alone has almost no value. It is entirely inconsistent to exhort developers to put more effort into the analysis and design activities and not attempt to reuse the information generated there."
Freeman [1983]

This fact is not just intuitive, it has also been confirmed empirically (see Matsumoto et al. [1991]). What's more, "*design reuse has the greatest potential leverage [...]*" (cf. [Biggerstaff and Richter, 1987, p. 48]) as

"[...] reuse of software-documents from the early phases is particularly interesting for reuse, because they capture the semantics of the application domain better than those of later phases."
[Zendler, 1997, p. 83, our translation]

Yet, today's so called component technologies components like, say, Java Beans and COM-components, but also Components in the UML are nothing but (commented) pieces of code. They are *programming components*, that is, low-level entities, suitable for programming, but not for the other phases of the system development life cycle:

"A higher form of software reuse is needed to overcome the inherent limitations of code reuse. [...] Therefore, we must think in terms of reusing higher-level software abstractions, such as program designs and architectures."
[McClure, 1992, p. 238]

So, what is called for are *software engineering components* (see Störrle [1999a,c]).

In order to effectively support reuse, an architectural unit must be a truly self-contained unit to allow for a separation of concerns, and thus for concurrent engineering. It is a complete system in itself, typically a functional unit in the real world, with its own life cycle and administration. It may describe a rather large portion of the real world, such as an automatic teller machine, an organizational unit, or a database. It is in many respects self-sufficient, or self-contained. It is independently developed

⁴See McIlroy [1969] for the original article by McIlroy, Schäfer et al. [1994] for an introduction, and Prieto-Diaz [1994] for a historical overview. A survey of tools for reuse and empirical results can be found in Zendler [1997].

(“concurrent engineering”) or otherwise procured, and may be analyzed, deployed and reused on its own. For different development phases and their different activities, different aspects of an architectural unit are relevant, so they need to have appropriate semantic representations for all relevant points of view on the system.

In order to allow for reuse of designs, however, a universally understood and expressive language for designs is needed. This has long been missing, but is now available: the UML.

1.2.3. Transparent formality

The availability of an adequate concept of architectural unit and a suitable software design language are useful in themselves, but their true potential can be exploited only with sophisticated tool support. Today's tools are basically editors, sometimes with some additional functionality like simulation or code generation—but nowhere near what is technically feasible today. The functionalities we have in mind can only be provided by formal methods, that is, they require semantic representations of designs.

Most formal methods based approaches pursued what Lano has called the “overt” approach to formality (cf. Lano [1995]): designs are expressed entirely or to a large degree in some formalism like CSP or Z. This is the approach taken by Wright and B, to name but two. It requires highly skilled personnel with a substantial background in formal methods. The effort of learning to use and actually using such tools effectively often outweighs the return in terms of increased productivity.

Lano contrasts this with a “covert” approach to formality, where all formality is hidden. Design diagrams are interpreted as formal specifications, but there are no changes to the design language. We prefer to call this approach *transparent formality*, as the presence and nature of the underlying formal model is completely transparent to the user. Lano admits that “*the covert approach [is] more immediately acceptable and usable within industrial software development [...]*” (cf. [Lano, 1995, p. 12]). We agree.

Addressing the needs of industrial software production, what is called for is push button-technology, i.e. solutions that use formal methods to provide advanced functionality, but hide the formalism and the technicalities. This does not work for all systems and for all properties, but it does work for a large number of interesting practical problems. And it comes virtually for free, so nothing is lost if it is not applicable in a particular case.

As our approach is squarely based on the UML, we need a formal semantics for (parts of) the UML. Up to now, almost all approaches to formalize (parts of) the UML share two fundamental flaws. First, they start at the diagram level, that is, they translate the concrete syntax into some semantic domain. However, semantics should be defined on the abstract syntax, that is, the metamodel (see Figure 1.1). Second, the translations are given in a very informal way, typically only by example. Both of these obstacles must be overcome in a transparent formality approach.

Consider an example. Intuitively, a developer might ask a question like: given a sequence diagram and a state transition diagram, does the sequence diagram represent a run of the state transition diagram? Such a question can be answered formally (see sketch).

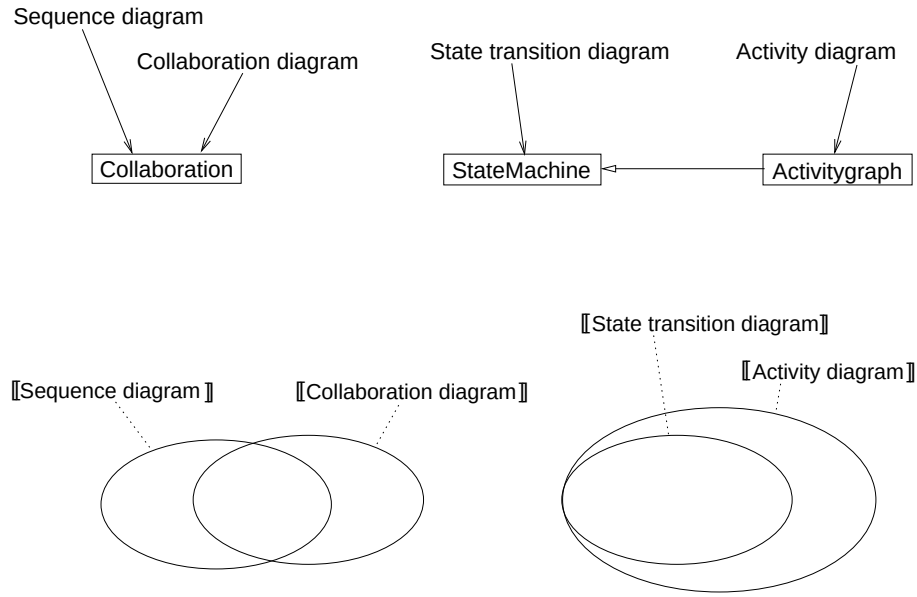
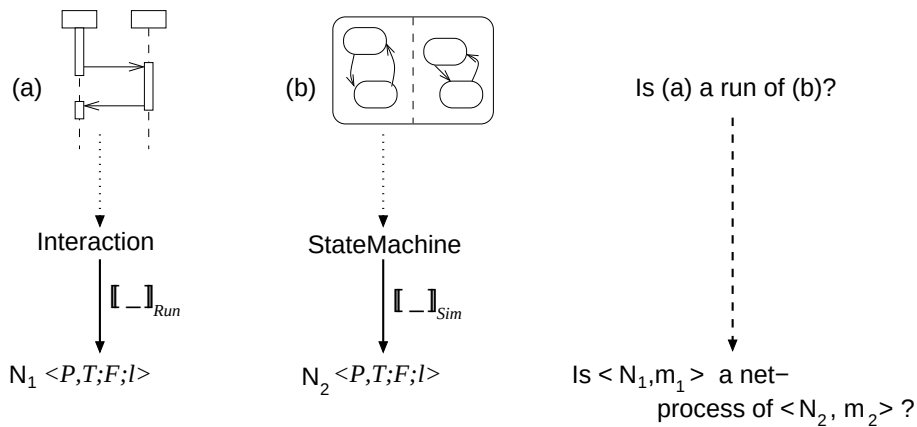


Figure 1.1.: A formal semantics of UML should be defined on the abstract syntax (i.e. metamodel) rather than the concrete syntax (i.e. diagram types): different kinds of diagrams map to the same metaclasses (top), so that the classes of models overlap (bottom). The semantic braces are used in a loose way.



Model fragments are expressed using the concrete syntax (the two tiny diagram-icons in the first row). We start from their abstract syntax representation, which are instances of the metaclasses *Interaction* and *StateMachine* (middle row). With a transparent-formality approach, these can be transformed automatically into appropriate semantical structures, say, Petri-nets (bottom row). Similarly, an intuitive property of the pair of diagrams can be translated into a formal property on the respective semantical model. In this case, the intuitive question of whether a sequence diagram presents a run of a state transition diagram is turned into the formal question of whether the one Petri-net is a net-process of the other.

1.2.4. Research objectives

Our approach translates into the following research programme. First, we ought to examine the context of architectural models according to our vision, i.e. we should identify the stakeholders and their

purposes by exploring usage scenarios. We then could define a notion of software architecture such that these concerns are addressed as good as possible. Special emphasis should be put on the perspective of the software architect.

Second, a digest of architectural modeling concepts must be compiled (drawing on, e.g. existing ADLs, ROOM and the IEEE standard P1471), with our notion of architectural unit as its cornerstone. This collection must be attuned and welded together to form a coherent, seamless whole. This requires an embedding of architectural modeling concepts into the UML metamodel such that as few changes and extensions as possible become necessary, so that the concepts, notations, processes and tools of the UML remain usable.

Third, we have to provide a formalization, so that the underlying formal methods are completely transparent to a potential user, so as to maximize the practical applicability of our work. We should identify the architectural properties that are relevant to the software architect, and define them formally, so that they, too, become amenable to automated analysis.

Fourth, we ought to describe (part of) a development process that integrates the concepts, notations, semantics, and properties tightly. We should provide examples that demonstrate the practical value of the concepts, notations, semantics, and properties we have created.

1.3. Organization of this book

1.3.1. Interpreting the title

Now take a little while to reconsider the title of this book: "Models of Software Architecture: Design and Analysis using UML and Petri-nets". Paraphrasing Herbert Simon here, we do not refer to software architecture but *models* of software architecture. It is these whose design and subsequent analysis this book is concerned with. And *design* and *analysis* are meant in the original sense, that is, as creation and examination rather than an earlier and a later phase in the software development cycle—which would make no sense at all, since we firmly believe that architecture is an issue throughout the whole lifecycle of a system. UML and Petri-nets, the last two items of the title, are the means we use to this end: UML for expressing architectural models, and Petri-nets as a semantic domain for such models that is particularly suitable for their investigation.

1.3.2. Straight sequential order

In Part I on "Concepts and Syntax", we first explore the notions connected with software architecture and define the concepts of architectural modeling (Chapter 3). They are embedded into the UML metamodel, so as to use as much of the UML as possible: its metamodel, notations, and thus also tools (Chapter 4). As the embedding is indeed quite smooth, only very few syntactical elements have to be added in order to cater for a whole new system of concepts (Chapter 5). We give a number of simple examples to demonstrate the expressiveness of our concepts, and the intuitiveness of our syntax.

In Part II on "Semantics", we first describe the overall approach, and give some examples of properties that may be analyzed on the basis of a formal semantics (Chapter 6). Then, the concepts introduced in Part I are given formal interpretations. Starting from a physical metamodel (Chapter 7), and an explanation of the semantical domains (Chapter 8), formal translation rules are given for all constructs of the abstract syntax and for several pragmatic circumstances (Chapter 9).

In Part III, we turn to "Pragmatics" of architectural modeling. We first give precise definitions of the properties to be analyzed using the semantics (Chapter 10), and sketch methodological guidelines

on the usage of the syntax, concepts, and techniques provided before (Chapter 11). Finally, as the quality of the concepts, syntax, semantics, and development guidelines is ultimately determined by their practical usability, we provide examples (Chapter 12).

1.3.3. A software engineer's view

The structure of this book can be seen as a development project, too, and the Parts become phases, and the Chapters become milestones within these phases.

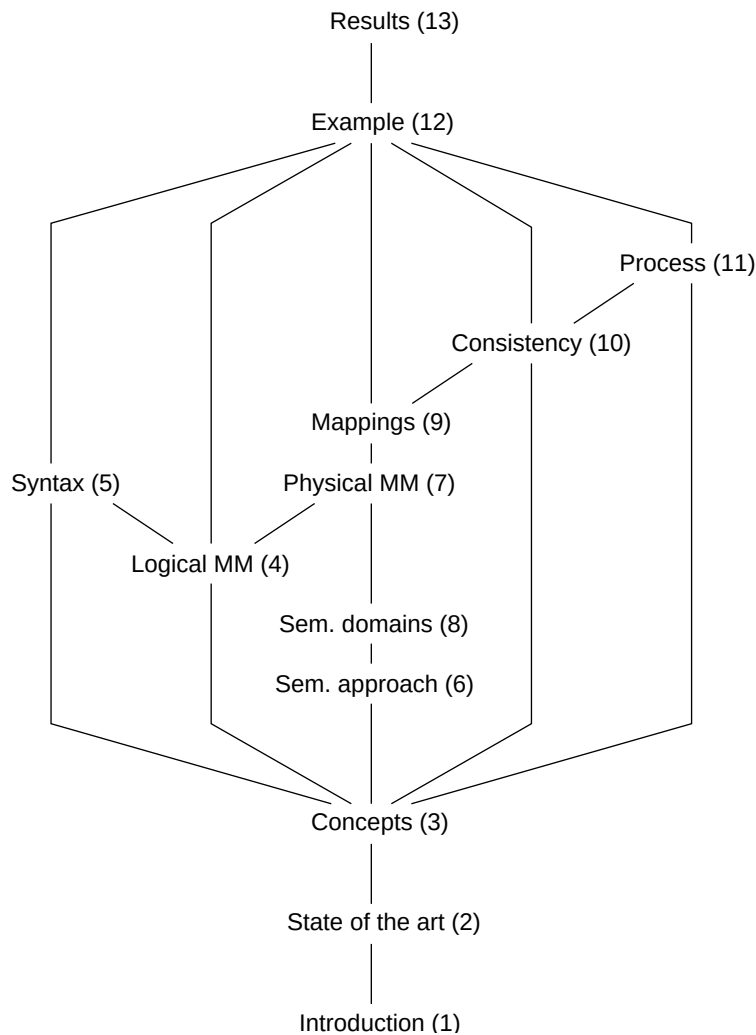
Seen this way, we start from an initial vision laid down in Section 1.1.1 as the research objectives. These questions are then turned into more concrete requirements (Chapter 1), triggering the analysis phase (Part I). The analysis begins with a definition of the context (i.e. the stakeholders, Section 3.1), the requirements (Section 3.3), a set of use cases (aka. scenarios, Section 3.4), and an initial, i.e., conceptual understanding of the application domain (Section 3.5). From the requirements, a logical model is developed (Chapter 4), and the analysis is completed by a customer-readable specification of the user interface (Chapter 5).

In the design phase (Part II), the further steps of the project plan are laid out (Chapter 6). Then, the logical model is transformed into a physical model, containing the data structures (Chapters 7 and 8) and functions (Chapter 9).

The development culminates in Part III, containing a kind of user guide with sample properties, a kind of tutorial, and examples (Chapters 10-12). The overall results of the “development project” are summarized and discussed in Part IV. At this stage, it is time to reconsider the initial vision and the requirements derived from it: Have they been adequately addressed in this book? We believe that this is indeed the case.

1.3.4. Chapter dependencies

The casual reader probably does not want to plough through this book from beginning to end in sequential order, but rather browse directly particular chapters. To support this, we show now the logical dependencies of the chapters, presented in the following graph. Read it as a Hasse-diagram, that is, starting at the bottom, and omitting transitive edges.



1.4. Reading advice

1.4.1. Prerequisites for reading

This book is not self-contained: some solid background in several areas of Software Engineering is required. We assume that the reader has a working knowledge of the syntax and semantics of the UML, as defined by OMG UML 1.3. In Appendix A we discuss some of the finer points of the UML, in particular wrt. its semantics. The reader might wish to consult it from time to time, when bits and pieces of the UML metamodel seem to be falling from the skies at random order. Familiarity with Petri-nets and other theories of concurrency are also helpful: though all used concepts are formally defined, we cannot provide a tutorial on this rich theory here. Also, a certain degree of familiarity with the usual ADLs is assumed.

This work combines several distinct areas of research, namely formal methods, software architecture, the UML, and CASE. Most readers will not be familiar with all of them to the same degree, so that in the interest of accessibility, we provide a wee overview of the fundamentals of UML in Appendix A and a glossary in Appendix B. It also contains a list of selected commented reading.

1.4.2. Typographic conventions

Throughout this work, we follow some typographic and stylistic convention.

- We use some abbreviations and acronyms in the text. Since most of them are standard abbreviations anyway, they are introduced upon their first usage. Also, they are explained in the glossary (Appendix B) and referenced in the index. Also, citations are referenced in the index by their authors.
- As far as the UML is concerned, we adopt the typographic conventions as stated in the standard document, save that we use the same font for figures and text, i.e. the name of a UML-metaclass always appears like this. This is particularly useful when the UML-terminology is in conflict with general terminology, e.g. to distinguish feature and Feature, or interfaces, Java-Interfaces and Interfaces. Also, association roles are always used in singular, even if their multiplicity is greater than one.
- We also adopt the conventions of the UML standard document for concepts defined here, i.e. extensions of the UML are treated exactly the same as native UML concepts. For instance, the names of associations with multiplicity greater than one are given in singular.

1.4.3. Using the apparatus

In order to increase reading effectiveness, in this book considerable effort has been put on the apparatus and the reader is kindly asked to make use of it. First of all, there is the table of contents: particular care has gone into the wording of section headings. It alone should suffice to quickly find the right place for every question addressed. Then Appendices A and B (Introduction to UML and Glossary) will be of particular interest for readers with only a casual interest. Appendix C presents a table of the semantics rules, giving the page of their definition, their argument and result type, and their usage in Chapter 10. This will be of use when it comes to understanding the finer points of the formal semantics and its applications. The references are quite extensive, and should cover pretty much all of the relevant literature in the field. The index contains terms, abbreviations, metaclasses and cited authors for easier reference.

2. State of the art

In this chapter, we survey existing approaches in five important areas: we consider notions of software architecture, notions of architectural units, architectural languages, UML semantics, and architectural processes in turn. Each section is structured alike: first, we define the criteria of comparison. Then we describe each of the contender in turn, followed by critique including a synoptical table with all criteria and all contenders.

2.1. Notions of software architecture

The term software architecture is currently very much en vogue. At the Software Engineering Institute's (SEI) web-site (cf. CMU SEI [2000]), about 75 different definitions are listed. As introductory reading, we recommend Clements and Northrop [1996] and Perry and Wolf [1992], both of which give an excellent overview. For more up to date resources, cf. Cattaneo [1996], CMU SEI [2000], Schneider [2000b]. An inspiring treatment of the aspects and purposes of architectures is found in Hammer [1997].

Criteria

The many definitions of software architectures are made for and by different kinds of people, who, according to their interest, emphasize different aspects. On the one hand, there is a varying degree of generality: what things are considered to be architectures? On the other hand, we are interested in the capacity for operationalization: can the definition be given a semantics, e.g. in some mathematical formalism, or at least in terms of the constructs of some programming language.

Contenders

Architecture for CEOs On the one hand, there are some very abstract and general accounts of architecture. Their generality allows to include everything one could possibly consider being an architecture. Following one particularly catchy slogan attributed to Tom DeMarco “*architecture is a framework for change*”. D’Souza and Wills state rather flatly that architecture is “*the set of design decisions [...] that keeps its implementors and maintainers from exercising needless creativity*” (cf. [D’Souza and Wills, 1999, p. 482]). Another classic is due to Luckham et al., who suggest that “*An architecture is a template for a family of systems*” (cf. [Luckham et al., 1995, p. 337]). Bronsard et al. [1997] describe software architecture as “*blueprints describing system composition*” (cf. [Bronsard et al., 1997, p. 19]). During the drawing-on-a-napkin stage of a development project, such vagueness is an important feature. The down side is poor operationalization, of course.

Architecture for programmers From a less elevated standpoint, other qualities of architecture come into view: low level definitions of architecture emphasize a system’s physical static

structure, plus a little behavior. Representative examples are provided by Luckham et al.: “[...] an architecture consists of a set of specifications [...] of modules, a set of connection rules that define direct communication between interfaces, and a set of formal constraints that define legal and/or illegal patterns of communication.” (cf. [Luckham et al., 1995, p. 337]), and by Larman: “[...] architecture refers to the high-level structure of subsystems and components, and their interfaces, connections, and interactions.” (cf. [Larman, 1998, p. 437]). The definition given by Catalysis is more abstract, but is still basically the same: “The architecture of a system consists of the structure(s) of its parts [...], the nature and relevant externally visible properties of those parts [...], and the relationships and constraints between them [...]” (cf. [D’Souza and Wills, 1999, p. 482]). Still the same definition, but from a formal methods rather than a programming point of view is given by Allen and Garlan: “a software architecture can be defined as a collection of computational components together with a collection of connectors, which describe the interactions between the components.” (cf. [Allen and Garlan, 1997, p. 219]). By providing concrete criteria and required/optional constituents of software architectures, these definitions inevitably restrict the class of things they can accept as software architectures, while preparing the ground for notations, tools, and processes.

Critique

We have listed two types of definitions: low-level definitions that focus on parts and properties, and high-level definitions that highlight the purposes of architectures and the people involved. We believe that both of these aspects are equally relevant, so that neither type of definition is sufficient in itself. Thus, we adopt a third, more comprehensive kind of definition, as given by the IEEE Architecture Working Group (IEEE AWG): “architecture [is] the fundamental organization of a system embodied in its components, their relationships to each other and to the environment and the principles guiding its design and evolution.” (cf. [IEEE Architecture Working Group, 1999a, p. 8, Def. 3.1.5]). Having a rather practical goal, however, we need to make this definition even more specific by focusing on particular architectural elements, forms, and rationales.

2.2. Notions of architectural units

The idea of a components and a component-oriented discipline of programming is as old as the term software engineering, cf. McIlroy [1969]. There have been numerous proposals as to what is the “right” notion of component, ranging from classical programming language concepts like package and class, via Business Objects (BO) to concepts from design languages.

Criteria

We identify the following criteria for fitness as a concept of architectural unit: (1) they must be able to represent architecture-level entities, i.e., large-scale and abstract entities; (2) they must be self-contained and strongly encapsulated entities with a footprint throughout the whole architectural lifecycle, i.e., during all development phases; (3) as a consequence of the previous two, architectural units must be units of analysis, design and maintenance as well as units of reuse. In other words, architectural units must cover all phases of a system lifecycle. Being the main building blocks of architectures, the precision, that is, the capacity for operationalization is also important for architectural units. Either, this can be realized by a formal semantics, or by a “hard-coded” semantics in the form of

a programming language mapping. Lastly, since our approach is UML based, the degree of proximity to the UML is an important aspect for us.

Contenders

Modules Starting in the 70's, module-concepts for large-scale units have been introduced in several programming languages (for a historical discussion, see [Louden, 1993, p. 265-267]). They bore names like Modules (Modula), Clusters (CLU) then, and Components (COM), Packages and Beans (Java) now. The purpose of these constructs is to provide units of abstraction, compilation, and loading (see [Wirth, 1983, pp. 80-81]), i.e. some, but not all of the aspects of encapsulation. The classes of object-oriented programming languages are technically quite similar to modules, but offer also inheritance and instantiation. Technologies like Java Beans or COM components are just classes with some additional constraints concerning naming conventions and usage.

Business Objects A BO represents an entity that is relevant to the application domain, encapsulating a piece of functionality or data with meaning to the end-user. They exist on a much higher level of granularity than the objects of programming languages, though they are usually realized as such. At least three different approaches to BOs have been proposed, namely IBM's Cooperative Business Objects (CBO) (cf. Sims [1994]), the Open Applications Group's (OAG) BOs (cf. Conelly [1999]), and the OMG's Business Object Component Architecture (BOCA) (see OMG BODTF [1998] for the current version and Flatscher [1999] for a historic overview). Both the CBO and the OAG BO approach seem to be deprecated, and so we focus on BOCA. The BOCA is to provide the "*Business Object Facility [i.e.] the infrastructure [...] required to support business objects operating as cooperative application components in a distributed object environment*" (cf. [OMG BODTF, 1998, p. 10]). The OMG claim that "*the combination of UML/BOCA and CORBA provides the closest known analogy to full-lifecycle transition free automation of information systems yet produced*" (cf. [OMG BODTF, 1998, p. 220]). It features a (textual) Component Description Language (CDL) and is aligned with UML, that is, there is a bijective mapping between the concepts of UML and BOCA.

Design language components There are also component-like concepts in design languages like UML Components, HOOD objects, and SDL blocks. For instance, in UML "*a component represents a physical piece of implementation of a system [...]*" (cf. [OMG UML 1.3, p. 2-29]), i.e. it is exclusively a piece of machine code, without any other aspects to it, such as a design-level specification, or a semantic representation of its structure, or proper behavioral interfaces. They are mainly the design-phase equivalent of modules in the original sense, sharing their disadvantages. So, these concepts do not realize architectural units in our sense.

ROOM actors The Real Time Object Oriented Modeling (ROOM) approach (see Selic et al. [1994]) introduces actors, that are connected via ports that abstract from the inner workings of the actor by playing a role in an explicitly modeled protocol. Actors may consist of a nested system of connected actors, resulting in a recursive structure. These concepts are expressive enough to be relevant for practical problems, demonstrated by the successful application of ROOM in industrial settings for many years. ROOM actors lack a formal semantics, but are operationalized by a tool. The ROOM-methodology cleverly selects

existing concepts and notations (e.g. the SARA-syntax) rather than contributing genuinely new ideas.

HOOD objects are “*models of a real-world entity or a software solution entity*” (cf. [Robinson, 1992, p. 31]), that “*may be an instance of a class*” [op cit., p. 32]. So, a HOOD object is *not* an object of an OO Programming language—it merely *may* be realized by one. It may have attributes, data and state, and may be composed of other HOOD objects. The interfaces of HOOD objects are sets of operations. Between HOOD objects, there are “uses” relationships, the operations of a compound object and the operations of its composite may be linked by “implements” relationships. The environment of an object may be described explicitly as an environment object. HOOD does have a bias towards Ada, but is not restricted to it.

UML-RT capsules In Lyons [1998], Selic and Rumbaugh [1998], an approach is described to integrate the ROOM concepts into the UML. Since both the traditional term “component” and the ROOM term for architectural units (actor) have already been used in the UML, UML-RT introduces the name capsule for architectural units. The descriptions have been very superficial so far, however. In particular, there is no tight embedding into the metamodel, and many important concepts are plainly omitted, or have no adequate syntax.

UML SubSystems Since version 1.3, the UML features the concept of SubSystem (see [OMG UML 1.3, pp. 2-174, 175, 178, 3-19–25])). Though only very sketchily defined, it seems to be very close to our understanding of architectural unit, and there are now additional sources like Övergaard and Palmkvist [2000] which confirm this point of view. Unsurprisingly, SubSystem and Capsule are close neighbors in the metamodel (see Figure 4.3), and Capsule could probably made a stereotype of SubSystem.

Critique

Programming-language constructs like modules, classes and JavaBeans are basically programming-level concepts, so their capacity to represent large-scale subsystems, throughout the complete lifecycle is quite limited. In other words, they are not self-contained, and they do not close the semantic gap between application domains and available technology. They are programming components, not software engineering components. On the other hand, reuse is most effective in the early phases (see Matsumoto et al. [1991]). In other words: reuse is more effective in the problem domain than in the program domain. True components would thus be high-level components—objects are not.

They also provide only a limited amount of encapsulation: modules, objects and Beans (1) offer a signature, not a behavioral type; and (2) they specify only the offered, not the required interfaces. Inheritance, the one characteristic difference between object-oriented and modular programming languages turns out to be a particularly problematic point. On the one hand, it does not occur naturally between architectural units (see Medvidovic [1999], Pfister and Szyperski [1998]). Even if it did, it could be replaced by delegation (cf. e.g. Frank and Halter [1996]). But, most importantly, inheritance, is simply not wanted in architectural modeling, as “*inheritance breaks encapsulation*” (cf. [Pfister and Szyperski, 1998, p. 5]), and encapsulation is essential for reuse (see also Buschmann et al. [1998], Pountain and Szyperski [1994] or [Pountain and Szyperski, 1994, p. 60]). Summing up, Udell flatly states that “*object technology failed to deliver in the promise of reuse*” (cf. [Udell, 1994, p. 46]). More precisely, the object-orientation paradigm is neither necessary nor sufficient to allow for effective reuse. For further discussions and motivations for architectural units different from objects and

classes, see Linden and Müller [1995], Nierstrasz [1991], Zendler [1998].

Closing the semantic gap is the explicit goal of BO-approaches. The deficiencies of classes are ameliorated by additional constraints. BOs try to be technology-independent, but fail to provide a formal semantics in stead of the “hard-coded” semantics given by a programming language mapping. Also, the existing approaches are not adequately aligned with the UML. Even the OMG itself concedes inadequacies in the current alignment of BOCA/CDL and UML: “*a totally isomorphic mapping between the UML and technology implementation targets has yet to be achieved and should be considered research.*” (cf. [OMG BODTF, 1998, p. 220]) (see also [OMG BODTF, 1998, Chapter 6]). To name but three deficiencies, the current alignment (1) relates BOCA to UML 1.1, not UML 1.3; (2) there are both important parts of UML and of BOCA/CDL which are not covered; (3) the UML lives on a much higher level of abstraction than the BOCA or CDL: BOCA-components (i.e. the entities described by CDL) are programming components, not design components. Since communication is a vital architectural concern, and the UML is *the* medium of communication in software engineering, this is a considerable disadvantage.

The concepts that come closest to our idea of architectural units are the notions of ROOM actor and UML SubSystem. As ROOM is now deprecated, and will be integrated into the UML, we focus on SubSystems in the remainder. While similar in intention and, apparently, in the pragmatics, there are also a number of decisive differences. Table 2.1 presents an overview of this comparison.

encapsulation SubSystems offer a set of Operations to their environment, which makes for a fairly weak encapsulation:

- SubSystems have no behavioral interfaces: Interfaces and Operations are just signatures, i.e. collections of uninterpreted symbols (cf. Liskov and Wing [1994]).
- SubSystems define only the inbound interfaces, not the outbound ones. There are no compulsory definition of dependencies to other SubSystems. References to other architectural units may be scattered throughout the realization of a SubSystem.

ROOM-actors or Capsules, on the other hand, feature Ports with ProtocolRoles, rigidly enforcing encapsulation. ProtocolRoles specify both inbound and outbound communications, and they use Signals to do so, i.e. they allow for asynchronous communication.

technology bias SubSystems use object-oriented features of the UML like Operations and inheritance. So, there is a certain amount of technology bias. Many legacy applications, however, are perfectly legitimate as architectural units, but do not fit into this paradigm. Capsules on the other hand use only Signals and delegation. Being tightly embedded into the UML metamodel, there are references to OO-concepts, but they are not relied on.

syntax The UML does not offer a convenient syntax for SubSystems—only the standard notations for Packages. For Capsules, on the other hand, both an abstract and a graphical concrete syntax are defined and illustrated by many examples.

semantics As with many parts of the UML, there is currently no consensus on the precise semantics of Packages. The additional semantical constraints for SubSystems are expressed in OCL and thus reasonably precise. Capsules, on the other hand, have both an explicit and clear natural-language semantics, and a formal semantics (see Chapter 9 below).

pragmatics Apart from Övergaard and Palmkvist [2000] and a couple of incidental remarks in the UP, there is currently no approach explaining in detail, how SubSystems may be used.

approach	encapsulation and self-containedness	architecture level abstraction	phase coverage	precision	proximity to UML
modules	(✓)	×	×	✓	(×)
OO-classes	(×)	×	(×)	✓	(✓)
Beans et al.	(✓)	(×)	(×)	✓	(×)
CBOs	✓	✓	(✓)	(✓)	×
OAG BOs	✓	✓	(✓)	(✓)	×
BOCA/CDL BOs	✓	✓	(✓)	(✓)	×
UML SubSystems	(✓)	✓	✓	(×)	✓
UML Components	(✓)	×	×	✓	✓
ROOM actors	(✓)	✓	×	✓	✓
HOOD objects	(✓)	✓	(✓)	(✓)	×
UML-RT capsules	(✓)	(✓)	×	(✓)	✓
SDL blocks	(×)	✓	×	✓	×

Table 2.1.: Approaches for architectural units.

2.3. Languages for architectural modeling

Criteria

The requirements elicited in Chapter 3 imply the following quality criteria for architectural modeling languages (AML): a good AML must feature (1) a comprehensive and adequate system of concepts for all relevant aspects of architecture; these concepts must be equipped with (2) a convenient syntax (preferably a visual notation); (3) a precise semantics (preferably a formal one); (4) process support; and (4) again acknowledging the paramount importance of the UML, a clear and operational relationship between to the UML.

We have given a short overview of which aspects of architectures we consider to be the relevant in Chapter 1. A full account follows in Chapter 3 below. For the time being, we just mention the capacity to explicitly represent both concrete architectures and architectural styles, concepts for views/viewpoints, and an account of a system's context.

Concerning semantics, obviously, a mathematically precise semantics is advantageous, but from a pragmatic viewpoint, a compiler-like tool is as good as a definition of a semantics—and for some circumstances, even a very precise natural-language text is adequate.

Contenders

We distinguish between three main kinds of AMLs. First, there are academically inspired ADLs. They often feature formal semantics, but lack notational and pragmatic support. A brief overview is presented in Medvidovic and Taylor [1997]. Second, there are more industrially inclined software design languages, like the UML or Specification and Description Language (SDL), offering powerful tools and processes, but lacking precise definitions. Third, there are extensions to the UML. We first deal with ADLs.

SARA Dating back to the late 70s, System Architect's Apprentice (SARA) is probably the oldest ADL (see Estrin et al. [1986]). It already introduces the graphical notation used many

times afterwards (e.g. in ROOM and here), and is based on a Petri-net semantics. It comes with extensive tool-support and a multi-view-concept. Its analysis targets include performance aspects.

Rapide Luckham et al. [1995] Rapide is an “*event-based, concurrent, object-oriented language specifically designed for prototyping system architectures*” (cf. [Luckham et al., 1995, p. 336]). It is based on a “*POSET Execution Model*” similar to Petri-net processes. It comprises tool-set for simulation and analysis of models. Specifications are written in an Ada-like textual form as a set of constraints. It does not seem to have a graphical notation of its own, but some sketches reminiscent of SARA are used for examples.

Wright Wright is mainly concerned with architectural styles rather than with concrete architectures (see Allen [1997], Allen and Garlan [1997]). Converse to our terminology, it seems to identify architectural style and ADL, which appears to be a confusion of concepts and syntax. Wright is based on CSP. It makes use of CSP-based tools for analysis purposes.

Aesop Aesop is also not really an ADL, but rather a style-description framework to generate style-specific design environments. So, its input formalism (a “fable”) is a kind of meta-ADL (see Garlan et al. [1994], see also the short characterization in [Medvidovic, 1999, p. 47]). It does not really fit *into* this comparison, but rather denotes one end of a spectrum.

Darwin While all of the aforementioned ADLs have their focus on the type-level, Darwin (see Magee et al. [1995]) emphasizes the instance-level.¹ Also, Darwin makes a point of structure being dynamic rather than static. Consequently, the formalism on which Darwin is based is the π -calculus.

C2/SADL Components-and-Connectors (C2) is an architectural style (see Medvidovic [1999]). The notion of style that the C2-authors use is not in line with our terminology. From our point of view, a style is an element in a language for architectural modeling. In C2-terminology, a style is a more general entity, that can be used to *derive* ADLs from it, as one example the Simple ADL (SADL). So, in a way, SADL is an instance of C2 like the UML metamodel is an instance of the MOF.

The second group of languages for architectural modeling have a more practical background.

SDL The SDL is an international standard (see SDL 2000 for the latest version). It has been widely used in telecommunications area, but only rarely outside this domain. It provides blocks as architectural elements. Among others, SDL uses Message Sequence Charts (MSC) to describe both scenarios and complete behavior ad hoc notation for static structure.

RM-ODP The Reference Model of Open Distributed Processing (RM-ODP) (see ISO RM-ODP [a,b,c,d]) is an internationally standardized reference model for telecom applications. In the terminology used in this book, it is a very detailed description of a style, defining five viewpoints (enterprise, information, computational, engineering, and technology viewpoints). It is not restricted to the telecommunications domain, but has found little acceptance outside this area. It is defined using various standardized telecommunication

¹The only other account of dynamic structure of software architecture that we know of is Allen et al. [1998].

notations (like SDL) and OMT. The reference model has the architectural elements nucleus, object, contracts, interfaces, and function. Unlike most other styles, it also provides a partial formal semantics using Z (and to a lesser degree, also the π -calculus).

HOOD The European Space Agency (ESA) has developed an object-based methodology called HOOD (Hierarchical OO Design) that uses HOOD objects as their mechanism for coarse grained, architectural modeling, see Robinson [1992]. HOOD uses of traditional data-flow diagrams and state transition diagrams, and introduces so called HOOD-diagram, essentially a restricted variant of traditional class diagram. They are also the basis for an early form of collaboration-diagrams, though there is no relation to role-modeling. Though this is not made a methodological point, separate and well-distinguished viewpoints are used consistently in HOOD. HOOD relies exclusively on objects for abstraction, introducing active and passive objects. It seems that the only form of aggregation available is composition.

The third group of languages for architectural modeling are directly building on the UML.

Selic and Rumbaugh [1998] have tried to integrate concepts from ROOM into the UML under the name UML for real-time (UML-RT) (see also Lyons [1998], Selic [1999]). In UML-RT, both architectural units and POIs are realized by $\ll active \gg$ Classes.

Egyed and Medvidovic [1999] present a very similar approach. They provide a mapping of C2-style architectures into UML, using stereotypes of Class and Association, and some OCL-constraints.

Kandé and Strohmeier [2000] present probably the most recent of approach in this line of work. They, too, provide a syntax reminiscent of ROOM, and build conceptually on the IEEE standard P1471. Concepts like viewpoint, view, and architecture are introduced by meta-modeling, that is, as first class citizens.

Hofmeister et al. [1999] is probably the earliest approach to embed AML-concepts into UML. They use stereotypes of Class, Component, Package for various views of architectural units, but fail to provide an integrated concept of architectural unit.

Critique

Most ADLs have a reasonably clear concept of architectural unit (usually under the name component) and architecture. Their focus is not on industrial applications, but on research, i.e., on exploring concepts and semantics rather than visualization, process support and so on. So, they usually include a formal semantics, but feature only textual or ad hoc graphical notations. Being strongly biased towards formal methods, using them requires great deal of theoretical background on behalf of the user. Thus, they are not usable for the majority of practitioners. Being older than the UML, they have no relationship to the UML by themselves. There have been attempts for embedding, but these have remained rather superficial (see the third group of contenders below). Garlan and Kompanek [2000] discuss some of the problems of integrating UML and ADLs. For surveys of ADLs see Cattaneo [1996], Schneider [2000b], and [Medvidovic, 1999, Section 3.9]. Apart from SARA and Rapide, none of these deal with system context.

The second group of contenders could also be called industrial AMLs. In contrast to the ADLs just discussed, they typically do not have a formal semantics and feature only a small set of architectural

concepts, but offer graphical notations, and sometimes complete development methods and integrated toolsets to support the development process. They are also older than the UML, but have a similar focus. Interestingly, the UML itself does not provide any useful definition of software architecture at all: “[architecture is] the organizational structure and associated behavior of a system.” (cf. [OMG UML 1.3, p. B3]) In the UML, most concepts of architectural modeling are either missing altogether (viewpoints, configuration etc.) or are there as vague concepts but not as first class citizens (i.e. as part of the metamodel, e.g. view), or would have to be simulated somehow (e.g. protocol, and protocol-role). None of the industrial AMLs have a concept of architectural style. Concerning concepts for architectural unit, see previous section.

The third group of approaches is the most recent, which is trying to integrate ADLs or industrial AMLs into the UML. All of the mentioned approaches provide only superficial and incomplete treatments. They all try to show, how the basic AML concepts (i.e. the C2-style) can be mapped to the UML, none of them provides a real embedding into the UML metamodel. They all map the main concepts to stereotypes of Class (plus, possibly, OCL constraints). In other words, they map architecture-level AML concepts to implementation-level concepts of the UML, irrespective of the implied technological bias. UML’s SubSystem are widely ignored. None of these approaches shows how a reasonably precise semantics could be defined. The approaches in this group inherit to a certain deficiencies of the UML, most notably the absence of a proper concept of view/viewpoint, architectural style, and system context. Table 2.2 presents an overview of this comparison. Other comparisons are presented in [Medvidovic, 1999, p. 51], Cattaneo [1996] and Allen [1997].

Approach	concepts				expressive notations		early phases	precise semantics	process support
	view	style	POI	context	structure	behavior			
SARA	✓	×	(✓)	(✓)	s/–	✓	×	✓	×
Rapide	×	(×)	✓	×	s/–	✓	×	✓	×
Wright	×	✓	✓	×	s/–	✓	×	✓	×
Aesop	×	✓	✓	×	s/d	✓	×	✓	×
Darwin	×	(×)	✓	×	s/d	✓	×	✓	×
C2/SADL	×	✓	✓	×	s/–	✓	×	✓	×
SDL	(✓)	×	(✓)	×	s/(d)	✓	(✓)	(✓)	✓
RM-ODP	(✓)	×	×	✓	s/(d)	✓	(✓)	✓	✓
UML	(✓)	×	×	×	(s/d)	✓	(×)	(×)	✓
ROOM	(×)	×	✓	(×)	s/–	✓	(×)	✓	(✓)
UML/RT	×	(×)	(✓)	(✓)	(s/d)	✓	×	(×)	(×)
HOOD	(×)	×	(✓)	(✓)	s/–	✓	(✓)	(✓)	(✓)
Egyed et al.	×	(×)	(×)	×	(s/d)	✓	×	(×)	(×)
Kandé et al.	×	(×)	(×)	×	(s/d)	✓	×	(×)	(×)
Hofmeister et al.	×	(×)	(×)	×	(s/d)	✓	×	(×)	(×)

Table 2.2.: Approaches to modeling of software architecture. In the column-structure, we use the abbreviations static and dynamic. The approaches UML/RT, Egyed et al., Kandé et al., Hofmeister et al. refer to Selic and Rumbaugh [1998], Egyed and Medvidovic [1999], Kandé and Strohmeier [2000], Hofmeister et al. [1999]

2.4. UML semantics

Criteria

There are a number of criteria to compare formal semantics of UML. We have omitted those that are irrelevant for the pursuit of our vision.

UML based First of all, many we state whether a semantical approach is based on the UML at all—many are based on precursors which share concepts with the UML.

Domain From a formal point of view, the semantics is mapping. It may use as its domain either the metamodel or a notation, that is, the abstract or the concrete syntax. For the reasons detailed in Section 1.2.3, we find it extremely important that semantics use the metamodel as their domain.

Completeness A mapping may cover all diagrams or models of a given kind, or just special cases, i.e. it may achieve different degrees of completeness.

Precision The definition may be given with varying precision, ranging from mere examples to a complete algorithm. Pursuing a transparent approach to formal methods, only mappings with high precision can be considered.

Scope There is a fundamental difference between semantics of static and dynamic models, and in order to integrate both, they have to be within the scope of a semantics.

Codomain The semantical mapping may use any of a vast array of different formalisms as its codomain, determined by purpose and focus.

A number of combinations of the above criteria are (currently) impossible to achieve, some are difficult to distinguish. For instance, most approaches have both an analytic and a constructive aspect to them, while it is virtually impossible to focus on a system of notations/metaclasses and still cover all of UML's features.

According to this terminology, the semantics we have provided in Chapter 9 is constructive, metamodel-based, transparent and precise. It focuses on systems of metaclasses for the dynamic aspect and uses Petri-nets as its basis (see the last row of Table 2.3). In the remainder of this section, we discuss only approaches, that match ours in at least one of these aspects. This excludes most of the formal semantics for individual software engineering notations that have been proposed over the years, and whose descendants have made their way into the UML.

Contenders

Lano [1995] presents a system of integrated semantics based on the system model of Z++/VDM++. The semantics he provides are not focused on UML, though, and neither precise², nor transparent.

Moldt [1996] presents a well-balanced system of related *colored Petri-nets* semantics, dealing with state charts and class diagrams (and some structured analysis diagrams). His focus is

²Consider for instance the treatment of State Transition Diagrams (p. 219ff) that leaves out exactly the interesting features, i.e. interrupt arcs and history states.

clearly on pre-UML notations, and the semantics provided are only by example. The approach is quite similar to ours in that a large number of consistency conditions are defined, but they are only of syntactic nature. It does not exploit the formal semantics for any pragmatic purposes.

Rumpe [1996] considers class diagrams and dynamic models, but replaces state transition diagrams by hierarchical automata, and provides a stream-semantics for them. The semantics are neatly integrated, yet incomplete and not at all dealing with UML.

Andersen [1997] presents an informal (yet rather precise) semantics for roles, replacing StateMachines by labeled transition systems. His work does not provide a semantics for diagrams, nor for (parts of) the metamodel.

Knapp [2000] defines three formal semantics of Collaborations, cast in LTL, Maude and Event spaces. These semantics are shown to be implementing each other. The emphasis is on exploring the true meaning of Collaboration, not on tool support or integration with other aspects of models. So, while the semantics is very precise, and based on the metamodel of UML 1.3, its purpose is the understanding of UML rather than making it available for practical, constructive purposes. Also, its scope is rather limited, as it deals with a single metaclass only, not with a system of interrelated metaclasses.

Lilius and Paltor [1999a] present one of the few really practical approaches that come with automated tool support: they provide a set of matching semantics for interaction and state transition diagrams which are translated into PROMELA in order to be analyzed by the SPIN model-checker. On the down side, they do not provide a translation from concrete or abstract syntax to semantics, but start with their own formalism, which is approximately on the level as the result of the translation we have presented above. So, they do not really provide a semantics, but rather use one. Also, many features are left out in their approach (in particular history states), contradicting to their claims (cf. Lilius and Paltor [1999b]).

Gehrke et al. [1998] is another Petri-net based approach, dealing with collaboration and activity diagrams. They focus on understanding the metamodel, however, and on implications for the development process. Their translations are given by example only, and not in sufficient detail. It is interesting, though, that the semantical structures they provide for interactions are very similar to those we came up with.

Firley et al. [1999] focus on real-time annotations to interaction diagrams. Their semantics is given by example only, and is not metamodel-based.

There are many more interesting publications on this subject matter, most notably Reggio et al. [2000], which is probably the most complete and up to date account of this field of study.

Critique

It is safe to say, that currently, the formal semantics of UML is an entirely unsolved problem. There are many approaches for individual aspects, and even the first approaches that cover more than one aspect, but neither are all aspects covered in this way, nor are the approaches presented precise enough, nor are they in any way acceptably comprehensive.

Most approaches start from the concrete syntax and leave out exactly the interesting features. They are mostly defined by example, that is, in a very superficial way. Lilius and Paltor [1999a] start

from an abstraction level, which is already quite semantical, but do not explain how they got there in the first place. The notable exception to this rule is Knapp [2000]—but this approach is restricted to a very narrow slice of the UML. Table 2.3 (top) presents an overview of this comparison.

Every semantics has its purpose. Most of those we have mentioned serve to study the UML, and they do serve their purpose. Dealing with translation technicalities does not help elucidating the UML semantics. Our purpose, however, is not the penultimate understanding of and insight in whatever crumpets of software engineering wisdom may solemnly slumber in the behemoth of paper that is the UML standard. Our purpose is to demonstrate that, and how, ADL concepts may be integrated into the UML seamlessly, provided with a semantics to allow for powerful automated tool support. Here, translation technicalities do matter. In fact, they are instrumental. No existing semantics addresses this point satisfyingly.

UML disciples of strict observance might find our approach too lenient, if not downright heretic, as we are more interested in the big picture than in minute details.

2.5. Architectural process

Criteria

An “*architecture-centric*” process is one in which “*a system’s architecture is used as a primary artifact for conceptualizing, constructing, managing and evolving the system under development*” (cf. [Jacobson et al., 1999, p. 443]). Thus, the architecture must be created and used with due diligence, that is, the activities involved need to be well-defined and precisely described. So, there has to be an adequate notion architecture in the first place; when in the overall process is an architecture defined; by whom is it used in what way and to what purpose in which activity, which viewpoints does it contain; does it deal with architectures, styles and product lines alike; if it is structured in patterns, how many facettes of classification and description are used, and so on.

Then, the extent to which the subprocesses procurement, reuse, and maintenance are addressed is important (the standard development subprocesses are not considered). There should be notions of architectures, product line architectures and styles. Additionally, all relevant stakeholders should be addressed explicitly. Obviously, there should be the roles of software architect and client. There might be (special cases of) “developers” as well (e.g. reuse engineer, quality assurance engineer etc.), and of course users. Then, we pay attention to whether process and its product have a recursive (or fractal) structure. Finally, only technology neutral processes are widely applicable—if a process relies on, say, an object-oriented approach being taken, then this is definitely a strong bias, and restricts the versatility of an approach.

Contenders

Catalysis (see D’Souza and Wills [1999]) is a development method emphasizing the (re)use of components, and frameworks. It is based on UML 1.1, but makes substantial modifications and extensions, both syntactically and semantically. Interestingly, Catalysis uses process patterns to describe its development process, thus establishing an isomorphy between the process and its product. The process patterns are described with two to three slots (Intent, Considerations, Strategy), filled with prose.

Unified Process The UP (see Jacobson et al. [1999]) is a general-purpose full-scale software development process, and is by no means restricted to architectural design. It does claim to

Approach	UML based	Metamodel as domain	completeness	precision	scope	codomain
Lano [1995]	×	×	×	×	s/d	Z++, VDM, and others
Moldt [1996]	×	×	✓	×	s/d	colored Petri-nets
Rumpe [1996]	×	×	✓	✓	s/d	streams, automata
Andersen [1997]	×	×	×	✓	s/d	labeled transition systems
Lilius and Paltor [1999b]	✓	×	(✓)	(✓)	-/d	PROMELA
Gehrke et al. [1998]	✓	×	×	×	-/d	Petri-nets, transition systems
Firley et al. [1999]	✓	×	×	×	-/d	automata
Knapp [2000]	✓	✓	(×)	✓	s/d	Maude, LTL, event spaces

Approach	precision	proc.	reuse	extent	a	s	p	all stake-holders	recursive product	recursive process	technology neutral
Catalysis	(✓)	×	(✓)	(✓)	✓	(×)	×	×	(✓)	(✓)	×
Unified Process	✓	×	(×)	(×)	✓	(✓)	×	✓	(×)	×	×
HOOD	(✓)	×	(✓)	(×)	✓	×	×	(×)	✓	✓	(×)
ROOM	(✓)	×	(✓)	(×)	✓	(✓)	×	×	✓	×	(×)
Hesse [1997]	×	×	(✓)	(✓)	✓	×	×	×	✓	✓	✓
Bayer et al. [1999]	(✓)	×	✓	(✓)	×	(✓)	✓	×	×	×	✓
Braun [1999]	✓	(✓)	✓	×	✓		(✓)	×	(✓)	(✓)	✓
Atkinson et al. [1998]	✓	×	(✓)	(✓)	✓	×	×	×	✓	(✓)	✓
Ochs et al. [2000]	✓	✓	✓	(✓)	✓	(✓)	(✓)	×	(✓)	(×)	✓

Table 2.3.: Approaches to UML semantics (top). The second last column shows, what kinds of models are mapped (static an/or **d**ynamic ones). Approaches to architectural processes (bottom). In the last three columns of “extent”, we use the abbreviations **a**rchitecture, **s**tyle, **p**roduct line architecture; proc. and maint. abbreviate procurement and maintenance, respectively.

be architecture-centric, though, and tries hard to live up to this ideal. It acknowledges that different stakeholders need different (sets of) views. The role of architect is elaborated (see [Jacobson et al., 1999, p. 324]). A small example, quite similar to that in Chapter 12 below is given on pages 78-81. Interestingly, the UP uses SubSystems as architectural units in this example.

HOOD (see Robinson [1992]) is a practical approach focusing on embedded systems in projects with high quality requirements, such as infrastructure, space and aviation projects. Both a system's architecture and its context are addressed explicitly, and result in a "Architectural Design Document" which contains a DFD-style context and top-level functionality description. HOOD addresses reusability and maintainability. It is geared towards Ada, and features extensive tool-support. HOOD offers a recursive process, and recursively structured products.

ROOM (see Selic et al. [1994]) is also a field-proven methodology. It has been used most in the domains of real-time, embedded and telecommunication systems. It does not provide a process proper, but it gives many examples that amount to a cookbook-style template process.

Hesse [1997] presents some interesting ideas about fractal processes and reuse, though in a very abstract and unprecise way.

Bayer et al. [1999] is a "*methodology for developing software product lines*". It is rather precisely defined, and covers procurement as well as reuse and evolution. It is technology neutral and comes with extensive tool support.

Braun [1999] presents a process for the reuse of COTS software. It is defined in a reasonably precise way (in an Integration Definition for Function Modeling (IDEF) like format, see National Institute of Standards and Technologies [1993]), though little detail is provided in Braun [1999]—we understand that there is a more detailed, yet unpublished description.

Atkinson et al. [1998] present a "*fractal-like process*" that results in a "*recursive product model*". The description given is quite superficial, but there are related publications like Ochs et al. [2000], which deals with a detailed procurement.

Critique

Almost all current software development processes pay lip service to the importance of software architecture, but few take it serious. For instance, Catalysis is not very closely related to the UML—the many or modified concepts are not properly integrated with the rest of the UML. Admittedly, this would have been a rather ghastly and futile task, given the state of the version 1.1 of the metamodel, and the difference to the 1.3 version. Though we definitely applaud the pattern-structure of its process, we have to criticize both the small amount of patterns that are directly related to architecture, and the poor degree of precision used to describe them. The traditional design patterns are structured by much more slots (13 in Gamma et al. [1995], 14 in Buschmann et al. [1998], respectively), some of which may be described in much greater detail and formality, in UML, say.

HOOD and ROOM, though a little dated, are architecture-based by heart: architecture is a central concern, and it is integrated in their concepts, and not just added on top, as in Catalysis. Both ROOM

and HOOD are pre-UML, however, and thus have no clear relationship to it (cf. the section on AMLs above).

The UP also really integrates the notion of software architecture. However, “architect” is only one of many roles involved, and a comparatively minor one. Also, architecture related activities are scarce, and a little vague. This seems to be inconsistent with the emphasis that is put on architecture throughout the UP. Reuse and evolution are not addressed at all on the architecture level, and there is a strong technology bias towards object-oriented technology. Many details are left unclear. Some deficiencies of the UML are inherited, in particular, the lack of well-defined concepts for system context, view and viewpoint. The usage of SubSystems is not explained in any detail, save a (small) example. It is not clear how many of the aspects of SubSystem can be integrated pragmatically.

In Atkinson et al. [1998], Bayer et al. [1999], Braun [1999], only brief overviews are provided, though there seem to be more detailed descriptions, too, whereas Hesse [1997] is indeed superficial, and is only considered here for the originality of the ideas presented. Table 2.3 (bottom) presents an overview of this comparison.

2.6. Chapter summary

In this chapter we have surveyed the state of the art concerning the five areas that involved in our approach: notions of architecture and architectural unit, languages for describing architectural artifacts like styles and concrete architectures, UML semantics, and architectural processes. We have yielded the following results.

First, we noted that there is a large number of notions of architecture, and probably each and every possibly conceivable concept or concern is defined somewhere. However, there is no single unifying framework that incorporates *all* important concepts. Also, existing approaches have no clear and manifest relationship to software reuse concepts. In particular, the existing concepts for architectural units lack these feature, and they are not well-integrated in other conceptual frameworks like the UML.

A similar situation is observed for AMLs. Here, there is a clear separation between semantics-based academic ADLs and more practically minded general-purpose software description languages—but no comprehensive approach unifying the best of both worlds. Consequently, there are relatively few attempts to support that part of the software process that is genuinely architecture-related.

An integrative approach would try to bring together the best of all approaches, and unify them in a well-balanced system, which is both formal enough to allow for tool support, and intuitive enough to be effectively usable in practical day-to-day work and as a means of communication with non-engineers. Given the enormous potential impact such an approach might have, it is not astonishing, that several such attempts have been made recently: a number of working groups have suggested architecture-level extensions of the UML. Such approaches do indeed unify field-proven notations and concepts with process and tool support. However, these approaches have so far not been entirely convincing: first, they are not broad enough wrt. the coverage of concepts. Second, the degree of amalgamation is rather poor—the concept remain unconnected with themselves, and with the UML. Third, there is so far no satisfying formal basis for the UML, which is necessary both for achieving precision and clarity of the presentation, and for allowing for tool support, which we deem indispensable.

So, in a nutshell, there is some work to be done.

Part I.

Concepts and Syntax

3. Concepts for architectural modeling

In this chapter, we investigate the notions of software architecture and architectural unit from a pragmatic point of view. We determine the groups of people affected by software architectures—its stakeholders—and some of their concerns wrt. a systems architecture (stakes). In particular, we examine the role software architecture plays for one especially important group of stakeholders, namely software architects (cf. [D’Souza and Wills, 1999, p. 486], Goldberg [1998], [IEEE Architecture Working Group, 1999a, pp. 11, 15, section 5.2], [Clements et al., 1996, p. 20]).

3.1. Stakeholders and their stakes

We believe that the architecture of a system is the basis of communication about it. Thus, it affects *all* of its *stakeholders* (see also [Marshall, 1999, p. 14-26] on the concept of stakeholder in the context of system development). We will explain the way the different groups are affected in turn.

Developers First of all, software architecture and designs thereof affect *developers* irrespective of what role they play, i.e. they may be working as programmers, reuse engineers, software architects, quality assurance staff or whatever. All of them use architectural designs in their daily work. Also, all phases of software development are affected by software architecture, i.e. throughout the whole life-cycle of a system, there is (some form of) architecture, and it can be used beneficially from the elicitation of requirements, through the definition of the systems boundaries, the initial development, the maintenance and evolution of a system, but also to integrate legacies, or scavenge them for other applications.

In this book, we are particularly concerned with the role of *software architect*. It is succinctly described in the WWISA-charter, where it says “*client advocacy is the cornerstone of the architect’s role*”, that is, the software architect is the one understanding the clients needs, and translating them into the form understood by the software engineers. He discusses and assesses several alternative designs, plans and controls the realization of the one agreed upon, and is responsible and accountable—within reasonable limits—for the overall effectiveness of the project. In a nutshell, a software architect is to a software system what a building architect is to a building. Thus the name.

Management Next, *managers* of software projects are affected. They are in charge of planning and organizing the development work, i.e. allocating resources such as manpower and budget, and controlling the progress of the development work. An architecture, explicit or not, is the basis of their work.

Higher-level management have an interest in reducing time to market, and achieving economies of scale through product families and reuse. In the case of concurrent projects, they decide on their relative priorities, depending on criteria like expected costs and pay-back, the companies vision and strategy etc. Thus, they need accurate data on the projects’

progress and timeliness, possible economies by reuse etc., that are to be provided by the project management on the basis of the respective system architectures.

There are also many companies, whose very existence depends on the proper working and effective evolution of software systems. Since the evolution and maintenance of software systems is a genuinely architectural issue, to some degree, even the non-technical *top-level management* is affected by the quality of the software architecture (mission critical) systems.

Clients and users Also, *customers* might need to assess the technical and performance requirements of system, based on its architecture. Finally, the *users* are usually not directly involved in or aware of a system's architecture, but are nevertheless affected by its qualities: they desire a stable, performant, ergonomic, and adequate system, even under the conditions of sustained evolution.

In this work, we are mostly interested in the developer's view, therefore we now elaborate on their view by developing a number of usage scenarios. These scenarios are given so as to illustrate the spectrum of activities involving the architecture, not in order to exhaust it (see e.g. Bosch et al. [1998] for another list of architectural stakeholders and associated activities).

3.2. Relationships between stakeholders

So far, we have considered the stakeholders and their stakes in isolation. We will now move on to their relationships, shedding further light on their tasks and responsibilities. Consider Figure 3.1. Here, the five main stakeholders are presented with their relationships. We have omitted their various roles for clarity. Of course, there may be cases where some of these roles coincide.

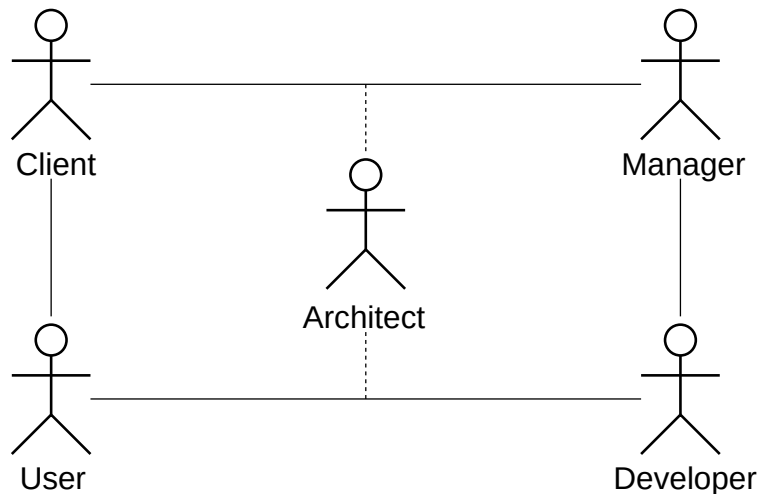


Figure 3.1.: The architect mediates between the other stakeholder.

User & Client The user is either a customer or employee of the client. Either way, it is the client, who carries responsibilities for the user. Not that being a customer is not necessarily a commercial relationship (consider for example a government agency and a citizen).

Client & Manager The client commissions the construction or maintenance of a particular system by negotiating a contract with the management. The client needs to communicate his goals and business needs to the manager. Conversely, the manager needs to communicate technical constraints.

Manager & Developer The manager then must organize the development process proper, including all activities related to human resources, budget and so on. Both the developers and the clients are his customers.

Developer & User The developer, then, has the user as their customers, to specify the requirements in detail. This includes cases where there are no "real" users, but proxies acting in their stead.

So far, the roles are similar to the roles found in traditional, non-architecture aware development. However, so far, architectural issues are not taken care of: Dealing with them is left to chance. Here, the software architect comes into play.

Client, Manager, Architect The architect facilitates the negotiations among clients and managers, reconciling technical and business requirements. His job is to assess the feasibility and relative benefits of alternatives, mediating the diverging interests guided by his technical expertise, experience, tools and professional standards.

User, Developer, Architect On the other hand, following the development process through to the more constructive stages, he moderates the development process, in particular the requirements elicitation, by advocating the client's strategic goals. He is to monitor the project progress, and update the client and management on all relevant incidents, e.g. when unforeseen technical difficulties arise.

Ideally, a software architect would have experience in the respective domains of all the other stakeholders. That being quite unrealistic, however, there is only one choice: Since the technological skills are indispensable, the role of software architect has to be played by a developer. The role of the software architect are not dissimilar to that of the system analyst of the pre-OO methods, if it were not for the monitoring and mediating aspects of his job.

3.3. Further implications of architecture-based software construction

In this section, we raise some commercial, legal, organizational and cultural issues connected with software architecture. As these issues are beyond the scope of this book, yet their paramount importance demands consideration. So, we mention them, but only in passing. For an assessment of the impact of reuse/componentware to the development process see e.g. Boehm and Belz [1988], Griss et al. [1994].

3.3.1. Development process and system life cycle

Obviously, the reuse of system components affects the development process of software systems, and the system life cycle as a whole. Firstly, building a component is different from building a complete system in that a component is always intended to be reused in different contexts, or evolved over a prolonged period of usage (not necessarily a long period of time). If one is to adopt the type of

capsules proposed in this dissertation, a number of aspects have to be covered, that are rarely, if ever, encountered for monolithic, one-of-a-kind, or isolated systems.

Secondly, building a system from components is different from building a system from scratch, without using components: Assuming that enough suitable components are available through one channel or the other, the emphasis is shifted away from realization to specification, and from programming towards designing and testing, thus changing the ratio of the activities carried out. Also, new activities related to using components are introduced (e.g. querying for a component, purchasing it, adapting it, replacing it etc.).

Thirdly, the purpose of using a componentware approach is to prolong the life cycle of a system, i.e. allowing for its functional evolution, adaptation to different environments and contexts, and derivation of variants.

3.3.2. Organizational infrastructure

It has been noted before, that using components also imposes special requirements on the organization that is to use or to build components (see e.g. Goldberg [1998], [McClure, 1992, p. 230] and [D'Souza and Wills, 1999, p. 456])¹. The new activities implied by the componentware paradigm have to be supported by an appropriate organizational infrastructure, too, and they have to be organized in a way, such that making components reusable and actually reusing them is directly beneficial to respective developers (see e.g. Griss et al. [1994] and Goldberg [1998]). All in all, software reuse is knowledge management, with all implications and repercussions.

3.3.3. Component markets

We now turn to an aspect of reuse and the vision of componentware, that is often neglected in technical papers, though it would seem this is indeed crucial: the market aspect of componentware. Large scale, big time reuse of components can only be successful, if there is a large enough market for commercial off-the-shelf components, so that is actually worthwhile trying to procure components rather than manufacturing them, or have them manufactured to the specific purpose. That, however, requires some policy of payment that addresses both the legitimate interests of vendors and purchasers. Buying and selling components, however, really is trading knowledge about some domain. So, all the problems related to protecting intellectual property have to be addressed in this context.

From a market perspective, the interests of vendors and purchasers of components come into view. The purchasers need to have access to a large number of technical details in order to use the component, and exploit its full potential, i.e. making the purchase of the component in question a worthwhile thing to do. The vendor on the other hand, needs to make sure that he gets adequate payment and in particular, that his product is protected against piracy. The classical solutions or protection against piracy obviously do not work in this setting.² So, the only (easy) way out for the vendor is to hold back technical information, and only disclose executable binaries, from which it is hard to reverse-engineer and access the knowledge that accounts for the value of the component. That, however, defeats the whole purpose, since interoperability is one of the key issues with components.

¹See [McClure, 1992, p. 228] for problems of reuse in general.

²For end-user products, protection of intellectual property is sometimes done by extra hardware ("dongles"), password schemes or licensing software ("license servers"), or code escrowal. Obviously, none of these approaches can be taken for software components: they are supposed to be used in various systems, including embedded systems, and you don't want your airbag to prompt you for a password every now and then...

So, there is a need of giving away the executable along with just the right piece of information to make the components usable. In other words, there should be a selection of viewpoint or view associated with published components, so that the intellectual property of the provider is protectable to the degree desired, depending on how many levels of abstraction are visible to the user. Yet, the user has access to the technical detail he requires.

Obviously, the coming into existence and flourishing of component markets is to some degree dependent on these economic requirements being satisfied. Since software engineering components are much better suited to satisfy them than programming components like Java Beans and COM-components, it is our belief that the approach pursued in this book has some potential to promote the wide-spread use of components.³

3.4. Usage scenarios

We list a number of architecture-specific activities in the software development process. The activities are organized in groups according to development phases, and development roles (i.e. stakes), respectively. Neither the list nor the grouping is by any means complete or definitive, it just serves to make clear the scope of activities an architectural description may be involved in. Scenarios similar to the following, and further ideas are described e.g. in [D’Souza and Wills, 1999, p. 486–490], [IEEE Architecture Working Group, 1999a, sections 4.3 & 5], Rechting [1991], Horn et al. [1999], and [Hammer, 1997, p. 305, 309]. We will define some of the following scenarios in greater detail in Chapter 11.

3.4.1. Initial design

We find five scenarios within the initial design (see Figure 3.2).

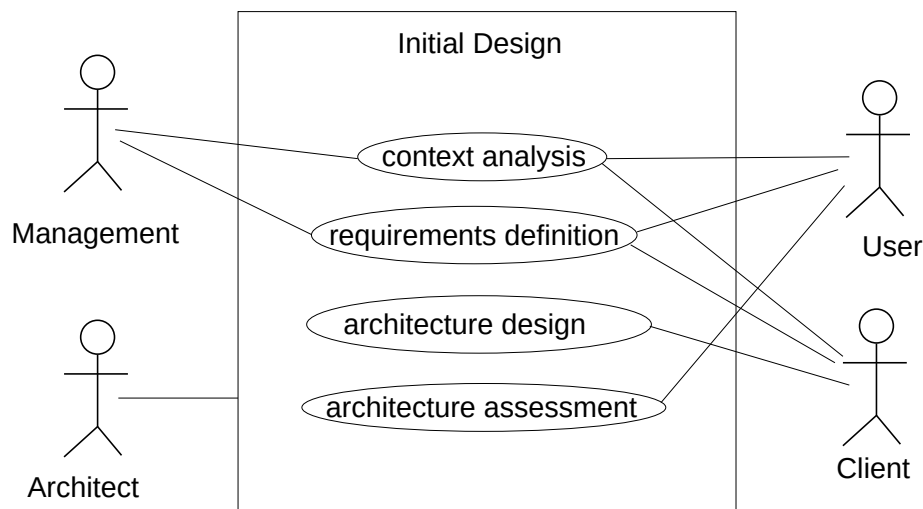


Figure 3.2.: The initial design comprises the scenarios “context analysis”, “requirements definition”, “architecture design”, and “architecture assessment”. An actor associated to all use cases is shown with a single association to the boundary of the system.

³This situation is of course exemplary for the intertwining of technical and economic aspects we witness so often today.

Context analysis

First of all, the *context* of a system must be analyzed: who are the stakeholders, what are their stakes and *goals*, what technical, legal and organizational constraints are there, what are the business processes, what part of a system is to be mechanized, and what part is to be left unsupported, i.e. where is the system boundary (on the role of goals in architecture, see e.g. [Regnell et al., 1997, p. 277], Buhr [1998] or IEEE Architecture Working Group [1999a], where it is even a proper metaclass). The context analysis might result in an architectural design consisting just of the relevant actors and sub-systems involved, but little or no details as to their precise structure, behavior or other features. In particular, the system that is to be developed is just an entity, an architectural unit without further features. In Section 5.2.3 below, we will introduce context diagrams for this purpose, an extension of traditional use case diagrams. See also Marshall [1999] for a discussion of these notions.

Requirements definition

Having identified the relevant stakeholders and their constellation in a context analysis, an initial plan can be created concerning the technical, legal, organizational, administrative and other requirements in greater detail. In particular, parameters such as throughput/number of users and available computing platform and power, and network capacity have to be determined. Also, the relevant aspects or viewpoints for a system need to be defined at this stage of development. Finally, the management parameters like budget, schedule, strategic purpose of the overall project need be determined.

Architecture design

The results of the context analysis and the requirements definition can then be used to make an initial architectural design. Starting from the description of the context, a first stage of refinement could add the protocols of interaction between the various subsystems, abstract descriptions of their behavior, and possibly also a structural, functional or behavioral decomposition of the system. This first draft of the physical structure formalizes the requirements and the constraints elicited by the context analysis and the requirements definition. In particular, if existing subsystems such as legacies are to be integrated, their usage must be planned at this stage, along with any middleware that is to be used.

Architecture assessment

The initial architectural design has to be continually assessed. There are several techniques, ranging from validation techniques such as walk-throughs, design inspections, and interactive simulation runs, to more rigid techniques such as the formal analysis of traces, consistency checking of the various parts, and might even include the analysis of quantitative system properties like load, throughput, reliability, safety, and so on. Except the first two, all of these techniques can be supported by automatized tools to a considerable degree, opening opportunities for raising the cost-effectiveness of software construction.

3.4.2. The reuse cycle

It has been noticed many times, that the architecture level is the appropriate level for effective reuse. On the one hand, the effectiveness of reuse is obviously the higher, the earlier it occurs, and the larger the parts are, that are reused.⁴ On the other hand, the effectiveness of reuse critically depends on the

⁴This is both an intuitive fact and an empirical finding (see [Zendler, 1997, p. 83], Matsumoto et al. [1991]).

frequency of reuse, i.e. the number of times a component is actually reused. Obviously, reuse is easier for similar systems, such as members of a product line. We will not consider the organizational and legal issues that are of prime importance for the effectiveness of reuse. We find four scenarios within the reuse cycle (see Figure 3.3).

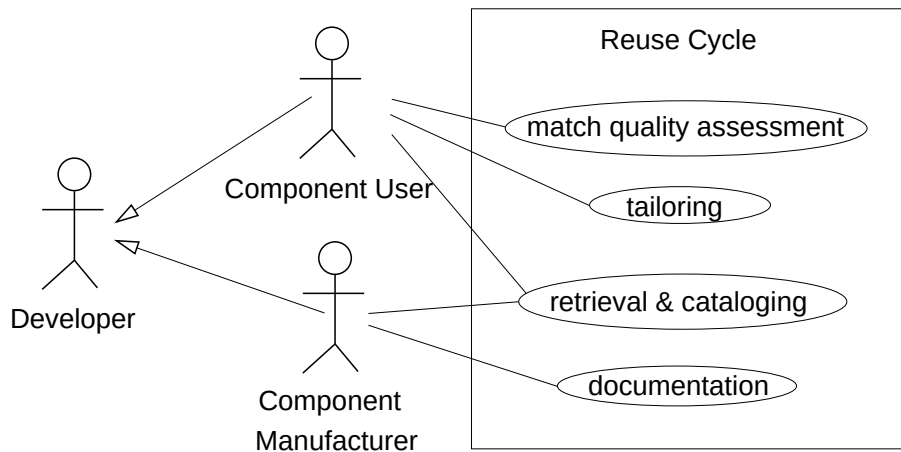


Figure 3.3.: The reuse cycle comprises the scenarios “retrieval and cataloging”, “match quality assessment”, “tailoring” and “documentation”.

Retrieval and Cataloging

For a specific gap in an architecture of some system, suitable components must be retrieved from (large) databases of off-the-shelf components. This requires specifying a profile of the candidate component, and finding the best matches in the database. Dually, components need to be prepared for retrieval, that is, they need to be described according to some cataloging scheme. Assuming a perfect database that holds the right component for each and every requirement, the effectiveness of reuse critically depends on actually finding the right components, which is mainly determined by the quality of the measures determining the match, and the quality of the cataloging.⁵ Due to the effort involved, it is practically impossible to do a manual cataloging. So, components must be organized in a way that allows for easy, and if possible, automatic classification.

Match Quality Assessment

Once retrieved according to coarse, mechanical matching criteria, it is necessary to assess with greater scrutiny, greater computational effort, and possibly manually, how well a component really matches the specific site under consideration, and what effort will have to be spent tailoring the component.

Tailoring

Typically, a retrieval will yield a non-perfect match, its qualities and deficiencies being revealed by the previous assessment. Supposing that the selected component is to be used anyway, it needs to be adapted. There are basically three complementary types of adaptations to be considered. First,

⁵An overview of the different approaches, and a mathematical treatment of the relationships between the variables involved can be found in Zendler [1997].

components may have switches and parameters that could be used for customization. Second, there are simple adaptations like aliasing of signals, transposition and projection of parameters, or simple pre- or post-processing that can be done by a wrapper. Third, a component may need real modifications, like an architectural restructuring (see e.g. p. 222).

Documenting a system

Many systems have little or no documentation of their architectures, but need to be integrated into reuse schemes just the same. For instance, so as to facilitate future changes and to allow for the extraction of fragments of code or business logic ("components") from legacy systems (i.e. re-architecting a system), so as to prepare for legacy scavenging, or business-logic archaeology. This resembles the development of systems in its result, but requires different techniques and procedures. A particular instance of system documentation is the extraction of styles, i.e. recurring architectural patterns, so as to learn for future designs. Once identified and extracted, a style must be particularly well documented, and possibly (formally) analyzed, validated, audited, assessed and certified.

Concluding this section, we want to emphasize again the primeval purpose of architecture, for all stakeholders, and within all conceivable scenarios: being a plan of the system under development/maintenance, an architecture is the technical basis of the *communication* of collaborating people. It is a necessary, albeit not sufficient prerequisite for teamwork on large systems.

3.4.3. The implementation phase

In the implementation phase, an architecture becomes a blueprint for realization. It can guide the coding and it can be used as a basis for the test both of individual units, and of the integration of units. It may even serve as a contract for out-sourcing. We find three scenarios in the implementation phase (see Figure 3.4).

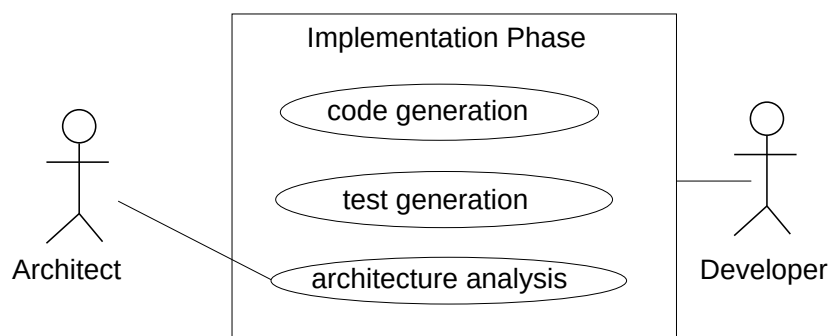


Figure 3.4.: The implementation phase comprises the scenarios "code generation", "test generation", and "architecture analysis".

Code generation

Many pieces of code can be generated automatically today, like make-files, database schemas, or other artifacts involved in the creation of an executable system. It is unlikely, that we will ever be able to generate all of the code of an application automatically from the specification in a practical way. It is

more likely, that we will be able to generate the coarse-structure automatically, arranging a great number of small, hand-written pieces of code (i.e. on the level of method-bodies). One application is the (correct) reconfiguration of architectures in terms of deployment, so as to accommodate changed requirements concerning the load or net traffic. Similarly, changes of the functional, behavioral, or other requirements might have to be accommodated in the architecture as well—in a well-designed architecture, little new coding should be necessary, most of the changes are on the architectural design-level, and once made, should be carried through automatically. Obviously, code reuse and componentware are important contributions here.

Test generation

Even if no automatic generation of code is possible, the architecture can be used as a guide for the implementation. For this purpose, it is important to endow architectural units with specific points of interaction (later on, we will call these “ports”). They may be used as probing-points, where actual values and behaviors of the actual implementation can be checked against the specification, similar to the way assertions (like in Eiffel or C++) can be used for code-instrumentation.

Architecture analysis

Next, the choice and adaptation of components has to be evaluated. While a successful match quality assessment and tailoring ensure that interfaces match locally, the global interplay of components needs to be assessed as well. For instance, often a performance/load or load balancing analysis is required. Basically, all the techniques of the architecture assessment can be applied here, with a different, more technical emphasis, and even greater potential for tool support.

3.4.4. The maintenance phase

Software changes—it is vital to accept, and even embrace this fact. Doing so implies that the maintenance and evolution of software is considered an essential part of the lifecycle of many a system. This is particularly true for those systems that we deal with in this book: systems that are large and long-lived enough so we care for their architecture in the first place. Obviously, constant change is a constant source of new errors and architecture erosion, and so, there is a definitive need for process support at this phase, too. It has been mentioned many a time that software “maintenance” is really a misnomer, and that there are different kinds of maintenance to be distinguished:

Today, the quality of many software systems is deplorable. And even an initially satisfying system seem to be bound to fall into disrepair sooner or later, when subjected to constant change: *“program maintenance is an entropy-increasing process, and even its most skillful execution only delays the subsidence of the system into unfixable obsolescence”* (cf. [Brooks, 1975, p. 123]). Change, however, is not an external force exerted by impertinent users: it is part of the very nature of software, for software only ever exists with and within its context, and contexts change.

“Software is not at all like wood or steel. Its paint does not chip and it does not rust or rot. Software does not need dusting, waxing, or cleaning. It often does have faults that do need attention, but this is not maintenance, but repair. Repair is fixing something that has been broken by tinkering with it or something that has been broken all along. Conversely, as the environment around software changes, energy must be expended to keep it current. This is not maintenance; holding steady to prevent decline. Evolution is changing to move ahead.”

[Cox, 1986, p. 6]

We have identified the dimensions impetus, impact and instruments of change that help us to distinguish between kinds of change.

impetus The impetus to change a system may originate from within or from its environment, so we distinguish between external and internal impetus. In the former case, we speak of maintenance, in the latter, the term evolution is more appropriate. In [Boehm, 1981, p. 54f], the terms update and repair have been used, where the latter is again subdivided into corrective, adaptive and perfective repair, i.e. fixing bugs, adapting to a changed environment (e.g. porting to a new platform), and improvement of, say, performance.

impact The (expected) severity of the impact to the system's overall structure and operation is determined by the degree of locality. Small, local changes do not affect the system's architecture, and so can be neglected from the architecture perspective. To some extent, one may even define the architecture or architectural style as that which does not change (easily), i.e. some kind of system invariant. Then, a change that does not affect this invariant could be called evolutionary, while a change that does, would have to be called revolutionary.

instruments Finally, the kind of change can be described by the instruments that are available (or even just conceivable) to support implementing it. This ranges from tasks where there are automatic tools, via modestly interactive tasks to genuinely human tasks, e.g. installing a patch, design debugging and programming, respectively. Similarly, there are procedures of varying rigor, i.e. genuinely creative versus mechanical tasks. Techniques like walk-through and inspection are somewhere half way.

In the remainder of this section, we deal only with changes with a big impact and at least partially rigorous and semi-automatic instruments. We will look into three kinds of maintenance, namely the replacement of individual architectural units, their structural refactoring, and their re-specification (see Figure 3.5). Note that these are ordered in increasing magnitude of expected impact. In Section 11.3, we shall provide a language of patterns for processes specifically for architecture-related tasks, including maintenance.

Replacement of units

Individual units may be replaced, e.g. to patch errors, or to make use of a more efficient or reliable unit. Ideally, we would like to consider only the unit that is to be replaced, its immediate environment, and the replacement. Again, the self-containedness of the units is crucial, as in the reuse cycle, see Section 3.4.2. However, there is a fundamental difference: there, we considered the reuse of a component, here, we consider the reuse of its environment. Therefore, replacability requires a different form of self-containedness than reusability. In the former case, we need to control the dependencies *on* the thing to be replaced, in the latter, we care more for the dependencies *off* the thing to be replaced.

Architectural refactoring and redeployment

Inevitably in the process of developing a system, there comes the point where the initial design has to be overthrown, e.g. to achieve better performance and load distribution by redeployment, which usually implies a restructuring, too. Another reason might be the introduction of new functionalities, requiring additional "hooks", that is, interfaces for their specific purposes.

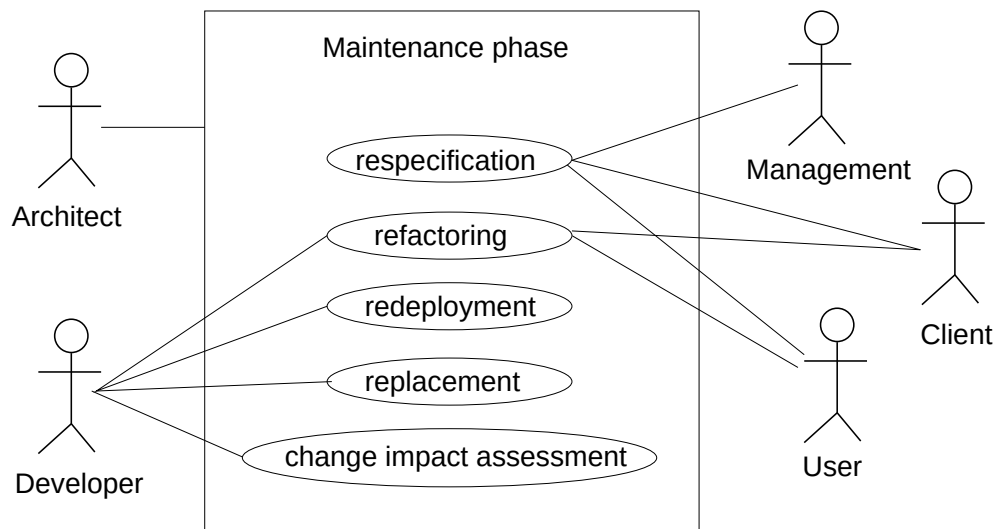


Figure 3.5.: Maintaining an architecture comprises the scenarios “replacement”, “refactoring”, “redemption” and “respecification”. The scenario “change impact assessment” helps tracing the effects of the changes, thus focusing the whole effort.

Respecification

Finally, the context of a system may change, that is, the platform, the requirements, the expected functionality, usage and load characteristics, or even the domain. In these cases, it is not enough to just adapt the implementation, or the design, but the whole architecture must be reconsidered. If this is not done, the quality of the architecture deteriorates, and, eventually, breaks down.

Change impact assessment

It is of course highly desirable, to restrict the effect of the changes, so that as much as possible of the previous work may be reused. To facilitate this, a change impact analysis (see Bohner and Arnold [1996]). has to be carried out. This is supported by a strong (behavioral) encapsulation of the architectural units, and requires a formal semantics, so that rigorous and (semi-)automatic analysis methods can be applied.

3.5. Main concepts

We have so far determined the stakeholders and their stakes, with particular emphasis on the developers point of view. To accommodate activities as diverse as these, a comprehensive concept for the elements of architectures becomes necessary, which we will now derive. Many works on architecture focus on either semantical or implementation aspects of architecture, and tend to avoid the pragmatical issues raised above. Consequently, the notions of component they provide are biased, and we believe they are not appropriate for software engineering purposes.

3.5.1. Views and viewpoints

We have determined the relevant activities for working on the architecture level by exploring a variety of usage scenarios. Every activity has its specific requirements concerning the concepts and notations needed, focusing on a particular part, aspect, or level of abstraction of the system under development. Such a focus is realized by a viewpoint. We adopt the terminology of IEEE AWG *P1471*:

Definition 3.5.1 (viewpoint, view)

A *viewpoint* is “a specification of the conventions for constructing and using a view.” (cf. [IEEE Architecture Working Group, 1999a, Def. 3.1.10]). A *view* is “a representation of a whole system from the perspective of a related set of concerns” (cf. [IEEE Architecture Working Group, 1999a, Def. 3.1.9]), i.e. a description of a concrete system from a particular viewpoint. In a non-technical sense, it is an instance (or reification) of a viewpoint. □

Using views and/or viewpoints has three main benefits. First, it promotes a better separation of concerns, and thus improves the development process.

Second, inconsistencies may arise from having different views of the same system. Many of these inconsistencies can be detected automatically, allowing for effective tool support on the design level already (“design debugging”, see below). This increases the validity of models, and can save substantial effort in later phases.

Third, using viewpoints can be exploited for the automatic cataloging and retrieval of software components from component databases. As we have sketched in Section 3.4.2, reusing a component involves querying a component database. Assuming that the sought after components are available at all, the effectiveness of reuse critically depends on finding the *right* components, which in turn depends on the quality of cataloging them in the first place. Here, *facette classification* has proved a valuable tool over the years (see e.g. Biggerstaff and Richter [1987], Prieto-Diaz [1989, 1991], Schäfer et al. [1994], Zendler [1997]). But manual classification is expensive, tedious and error-prone and is thus a major obstacle to effective reuse. This obstacle can be (partly) overcome by using viewpoints as *facettes*: the relevant viewpoints are provided in the course of system development anyway, so that no extra effort has to be put into a classification. The terms or elementary classes within the *facettes* are the elements of the respective viewpoint. Thus, the classification is done quasi automatically.

Of these three benefits, only the first seems to have been examined before. Yet, their significance can hardly be overestimated. We will elaborate on them in Part III.

Most authors seem to agree that it is indispensable to use several different viewpoints to design and construct systems (cf. the insightful discussion of viewpoints of architectures given by Hammer [1997]). Just how many viewpoints there should be, however, and which, is subject to intense debate. Some examples of possible viewpoints are:

functionality i.e. what the architectural unit is supposed to accomplish.

structure i.e. the kind and configuration of architectural units involved to refine a given unit.

behavior i.e. what external stimuli (mediated by the points of interaction) the architectural unit responds to, and in which way.

implementation i.e. how the functionality is accomplished technically.

interfaces i.e. the points of interaction, and the interactions they permit.

platform i.e. requirements of the technical environment, such as operating systems etc.

administration i.e. the data related to managing the development process.

This is but a short collection of obviously useful viewpoints. More examples, together with the conceptual embedding and concrete syntax, will be presented in Sections 4.2.2 and 5.3, respectively. In Appendix D, there is a complete table of all viewpoints used in this book. Note that a viewpoint is a special kind of aspect, but there are aspects that do not have a corresponding viewpoint (quality, or concurrency, say).

Most approaches fix a certain set of viewpoints once and for all (see e.g. Davis and Williams [1997], ISO RM-ODP [a], Jacobson et al. [1999], Kruchten [1995] or [Booch et al., 1999, p. 31]), but fail to give sufficient justification for their choice (an overview of some positions is presented in Table 3.1).

We agree on the paramount importance of views and viewpoints as such (which has been validated empirically, too, see Turgeon and Madhavji [2000]), but to us, it seems obvious that the relevant viewpoints do vary among systems, depending on technical, business, legal, pragmatical and several other factors. So, there cannot be a single right set of viewpoints for all circumstances. Thus we allow arbitrary viewpoints, but require them to be fixed during requirements elicitation. Views (and viewpoints) may be complementary and overlapping, but they all refer to the same system (i.e., architectural unit) and its parts, so that their totality is supposed to be a complete and consistent description of the system. Both properties are non-trivial.

	Davis/Williams	Kruchten	Amigos	RM-ODP
administration	–	–	–	–
behavior	–	–	–	information
context	–	–	–	(enterprise, technology)
deployment	(component)	physical	deployment	engineering
domain	domain	logical	design	(enterprise)
functionality	context	scenarios	use case	–
implementation (execution)	component –	(development) process	implementation process	– –
interfaces	interfaces	–	–	(computation)
platform	platform	–	–	(technology)
structure (static)	– (component)	– (development)	– –	(computation) –

Table 3.1.: Rough comparison of different systems of viewpoints. Items in parentheses indicate parts of the view on the previous row (first column), or approximate matches (other columns). The table headings Davis/Williams, Kruchten, Amigos, and RM-ODP refer to Davis and Williams [1997], Kruchten [1995], Booch et al. [1999], and ISO RM-ODP [a], respectively.

3.5.2. Architectural units

Bundling several views, an architectural unit constitutes an independent system. It is a *self-contained entity* in the sense that it is an entity for all purposes and activities of software engineering. We have mentioned architectural units as units of design, planning, production, evolution, reuse, formal analysis, deployment, replacement etc. Pfister & Szyperski mention components as units of abstraction, analysis, compilation, distribution, loading, dispute, deployment, existence, maintenance and several others, see Pfister and Szyperski [1998].

An architectural unit is rigorously *encapsulated* so as to allow for compositional design and analysis. This point is particularly important, with a view to the state space explosion problem. In order

to allow for adequate modeling of problem-domain entities, an architectural unit is also a potentially *large* entity, with significance to the stakeholders and actors in its context. The significance of the context for an architectural unit lies in the role of architecture as a central medium of communication for all stakeholders involved. We use the term context in the classical system-theoretic sense. Since architectural units are systems in their own right, there is no difference between context of an architectural unit, and the context of a system as such. Depending on the specific focus, the context might be documented by viewpoints for the technical environment, the business goals, or the functionality of a system, where the users are represented as actors in use cases.

An architectural unit may be implemented using any appropriate technology, for e.g. legacy applications are quite naturally architectural units, and many legacies are not realized using the object paradigm. Still, they need to be integrated as architectural units, too. These characteristics can be condensed into the following definition.

Definition 3.5.2 (architectural unit, context)

An *architectural unit* is a rigorously encapsulated software system with a life-cycle of its own: there are no other dependencies than those on well-specified points of interactions. An architectural unit is self-contained under all software-engineering activities of all phases of the system life-cycle. It consists of a set of views (i.e. instances of viewpoints), to describe its internal and external properties wrt. all the activities it can be subjected to. A unit is always explicitly embedded in a context. An architectural unit is entirely independent of any implementation technology.

The *context* of a system is the totality of its technical, organizational, and legal side conditions, its strategic goals and other (non-functional) requirements, the business processes affected by the system's functionality. It defines the stakeholders, boundaries and, possibly, constraints on its internal workings, such as compulsory units or deployment. In doing so, it establishes a mapping between systems and actors on the one hand, and functionality on the other, that is, which tasks in a business process are automatized and which are not. The context also fixes intended future development paths, and the viewpoints that are considered relevant for construction and evolution. □

Note that the level of precision (i.e. amount of detail) and the level of abstraction (i.e. scale or model layer) are independent of each other. By focusing on a system's architecture, we fix a level of abstraction, leaving open the amount of detail to be filled in into the model. However, the self-containedness of architectural components allows to handle the amount of detail that one can hope to deal with effectively with machine support, in other words: can hope to handle effectively at all.

There are at least three kinds of architectural units that we are able to distinguish at this stage: black-box units without specification details save interfaces (examples are legacies and very small units); proper units with detailed specifications such as third party reusable components; and virtual units, that do not exist as chunks of code, but rather as a logical block of functionalities whose physical realization is distributed over several other implementation units. Black-box and virtual units are less detailed models than proper units, and so their use is restricted—but at least, they can be modeled.

3.5.3. Architectural glue and points of interaction

In order to achieve the degree of self-containedness we strive for, interaction with and of architectural units is limited to specific points of interaction, which are the only facilities of interaction an architectural unit possesses. These points of interaction can then be used to glue architectural units together to form a coherent whole.

In our approach, we put very little emphasis on this glue—in some other approaches such as Allen [1997], Garlan et al. [1994], the glue is on a par with the architectural units. In fact, in our

terminology, they *are* architectural units. Thus we tend to specify them as such. The glue, on the other hand, usually is entirely passive, i.e. without behavior. For modeling convenience, we allow “active glue” too, whose behavior may be specified explicitly. Glue contains constraints on the kinds of points-of-interaction it attaches to, i.e. glue may be more or less specific, it does not necessarily “stick” to all “surfaces”.

Definition 3.5.3 (point-of-interaction, glue)

A point-of-interaction (POI) is an abstract behavioral interface of an architectural unit. All requested and offered services of an architectural unit are requested and offered through points-of-interaction only. Every POI belongs to exactly one architectural unit. Architectural units are related by *glue*, which may attach to points-of-interaction only, but not to architectural units. The glue contains constraints on the kind of POIs that it attaches to. Glue may be active or passive. □

The distinction between units and their POIs on the one hand, and glue on the other reflects the architectural distinction between computation and communication. A POI may be either an abstraction or a transformation of the internal behavior of a unit.

3.5.4. Architecture

The purpose of architectural modeling is to understand, organize and communicate a system’s (planned or existing) architecture. The architecture is distinguished from the rest of the system by being to a lesser degree liable to change. Thus it is as well a template for product families, a basis for reuse and legacy scavenging as it is a basis of evolution. It defines and documents the system context and requirements, and is a guide during the realization of the system, helping to understand and control the creation or evolution of a system by allowing experiments on models rather than the real system (e.g. comparing the behaviors of the model and the real system, e.g. by probing ports).

A software architecture is present during the whole life-cycle, from initial conception through development, deployment, usage, evolution, reuse until the eventual dismantling of the system. It always exists, even if its trivial or chaotic. It is subject to all activities, and encompasses both the highest-level of abstraction and the code-level. It exists in a context (structural, technical, business, strategic, requirements, non-functional requirements) and is of concern to all stakeholders.

So, we arrive at the following understanding of architecture of software systems:

Definition 3.5.4 (software architecture)

A *software architecture* comprises (1) a constellation of independent architectural units, their external properties and the (dynamically changing) connections between them; (2) an embedding of the system into its context, including the architecture’s *rationale*.⁶ □

Our definition can be seen as a specialization of both the IEEE’s definition and the definition of Perry & Wolf (see Section 2.1 for an in-depth discussion). The criteria for our specialization can be derived from the purposes we have considered relevant for architectures above.

It is very important to note that this definition of architecture is *not* a structural one—it may merely *include* structure. Note also, that it provides a very natural way to integrate the various views of a system, in particular those from structural, behavioral and functional viewpoints. Viewpoints that make for views without internal structure do not prevent further structuring, they merely require that these views be (transitively) present in all of the architectural units constituting a system. General

⁶Webster’s Dictionary: (1) an explanation of controlling principles of opinion, belief, practice, or phenomena; (2) an underlying reason.

principles governing the design of a system (e.g. “use message passing style”, “Java will be used as implementation language”) might be examples for such views.

3.5.5. Architectural style and product line architecture

There are widely differing opinions what architectural style is. The spectrum of interpretations ranges from styles as general paradigms of software development and the class of models generated⁷ to the semantics of architectural notations: “*architectural style is an interpretation from syntax to semantics*” (cf. [Abowd et al., 1995, p. 327]). The various ADLs each have their own, rather restricted and quite technical meaning of style.

According to Buschmann et al., a style is like an (architectural) *pattern*, but “*it is independent from its realization, and does not directly refer to a concrete application problem it is intended to solve*” (cf. [Buschmann et al., 1998, p. 393]). On the other hand, in the UML a pattern is “*a synonym for a template collaboration that describes the structure of a design pattern*” (cf. [OMG UML 1.3, p. 115]). Thus, in the UML, a style would be a “*template collaboration*”. We find this definition rather confusing.

A succinct definition is given by Shaw and Clements, who define a style as “*a set of design rules that identify the kinds of components and connectors that may be used to compose a system [...] together with [...] constraints on the way the composition is done.*” (cf. [Shaw and Clements, 1996, p. 1]). A similar position is expressed in Medvidovic [1999]. Thus, a style is understood as a vocabulary to express architectures.

On the other hand, an “*architectural style is that which abstracts elements and formal aspects from various specific architectures*” (cf. [Perry and Wolf, 1992, p. 45]), and “*there is no hard dividing line between where architectural style leaves off and architecture begins*” (cf. [Perry and Wolf, 1992, p. 45]). So, style has a rather close relationship to architectural unit. We use this understanding as the basis of our own definition.

Definition 3.5.5 (architectural style, product line architecture)

An *architectural style* is a template for an architectural unit, and may contain all the elements that an architectural unit may contain. In particular, it contains (1) architectural units, or partial specifications, templates or constraints for them (pragmatic or other); and (2) glue with behavioral constraints. It may also contain (3) partial specifications of the static and dynamic structure of these architectural units and glue. Finally, (4) a style contains also an account of its rationale and usage conditions. All of these aspects are stored in appropriate views of the style. Being a style is a role of an architectural unit, i.e. the style may be derived from some capsule in a natural way.

In contrast to a style, a product line architecture is a concrete architecture that is paradigmatic for a set of other architectures, i.e. with defined extension and variation points. While a style is realized or implemented, a product line architecture is refined and extended in order to be applied. Thus, a PLA typically contains different kinds of ModelElements than a PLA, and exists on a more concrete level of abstraction. □

Architectures can be described as concrete representatives of styles. A style is to an architecture what a role model is to an object model.

The purpose of a style as we understand it is threefold. First, applying a style is a documentation of the design and implicitly a discussion of the design alternatives. Second, applying a style means

⁷See [Shaw, 1995a, p. 28 & 38], who equates “*design strategies*” with style and mentions as examples “*object-oriented, state-based, feedback, and real-time development*”.

reusing a specification, or rather, an (incomplete) specification *template*. Third, under certain circumstances, not only the specification may be reused, but also the results of a (formal) analysis of the style.

Another notion frequently mentioned in this context is that of Product Line Architecture (PLA). A product line is a family of closely related products, that share large parts of their specification and realization, but differ also in some relevant aspects. Typical examples are mobile phones or the platform approach in automotive.

Product lines are obvious candidates for large scale reuse, and thus similar to a style. Also, a PLA has similar descriptive requirements as styles. Typically, however, a style is much more abstract than a PLA, allowing for much greater variation between individual realizations. In fact, there are different mechanisms of reusing styles and PLAs: a style is a specification, and its realizations must satisfy an implements-relationship. The style is, then, not part of the realization. A PLA, on the other hand, is a template that is taken as is, and then modified, extended or pruned to fit the needs of the product that is to become a member of the family. Therefore, a PLA must be much more concrete than a style, and it must provide a set of modification, extension, and pruning points (“hotspots”), along with guidelines and constraints on their usage. So, in terms of increasing degree of detail, there is an ascending order from styles, to actual architectures, with PLAs situated in the middle. There are no strict dividing lines between the three, however.

3.6. Chapter summary

In this chapter we have introduced the main concepts of architectural modeling. We have justified the choice and precise details of these concepts by analyzing the stakeholders and their concerns, i.e., by eliciting the requirements first. In contrast to most existing approaches, we emphasize the early phases, reuse-related concerns and maintenance of systems.

We have observed, that by its very nature, software architecture needs to be expressed in a way that is accessible to as many people as possible. Currently, the UML is the most widely understood language for software designs, so we consider it the right starting point: in the next chapter, we shall realize the concepts we have introduced above by an UML extension.

4. Logical metamodel

In this chapter, we will explain how the concepts we have introduced in Chapter 3 can be realized as an extension of the UML logical metamodel¹, and how architectural models can be expressed using UML notions and notations. An overview of the logical metamodel is given in Appendix A.2. The description here is intuition oriented, a more precise account is given in the following subsection, and later in the formal semantics.

4.1. Preliminary remarks

There are many different approaches to architectural modeling, each with their own strengths and weaknesses. So, rather than picking a particular one to embed it into the UML, we try to compile the most important and advanced concepts of the whole field, and adapt them so that they neatly fit into the UML.

4.1.1. A note on terminology

Regarding terminology, we have tried to stick to the names familiar from literature on software architecture and the UML. However, this is not always possible, as some terms are too heavily overloaded. Thus, at some points, we had to adopt other conventions, or even make up our own terminology. While inevitably carrying an element of discretion, in making up new names we have tried to satisfy the conventions of UML and at the same time appeal to intuition, which are competing goals at times. For the reader familiar with (one of) the approaches to architectural modeling, Chapter 2 provides a brief overview of the relation of the terminology used here and in other approaches.

4.1.2. Approach

The goal of the embedding of concepts for architectural modeling into the UML's metamodel is to use as much as possible of what is already there in UML. This is done by modeling on the level of the metamodel ("metamodelling"), i.e. using the concepts provided by the UML metamodel. One advantage of this approach is, that we have to introduce almost no new syntax at all. The obvious disadvantage is, that some of the oddities of the UML metamodel also appear in our approach. Therefore, the concepts and their embedding is not always completely intuitive and natural.

The UML metamodel is a logical metamodel, that organizes the concepts irrespective of any implementation issues. Usually, the logical metamodel is referred to as *the* metamodel. It is accompanied by a physical metamodel, that implements the logical metamodel in a loose sense. The differences are of technical nature only, such as adapting differing naming conventions, adding or removing attributes, resolving problematic logical constructs such as association classes or multiple inheritance

¹When no confusions can arise, we will call the logical metamodel just metamodel, as is done throughout the UML anyway.

(see Section 7.1.1 for a more detailed discussion). The physical metamodel is supposed to be a direct representation of both the graphical syntax and its textual form (see Part III and Section 6-3 of OMG UML 1.3, respectively), though few tools seem to actually implement the physical metamodel, or offer to export models compatible with the Extensible markup language (XML) Document type definition (DTD) that specifies the textual representation of the physical metamodel (in fact, the DTD is claimed to be generated automatically from the physical metamodel).

We will mirror the UML approach by giving a logical metamodel in this section, followed by a physical metamodel in Section 7, that has an isomorphic textual representation to improve handling, in particular wrt. the semantical mappings. Our implementation actually uses our own physical metamodel.

Since the concepts for architectural modeling are concepts on the same footing as UML concepts, we will use the same typographic conventions for them (see p. 11).²

We treat the embeddings in small groups of related concepts. We also note aggregations and some other associations, where helpful for the understanding of the embedding. Not all associations, however, are always documented, so as to not clutter up the diagrams with unnecessary detail. At the end of this chapter we give an overview of all embeddings.

Note that the concepts we presented here comply with the IEEE standard on architectural descriptions, P1471 (see IEEE Architecture Working Group [1998, 1999a,b]). In [IEEE Architecture Working Group, 1999a, p. 11, Fig. 1], a survey of the notions of P1471 is presented. It is easy to see that all of these notions are embedded in our terminological framework, even though they will not all appear by name in the metamodel extensions we propose.

4.1.3. Synopsis of embedding

In this chapter, we introduce a closely knit system of metaclasses. In order to provide a little overview, we now present a synopsis of all the metaclasses, and their embedding into the plain-UML metamodel (see Figure 4.2). Each concept is presented with a *description*, a brief account of its *pragmatics*, its *embedding*, and its *constraints* expressed in OCL. The relationship between the most important of these metaclasses and the notions introduced in Chapter 3 is visualized in Figure 4.1.

One of the most important and valuable features of our conceptual embedding is its seamless integration with the UML metamodel. Thus, however, the embedding heavily relies on intricacies of the UML metamodel. The reader unfamiliar with the UML may wish to consult Appendix A.

4.2. Architectural structures

In this section we describe the concepts of concrete architectures and their embeddings, i.e. View, Capsule, Port, Connector, ConnectorEnd and Configuration. Figure 4.4 gives an overview of these concepts, their relationships, and embeddings in the UML metamodel.

²In fact, as we will see above, they *are* UML concepts in the literal sense since stereotyped ModelElements are actual ModelElements.

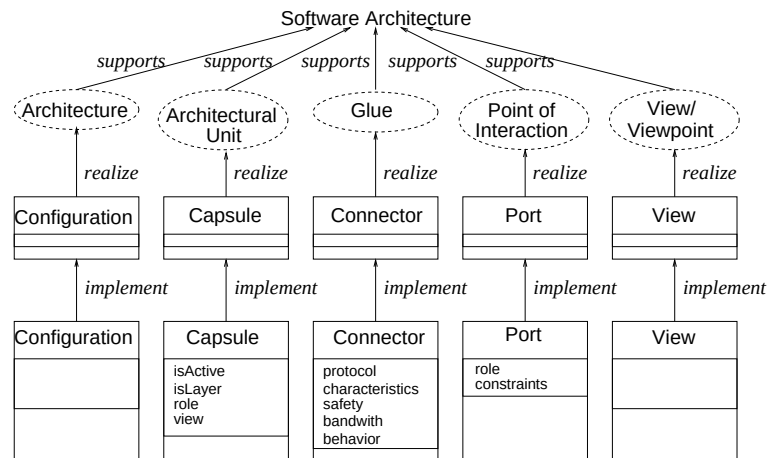


Figure 4.1.: The relationship between the main concepts introduced in Chapter 3 and the metaclasses defined in this chapter.

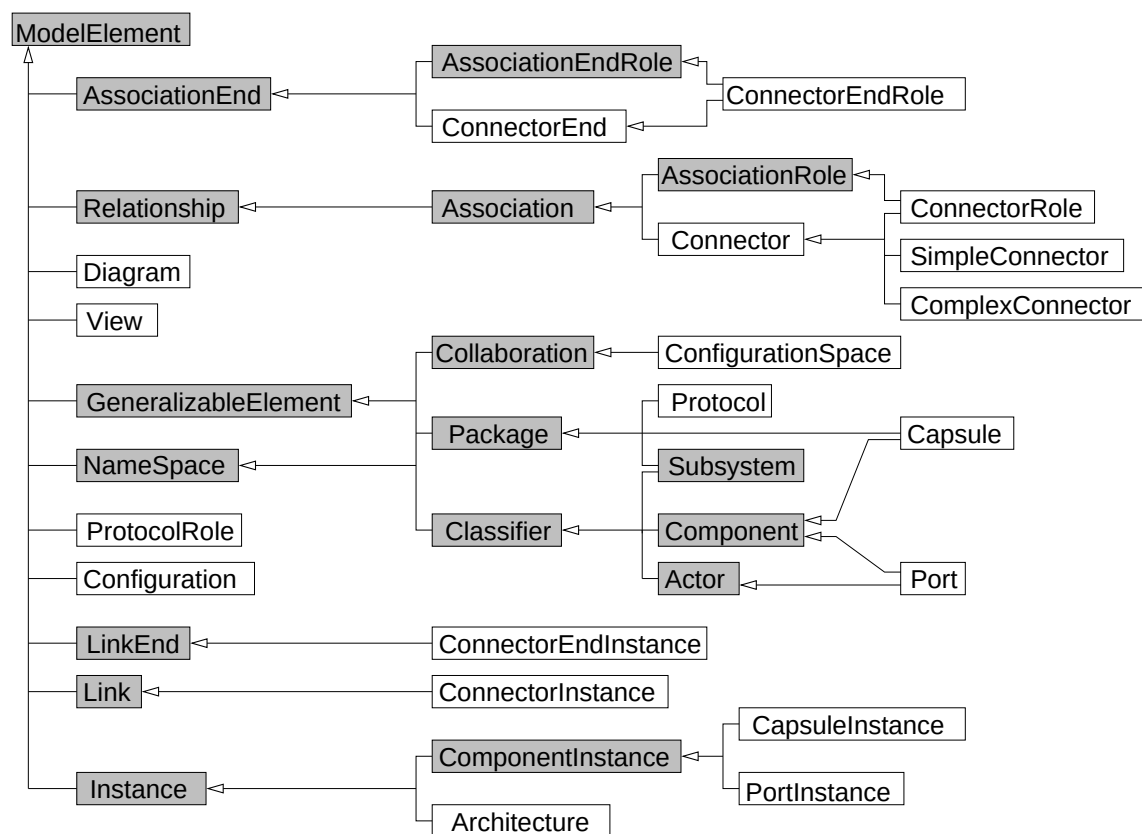


Figure 4.2.: Embedding of the main concepts of architectural modeling into the UML metamodel. Shaded boxes represent metaclasses of pure UML, white boxes represent concepts introduced here.

4.2.1. Capsule

Description

The elementary building blocks of architectural designs are called Capsules.³ They realize architectural units in the sense of Chapter 3. As such, Capsules are the prime entities of reuse, concurrent engineering, analysis, deployment and so on. Note that there are of course other possible realizations, too, but with not quite as advantageous properties.

To cater for the multitude of usages, a Capsule may be equipped with an arbitrary number of Views (see Section 4.2.2 below).

In the run of its *life-cycle*, a Capsule may play different roles. Since the state of the life-cycle is of paramount pragmatic importance, it has to be documented within the Capsule. We currently distinguish between the following roles.

- project A system under development needs infrastructure for the development process as such, such as the assignment of responsibilities to the members of the development team associated to the development project, a project schedule (and budget), development goals, and various other aspects of project management. It may also contain test- and measurement-*“scaffolding”* (cf. [Brooks, 1975, p. 148]) to support the development.
- component A reusable component does not need most of the administration data from the initial creation, but the reuse process needs its own administrative infrastructure, for instance references to prior applications, and information relevant for commercial distribution (e.g. accounting). Also, parts of the scaffolding become obsolete, too, but other technical information is necessary, such as the development and update history, extensive documentation with patterns and styles, and test protocols.
- style A Capsule may also play the role style. As we have discussed in Section 3.5.5, a style is a template specification of a Capsule, or *“a set of design rules that identify the kinds of components and connectors that may be used to compose a system [...] together with [...] constraints on the way the composition is done.”* (cf. [Shaw and Clements, 1996, p. 1]). Since *“there is no hard dividing line between where architectural style leaves off and architecture begins”* (cf. [Perry and Wolf, 1992, p. 45]), it seems quite adequate to consider a style as a special role of Capsule.
- pla A Capsule can also be used as a template for a PLA, i.e. family of products that all share some parts of their architecture. Typically, the following items will be shared: the relevant viewpoints, a tailored workflow with a list of applicable process patterns, a domain model, sets of applicable components, styles, and possibly individual members of the product families for reference.
- virtual unit A Capsule realizes a self-contained architectural unit. Although this can be achieved in many cases, it is sometimes not practical to do so, e.g. when reusing components that rely on a broader concept of software architecture than ours. For instance, there may be a set of services that belong together logically, but not physical, as they are distributed or mobile. Such (and other) Capsules are called virtual. Many benefits of architectural units do not apply to virtual Capsules, but it is still necessary to be able to integrate them, at least in the design.

³Note that our whole approach is inspired by ROOM, though it works on a much higher level of abstraction, and is tightly integrated with UML, which is not the case with ROOM.

During its lifecycle, a Capsule may first be a style, then be refined to be a pla, and finally become a component (see sketch).

style —————> pla —————> component

Any Capsule may or may not be virtual or project.

Pragmatics

Initially, a Capsule has to be created. During this stage, it is probably incomplete, inconsistent, unreliable, and may well be discarded again altogether. We call this the project-role of a Capsule. Later on, it may mature to a reusable component or style. During these stages, changes must be monitored much more closely, and much higher quality criteria will have to be met. For different roles, different Views will be needed and required, to different degrees of precision and completeness, enabling and participating in different tasks.

A Capsule may change its role dynamically: after the initial development it may be transformed into a reusable component, say. Alternatively or subsequently, a project may be turned into a style or product family, that is, the knowledge gathered in developing it may be abstracted to a degree that it becomes reusable as a template rather than as a system. Also, a Capsule may play more than one role at a time.

The life-cycle state of a Capsule may be described by the goal of the development and the degree of maturity it has achieved towards this goal.

Embedding

Capsules are the main building blocks of architectural designs, and thus the entities of deployment and design (amongst others). So, they are made both a Component and a Package (see Figure 4.3). A Capsule may contain a number of ModelElements, virtue to Classifier being a kind of Namespace, which in turn owns arbitrary ModelElements. In particular, a Capsule may contain Views, Ports, a ConfigurationSpace and a (single!) Configuration.

In the UML, a pattern is a “*template collaboration*” (cf. [OMG UML 1.3, p. 3-114]). Since a style is rather similar to an architectural pattern, it is tempting to embed style as a subclass of Collaboration. However, it is not clear what a template collaboration is,⁴ and a Collaboration is restricted to contain the ModelElements prescribed by the metamodel, while a pattern—and a style—should be allowed to contain arbitrary specification elements. So, we prefer to make style a special kind of Capsule.

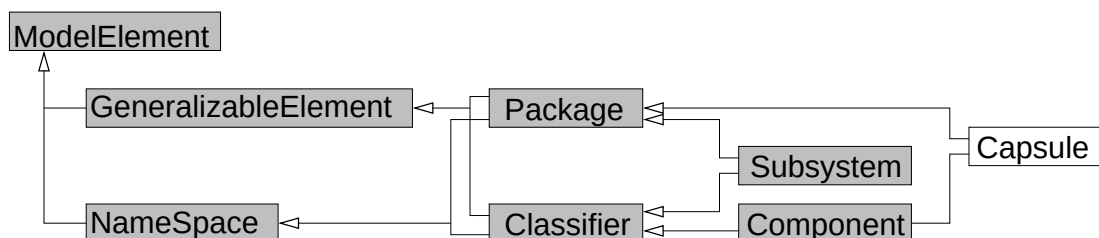


Figure 4.3.: Capsule and SubSystem are close neighbors in the UML metamodel.

⁴It is not even entirely clear, what a Collaboration is, judging by the amount of literature dealing with the problem, cf. Knapp [1999].

Here, we can either make style a subclass of Capsule, introduce a “life-cycle view” or introduce a role attribute. Since roles may change dynamically, the first option is too inflexible. A dedicated View would have provided more freedom, but unlike all other Views, every Capsule has a role, always. So, the role should be a first class citizen, i.e. modeled explicitly. Also, an attribute facilitates expressing certain constraints on Capsules, using OCL.

So, we chose the third option: the attribute role holds a set of the items project, style and component. Being a Capsule, a style is a Package and can import and use other styles; also, this way style is a GeneralizableElement, and may thus participate in an inheritance hierarchy (though there is no formal subtype relationship).

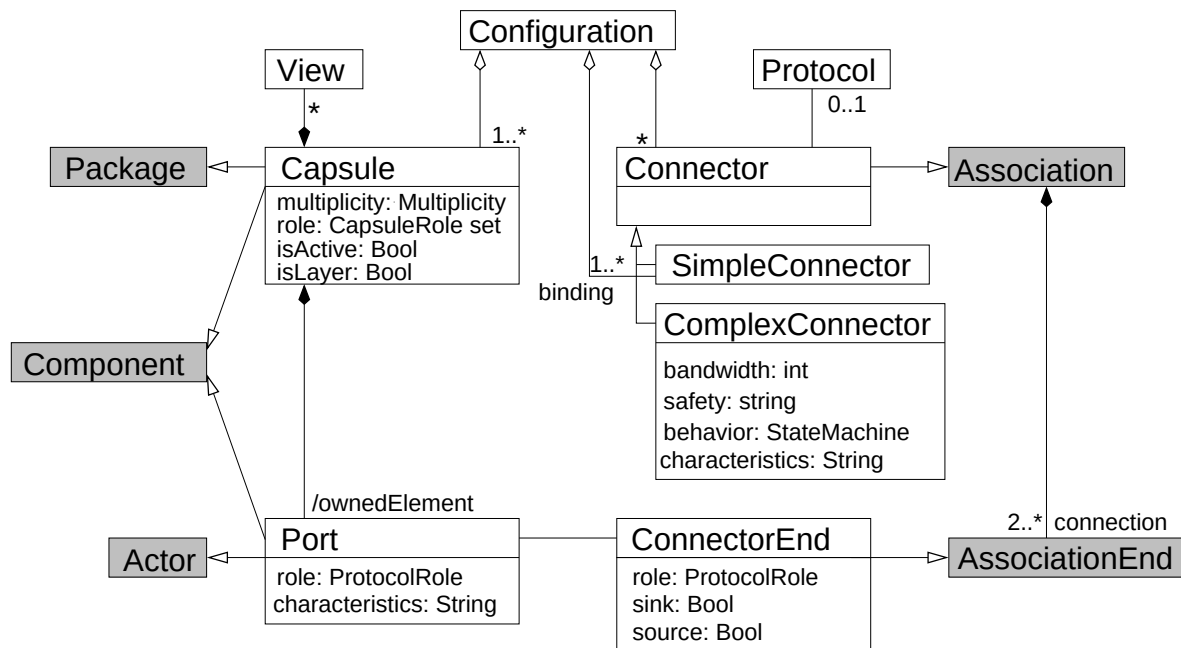


Figure 4.4.: A Configuration consists of Capsules and Connectors. Capsules are Packages, and may own Ports, to which the ConnectorEnds of Connectors are attached. Note that several ConnectorEnds may be attached to a single Port.

We sometimes identify Capsules and their roles, i.e., instead of “a Capsule that plays the role project” we just say project, and similarly for style, component and product family. Capsule has the following attributes and associations.

isActive: Boolean Attribute with the same meaning as the active class-stereotype.

isLayer: Boolean Attribute that indicates whether the respective Capsule is a layer in a layered system.

role: CapsuleRole **set** Where CapsuleRole is an enumeration of project, component, style, product family, and virtual. For pragmatic purposes, it is helpful to be able to distinguish between the different roles of a Capsule. The value project stands for ongoing development projects, component stands for reusable components, style stands for Capsule-templates, and virtual stands for Capsules that are not truly self-contained, which is often the case for legacies. During the life-cycle of a Capsule, its role may change, e.g. from project to component, and

on to style. Capsules may also play more than one role at a time, e.g. component and project for a Capsule which is a development project aiming at producing a reusable component.

view: View **set** The set of all Views specified for the Capsule. A View may refer to any of the ModelElements that a Capsule owns or imports by being a Package.

multiplicity: MultiplicityKind similar to the multiplicity of, say, ClassifierRole.

Constraints

1. Views are unique, i.e., there is always at most one View of a kind.

```
context    Capsule
inv:       self.view ->forall(v1,v2 | v1.kind = v2.kind implies v1 = v2)
```

2. A Capsule with the role component has at least one Port, a Capsule with the role style has no Ports. A Capsule with the role pla has a domain view, and a virtual unit can not be a component.

```
context    Capsule
inv:       self.role ->includes(component) implies
           self.view ->select(name = "interfaces") ->notEmpty
inv:       self.role ->includes(style) implies
           self.view ->select(name = "interfaces") ->isEmpty
inv:       self.role ->includes(pla) implies
           self.view ->select(name = "domain") ->notEmpty
inv:       self.role ->includes(virtual unit) implies
           not self.role ->includes(component)
```

3. A virtual Capsule may not play the role component.

```
inv:       self.role ->includes(virtual) implies
           not self.role ->includes(component)
```

4. A Capsule may play at most one of the roles style, style and component.

```
inv:       self.role ->includes(pla) implies
           self.role ->not includes(component or style)
inv:       self.role ->includes(component) implies
           self.role ->not includes(pla or style)
inv:       self.role ->includes(style) implies
           self.role ->not includes(component or pla)
```

5. A Capsule may not own public ModelElements other than Ports. All Ports must be public.

```

context    Capsule
inv:       self.ownedElement ->
           forall(visibilityKind = "public" implies oclIsTypeOf (Port))
inv:       self.ownedElement ->forall(oclIsTypeOf(Port) |visibilityKind = "public")

```

6. A Capsule may not own public ModelElements other than Ports. All its importedElements must be private.

```

context    Capsule
inv:       self.ownedElement ->
           forall(visibilityKind = "public" implies oclIsTypeOf (Port))
inv:       self.importedElement ->forall(visibilityKind = "private")

```

7. A Capsule must be a self contained NameSpace, i.e. as a Package it must carry the top-level-stereotype.

```

context    Capsule
inv:       self.stereotype ->exists(topLevel)

```

8. The inbound sets of Signals of any two Ports must be disjoint.

```

context    Capsule
inv:       forall p1,p2 |let
           p1 : Port = self.owns.isKindOf (Port)
           p2 : Port = self.owns.isKindOf (Port)
           p1 ≠ p2 implies p1.in disjoint p2.in

```

4.2.2. View

Description

Software systems—that is, Capsules—can have quite large and complex designs. To improve handling of the designs, it may be looked at from a multitude of angles, each focusing on some special aspect of the system, i.e. its behavior, functionality, structure, performance and so on. Also, every activity implicitly defines the items necessary to it being accomplished. Thus, we have defined Capsule to contain a set of Views.

The idea of multiple aspects of a system being represented separately is central to the UML. Complying to the IEEE standard on software architecture IEEE Architecture Working Group [1999a] we use the concepts of viewpoint and view. There is no similar concept in the UML metamodel.

Since a Capsule is a package, technically a view is a filter on a Package that displays the elements belonging to a view, so that they can be shared between different view-Packages. We acknowledge the existence and necessity of a principally unbounded number of relevant viewpoints: there is no convincing reason for restricting ourselves to, say, the 4+1 views of at the heart of the UML (see Section 3.5.1 above). Conversely, not all Capsules need to have instances of all viewpoints (during all development phases). Consider the following examples of viewpoints we have found useful.

interfaces An interfaces view is the simplest abstraction of a system: it just presents the system (i.e. a Capsule) as black box. It also needs to contain (some) Ports, since these are the only points of interaction of a Capsule. Without any Ports, a Capsule is inert, and has no externally visible aspect except its existence. The interfaces view is a useful abstraction, especially in the early phases of development.

functionality A functionality view describes the functionality of a Capsule. It can be described simply by a UseCase, and possibly a Collaboration between the Ports and the UseCases with a number of Interactions. Since a Port is an Actor, the scenarios of use cases can be described quite precisely by the ProtocolRole of the Port. When refining the Capsule, these specifications remain, and ensure a smooth transition between development phases

behavior The behavior view of a Capsule can be described by a StateMachine. The events used in the inscriptions of the arcs of the state transition diagram may refer to the signals sent or received by the Ports.

domain A Capsule may contain a logical model of the application domain, i.e. a domain view, containing its structure and logic (i.e. processes, or any other dynamic aspects). Domain models may be represented by an analysis-level class diagrams and activity or interaction diagrams. See Marshall [1999] (in particular pp. 62) for a detailed description of enterprise modeling with UML. See e.g. Barros et al. [2000] for business modeling with UML.

implementation An implementation view contains items such as source code, binaries, programming-level interfaces, along with test-data and information for tracing between design and code. Supporting the “scaffolding” of other views, there may also be a dummy representation of the implementation.

execution An execution view contains run-time entities, such as threads and processes, objects, temporary files and so on.

structure A Capsule can be refined, i.e. it can be an abstraction of a Configuration, representing the static structure of the architecture of this Capsule. The dynamic structure may be represented as a ConfigurationSpace, that is, a set of Interactions on the Capsules defined the Configuration—the latter are the representedClassifiers of the Collaboration. Together, the static and the dynamic structure are represented in the structure view. The structure view also contains the binding of the Ports of the sub-Configuration to the Ports of the Capsule.

deployment Being a proper system, a Capsule need not be local to a single Node: a sub-architecture may be deployed to a topology.

goals The goals view is available only for Capsules playing the role project. It may contain text-files, context diagram, and specific TaggedValues on Actors (cf. Buhr [1998], Regnell et al. [1997], and IEEE Architecture Working Group [1999a]).

administration An administration view may contain all documents related to schedule, budget, organization etc. It also contains details of the development process that are at the discretion of the developer, such as how the transition from one view to another is accomplished exactly. Thus, the administration view also contains (part of) the development history.

environment Software is developed for a number of platforms. Apart from ordinary computers, software development also targets devices such as cellular phones and palm computers, set top boxes and game consoles, all kinds of embedded systems, e.g. car navigation appliances, stereos etc. When a component-Capsule is to be reused, it might make assumptions on the platform, memory capacity, operating systems features (e.g. preemptive/non-preemptive) and so on. Such requirements can be compiled in an environment view.

Like the implementation view, all other views may contain test-data of the respective viewpoint and level of abstraction, too. So, a functionality view could include descriptions of test cases, a behavior view could include test runs and so on. We abbreviate views by their first letter, or, if there are ambiguities, their first few letters.

Pragmatics

Determining just which viewpoints are relevant for some particular undertaking has to be done during requirements analysis. In which order respective views are then provided, and how they are used is not in any way restricted by our terminological framework. See Chapter 11 for some suggestions.

Embedding

There are basically three alternatives for modeling View: directly as a ModelElement, as a “compartment”, or as a kind of Package. A “compartment”, on the one hand, is not a proper concept at all: there is no such metaclass or meta-attribute. Its semantics is undefined. Second one could use (a stereotype of) Package. Nevertheless, according to its significance, we decided to introduce the view-concept should be as a first-class citizen of the metamodel, i.e. we implement View directly as a subclass of ModelElement.

A View refers to an arbitrary number of ModelElements as me, and owns a set of Diagrams as diagram. A Diagram represents a sub-grouping of PresentationElements. Its PresentationElements are called pe, and the ModelElements are called its me. Both View and Diagram are subclasses of ModelElement. See Figure 4.5 for a schematic representation of the embedding. View has the following attributes and associations.

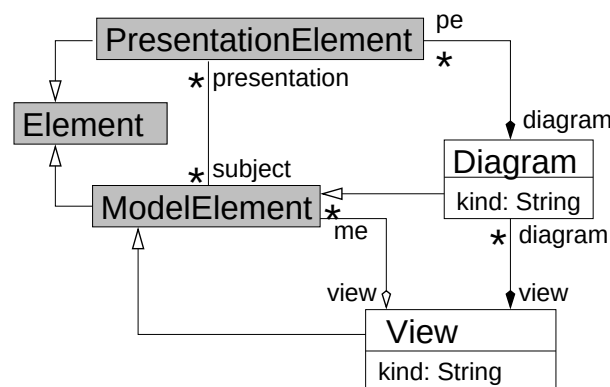


Figure 4.5.: Embedding of View, PresentationElement, and Diagram.

diagram: Diagram **set** The Diagrams that belong to a View. Diagrams are used to structure the PresentationElements referred to by a View. Indirectly, they also express a structure of the ModelElements represented by the PresentationElements.

me: ModelElement **set** The ModelElements belonging to a View. Since several Views may refer to the same ModelElement (e.g. Ports), a weak aggregation must be used.

kind: String The pragmatic function of a view. This attribute should not be an enumeration type (like role of Capsule), as our approach does not in any way restrict the number or function of viewpoints, and so this attribute may take on arbitrary values.

Constraints

1. Every PresentationElement of (some Diagram of) a View must be the presentation of some subject.

```
context    View
inv:       self.me.presentation includesAll self.diagram.pe
```

2. The interface view contains only Ports.

```
context    View
inv:       self.kind = "interface" implies self.me.oclIsTypeOf(Port)
```

3. The behavior view contains only StateMachines and Collaborations. There may only be one such StateMachine.

```
context    View
inv:       self.kind = "behavior" implies self.me.forAll
           (v | v.oclIsKindOf(StateMachine) or v.oclIsTypeOf(Collaboration))
           and self.kind = "behavior" ->collect
           (self.me.oclIsTypeOf(StateMachine)) ->count ≤ 1
```

4. The structure view contains a single Configuration and a set of Connectors that establish the binding between the Configuration and the Ports of the Capsule. It may also contain a ConfigurationSpace.

```
context    View
inv:       self.kind = "structure" implies self.me.oclIsTypeOf(Configuration)
           or oclIsTypeOf(Connector) or oclIsTypeOf(ConfigurationSpace)
           and self.kind = "structure" ->collect
           (self.me.oclIsTypeOf(Configuration)) ->count = 1
           and self.kind = "structure" ->collect
           (self.me.oclIsTypeOf(ConfigurationSpace)) ->count = 1
```

The views formalized here are of special interest in the semantics (see Section 7.4.2 and Chapter 9). Other views may be specified similarly, but this is really up to the application. Note that all views are optional, but if they are present, their contents are required, too.

4.2.3. Port

Description

We have reiterated upon the paramount importance of self-containedness for architectural units, and thus for Capsules: a Capsule must hide not only implementation details, as is usual for modules, abstract data types, objects etc., but also any dependencies on its environment, functional or other. In order to achieve this, we introduce the concept of Port to realize POIs. Its closest relatives in the UML are Interface and `«active»Class`. The set of all Ports of a Capsule is a specification of the system boundary of the Capsule. All interactions of the system (read: Capsule) have to take place via some Port. To this end, a Port may be attached to a ConnectorEnd of a Connector. It constitutes a *contract* between the Capsules connected by that Connector. A Port that is not attached to a Connector of a Configuration is called a *free* Port. Both Ports and ConnectorEnds play a ProtocolRole, so that the attachment may be checked for compatibility. As a special case, we also allow *relay* Ports, that is, Ports without behavior of their own, that just relay whatever Signals are sent to them.

Pragmatics

Note, that unlike Interfaces in UML, Ports control not only access *to* the features of an entity, but also *by* that entity.⁵ Thus, dependencies *on* as well as dependencies *of* a Capsule are made explicit, which is a crucial contribution to the self-containedness of Capsules. On the other hand, this requires additional effort in creating a Capsule. However, in many cases, predefined ProtocolRoles will be available for specifying Ports.

There are several kinds of Ports, distinguished by the kind-attribute. First, there is the standard case, that is, Ports that abstract from the internal behavior of their Capsule and transform this behavior. They are called transformers. An easier case are so called filters, which merely mask out certain Signals. Another frequent case is a Port which just passes on whatever it is handed, such as the Ports of a refined Capsule. Following ROOM, such Ports are called relays.

If a Port represents an individual thread of control (such as when it controls timeouts or error-correction by itself) it is called active. If a Port has only incoming or outgoing Signals, it is called unidirectional. If the role of a Port specifies a kind of document type like PostScript, or HTML, the Port is called document-type Port.

Obviously, more than one of these properties can hold at any one time, so kind is similar to role of Capsule, i.e., it is a set-valued attribute.

Ports may be implemented in different ways, depending on the kind of POI they are representing. For instance, one might use an Interface, a set of Classes, an `«active»Class` or a SubSystem. Relay Ports are not implemented at all. See Section 12.2.1 for some examples.

Embedding

Pragmatically, a Port specifies the system boundary of a Capsule, i.e. it is the point of interaction of the UseCases of a Capsule, and so Port must be a subclass of Actor. Also, since Capsules are to appear

⁵In UML, for example, this would refer to Features of a Classifier, in Java there are attributes of classes or interfaces, and in architectural modeling, there are Signals of a Port's ProtocolRole.

in deployment diagrams, Ports have to be there too, and so, Port must inherit from Component. It has two Features of the type Interface by the names in and out, and an facultative StateMachine that models its ProtocolRole, see Figure 4.4 for the embedding. More abstractly, constraints on Signals may also be expressed using OCL-constraints.

Port is an Actor, and thus a Classifier.⁶ In a Collaboration, a Classifier is represented by a Classifier-Role. ClassifierRoles have an attribute multiplicity, indicating the number of times it may occur in the Interactions of the Collaboration. Thus, in interaction diagrams, Ports may occur with a multiplicity. Port has the following attributes and associations.

characteristics: String Similar to the characteristics of Connector (cf. Section 4.2.5).

role: ProtocolRole Similar to the role of ConnectorEnd (cf. Section 4.4).

constraints: Constraint **set** A set of constraints on the role, i.e. on the (sets of) incoming and/or outgoing Signals.

kind: PortKind **set** Where PortKind is an enumeration of (currently) relay, transformer, filter, active, and unidirectional.

Constraints

1. Ports with kind unidirectional must have an empty set of incoming or outgoing Signals.

```

context      Port
    self.kind = unidirectional ->self.role.in ->isEmpty
or
    self.kind = unidirectional ->self.role.out ->isEmpty
  
```

There are no further constraints on Port. Connectors are always part of a Configuration. The Ports of Capsules of a sub-Configuration belong to a different NameSpace. Thus, Connectors can not tunnel Capsule boundaries.

4.2.4. Configuration, Architecture

Description

A Configuration is an ensemble of architectural units called Capsules, glued together by Connectors via interfaces of the building blocks called Ports (see Figure 4.4). That is, a Configuration is the realization of a set of glued together units. There is no counterpart to Configuration in the UML metamodel so far. An instance of a configuration is called Architecture.

Pragmatics

A Configuration represents the static structure of an architecture.

⁶A Port is also a Component, but Component is also a Classifier.

Embedding

From a theoretical perspective, a Configuration is quite similar to a Collaboration: both describe an ensembles of collaborating entities. However, Configuration is one of the central notions of architectural modeling, so we chose to include it as metaclass of itself, and as an immediate subclass of ModelElement. Configuration has the following attributes and associations.

capsule: Capsule **set** The Capsules that form the Configuration.

connector: Connector **set** The Connectors among the Capsules of the Configuration.

binding: SimpleConnector The binding connects the free Ports of a Configuration with the Ports of the hosting Capsule.

Constraints

1. The ProtocolRoles of Ports connected by a Connector have non-empty incoming (outgoing) sets of Signals, if the respective ConnectorEnd is a source (sink).

```
context    Configuration
inv:      let con: ConnectorEnd = self.connector.connection
          in con.source implies con.port.role.in ->notEmpty
          and con.sink implies con.port.role.out ->notEmpty
```

4.2.5. Connector, ConnectorEnd

Description

A Connector associates two or more Ports. To this purpose, a Connector consists of n ConnectorEnds. Every ConnectorEnd is attached to exactly one Port. Connectors with exactly two ConnectorEnds are called *binary* Connectors. Connectors where all ConnectorEnds play the symmetric role (such as the sink and the source for a particular format) are called *symmetric*. A Connector whose ConnectorEnds are all either only sources or only sinks is called *unidirectional*. Connector is an abstract class with two subclasses, SimpleConnectors and ComplexConnectors. SimpleConnectors have no behavior of their own. All information they transmit is assumed to be transmitted instantaneously and without errors, i.e., with infinite *quality of service*. ComplexConnectors, on the other hand do have behavior, that is, they can be modeled as asynchronous and/or faulty channels.

Pragmatics

Connectors realizes the glue to connect Capsules realizing architectural units. Different kinds of Connectors can be used to model different calling mechanisms, i.e. SimpleConnectors exhibit the characteristics usually associated with local procedure call or method invocation: information exchange via a SimpleConnector can be thought of as conceptually instantaneous, of perfect quality and synchronous.

ComplexConnectors, on the other hand, can be used to model remote connections such as Remote Procedure Call (RPC) or Remote Method Invocation (RMI), which typically are asynchronous, and may have a behavior of their own, such as delay, transmission errors, reordering, corruption or loss of packets etc., limited bandwidth or even processing of their own. For completeness (and less precise specifications), we add attributes to express safety other characteristics as plain text.

Embedding

Connector and ConnectorEnd are subclasses of Association and AssociationEnd, respectively, see Figure 4.7. Thus, they inherit the respective attributes and associations, e.g. Connector has two or more ConnectorEnds, and ConnectorEnd has a multiplicity. Connectors have the following attributes and associations.

protocol: Protocol A Connector (or ConnectorInstance) may be associated to a Protocol that specifies its behavior, that is, the behavioral types of all its ConnectorEnds (ConnectorEndInstances).

SimpleConnectors have no attributes or associations beyond those of Connector. ComplexConnector could have been realized as a subclass of AssociationClass, thus inheriting from Classifier the ability to own a StateMachine. But then, the two conceptually similar classes SimpleConnector and ComplexConnector would have no common abstract class Connector, and they would be placed too far away from each other. So instead, we introduce ComplexConnectors as a subclass of Connector with the following attributes.

characteristics: String Specifies the characteristics of the Connector, i.e. average number of errors in a given interval of time expected for this connector, the transmission rate (maximum number of bits transmittable via this connector in a given interval of time), the time intervals and quality-of-service-parameters.

safety: int The degree of safety of transmissions across this Connector.

bandwidth: int The number of packets transmitted over the Connector per time unit.

behavior: StateMachine The behavior of a ComplexConnector can be modeled explicitly as a StateMachine. This way, all kinds of errors or delays or other qualities of service may be captured.

4.2.6. ConfigurationSpace

Description

The static structure of a system is conveniently represented as a Configuration or an Architecture. Many systems, however, have a dynamic structure as well, i.e. their structure evolves over time. So, a Configuration is really a snapshot of a systems structural evolution. The collection of snapshots and their relationships are represented by a ConfigurationSpace.

Pragmatics

Since all activities in a Configuration or Architecture emerge from Ports and PortInstances, respectively, the evolution itself can also be restricted by their ProtocolRoles, so the dynamic structure may be subject to consistency analysis as well. Note that we use Interactions only as scenarios, i.e. to describe sample runs. Thus, we need not care about activator.

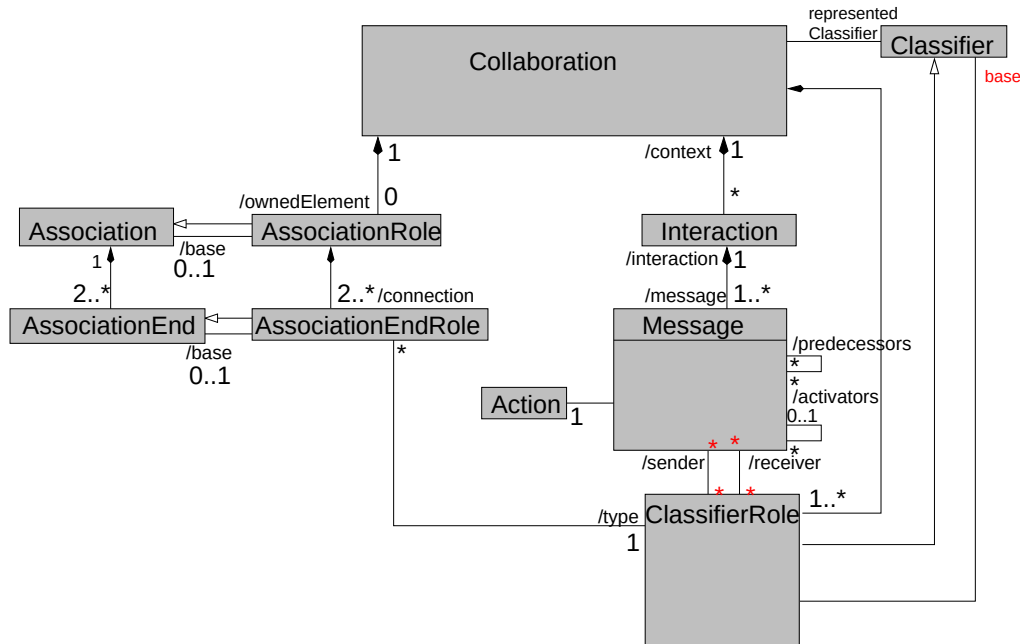


Figure 4.6.: Collaborations in the UML, abridged from [OMG UML 1.3, p. 2-104].

Embedding

Intuitively, ConfigurationSpace can easily be seen as a role model as both Capsule and Port are Classifiers, ClassifierRole can be used to represent them in Interactions. In a Collaboration Classifier and Instance are replaced by ClassifierRole, and Association and AssociationEnd are replaced by AssociationRole and AssociationEndRole (see Figure 4.6).

However, a number of additional metaclasses are required for role models, see Figure 4.7, bottom. Since we use subclasses of Association and AssociationEnd for ConfigurationSpaces, we also need subclasses of AssociationRole and AssociationEndRole. We call these ConnectorEnd and ConnectorEndRole.

Constraints

1. The representedClassifiers of a ConfigurationSpace may be only Ports or Capsules.

```

context    ConfigurationSpace
inv:       self.representedClassifier.oclIsTypeOf(Port)
           or self.representedClassifier.oclIsTypeOf(Capsule)

```

2. A ConfigurationSpace may own only ProtocolRoles, ConnectorRoles, and ConnectorEndRoles.

```

context    ConfigurationSpace

```

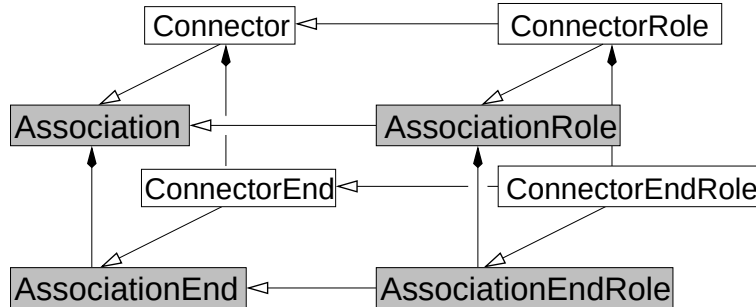
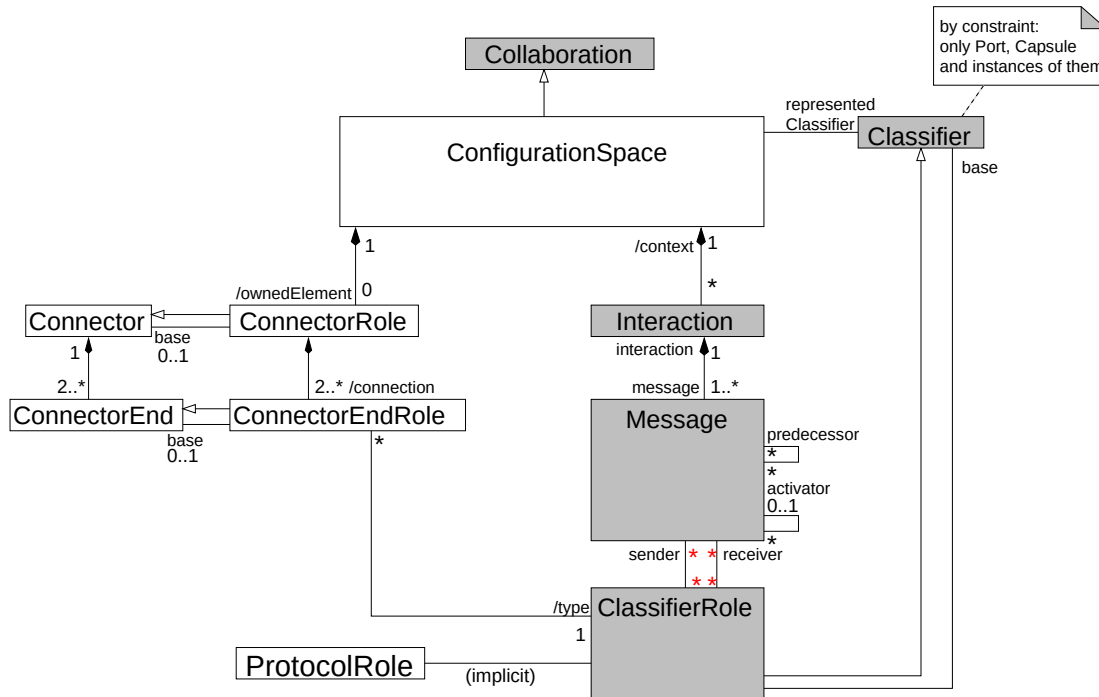



Figure 4.7.: ConfigurationSpace is a subclass of Collaboration, closely matching its structure (top). In order to use the structure of the UML metamodel for Collaboration, it is necessary to also duplicate the “square” of [OMG UML 1.3, p. 2-104] (bottom). Shaded boxes represent UML metaclasses, white boxes represent concepts introduced here.

```

inv:      self.ownedElement.oclIsTypeOf(ProtocolRole)
or        self.ownedElement.oclIsTypeOf(ConnectorRole)
or        self.ownedElement.oclIsTypeOf(ConnectorEndRole)

```

3. The type of a ClassifierRole in a ConfigurationSpace must be a ConnectorEndRole.

```

context   ConfigurationSpace
inv:      self.representedClassifier.base.type.oclIsTypeOf(ConnectorEndRole)

```

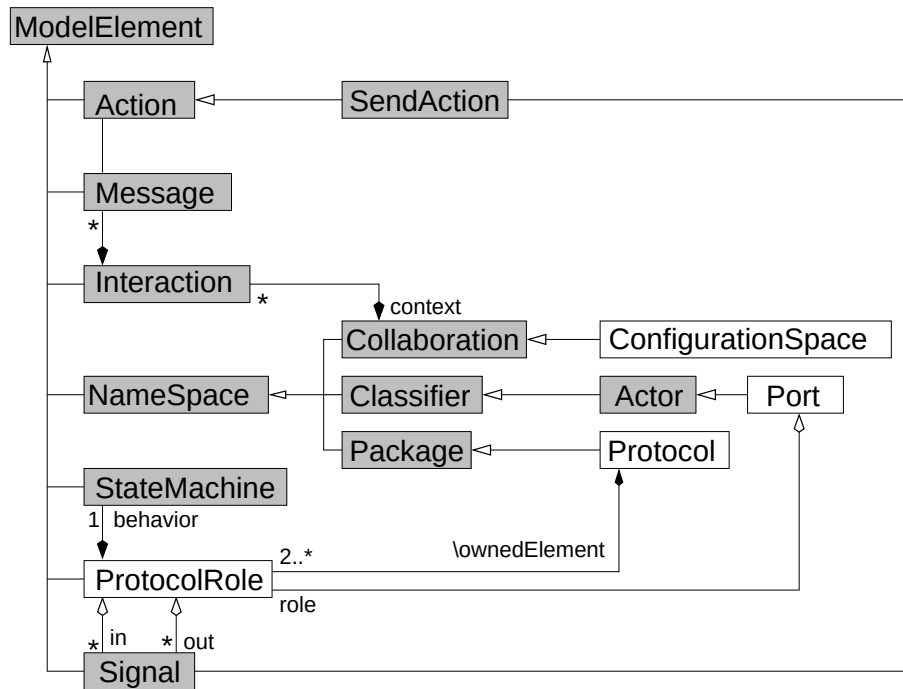


Figure 4.8.: Embedding of ConfigurationSpace and Protocol: the former is a Collaboration of Ports and Capsules, and instances thereof. The latter is a special kind of Package, with some constraints on its contents. Shaded boxes represent UML metaclasses, white boxes represent concepts introduced here.

4.3. Instance level

Description

The concepts we have introduced thus far all live on the type level, i.e. they are similar to Class, rather than Instance. Now we introduce the respective Instances as well. For Port, Capsule, Connector and ConnectorEnd, there are PortInstance, CapsuleInstance, ConnectorInstance, and ConnectorEndInstance. A Configuration may be instantiated to an Architecture.

Pragmatics

Architecture, PortInstance, CapsuleInstance, ConnectorInstance, and ConnectorEndInstance are used exactly as Configuration, Port, Capsule, Connector and ConnectorEnd.

Embedding

The embedding is straightforward: ConnectorInstance and ConnectorEndInstance are subclasses of Link and LinkEnd; Architecture is a subclass of Instance, and PortInstance and CapsuleInstance are subclasses of ComponentInstance, see Figure 4.2.

Constraints

The constraints of Port, Capsule, Connector, ConnectorEnd, and Configuration apply analogously.

4.4. Pattern level**4.4.1. ProtocolRole****Description**

A ProtocolRole consists of two sets of incoming and outgoing Signals, and a StateMachine that specifies the possible behaviors of the role. A ProtocolRole is a kind of *behavioral type* (cf. Liskov and Wing [1994]).

Pragmatics

A ProtocolRole specifies a behavioral type. It can be used as a role-declaration either of a Port or in a Protocol.

Embedding

ProtocolRole is a vital concept to our approach, and it does not resemble any of the standardized ModelElements, so we make it a direct child of ModelElement.

Constraints

1. The ProtocolRole attached to a Port must be public.

```
context    ProtocolRole
inv:      self.in ->forall(visibilityKind = "public")
inv:      self.out ->forall(visibilityKind = "public")
inv:      self.behavior.visibilityKind = "public"
```

4.4.2. Protocol**Description**

A Protocol is a specification of the interactions of a number of Ports. It consists of a set of ProtocolRoles and a Collaboration among these. Like a ProtocolRole is a kind of behavioral type of a Port, a Protocol can be used as a kind of behavioral type of a Connector.

Pragmatics

We distinguish three common types of protocols. First, there are the *control* protocols, that are user-defined and have no further constraints to them. Second, there are the *document format* protocols, i.e. all protocols that describe the exchange of documents of a certain type, like HTTP or PostScript. Typically, such protocols are described as a context-free grammar of the format. Thirdly, there are *interface types*, like USB or TCP/IP. Typically, these are not described in detail, unless one is concerned with actually implementing an USB-slave, say. There may be hierarchies of protocol roles.

Embedding

Protocols contain varying and diverse ModelElements. Also, it is useful to be able to include one Protocol in another, e.g. for describing sub-Protocols. So, Protocol should be a subclass of Package.

Constraints

1. A Protocol may own only ProtocolRoles and a Collaboration between them.

```
context    Protocol
inv:       self.ownedElement ->
           (oclIsTypeOf(ProtocolRole) or oclIsTypeOf(Collaboration))
```

2. There must be exactly one Collaboration in a Protocol.

```
context    Protocol
inv:       self.ownedElement ->collect
           (oclIsTypeOf(ProtocolRole) or oclIsTypeOf(Collaboration)) ->count = 1
```

3. The Collaboration must have the ProtocolRoles as its base.

```
context    Protocol
inv:       let col: Collaboration =
           collect(self.ownedElement.IsTypeOf(Collaboration))
           and roles: set(ProtocolRole) =
           collect(self.ownedElement.IsTypeOf(ProtocolRole))
           in
           col.base isSubsetOf roles
```

4. A Protocol may import or own only other Protocols.

```
context    Protocol
inv:       self.importedElement ->IsTypeOf(Protocol)
```

4.4.3. Abstraction

Description

Abstraction is a subclass of Dependency. Dependencys are binary and directed: they go from exactly one client (also called implementation) to exactly one supplier (also called specification, see [OMG UML 1.3, p. 2-19]). Abstraction has also an attribute mapping of type mappingExpression, that may “*evaluate to*” a Mapping, of explicitly unspecified nature (see [OMG UML 1.3, p. 79]). It can be used to add further detail to an Abstraction. Additionally, there are four stereotypes for Abstraction, «derive», «realize», «refine», and «trace», the meaning of which is vague (see [OMG UML 1.3, p. 2-19]).

Neither Abstraction, nor mappingExpression, nor the stereotypes are given but the skimpiest of descriptions in the UML: a mappingExpression “*evaluates to a Mapping*” and “*the format of the mapping is currently unspecified in UML.*” (cf. [OMG UML 1.3, p. 2-79]) As far as can be inferred from the description in [OMG UML 1.3, p. 2-19], of the four stereotypes for Abstraction, only `«realize»` seems to be relevant for our purposes. No semantics or constraints are provided by the standard.

Pragmatics

Nothing is known about the intended pragmatics of Abstraction in the UML (save [OMG UML 1.3, p. 15]). In this book, we use it to define a formal refinement-relationship between Capsules and Views, in particular between Capsules with roles component and style. We will interpret only Abstractions between Views and Capsules. We will see an example of such a relationship in Figure 5.27 in Section 5.5.5. See Sections 11.3, 11.3, and particularly Section 10.4 on when and how such relationships are established during development.

Embedding

Abstraction is a subclass of Dependency, and thus of Relationship. It is a standard metaclass, and no changes are made, but we interpret it rather freely, so that we have to mention it here.

Constraints

1. The client must be a subclass of the supplier. Note that this constraint must not be demanded for arbitrary Abstractions or Dependencies.

```
context    Abstraction
inv:       self.kind = “realize” implies
           self.clientoclIsKindOf(self.supplier)
```

4.5. Chapter summary

In this chapter, we have defined a logical metamodel to realize the concepts introduced in the previous chapter. We have proposed a conservative extension to the UML metamodel, that guarantees a tight integration of our approach into the UML, but leaves the UML unchanged. All notations, techniques, tools and processes that use the UML remain usable, but can now be extended to cover the architecture level to the full. We have discussed alternatives for embeddings, implications of our proposals, and pragmatic issues related to our concepts.

Following the UML jargon, we refer to this conceptual model as the *abstract* syntax. In the following chapter, we provide a *concrete* syntax, that is, a notation that maps to these concepts. Due to our embedding, we do not have to define a new diagram type proper, only variants of existing notations. It suffices to define the notation as such—the mapping is an obvious bijection.

5. Concrete syntax

5.1. Syntactic Elements

Those metamodel elements that are used without changes (e.g. StateMachine) keep their standard syntax, of course. Those of the concepts we have presented above that are merely restricted versions of plain UML concepts can be represented syntactically by the usual UML syntax. This applies e.g. for Connector, Connection, Protocol and Configuration. Those concepts whose superclasses in the UML metamodel have no graphical equivalent at all, such as ConnectorRole, and ArchitecturalMessage, still need no graphical description.

For the central concepts representing architectural units (i.e. Capsule and Port), on the other hand, we do add notation: the traditional "box-and-line" diagrams known in many a modeling language for decades. However small and unassuming this addition might appear, there are some remarkable repercussions and change requirements on the level of diagram types that we shall discuss separately.

5.1.1. Port and PortInstance

A Port can be represented syntactically by a square with the port's name in it, (see Figure 5.1). In UML, Objects are depicted using the same graphical symbol as for Classes, but their names being underlined. We use the same convention to distinguish a Port from a PortInstance (see Figure 5.1, (a) & (b)). We also distinguish between active and passive Ports¹ i.e. such that are able to start interactions with other Ports connected to them by themselves, and those that need to be triggered from the outside. Again, we use the convention of UML, where active classes are marked with a heavy outline (see Figure 5.1, (c)). Of course, there are also active PortInstances, marked by both a fat outline and an underlined name.

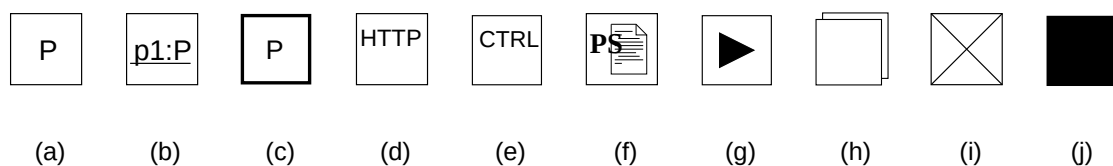


Figure 5.1.: The graphical symbols for (left to right): a plain Port, PortInstance, active Port, a HTTP-Port, a control-Port, a Port accepting and/or emitting the PostScript document-format, a unidirectional Port, a Port with multiplicity higher than one, a relay, and a filter-Port.

The internals of a Port can be specified by whatever is appropriate, in the context of UML, the behavior will usually be described by a State Transition Diagram. However, Interaction Diagrams or Activity Diagrams are also legal. For applications, where files are transmitted via the ports, it can be advantageous to describe the Ports behavior by a grammar of the file format (such as, say, PostScript,

¹We use the terms active and passive in the sense coined in OMG UML 1.3, see in particular the glossary and p. 2-27.

see (f)). The sets of acceptable and emittable signals may be represented either as a Interface, or as plain lists of the symbols. Further properties of Ports might be given a syntactic representation, too (see (f) for a Port with a pragmatically distinguished role and (g) for a unidirectional Port). Finally, there are relay Ports, that is, Ports without behavior of their own, that just relay whatever Signals are sent to them.

For simplicity, the name of a Port may be omitted if it is clear from the context or irrelevant.

5.1.2. Capsule and CapsuleInstance

The prime concept of our approach is that of architectural unit, represented by Capsules. We introduce as a syntactical element a rectangle with its Ports sitting on the outline. Not all of the ports of a Capsule need be displayed. A Port may also appear more than once, if it plays several ClassifierRoles. In that case, the role has to be indicated in the usual UML-way (see [OMG UML 1.3, 3-120]). This symbol is genuinely new to the UML (see Figure 5.2 a), though it is not at all new in itself: it also appears in many other approaches, including UML/RT Buhr [1998], Buhr and Casselman [1996], Luckham et al. [1995], Lyons [1998], Magee and Kramer [1996], Selic and Rumbaugh [1998], Shaw et al. [1995], Simpson [1998]. The earliest publication we found using it dates back to 1986, see Estrin et al. [1986], but it seems to have been in the folklore even longer.

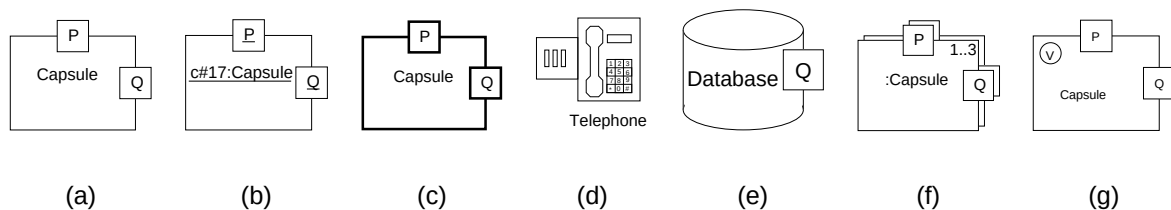


Figure 5.2.: The graphical symbols different types of Capsules. From left to right: a plain Capsule, a CapsuleInstance, active Capsule, two Capsules representing a phone and a database, and a Capsule-type that may appear with at least 1, and at most 3 instances. The rightmost icon represents a virtual Capsule.

Similar to PortInstances and active Ports, we introduce special symbols for CapsuleInstances and active Capsules (see Figure 5.2, (b) & (c)). As Capsule is the most used concept in architectural design, it is worthwhile to distinguish a number of special cases. One of the most frequent examples of architectural units are databases. We use the traditional drum-shaped graphical symbol to depict databases, additionally endowed with the Ports of a Capsule (see Figure 5.2, (d)). Other notations may be introduced in the same way (e.g. e for a phone).

Being Classifiers, both Capsules and Ports can be viewed upon as ClassifierRoles, and as such they have a multiplicity. In the UML, this is represented by explicitly displaying it in the top right corner (see Figure 5.2, (h)), and possibly by a specific version of the original symbol. As styles are special kinds of Capsules, there is even a graphical representation for styles.

For simplicity, the name of a Capsule may be omitted if it is clear from the context or irrelevant.

5.1.3. Connector and ConnectorInstance

Connectors are Associations, and they are represented as such, i.e. by a solid line (Figure 5.3 a). Unidirectional Connectors may be decorated by an arrowhead, similar to the Message-directions in collaboration diagrams. n -ary Connectors are presented by tree. SimpleConnectors can be used if the

information exchange is conceptually transparent, synchronous and instantaneous, such as with local procedure call or method invocations.

ComplexConnectors are presented as double lines, possibly decorated by the kind of behavior exhibited by the Connector, such as delay, loss of information, corruption of messages, independent processing etc. Other syntactic forms of ComplexConnector may include symbols such as the one in Figure 5.3 (f).

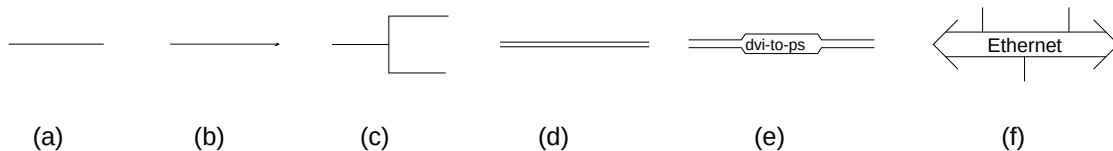


Figure 5.3.: Syntactical representations for different types of Connectors (a to c): plain, unidirectional, and ternary Connectors. All of these are SimpleConnectors, i.e. they have no behavior of their own. ComplexConnectors do have behavior, e.g. errors, delay, or proper processing. They are presented as a pair of parallel lines, possibly with the respective behavior indicated in the broader middle section (d to f): plain, processing, bus-like ComplexConnectors.

5.1.4. Nodes

There are a number of traditional symbols for various items of data processing hardware, in particular computers of different sizes. It might be helpful to adapt these traditional symbols for physical nodes in the sense of UML (i.e. Nodes). This way, they can simply be used in UML deployment diagrams.

5.1.5. ProtocolRole, Protocol

A protocol is the specification of a set of Roles, each of which have their own behavior represented by a StateMachine. ProtocolRoles have no immediate corresponding PresentationElement but, in designing a Protocol, concurrent sub-states of StateMachines may be used to represent (the behavior of) the ProtocolRoles, i.e., a Protocol appears as a StateMachine with a concurrent region for each of its ProtocolRoles (see Figure 5.4). It is important to observe that this is possible only if the Connector sporting this Protocol is synchronous, for otherwise, the standard (synchronous) semantics of StateMachines does not fit. This effect may be achieved with an appropriate implementation for the event queue, that represents asynchronous communication adequately.

5.1.6. Other metaclasses

There are no PresentationElements as presentations of View, Diagram, ConnectorEnd and ConnectorEndInstance. Configuration and Architecture are represented graphically as capsule diagrams.

5.2. Usage of Capsule in diagram types

As Capsules are to realize architectural units, they are supposed to be usable in all phases and activities of software engineering. Thus, they need to appear in different views on a system, i.e. in different diagram types. In the following we consider all the nine diagram types of the UML in turn wrt. the role Capsules may play there virtue to their metamodel-embedding.

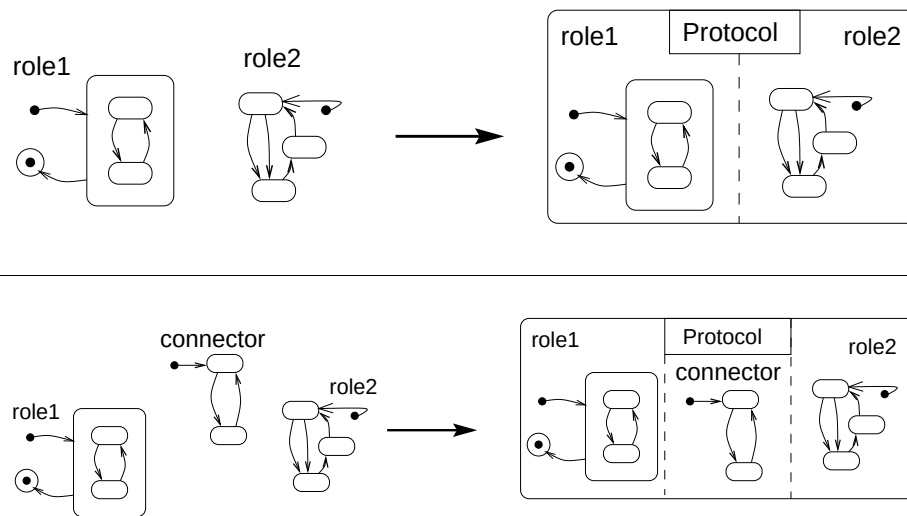


Figure 5.4.: Representing a Protocol as a single StateMachine: for SimpleConnectors, each individual Protocol-Roles becomes a concurrent region of a StateMachine (top); for ComplexConnectors, the additional behavior of the Connector becomes an additional region.

5.2.1. Structural diagrams

The UML features four diagram types, that are predominantly occupied with structural aspects: package, component, deployment and class diagrams. Other diagram types such as interaction diagrams also contain static aspects, but only to a lesser degree.

Component diagram

In UML, component diagrams consist of a set of Components² connected by Dependencies. They have a type form only, but no instance form, i.e. there are Components, but no ComponentInstances (see [OMG UML 1.3, p. 165]). No justification is given for this rather awkward restriction, so we will drop it where appropriate.

Since component diagrams may only contain Dependencies, but no other Relationships and in particular no Associations, component diagrams can not be used to show Configurations.

Deployment diagram

Being Components, Capsules and Ports have a deploymentLocation, and may appear in deployment diagrams. Similarly, CapsuleInstances and PortInstances are ComponentInstances, and thus residents of a NodeInstance. As an additional constraint, we require that if in a deployment diagram there are Capsules or Ports, then there may be no ordinary components (similar for CapsuleInstances and PortInstances). Actually, this way we define a stereotype for a diagram type, a sub-diagram type, similar to object diagrams being a subtype of class diagrams.

Note that while physical Nodes are connected by Associations, the Dependencies of residents of Nodes may be shown directly, crossing the boundaries of the Nodes, see [OMG UML 1.3, Figs. 3-82, 3-83]. In instance-level diagrams, NodeInstances are connected by Links, so these should be al-

²It seems, that other kinds of Classifier, at least Interfaces, might be allowed in component diagrams as well.

lowed to appear in component and deployment diagrams as well, but this is not mentioned in the standard. Associations such as Connectors are realized by Dependencys such as Usage or Binding, depending on whether or not a PortInstance is realized by a binary object. In order to simplify the notation we introduce syntactical stereotypes for these Dependencys, that have as their graphical representation the same symbol as the Connector they realize, (see Figure 5.5).

In component diagrams, confusion might arise concerning the mapping of Relationships to Links, i.e. logical to physical connection. Strictly speaking, no Relationships may appear in component diagrams at all, neither within nor among Nodes. Clearly, this is overly restrictive. One possibility would be color-coding, in analogy to electric cords: different lines within the same cord are given sheaths of different colors. Similarly, different Connectors (or indeed: Relationships) that use the same Link can be drawn in different colors. We distinguish between a type and an instance form of deployment diagrams, exactly analogous to class and object diagrams.

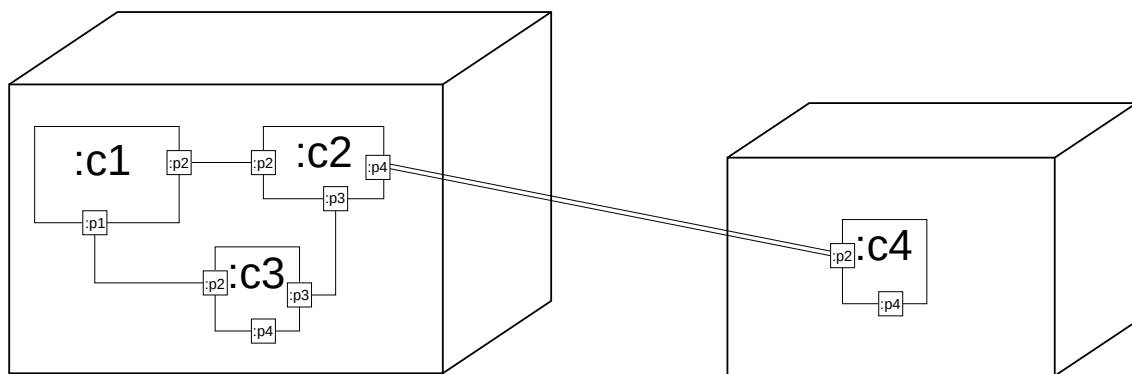


Figure 5.5.: A deployed Configuration.

Static structure diagrams

A static structure diagram may contain Classes, Objects, Packages, Interfaces and all their Relationships. One might stipulate that class, object, and package diagrams contain only Classes, Objects, and Packages, respectively, plus appropriate Relationships and Links, and possibly Interfaces. Such a definition, however, is not part of the UML standard. In practice, all these elements are frequently mixed together, resulting often in silly graphs of indeterminate semantics.

Capsule diagram

We introduce two other special cases of structural diagrams, the so called capsule and capsule instance diagrams (also known as architectural structure and architectural instance diagrams). This notation can be traced back to Estrin et al. [1986]. A capsule diagram consists of a set of Capsules, Ports and Connectors, where every Port must be attached to exactly one Capsule, and Connectors may be attached to Ports.

The instance form contains CapsuleInstances, PortInstances and ConnectorInstances. A capsule instance diagram consists of a set of CapsuleInstances, PortInstances and ConnectorInstances, where every PortInstance must be attached to exactly one CapsuleInstance, and ConnectorInstances may be attached to PortInstances. All the conventions for ordinary static structure diagrams apply.

5.2.2. Behavioral diagrams

Capsules do not appear in state transition diagrams. In activity diagrams, a swimlane (i.e. Partition) might be assigned to a Capsule or Port, but this is inconsequential.

Interaction diagrams

Capsules and Ports are Classifiers, thus they can be represented by ClassifierRoles, and so they can appear without further ado in all interaction diagrams. Also, a ProtocolRole is a ClassifierRole, and thus can also appear in all interaction diagrams. Note that in interaction diagrams both the creation and deletion of roles, and also the dynamic interconnection of roles is expressible.

Activity diagrams

As all notations of the UML, activity diagrams can be used to describe the internals of a Capsule. They can also be used to describe scenarios, similar to interaction diagrams, where the swimlanes take on a role similar to that of roles in an interaction diagram. It is currently not clear, whether activity diagrams might be useful in the description of workflows or business processes spreading several capsules, or whether they can be used in a way similar to use case maps Buhr [1998].

State transition diagrams

State transition diagrams play an important role in modeling behavior, such as the behavior of ProtocolRoles or Capsules. As such, state transition diagrams (or tabular representations thereof) may appear inside Protocols, Ports, and Capsules. None of the concepts we have introduced above, however, appears in state transition diagrams (other than by reference to their name). So, there are no implications for state transition diagrams, too.

5.2.3. Context diagrams

In the UML, there is a single form of context diagram, the use case diagram. It consists of Actors and UseCases connected by a selection of Relationships in the following way (see [OMG UML 1.3, p. 3-89, section 3.53.2]). Consider Figure 5.6 as an example.

	Actor	UseCase
Actor	Generalization	Association
UseCase	Association	Generalization, Association, Includes, Extends

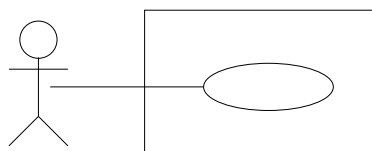


Figure 5.6.: A simple use case diagram: the system-bounding box is just an optional graphical feature without meaning.

The standard mentions also, that Interfaces are allowed in use case diagrams, but they are not mentioned any further, and in particular, nothing is said about who might offer or use these interfaces. It is also not entirely obvious, what an Interface might do in a use case diagram, if a use case describes only functionality.

A set of use cases may be enclosed by a box, to denote the boundary of the system under development. This is just a visual cue, however, there is no corresponding concept of system or system boundary in the UML metamodel. Note that Subsystems are not allowed to appear in use case diagrams, though a Package may of course contain UseCases.

It has been noticed before, that with these expressive means, it is impossible to model the technical or business context of a system (see e.g. Allweyer and Loos [1998], Arlow et al. [1998]). We think that this is one of the most serious shortcomings of the UML. When it comes to modeling a system architecture, the various aspects of a system's context are of prime importance. For instance, when starting a system development, it is not necessarily obvious, what parts of the system and its functionality are suitable to automatization. So it is advisable to first model the whole system, and only afterwards decide where the system boundary is to be drawn.

In the following, we propose a very subtle modification to the UML, that solves this problem to a considerable degree. We enhance use case diagrams by allowing also Capsules and Ports to appear there, with the following Relationships:

	Actor	Port	UseCase
Actor	Association, Generalization	Association	Association
Port	Association	Connector	Association
UseCase	Association	Association	Association, Generalization, Includes, Extends

Note that Ports are Actors already, and thus, strictly speaking, need not be introduced to use case diagrams separately. Capsules do not take part in any connections directly, but only via the Ports that are attached to them, so they do not appear in this matrix. Considering this, the only real change we make to the semantics of UML is allowing Associations between Actors.

It is now possible to draw a diagrams like in Figures 5.7. The diagram looks very familiar, but contains a lot more information than the traditional use case diagram. In Figure 5.7, there is a system with a use case, an actor of the system interacting with that use case, and another actor, interacting with the first actor. This second actor is part of the system's context, and may be the rationale for the system's development. It is important to be able to represent such contextual elements, too.

Figure 5.8 shows notational possibilities opened up by the multi-viewpoint concept. On the left, there is a system, represented by a Capsule, its Port, and an Actor. The Port describes exactly the interactions between the system and the Actor. It is possible to precisely specify the interactions between the system and the environment as the ProtocolRole associated to the Port. This can later be the basis for a system test. The Capsule may also be looked at from the static structure viewpoint (Figure 5.8, right), or indeed any other viewpoint. If the Capsule is refined and the sub-capsules again are looked at from the functionality viewpoint, this amounts to a form of *hierarchical use case s* (more on this in Section 5.3.4).

Several further enhancements have been proposed for use case diagrams Buhr [1998], Regnell et al. [1997] (cf. Section 2.3). Note that modeling the physical topology of computing nodes, i.e. really a part of the deployment might belong to modeling the context, too. Concerning legal issues and other context issues, the physical location might also become important (e.g. it might be of crucial importance, what legislation is relevant for a web-server).

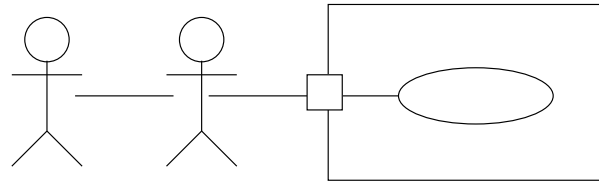


Figure 5.7.: By virtue of allowing actors to be connected by associations, part of the business context can be modeled, here exemplified by the customer.



Figure 5.8.: Enhanced use case diagram showing a system and an actor (left). The system is represented by a Capsule, its Port describes exactly the interactions between the system and the Actor. When displaying the functionality view of the Capsule (right), the resulting diagram looks very familiar, but contains a lot more information.

5.3. Diagrammatic representations for viewpoints

So far, we have considered in which of the standard UML diagrams Capsules and Ports may appear. Now we examine how the standard diagrams may be used to display the various viewpoints of a Capsule.

5.3.1. Viewpoints identical to diagram types

Some viewpoints can be represented more or less directly by a certain UML diagram type, so that there is a simple correspondence viewpoint–diagram type and view–diagram. This holds for example for the following viewpoints:

interfaces The interfaces viewpoint of a given Capsule consists just of the Capsule and its Ports.

behavior The behavior of a Capsule can be described by state transition diagram. Note that there is no graphical connection between the Ports and the state transition diagram

implementation An implementation viewpoint can be described by a component diagram, possibly with some Packages also. Additionally, object-diagrams might describe execution-states.

structure A Capsule can be refined, i.e. it can be an abstraction of a Configuration, representing the static structure of the architecture of this Capsule. It can be represented as a capsule diagram, connected to the Ports of the enclosing Capsule by Connectors, i.e. the binding. The dynamic aspect of the structure is captured by a ConfigurationSpace.

deployment Being a proper system, a Capsule need not be local to a single Node: a sub-architecture may be deployed to a topology, represented by a standard deployment diagram.

In fact, everything that can be expressed in the UML constitutes a proper viewpoint. For the interfaces, behavior, domain, implementation and structure viewpoints, there are both type and instance variants (i.e. similar to class and object diagrams) with appropriate additional constraints.

5.3.2. Viewpoints derivable from diagram types

Other viewpoints might include additional ModelElements, or more than one instance of one or more diagram types. This is the case for the following viewpoints.

functionality A functionality viewpoint can be described by a traditional use case diagram. However, the functionality requires also scenarios among the Actors (i.e. Ports) and the Use-Cases (e.g. in the form of interaction diagrams), and the ProtocolRoles of the Ports (e.g. a state transition diagram and yet more interaction diagrams). Note that our understanding of a use case is closer to the original notion (see e.g. Jacobson et al. [1992]) than what the UML calls use case.

context The context view includes on the one hand one or more context diagrams, i.e. interfaces and actors, and possibly use cases. On the other hand it may contain additional annotations and constraints, a description of the project goal and other requirements or contextual items. All of these can be seen as Comments on the context diagram.

performance A performance view can be derived from either the structure view or the deployment view as inscriptions on performance-related aspects of the respective ModelElements (see Figure 5.14, bottom).

domain The structure of an application domain of a Capsule can be described by an analysis-level class diagram. The domain logic (i.e. the workflow) in an application domain could be modeled by UML activity diagrams.³

Table 5.1 shows the viewpoints mentioned before with the ModelElements that can be part of it. Note, that they all contain Ports, i.e. the interfaces of the enclosing Capsule. All viewpoints may contain additional Comments.

Abstract examples of these viewpoints are given in Figure 5.9. The list given before is by no means exhaustive, one might also consider viewpoints for reliability/availability, usability, safety, security, schedulability, and many more.

5.3.3. Viewpoints without UML representation

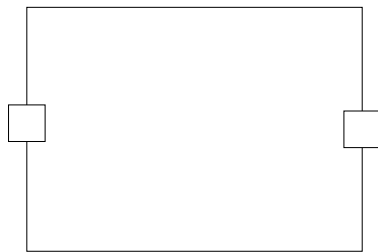
For many users, the UML notations are too technical, yet it is the users that need to provide and understand these models. Also, it might be the case that there is no appropriate diagram type in the UML, or that there is no diagrammatic representation of a certain aspect at all, or that for some purposes these aspects are best shown in a textual form in a non-UML diagram.

schedule There are a large number of diagrammatic representations of aspects of project management, such as PERT-charts and Gantt-charts.

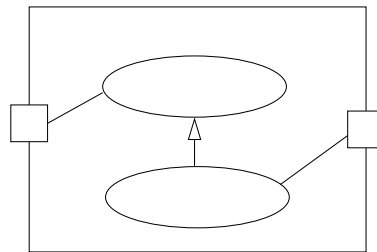
³However, an adequate model of the domain logic often plays a crucial role in process improvement: here, the domain experts play an important role, yet they frequently lack the technical background to use bare UML. So, simpler notations have been invented to better support the users. For instance, for the introduction of standard ERP software, so called event process chains have been used to good effect.

viewpoint	abbr.	contained elements
interfaces	i	Capsule, Port
behavior	b	Capsule, Port, StateMachine
domain	d	Capsule, Port, Class, Relationship, ActivityGraph
implementation	im	Capsule, Port, Component, Package, Dependency
structure	s	Capsule, Port, Connector
deployment	de	Capsule, Port, Component, Package, Node, Relationship
functionality	f	Capsule, Port, UseCase, Relationship, ActivityGraph, Collaboration

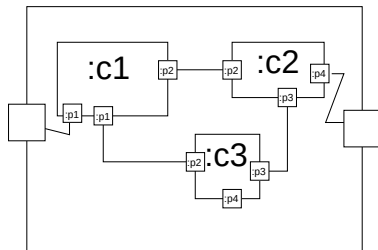
Table 5.1.: Some examples of viewpoints and ModelElements contained in them. The viewpoints context, performance, and administration have no UML representation. See also Figure 5.10 and Appendix D.



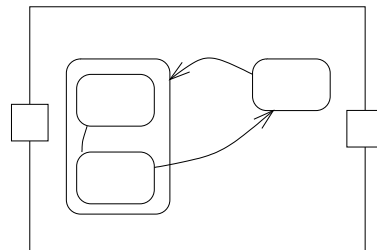
(a)



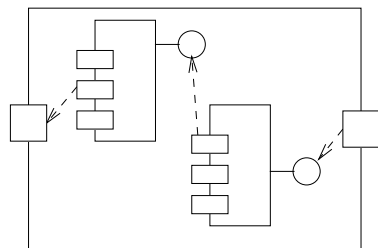
(b)



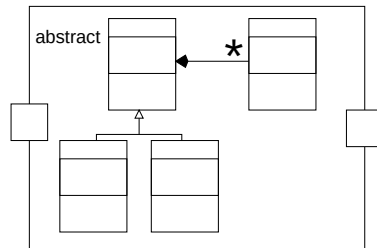
(c)



(d)



(e)



(f)

Figure 5.9.: Some views of a Capsule: interfaces (a), functionality (b), architecture (c), behavior (d), implementation (e), and domain structure (f).

environment The execution environment of a Capsule might simply be described as a natural language text, or a table.

budget A project’s budget is probably best represented as a spreadsheet, possibly with additional natural language texts.

The following sketch Figure 5.10 presents a synopsis over these viewpoints. Each of them is represented by a small diagram or set of diagrams that may be used in the respective viewpoint.

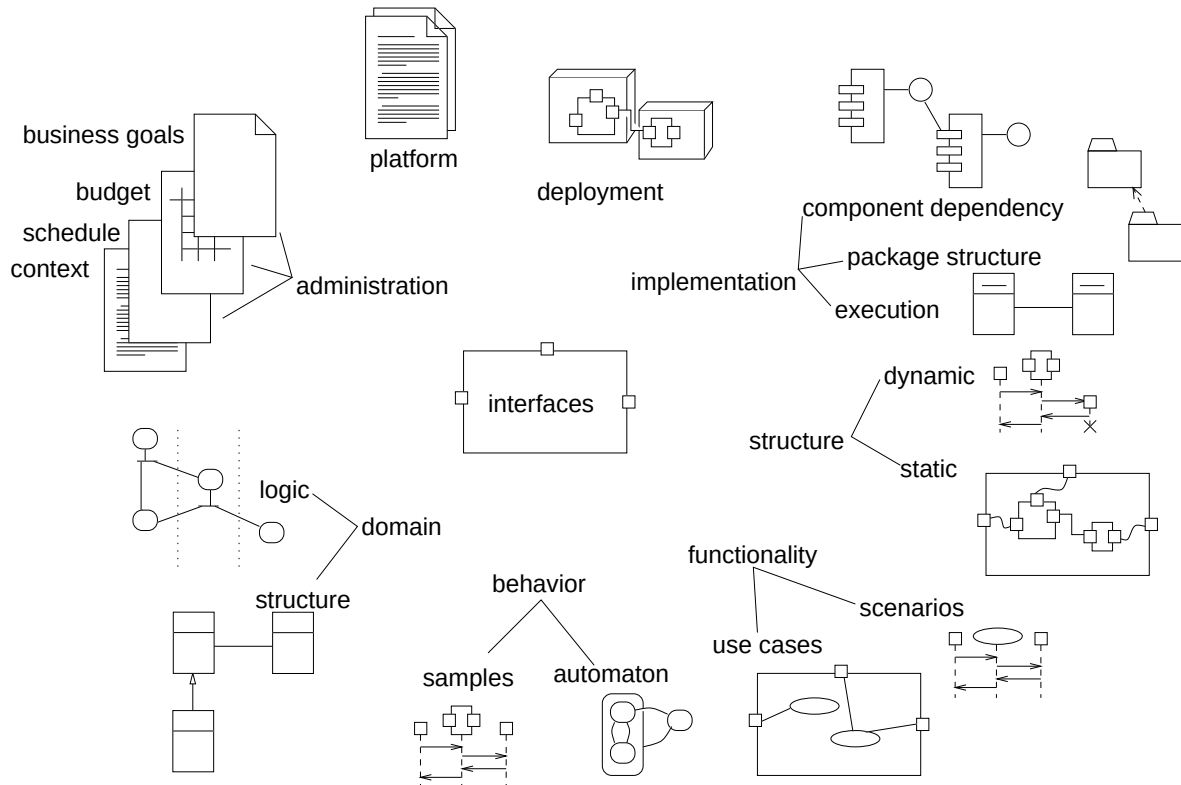


Figure 5.10.: Overview of views, and the modeling elements and diagrams they may contain.

5.3.4. Recursive presentation of viewpoints

A special role is played by the structure view: it may contain sub-Capsules, each with its own set of views. Practical experience suggests that it is sometimes helpful to display the “internal” views as being graphically embedded in the outline of the Capsule they belong to, in order to build recursive presentations of views as in Figure 5.11. Another example of a recursively presented view will be given with the layered system of Figure 5.24. Note that the system shown there is also an example of how it can be advantageous to show the Capsules of a Configuration with different views: for the underlying layers of a layered system, only their functionality is of interest, if the system is trusted to satisfy its requirements. The top-most level, however, needs to be described from more or just different viewpoints. Generally, all viewpoints can be used recursively, though not all possibilities make sense pragmatically. For recursive versions of the respective views, Capsule, Port and Connector need to be added in Table 5.1.

A recursive presentation of the functionality view results in something reminiscent of a classical data flow diagram. It is in fact not a data flow diagram, however, since the decomposition is structural, not functional. On the contrary, a data flow diagram may be seen as a degenerate case of a context diagram.

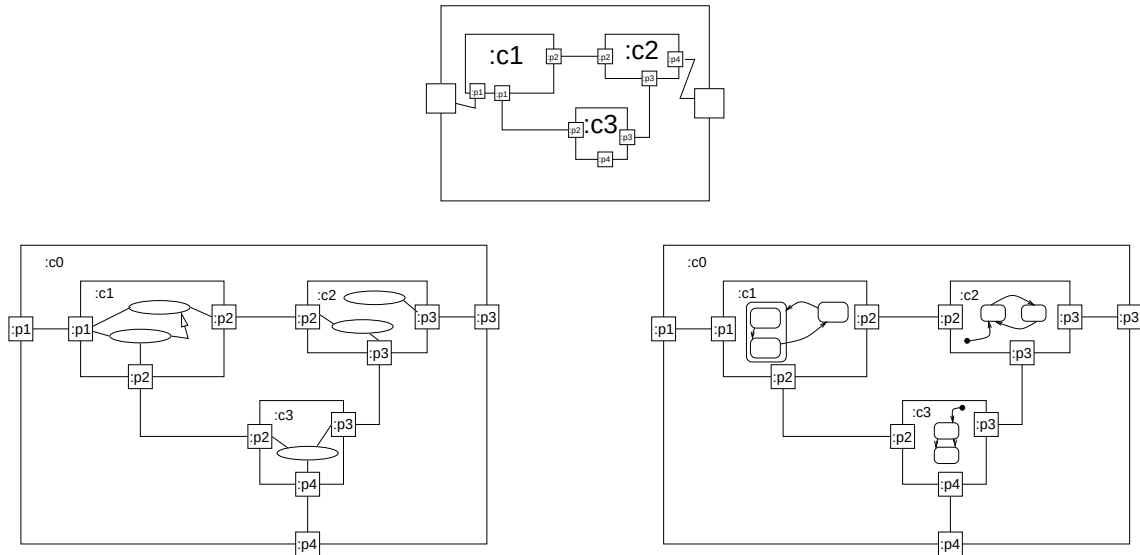


Figure 5.11.: Two examples of recursive presentations of views: architectural view of a capsule (top), and the functionality and behavior view of one further refinement level (below, left and right, respectively).

5.4. First examples

In this section, we provide some first small examples of how the concepts and syntax provided in the previous chapters can be used for architectural modeling. The examples are somewhat artificial, in that they serve only to demonstrate the expressiveness of the syntax.⁴ We will turn to issues like analysis of designs later, after having introduced the formal semantics.

5.4.1. A simple POS-support system

Consider as our first example a point of sale (POS) support system, e.g. for banking services. Customers will walk up to the counter, and request certain services from the clerk, such as transferring money from one account to another, say, or withdrawing money. A clerk provides these services with the help of the support system. This situation is described in Figure 5.12 (top, middle). Assume for the sake of argument, that this system is realized in a classical three-tier *client/server-architecture*, i.e. with a set of identical front end components for the (graphical) presentation and user interaction, a single application server that executes the business logic, and a database server to store and retrieve the data driving the business process. This refinement of the initial architecture is shown in Figure 5.12,

⁴We have tried to obtain real-life large-scale examples from several banks and mobile phone manufacturers, but to no avail.

bottom).⁵ In the remainder, we will concentrate on the architecture of the banking system, i.e. we will omit the context it is embedded in. Of course, there is not just a single branch of the bank and a single clerk, but several of them, but we model only one representative of clerk, i.e. we model the *role* clerk. The front-end is running on a machine local to the branch, while the back-end is running on a central sever of the bank, (see Figure 5.12) (bottom).

So far, we have provided the embedding of the system, and several views on its architecture: its functionality, structure and deployment, and a little bit of its behavior. We have not described formally the scenarios, or the user interactions.

Now suppose, that the architecture is to be enhanced so as to provide these services outside the opening hours of the bank. For withdrawing money, automated teller machines (ATM) are installed, and for money transfers, customers may use a simple internet access. Later, this route of access might be enhanced so as to offer a larger variety of services.

First of all, the added value for the customer is documented as a goal, namely that of having the bank's services available day and night. Second, the new elements of the architecture need to be introduced: ATMs and an internet interface. Both of them will require adapters for the interface of back-end. For clarity, we use extra Port-symbols, though we make no semantic change. This new architecture is shown in Figure 5.14 (top). We can provide a deployment view, too, see Figure 5.14 (middle). An interesting aspect of such a system is whether the performance will carry the expected load. To document the available resources and the expected demands, a load view can be created (see 5.14, bottom). Here, all the relevant data are annotated to the elements of the deployment view. Similarly, a security-view might be created.

Chapter 12 will elaborate on this example.

5.4.2. A call-by-call least cost router

In this section, we describe the case-study presented in Mattern et al. [1998] with the concepts and notations presented above. The case study describes the implementation a Least cost router (LCR). It has been carried out at the TH Darmstadt as a one-term-assignment given for masters students.

Background

With the liberalization of the telecom market in Germany, a large number of providers have entered the market place, fiercely competing for customers, mainly by the price of long-distance calls. Here, the most popular way of choosing a provider is call-by-call-selection, i.e. the choice of a provider for every single call. To select a particular provider, a customer has to dial the provider's individual five-digit-code, the area code, and finally the desired number itself. Depending on the provider, billing might be carried out by one of several methods.

With the growing competition, the number of tariffs has grown enormously. New deals are being offered almost on a daily basis, so that today it is virtually impossible for an individual to keep up to date with the competing offers. So, it is currently a very tedious and time-consuming job to find out, for any given day, time of day, distance and length of call, which provider offers the best deal. Thus, there is now an end-consumer market for devices that ensure just this automatically. This functionality is called least cost routing.

⁵The middle tier might be further refined as a layered system (see Section 5.5), with a detailed description of the behavior of the top layer and coarse descriptions of the functionality of the lower layers, say.

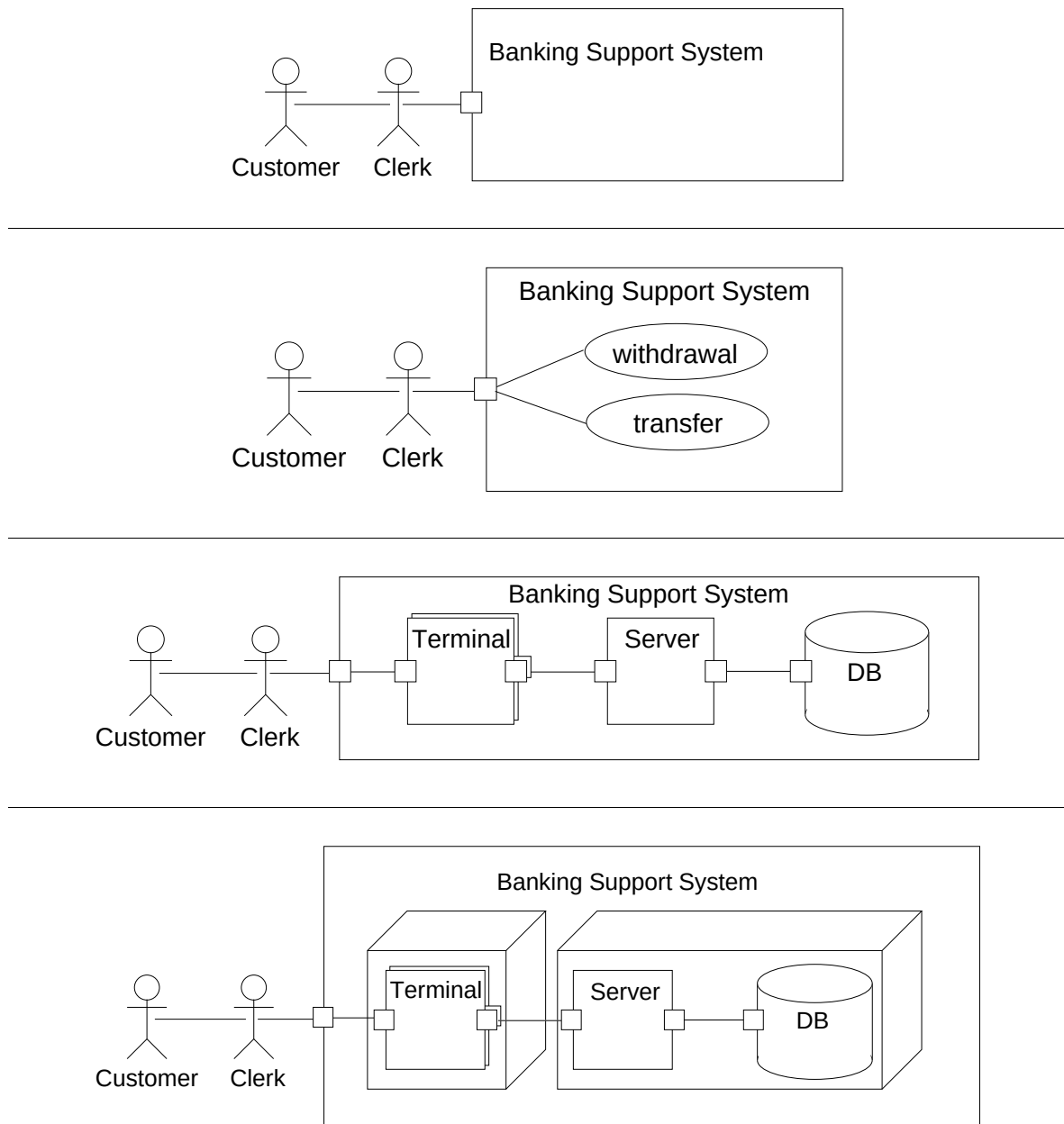


Figure 5.12.: Four views on a POS-support system (from top to bottom): the interfaces, functionality, architecture and deployment view. All views are shown embedded in the systems context.

Design alternatives discussed in the original case study

As there are frequent changes to the available tariffs and providers, updates of the routing data is a crucial issue for least cost routing. Depending on the hardware assumed to be available, the convenience expected by the user and the quality of routing, several architectural alternatives are suggested and discussed in [Mattern et al., 1998, pp. 4–5, 9–16].

1. As the first family of alternatives, architectures without a private branch exchange (PBX), but

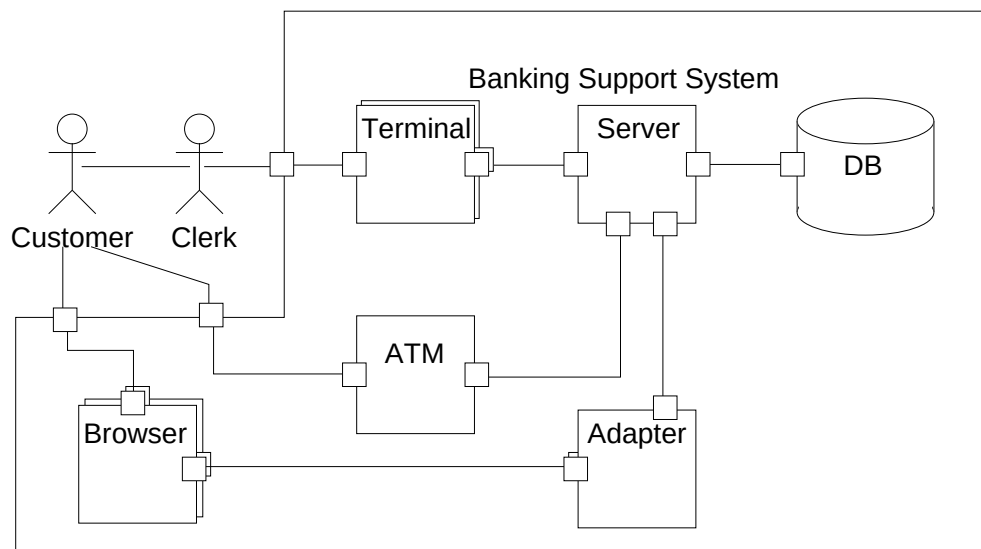
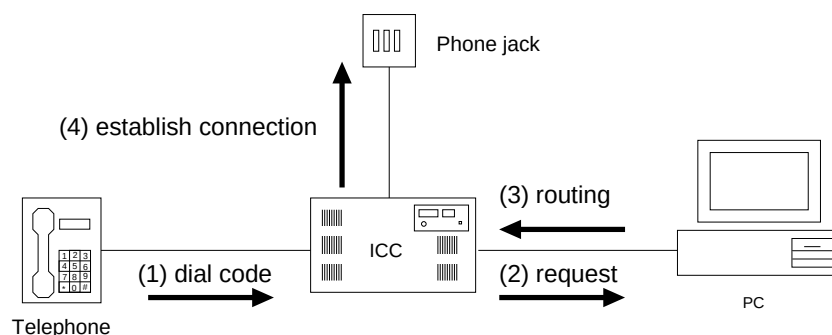


Figure 5.13.: An enhanced architecture for internet banking.

with external LCR-boxes are presented. There is a choice as to how the LCR-box is to be programmed: (a) by the user directly, using the phone attached; (b) by a PC that is also attached and that can be used to download the relevant data from, say, the internet, and then upload it into the LCR box; or (c) by a remote maintenance server uploading current data into the LCR-box directly.

- As a second family of alternatives, an internal telecommunication system is assumed to be present. This is usually the case for companies, but also for a number of private households. Note that in Germany, there is a large number of households equipped with ISDN-adapters, that are actually PBXs. Here, there is a choice between (a) internal and (b) external LCR, i.e. the LCR can be part of the ISDN-adapter/exchange, or resides in a separate LCR box attached to the switching device. This requires calls to be originated from the PC. The original case study presents the following diagram to illustrate a scenario of the architectural alternative 2 (b). The diagram is abridged from [Mattern et al., 1998, Fig. 3].



In this case study, a particular make of telecommunication system with the product name *ICC* was used, as it was available for free. It may be considered as representing a legacy.

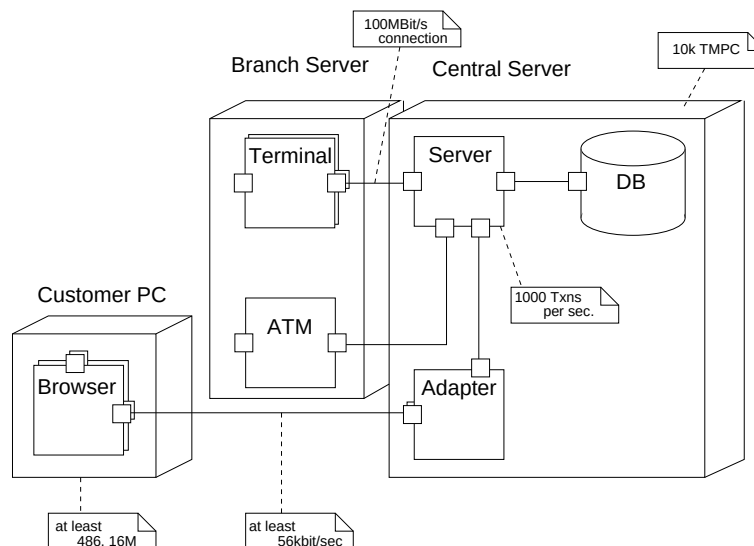
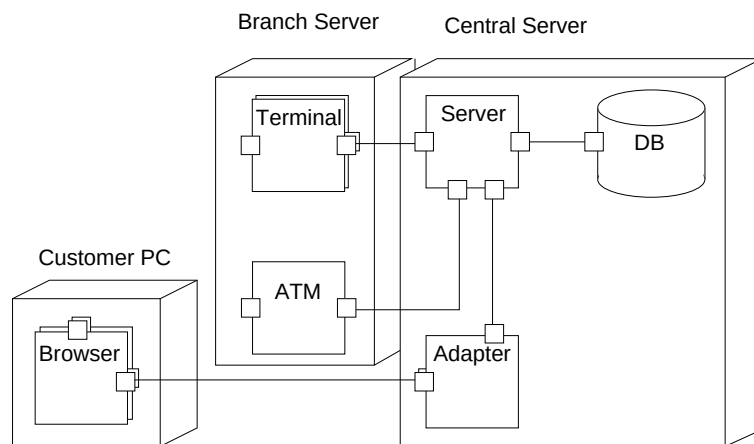
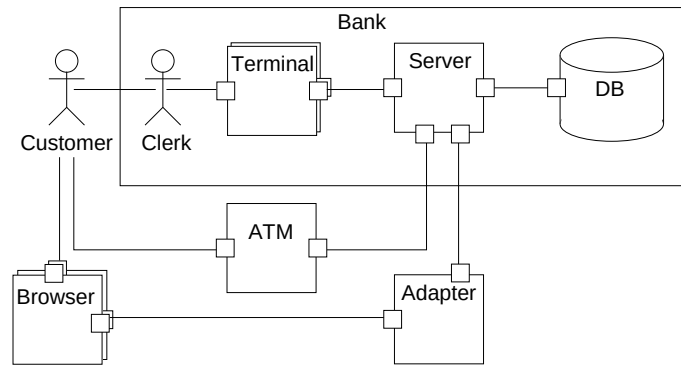
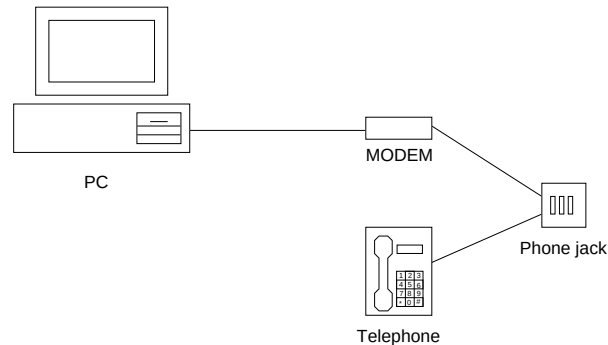


Figure 5.14.: Adapting the architecture for internet banking (top); a deployment view (middle); and a deployment view enhanced with load and performance figures (bottom).

3. The last alternative is a software solution, requiring neither exchange nor separate LCR-box. The architecture is illustrated by the following diagram (abridged from [Mattern et al., 1998, Fig. 1]).



Obviously, options 1 (a), and 3 are very inconvenient to use. Option 1 (b) still requires a good deal of effort from the user. Option 1 (c) is certainly the most convenient, but requires hardware of considerable sophistication (and memory) for the LCR box, which currently is not worthwhile. Option 2 (a) is convenient at no extra hardware cost (ISDN-adapters and PBXs are usually quite sophisticated), but appears to be rather cumbersome to implement technically. So, for the case study, option 2 (b) was chosen.

In the case study, there are just two sketches concerning the architecture (the ones we presented with the alternatives). They are used informally, and lack both expressiveness and precision. With the concepts we have introduced above, it is possible to express the architectural alternatives in a clear and UML-compliant way. First consider the basic architecture: two prototypical phones are connected via the public telecommunication network (Figure 5.15, top). This initial design can be refined to allow for the functionality of making a call. Since it is an important point that there are several providers, and since the functionality they offer differs substantially (e.g. in terms of price, billing procedure, availability etc.), these should become different use cases, possibly using the same sub use cases for the phone connection as such. Figure 5.15, bottom shows the availability of different kinds of calls.

Both of these diagrams can also be refined by adding behavior to the interfaces, i.e. the protocols that are to take place via the Connectors. In Figure 5.16, a simplified version of the behavior of phones is displayed. Note that the phone protocol is symmetric in nature, so that this single diagram describes all protocol roles that occur in the architecture, as far as we have designed it up to now. With the behavior of its interface captured, it is possible to abstract from the internal make up of the phone itself.

We are now capable of precisely describing a phone call taking place in this system, (see Figure 5.15) bottom. Note that this is a UML diagram, according to the semantic extensions we have defined above.

Modeling alternative architectures

The design alternatives can be written down unanimously using the concepts we have defined. Consider first Figure 5.17, describing design alternative 1 (a), i.e. the phone-programmable external LCR-box. In order to adequately describe this architecture, we must define the interfaces, functionality and behavior of the LCR-box itself. Note that the behavior of the interface of the LCR-box towards the local phone (Figure 5.17, middle) is a specialization of the behavior of an ordinary phone (Figure 5.16).

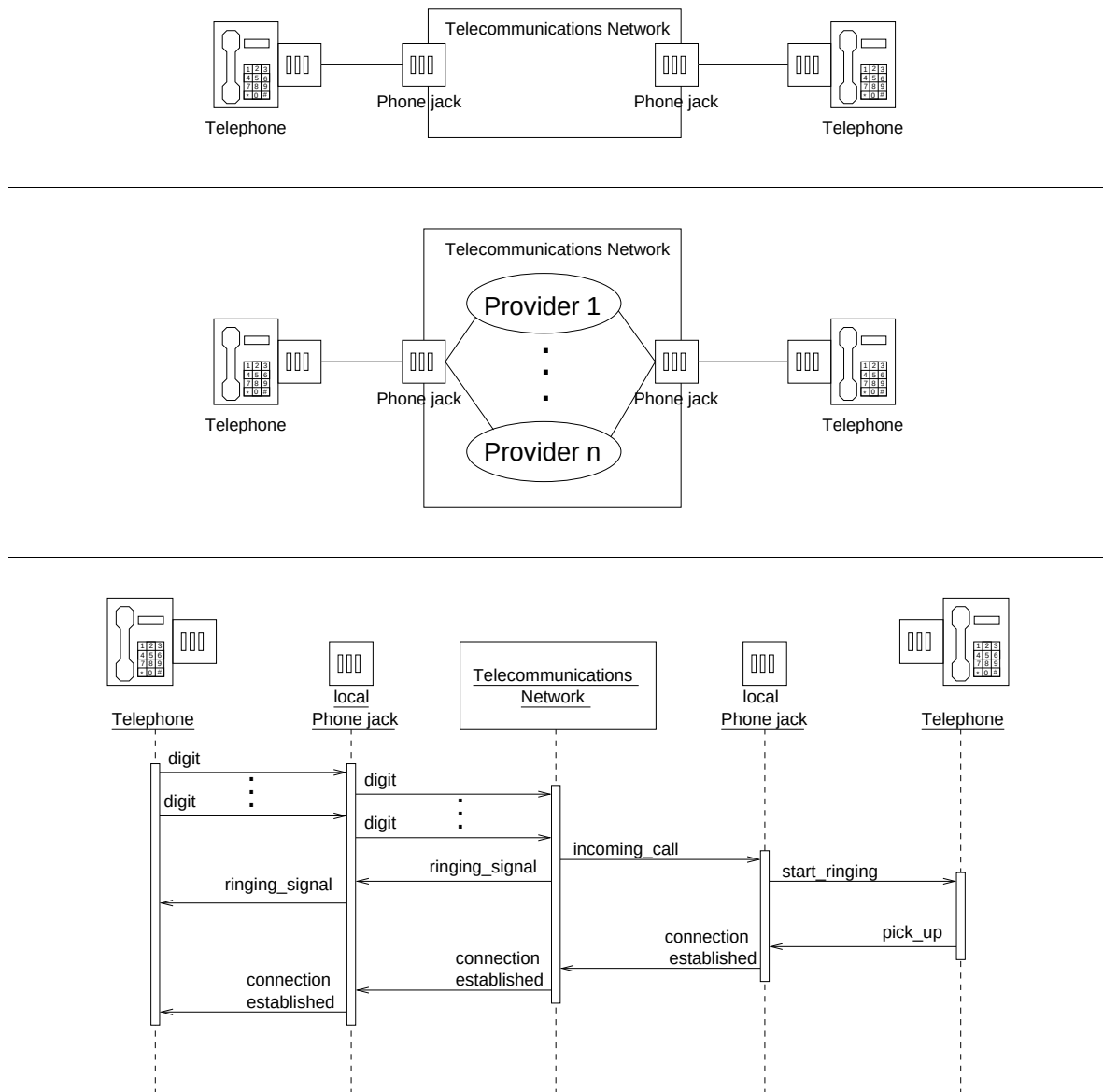


Figure 5.15.: Plain phone connection: context view (top), and functionality view (middle), and a scenario for initiating a call (bottom). The services of different providers amount to different use cases.

Figures 5.17, 5.18 and 5.19 show the alternative architecture, with their respective sub-options.

All of the above Configurations are formalizations of the design alternatives discussed informally in the case study. We have used the plain UML notations and the extensions introduced above. Of the five alternative architectures discussed in said case study, the alternative with an exchange and a PC-based LCR has been chosen (see Figure 5.18, middle). So far, we have only provided Configurations, but have left open many important details, in particular some of the interfaces, and the realization of the LCR-functionality. While the interface of the LCR-application to the exchange is quite simple conceptually (see Figure 5.20, bottom left), it involves some effort on the implementation level. The architecture actually implemented by the case-study used the Telephony Services API (TSAPI),

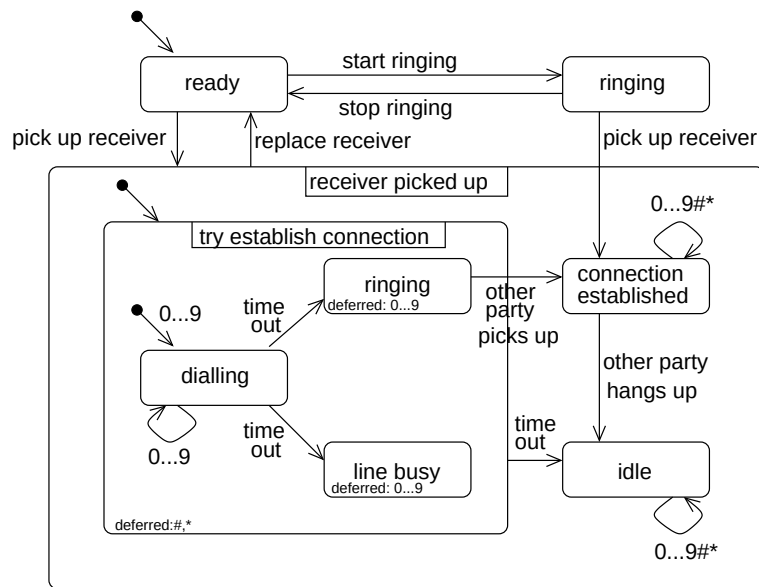


Figure 5.16.: Protocol role of an ordinary phone over its jack (simplified).

Java Telephony API (JTAPI), Common Object Request Broker Architecture (CORBA), and the C++ and Java-mappings for CORBA, and gives the following diagram taken and slightly abridged from [Mattern et al., 1998, Sec. 3.4.3, p. 11]. It illustrates the way the transition from the TSAP-interface of the ICC telecommunications switch to the JTAP-interface that the designers chose to use for their LCR-application (see Figure 5.20, bottom left).

Obviously, this architecture is too complex to be defined exhaustively in an introductory example. But this is not necessary, we can simply abstract away the technicalities of CORBA, say, and use it “as is”. The illustration of the case study can be mapped to a proper Configuration, see Figure 5.20. There, the interfaces, functionality and structure view of the LCR-application are shown. Observe how the informal sketch used in the case study becomes precise by the conceptual mould provided. The protocol is rather simple, as long as one considers only the design level. A further level of refinement shows the functionality of the CORBA services used, and the implementation of the LC-router proper.

5.5. Styles

In this section, we introduce some examples for architectural styles. Buschmann et al. [1998] describe in their Chapter 2 a number of architectural patterns, that closely correspond to the styles we present in Sections 5.5.1 and 5.5.2, and Figure 5.24. See also Shaw [1995b,c]. We have outlined above that architectural styles are really special roles of Capsules. When focusing on some particular aspects of a style, we will not show the enclosing Capsule, but just the contents of the particular view.

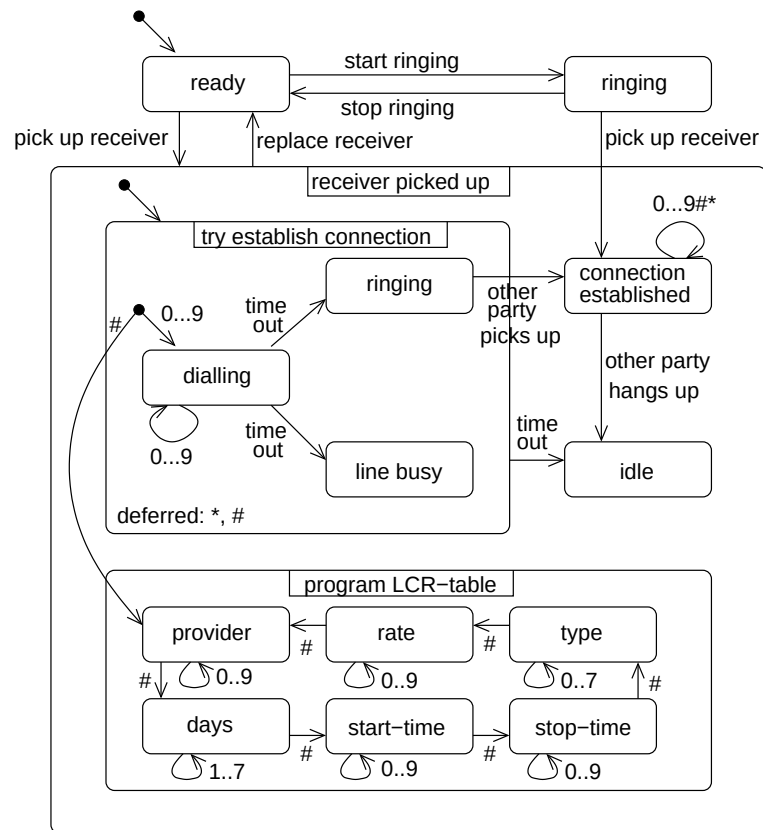
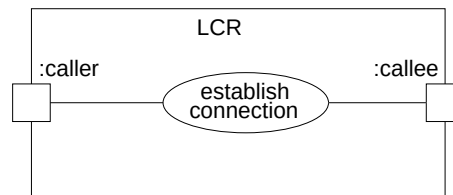
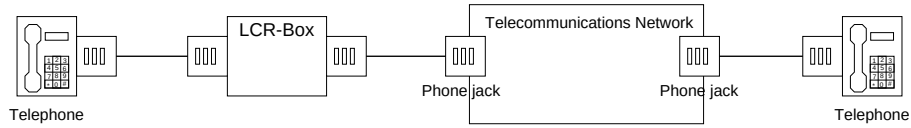


Figure 5.17.: Alternative 1 (a): a separate phone programmable LCR-box (top) with a functional description of the LCR-box (middle) and the behavior of the ProtocolRole of the LCR-box towards the phone (bottom).

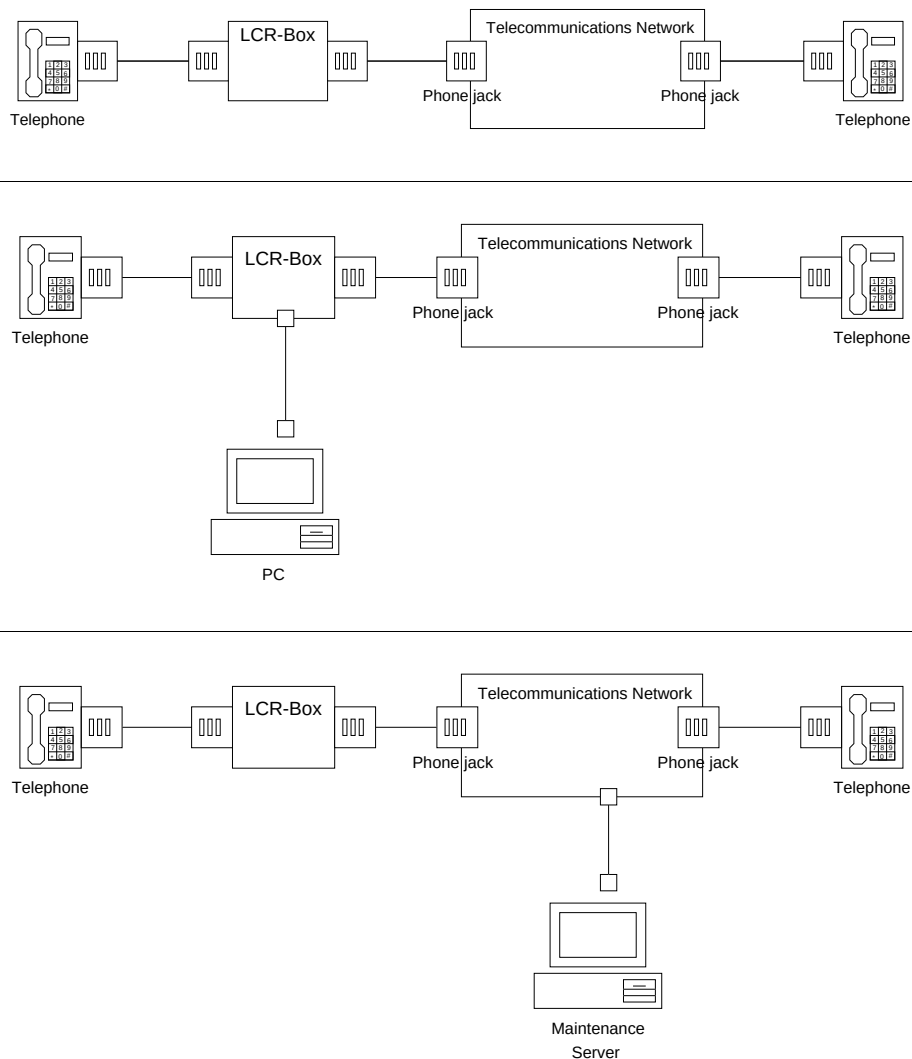


Figure 5.18.: Three alternative architectures for a LCR-solution with separate LCR-box (the first option discussed). These alternatives differ in the way the LCR-box is programmed: either by the phone (top); by an external PC (middle); or by a remote maintenance server (bottom).

5.5.1. Pipeline style

General description

The Pipeline style is also known as the Pipe/Filter style (see [Buschmann et al., 1998, p. 54-71]). It consists in a linear configuration of processing units. It is a particularly simple and reusable structure. This style is applicable wherever there are several stages of processing that require no feedback or control, other than synchronization on the data to be processed.

Configurations

A pipeline consists of a static Configuration, containing an ordered set $\{c_1, \dots, c_n\}$ of Capsules, each of which has at least two ports that are called *in* and *out*. Every pair $\langle c_i, c_{i+1} \rangle$, $0 < i < n$, of Capsules is

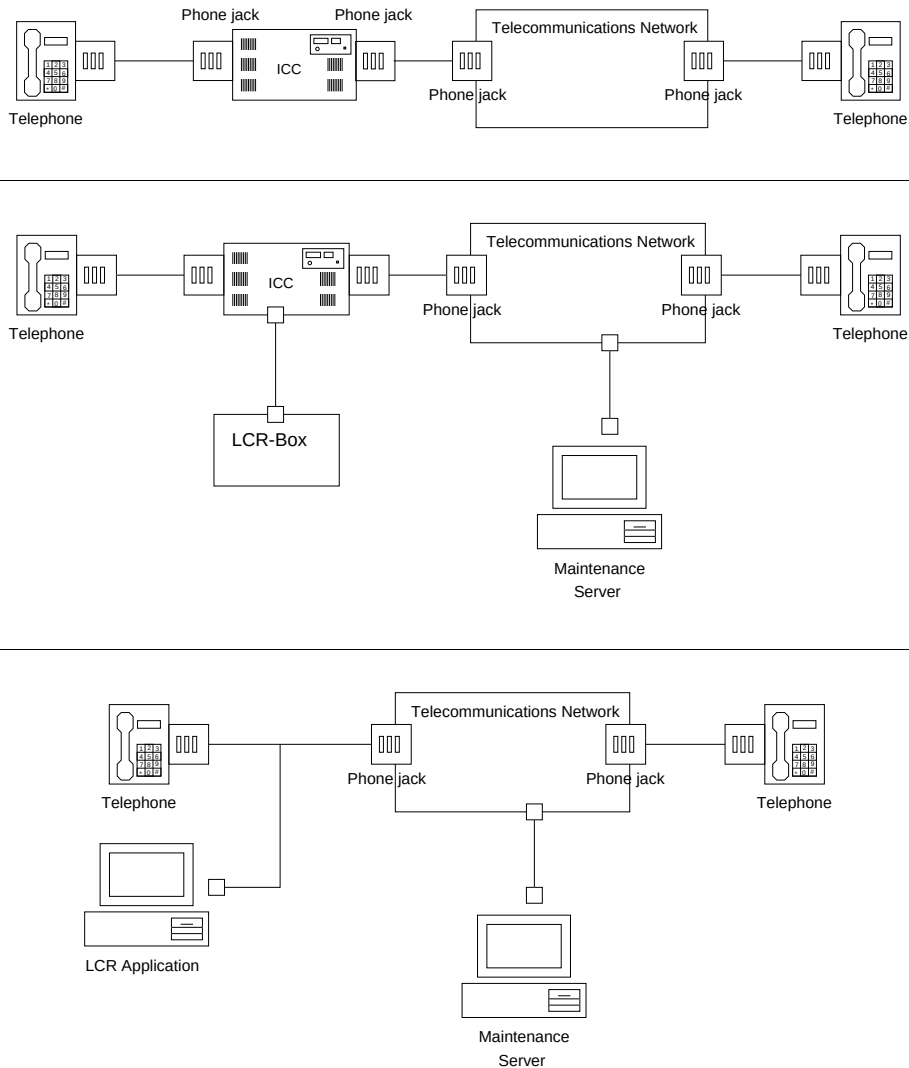


Figure 5.19.: The second option discussed above: two alternatives for LCR-solutions with separate exchange (top and middle), differing in the deployment of the LCR-functionality as such. The third option is a pure software solution (bottom), where the calls have to be initiated by an external PC, that is also responsible for the routing.

connected by exactly one binary unidirectional symmetric and synchronous Connector, where $c_i.out$ is the source, and $c_{i+1}.in$ is the sink of this Connector. There are no other Connectors between the Capsules of the pipeline, but there may be further Connectors to other Capsules outside the pipeline. All Connectors satisfy a file format Protocol. See Figure 5.21 for a sketch of the Configuration.

Interactions

Since all Connectors inside the pipeline are unidirectional and synchronous, Interactions of the Configuration contain neither loops, nor return-signals. Depending on whether the Capsules are buffering a certain amount of the processed data or not, the Interactions exhibit true concurrency.

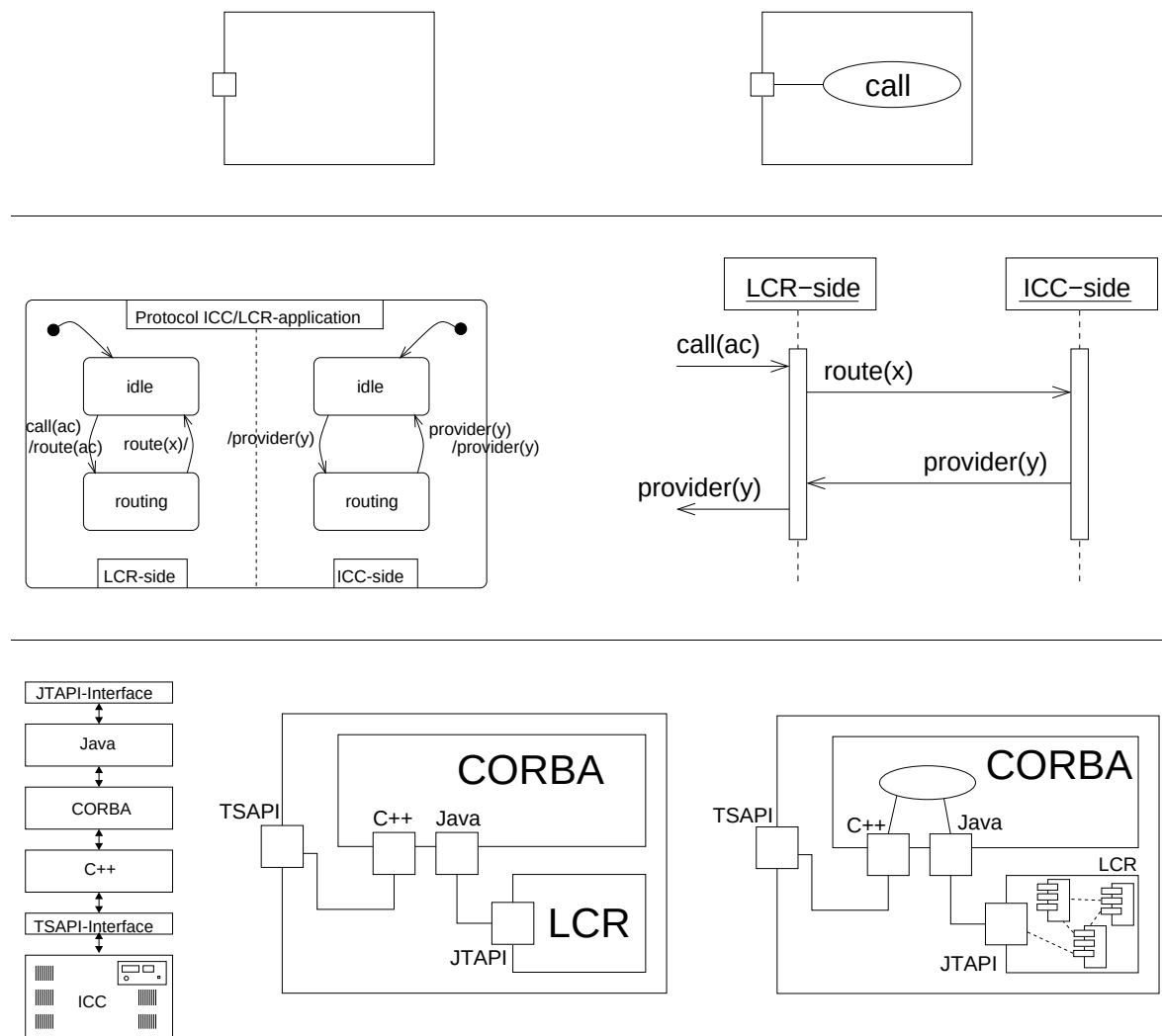


Figure 5.20.: The LCR-application in greater detail: interfaces, functionality and architecture view (top row, left to right); the protocol between the LCR-application and the ICC, and a scenario (middle row); and a recursive view of the LCR-application, showing the functionality of CORBA and the implementation of the router proper (bottom row middle and right). For comparison, we include the diagram provided with Mattern et al. [1998] (bottom row left)

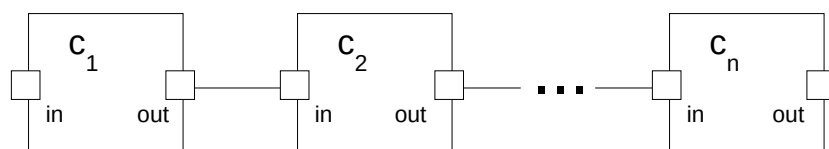


Figure 5.21.: A generic example of the configuration-part of the pipeline style.

5.5.2. Blackboard style

General description

The Blackboard style (see [Buschmann et al., 1998, p. 72-98]) consists of a dynamic configuration of a single passive storage unit and a varying number of active information sources or sinks. The purpose of a blackboard-architecture is to connect different servers and clients in a simple, dynamically reconfigurable way. Its prime advantages are flexibility. The blackboard-unit may become a bottleneck.

Configurations

A blackboard-architecture consists of Configuration of two Capsules called *Blackboard* and *Administrator*, and a dynamically changing, possibly unrestricted set of further Capsules called *Users*. The *Blackboard* has a Port called *control*, and a dynamically changing, possibly unrestricted set of Ports called *user-ports*. The *Administrator* has a Port called *board-control* which is connected with a binary synchronous Connector to the *control* of the *Blackboard*. It may have other Ports as well. Every *User* has a Port called *board*, that is connected to a *user-port* of the *Blackboard* by a binary synchronous Connector. See Figure 5.22 for the initial and a later Configuration of the structure. Note that this is an example of dynamic structure.

New *Users* are introduced by the *Administrator* by a register-operation. Similarly, the *Administrator* may remove *Users* by a unregister-operation. Figure 5.22 shows scenarios of the dynamic architecture as a ConfigurationSpace. The number of *Users* and *user-ports* is identical, unless there are uncompleted register- or unregister operations. One might also define specialized static blackboard-style for a fixed and static number of *Users*. This style can dispense with the *Administrator*-Capsule, but looks the same otherwise.

Interactions

There are no general constraints on the interactions of blackboard-architectures: any part may start or terminate interactions. The most typical scenarios is shown in Figure 5.23.

5.5.3. Layers

General description

One classic example of architectural style is the *layers architecture* (see e.g. [Buschmann et al., 1998, p. 32-53]). A layer is a subsystem encapsulating a certain level of abstraction, such as a layer in a communication protocol or operating system. Typically, there are stacks of layers, so that every layer has a top side and a bottom side, and layers are connected only to the two layers directly above and below it (save for the bottom-most and top-most layers of course, who have only one neighbor). Each layer provides some services, that can be requested from the layer above, and that request services form the layer below.

Among layers, the services of the underlying layer are assumed to be ubiquitous. Within a layer there is little coupling, and its architectural units are of equal rank, in that they access the services provided by the lower layer and/or provide services to the higher layer in similar ways. In other words, a layer is a collection of services of similar granularity.

Frequently, a layered architecture is associated with synchronous communication, i.e. the caller of a service waits till the callee returns the thread of control. Also, there are often many points

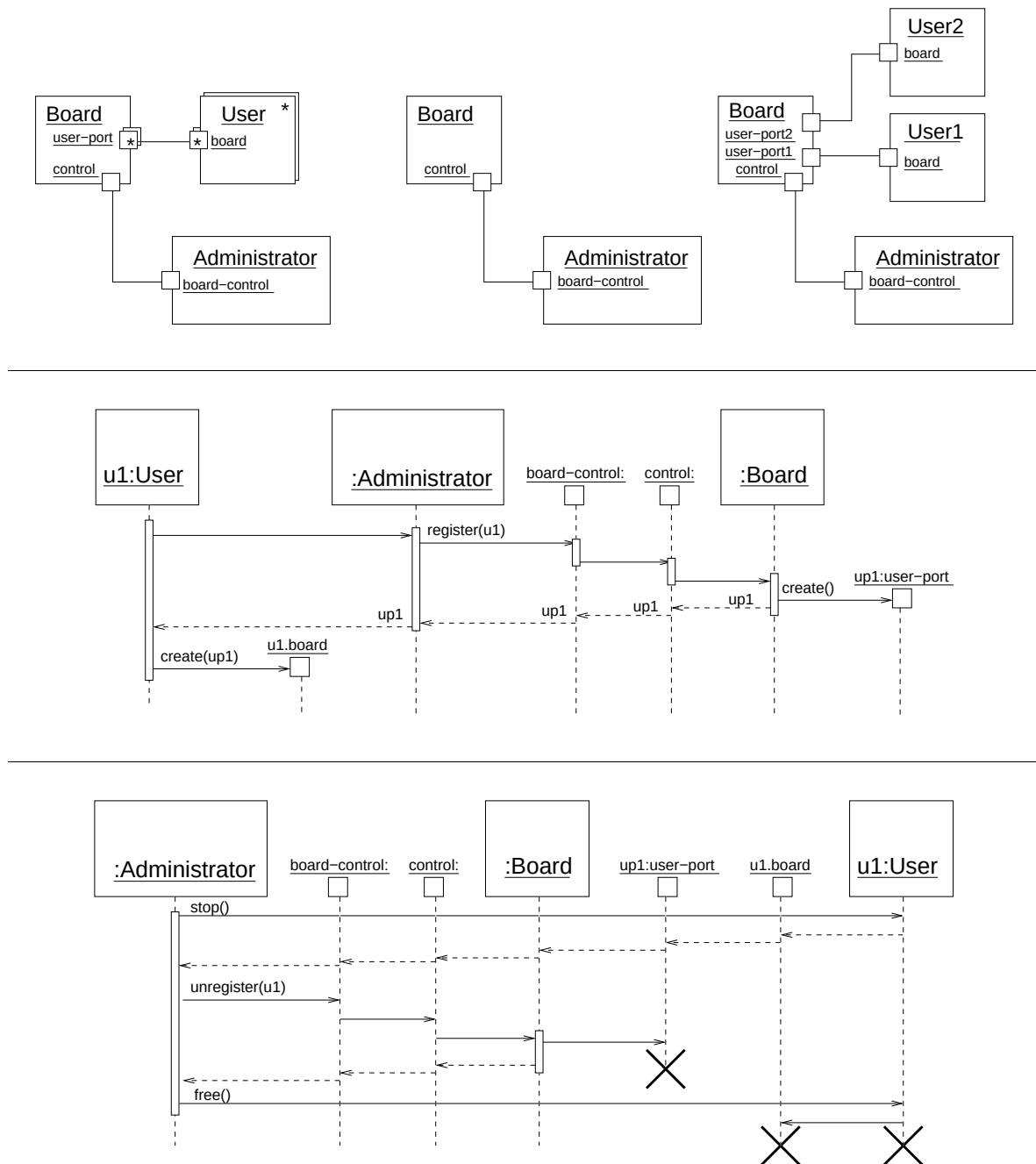


Figure 5.22.: The blackboard-style features a dynamic architecture: the generic description of the configuration space, the initial configuration, and a later configuration with two users of the blackboard (top row, left to right); two scenarios for registering and unregistering a user (middle and bottom rows).

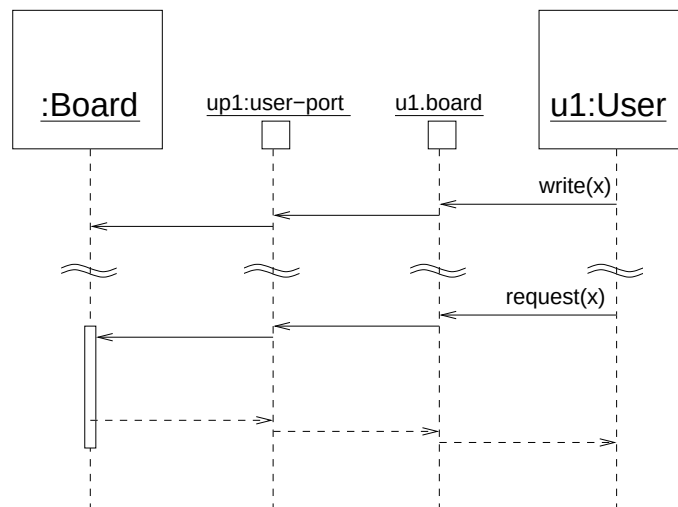


Figure 5.23.: Two scenarios of using a blackboard-architecture: writing a value x to the blackboard (top); requesting a value (bottom).

of interaction between layers, and so explicitly drawing all communication channels can become messy. Therefore, the abbreviation of Figure 5.24 is introduced: the capsules constituting the layers are stacked atop each other so that their outlines touch (see Figure 5.24). The shared boundary is replaced by the dashed line that was used for synchronous connectors before. In Figure 5.24 (right), we show a recursive view, where there are additional characteristic elements added to the different layers.

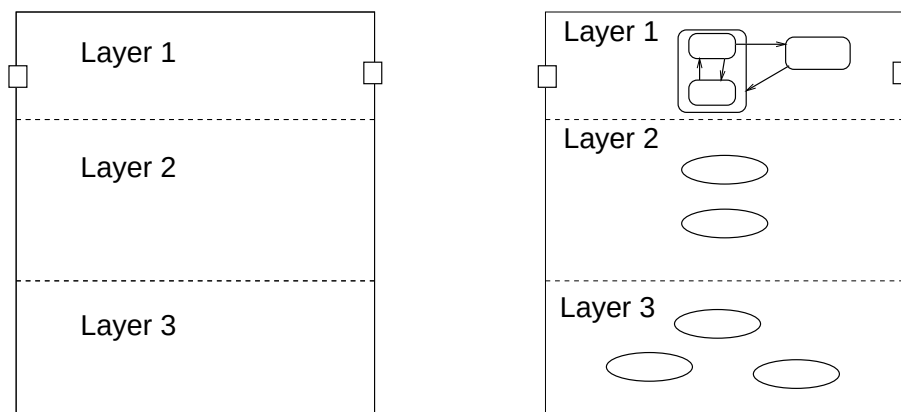


Figure 5.24.: Layered systems: a system refined by three layers (left); one further level of refinement of that system (right).

Configurations

The layers style requires a linear sequence of Capsules like the Pipeline style, but the communication typically is bidirectional, not unidirectional. The layers-Configuration is static, so there is no ConfigurationSpace attached.

Interactions

In Figure 5.25, we show a schematic scenario of how a service may be provided in a layered system. Typically, there are only asynchronous communications, possibly with nested flow of control.

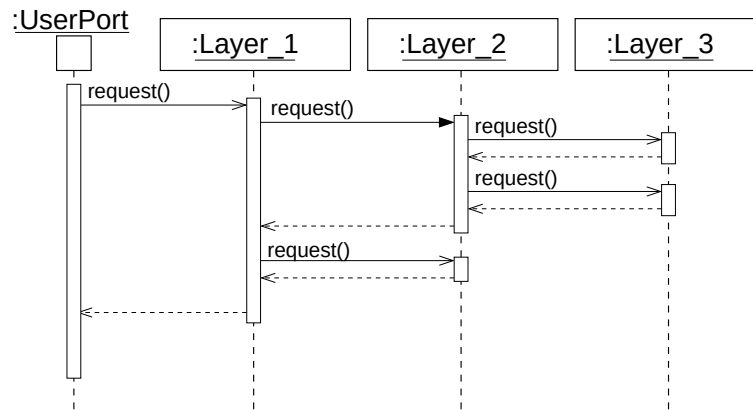


Figure 5.25.: A typical schematic scenario of providing a service: the request is initiated via the Port *UserPort*, and processed by requesting several services of the lower layer, which in turn provides these services by requesting other services from the layer beneath it.

5.5.4. Compute/Control style

General description

The Compute/Control style is also known as Master/Slave. It consists of a configuration of a processing unit and a controller unit. It is useful to separate computation and control, such as to improve reusability of the computational part, or to localize the control aspect for easier maintenance.

Configurations

A Compute/Control-architecture consists of a static Configuration of two Capsules called *Control* and *Compute*, respectively. The *Control*-unit has exactly two Ports, called *Ctrl.in* and *Ctrl.out*. This is the source of a single binary unidirectional symmetric Connector with a simple non-file format protocol. The sink of this Connector is a Port of the *Compute*-unit. The *Compute*-unit may have arbitrary further Ports and attached Connectors. See Figure 5.26 for a sketch of the Configuration.

Interactions

All Interactions of the Configuration start with a signal reception at the *Ctrl.in*-Port of the *Control*-unit. No signal is received or sent by any of the Ports of the *Compute*-unit unless there has been an according enabling signal from the *Control*-unit to the *Compute*-unit before.

5.5.5. Using a style

A Style is an abstract description of (some particular aspect of) a concrete architecture, a kind of comment. So, there is an Abstraction-relationship between a concrete architecture and a Style. When using

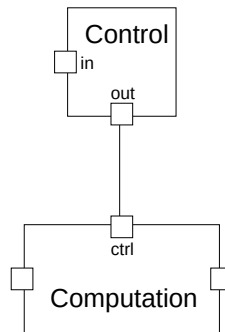


Figure 5.26.: A generic example of the configuration-part of the master/slave style.

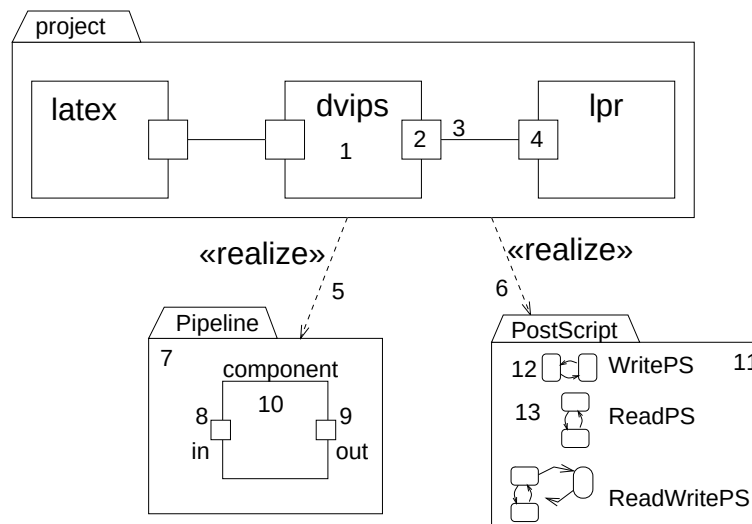
a Style, this relationship has to be established. Consider the example in Figure 5.27. It describes an architecture to implement the UNIX-command `latex file | dvips | psnup -2 | lpr`. Obviously, this is a typical application of the pipeline-style described above. Also, the Protocols `dvi` and `PostScript` are used. Using the pipeline-Style and the Protocols amounts to importing it, and providing a mapping from `ModelElements` of the architecture to `ModelElements` of the Style. In Figure 5.27, we have provided a part of this mapping, namely the Abstractions a and b.

Each of these two mappings possesses a mapping, which is of the elementary UML data type `MappingExpression`. Since its semantics is entirely undefined, we take it to mean a mapping in the mathematical sense, relating identifiers of `ModelElements` contained in the client and supplier of the Abstraction, see the tables in Figure 5.27. Note that not all elements of a Style or Protocol must be used, so that the Abstraction-mapping may well be partial. In the example, the ProtocolRole `ReadWritePS` of the `PostScript`-Style is not used. Of course, `ModelElements` of the concrete architecture may be mapped into several different Styles or Protocols (cf. the `dvips-out` Port).

5.6. Chapter summary

In this chapter, we have defined a concrete syntax for architectural modeling, and given some first examples of how it is to be used—a more in depth example is presented in Chapter 12. The notation we have presented here maps directly to the abstract syntax we have defined on the previous chapter. Thanks to the way we have embedded our conceptual model into the UML metamodel, all UML-related diagrams, techniques and so on can be used as before, and we don't even have to define a proper mapping from our concrete to our abstract syntax—it is already implied in the UML's own mapping from concrete to abstract syntax.

In Part II, we will define the semantics of (some of) the concepts of UML and our extension to it. In Part III, we will deal with pragmatic aspects.



MappingExpressions

	client		supplier
5:	1 (dvips)	$\mapsto$	10 (component)
	2 (dvips-out)	$\mapsto$	9 (out)
6:	2 (dvips-out)	$\mapsto$	12 (WritePS)
	4 (lpr-in)	$\mapsto$	13 (ReadPS)

Figure 5.27.: Using the pipeline-Style and the PostScript-Protocol. On the left, there is a Configuration (top), annotated by a Style (bottom); the two are connected by two Abstractions, a and b. On the right, there are the mappings of the Abstractions. The numbers are the identifiers of the respective ModelElements. For simplicity, we have also added the names of the respective ModelElements in the MappingExpressions.

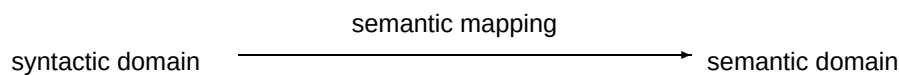
Part II.

Semantics

6. Approach

6.1. Overview

Our approach towards the formal semantics of (fragments of) the UML is rather straightforward, and very similar to what one would expect traditionally: from a formally defined syntactic domain we provide structural mappings into appropriate semantical domains (see sketch).



The UML is structured into a logical and a physical metamodel, similar to analysis and design models in ordinary software development. The former is usually referred to as *the* metamodel. It is represented as a class diagram. The physical metamodel comes in two representations, as a class diagram and a textual representation (a XML DTD). These two representations are supposed to be equivalent, as the textual representation is apparently generated automatically from the diagrammatic representation (see [OMG UML 1.3, p. 5.3–5.6]). The concrete graphical syntax of the UML, i.e. the diagram types and the OCL-grammar, are yet another representation of the physical metamodel, though a rather informal one. See the following sketch for a synopsis.

	physical metamodel	logical metamodel
represented as	class diagram & OCL XML DTD	class diagram & OCL

There is a large number of constraints defined on the logical metamodel. They are expressed in the OCL. Save technical and syntactical adaptations, these constraints apply just the same for the physical metamodel.

In our approach we provide conservative¹ extensions of the logical metamodel and the concrete syntax (see Part I). We provide a physical metamodel, that closely matches our logical metamodel with two exceptions: we add a number of concepts necessary for the implementation (see Part III), and we alter the representation of StateMachine (see the next chapter). We also provide a textual representation of the physical metamodel as a context-free grammar, as this is theoretically and technically easier to handle than the original XML DTD. In the context of the first sketch above, we take the textual representation of our version of the physical metamodel as the syntactic domain. See also Figure 6.1.

The conceptual structure of the UML is explained in Appendix A. It is mirrored by the approach presented here, as the Figure shows. Since the nodes in this graph are informal, there are also no formal relationships between them. Edges 1 and 3 are (almost) plain extensions, i.e. additions of new metaclasses and meta-associations (the exceptions are error patches rather than essential features for

¹By conservative we mean that our extensions just add to the standardized metamodel, but do not remove or change any concepts or properties. The one exception —Associations between Actors— is immaterial to our approach, and a good idea in its own right.

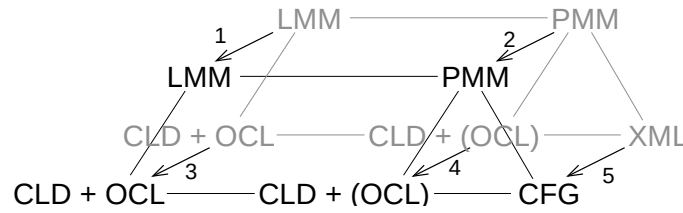


Figure 6.1.: Relationship between the elements of the UML metamodel (in the back, grey) and our approach (in the front, black).

architectural modeling). Thus, we say that our approach is a conservative extension. Edge 2 is a necessarily informal relationship, as it is determined by technical side conditions and the rationale. These, which, however, are different for the UML proper and our approach. Consequently, the edges 4 and 5 also are informal only.

6.2. Purposes

The driving force behind our approach is the vision of automated CASE support for the work of the software architect. This is the reason why we need to have a formal semantics. Exactly which semantic domains and mappings are required depends on the activities and tasks that are to be supported. Therefore, we now refine the usage scenarios of Section 3.4 into concrete activities, according to the degree of automated support that can be given.

- First, there are activities that require little or no formality to be supported. This applies in particular for some non-engineering activities required for *work organization*² and *project management* (e.g reporting, work scheduling), as these are informal by nature, and the *classification and retrieval* of components from component databases. Here, well structured documentation and meta-data are pivotal, and sufficient.
- Second, there is a range of *syntax checking*-like activities, such as name consistencies etc. They, too, require at most meta-data (like format conventions). We will assume all of these activities to be completed successfully before any more sophisticated activities are launched.
- Third, there are activities requiring simple semantics like dependency graphs to generate e.g. *make-files*. These can then be used for the automated *assembly* of system level code, test cases, and documentation from unit-level fragments of code, test cases and documentation, respectively. Similarly, dependency structures can be used to support *versioning* and *regression testing*.
- Fourth, *validation* is important to ensure that Validation addresses users, that is, people usually not familiar with software development, so that visualization and understandability are prime issues in validation. Here, interactive and possibly animated simulation has proved an effective tool. Simulation obviously requires an operational understanding of behavioral models, but not necessarily complete semantics. Other validation techniques include inspection and walk-through (see e.g. Wheeler et al. [1996]). They do not require operational semantics, but a consensus on the precise meaning of the models is essential.

²It is possible to formalize the development process as such, i.e. to support the design flow by tools.

- Fifth, there are procedures for *consistency checking*: every view can be considered as a partial specification of a system, implying a number of consistency conditions. Verifying these can be used to track down logical mistakes in behavior specifications, conditions, and branches, or overlooked concurrency phenomena and so on. All the constraints given in the UML standard document (and those given in Chapter 4) belong to this class.
- Finally, there are procedures for *quantitative analysis*, that aim at analyzing certain performance aspects of a system such as memory usage, throughput, load, residual and response times etc.

Figure 6.2 symbolically shows different views, the diagram types used in them, and some of the activities that use the respective views.

activity	involved views
project management	administration, functionality, environment
code generation	deployment, implementation, environment, execution
regression testing	implementation, behavior, functionality
validation	functionality, goals, domain structure, domain logic
(e.g. simulation, inspection)	functionality, behavior, functionality
consistency checking	functionality, behavior, deployment, implementation
quantitative analysis	deployment, implementation, environment

Figure 6.2.: Synopsis of the views (top); association of views to activities (bottom). The interfaces and structure views are always used, so we omitted them in the table.

Obviously, all of the activities from this list deserve tool support. However, we follow Brooks in that “[...] *conceptual integrity is the most important consideration in system design.*” (cf. [Brooks, 1975, p. 42]), since early errors are dearest. So, we deal only with the last four items, focusing on consistency checking. Quantitative analysis procedures, simulation and code generation are covered only in passing.

While the first two activities from the list above require no formality, or only a formal syntax, the third through fifth require also a formal semantics.

6.3. Consistency checking

Accounting for its paramount importance within our approach, we shall give an example of consistency checking by integrating complementary views. Suppose that the two sequence diagrams in Figure 6.3, (a) and (b) are both views of the same system, representing admissible sample runs. Obviously, these views are overlapping, since they both refer to some entity B. The two state machines in Figure 6.3, (c) and (d) each satisfy exactly one of the two views. The two state machines in Figure 6.3, (e) and (f) satisfy both. More formally, the following sets of views are consistent (apart from the singletons): $\{a,c\}$, $\{b,d\}$, $\{a,b,e\}$, $\{a,b,f\}$.

Possible inconsistencies like this may arise wherever there are behaviors and samples, such as within Protocols (where the behavior is the concurrent composition of all roles), between functionality and interfaces views, or between interfaces and behavior views of a single Capsule. Now imagine the maintenance of a large software system, where there are many Protocols and Capsules. Changing one of the sample runs can result in an inconsistency somewhere else. The implications of this inconsistency may be discoverable easily enough by inspection at design time already, but the size of the

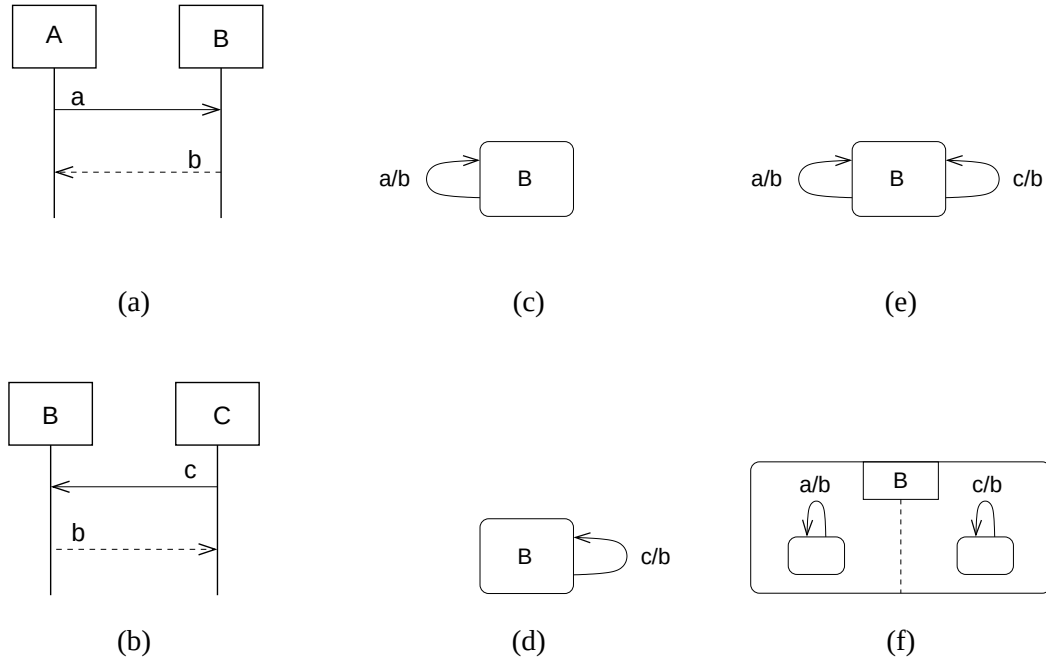


Figure 6.3.: Inconsistencies in ensembles of overlapping views: two views showing possible interactions of a system B with other systems A and C, respectively (see (a),(b)); two state machines for B that satisfy exactly one of these views ((c),(d)); two alternative state machines for B that satisfy both views ((e),(f)).

system (i.e. the amount of diagrams involved) will make this a tedious and time consuming task. In a typical development project, no such check will be done systematically. So, the inconsistency might be discovered later, during programming, during compilation, or still later during regression testing. Then, however, fixing an error is dearer than fixing it at design time. With behavioral abstraction and appropriate tool support built on formal semantics, this type of error can be found at the time of design, and in many cases, it can be found completely automatically. This scenario applies just the same for the initial construction or the reuse of subsystems.

One might object, that the inconsistencies sketched above are due to the usage of different overlapping views only. However, humans have limited information processing capabilities only (cf. the “bounded rationality” research by Simon and Newell), so that it is rather natural to present a system as a set of related projections (i.e. views). In referring to the same system, different views inherently have a certain overlap. Besides, as our approach shows, by consistency checking, the overlap can be used to provide valuable automated support for the software development.

6.4. Analysis targets

In the previous section, we have interpreted state transition diagrams and interaction diagrams with their intuitive meaning. More formally, they are concrete syntax representations of StateMachines and Interactions. For simplicity, we will carry on to equate these with their denotations, that is, with a form of automata and sample runs. In Chapter 9, we will define the semantics in a way justifying this

terminology.

The interfaces view can be used to derive an automaton representing the complete behavioral environment of a Capsule. Analogously, the free Ports of a Configuration can be used to derive an automaton representing the external behavior of the Configuration. This is admissible only if the ProtocolRoles of the respective Ports have pairwise disjoint sets of incoming and outgoing Signals, for otherwise there could be communications from one region (i.e. ProtocolRole) to another without this being specified by a Connector.

All of the analysis targets we mention in the following will be spelt out in greater detail in Chapter 10.

6.4.1. Consistency of individual views

In this section, we consider properties that can be proved or disproved by considering a group of ModelElements correlated by a single view. Recall that while a diagram usually belongs to a single view, most views consist of more than one diagram. So, intra-view consistency may well be inter-diagram consistency.

Interfaces The interfaces view just consists of a set of Ports. Each Port has a ProtocolRole attached to it, that in turn has a StateMachine and two sets of incoming and outgoing Signals. There are no consistency conditions beyond the constraints of ProtocolRole.

Functionality The functionality view consists of context diagrams with Actors, UseCases, and their relationships; and possibly interaction diagrams as scenarios of the UseCases. Recall that Port is a subclass of Actor, so that the Actors may be specified in greater detail than is customary for Actors (i.e. with sets of incoming and outgoing Signals and a StateMachine). If the Actors of a Capsule are not Ports, no semantic analysis can be carried out.

Behavior The behavior view of a Capsule consists of a state transition diagram and a set of interaction diagrams, i.e. a StateMachine and a Collaboration whose context is given by the Ports of the Capsule and the Capsule itself.

Structure The structure view consists of a capsule diagram, i.e. Capsules with attached Ports and Connectors between them. For binary Connectors, all outgoing Signals of the one Port must be incoming by the other Port, and the behaviors of both Ports together must be well-behaved (the meaning of which depends on the application). For n -ary Connector, similar conditions must hold.

There are no semantic consistency conditions for the implementation, deployment, domain viewpoints, since they have no formal meaning in our approach (yet). Neither Protocol nor Style are viewpoints proper in that the respective particular aspects they present are generic for classes of systems rather than specific for an individual system. Still, they are quite similar to viewpoints, so we treat them here.

- A Protocol consists of a set of ProtocolRoles and a Collaboration between them. The Interactions together with the derived automaton are similar to a behavior view, and the respective consistency conditions apply.
- A Style may consist of all conceivable modeling elements. In fact, a Style can be considered a generic or prototypical (partial) specification of a Capsule. Thus, all consistency conditions that apply for any view involving a Capsule are also applicable for Styles.

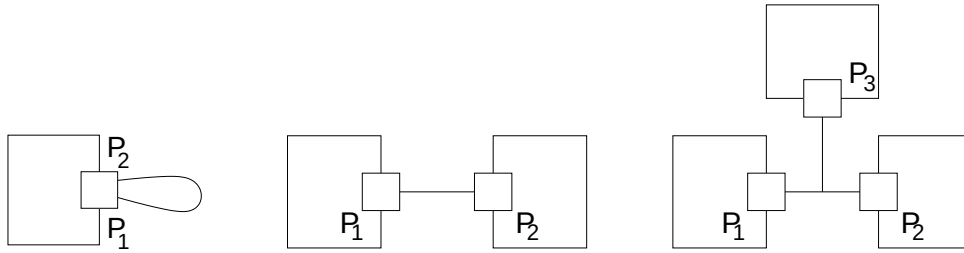


Figure 6.4.: The different cases occurring in the structure view: a short circuited Connector (left), a binary, and a n -ary Connectors (middle, right). Note that the labels p , q , and r refer to the ProtocolRoles of the ConnectorEnds, not the Ports.

6.4.2. Consistency among views

Interfaces vs. structure First, consider a Capsule that is refined by a Configuration. The Capsule's Ports are connected to (some of) the free Ports of the sub-Configuration. See Figure 6.5 for the different cases. (We will refer to these as internal and external Ports of a Capsule, respectively.)³ For binary Connectors, the external Port must be a behavioral subtype of the internal Port (see Liskov and Wing [1994], Palsberg and Schwartzbach [1999]). For n -ary Connectors, a generalization of this relationship must hold.

Interfaces vs. behavior, functionality The interfaces view can be seen as a loose specification of the externally visible behavior of a Capsule, by putting the StateMachines of the ProtocolRoles of all Ports in parallel. Then, the actual behavior specification (i.e. the StateMachine from the behavior view) must be an implementation of this loose specification.

Similarly, instead of the behavior view, the sample runs of the functionality view can be checked against the loose behavior specification derived from the interfaces view.

Static structure vs. behavior, functionality Another loose behavior specification can be derived from the free Ports of the (static) structure view. It too can be checked against the behavior and functionality views.

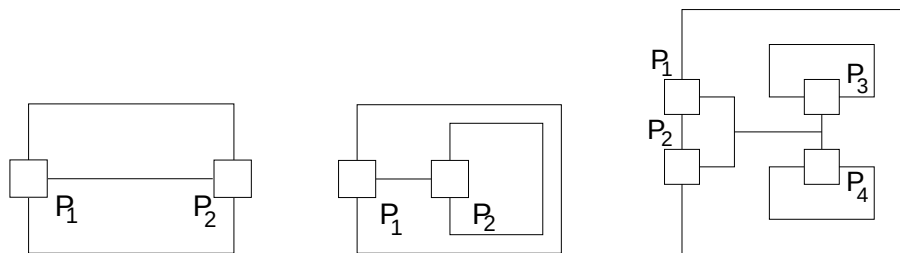


Figure 6.5.: The different cases occurring when analyzing the structure and interfaces views for consistency: a short-circuiting binding (left); nesting a Configuration with a binary Connector (middle); a n -ary Connector (right). The short-circuiting may occur, e.g., in the pipeline style. Note that the labels p , q , r , and s denote ProtocolRoles, not Ports.

³In ROOM, the interfaces Ports are called relay Ports (as opposed to so called end Ports), but these names are ambiguous.

6.4.3. Architectural refinement

One particularly interesting area of application is architectural refinement. We distinguish between (1) elaboration of a Capsule by adding (parts of) Views, (2) replacement of one Capsule with another, and (3) abstraction from a Capsule to a Style. A subdivision can be made between refinements, where only Views of the same kind are involved in a refinement (homogenous refinements), and such where this is not the case (heterogenous refinement). The last two forms of refinement are technically very similar, since they both provide abstractions between Capsules playing different roles. There is a pragmatic difference, however (see Part III).

6.4.4. Quantitative analysis

Qualitative aspects of models can be used to conduct further formal analyses. For instance, from firing frequencies of Ports and the buffering capacity of Connectors can be used to compute the average filling of buffers etc. See Section 9.6 for more details.

6.4.5. Code generation

Generation of code and test cases is obviously not an analysis target, but the procedures are quite similar, so we file it in this section, too.

The automated *generation* of code from predefined components can be supported using the dependency structure embodied in the structure view of a Capsule, i.e. its (recursive) Configuration: provided that every relevant architectural unit has an appropriate implementation view with code, unit-level test cases and documentation, the respective system level code, test cases, and documentation can be assembled automatically. This procedure requires only to use the static structure of an architecture—a tree—to determine the dependencies, and thus the compilation order.⁴ Activities such as *versioning*, *release control* and *regression testing* could be supported similarly.

6.5. Semantic domains

Obviously, there is no single formalism equally well suited for the whole range of purposes that need tool-support. So, a number of different semantic domains will be necessary for the different purposes.⁵

First, we use classical signatures to represent abstract data types, such as for the types of classes (i.e. interfaces in the Java-sense). We shall also use them within the behavioral model to formalize access to built-in pieces of functionality that we do not care to define formally, such as basic data types (like numbers, booleans, sets, lists, etc.), and associated functions.

Second, we introduce a notion of namespace capable of representing values, references, visibilities, and dependencies between namespaces, including sharing. This is necessary for a variety of static analyses and code generation.

Third, we need a behavioral model, certainly the single most important semantic domain. Here, we chose Petri-nets for a number of reasons:

⁴E.g., by first creating a `make`-file, which can then be used to generate an executable system.

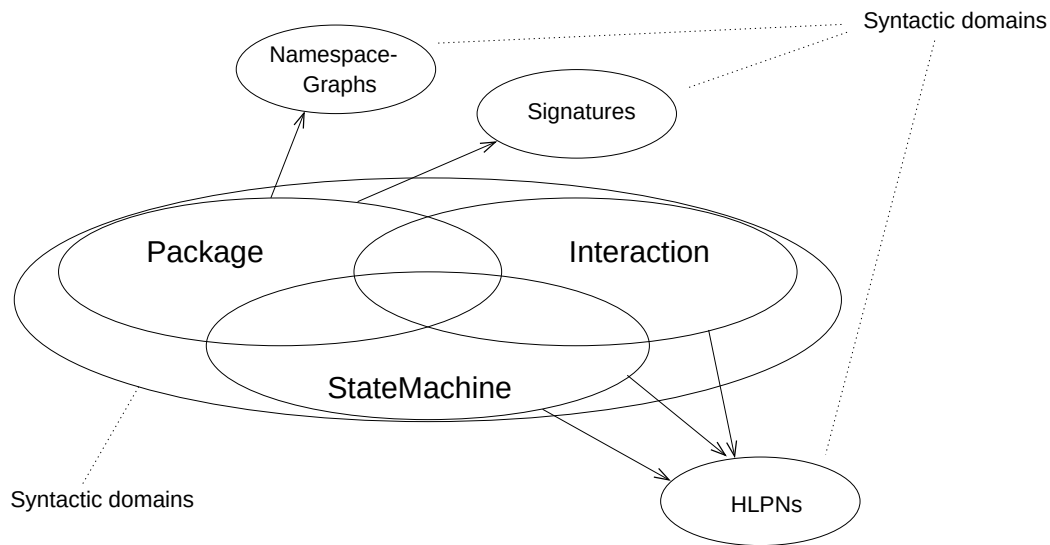
⁵Interestingly, the UML acknowledges this, at least for StateMachines: “[...] there are a number of semantic variation points to allow for different semantic interpretations that might be required in different domains of application” (cf. [OMG UML 1.3, p. 2-143]).

- First, Petri-nets are *expressive*: both states and events are first class modeling elements, making Petri-nets an unbiased semantic basis for integrating both state-centered and event-centered behavioral formalisms, such as state transition and sequence diagrams. Thus they are an ideal basis for unifying different types of notations. Also, architectural models are distributed models, so they exhibit asynchrony and true concurrency phenomena, that are very naturally expressed by Petri-nets. Furthermore, having been around for a very long time, probably all conceivable notational extensions and modifications have been proposed (e.g. place capacities, firing duration, firing probability, location).
- Second, Petri-nets have a sound theoretical foundation. There are many analysis methods, including the standard techniques like model checking, reachability analysis, and stubborn/lazy sets, but also some genuinely specific methods, like place and transition invariants, net graph reductions, and symmetry-analysis of state spaces. Petri-nets have so rich a theory, that *“there is only little need for new Petri-net theory, but a great need for Petri-net archaeology”* (cf. [Kindler, 1998, p. 18]).
- Thirdly, there is a number of pragmatic reasons to choose Petri-nets. For instance, there is a wealth of algorithms and tools, that we can draw upon, unequaled by any other formal modeling language (see Störrle [1998]). Then, Petri-nets have a natural visual representation which makes it easier to justify the semantic mappings intuitively. Some of the UML notations even bear a very close visual similarity to Petri-nets (e.g. activity and sequence diagrams). Finally, Petri nets have an operational semantics, i.e. they are executable, and there is a natural way of animating them, as well.

All of the semantic domains must allow for a tracing-relationship between corresponding semantic and syntactic objects be established by the semantic mapping.

6.6. Semantic mapping

As we have said, our approach is quite straightforward: we define semantic and syntactic domains, and mappings between them. So, the core of our formal semantics is a set of functions transforming syntactic objects into objects of the various semantic domains that we have outlined before, according to the different purposes intended with the semantics (see sketch for some examples).



Note that our goal is not to find the quintessential semantics of UML, but to provide the necessary basis for automated tool support, and to provide it in as simple a way as possible. Consider our semantics as an illustration to our approach.

Different purposes may not necessarily use disjoint portions of the domain, or different codomains and mappings, but often they do. One notable exception to this rule is already depicted in the sketch: both StateMachine and Interaction are mapped into the same semantic domain of Petri-nets. All semantic mappings are defined in a way that reflects the structure of the syntactic domain, i.e. the mappings are defined rule-wise, according to the rules the context-free grammar we use as our syntactic domain.

So, tracing from syntax to semantics becomes very easy. For instance, package diagrams are mapped into graphs of namespaces. Irregularities such as circular dependencies in the diagram are preserved in the semantic structure, and thus can be precisely defined and detected automatically there. The same principle applies for more complex properties (consider e.g. the UML scoping rules for protected import into Packages). Also, the semantics as such is more transparent, facilitating both informal and formal arguments concerning its correctness. For instance, structuring the mappings according to the syntactic structure provides us with the structural induction proof principle⁶.

6.7. Syntactic domain

As our syntactical domain, we use a textual representation of an abridged physical metamodel, expressed in Extended Backus Naur Form (EBNF). This approach has certain advantages over using the logical metamodel, or one of the descriptions of the physical metamodel of the UML. Additionally, we use OCL-expressions to express constraints that can not be expressed by a context-free grammar. Note that this is exactly parallel to classical programming languages.

First, using a textual rather than a diagrammatic representation makes the definition of the semantic rules technically easier,⁷ and we have already mentioned the benefits of rule-wise definition of semantic mappings. Also, such rules can later be used with few changes in an implementation.

Next, using the traditional notation for the syntax of programming languages, EBNF, rather than

⁶Though we currently make no use of it. See also Section 6.8 below.

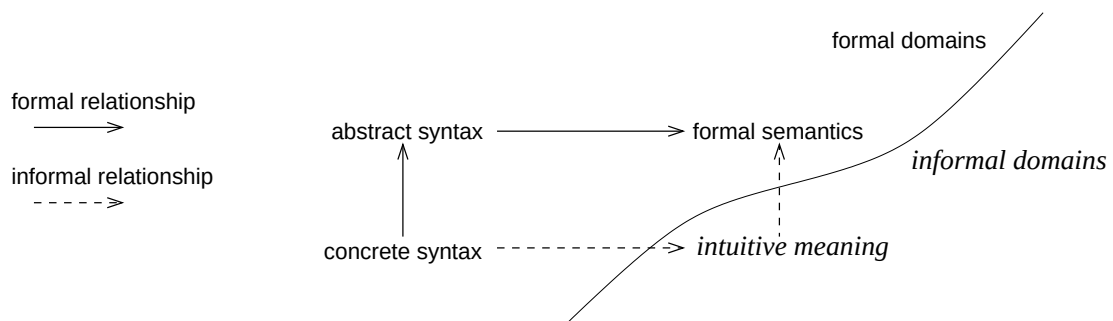
⁷This could have been circumvented by using a graph grammar-based approach, at the cost of prematurely excluding all non-graphical formalisms as semantic domains. The obvious example here are process algebras.

the XML DTD provided by the standard presents certain pragmatic advantages, such as more concise models and easier parsing: the XML description of the metamodel is more than 120 pages long and contains many details related to CORBA and MOF, which are irrelevant for our purposes. Also, EBNF is basically a context-free grammar, and has thus a well defined, simple semantics.

Then, using the physical rather than the logical metamodel as our starting point might seem unnecessary at first, when the logical metamodel is also called the "abstract syntax" of UML, suggesting it is easier to use in defining a semantics. This is not the case, quite the opposite in fact, and the additional layer of abstraction the physical metamodel provides, allows us to elegantly dispose of a number of Elements that do not occur in our approach. At the same time, we can eliminate certain defects of the UML metamodel without tinkering with the logical metamodel, making clear that the changes that we do make are conceptually irrelevant, and that our approach is indeed conservative wrt. the standard. In the next chapter, we will discuss the (adaptations to the) physical metamodel.

6.8. Adequacy

Our approach immediately raises questions concerning its formal status. Despite a massive academic and commercial effort, there is currently no generally agreed upon meaning of many UML constructs. The UML standard itself is by far not precise enough, and even in the parts where it is reasonably formal (in particular in the constraints) it is profoundly inconsistent (see Appendix A on some examples). Thus, providing a formal semantics for (a portion of) the UML is a transition from an informal to a formal domain (see sketch).



Obviously, formalizing an informal language is intrinsically normative, i.e. it *defines* correctness, and can not be subject to a formal proof.

It can, however, be subject to an informal argument concerning its adequacy, i.e. whether it captures the intuition. Such arguments critically rely on the transparency of the transition process from syntactic to semantic objects. Here, the choice of the semantic domains (in particular Petri-nets) is critical. We will elaborate on this issue in Section 10.5.

6.9. Chapter summary

In this Chapter we have given an overview over our approach towards semantics of UML, starting with the purposes it serves, and justifying in detail the way it is done. In the following chapters, we will put this plan into action, step by step. We start in the next chapter by defining the physical metamodel, a slight modification of the logical metamodel, that is expressed in a different formalism to allow for easier treatment.

7. Physical metamodel

The physical metamodel is an additional layer of abstraction separating the logical metamodel from implementations. It accommodates a number of minor adaptations and additional constraints imposed by an implementation, that are not part of the problem domain proper, and thus should be kept separate.

7.1. General principles

Metaclasses may be eliminated when they are purely conceptual and/or defined only to allow the formulation of constraints. Such eliminations considerably simplify the physical metamodel. They can be achieved by transforming a sub-class into a kind-of-attribute of its super-class, similar to the kind-attribute of `PseudoState` of the UML logical metamodel.

The purpose of implementation also demands a number of new concepts and attributes, that are relevant for the physical metamodel only (e.g. the sub-classes of `PresentationElement` and `isSelected`).

7.1.1. Transition from logical to physical metamodel

The logical metamodel describes the concepts used for architectural modeling, embedded in the UML. For a tool implementation and for the definition of the semantics, conceptual clarity is replaced by technical elegance as the most important concern. As it happens, there are some spots in the metamodel where a different way of modeling contributes significantly to the clarity and conciseness of the metamodel. In particular, we make the following changes in our transition from logical to physical metamodel.

- We use Java as our implementation language. Java does not allow arbitrary multiple inheritance, but the logical metamodel contains several occurrences of multiple inheritance. They have to be resolved in the physical metamodel. The simplest way of circumventing multiple inheritance is of course the elimination of the offending super-classes. This is possible when the attributes declared by the super-class are not used (as is the case in our approach for `GeneralizableElement`), or when the superclass has been defined for conceptual purposes only (e.g. `ConnectorRole`). Where this is not possible, it is sometimes possible to use the one kind of multiple inheritance which is allowed in Java, namely implementation of multiple types, called implementation of interfaces in Java¹ (cf. `Component`).
- We add a sub-hierarchy under `PresentationElement` to cater for implementation-related issues.

¹Java-interfaces are special cases of SML-signatures. They contain an arbitrary number of method declarations. So, one can not directly declare inherited attributes, but rather we have to resort to declaring a pair of set- and get-methods for each attribute.

- We completely omit all instance-related metaclasses, and add a boolean attribute `isInstance` to all instantiable metaclasses. Also, we omit `ConnectorRole` and `ConnectorEndRole`, as role, class and object can be treated almost the same in an editor.
- We omit some `PseudoStates` of `StateMachine`, and turn the association between `StateVertex` and `Transition` from 1-1 into n-n.
- Some concepts have a number of optional arguments, most notably `Transition`, which may have a guard, a trigger, and an effect. In defining the formal mapping, we would have to provide eight rules to catch all possible combinations of these arguments. To make the semantics simpler, we just assume optional arguments to be always provided, but possibly with null-values. For instance, a guards may take on the constant value “true”, and the effect may be the empty sequence of actions λ , and so on.
- All metaclasses are concrete classes to allow for top-down parsing. Otherwise, the kind of a `ModelElement` could not be determined at once by JavaCC compilers.

Obviously, these changes have only marginal impact on the UML semantics.

7.1.2. Textual representation

As a convention, we distinguish between non-terminals, **terminals** and *values* typographically like this. Non-terminals that represent a metaclass appear in exactly the same typographic make-up (concerning font, style, size, initial and intermediate capitals, underscores etc.) as in the UML and the rest of this document. There are two types of rules.

Basic rules

First, there are rules to describe the metaclasses proper. The productions for a metaclass start with a terminal that identifies the metaclass. For a `MetaClass`, this would typically be the symbol **mc**:. The attributes defined in this class are represented as tag/value-pairs, where the tag is a terminal with the name of the attribute, and the name of the value indicates the type of the attribute, i.e. a string-valued attribute name is represented as **name string**. Boolean attributes are represented as optional terminals, e.g. **[booleanAttribute]**. Sharing of classes (weak aggregation, the “white diamond”) is realized by reference to the identifiers of the respective classes (e.g. as *id* in `PresentationElement`). The rule for `MetaClass` ends with the non-terminal that represents the (unique) immediate super-class. The OCL-constraints associated to the various `ModelElements` are of course not implemented in the grammar. Altogether, `MetaClass` is represented according to the scheme

$$\text{MetaClass} ::= \mathbf{mc} : \mathbf{attribute\ type} [\mathbf{booleanAttribute}] \dots \text{SuperClass}$$

Here, `SuperClass` is again a metaclass. The top-class, `Element`, has the trailing tag **e**. instead of a super-class, to increase readability of source files. Multiple inheritance of types can be represented by adding interfaces just before the non-terminal for the super-class, i.e. like this:

$$\text{MetaClass} ::= \mathbf{mc} : \dots \text{Interface}_1 \dots \text{Interface}_n \text{SuperClass}$$

Interfaces are represented similar to classes, but without the leading tag and the reference to their immediate superclasses, i.e. as follows.

Interface ::= **i** : **attribute**₁ *type*₁ ... **attribute**_n *type*_n

There are some classes that do not satisfy this scheme, as they are not properly integrated into the UML metamodel by the standard anyway (cf. ElementImport and ElementOwnership).

Polymorphism rules

The rule-schemes we have presented so far have a problem: attributes with non-simple types cannot be represented correctly. Consider an attribute that may hold an arbitrary StateVertex, such as the attribute subvertex of StateMachine. The rule

CompositeState ::= **composite** : ... **subvertex** StateVertex* State

obviously provides the wrong deviations, as it is not possible to derive any of the subclasses of StateVertex. More abstractly, the rules we have presented so far do not take into account sub-typing, i.e. being able to replace objects of a subtype for objects of a super-type. This can be overcome by a set of polymorphism-rules according to the scheme:

MC ::= MetaClass | SubClass₁ | ... | SubClass_n

All SubClass_i are also metaclasses. Consequently, an abstract class has the form

MC ::= SubClass₁ | ... | SubClass_n

while a singleton type is represented as MC ::= MetaClass. The latter could be simplified, of course. It is kept the way it is, however, so that all rules of the grammar strictly adhere to a systematic structure, so that it will be easier to define a mapping to and from other systematic representations of the metamodel, most notably of course the class-diagram representation and the XML DTD. Although we have not done this, we are very confident that this would be an entirely technical task, i.e. it could be done automatically. Observe that the distinction between, say, ModelElement and ME is exactly the distinction between type and kind, made in the UML (see e.g. [OMG UML 1.3, p. 7-17f]).

7.2. PresentationElement

PresentationElement is an abstract metaclass for presenting ModelElements visually. The UML specifies a n-to-n-relationship² to ModelElement, with the associations presentation and subject. Other than this, there are no more details provided by the UML, so we fill in further detail ad lib.

We add the boolean attributes visible, selected, style, font, and fontSize. There are two types of PresentationElement, PNode and PEdge. There is no explicit association between Node and Edge: if there is to be one, it is implied by the represented ModelElements, and accessible as the subject of these. Node has a number of straightforward attributes like at, small, locked. Similarly, there are the attributes from, to and pointed of Edge. See Figure 7.2 for a synopsis.

²E.g., ClassifierRole, Class, and Object all share the same PresentationElement, a rectangle with the name printed inside. On the other hand, one and the same ModelElement may possess several PresentationElements, i.e. a Port may appear at different positions in different Views.

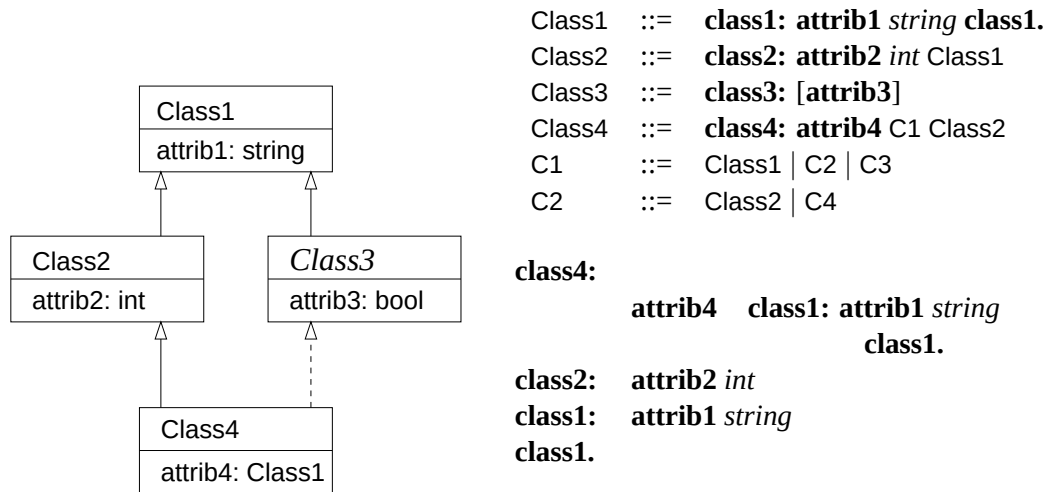


Figure 7.1.: A first example for the textual representation scheme: a class diagram (left), and its textual representation (right, top). A concrete model, that is, an instance of the model on the left, or, a word of the grammar on the right, is shown below.

7.3. Relationships

We use only a few kinds of Relationships, and add only the Connectors as a kind of Association. There are four stereotypes predefined for Abstraction, that is, there are effectively four subclasses, who seem to differ only in their pragmatics. We add a kind-attribute to Abstraction, but allow only the one value that is of interest in this context. See Figure 7.3 for a graphical representation of the relevant part of the metamodel.

7.4. Namespace

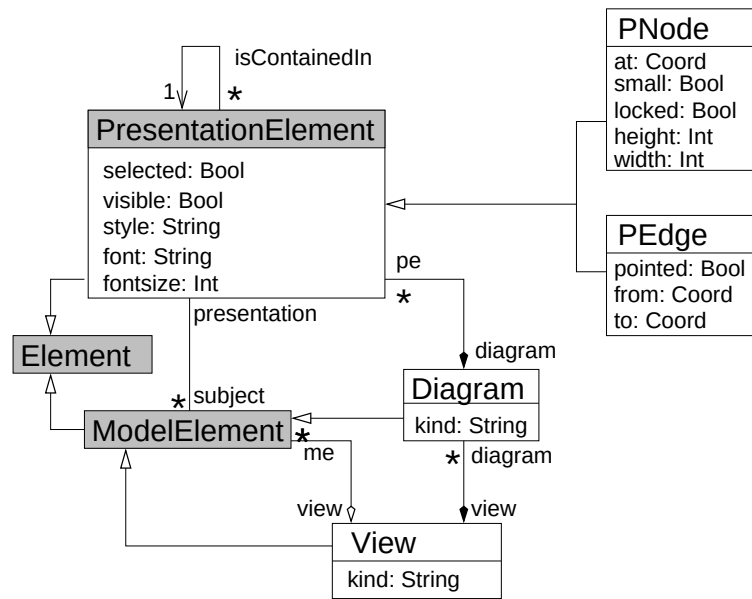
In the logical metamodel, both Port and Capsule have more than one direct superclass: Port is an Actor and a Component, and Capsule is a Package and a Component. The purpose for introducing Component as a superclass was to allow Capsules and Ports in deployment diagrams (and interaction and class diagrams, in the case of Capsules). So, the only aspect of the inheritance from Component that we use is the attribute deploymentLocation. So, we simulate this inheritance relationship in the physical metamodel as the implementation of an interface of type Component.

The inheritance from the metaclass GeneralizableElement is not used at all in our approach, and is thus removed altogether from the physical metamodel. It could be reintroduced as an additional interface, similarly to Component.

Also, all the instance-concepts are removed, and replaced by an additional boolean attribute isInstance of their respective type-counterpart. This applies for CapsuleInstance and PortInstance (see Figure 7.4).

7.4.1. ConfigurationSpace

The concepts Protocol and ConfigurationSpace are subclasses of Package and Collaboration, respectively. They differ from their superclasses only in some constraints, but are structurally identical (see



PE	::=	PresentationElement PNode PEdge
PresentationElement	::=	pe: subject me id* [selected] [visible] style string font string font-size int [small] isContainedIn id Element
PNode	::=	node: at coord height int width int [locked] PresentationElement
PEdge	::=	edge: from coord to coord [pointed] PresentationElement
View	::=	view: me id* diagram Diagram* ModelElement
Diagram	::=	diagram: kind string pe PresentationElement* ModelElement

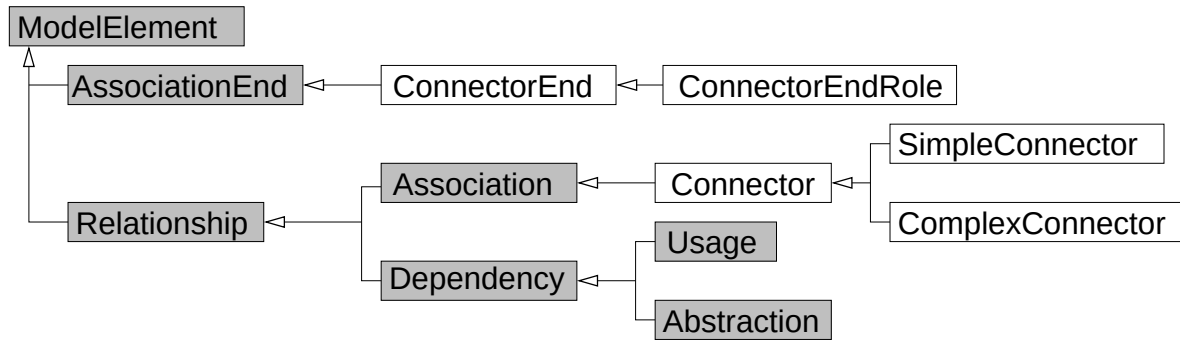
Figure 7.2.: The relationship between PresentationElement and ModelElement in the physical metamodel. Not all subclasses of Node and DiagramView are shown. Shaded boxes indicate standard UML meta-classes.

Figure 7.5).

ConnectorRole and ConnectorEndRole can be omitted, as they have a very similar representation as Connector and ConnectorEnd. Whether the Classifier or the role is intended, is always clear from the context.

7.4.2. Capsule

In the logical metamodel, the structure of Capsules is quite simple: they have just a few simple attributes, and views, that refer to ModelElements that the Capsule owns or imports, virtue to being a Package. We need to add only a boolean attribute isInstance to cater for CapsuleInstances.



R	::=	Relationship Connector D
D	::=	Usage Abstraction
Relationship	::=	relationship: ModelElement
Association	::=	association: connection AssociationEnd* Relationship
Connector	::=	connector: [isInstance] Association
ComplexConnector	::=	ccon: behavior StateMachine characteristics string safety string bandwidth int Connector
SimpleConnector	::=	scon: Connector
Dependency	::=	dependency: client id supplier id Relationship
Usage	::=	usage: Dependency
Abstraction	::=	abstraction: kind "realize" mapping ρ Dependency
AssociationEnd	::=	ae: multiplicity string type Classifier ModelElement
ConnectorEnd	::=	ce: [source] [sink] [isInstance] AssociationEnd

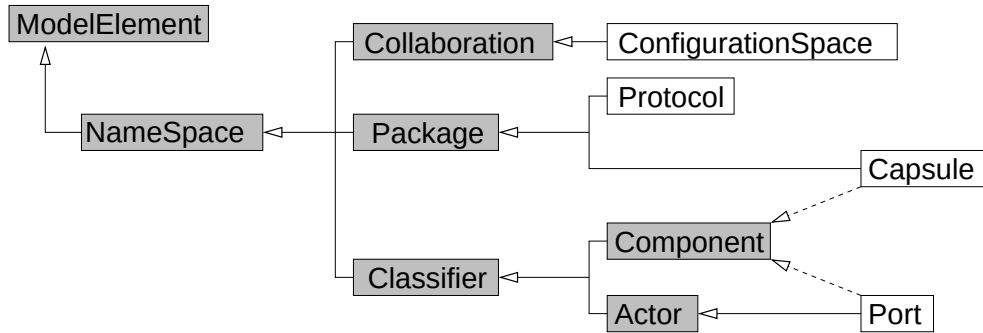
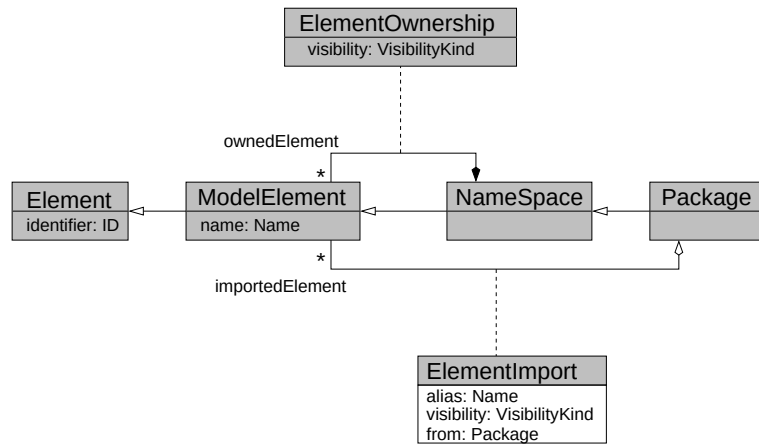
Figure 7.3.: Various kinds of Relationship: Relationship itself is abstract, and has no attributes. Association has a connection, which consists of two or more AssociationEnds. Dependency has a client and a supplier. Deviating from the standard, we assume that both client and supplier exist and are unique. Usage has no proper attributes, but Abstraction has a mappingExpression, which we take to be a mapping between identifiers. Connector is an Association, and ConnectorEnd is an AssociationEnd. The connection of Connectors contains only ConnectorEnds. Connector is abstract, with the two subclasses SimpleConnector and ComplexConnector.

7.5. StateMachine

7.5.1. Overview

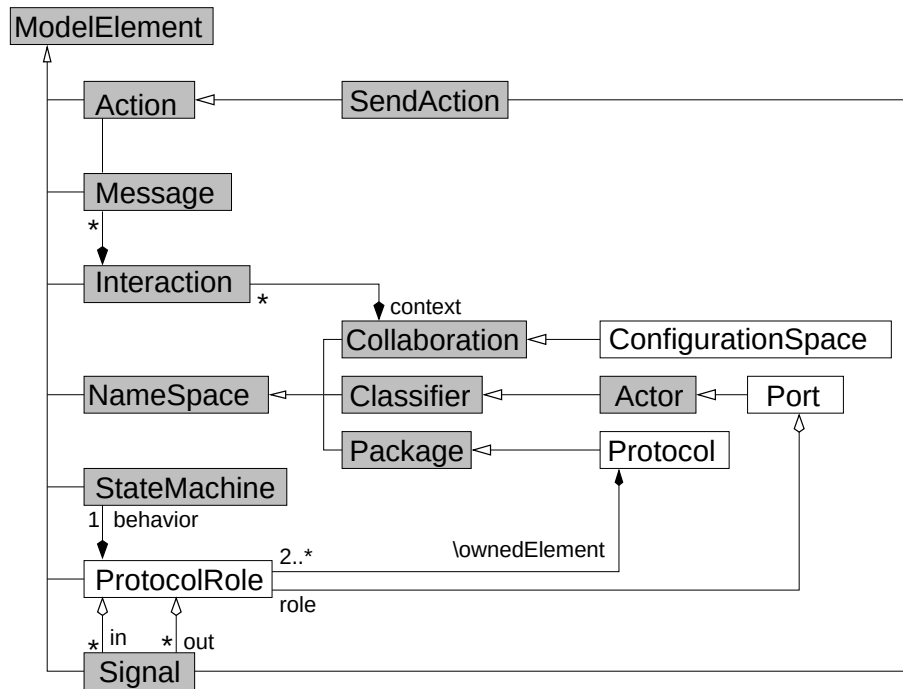
A StateMachine is modeled in a rather unorthodox way: it consists of a set of Transitions and a tree of StateVertexes, starting at a top-State. Actually, the top-state is a State, namely a CompositeState (see [OMG UML 1.3, p. 2-141, constraint 4]). The standard does not provide a convincing explanation for this way of modeling. In fact, it gives no comment whatsoever (see [OMG UML 1.3, p. 2-136]).

A Transition has non-empty sets of StateVertexes as target and source. Optionally, a Transition may have as its trigger an Event, as its guard a Guard, and as its effect an Action. StateVertex has two sub-classes, real States and StubStates, the former again being subdivided into SimpleState, CompositeState, and FinalState. StateVertex and State are abstract classes. States may have Actions associated to them as entry, exit, or doActivity. States may also have some Events associated as deferrable. Sim-



E	::=	Element ME PE
ME	::=	ModelElement NS Classifier View R AssociationEnd ConnectorEnd TaggedValue Comment Message Interaction EV ProtocolRole Configuration StateMachine Transition S A
NS	::=	Namespace Package Collaboration
P	::=	Package Capsule Protocol
Element	::=	e: id id e.
ModelElement	::=	me: name string presentation PE Element
Namespace	::=	ns: ownedElement ElementOwnership* ModelElement
ElementOwnership	::=	own ModelElement visibility (+ # −)
Package	::=	package: importedElement ElementImport* Namespace
ElementImport	::=	import ModelElement visibility (+ # −) alias name from id
Classifier	::=	classifier: Namespace

Figure 7.4.: Various kinds of Namespace: multiple inheritance from Component can be simulated by multiple implementation of Java-interfaces, while GeneralizableElement can be omitted altogether. Shaded boxes indicate standard UML metaclasses.



Protocol	::=	protocol: Package
ProtocolRole	::=	pr: behavior StateMachine in Signal* out Signal* ModelElement
ConfigurationSpace	::=	cs: Collaboration
Collaboration	::=	collaboration: interaction Interaction* Namespace
Interaction	::=	interaction: message Message* context id ModelElement
Message	::=	message: sender id receiver id predecessor id* action id ModelElement

Figure 7.5.: Protocols consist of ProtocolRoles and a role model, i.e. a Collaboration. Shaded boxes indicate standard UML metaclasses.

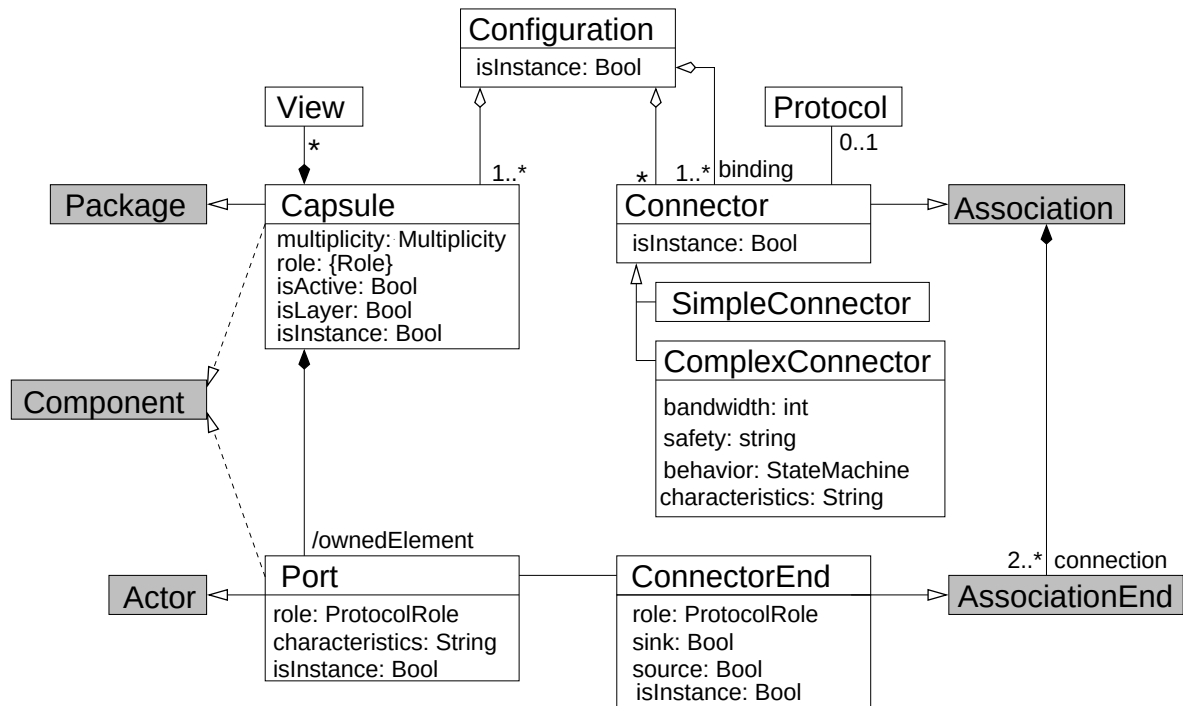
pleState and CompositeState have boolean attributes isInitial and history. A CompositeState owns a non-empty set of concurrent StateMachines. A CompositeState also owns a non-empty set of StubStates, that are used to capture Transitions starting or ending at, or cutting across the boundary of a CompositeState.

We add a string-valued attribute characteristics for quantitative analysis later on. It may contain information similar to the characteristics attributes of Port and Connector.

7.5.2. Differences to the logical metamodel

There are a few differences between logical and physical metamodel (cf. Figures 7.7 and 7.8). Basically, the physical metamodel is almost a projection of the logical metamodel, so that a mapping between logical and physical metamodel is simple.

First, the choice-kind of PseudoState is just syntactic sugar for a set of Transitions. It can be omitted from the abstract syntax without loss. Similarly, the deepHistory-kind of PseudoState can



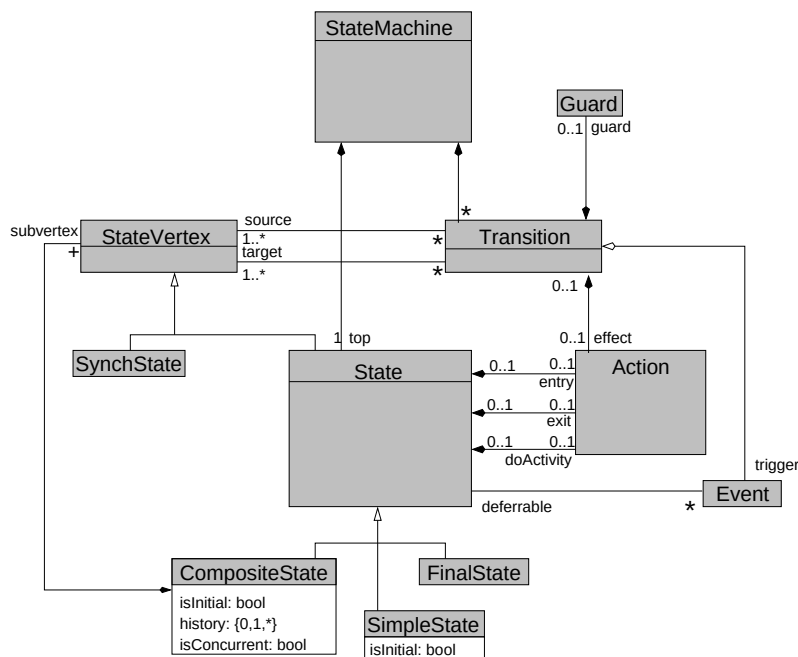
CapsuleRole	::=	project component style pla
Configuration	::=	configuration: capsule <i>id</i> * connector <i>id</i> * binding <i>id</i> * [isInstance] ModelElement
Capsule	::=	capsule: [isActive] [isLayer] CapsuleRole* <i>string</i> view View* [isInstance] multiplicity <i>string</i> ComponentInterface Package
Port	::=	port: role ProtocolRole multiplicity <i>string</i> characteristics <i>string</i> ComponentInterface Package
Component	::=	component: <i>file</i> <i>string</i> ComponentInterface Classifier
ComponentInterface	::=	ci: location <i>id</i>

Figure 7.6.: The physical metamodel for Capsule and related concepts. The differences to the logical model are mainly the isInstance-attributes and the binding-association. The dashed arrows indicate, where interface implementation replaces inheritance. Shaded boxes represent original UML metaclasses.

be simulated as shallowHistory-PseudoStates for all transitively nested substates of the state with the deepHistory-PseudoState.

Then, the initial-, and the shallowHistory-kinds of PseudoState can be replaced by boolean attributes isInitial and hasHistory of CompositeState and SimpleState, respectively.

The standard distinguishes between Transition, connecting exactly two arbitrary StateVertexes, and the “derived concept” compound transition, that “represents a ‘semantically complete’ path made of one or more transitions, originating from a set of states (as opposed to pseudostate) and targeting a set of states” (cf. [OMG UML 1.3, p. 2-147]). So, compound transitions correspond to what intuitively is called a transition, while a UML-Transition is just a line segment in a diagram. We think that such things are out of place in the abstract syntax, and that the derived concept really is the underlying basic



S	::=	FinalState SimpleState CompositeState
A	::=	Action SendAction ActionSequence
EV	::=	SignalEvent
StateMachine	::=	sm: top S transition Transition* ModelElement
StateVertex	::=	sv: ModelElement
SynchState	::=	synch: StateVertex
State	::=	state: deferrable EV* entry A exit A do.activity A StateVertex
SimpleState	::=	simple: [initial] State
FinalState	::=	final: State
CompositeState	::=	composite: [initial] history (0 1 *) (concurrent sequential) subver- tex S* State
Transition	::=	transition: source <i>id</i> target <i>id</i> trigger EV guard Guard effect A Mod- elElement
Guard	::=	guard: expr <i>expr</i> ModelElement
Event	::=	event: parameter string* ModelElement
SignalEvent	::=	signalevent: signal Signal Event
Signal	::=	signal: Classifier
Action	::=	action: (synch asynch) actualArgument <i>expr</i> * ModelElement
SendAction	::=	send: signal Signal Action
ActionSequence	::=	as: action A* Action

Figure 7.8.: The fragment of our physical metamodel dealing with StateMachine and related concepts. Shaded boxes indicate standard UML metaclasses.

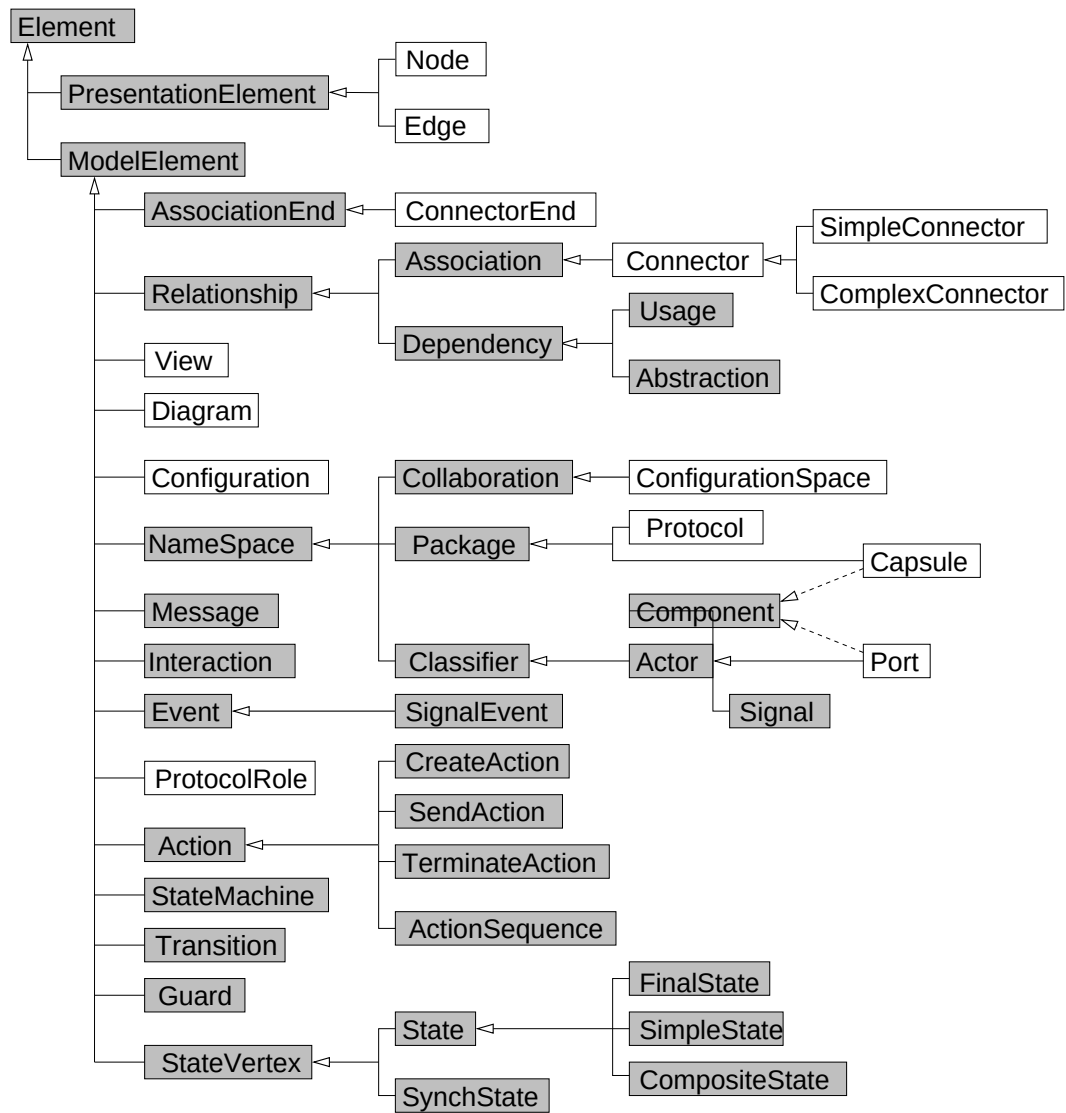


Figure 7.9.: Overview over the physical metamodel, as implemented. Dashed arrows indicate interface implementation replacing inheritance. Shaded boxes indicate standard UML metaclasses.

8. Semantic domains

8.1. General concepts

We understand the natural numbers to include zero. Tuples are always written with angle braces, i.e. $\langle A, B \rangle$ is a tuple with A as its first element, and B as its second element. The usual set union is recursively extended to tuples of sets in the following fashion: $\langle A_1, B_1 \rangle \cup \langle A_2, B_2 \rangle := \langle A_1 \cup A_2, B_1 \cup B_2 \rangle$. Since functions can be represented as sets, they can be written in the form $f = \{x_1 \mapsto y_1, x_2 \mapsto y_2, \dots\}$, i.e. $a \mapsto b$ represents $\langle a, b \rangle$. For a function $f : A \rightarrow B$, we write $f(A') = \{f(x) \mid x \in A'\}$ for $A' \subseteq A$. The restriction of function $f : A \rightarrow B$ to A' (written $f|_{A'}$) is defined as $\{\langle x, f(x) \rangle \mid x \in A \cap A'\}$. The definition of a partial function f is written $f : A \rightharpoonup B$. When a partial function is applied to value it is not defined on, we write $\perp$. The domain and codomain of a total or partial function f from A to B are written as $\text{dom}(f)$ and $\text{cod}(f)$, where $\text{dom}(f) = \{x \mid f(x) \neq \perp\}$ and $\text{cod}(f) = \{f(x) \mid x \in \text{dom}(f)\}$.

Often it is necessary to distinguish between different occurrences of isomorphic things by indexing. Several tags may be attached to a thing consecutively, forming a sequence of tags as the index. A thing without an index is assumed to have the empty sequence λ as its index. Indexes are noted as superscripts, i.e. $\text{thing}^{\text{index}}$. To tag a set of things X with an index i , we define the function $\text{tag}(X, i)$ as follows: $\text{tag}(X, i) := \{x^{j \cdot i} \mid x^j \in X\}$. Singleton sets may be denoted by their element. Indexing can also be used to introduce the disjoint union as $A \uplus B := \{\text{tag}(a, A) \mid a \in A\} \cup \{\text{tag}(b, B) \mid b \in B\}$.

For finite sequences $w \in A^*$, we define some standard notions and convenience functions:

- The empty sequence is always denoted λ .
- The *length*-function $|w|$ returning the number of elements in w .
- The *projection* function w_i that returns the i -th element of w . It is defined only for $i \in \{1, \dots, |w|\}$. $\lambda[i] := \lambda$ for $i > 0$.

The canonical generalization of functions $f : A \rightarrow B$ to sequences of arguments, i.e. $f^* : A^* \rightarrow B^*$ is defined as $f(\lambda) = \lambda$, and $f^*(w) = f(w_1).f^*(w[1])$ for $|w| > 0$. Generalizations of f to sets or tuples of arguments can be defined similarly.

Definition 8.1.1 (ordering relations)

Given a *carrier-set* C , a relation $R \subseteq C \times C$ is a *preorder*, iff it is reflexive and transitive, i.e. (1) $\forall c \in C : c R c$ and (2) $\forall a, b, c \in C : a R b \wedge b R c \implies a R c$. A preorder that is also *antisymmetric* is called a *partial order*, i.e. it has to satisfy the condition $\forall a, b \in C : a R b \wedge b R a \implies a = b$. $\square$

Definition 8.1.2 (graph)

Let N be a set of *nodes* and $E \subseteq N \times N$ a set of *edges* between them, then $\langle N, E \rangle$ is called a *graph*. The functions $\text{pre} : N \times \langle N, E \rangle \rightarrow 2^N$ and $\text{post} : N \times \langle N, E \rangle \rightarrow 2^N$ are defined as $\text{pre}(n) = \{n' \mid \langle n', n \rangle \in E\}$ and $\text{post}(n) = \{n' \mid \langle n, n' \rangle \in E\}$. $\square$

We assume to be present a pervasive universe of symbols called $\mathcal{S}$. These may be interpreted as signals in the application domain, i.e. instances of *Signal*. $\mathcal{S}$ consists of two disjoint sets of *plain* and the *marked* symbols, the latter are written with an overbar. A plain symbol s is turned into a marked symbol $\bar{s}$ by applying the *mark*-function, i.e. $mark(s) := \bar{s}$, and we have that $mark(mark(s)) = s$. Intuitively, marked symbols are supposed to stand for input, and plain symbols stand for output. This is purely intuition, however, and has no semantic repercussions whatsoever. The notation is chosen so as to remind of process algebras like CCS.

8.2. Low-level Petri-nets

Petri-nets are a well-known concept (see Baumgarten [1996], Jensen and Rozenberg [1991], Reisig [1985]), so we need give only a brief account.

8.2.1. Structure

Definition 8.2.1 (labeled low-level Petri-nets)

A tuple $N = \langle P, T; F; \beta \rangle$ is called a (*labeled*) *Petri-net*, iff P and T (the *net elements*) are disjoint sets called *places* and *transitions*. Arcs between them are described by $F \subseteq P \times T \cup T \times P$, the *flow relation*. The labeling function β maps net elements to strings of symbols, i.e. $\beta : T \rightarrow \mathcal{S}^*$. The *pre*- and *post*-sets of a net element x (written $\bullet x$ and $x^\bullet$) are the sets of net elements connected to x : $\bullet x = \{y \in X \mid y F x\}$ and $x^\bullet = \{y \in X \mid x F y\}$. Two nets are called *disjoint*, if the sets of their places and transitions are disjoint. The places and transitions constituting an arc can be selected from this arc by the *place* and *trans* functions, e.g. $place(\langle t, p \rangle) = p$. N is called *linear*, iff $\forall x \in P \cup T \bullet x \leq 1 \wedge x^\bullet \leq 1$. $\square$

As a convention, the simplest possible low level net, the *empty low-level net* $\langle \emptyset, \emptyset; \emptyset; \emptyset \rangle$ is abbreviated as N_0 . As we use the usual set operations generalized to tuples, they also apply to nets. N_0 is the neutral element for union.

Definition 8.2.2 (labeled low-level occurrence net)

Let C be a set of *conditions*, and E a set of *events*, then a Petri-net $\langle C, E; G; \gamma \rangle$ is called a *labeled occurrence net* (also: *condition-event* or *causal net*), iff

1. it is non *place-branching*, i.e. $\forall c \in C : c^\bullet \leq 1 \wedge \bullet c \leq 1$,
2. G is not circular, i.e. $G^+ \cap (G^-)^+ = \emptyset$;
3. G is well-founded;
4. and finitely branching, i.e. for all $c \in C$, $\{e \in E \mid e \in c^\bullet\}$ is finite.

$\square$

8.2.2. Behavior

The definitions in this section are standard. We adopt most of the notation from [Kiehn, 1989, Chapter 1], which is an excellent and concise account of these and the related notions.

Definition 8.2.3 (marking, enabling, occurrence, reachability for low-level nets)

Let $N = \langle P, T; F; \beta \rangle$ be a labeled low-level net. A *marking* is a function $m : P \rightarrow \mathbb{N}$. Markings can also be treated as words of the commutative monoid over P (i.e. multi-sets). The set of all markings of N is denoted $\mathcal{M}(N)$.

Given a marking m of N , a transition $t \in T$ is said to be *enabled* (written $N : m \xrightarrow{t}$), iff there is a token on every pre-place of t , i.e. $m \supseteq \bullet t$. If a transition is enabled, it may *occur* and reach marking m' (written $N : m \xrightarrow{t} m'$), where $m' = m - \bullet t + t \bullet$.

Enabling and occurrence may be generalized to words of transitions, and the notations overloaded as follows:

$$\begin{aligned} N : m \xrightarrow{w} m' &: \iff \exists \{m_1, \dots, m_{|w|}\} \forall 1 \leq i \leq |w| : m_{i-1} \xrightarrow{w_i} m_i \wedge m = m_0 \wedge m' = m_{|w|} \\ N : m \xrightarrow{w.t} &: \iff N : m \xrightarrow{w} m' \wedge N : m' \xrightarrow{t}, \end{aligned}$$

where $w \in T^*$. Wherever N is understood, it may be omitted in the notations.

The set of markings of N that are *reachable* from some $m \in M$, is defined as

$$\begin{aligned} \mathcal{R}_N(M, t) &:= \{m' \in \mathcal{M}(N) \mid \exists m \in M : m \xrightarrow{t} m'\} \\ \mathcal{R}_N(M, w) &:= \begin{cases} \mathcal{R}_N(\mathcal{R}_N(M, t), w') & \text{if } w = t.w' \\ M & \text{if } w = \lambda \end{cases} \\ \mathcal{R}_N(M) &:= \bigcup_{w \in T_N^*} \mathcal{R}_N(M, w) \end{aligned}$$

where $t \in T_N$ and $w \in T_N^*$. When the first argument is a single marking instead of a set, it is understood as a shorthand for the respective singleton set. When N is understood, it may be omitted. A marking m is called a *home marking* of N , iff $m \in \mathcal{R}_N(\mathcal{R}_N(\underline{m}))$ for the initial marking $\underline{m}$ of N . $\square$

In general, we always make the progress assumption. In the Petri-net jargon, that means that whenever anything can happen, eventually something must happen, or, more formally, if any transition is enabled, some transition must occur eventually.

These definitions are sufficient to play the token game, i.e. to execute a net. For interactive simulations, this is sufficient. For formal analysis, richer semantical structures are required.

8.2.3. Semantics

Based on the elementary notions introduced in the previous definition, different semantics may be defined for Petri-nets. In the following three definitions, we present the three most important of them. First we turn to the classic notion of language, i.e. sequential runs similar to those known from the Chomsky-hierarchy.

Definition 8.2.4 (firing sequences and languages of low-level nets)

Let $N = \langle P, T; F; \beta \rangle$ be a labeled low-level net with *initial marking* $\underline{m}$ and *terminal marking* $\bar{m}$. We define the set of all *firing sequences* $\mathcal{F}$, the *language* $\mathcal{L}$ and the *terminal language* $\mathcal{T}$ of N as follows:

$$\begin{aligned} \mathcal{F}(N, \underline{m}) &:= \{w \in T^* \mid N : \underline{m} \xrightarrow{w}\} \\ \mathcal{L}(N, \underline{m}) &:= \{\beta^*(w) \mid N : \underline{m} \xrightarrow{w}\} \\ \mathcal{T}(N, \underline{m}) &:= \{\beta^*(w) \mid N : \underline{m} \xrightarrow{w} \bar{m}\} \end{aligned}$$

There are also *weak* versions of the language and the terminal language, that are the respective sets of sequences projected to some given set $S \subseteq \mathcal{S}$ of symbols: $\mathcal{L}_S(N, \underline{m}) := \mathcal{L}(N, \underline{m})|_S$ and $\mathcal{T}_S(N, \underline{m}) := \mathcal{T}(N, \underline{m})|_S$. $\square$

It is well known, that the languages defined by labeled low-level Petri-nets include the context-free languages and are included by the recursive languages. They are incomparable to the context-sensitive languages. Their computational power is smaller than those of the primitive-recursive functions. See Jantzen [1979a,b] for a thorough discussion.

The language of a net removes both concurrency and choice (see Reisig [1985]) from the semantics, so that for many purposes, net languages are too weak. The process-semantics is *the* classic Petri-net semantics. It preserves concurrency, but removes choice.

Definition 8.2.5 (labeled process of low-level nets)

Let $N = \langle P, T; F; \beta \rangle$ be a labeled low-level net with *initial marking* $\underline{m}$ and *terminal marking* $\overline{m}$, where $\underline{m} \in \mathcal{M}(N)$ and $\overline{m} \in \mathcal{M}(N)$. Let $R = \langle C, E; G; \gamma \rangle$ be a labeled low-level occurrence net and $\eta : C \cup E \rightarrow P \cup T$ a surjective mapping. Then $\langle R, \eta \rangle$ is called a *net process* (or just: process, or: run) of N , iff:

1. η preserves sorts, i.e. $\eta(C) \subseteq P \wedge \eta(E) \subseteq T$.
2. η preserves environments, i.e. $\forall e \in E : \eta(\bullet e) = \bullet \eta(e) \wedge \eta(e^\bullet) = \eta(e)^\bullet \wedge \langle x, y \rangle \in G \implies \langle \eta(x), \eta(y) \rangle \in F$.
3. η preserves labels, i.e. $\forall e \in E : \gamma(e) = \beta(\eta(e))$.
4. η preserves the initial marking, i.e. $\eta(\{p \in C \mid \bullet p = \emptyset\}) = \underline{m}$.
5. R is finitely branching, i.e. $\forall e \in E : |\bullet e| < |\mathbb{N}|$.
6. R is place-bounded, i.e. $\forall e \in E : \bullet e \neq \emptyset \wedge e^\bullet \neq \emptyset$.

The run $\langle R, \eta \rangle$ is called *terminal*, iff $\{c \in C \mid c^\bullet = \emptyset\} = \underline{m}$. The set of all processes of N under $\underline{m}$ is denoted as $\mathcal{PROC}(N, \underline{m})$. $\square$

Processes may be generated from nets with initial markings by the standard procedure of unrolling. Processes (or arbitrary occurrence nets) may be transformed into event structures by stripping the conditions (i.e. places).

Definition 8.2.6 (event structure, see Definition 2.3.6 in Kiehn [1989])

Let $N = \langle C, E; G; \gamma \rangle$ be a labeled low-level occurrence net with the initial marking $\underline{m} = \{c \in C \mid \bullet c = \emptyset\}$. Then $ES = \langle E, G^+|_{E \times E}, \gamma|_E \rangle$ is the *event structure* of N with $\underline{m}$, written $ES(N, \underline{m})$. ES can be *expanded* by refining all events $e \in E$ with $|\gamma(e)| > 1$ into strictly ordered sub-event structures $e_1 \dots e_n$ such that $\gamma(e_i) = \gamma(e)_i$. So, $expand(\langle E, G, \gamma \rangle, e) := \langle E', G', \gamma' \rangle$ where

$$\begin{aligned} E' &:= E - \{e\} \cup \{e_1, \dots, e_n\} \\ G' &:= G|_{E - \{e\}} \\ &\quad \cup \{\langle e_i, e_{i+1} \rangle \mid 0 < i < n\} \\ &\quad \cup \{\langle e', e_1 \rangle \mid \forall_e : e' G e\} \\ &\quad \cup \{\langle e_n, e' \rangle \mid \forall_e : e G e'\} \\ \gamma' &:= \gamma|_{E - \{e\}} \cup \{e_i \mapsto \gamma(e)_i \mid 0 < i \leq n\} \end{aligned}$$

and $|\gamma(e)| = n$. It can be generalized to sets of events, i.e. $expand^*(ES, E') := expand^*(expand(ES, e), E' - \{e\})$ for some $e \in E'$ and $expand^*(ES, \emptyset) := ES$. We define $Expand(ES) := expand^*(ES, \{e \in E \mid 1 < |\gamma(e)|\})$. $\square$

Note that we do not exploit (or even define) any algebraic properties, or refinements, so that the labeling with words rather than symbols can cause no problems.

Definition 8.2.7 (partial word)

A tuple $\langle A, \prec, \gamma \rangle$ is called a (finite) *labeled partial order* over Σ , iff $\langle A, \prec \rangle$ is a (finite) partial order, and γ is a labeling function $\gamma: A \rightarrow \Sigma$. If A can be understood as a set of events, the class of isomorphic labeled partial orders over Σ is called a *partial word* (over Σ). When no confusion can arise, we use representatives of partial words (i.e. concrete elements of the isomorphism class) instead of the class proper.

We say that a partial word $pw = \langle E, \prec, \gamma \rangle$ is *weaker* than another partial word $pw' = \langle E', \prec', \gamma' \rangle$, iff $E = E'$, $\gamma = \gamma'$, and $\prec \supseteq \prec'$.

The set of partial words of a net N with initial marking $\underline{m}$ (written $PW(N, \underline{m})$) is defined by $PW(N, \underline{m}) := \text{Expand}(ES(N, \underline{m}))$. $\square$

In other words, an event structure is what remains of a process when taking away the places and adjusting the flow relation accordingly. A partial word is an isomorphism class of partial orders. A partial word is weaker than another, if it has exactly the same events and labels, but contains fewer ordering constraints, i.e. it is more general. So, a weak partial word is some kind of specification of a stronger partial word.

While event structures are still close to their origin (processes), partial words are more abstract. For any given finite net and initial marking, its event structures each are the strongest elements in a partial order of weaker and weaker partial words, where the bottom elements are the words of the net language (see Proposition 1.2.15 in [Kiehn, 1989, p. 19]).

Partial words and event structures can be represented graphically by a complete graph or a Hasse-diagram put sideways, whose root is the set of events enabled in the initial marking (that is, $\beta(w)$ where $N, \underline{m} : \eta(\underline{m}) \xrightarrow{w}$). Usually, partial words are written sideways, from left to right.

Other techniques may be used for expressing and checking properties of a Petri-net as well, including P- and T-invariants (see e.g. Jessen and Valk [1987], Reisig [1985]), model checking of temporal logic formulas, symmetries (see e.g. Jensen [1996], Schmidt [1996]), reductions (see e.g. Couvreur et al. [1993], Haddad [1990]), and refinements (see e.g. Kiehn [1989], Valette [1979], Vogler [1992]), lazy and stubborn sets (see e.g. Godefroid [1996], Valmari [1993]) to name but a few.

8.2.4. Behavioral equivalences

There are many different kinds of notions of behavioral equivalence between nets (and similar structures), see e.g. [Glabbeek, 1996, chapter 1] for a catholic survey. We will consider three different kinds of semantics: sequential, *interleaving (branching)* and *true concurrency* semantics.

Sequential vs. branching semantics

The simplest possible semantics of systems considers (all) the sequential runs of the system. However, there are semantical differences between systems, that can not be determined by looking at sets of sequential runs alone. Consider the two nets in Figure 8.1.

Obviously, both nets have the same sets of sequences of behaviors, namely $\{a.b, a.c\}$. However, they do differ in their branching structure, that is, at what point decisions are taken: The net on the right can do an a , and then either a b or a c . This choice is mad by action a already in the net on the left. If a were an invisible (or “internal”) action, synchronizing either (a) or (b) with some environment would lead to different deadlocks (see e.g. Glabbeek [1993]).

This type of semantics is based on the interleaving assumption: all actions are atomic and of no duration. So, no two actions happen at the same time, but either the one first and then the other, or

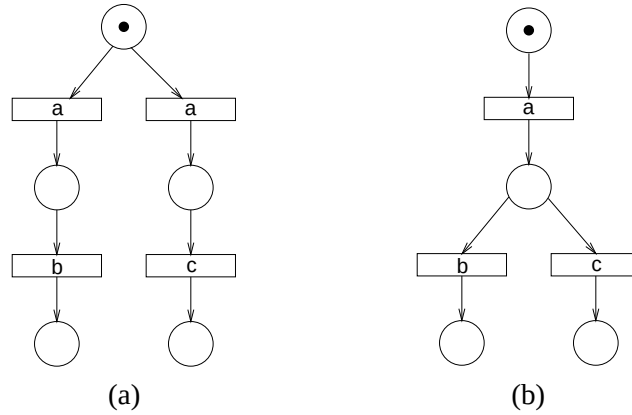


Figure 8.1.: Two nets with the same language, but different branching structures: the choice between b and c is made by a already, but can be observed only afterwards (left); the choice between b and c is made only after a (right). The given marking is the initial marking.

the other way round.¹ This assumption is justified by the idea that the (non-)equality of two systems is always determined by an *observer*, who can make only a single observation at any given time. So, the equalities derived from this paradigm are also called observational equalities.

The interleaving assumption is unrealistic in many cases, e.g. for distributed systems. It is these systems, however, that are in the focus of software architecture. So, we consider also semantics that take into account the causal structure ("true concurrency semantics") rather than mere observations.

True concurrency vs. interleaving

An interleaving point of view considers (permutations of) observable events, while true concurrency considers causal relationships between events. The difference between interleaving semantics and true concurrency semantics shows up when actions are not atomic, i.e. when they have distinctly observable beginnings and endings. This is the for instance the case when actions may be refined, or when they have temporal extent. Consider Figure 8.2.

All three nets of Figure 8.2 have the same sets of sequences of actions, namely $\{a.b.c, a.c.b, c.a.b\}$. They also bisimilar branching structures, that is, under the interleaving assumption they are (observationally) equivalent. However, all three nets can be distinguished when looking at the causal structure: in the net on the left, both a and b are independent from c , in the middle net, c depends on either a or b . The net on the right has causal dependencies between all events, see sketch.

Still, these phenomena are frequently abstracted away, as interleaving is sometimes easier to understand, and easier to use for technical reasons.² Obviously, true concurrency phenomena cannot be analyzed, when they are defined away.

¹So called step-semantics also consider sets of actions happening in lockstep, i.e. all of them at exactly the same time. Theoretically, there is no difference between simple and step interleaving.

²Note, however, that the first hallmark contributions to software architecture used Petri-nets or Petri-net like formalisms, too, see Estrin et al. [1986], Luckham et al. [1995].

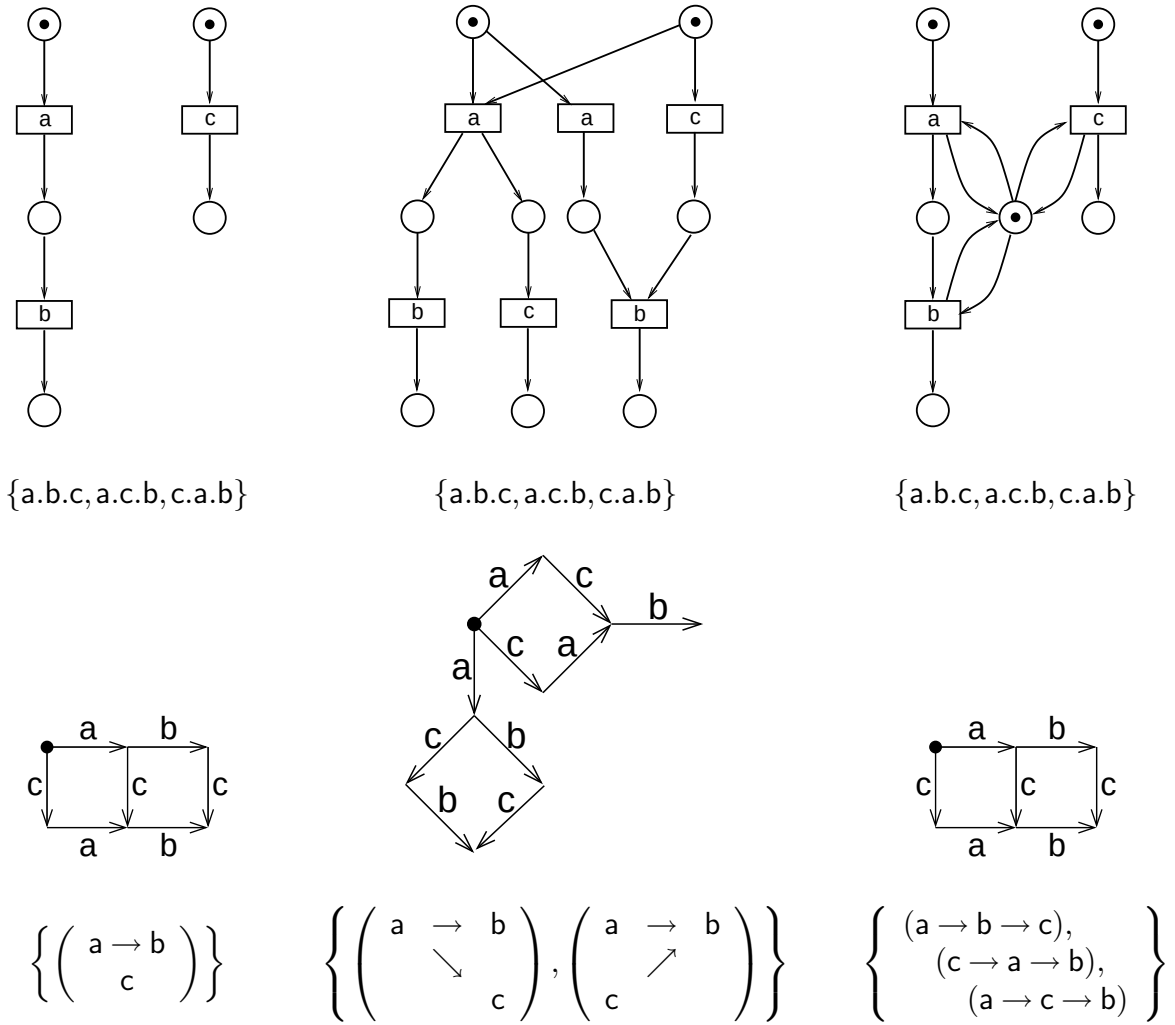


Figure 8.2.: True concurrency distinguishes more systems: three examples (top row, left to right) and their respective sequences, branching structures, and partial words (remaining rows, top to bottom). The given markings are the initial markings. The initial marking is also shown in the branching structure by a fat dot. The partial words are represented as a Hasse-graph of the ordering relation.

Language equivalence

We start with a family of sequential semantics for Petri-nets. They distinguish systems with different sequences of actions, but neither branching structure, nor concurrency. However, they are particularly cheap to compute, so that they are useful as a first check of equality.

Definition 8.2.8 (language equivalence)

Let N_i and N_s be two low-level nets (called implementation and specification) with initial markings m_i and m_s , and $S \subseteq \mathcal{S}$ a set of symbols. Then the specification $\mathcal{L}$ -simulates the implementation (written $\langle N_i, m_i \rangle \stackrel{\mathcal{L}}{\geq} \langle N_s, m_s \rangle$, where $\mathcal{L}$ stands for language) iff the language of the specification contains the language of the implementation (with respect to $S \subseteq \mathcal{S}$), i.e. $\mathcal{L}_S(N_s, m_s) \subseteq \mathcal{L}_S(N_i, m_i)$. If both nets implement each other, i.e. if they have the same languages, they are said to be *language equivalent*, written $N_i, m_i \stackrel{\mathcal{L}}{=} N_s, m_s$. $\square$

Interleaving equivalence

Bisimulations have been introduced by Park [1980], and made popular with process algebras Baeten and Weijland [1990], Milner [1989]³. They are rooted in logics, on the one hand, and game-theory on the other (see Thomas [1990]).

Definition 8.2.9 (branching simulation, branching equivalence)

Let N_i and N_s be two low-level nets (called implementation and specification) with initial markings $\underline{m}_i$ and $\underline{m}_s$, and $S \subseteq \mathcal{S}$ a set of symbols. Then the specification is said to $\mathcal{B}$ -simulate the implementation (written $\langle N_i, m_i \rangle \xrightarrow[\mathcal{S}]{\mathcal{B}} \langle N_s, m_s \rangle$, where $\mathcal{B}$ stands for branching) iff for every sequence of transitions the implementation may execute, there is also a sequence of transitions that the specification may execute, so that both transition sequences represent the same sequences of symbols in $S \subseteq \mathcal{S}$:

$$\langle N_i, m_i \rangle \xrightarrow[\mathcal{S}]{\mathcal{B}} \langle N_s, m_s \rangle : \iff \forall v \in T_s^*, m_s \xrightarrow{v}: \exists w \in T_i^*, m_i \xrightarrow{w}: \beta_s^*(v) =_S \beta_i^*(w)$$

where $v =_S w : \iff v|_S = w|_S$. Two labeled low-level nets $\langle N, m \rangle$ and $\langle N', m' \rangle$ are *interleaving equivalent* wrt. $S \subseteq \mathcal{S}$ (written as $\langle N, m \rangle \xrightarrow[\mathcal{S}]{\mathcal{B}} \langle N', m' \rangle$), iff $\langle N, m \rangle \xrightarrow[\mathcal{S}]{\mathcal{B}} \langle N', m' \rangle$ and $\langle N', m' \rangle \xrightarrow[\mathcal{S}]{\mathcal{B}} \langle N, m \rangle$. $\square$

The definition we use is based on the Definition of [Vogler, 1992, p. 28].

True concurrency equivalence

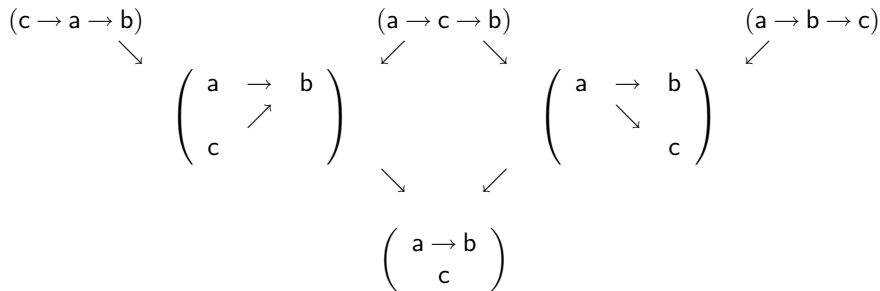
Definition 8.2.10 (true concurrency implementation and equivalence)

Let N_i and N_s be two low-level labeled net with initial markings m_i and m_s . They are called implementation and specification, respectively. We say that N_i $\mathcal{C}$ -simulates N_s under m_i and m_s with respect to a set of symbols $S \subseteq \mathcal{S}$ (written $\langle N_i, m_i \rangle \xrightarrow[\mathcal{S}]{\mathcal{C}} \langle N_s, m_s \rangle$, where $\mathcal{C}$ stands for concurrent), iff $PW_S(N_s, m_s) \subseteq PW_S(N_i, m_i)$. We say that the two nets are pw-equivalent, if they implement each other, i.e.

$$\langle N_i, m_i \rangle \xrightarrow[\mathcal{S}]{\mathcal{C}} \langle N_s, m_s \rangle : \iff \langle N_s, m_s \rangle \xrightarrow[\mathcal{S}]{\mathcal{C}} \langle N_i, m_i \rangle \wedge \langle N_i, m_i \rangle \xrightarrow[\mathcal{S}]{\mathcal{C}} \langle N_s, m_s \rangle.$$

$\square$

The nets in Figure 8.1 are pw-equivalent. The nets in Figure 8.2 are pairwise pw-inequivalent. In fact, there is a partial order on the respective partial words, see sketch.



In this sketch, the nodes of the graph are the partial words of the three nets. The arcs of the graph represent the *is-weaker-than* relationship between partial words.

True concurrency phenomena are intuitive and truly realistic. In particular, they are omnipresent in software architecture, as we understand it, that is, composed of independent, asynchronous architectural units.

³A practically minded introduction is Fencott [1996].

Relationships between equivalences

We have the following classic results concerning these equivalences.

Proposition 8.2.1 (relationship of equivalences)

Let N and N' be labeled low-level nets with initial markings m, m' . Then we have

1. $\langle N, m \rangle \stackrel{\mathcal{B}}{=} \langle N', m' \rangle$ implies $\langle N, m \rangle \stackrel{\mathcal{L}}{=} \langle N', m' \rangle$.
2. $\langle N, m \rangle \stackrel{\mathcal{C}}{=} \langle N', m' \rangle$ implies $\langle N, m \rangle \stackrel{\mathcal{B}}{=} \langle N', m' \rangle$.
3. $\langle N, m \rangle \stackrel{\mathcal{L}}{=} \langle N', m' \rangle$ implies $\langle N, m \rangle \stackrel{\mathcal{C}}{=} \langle N', m' \rangle$.

Proof 8.2.1

1. Obvious; see also Figure 1, p. 20 of Glabbeek [1996].
2. Idea: observe that unrolling a process from a net resolves choices, but retains concurrency. The true-concurrency equivalence regards *all* unrollings, however, so that choices are respected. See also proposition 1.2.15 of Kiehn [1989].
3. Follows immediately from (1) and (2) by transitivity. ■

The equivalences we have defined are insensitive to either weak or strong fairness.⁴ That is, they do not distinguish between fair and unfair implementations. This issue has to be treated separately.

All three of the implementation-relationships are transitive.

Proposition 8.2.2 (transitivity of equivalences)

$\stackrel{x}{\geq}$ is transitive for $x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$.

Proof 8.2.2

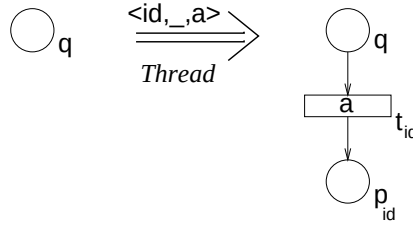
$x = \mathcal{L}$: Set inclusion is transitive.

$x = \mathcal{B}$: Bisimulations are transitive (see e.g. Proposition 5 of [Milner, 1989, p. 154]).

$x = \mathcal{C}$: the is-less-than-relation of partial orderings is transitive. ■

8.2.5. Low-level net operators

In this section, some operators for creating and combining low-level nets in different ways are defined. Mathematically, nets are just tuples of sets. So, the parallel composition of disjoint nets is simply their union. To mark this operation, we will denote it as $\cup$. We define all operators on a single net. In order to combine several nets, one has to take their union first, and then apply the operator, i.e. $Operator(N_1 \cup N_2, argument)$.

Figure 8.3.: Intuition of the *Thread*-operator.

8.2.6. Creating thread-like nets

The *Thread*-operator builds a linear occurrence net from a sequence w of labels, so that afterwards, the net language $\mathcal{L}$ contains only prefixes of w . See also Figure 8.3.

Definition 8.2.11 (*Thread-operator*)

Let N be a low-level net and $\langle id, p, a \rangle$ a triple representing the identifier, predecessors and signal of a message emitted or absorbed by the thread. Then the *Thread*-operator is defined as:

$$Thread(N, \langle id, -, a \rangle) := N \cup \langle \{p_{id}\}, \{t_{id}\}; \{\langle t_{id}, p_{id} \rangle, \langle p_{id}, t_{id} \rangle\}; \{t_{id} \mapsto a\} \rangle.$$

where p_{id} and t_{id} are new places and transitions, respectively, indexed by the identifier id provided with the second argument. It can be generalized to sequences of triples as the second argument by $Thread^*(N, \lambda) := N$ and $Thread^*(N, m.msgs) := Thread^*(Thread(N, m), msgs)$. $\square$

Note that id may be marked, i.e., $\overline{id}$. Obviously, it is indeed the case that $\mathcal{F}(Thread^*(msgs, roles), \{p \mid \bullet p = \emptyset\}) = signal^*(msgs)$, where $signal(\langle -, -, a \rangle) := a$.

8.2.7. Connecting nets asynchronously

Intuitively, the asynchronous connection of nets amounts to inserting a new place to a net as some kind of communication channel, and connecting it to matching pairs of transitions that put and get messages to that channel. The intuition is illustrated in Figure 8.4 below.

Definition 8.2.12 (*Connect-operator*)

Let $N = \langle P, T; F, \beta \rangle$ be a low-level Petri-net, and id an identifier. Then $Connect(N, id)$ is defined as $\langle P \cup P', T; F \cup F'; \beta \rangle$ where

$$\begin{aligned} P' &= \{buffer_{id} \mid t_{id} \in T \wedge \overline{t_{id}} \in T\} \\ F' &= \{\langle t_{id}, buffer_{id} \rangle, \langle buffer_{id}, \overline{t_{id}} \rangle\}. \end{aligned}$$

and $buffer_{id}$ is a new place indexed by the identifier id already attached to the transitions that are to be connected. The *Connect*-operator can be canonically generalized to sets of symbols by $Connect^*(N, \emptyset) := N$ and $Connect^*(N, \{s_1, \dots, s_n\}) := Connect^*(Connect(N, s_1), \{s_2, \dots, s_n\})$. $\square$

⁴In the Petri-net jargon, a transition is weakly fair, if it must occur eventually if it is enabled permanently. A transition is called strongly fair, if it must occur eventually if it is enabled infinitely often. Note that in the Petri-net world, fairness can be for individual transitions (i.e. actions or events), which is much easier to implement technically than fairness for *all* transitions, which is commonly assumed in other formalisms.

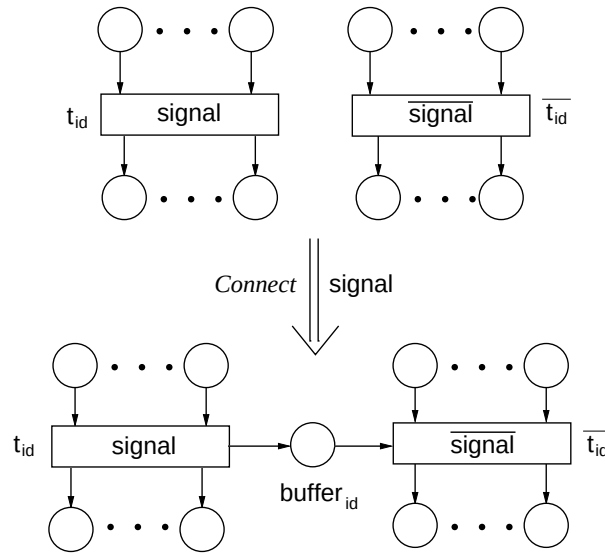


Figure 8.4.: Intuition of the *Connect*-operator. If some net element is shown without inscriptions, the inscription may be assumed to be the empty word.

Connecting occurrence nets this way preserves the individual behaviors of occurrence nets. That is, given a set $\{N_1, \dots, N_n\}$ of occurrence nets with initial markings $m_1, \dots, m_n$ (i.e. $m_i = \{p \mid \bullet p = \emptyset\}$), we have

$$\langle \text{Connect}(\bigcup_{i \in \{1, \dots, n\}} N_i, S), \bigcup_{i \in \{1, \dots, n\}} m_i \rangle \stackrel{x}{\mathcal{S}} \langle \bigcup_{i \in \{1, \dots, n\}} N_i, \bigcup_{i \in \{1, \dots, n\}} m_i \rangle$$

for $x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$ and $S \subseteq \mathcal{S}$.

8.2.8. Synchronizing nets

The *Synch*-operator is used to synchronize sets of transitions, i.e. to merge them into a single transition. It is usually used in conjunction with a union of nets as follows: $\text{Synch}(N \cup N', id)$, where id is an identifier. In general, applying *Synch* does not preserve any behavioral equivalences.

Definition 8.2.13 (*Synch*-operator)

Let $N = \langle P, T; F, \beta \rangle$ be a low-level Petri-net, typically the union of several disjoint nets, and $T'' \subseteq T$ a set of transitions. Then $\text{Synch}(N, s)$ is defined as $\langle P, T'; F'; \beta' \rangle$ where

$$\begin{aligned} T' &:= T - T'' + \{t_{\text{new}}\} \\ F' &:= F|_{P \times T' \cup T' \times P} \\ &\quad \cup \{ \langle p, t_{\text{new}} \rangle \mid \exists t \in T'': p F t \} \\ &\quad \cup \{ \langle t_{\text{new}}, p \rangle \mid \exists t \in T'': t F p \} \\ \beta'(t) &= \begin{cases} \beta(t) & \text{if } t \in T \cap T' \\ \beta(t_{id}) \cdot \beta(t_{id}) & \text{if } t = t_{id} \cdot \bar{t}_{id} \end{cases} \end{aligned}$$

This operator may be extended to sets of sets of transitions $\mathcal{T}$ by defining $\text{Synch}^*(N, 0) := N$ and $\text{Synch}^*(N, \mathcal{T}) := \text{Synch}^*(\text{Synch}(N, T), \mathcal{T} - T)$ where $T \in \mathcal{T}$. Finally, the *Synch*-operator may be gen-

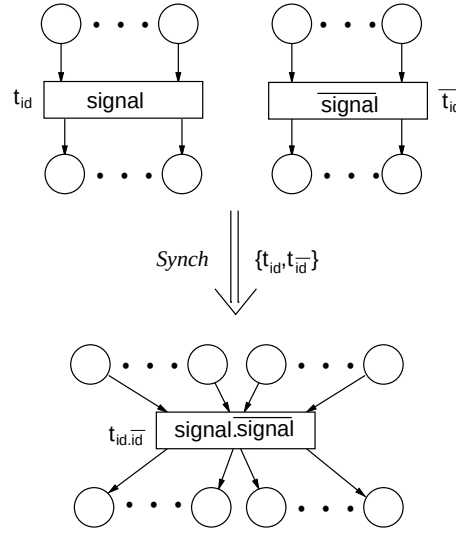


Figure 8.5.: Intuition of the *Synch*-operator. If some net element is shown without inscriptions, the inscription may be assumed to be the empty word.

eralized to sets of symbols that indicate sets of sets of transitions carrying that symbols as their inscriptions, to be used for synchronization. Then we have

$$\text{Synch}(N, L) := \text{Synch}^*(N, \{\{t \in T_N \mid l \in \beta(t)\} \mid l \in L\})$$

□

8.3. Chapter summary

In this chapter, we have defined the semantical domain of our semantics, that is, low-level Petri-nets. The operators we have introduced are non-standard, and make the definition of the semantical mappings considerably easier. In the next chapter, we shall define these mappings.

9. Semantic mappings for models

In this chapter, we provide denotational semantical mappings from expressions of the textual representation of the physical metamodel (viz. the context-free grammar of Chapter 7) into the semantical domains (Section 8). We distinguish between semantical mappings for the context, for creating executable models, for creating models for quantitative analysis and so on, according to the resulting mathematical objects and the purposes they are subjected to. It is assumed, that suitable semantics are available for Guard, Event, Action and several other inscriptions. We will not provide mappings for them, but assume them given (e.g. $\llbracket \text{Guard} \rrbracket$ and so on).

9.1. Preliminaries

We always define semantical mappings as families of clauses that share the name and are distinguished by an index (that may be omitted if there is only one member of the family). Each member of the family has a unique argument type, while the result types may coincide. See Appendix C for a complete overview of all semantical mappings.

9.1.1. Notation

Each rule is printed starting with the name of the mapping, possibly indexed by the number of the clause. The name of the mapping is also subscripted to each clause. Optionally, a further transformation argument may be superscripted to a clause. So, the general form of the individual clauses of semantical mapping functions is as follows:

$$\text{mapping}_{\text{clause}} \quad \llbracket \text{expression} \rrbracket_{\text{mapping}}^{\text{argument}} := \text{mathematical object}$$

Each clause has a *expression* enclosed in semantics brackets on its left hand side, and a *mathematical object* as the result of the translation on its right hand side.

9.1.2. Elementary rules and rule-schemes

Terminals and variables denote themselves, i.e. the respective values. There are some very simple transformations, that we specify by rule-schemes only, such as the attribute selection functions. In defining these rule-schemes, we use the meta-variables TYPE, ATTR and SUPER to denote those parts of an expression concerned with the type, the attributes and the superclass of a metaclass, respectively. Recall that the textual representation of the physical metamodel strictly follows a scheme, where there is a production for each metaclass *MetaClass*, starting with a tag followed by the attributes of *MetaClass*, and a non-terminal indicating the unique direct *SuperClass* (which is of course a *MetaClass*):

$$\text{MetaClass} ::= \overbrace{\text{tag}}^{\text{TYPE}} : \overbrace{\text{attrib type} \dots}^{\text{ATTR}} \overbrace{\text{SuperClass}}^{\text{SUPER}}$$

We use ... to denote irrelevant subexpressions. The subexpression ends either at the end of the expression, or at the subexpression given (in the example the SUPER-clause). Consider the ID-, NAME-, TYPE- and KIND-functions as simple examples for a rule schemes, that select the **id**- and **name**-attributes, and determine the type of a metaclass, respectively:

$$\begin{aligned}
 ID_1 : \quad & \llbracket \mathbf{e} : \mathbf{id} \, id \, \mathbf{e} \rrbracket_{ID} &:= id \\
 ID_2 : \quad & \llbracket TYPE \, ATTR \, SUPER \rrbracket_{ID} &:= \llbracket SUPER \rrbracket_{ID} && \text{if } TYPE \neq \mathbf{e} \\
 \\
 NAME_1 : \quad & \llbracket \mathbf{me} : \mathbf{name} \, name \rrbracket_{\text{ModelElement}} &:= name \\
 NAME_2 : \quad & \llbracket TYPE \, ATTR \, SUPER \rrbracket_{NAME} &:= \llbracket SUPER \rrbracket_{NAME} && \text{if } TYPE \neq \mathbf{me} \\
 \\
 TYPE : \quad & \llbracket TYPE \, ATTR \, SUPER \rrbracket_{TYPE} &:= TYPE \\
 \\
 KIND_1 : \quad & \llbracket TYPE \, ATTR \, SUPER \rrbracket_{KIND}^{id} &:= \begin{cases} \{TYPE\} \cup \llbracket SUPER \rrbracket_{KIND} & \text{if } self = id \\ \emptyset & \text{otherwise} \end{cases} \\
 KIND_2 : \quad & \llbracket TYPE \, ATTR \, SUPER \rrbracket_{KIND} &:= \{TYPE\} \cup \llbracket SUPER \rrbracket_{KIND}
 \end{aligned}$$

Additionally, we use the symbol *self* as a macro-like abbreviation for $\llbracket SUPER \rrbracket_{ID}$. We use sequences to represent sets. In general, there may be sets of objects of any metaclass X, i.e. the expression X^* denotes a set of X. We understand the following rule scheme to be defined for such sets, *unless defined differently* (*arg* is an arbitrary argument).

$$\begin{aligned}
 \llbracket X \, X^* \rrbracket^{arg} &:= \llbracket X \rrbracket^{arg} \cup \llbracket X^* \rrbracket^{arg} \\
 \llbracket \lambda \rrbracket^{arg} &:= \emptyset
 \end{aligned}$$

When using a view as a parameter to a mapping, we use the first letter of the view's kind to abbreviate the kind, i.e. instead of functionality, we write f, and similar for the behavior, structure, and interfaces views. We abbreviate the implementation view-kind as im.

9.2. Event, Action, Operation, and Signal

Before we can go on to define the semantics of StateMachine and Interaction, we need to establish a fundamental point about the relationship of behavioral elements in the UML metamodel. If interaction diagrams are to represent runs of state transition diagrams, for instance, there has to be a semantic connection between the two respective metaclasses, that is, between Collaboration and StateMachine. Similarly, if one concurrent region of a StateMachine is to communicate with another, there must be some path to exchange information. Consider Figure 9.1 which shows the relevant portion of the metamodel.

There are two such paths, both quite similar, the one via Signal, the other via Operation. Consider first the Signal-path. On the one hand, Transitions are triggered by Events, which may be SignalEvents. A SignalEvent is associated with exactly one Signal. On the other hand, Messages have as their action an Action, that may be a SendAction, which, again, is associated with a Signal. So, if some Transition fires, it may cause a SendAction with some Signal, that stimulates a SignalEvent triggering another Transition.

Similarly, relating StateMachines and Interactions, the action of a Message may be a SendAction. Thus, the occurrence pattern of Signals in an actual run of a StateMachine can be captured in an Interaction, *as long as one is concerned with SendActions and SignalEvents only*. How the information transfer is to be accomplished for other types of Event and Action, and in particular the role of Stimulus is as yet unclear in the UML community (see also the UML unresolved issues database, OMG Issues DB).

The Operation-path is structurally similar, but uses CallAction, Operation, and CallEvent instead.

In the remainder, we treat only the Signal-path, because first, architectural units are large, self-sufficient entities, they will run in individual threads or processes, possibly even on different machines, so it is assumed that they communicate mostly asynchronously, and a Signal is a “*specification of an asynchronous stimulus between two instances*” (cf. [OMG UML 1.3, p. 2-94]). Second, architectural units are supposed to have no technology bias, but an Operation is a “*service that can be requested from an object*” (cf. [OMG UML 1.3, p. 2-39]). Operations could be treated very similarly, if that were required.

We model this information exchange mechanism by inscribing Petri-net transitions with (sequences of) symbols, i.e. words from S^* . We use plain symbols for Signals emitted by SendActions as the effect of a Transition or the action of a Message, and marked symbols for Signals consumed by SignalEvents as the trigger of a Transition.

Note that the standard also declares that a Stimulus “*reifies a communication between two instances [as] a signal sent [...] or an invocation of an operation*” (cf. [OMG UML 1.3, p. 2-94]). This seems to indicate an important role for Stimulus in mediating these communication paths. However, any further details of this mechanism are left to the imagination of the reader.

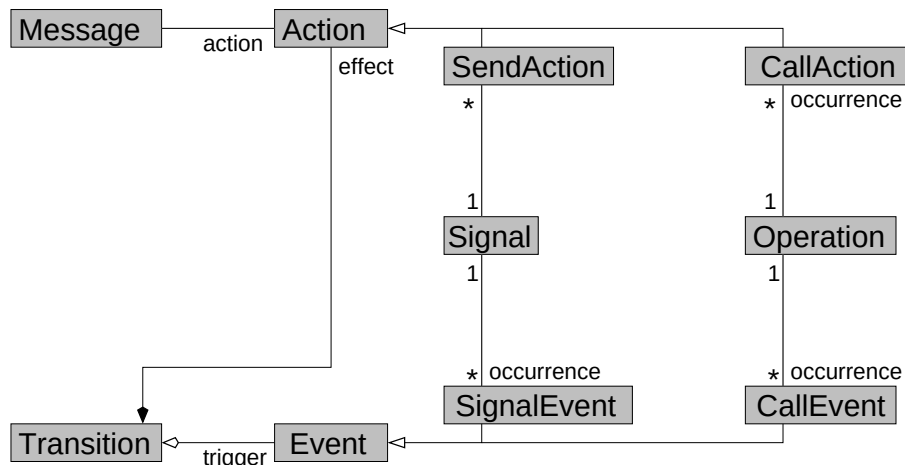


Figure 9.1.: The relationships between the metaclasses Event, Action, and Signal in the UML metamodel

$$\begin{aligned}
 \text{SigIds}_1 &: \llbracket \text{Capsule} \rrbracket_{\text{SigIds}}^{\text{view.type}} := \bigcup_{id \in \llbracket \text{Capsule} \rrbracket_{\text{Ids}}^{\text{view.type}}} \llbracket \text{Capsule} \rrbracket_{\text{SigIds}}^{id} \\
 \text{SigIds}_2 &: \llbracket \text{Capsule} \rrbracket_{\text{SigIds}}^{id} := \llbracket \text{SUPER} \rrbracket_{\text{SigIds}}^{id}
 \end{aligned}$$

$$\begin{aligned}
\text{SigIds}_3 : \llbracket \text{package : importedElement} \rrbracket_{\text{SigIds}}^{id} &:= \llbracket \text{ElementImport}^* \rrbracket_{\text{SigIds}} \cup \llbracket \text{SUPER} \rrbracket_{\text{SigIds}} \\
\text{SigIds}_4 : \llbracket \text{ns : ownedElement} \rrbracket_{\text{SigIds}}^{id} &:= \llbracket \text{ElementOwnership}^* \rrbracket_{\text{SigIds}} \\
\text{SigIds}_5 : \llbracket \text{import ME} \dots \rrbracket_{\text{SigIds}}^{id} &:= \begin{cases} \{\llbracket \text{ME} \rrbracket_{\text{SigIds}}\} & \text{if } id = \llbracket \text{ME} \rrbracket_{\text{ID}} \\ \emptyset & \text{otherwise} \end{cases} \\
\text{SigIds}_6 : \llbracket \text{own ME} \dots \rrbracket_{\text{SigIds}}^{id} &:= \begin{cases} \{\llbracket \text{ME} \rrbracket_{\text{SigIds}}\} & \text{if } id = \llbracket \text{ME} \rrbracket_{\text{ID}} \\ \emptyset & \text{otherwise} \end{cases} \\
\text{SigIds}_7 : \llbracket \text{ia : message} \rrbracket_{\text{SigIds}}^{role} &:= \llbracket \text{Message} \rrbracket_{\text{SigIds}}^{role} \\
\text{SigIds}_8 : \llbracket \text{Message} \rrbracket_{\text{SigIds}}^{role} &:= \begin{cases} \{\llbracket \text{Message} \rrbracket_{\text{ID}}\} & \text{if } role \in \llbracket \text{Message} \rrbracket_{\text{Roles}} \\ \emptyset & \text{otherwise} \end{cases} \\
\text{Signal}_1 : \llbracket \text{message :} \dots \text{action Action} \dots \rrbracket_{\text{Signal}} &:= \llbracket \text{Action} \rrbracket_{\text{Signal}} \\
\text{Signal}_2 : \llbracket \text{sendaction : signal Signal} \dots \rrbracket_{\text{Signal}} &:= \llbracket \text{Signal} \rrbracket_{\text{NAME}} \\
\text{Signal}_3 : \llbracket \text{actionsequence : action Action}^* \dots \rrbracket_{\text{Signal}} &:= \llbracket \text{Action}^* \rrbracket_{\text{NAME}} \\
\text{Signal}_4 : \llbracket \text{Action Action}^* \rrbracket_{\text{Signal}} &:= \llbracket \text{Action} \rrbracket_{\text{Signal}} \cdot \llbracket \text{Action}^* \rrbracket_{\text{Signal}} \\
\text{Signal}_5 : \llbracket \lambda \rrbracket_{\text{Signal}} &:= \lambda \\
\text{Signal}_6 : \llbracket \text{signalevent : signal Signal} \dots \rrbracket_{\text{Signal}} &:= \llbracket \text{Signal} \rrbracket_{\text{NAME}} \\
\text{Signal}_7 : \llbracket \text{ia : message Message}^* \dots \rrbracket_{\text{Signal}} &:= \llbracket \text{Message}^* \rrbracket_{\text{Signal}} \\
\text{Signal}_8 : \llbracket \text{Capsule} \rrbracket_{\text{Signal}} &:= \llbracket \text{Capsule} \rrbracket_{\text{In}} \cup \llbracket \text{Capsule} \rrbracket_{\text{In}} \\
\text{Signal}_9 : \llbracket \text{Capsule} \rrbracket_{\text{Signal}}^{\text{bindings}} &:= \llbracket \text{Capsule} \rrbracket_{\text{In}} \cup \llbracket \text{Capsule} \rrbracket_{\text{In}} \\
\text{Signal}_{10} : \llbracket \text{Connector} \rrbracket_{\text{Signal}} &:= \llbracket \text{SUPER} \rrbracket_{\text{Signal}} \\
\text{Signal}_{11} : \llbracket \text{association : connection} \rrbracket_{\text{Signal}} &:= \llbracket \text{AssociationEnd}^* \rrbracket_{\text{Signal}} \\
\text{Signal}_{12} : \llbracket \text{ae :} \dots \text{type} \rrbracket_{\text{Signal}} &:= \llbracket \text{SUPER} \rrbracket_{\text{Signal}}
\end{aligned}$$

For our purposes, the name of a Signal is sufficient as its semantics. Note that the in and out attributes of ProtocolRoles are of type Signal set rather than Event or Action set, and that Subsystems use Operations (see also Section 2.2).

9.3. Creating a sample run from Interaction

We use Interactions as deterministic scenarios, restricting ourselves to ClassifierRoles that dispatched always only one Message at any time. In other words, we do not take into account activator, branches, and roles that initiate or accept more than one message at one time. Expressions are constants (i.e. variables have a fixed value). So, inscriptions can be considered as symbolic inscriptions without operational meaning. All in all, this makes the Interaction-semantics a comparatively easy case. Concerning the relationship to StateMachine, see the previous Section.

9.3.1. Intuitive semantics of Interaction

An Interaction always refers to a context, which is a set of ClassifierRoles as part of a Collaboration. All Classifiers and Instances can be represented by a ClassifierRole. Each role represents a thread or co-routine that may exchange Messages. With the restrictions we use, semantically an Interaction is a directed acyclic graph (and a partial order, see [OMG UML 1.3, p. 2-114]) of Messages. Since a Petri-net process is a partial order of events, there is a straightforward mapping between the two. Intuitively, an Interaction represents both a *system run*, i.e. a collaborative effort of all roles involved, and a set of *individual runs* for each role on its own.

9.3.2. Mapping

There are two ways that an Interaction can be used: as a system run, i.e. as a run of an ensemble of roles, and as an individual run, i.e. a run of one of the roles in isolation. The latter can then be checked against the individual behavior specifications of the roles. We assume in the remainder, that at any point in time, a role may dispatch or receive only a single Message.

The mapping for Interaction takes into account the dual role of Interactions. In order to create a system run, first a set of individual threads is created, one for each role involved in the system run. Then, these threads are connected together. This way, both individual runs and collaborative runs can be created using the same semantical mappings.

Starting from a Collaboration, the Interaction with identifier i is selected.

$$\begin{aligned}
 \text{Run}_1 : \quad & \left[\begin{array}{l} \text{collaboration} : \dots \text{interaction} \\ \text{Interaction}^* \dots \end{array} \right]_{\text{Run}}^i := [\text{Interaction}^*]_{\text{Run}}^i \\
 \text{Run}_2 : \quad & [\text{Interaction} \text{ Interaction}^*]_{\text{Run}}^i := \begin{cases} [\text{Interaction}]_{\text{Run}} & \text{if } [\text{Interaction}]_{\text{ID}} = i \\ [\text{Interaction}^*]_{\text{Run}}^i & \text{otherwise} \end{cases} \\
 \text{Run}_3 : \quad & [\lambda]_{\text{Run}}^i := N_0
 \end{aligned}$$

Then, the run itself is created by connecting a set of individual threads. For each ClassifierRole that is involved in the Interaction, an individual thread is created.

$$\text{Run}_4 : [\text{Interaction}]_{\text{Run}} := \text{Connect}(\bigcup_{r \in R} [\text{Interaction}]_{\text{Thread}}^r, \bigcup_{r \in R} [\text{Interaction}]_{\text{SigIds}}^r)$$

where $R = [\text{Interaction}]_{\text{Roles}}$ abbreviates the set of all roles.

$$\begin{aligned}
 \text{Roles}_1 : \quad & [\text{ia : message Message}^* \dots]_{\text{Roles}} := [\text{Message}^*]_{\text{Roles}} \\
 \text{Roles}_2 : \quad & [\text{message : sender } s \text{ receiver } r \dots]_{\text{Roles}} := \{s, r\}
 \end{aligned}$$

The second argument provides the set of all Signals that are exchanged between roles in the Interaction.

A thread represents a run of an isolated role. It is created using the *Thread**-operator,¹ which translates an ordered sequence of Message representations into a linear net. The first argument of the operator is a simple net that has just an initial place named after the role.

¹*Thread** is the canonical generalization of *Thread* to sequences of Message-representations.

$$\text{Thread} : \left[\begin{array}{l} \mathbf{ia : message} \\ \mathbf{Message^* \dots} \end{array} \right]_{\text{Thread}}^{\text{role}} := \text{Thread}^* \left(\begin{array}{c} \langle \{role\}, \emptyset; \emptyset; \emptyset \rangle, \\ \text{sort}(\llbracket \text{Message}^* \rrbracket_{\text{Msg}}^{\text{role}}) \end{array} \right)$$

The individual threads are created from an Interaction by mapping its Messages into tuples representing the identifier, the predecessors, and action of that Message. These tuples are sorted according to the partial order given by the predecessor-relation: $\text{sort}(w)$ generates an arbitrary permutation of w , such that $\forall_{1 \leq i < j \leq |w|} : id(w_j) \notin \text{pred}(w_i)$, where $id(\langle x, -, - \rangle) = x$ and $\text{pred}(\langle -, x, - \rangle) = x$.

$$\text{Msg}_1 : \left[\begin{array}{l} \mathbf{message : sender s receiver r} \\ \mathbf{predecessor p^* action A \dots} \end{array} \right]_{\text{Msg}}^{\text{role}} := \begin{cases} \langle self, p^*, \llbracket A \rrbracket_{\text{Signal}} \rangle & \text{if } role = s \\ \langle mark(self), p^*, mark(\llbracket A \rrbracket_{\text{Signal}}) \rangle & \text{if } role = r \\ \lambda & \text{otherwise} \end{cases}$$

$$\text{Msg}_2 : \llbracket \text{Message Message}^* \rrbracket_{\text{Msg}} := \llbracket \text{Message} \rrbracket_{\text{Msg}} \cdot \llbracket \text{Message}^* \rrbracket_{\text{Msg}}$$

$$\text{Msg}_3 : \llbracket \lambda \rrbracket_{\text{Msg}} := \lambda$$

Note how we use the *mark*-function to mark the identifier of the message as well. This is to distinguish the sender and receiver of a message when connecting several threads. A run of a role in isolation can simply be determined by selecting the identifier of the role, and applying a simplified rule *Run*₄:

$$\text{Run}_5 : \llbracket \text{Interaction} \rrbracket_{\text{Run}}^{\text{role}} := \llbracket \text{Interaction} \rrbracket_{\text{Thread}}^{\text{role}}$$

Due to the definition of the *Thread*-operator, the initial marking of processes can be determined statically. It is not necessary to determine a terminal marking.

$$\text{Initial}_1 : \llbracket \text{Interaction} \rrbracket_{\text{Initial}} := \{p_r \mid r \in \llbracket \text{Interaction} \rrbracket_{\text{Roles}}\}$$

We have that $\langle \llbracket \text{Interaction} \rrbracket_{\text{Run}}^r, \llbracket \text{Interaction} \rrbracket_{\text{Initial}}^r \rangle \stackrel{x}{S} \geq \langle \llbracket \text{Interaction} \rrbracket_{\text{Run}}, \llbracket \text{Interaction} \rrbracket_{\text{Initial}} \rangle$ for all $r \in \llbracket \text{Interaction} \rrbracket_{\text{Roles}}$ and $S = \llbracket \text{Interaction.message} \rrbracket_{\text{ID}}$. This is obvious for $x = \mathcal{L}$. For $x = \mathcal{B}$, observe that there is no branching in linear nets as are created by the *Thread*-operator, and the proposition is true for $x = \mathcal{L}$. For $x = \mathcal{C}$, observe that the *sort*-function preserves the order ($\text{sort}(x)$ is a linearization of the partial order x), so that the partial word for the individual run is a weakening of the partial word for the collective run.

9.3.3. Examples

Figure 9.2 shows an example of how an interaction diagram (i.e. a single Interaction) can be mapped to a Petri-net process (see Figure 9.3 for the abstract syntax representation of the interaction diagram).

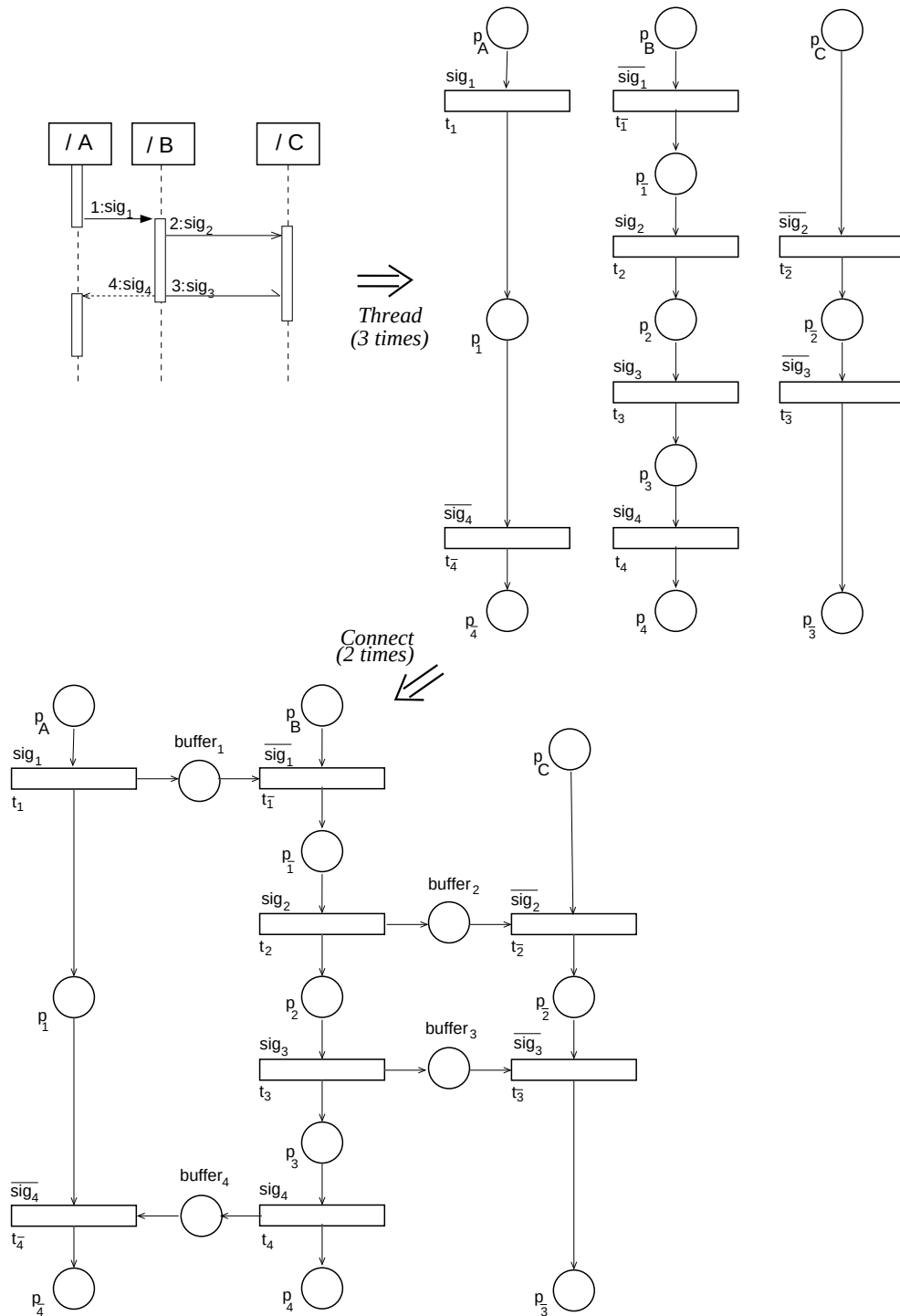


Figure 9.2.: Semantics of Interaction: a simple example including all four kinds of communications as well as object creation and deletion presented as a sequence diagram (top left); the Petri-net processes corresponding to individual runs (right middle); and the Petri-net process representing the whole sequence diagram, i.e. a system run (bottom left). We have added the identifiers of ModelElements and the labels of net-elements to improve readability.

```

collaboration:
  kind collaboration-kind
  interaction
    ia:
      message
        message:
          sender id4
          receiver id5
          predecessor  $\emptyset$ 
          action foo
        me:... e: id id1 e.
        message:
          sender id5
          receiver id6
          predecessor  $\{id_1\}$ 
          action create
        me:... e: id id2 e.
        message:
          sender id5
          receiver id6
          predecessor  $\{id_1, id_2\}$ 
          action delete
        me:... e: id id3 e.
        message:
          sender id5
          receiver id4
          predecessor  $\{id_1, id_2, id_3\}$ 
          action return
        me:... e: id id4 e.
      context  $\{id_4, id_5, id_6\}$ 
    me:... e: id id4 e.
ns: ownedElement
  :
me: name example e: id id1 e.

```

Figure 9.3.: Abstract syntax representation of the sequence diagram of Figure 9.2.

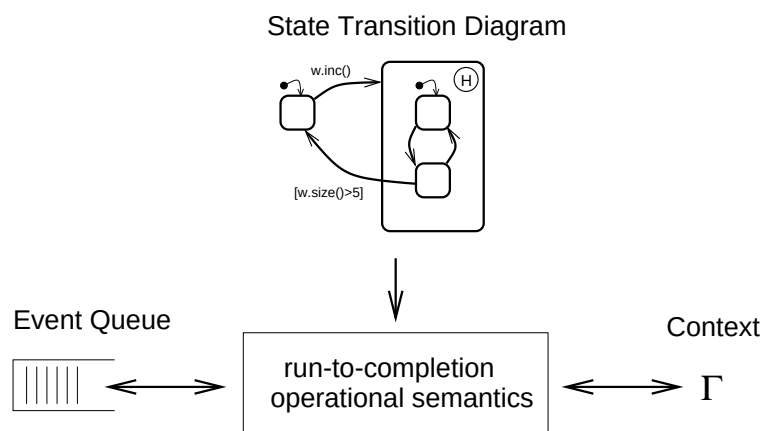
9.4. Creating a Petri-net from StateMachine

In the semantics of StateMachine, some leeway for interpretation is left on purpose: “[...] *there are a number of semantic variation points to allow for different semantic interpretations that might be required in different domains of application*” (cf. [OMG UML 1.3, p. 2-143]). So, a faithful formal semantics of StateMachine is necessarily loose. For an interactive simulation semantics, this is no problem: any detail left open by the specification can be filled in quite naturally by the user through the interaction that is necessary anyway. In fact, for a simulation semantics, all of the annotations can be discarded for the time being. This makes the semantics more loose than it necessarily needs to be, but it is a helpful starting point. Later on, this loose semantics can be strengthened, by adding components to interpret (some of) the inscriptions (see e.g. Hußmann et al. [2000], Richters and Gogolla [2000], Wittmann [2000] for a compiler and an interpreter for OCL, respectively).

We will first explain the semantics of StateMachine as far as it is determined by the UML, and then fill in the necessary detail, thus clearly separating what is implied by the UML, and what is added by us. In particular, the operational semantics for NameSpace can be used here.

9.4.1. Intuitive semantics of StateMachine

There are three elements involved in defining the operational semantics of StateMachine: the automaton proper, the context, and the event queue (see sketch). The standard describes these as “*a hypothetical machine that implements a state machine specification*” (cf. [OMG UML 1.3, p. 2-143]).



The automaton is described by a state transition diagram, where each Transition may be inscribed with an Event as the trigger, a Guard, and an Action as the effect. The UML does not fix an inscription language, though OCL can probably be considered canonical here.

The context may be any “ModelElement *capable of dynamic behavior, such as a classifier* [...]”, which provides the context for that StateMachine.” (cf. [OMG UML 1.3, p. 3-132]). It is not specified what effects executing the StateMachine has on the context.

The event queue stores Events that are generated and consumed by the StateMachine (and the BehavioralFeatures of the context). The dequeuing policy is deliberately under-specified: “*the order of dequeuing is not defined, leaving open the possibility of modeling different priority-based schemes*” (cf. [OMG UML 1.3, p. 2-149]). Events created by the previous step that are not consumed by the current step are discarded, unless they are declared as deferred. A related issue is discussed in Section 9.4.1.

Intuitively, executing a StateMachine amounts to finding a maximal set of non-conflicting enabled transitions (a step), executing it, and entering the resulting state. Evaluating the trigger, guard and effect of a transition may change the event queue and the context.

State configuration

The intuitive notion of the “state” of a StateMachine is captured by the notion of *state configuration*, i.e. “*a tree of States starting with the single top-State at the root down to individual SimpleStates at the leaves*” (cf. [OMG UML 1.3, p. 2-145]).

Regions

Note that by [OMG UML 1.3, p. 2-139, constraint 5], concurrent CompositeStates may only have sequential CompositeStates as their subvertexes.² They are called *regions* in the UML, and are “*usually associated with concurrent execution*” (cf. [OMG UML 1.3, p. 2-133]). In fact, no other interpretation is known, although there is considerable leeway as to the meaning of concurrency (see below), and the example of [OMG UML 1.3, Figure 2-23, p. 2-149] clearly contradicts this interpretation of regions. Regions have no counterpart in a state transition diagram (see also p. 2-145).

Compound transitions

The Transitions of the metamodel are just line segments of the arcs of a state transition diagram. Segments are joined together by StateVertexes other than State. A set of joined-together Transitions is called a *compound transition* in UML. This notion is instrumental to the definition of the semantics: “*the transition execution semantics [...] refer to compound transitions.*” (cf. [OMG UML 1.3, p. 2-147]). We have adapted the metamodel so that Transition can be used to represent compound transitions, too.

High-level transitions

Transitions connected to CompositeStates are called *high-level transitions* or *group transitions*.³ They are different from ordinary transitions as they may represent a whole family of transitions, possibly one for each sub-configuration of a composite state.

Step and transition firing

Intuitively, executing a transition (“firing it”, both in UML and Petri-net terminology) means going from one state configuration to another in a so called “run-to-completion-step” (see below). A Transition can be executed, when it is *enabled*, i.e. when (a) the Guard is evaluated to true in the current context, and (b) the event queue holds all triggers required for said Transition. The firing results in the effect being executed. This Action may have side-effects, in particular it may generate new Events that are then added to the event queue, and it may update the context. Several compound transitions may be executed in a single step (i.e. in lockstep parallel, but not concurrently), if they are not in conflict with each other, i.e. when the state configurations that the transitions of the step leave or enter are pairwise disjoint.

Run-to-completion step

UML features a so-called run-to-completion-step semantics, i.e. maximal sets of concurrently enabled transitions are executed together⁴, and their effects are not propagated until all of them have been finished. In particular, no new Events are generated afresh and enqueued until all of the transitions in a step have been completed, i.e. consumed their triggers and read their values from the context. So, UML StateMachines have a step-semantics, but no true concurrency semantics (see the wee digression of Section 8.2.4).

²We do not consider the pointless case of concurrent CompositeStates as subvertexes of a CompositeState.

³The standard mistakenly speaks of transitions *originating* from composite states, [OMG UML 1.3, p. 2-147].

⁴This amounts to the maximum firing rules in the Petri-net terminology. Note that this firing rule endows Petri-nets with Turing-power.

The run-to-completion-assumption seems to impose a (step-)interleaving semantics: “*the dynamic semantics [...] are based on the premise that a single run-to-completion-step applies to the entire state machine and includes the concurrent steps taken by concurrent regions in the active state configuration*” (cf. [OMG UML 1.3, p. 2-150]). So, the UML semantics of StateMachine defines away an important class of realistic phenomena, that occur *despite of the run-to-completion-assumption*: They are re-introduced by collaborating Classifiers that each may have their own StateMachine. The run-to-completion-assumption, however, is restricted to individual StateMachines. Making this assumption for whole systems, on the other hand, is obviously entirely unrealistic, and cannot possibly be implemented by all but the most trivial systems.

Event space

The semantics of StateMachine is defined with reference to a single, isolated StateMachine only. However, no object (or architectural unit) is an island, and so there are always *systems* of interacting StateMachines (see the previous digression). Just how these interact is a difficult question.⁵ For instance, if there were just a single event queue for all StateMachines, there could be side-effects transmitted via the event queue (similar to the context). When using StateMachines as the description of a system’s behavior (e.g. in the sense of state charts), this is clearly unwanted. So, we introduce the notion of *event space*⁶, which is a kind of namespace for events, i.e. a scope of event-emitting and -processing units that share an event queue. Note that if two items share an event space, they must also share a NameSpace, but the concepts are in principle orthogonal. By default, every StateMachine has its own distinct and encapsulated opaque event space. They may be connected explicitly by specific operations, but remain distinct save for the connection-points. As event spaces are an entirely semantical concept, no metamodel-classes need be introduced.

9.4.2. Loose semantics for StateMachine

Approach

The UML terminology concerning StateMachines is surprisingly reminiscent of the traditional Petri-net terminology, and the intuition behind many a concepts seems to be exactly that familiar from Petri-net theory, in particular the notions of firing, conflict and step. So it comes as no surprise, that there is a pretty straightforward mapping from StateMachines to Petri-nets, based on the following schema.

SimpleState, FinalState	↦ place
state configuration	↦ marking
compound transition	↦ transition

Two simple examples of state transition diagrams and the corresponding Petri-nets, according to this translation scheme, are shown in Figure 9.4 (more elaborated examples follow below). Note that we have inscribed the arcs only with identifiers for easier reference, but not with proper OCL-inscriptions.

A simulation semantics does not necessarily require the event queue and the context to be implemented: For the purpose of validation, a simulation must be carried out interactively anyway, i.e. with

⁵In fact, it is an open issue in the UML issues-database.

⁶There is no similarity with namespaces in the sense of Java Threads, and a faint conceptual similarity to Linda’s tuple-space or JiniSpaces.

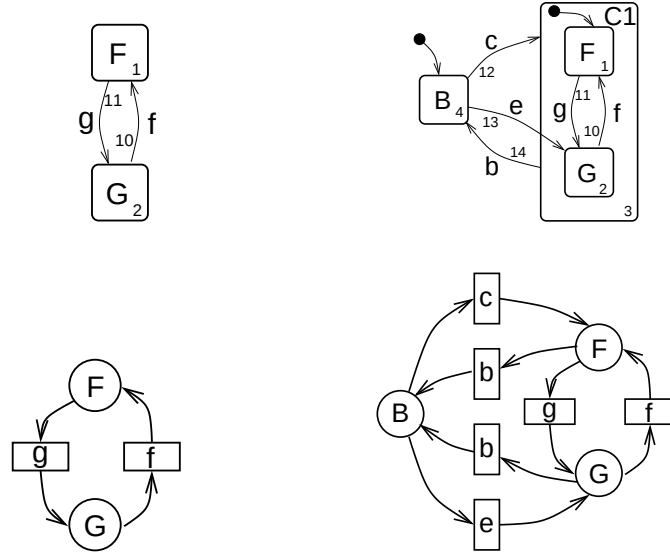


Figure 9.4.: Examples 1 and 2 for the translation of transitions: flat state transition diagram (top); state transition diagram with composite sequential state (bottom). The abstract syntax representation of example 2 is found in Figure 9.6.

the user deciding which steps are to occur when.⁷ So, we need not bother with the evaluation of Guards, the realization of side-effects of Actions, in particular the generation of Events, and the consumption of Events as triggers. Thus, low-level Petri-nets are sufficient as a domain for the simulation semantics. Of course, this approach does not work for all formal analysis methods.

Auxiliary functions

First we need a couple of simple auxiliary functions. The first one determines the value of the history-attribute for all states as a mapping from state-ids to a value from $\{0, 1, *\}$.

$$\begin{aligned}
 H_1 : \llbracket \text{synch} : \dots \rrbracket_H &:= 0 \\
 H_2 : \llbracket \text{simple} : \dots \rrbracket_H &:= 0 \\
 H_3 : \llbracket \text{final} : \dots \rrbracket_H &:= 0 \\
 H_4 : \left[\begin{array}{l} \text{composite} : \dots \text{history } h \dots \\ \text{subvertex } S^* \text{ SUPER} \end{array} \right]_H &:= \{ \llbracket \text{SUPER} \rrbracket_{ID} \mapsto h \} \cup \llbracket S^* \rrbracket_H
 \end{aligned}$$

When StateMachine (i.e. **sm: top** $S \dots$) is understood, we abbreviate $\llbracket S \rrbracket_H(s)$ as *history*(s).

Next, the function K determines the state-kind for all states as a mapping from state-ids to a value

⁷With all three elements of the semantics of

StateMachine present, i.e. in a less loose semantics, the choice of steps would be smaller, but no new possibilities would be added.

from $\{\mathbf{simple}, \mathbf{final}, \mathbf{conc}, \mathbf{seq}\}$.

$$\begin{aligned}
K_1 : \llbracket \mathbf{synch} : \dots \rrbracket_K &:= \emptyset \\
K_2 : \llbracket \mathbf{simple} : \dots \mathbf{SUPER} \rrbracket_K &:= \{\llbracket \mathbf{SUPER} \rrbracket_{ID} \mapsto \mathbf{simple}\} \\
K_3 : \llbracket \mathbf{final} : \mathbf{SUPER} \rrbracket_K &:= \{\llbracket \mathbf{SUPER} \rrbracket_{ID} \mapsto \mathbf{final}\} \\
K_4 : \left[\begin{array}{l} \mathbf{composite} : \dots \mathbf{concurrent} \\ \text{subvertex } S^* \mathbf{SUPER} \end{array} \right]_K &:= \{\llbracket \mathbf{SUPER} \rrbracket_{ID} \mapsto \mathbf{conc}\} \cup \llbracket S^* \rrbracket_K \\
K_5 : \left[\begin{array}{l} \mathbf{composite} : \dots \mathbf{sequential} \\ \text{subvertex } S^* \mathbf{SUPER} \end{array} \right]_K &:= \{\llbracket \mathbf{SUPER} \rrbracket_{ID} \mapsto \mathbf{seq}\} \cup \llbracket S^* \rrbracket_K
\end{aligned}$$

Similar to *history*, when StateMachine (i.e. **sm: top S...**) is understood, we abbreviate $\llbracket S \rrbracket_K(s)$ as *kind(s)*. Thirdly, the function I determines all the States that are initial-states as a set of state-ids.

$$\begin{aligned}
I_1 : \llbracket \mathbf{synch} : \dots \rrbracket_I &:= \emptyset \\
I_2 : \llbracket \mathbf{simple} : \mathbf{initial} \mathbf{SUPER} \rrbracket_I &:= \{\llbracket \mathbf{SUPER} \rrbracket_{ID}\} \\
I_3 : \llbracket \mathbf{simple} : \mathbf{SUPER} \rrbracket_I &:= \emptyset \\
I_4 : \llbracket \mathbf{final} : \dots \rrbracket_I &:= \emptyset \\
I_5 : \left[\begin{array}{l} \mathbf{composite} : \mathbf{initial} \dots \\ \text{subvertex } S^* \mathbf{SUPER} \end{array} \right]_I &:= \{\llbracket \mathbf{SUPER} \rrbracket_{ID}\} \cup \llbracket S^* \rrbracket_I \\
I_6 : \left[\begin{array}{l} \mathbf{composite} : \mathbf{history} \dots \\ \text{subvertex } S^* \mathbf{SUPER} \end{array} \right]_I &:= \llbracket S^* \rrbracket_I
\end{aligned}$$

As before, when StateMachine (i.e. **sm: top S...**) is understood, we abbreviate $\llbracket S \rrbracket_I(s)$ as *initial(s)*. Finally, the function Sub determines the sub-states for all states, i.e. the relation “is a subvertex of”. It is literally a relation over state-ids.

$$\begin{aligned}
\text{Sub}_1 : \llbracket \mathbf{synch} : \dots \rrbracket_{\text{Sub}} &:= \langle \emptyset, \emptyset \rangle \\
\text{Sub}_2 : \llbracket \mathbf{simple} : \dots \mathbf{SUPER} \rrbracket_{\text{Sub}} &:= \langle \{\llbracket \mathbf{SUPER} \rrbracket_{ID}\}, \emptyset \rangle \\
\text{Sub}_3 : \llbracket \mathbf{final} : \dots \mathbf{SUPER} \rrbracket_{\text{Sub}} &:= \langle \{\llbracket \mathbf{SUPER} \rrbracket_{ID}\}, \emptyset \rangle \\
\text{Sub}_4 : \left[\begin{array}{l} \mathbf{composite} : \dots \\ \text{subvertex } S^* \mathbf{SUPER} \end{array} \right]_{\text{Sub}} &:= \llbracket S^* \rrbracket_{\text{Sub}} \cup \{ \langle \llbracket \mathbf{SUPER} \rrbracket_{ID}, s \rangle \mid s \in \llbracket S^* \rrbracket_{ID} \}
\end{aligned}$$

Again, when StateMachine (i.e. **sm: top S...**) is understood, we abbreviate $\llbracket S \rrbracket_{\text{Sub}}$ as $<$. This relationship is irreflexive and transitive.

State configurations Frequently, we need to determine certain (sets of) state configurations. An individual state configuration is represented as a word of state-identifiers. Strictly speaking, a set would suffice, but a word is more intuitive when taking into account the tree-like structure of StateMachine. See also Kishinevsky et al. [1997]. We define the concatenation of sets of words (written $\otimes$) as $A \otimes A' := \{a.a' \mid a \in A \wedge a' \in A'\}$ and $\bigotimes_{\{A_1, \dots, A_n\}} = A_1 \times \dots \times A_n$.

All State Configurations The function *asc* computes all state configurations for a state with given identifier, i.e. $asc : id \rightarrow 2^{id^*}$. For FinalStates and SimpleStates, these are just the names of these States. For a sequential CompositeState this is the union of all state configurations of all of its subvertexes. For a concurrent CompositeState, this is the concatenation of all the state configurations of all its concurrent regions.

$$asc(s) := \begin{cases} \{s\} & \text{if } kind(s) \in \{\mathbf{simple}, \mathbf{final}\} \\ \bigcup_{s' < s} asc(s') & \text{if } kind(s) = \mathbf{seq} \\ \bigotimes_{s' < s} asc(s') & \text{if } kind(s) = \mathbf{conc} \end{cases}$$

Initial State Configurations The function *isc* computes the initial state configurations for a state with a given identifier, i.e. $isc : id \rightarrow 2^{id^*}$. Final states cannot have an initial state configuration by definition. Simple states have as their initial state configuration themselves, iff the respective attribute is set. The initial state configuration of a sequential CompositeState is the concatenation of the initial state configurations of all of its concurrent regions. Note that the order in the words representing state configurations is irrelevant – they represent bags.

$$isc(s) := \begin{cases} \{\lambda\} & \text{if } kind(s) = \mathbf{final} \\ \{s \mid initial(s)\} & \text{if } kind(s) = \mathbf{simple} \\ isc(\{s \mid s < s', initial(s')\}) & \text{if } kind(s) = \mathbf{seq} \\ \bigotimes_{s' < s} isc(s') & \text{if } kind(s) = \mathbf{conc} \end{cases}$$

Entering State Configurations The function *esc* computes the entering state configurations of a state with a given identifier *s*, i.e. all the state configurations of *s* that may be reached by Transitions entering *s*. Thus, its type is $esc : id \rightarrow 2^{id^*}$. Since this function also depends on the history attribute, the definition is quite complex. This complexity mainly stems from the shallow history.

For FinalStates and SimpleStates, these are just the names of these States, i.e. the same as *asc*. For a sequential CompositeState, the entering state configurations is the union of the entering state configurations of its initial subvertex (the union is just a technicality). For a concurrent CompositeState, the entering state configuration depends on the value of the history-attribute: if there is no history, the entering state configuration is the concatenation of the entering state configurations of all the subvertexes (similar to *asc*); for deep history, any state configuration can be an entering state configuration, i.e. the result is again the same as for *asc*. Finally, for shallow history, the set of entering state configurations is the concatenation of all the state configurations of all its concurrent regions.

$$esc(s) := \begin{cases} \{s\} & \text{if } kind(s) \in \{\mathbf{simple}, \mathbf{final}\} \\ \bigcup_{s' < s, initial(s')} esc(s') & \text{if } kind(s) = \mathbf{seq} \\ \bigotimes_{s' < s} esc(s') & \text{if } kind(s) = \mathbf{conc}, history(s) = 0 \\ \bigotimes_{s' < s} \bigcup_{s'' < s'} esc(s'') & \text{if } kind(s) = \mathbf{conc}, history(s) = 1 \\ asc(s) & \text{if } kind(s) = \mathbf{conc}, history(s) = * \end{cases}$$

We assume all of these auxiliary functions to be always present.

StateMachine and State

With these preliminaries, we can now proceed to the definition of the semantics of StateMachine. We first consider only basic StateMachines and leave out SynchStates, StubStates, PseudoStates, SubmachineState and Guards. For the time being, we also disregard the concrete values of history attributes (we do consider *all* possible values, though).

A StateMachine is translated into an inscribed low-level petri-net as follows. First, all simple and final states become places, and each Transition is turned into a set of transitions, according to the state configurations the Transition connects. Every transition is then inscribed with the denotation of the Signal of the trigger and effect of the corresponding Transition.

$$SM_1 : \llbracket \mathbf{sm} : \mathbf{top} \ S \ \mathbf{transition} \ Transition^* \dots \rrbracket_{SM} := \langle P, T; F; \beta \rangle$$

where

$$P = \{p \in dom(\llbracket S \rrbracket_{TYPE}) \mid \llbracket S \rrbracket_{TYPE}(p) \in \{\mathbf{simple}, \mathbf{final}\}\}$$

$$T = \llbracket Transition^* \rrbracket_T$$

$$F = \{ \langle p, t_{from.id.to} \rangle \mid t_{from.id.to} \in T, p \in P, p \in from \} \\ \cup \{ \langle t_{from.id.to}, p \rangle \mid t_{from.id.to} \in T, p \in P, p \in to \}$$

$$\beta = \llbracket Transition^* \rrbracket_{TE}$$

The sending of Signals within a StateMachine can not be modeled in the loose semantics, because it depends on the context and the event queue, which other Transition is to be triggered by a SendAction.

Transition

As we have said before, the compound transitions of UML can be represented directly as Petri-net transitions. In a first step, the transitions are determined and the corresponding appropriate state configurations are computed. Consider the example in Figure 9.5 that illustrates the difficulties of this step.

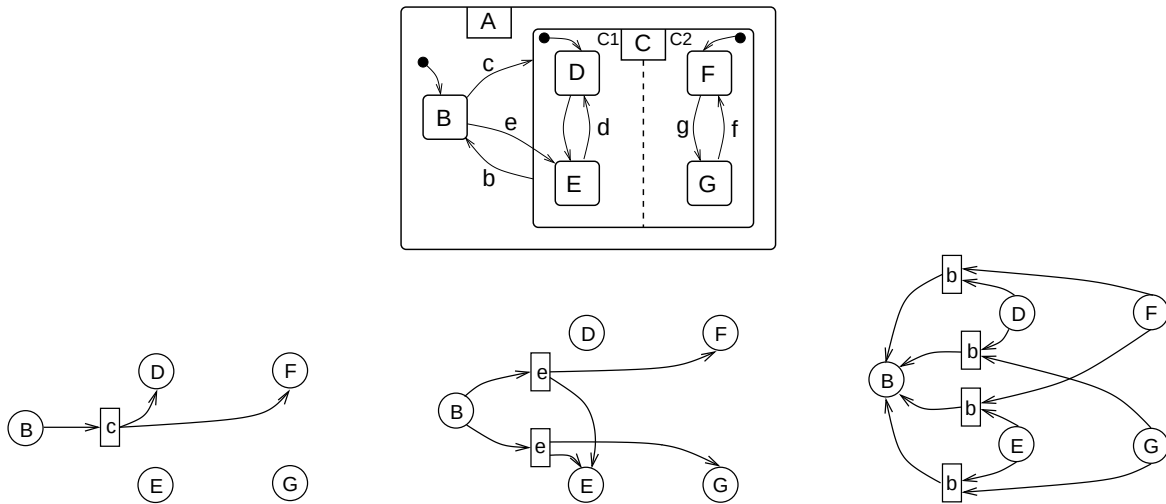


Figure 9.5.: Example 3 for the translation of transitions: state transition diagram with composite concurrent state. The translations for the high-level transitions c, e, and b (left top, middle, bottom) have been split up to increase overview. The abstract syntax representation for this example is shown in Figure 9.7.

The two concurrent subvertex-regions of C are included in this diagram by their names C1 and C2. We have given names to the transitions for easier reference, but not triggers etc., as these are not of interest in this context. Suppose that all Transitions are always enabled, and that there are no conflicts.

We have listed all the reachable state configurations and the transitions between them. Obviously, the transitions c and b link several pairs of state configurations. So, we sometimes need a set of Petri-net transitions to represent a single compound Transition.

The set of all transitions is determined by the cross-product of all the state configurations a Transition may exit, and all the state configurations it may enter.

$$T_1 : \llbracket \text{transition} : \text{source } id_1 \text{ target } id_2 \dots \rrbracket_T := \left\{ t_{from.id.to} \left| \begin{array}{l} from \in asc(id_1), \\ id = \llbracket \text{SUPER} \rrbracket_{ID}, \\ to \in esc(id_2) \end{array} \right. \right\}$$

The arc inscriptions (trigger and effect) are extracted as a sequence of symbols representing the respective Signals. The attributes trigger and effect are optional: if missing, they are treated as mapping to the empty sequence λ . To distinguish between the two, triggers are marked.

$$TE_1 : \llbracket \text{transition} : \dots \text{trigger EV} \dots \rrbracket_{TE} := \left\{ t \mapsto mark(\llbracket EV \rrbracket_{Signal}) \cdot \llbracket A \rrbracket_{Signal} \mid \text{where } t \in \llbracket \text{Transition} \rrbracket_T \right\}$$

$$TE_2 : \llbracket \text{transition} : \dots \text{effect A} \dots \rrbracket_{TE} := \{ t \mapsto \llbracket A \rrbracket_{Signal} \mid t \in \llbracket \text{Transition} \rrbracket_T \}$$

$$TE_3 : \llbracket \text{transition} : \dots \text{trigger EV} \rrbracket_{TE} := \{ t \mapsto mark(\llbracket EV \rrbracket_{Signal}) \mid t \in \llbracket \text{Transition} \rrbracket_T \}$$

$$TE_4 : \llbracket \text{transition} : \dots \rrbracket_{TE} := \{ t \mapsto \lambda \mid t \in \llbracket \text{Transition} \rrbracket_T \}$$

In a loose semantics, the S emanating from the mappings of Actions and Events are not interpreted, so that e.g. a literal representation is just fine. A stricter semantics could abstract the context into a signature Σ , and provide interpretations of Guard, Action and Event as terms over an appropriate signature.

The sets of incoming and outgoing signals of a StateMachine are needed to determine the argument S for implementation and equality-relations. They can be determined by the following rules.

$$In_1 : \llbracket \text{transition} : \dots \text{trigger EV} \dots \rrbracket_{In} := mark(\llbracket EV \rrbracket_{Signal})$$

$$In_2 : \llbracket \text{sm} : \dots \text{transition Transition}^* \dots \rrbracket_{In} := \llbracket \text{Transition}^* \rrbracket_{In}$$

$$Out_1 : \llbracket \text{transition} : \dots \text{effect A} \dots \rrbracket_{Out} := \llbracket A \rrbracket_{Signal}$$

$$Out_2 : \llbracket \text{sm} : \dots \text{transition Transition}^* \dots \rrbracket_{Out} := \llbracket \text{Transition}^* \rrbracket_{Out}$$

9.4.3. Examples

A simple transition, such as the one presented in Figure 9.4 (left) maps onto a single Petri-net transition, irrespective of whether the source or target of the Transition are CompositeStates or not.

Figure 9.4 (right) shows two similar cases: e is a simple transition again, irrespective of the fact that it crosses a state boundary. Transition c enters a CompositeState, but it is also mapped to a single transition, as there is only one state configuration it can enter, namely the initial state configuration. Transition b, on the other hand, can exit from all state configurations of C1, i.e. it has to be mapped onto as many transitions.

Figure 9.5 shows a different case. Transition c again enters the unique initial state configuration of C, but being composed of two regions, C1 and C2, there are two places in this state configuration, D and F. Thus, transition c has two outgoing arcs. Transition e is also mapped onto a single state

configuration of C, namely the one that contains E and the initial state (configuration) of C2. Were there a history-attribute for C, then e would have to be mapped to {E, F} and {E, G}. Transition b, on the other hand, is now mapped onto four transitions, one for each sub-configuration of C.

The respective abstract syntax representations of the StateMachines of these examples are given in Figures 9.6 and 9.7.

```

sm:
  top
    composite
      initial history 0 sequential
      subvertex
        simple:
          initial
          state:
            deferrable 0 entry ... exit ... do_Activity ...
          sv: me: name F e: id id1 e.
        simple:
          state:
            deferrable 0 entry ... exit ... do_Activity ...
          sv: me: name G e: id id2 e.
      state:
        deferrable 0 entry ... exit ... do_Activity ...
      sv: me: name C1 e: id id3 e.
    transition
      transition:
        source id2 target id1
        trigger ... guard ... effect ...
      me: name f e: id id10 e.
      transition:
        source id1 target id2
        trigger ... guard ... effect ...
      me: name g e: id id11 e.
  me: name state machine example 1 e: id id0 e.

```

Figure 9.6.: Example: abstract syntax representation of the StateMachine of the second example for StateMachine (Figure 9.4, top right).

9.4.4. Strict semantics

So far, the semantics we have presented is a loose semantics, in that it disrespects certain features of StateMachine. Consider Figure 9.8. It shows the elements of the StateMachine-semantics in UML. The nets we have created so far (that is, with the loose semantics) cared only for the state transition diagram, but left out the context, the event queue, and the RTC-steps. Thus, the behaviors of the nets we have created up to now include all admissible behaviors specified by the respective StateMachine, but also allow for others. We will add the other aspects of StateMachine one by one.

Run-to-completion semantics

Executing a (compound) transition is done in three steps (collectively referred to as a run-to-completion step in the UML). First, a “*maximal conflict-free*” set of transitions is selected: “*the set of*

```

sm:
  top
    composite
      initial history 0 sequential
      subvertex
        simple:
          initial
          state:
            deferrable 0 entry ... exit ... do_Activity ...
          sv: me: name B e: id id4 e.
          composite:
            :
            see Figure 9.6, ModelElements id1, id2
            :
          sv: me: name C1 e: id id3 e.
        state:
          deferrable 0 entry ... exit ... do_Activity ...
          sv: me: name A1 e: id id5 e.
      transition
        transition:
          source id4 target id3
          trigger ... guard. ... effect ...
          me: name c e: id id12 e.
        transition:
          source id4 target id2
          trigger ... guard. ... effect ...
          me: name e e: id id13 e.
        transition:
          source id3 target id4
          trigger ... guard. ... effect ...
          me: name b e: id id14 e.
        :
        see Figure 9.6, ModelElements id10, id11
        :
    me: name state machine example 2 e: id id0 e.

```

Figure 9.7.: Example: abstract syntax representation of the StateMachine in Figure 9.5. For clarity, we omitted those states and transitions that have been shown in Figure 9.6 above already.

transitions that will fire is a maximal set” (cf. [OMG UML 1.3, p. 2-151])). All the Events that are needed as triggers for the Transitions in this set have to be in the event queue, but are they *not* consumed by triggering.

Then, in the second phase, all Events in the queue are discarded, unless they are declared as deferred in one of the States of the current state configuration. This behavior can be realized either by

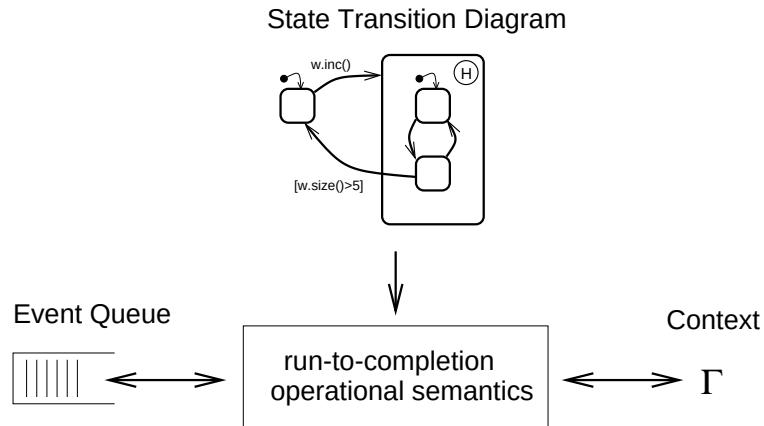


Figure 9.8.: The structure of the StateMachine semantics.

flush-arcs, or by firing priorities, thus implying non-low level Petri-nets. However, the requirement of maximal steps already demanded the Petri-net maximum firing rule, and so, full Turing-power.

In the third phase, the step is actually executed, that is, the effects of all its transitions are executed. Executing an Action may result in side effects: on the one hand, variables in the context may be updated. On the other hand, new Events may be generated and enqueued into the event queue. A cycle of these three phases is called a *RTC-step*.

As this part of the strict semantics is always the same for all StateMachines (save a parameter for the event queue, see below), we do not provide semantic rules, but just a sample implementation, see Figure 9.9. Non-deferred Events simply remain in the event queue as they are.

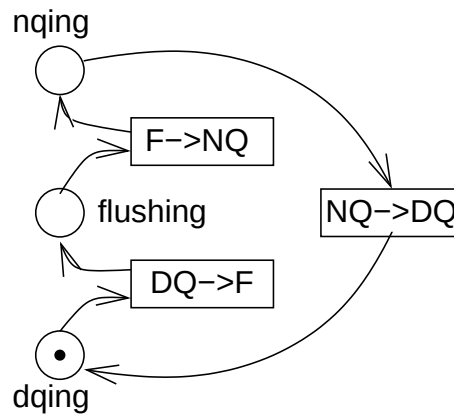


Figure 9.9.: A Petri-net model for an abstract machine implementing the RTC-step.

Adding the event queue

The general approach is to create separately realizations of the event queue and the StateMachine proper as nets, and then combine them by synchronizing transitions that represent occurrences of the same Signal. We introduce a function `SMstrict` for this purpose.

$$\text{SMstrict} : \llbracket \text{sm} : \text{top S transition Transition}^* \dots \rrbracket_{\text{SMstrict}}^n := \text{Synch} \left(\begin{array}{c} \llbracket \text{StateMachine} \rrbracket_{\text{SMs}} \cup \llbracket \text{StateMachine} \rrbracket_{\text{EQ}}^n, \\ \llbracket \text{StateMachine} \rrbracket_{\text{In}} \cap \llbracket \text{StateMachine} \rrbracket_{\text{SM}}^{\text{Out}} \end{array} \right)$$

Not that the synchronization is done with the help of all the Signals that may occur in the StateMachine syntactically. Some technical changes become necessary in the functions SM etc., so we introduce strict variants of these, i.e. for the SM-function we introduce the function SMs, for the T-function we introduce a function Ts and so on.

With a few exceptions, current formal methods can deal only with finite state spaces automatically. So, it is important to create only finite semantical models—and the keen reader will have noticed the parameter n to the EQ-function. It serves to restrict the length of the event queue, i.e. the number of Events that may be stored. This restriction does not restrict the generality of our model, however, for any state transition diagram is finite, and thus can only contain finitely many Transitions, and ActionSequences are finite by definition anyway. So, any RTC step may only trigger or consume finitely many Events. The parameter, then, can be chosen large enough, and is there only for convenience.

On the other hand, there might also be infinitely many *kinds* of Event to be stored in the event queue. The standard, however, does not specify parameters for Signals, SignalEvents, or SendActions. So, again, finite state transition diagrams may contain only finitely many different kinds of Event. Apart from this, we trust that almost all real life design models can be dealt with by finite semantical models (read: state spaces), anyway.

Figure 9.10 shows Petri-net models for different cases of event queues, as created by the EQ-function. Consider first the net on the top left. For convenience, we use a *high-level Petri-net* here. The net represents an event queue with two kinds of Event, namely a and b. Unfolding the high-level net to a low-level net yields the net on the right of Figure 9.10. Here, we have assumed that the parameter n for the maximum length of the queue is 2, so there must be $\sum_{i=0}^n c^i = \frac{1-c^{n+1}}{1-c}$ places in the low-level net, where c denotes the number of Events, i.e. $|\{a, b\}|$ (see [Graham et al., 1989, (2.26)] for the closed form).

The standard leaves the implementation of the event queue open deliberately: “*the order of dequeuing is not defined, leaving open the possibility of modeling different priority-based schemes*” (cf. [OMG UML 1.3, p. 2-149]). In order to capture this abstract specification, we realize no queuing policy at all, and all queued Events may be dequeued again at any time. The following function EQ creates an event queue of length n .

$$\text{EQ} : \llbracket \text{SM} \rrbracket_{\text{EQ}}^n := \langle P, T, F; \beta \rangle,$$

where

$$\begin{aligned} P &= \{p_q \mid q \in Q\} \\ T &= \{nq_{e,q} \mid e \in \mathcal{E}, p_q \in P, |q| < n\} \cup \{dq_{e,q} \mid e \in \mathcal{E}, p_q \in P, q \neq \lambda\} \\ F &= \{\langle nq_{e,q}, p_{e,q} \rangle, \langle p_q, nq_{e,q} \rangle \mid e \in \mathcal{E}, q \in Q, |q| < n\} \\ &\quad \cup \{\langle dq_{e,q,e}, p_q \rangle, \langle p_{q,e}, dq_{e,q,e} \rangle \mid e \in \mathcal{E}, q \in Q, q \neq \lambda\} \\ \beta &= \{dq_{e,q} \mapsto dq(e), nq_{e,q} \mapsto nq(e) \mid nq_{e,q}, dq_{e,q} \in T\} \end{aligned}$$

with the abbreviations $\mathcal{E} = \llbracket \text{SM} \rrbracket_{\text{In}} \cap \llbracket \text{SM} \rrbracket_{\text{Out}}$ and $Q = \bigcup_{i=0}^n \mathcal{E}^i$ to denote the set of all Events that occur syntactically in a StateMachine, and the set of all queue-states, respectively.

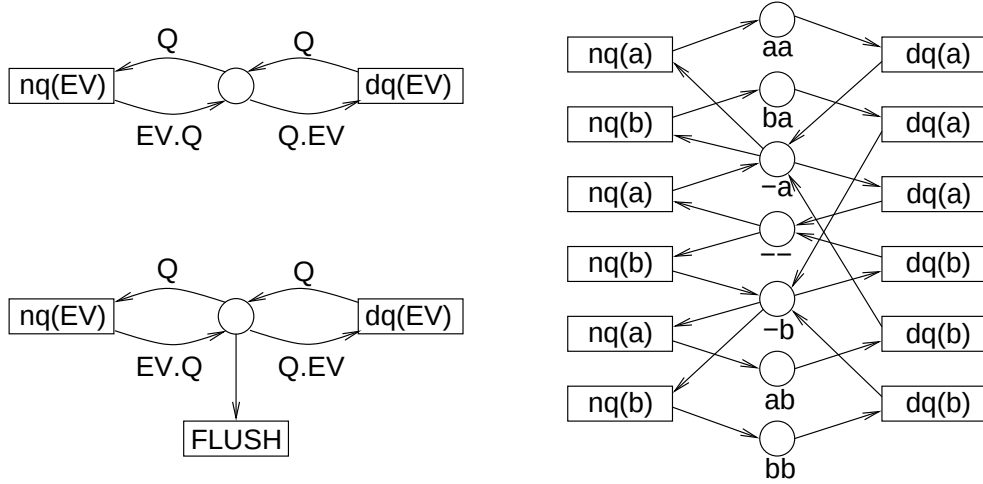


Figure 9.10.: The event queue as high-level Petri-nets: all Events are deferred (left top); no Events are deferred (left bottom); the latter net expanded to a LLPN (right). The notation Q, EV, and EV.Q refers to: the event queue data structure, some event, and the event prepended to the queue (i.e. enqueued), respectively. Similarly, Q.EV denotes the event EV enqueued to Q.

The RTC-semantics basically splits up every RTC-step into two phases for the consumption and creation of Events, respectively. This must be reflected in the model of the StateMachine proper: for the loose semantics, each Transition was translated straight into a transition. For a strict semantics, each Transition must be translated into one or two transitions, depending on whether or not both trigger and effect are specified for the transition. See the SMs-function.

$$\text{SMs} : \llbracket \text{sm} : \text{top } S \text{ transition Transition}^* \dots \rrbracket_{\text{SMs}} := \langle P, T; F; \beta \rangle$$

where

$$\begin{aligned} P &= \{p \in \text{dom}(\llbracket S \rrbracket_{\text{TYPE}}) \mid \llbracket S \rrbracket_{\text{TYPE}}(p) \in \{\text{simple}, \text{final}\}\} \\ &\quad \cup \{p_t \mid t \in \llbracket \text{Transition} \rrbracket_{T_s} \cup \llbracket \text{Transition} \rrbracket_{T_{s'}}\} \\ T &= \llbracket \text{Transition}^* \rrbracket_{T_s} \cup \llbracket \text{Transition}^* \rrbracket_{T_{s'}} \\ F &= \{\langle p, t_{\text{from.id.to}} \rangle \mid t_{\text{from.id.to}} \in T, p \in P, p \in \text{from}\} \cup \{\langle p_t, t \rangle \mid t \in \llbracket \text{Transition}^* \rrbracket_{T_{s'}}\} \\ &\quad \cup \{\langle t_{\text{from.id.to}}, p \rangle \mid t_{\text{from.id.to}} \in T, p \in P, p \in \text{to}\} \cup \{\langle t, p_t \rangle \mid t \in \llbracket \text{Transition}^* \rrbracket_{T_s}\} \\ \beta &= \llbracket \text{Transition}^* \rrbracket_{T_{Es}} \end{aligned}$$

The interesting difference to the SM-function is that Transitions are now translated into pairs of transitions with an intermediate place. In the places (the arcs and inscriptions are just adapted to this change). The first half is again just like for the original SM-function. The second half creates an additional place for each transition. The parameter i specifies the length of the queue.

$$\begin{aligned} T_s : \llbracket \begin{array}{l} \text{transition : source } id_1 \\ \text{target } id_2 \dots \end{array} \rrbracket_{T_s} &:= \left\{ t_{\text{from.id}} \mid \begin{array}{l} \text{from} \in \text{asc}(id_1), \\ id = \llbracket \text{SUPER} \rrbracket_{ID} \end{array} \right\} \\ T_{s'} : \llbracket \begin{array}{l} \text{transition : source } id_1 \\ \text{target } id_2 \dots \end{array} \rrbracket_{T_{s'}} &:= \left\{ t_{id.to} \mid \begin{array}{l} id = \llbracket \text{SUPER} \rrbracket_{ID}, \\ to \in \text{esc}(id_2) \end{array} \right\} \end{aligned}$$

We give a clause for TEs only for the one case, where both trigger and effect are specified. The other cases are analogous to the Rules TE₂ to TE₄ and are omitted here.

$$\begin{aligned} \text{TEs : } \llbracket \text{transition} : \dots \text{trigger EV} \dots \text{effect A} \dots \rrbracket_{\text{TEs}} &:= \\ \{t_1 \mapsto \text{mark}(\llbracket \text{EV} \rrbracket_{\text{Signal}}), t_2 \mapsto \llbracket \text{A} \rrbracket_{\text{Signal}} \mid t_1 \in \llbracket \text{Transition} \rrbracket_{\text{Ts}}, t_2 \in \llbracket \text{Transition} \rrbracket_{\text{Ts}'}\} \end{aligned}$$

In the strict semantics so far, we have treated all Events as deferred, that is, they remain in the queue until they are consumed. Of course, the opposite behavior, the plain behavior according to the standard, can also be realized as a Petri-net: by representing the two phases of the RTC-step explicitly, the queue can be flushed on termination of the second phase, see Figure 9.10 (bottom left). In order to realize priority-based event-queues, a priority-based Petri-net semantics has to be assumed.

Adding Context

We turn now to the context of a StateMachine. The overall approach is quite similar to that of adding an event queue: a separate model is created and synchronized with the model for the StateMachine proper. Figure 9.11 gives an idea of how Petri-net models of a context might look like.

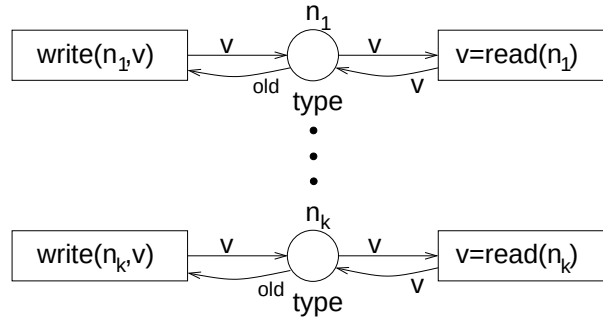


Figure 9.11.: A HLPN implementing a NameSpace as the context of a StateMachine.

Obviously, a direct translation to low-level Petri-nets is rather difficult. So, one might use high-level Petri-nets. They can be seen as enhancing low-level nets by types for tokens, guards for transitions and variable inscriptions for arcs. A synopsis of the most important kinds of high-level nets can be found in Jensen and Rozenberg [1991].

Semantically, the context of a StateMachine is a NameSpaces, but if we want to stick with the basic Petri-net model we already have, the context must be represented as a Petri-net, too. One possibility is an operational translation of NameSpaces according to the following scheme. Variable with names $n_1, \dots, n_k$ are represented as k places of the right type. Looking up and updating values are mapped onto transitions that read from and write to that place, respectively (see Figure 9.11). Connecting context to the other models is done in a similar way to adding the event queue.

There are some dialects of High-level nets that allow extensive textual annotations. For instance, colored nets as used in the *Design/CPN*-tool may have a so called “global declaration node”. This may be used to model the context in a simpler way.

Putting it all together

Now that we have translated all the elements of a strict interpretation of a StateMachine individually, we can join them together. The order is arbitrary. Consider first the event queue and the diagram: here,

the places representing the phases of the abstract machine become run places for all the transitions of the respective phases, that is, the RTC's place flush becomes the run place for all the event queue's flush-transitions and so on. A run place is traditionally considered as an exclusive resource for some transition, but we may use read arcs here, opening the way for a concurrent interpretation of the RTC-semantic, that goes beyond the artificial limitation found in the standard: *"It is possible to define state machine semantics by allowing the run-to-completion steps to be applied concurrently [...] However, such semantics are quite subtle and difficult to implement."* (cf. [OMG UML 1.3, p. 2-150])

Next, the diagram can be added by synchronizing its transitions with those of the event queue. Here, like and like are simply merged. Finally, the context can be added in in a similar fashion.

One has to take care about one rather subtle point: the transitions switching from one phase of the RTC-step to another have to have a lower priority than all other transitions. Otherwise, a phase may be skipped, corrupting the event queue. Unfortunately, this feature is known to transport low-level nets to full Turing-power, thus seriously impeding the analyzability of such models. One might see this as an argument against the way we model the strict semantics here. However, the UML standard already demands RTC-steps to be maximal (see above), so that we also have to employ the maximal firing rule already—which is also known as pushing low-level nets to Turing-completeness. So, any problems with computability arise from the standard alone, anyway, and nothing is lost by the peculiarities of our way of modeling.

Note that this is not (yet) a fairness issue.

9.4.5. Example

In this section, we present an example of a StateMachine interpreted with the strict semantics. In order to keep this example reasonably simple, we have to resort to using an extremely simple state transition diagram, see Figure 9.12, top right. Here, there are two concurrent regions exchanging a Signal denoted as *s*. The intuitive justification for this example is given in Sections 10.2.5 and 12.2.3.

A strict interpretation of this state transition diagram also has to take into account the event queue and the abstract machine implementing the RTC-step, see Figure 9.12, top.⁸ Their individual semantics is given in Figure 9.12, bottom.

In the example, there is only a single kind of Event that affects the queue, namely the SignalEvent with the signal denoted as *s*. Thus, there are two transitions to enqueue and dequeue *s*. Note that dequeuing is not consumptive. There is also a pair of transitions to enqueue and dequeue the silent action τ which is used for Transitions with no trigger and/or effect. Finally, the event queue has a transition for removing non-deferred Events. The abstract machine has three cycles for enqueueing, dequeuing and flushing the queue.

The three elements of the strict interpretation of StateMachine have to be put together by the procedure sketched in the previous section: the places of the RTC-implementation are used as run places, transitions denoting the same Signals in the event queue and the diagram are synchronized. The resulting net is shown in Figure 9.12, bottom. For clarity, the synchronization is shown by connecting the respective transitions rather than by actually merging them.

Figure 9.13 now shows a process of this net, that is, the result of letting it run. In this example, this is also the only terminal concurrent run. Again, we present the synchronization by connecting the respective transitions with a dashed line.

⁸For simplicity, we omit the context, which is not used in this example. In a concrete implementation, one might use a tool like Design/CPN to execute or analyze such models. There, the context should be modeled as a so called CPN-region, that is, it should be encoded directly.

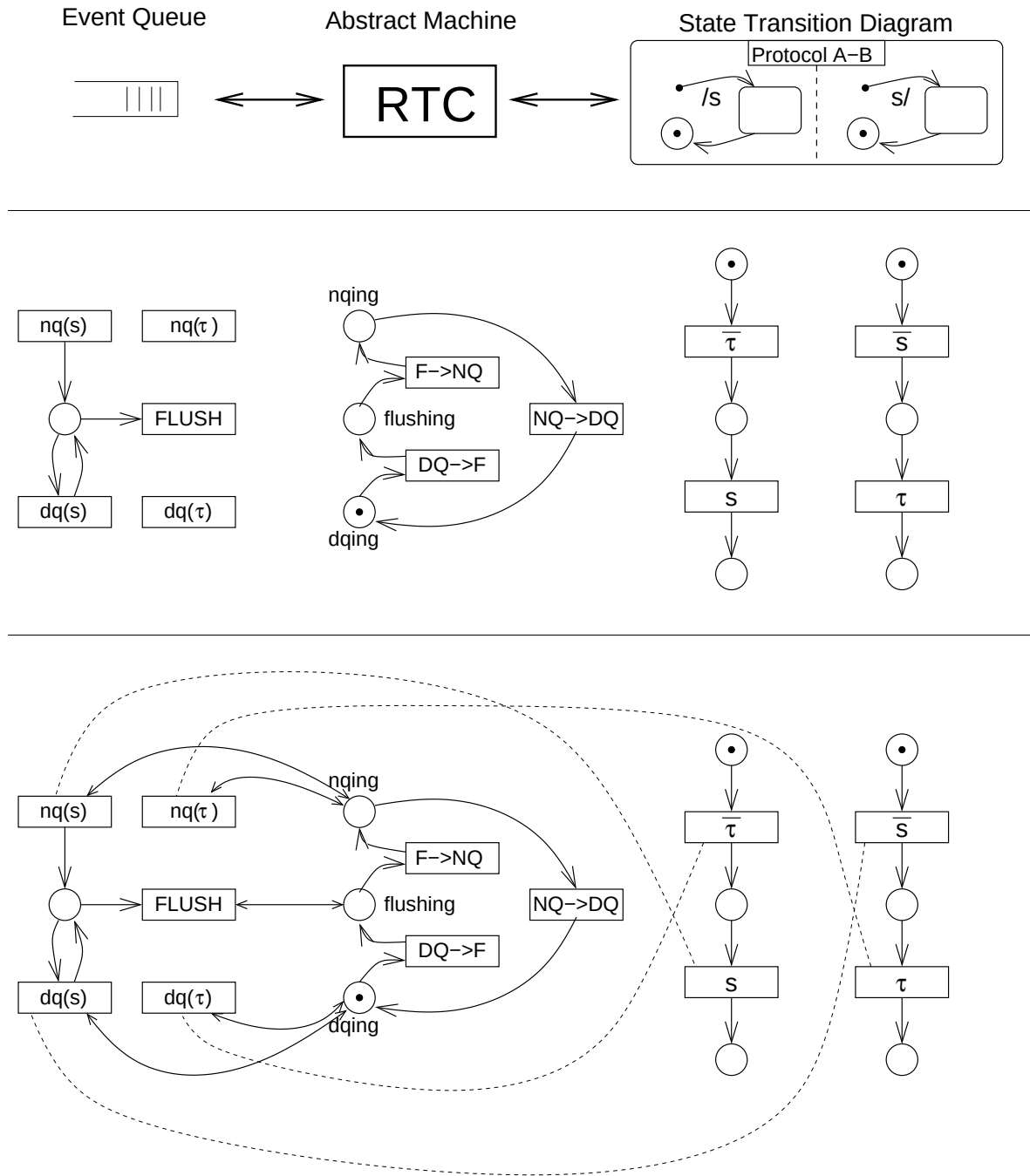


Figure 9.12.: Strict interpretation of StateMachine: the event queue, the abstract machine implementing the RTC-step, and the state machine proper (top, left to right). Below there are the respective semantical structures. The bottom line shows the individual elements connected. A sample run of this ensemble is represented in Figure 9.13.

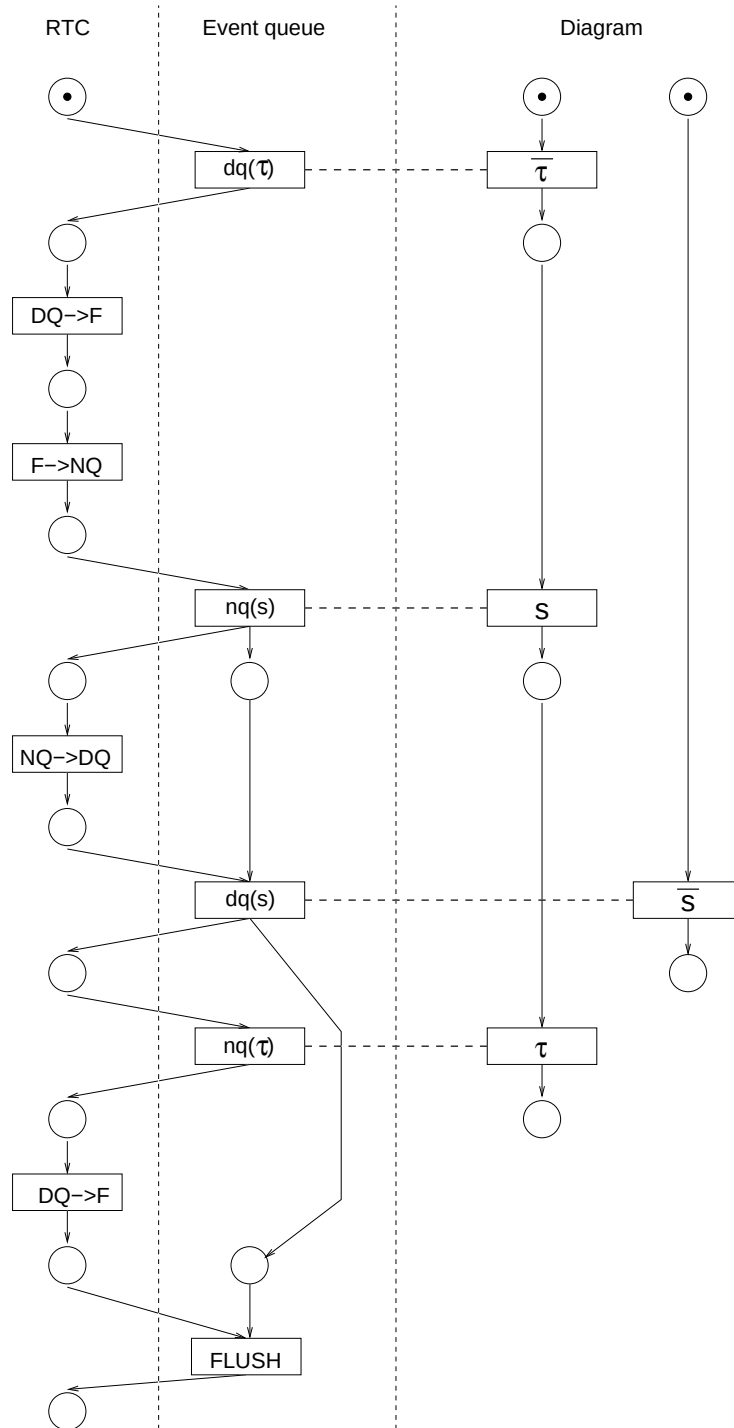


Figure 9.13.: A run of the StateMachine of Figure 9.12 with both the event queue and the RTC abstract machine, but not the context. The behaviors contributed by the abstract machine, the event queue and the diagram as such are shown in swimlane-like regions.

In a tool like Design/CPN, one would probably model the elements of a strictly interpreted StateMachine exactly as we have done here, using "fusion places". The context would probably be coded directly as a textual annotation (see above).

9.4.6. Initial and terminal states

In the UML, configurations of FinalStates and States whose initial-attribute is true constitute initial and terminal state configurations. They correspond to initial and terminal markings (i.e. sets of places) in the Petri-net terminology. The function *isc* can be used to determine the initial state configuration. The set of terminal state configurations is determined as the set of FinalStates that are direct substates of top.

$$\begin{aligned} \text{Initial}_2 : \quad & \llbracket \text{sm : top } S \dots \rrbracket_{\text{Initial}} &:= \text{isc}(S) \\ \text{Terminal}_2 : \quad & \llbracket \begin{array}{l} \text{sm : top composite : } \dots \\ \text{subvertex } S^* \dots \end{array} \rrbracket_{\text{Terminal}} &:= \{s \mid \text{kind}(s) = \mathbf{final}\} \end{aligned}$$

Recall that *kind*(*s*) is an abbreviation for $\llbracket S \rrbracket_K(s)$. The rule System is used when creating a namespace (graph) from a Capsule.

Further features

History can be simulated by simply storing the state configuration of a CompoundState as a value of the context. Since every Classifier has at most one StateMachine, we simply define a special variable HISTORY of type string, that is part of all contexts. Storing and retrieving the state is then realized as storing and retrieving any other arbitrary value.

Activities (i.e. entry, exit, and doActivity) can be considered by an additional translation step that refinements places representing States by nets with appropriate structure. See fig:DoEntryExit:s sketch for a first sketch.

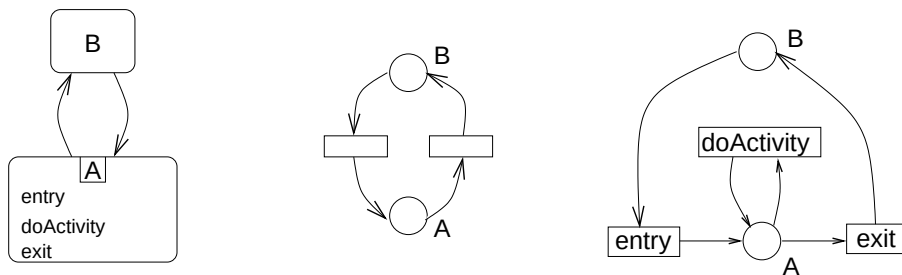


Figure 9.14.: A first idea of how to treat entry-, exit-, and doActivities.

Time, priority, and probability of, say, Transitions need additional inscriptions such as the characteristics attribute we have introduced. The standard does not say very much on this issue. Semantically, such conditions would result in additional annotations of Petri-nets, in particular on the nets representing the event queue. We make some suggestions how this can be done in Section 9.6.3.

9.5. Semantics of Capsule and Style

A Capsule serves different purposes, and so there are several semantical mappings selecting and assembling the parts needed.

9.5.1. Auxiliary functions

We first define an auxiliary function “Ids” to extract all identifiers of ModelElements of a given type from a particular view of a Capsule. The parameters τ and $view$ stand for a kind of ModelElement and a view, respectively. They are optional: if omitted, all types and views are selected.

$$\begin{aligned}
 Ids_1 : \llbracket \mathbf{capsule} : \dots \mathbf{view} \text{ View}^* \dots \rrbracket_{Ids}^{view} &:= \llbracket \text{View}^* \rrbracket_{Ids} \\
 Ids_2 : \llbracket \mathbf{view} : \mathbf{me} \text{ id}^* \mathbf{kind} \text{ v_kind} \dots \rrbracket_{Ids}^{view} &:= \begin{cases} id^* & \text{if } view = v_kind \\ \emptyset & \text{otherwise} \end{cases} \\
 Ids_3 : \llbracket \text{Capsule} \rrbracket_{Ids}^{\tau} &:= \llbracket \text{SUPER} \rrbracket_{Ids}^{\tau} \\
 Ids_4 : \llbracket \mathbf{package} : \mathbf{elementImport} \text{ ElementImport}^* \text{ Namespace} \rrbracket_{Ids}^{\tau} &:= \llbracket \text{ElementImport} \rrbracket_{Ids}^{\tau} \cup \llbracket \text{Namespace} \rrbracket_{Ids}^{\tau} \\
 Ids_5 : \llbracket \mathbf{import} \text{ ME} \dots \rrbracket_{Ids}^{\tau} &:= \begin{cases} \{\llbracket \text{ME} \rrbracket_{ID}\} & \text{if } \tau = \llbracket \text{ME} \rrbracket_{TYPE} \\ \emptyset & \text{otherwise} \end{cases} \\
 Ids_6 : \llbracket \mathbf{own} \text{ ME} \dots \rrbracket_{Ids}^{\tau} &:= \begin{cases} \{\llbracket \text{ME} \rrbracket_{ID}\} & \text{if } \tau = \llbracket \text{ME} \rrbracket_{TYPE} \\ \emptyset & \text{otherwise} \end{cases} \\
 Ids_7 : \llbracket \text{Capsule} \rrbracket_{Ids}^{\tau, view} &:= \llbracket \text{Capsule} \rrbracket_{Ids}^{\tau} \cap \llbracket \text{Capsule} \rrbracket_{Ids}^{view} \\
 Ids_8 : \llbracket \text{Protocol} \rrbracket_{Ids}^{\tau} &:= \llbracket \text{SUPER} \rrbracket_{Ids}^{\tau} \\
 Ids_9 : \llbracket \mathbf{collaboration} : \mathbf{interaction} \text{ Interaction}^* \dots \rrbracket_{Ids} &:= \{self\} \cup \llbracket \text{Interaction}^* \rrbracket_{Ids} \\
 Ids_{10} : \llbracket \text{Connector} \rrbracket_{Ids} &:= \llbracket \text{SUPER} \rrbracket_{Ids} \\
 Ids_{11} : \llbracket \mathbf{association} : \mathbf{connection} \text{ AssociationEnd}^* \dots \rrbracket_{Ids} &:= \llbracket \text{AssociationEnd}^* \rrbracket_{ID}
 \end{aligned}$$

The last rule becomes necessary, because an Interaction is always part of a Collaboration, and never exists on its own as a ModelElement in an ElementOwnership. Similarly, AssociationEnds only exist within their Association.

Similarly, we define a function “Names” to select the names of all ModelElements of a given type from a particular view of a Capsule. Again, the optional parameters τ and $view$ stand for a kind of ModelElement and a view, respectively.

$\text{Names}_1 :$	$\llbracket \text{Capsule} \rrbracket_{\text{Names}}$	$:= \llbracket \text{Capsule} \rrbracket_{\text{Names}}^{\llbracket \text{Capsule} \rrbracket_{\text{Ids}}}$
$\text{Names}_2 :$	$\llbracket \text{Capsule} \rrbracket_{\text{Names}}^{\tau}$	$:= \llbracket \text{Capsule} \rrbracket_{\text{Names}}^{\llbracket \text{Capsule} \rrbracket_{\text{Ids}}^{\tau}}$
$\text{Names}_3 :$	$\llbracket \text{Capsule} \rrbracket_{\text{Names}}^{\tau, \text{view}}$	$:= \llbracket \text{Capsule} \rrbracket_{\text{Names}}^{\llbracket \text{Capsule} \rrbracket_{\text{Ids}}^{\tau, \text{view}}}$
$\text{Names}_4 :$	$\llbracket \text{Capsule} \rrbracket_{\text{Names}}^{\text{ids}}$	$:= \llbracket \text{SUPER} \rrbracket_{\text{Names}}^{\text{ids}}$
$\text{Names}_5 :$	$\llbracket \text{package : elementImport} \text{ ElementImport* Namespace} \rrbracket_{\text{Names}}^{\text{ids}}$	$:= \llbracket \text{ElementImport} \rrbracket_{\text{Names}}^{\text{ids}} \cup \llbracket \text{Namespace} \rrbracket_{\text{Names}}^{\text{ids}}$
$\text{Names}_6 :$	$\llbracket \text{import ME} \dots \text{alias name} \dots \rrbracket_{\text{Names}}^{\text{ids}}$	$:= \begin{cases} \{ \text{name} \} & \text{if } \llbracket \text{ME} \rrbracket_{\text{ID}} \in \text{ids} \\ \emptyset & \text{otherwise} \end{cases}$
$\text{Names}_7 :$	$\llbracket \text{ns : ownedElement} \text{ ElementOwnership*} \dots \rrbracket_{\text{Names}}^{\text{ids}}$	$:= \llbracket \text{ElementOwnership*} \rrbracket_{\text{Names}}^{\text{ids}}$
$\text{Names}_8 :$	$\llbracket \text{own ME} \dots \rrbracket_{\text{Names}}^{\text{ids}}$	$:= \begin{cases} \{ \llbracket \text{ME} \rrbracket_{\text{NAME}} \} & \text{if } \llbracket \text{ME} \rrbracket_{\text{ID}} \in \text{ids} \\ \emptyset & \text{otherwise} \end{cases}$
$\text{Names}_9 :$	$\llbracket \text{Protocol} \rrbracket_{\text{Names}}^{\tau}$	$:= \llbracket \text{SUPER} \rrbracket_{\text{Names}}^{\tau}$

We will use these functions frequently, when defining formal properties of models. Where no confusion can arise, we abbreviate the names of views by their initial letter, i.e. b instead of behavior, s instead of structure, and i instead of interfaces.

9.5.2. ProtocolRole and its users

When it comes to consistency checking, ProtocolRoles are used in many places, e.g. for Ports and ConnectorEnds, and thus for Connectors, Protocols and Capsules. First, the incoming and outgoing Signals of a ProtocolRole may be selected by clauses of the functions In and Out.

$$\begin{aligned} \text{In}_3 : \quad & \llbracket \text{pr : in Signal* out} \dots \text{behavior} \dots \rrbracket_{\text{In}} &:= \llbracket \text{Signal*} \rrbracket_{\text{NAME}} \\ \text{Out}_3 : \quad & \llbracket \text{pr : in} \dots \text{out Signal* behavior} \dots \rrbracket_{\text{Out}} &:= \llbracket \text{Signal*} \rrbracket_{\text{NAME}} \end{aligned}$$

Then, the behavior of the ProtocolRole may be extracted. It consists of the StateMachine, and its initial and terminal states.

$$\begin{aligned} \text{SM}_3 : \quad & \llbracket \text{pr :} \dots \text{behavior StateMachine} \dots \rrbracket_{\text{SM}} &:= \llbracket \text{StateMachine} \rrbracket_{\text{SM}} \\ \text{Initial}_3 : \quad & \llbracket \text{pr :} \dots \text{behavior StateMachine} \dots \rrbracket_{\text{Initial}} &:= \llbracket \text{StateMachine} \rrbracket_{\text{Initial}} \\ \text{Terminal}_3 : \quad & \llbracket \text{pr :} \dots \text{behavior StateMachine} \dots \rrbracket_{\text{Terminal}} &:= \llbracket \text{StateMachine} \rrbracket_{\text{Terminal}} \end{aligned}$$

Obviously, the last three rules are almost identical. This situation keeps recurring, so that in the remainder, for convenience and clarity, we will abbreviate such families of rules like this:

$$x_3 : \quad \llbracket \text{pr :} \dots \text{behavior StateMachine} \dots \rrbracket_x := \llbracket \text{StateMachine} \rrbracket_x$$

where $x \in \{\text{SM}, \text{Initial}, \text{Terminal}\}$. As we have said, Ports and ConnectorEnds are specified using ProtocolRoles, so that we can extract their various aspects simply by the following rules.

$$\begin{aligned}
x_4 : \llbracket \mathbf{ce} : \mathbf{role} \text{ ProtocolRole} \dots \rrbracket_x &:= \llbracket \text{ProtocolRole} \rrbracket_x \\
x_5 : \llbracket \mathbf{port} : \mathbf{role} \text{ ProtocolRole} \dots \rrbracket_x &:= \llbracket \text{ProtocolRole} \rrbracket_x
\end{aligned}$$

where $x \in \{\text{SM}, \text{Initial}, \text{Terminal}, \text{In}, \text{Out}\}$. The extraction is a little more complex for Capsules, as these have a large number of different specification elements. To select the right ones, the type of the `ModelElement` and the view it is contained in may be used.

$$\begin{aligned}
x_6 : \llbracket \text{Capsule} \rrbracket_x^{v, \tau} &:= \llbracket \text{Capsule} \rrbracket_x^{\llbracket \text{Capsule} \rrbracket_{ids}^{v, \tau}} \\
x_7 : \llbracket \mathbf{capsule} : \dots \text{Package} \rrbracket_x^{ids} &:= \llbracket \text{Package} \rrbracket_x^{ids} \\
x_8 : \llbracket \mathbf{package} : \mathbf{importedElement} \begin{array}{l} \text{EI}^* \text{ Namespace} \end{array} \rrbracket_x^{ids} &:= \llbracket \text{EI}^* \rrbracket_x^{ids} \cup \llbracket \text{Namespace} \rrbracket_x^{ids} \\
x_9 : \llbracket \mathbf{ns} : \mathbf{ownedElement} \text{ EO}^* \rrbracket_x^{ids} &:= \llbracket \text{EO}^* \rrbracket_x^{ids} \\
x_{10} : \llbracket \mathbf{ei} : \dots \text{ME} \rrbracket_x^{ids} &:= \begin{cases} \{\llbracket \text{ME} \rrbracket_x\} & \text{if } \llbracket \text{ME} \rrbracket_{ID} \in ids \\ \emptyset & \text{otherwise} \end{cases} \\
x_{11} : \llbracket \mathbf{eo} : \dots \text{ME} \rrbracket_x^{ids} &:= \begin{cases} \{\llbracket \text{ME} \rrbracket_x\} & \text{if } \llbracket \text{ME} \rrbracket_{ID} \in ids \\ \emptyset & \text{otherwise} \end{cases}
\end{aligned}$$

where $x \in \{\text{SM}, \text{Initial}, \text{Terminal}, \text{In}, \text{Out}, \text{Run}\}$, and v is a view and τ a kind of `ModelElement`. Not all combinations of arguments are sensible, of course, so we list the admissible ones explicitly in Table 9.1.

value of x	argument (kind of <code>ModelElement</code>)					
	StateMachine	b,sm	i,port	Interaction	b,ia	f,ia
Run	×	×	×	✓	✓	✓
SM	✓	✓	✓	×	×	×
Initial	✓	✓	✓	✓	✓	✓
Terminal	✓	✓	✓	×	×	×
In	✓	✓	✓	✓	✓	✓
Out	✓	✓	✓	✓	✓	✓

Table 9.1.: Admissible combinations of arguments for clauses x_6 .

9.5.3. Executable models from Configurations

Intuitively, a Capsule is often seen as a unit with behavior—but just which of the many specification elements of a Capsule should be used to determine “the” behavior? Obviously, the Ports should play a role, and the behavior view. However, in the absence of the latter, the structure view can also be used, as its Configuration is a composition of several Capsules again.

We shall now show, how a comprehensive behavioral model of a Capsule can be created. Due to the complexity of models expected from this semantics, its prime purpose will be simulation, so the rule is called Sim. To start with, we distinguish between different cases of the specification.

$$\text{Sim}_1 : \llbracket \text{Capsule} \rrbracket_{\text{Sim}} := \begin{cases} \text{Synch}(\llbracket \text{Capsule} \rrbracket_{\text{SM}}^{i,\text{sm}} \cup \llbracket \text{Capsule} \rrbracket_{\text{SM}}^{b,\text{sm}}, \llbracket \text{Capsule} \rrbracket_{\text{Signal}}^{\text{bindings}}) & \text{if there is a b-view} \\ \text{Synch}(\llbracket \text{Capsule} \rrbracket_{\text{SM}}^{i,\text{sm}} \cup \llbracket \text{Capsule} \rrbracket_{\text{SM}}^{s,\text{config}}, \llbracket \text{Capsule} \rrbracket_{\text{Signal}}^{\text{bindings}}) & \text{if there is no b, but a s-view} \\ \llbracket \text{Capsule} \rrbracket_{\text{SM}}^{i,\text{sm}} & \text{otherwise} \end{cases}$$

Turning to the last case first, we see that if there is neither a structure nor a behavior view, that is, if the Capsule is an empty shell, only the Ports can be used to create a behavioral model. Since the Ports are the exclusive points of interaction of a Capsule, they define the behavioral environment of a Capsule. Since Ports are by definition independent from each other, this model can be created simply as the net-union of the behavioral models of the individual Ports.

The first and the second case are structurally similar. In both cases, the same model as in the third case is created, but with a model from either the behavior or the structure view added—and not just added, but synchronized with the shell-model. We here decide, that a behavior model is to take precedence over a structural refinement, as the latter is probably more complex than the former, and methodologically, they are supposed to be behaviorally equivalent. Whether this is actually the case may later be verified formally (see Chapter 10).

We now turn to the behavioral model of the Configuration. A Configuration contains a set of Capsules and Connectors between them, and some binding Connectors. The bindings are used to synchronize the shell-model with the structure-model. The structure model is created by recursively translating all its Capsules and Connectors separately, and connecting them, according to the Connectors.

$$\begin{aligned} \text{Sim}_2 : \llbracket \text{Capsule} \rrbracket_{\text{Sim}} &:= \llbracket \text{Capsule} \rrbracket_{\text{Sim}}^{\llbracket \text{Capsule} \rrbracket_{\text{Sim}'}} \\ \text{Sim}_3 : \llbracket \text{Capsule} \rrbracket_{\text{Sim}}^{\text{caps},\text{con},\text{bdg}} &:= \text{Connect}(\llbracket \text{Capsule} \rrbracket_{\text{Sim}}^{\text{caps}} \cup \llbracket \text{Capsule} \rrbracket_{\text{Sim}}^{\text{con}}, \llbracket \text{Capsule} \rrbracket_{\text{Bdg}}) \end{aligned}$$

where

$$\text{Sim}' : \left[\begin{array}{l} \text{configuration : capsule } id_1^* \text{ connector} \\ id_2^* \text{ binding } id_3^* \text{ ModelElement} \end{array} \right]_{\text{Sim}'} := \langle id_1^*, id_2^*, id_3^* \rangle$$

SimpleConnectors have no behavior, so they are translated directly to the empty net. For ComplexConnectors, the semantics of its behavior is created. This is the same schema that is used for the (intuitive) semantics of Protocols, see Figure 9.15.

$$\begin{aligned} \text{Sim}_5 : \llbracket \text{ccon : behavior StateMachine} \dots \rrbracket_{\text{Sim}} &:= \llbracket \text{StateMachine} \rrbracket_{\text{SM}} \\ \text{Sim}_6 : \llbracket \text{scon :} \dots \rrbracket_{\text{Sim}} &:= N_0 \end{aligned}$$

The bindings and Connectors are simply translated into pairs (sets) of identifiers that are to be connected. Note that we here restrict ourselves to binary bindings for technical reasons.

$\text{Con}_1 :$	$\llbracket \text{association : connection} \rrbracket_{\text{Con}}$	$:= \llbracket \text{AssociationEnd}^* \rrbracket_{\text{Con}}^\lambda$
$\text{Con}_2 :$	$\llbracket \text{ae : ... type Classifier ...} \rrbracket_{\text{Con}}$	$:= \{ \llbracket \text{Classifier} \rrbracket_{\text{ID}} \}$
$\text{Con}_3 :$	$\llbracket \text{connector : ...} \rrbracket_{\text{Con}}$	$:= \llbracket \text{SUPER} \rrbracket_{\text{Con}}$
$\text{Con}_4 :$	$\llbracket \text{ce : ...} \rrbracket_{\text{Con}}$	$:= \llbracket \text{SUPER} \rrbracket_{\text{Con}}$
$\text{Con}_5 :$	$\llbracket \lambda \rrbracket_{\text{Con}}^x$	$:= x$
$\text{Con}_6 :$	$\llbracket \text{AE AE}^* \rrbracket_{\text{Con}}^x$	$:= \llbracket \text{AE} \rrbracket_{\text{Con}} \cdot \llbracket \text{AE}^* \rrbracket_{\text{Con}}^x$

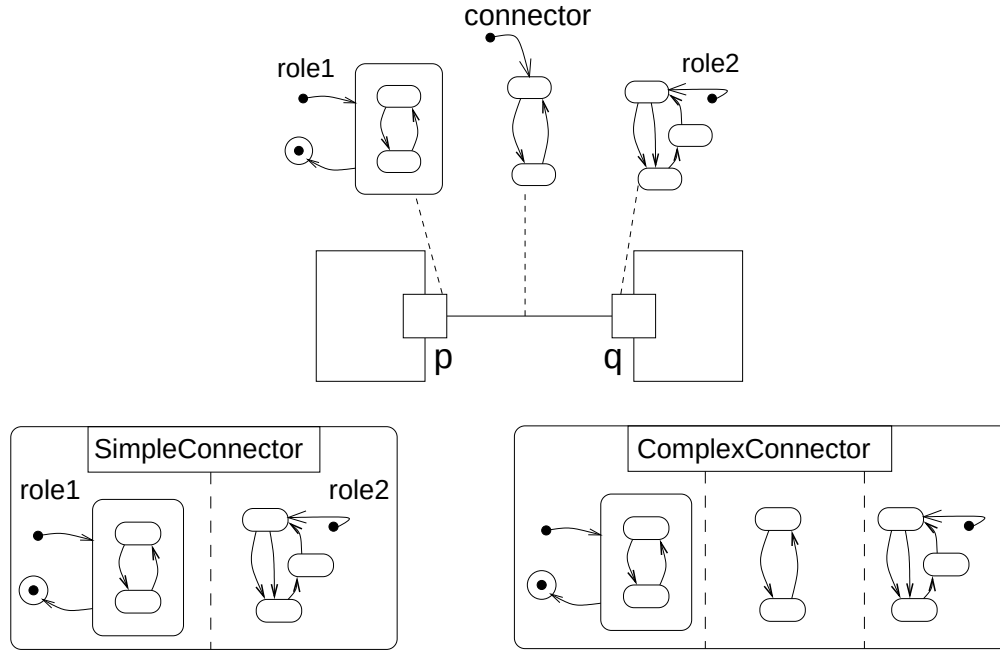


Figure 9.15.: Simulation-semantics of Configuration: a Configuration and the StateMachines of the ProtocolRoles (top); if the Connector is a ComplexConnector, it has as its behavior another StateMachine that mediates between the two ConnectorEnds (bottom, left); if the Connector is a SimpleConnector, however, the behavior of (this part of) the Configuration is just the parallel composition of the StateMachines of the ProtocolRoles. (bottom, right).

$$\text{Bdg}_1 : \llbracket \text{Capsule} \rrbracket_{\text{Bdg}} := \left(\llbracket \text{Capsule} \rrbracket_{\text{Ids}}^{s, \text{port}} \cdot \llbracket \text{Capsule} \rrbracket_{\text{Con}}^{\text{binding}} \right) \cup \left(\llbracket \text{Capsule} \rrbracket_{\text{Con}}^{\text{binding}} \cdot \llbracket \text{Capsule} \rrbracket_{\text{Ids}}^{s, \text{port}} \right)$$

Finally, we need new rules for initial and terminal states of configurations, too.

$x_{12} :$	$\llbracket \text{configuration : capsule id}^* \rrbracket_x$	$:= \llbracket \text{Capsule}^* \rrbracket_x + \llbracket \text{Connector}^* \rrbracket_x$
$x_{13} :$	$\llbracket \text{Capsule} \rrbracket_x$	$:= \llbracket \text{Capsule} \rrbracket_x^{b, \text{sm}}$
$x_{14} :$	$\llbracket \text{Capsule} \rrbracket_x^{\text{sub}}$	$:= \llbracket \text{Capsule} \rrbracket_x^{s, \text{config}}$
$x_{15} :$	$\llbracket \text{ccon : behavior StateMachine ...} \rrbracket_x$	$:= \llbracket \text{StateMachine} \rrbracket_x$
$x_{16} :$	$\llbracket \text{scon : ...} \rrbracket_x$	$:= \emptyset$

where $x \in \{\text{Initial}, \text{Terminal}\}$.

9.5.4. Dependency

Relationship is abstract and has no attributes of its own. Its subclass Dependency specifies a client and a supplier, that are interpreted as implementation and specification, respectively. Its subclass Usage has no further attributes, its subclass Abstraction has a mapping that elaborates on the precise nature of the Abstraction.

$$\begin{aligned}
 R_1 : \llbracket \text{dependency : client } impl \rrbracket_R^{\text{mapping}} &:= \langle impl \mapsto spec, mapping \rangle \\
 R_2 : \llbracket \text{abstraction : kind "realize" mapping } \rho \dots \rrbracket_R &:= \llbracket \text{SUPER} \rrbracket_R^\rho \\
 R_3 : \llbracket \text{Usage} \rrbracket_R &:= \llbracket \text{SUPER} \rrbracket_D^0
 \end{aligned}$$

Dependencies can be exploited in several ways. On the one hand, it can be used to formalize the dependencies between two Capsules, such as a specification and an implementation, such as when refining an architecture, or between a component-Capsule and a style-Capsule. On the other hand, it can be used to represent dependencies between pieces of code implementing Capsules. We currently deal only with $\llbracket \text{realize} \rrbracket$ -Abstractions.

Style abstraction and architectural refinement

As an example for style abstraction, consider Figure 5.27 above for the syntax, and the following Figure 9.16 below for the abstract syntax and resulting semantical objects. The case of architectural refinement is identical, as styles are Capsules.

abstraction:	abstraction:
mapping $\{id_1 \mapsto id_9, id_2 \mapsto id_{10}\}$	mapping $\{id_2 \mapsto id_{12}, id_4 \mapsto id_{13}\}$
dependency:	dependency:
client id_1 supplier id_7	client id_3 supplier id_{11}
relationship:	relationship:
me: name a	me: name b
e: id id_5 e.	e: id id_6 e.
$\langle id_1 \mapsto id_7, \{id_1 \mapsto id_9, id_2 \mapsto id_{10}\} \rangle$	$\langle id_3 \mapsto id_{11}, \{id_2 \mapsto id_{12}, id_4 \mapsto id_{13}\} \rangle$

Figure 9.16.: Translation of Abstractions: abstract syntax representation of the two Abstractions of Figure 5.27 (top); resulting semantical structures (bottom).

Code dependencies

In Section 6.4.5 we have briefly sketched how the static structure and the implementation view of a Capsule can be exploited for code-generation by computing the dependency between the Components that implement the Capsule. We will now show this in detail.

There are two kinds of dependencies to be considered: those that are explicitly stated by usage Relationships between Components in the implementation view, and those that are implicit in the aggregation hierarchy of the sub-Configuration of structure view.⁹ The structure view has precedence over the implementation view, i.e., if a sub-configuration is used, it is assumed that the implementation view is obsolete, as far as Dependencies are concerned.

$$D_1 : \llbracket \text{Capsule} \rrbracket_D := \begin{cases} \llbracket \text{Capsule} \rrbracket_D^{im} & \text{if } \llbracket \text{Capsule} \rrbracket_{Ids}^{s,capsule} = \emptyset \\ \llbracket \text{Capsule} \rrbracket_D^s & \text{if } \llbracket \text{Capsule} \rrbracket_{Ids}^{s,capsule} = \emptyset = \llbracket \text{Capsule} \rrbracket_{Ids}^{im,component} \\ \langle \{self\}, \emptyset \rangle & \text{otherwise} \end{cases}$$

The resulting dependency structure is a graph, where the nodes are Component or Capsule identifiers, and the edges are dependencies between them. The implementation view of a Capsule represents Components and usage relationships. The structure view represents the structural refinement of a Capsule. Sub-Capsules may of course be refined again, so D_3 must be defined recursively.

$$D_2 : \llbracket \text{Capsule} \rrbracket_D^{im} := \left\langle \begin{array}{c} \{self\} \cup \llbracket \text{Capsule} \rrbracket_{Ids}^{im,component} \\ \{ \langle self, x \rangle \mid x \in \llbracket \text{Capsule} \rrbracket_{Ids}^{im,component} \}, \\ \llbracket \text{Capsule} \rrbracket_{Ids}^{im,usage} \end{array} \right\rangle$$

$$D_3 : \llbracket \text{Capsule} \rrbracket_D^s := \left\langle \begin{array}{c} \{self\} \cup \llbracket \text{Capsule} \rrbracket_{Ids}^{s,capsule}, \\ \{ \langle self, x \rangle \mid x \in \llbracket \text{Capsule} \rrbracket_{Ids}^{s,capsule} \}, \\ \llbracket \text{SUPER} \rrbracket_D \end{array} \right\rangle$$

$$D_4 : \llbracket \text{Package} \rrbracket_D := \llbracket \text{SUPER} \rrbracket_D$$

$$D_5 : \llbracket \text{ns : ownedElement} \rrbracket_D := \text{ElementOwnership}^*$$

$$D_6 : \llbracket \text{own ME} \dots \rrbracket_D := \begin{cases} \llbracket \text{ME} \rrbracket_D & \text{if } \llbracket \text{ME} \rrbracket_{\text{TYPE}} = \text{capsule} \\ \langle \emptyset, \emptyset \rangle & \text{otherwise} \end{cases}$$

Nodes representing Components are labeled by a filename that is extracted from the file attribute of the respective Component. Nodes representing Capsules are unlabeled.

$$D_7 : \llbracket \text{component : file string} \dots \rrbracket_D := \{self \mapsto string\}$$

Recall that a Component is a “*physical [...] part of a system*” (cf. [OMG UML 1.3, p. 2-29]), so we assume that its file attribute is indeed pointing to an existing file that stores its implementation as, say, source code. Of course, further subclasses of Component may differentiate between source- and object-code. This would result in similar but distinct semantics for the creation of link-files and make-files.

Consider Figures 9.18 and 9.17 for an example.

⁹We disrespect the resident attribute of Component, as its meaning is unclear.

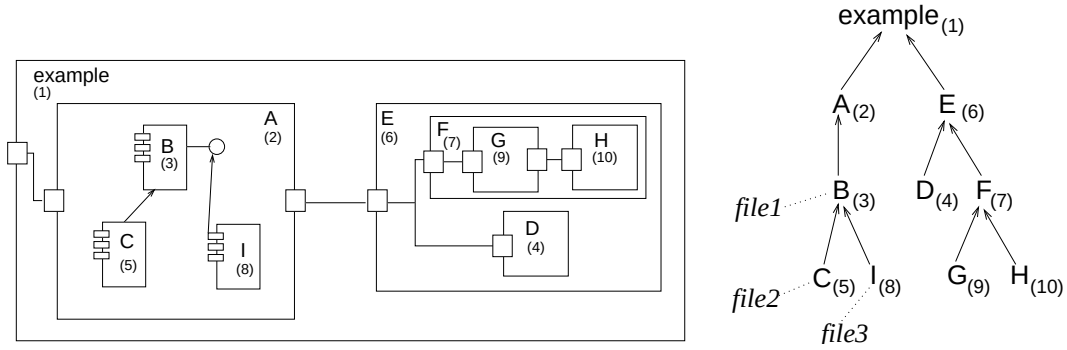


Figure 9.17.: A capsule diagram (left) whose abstract syntax tree is shown in Figure 9.18. Translating the abstract syntax tree results in a dependency graph (right). The small numbers in brackets are the identifiers of the respective ModelElement.

9.6. Quantitative models of Capsules

In this section we show how some simple kinds of quantitative analysis can be conducted. The methods we use are taken from standard textbooks: see Peterson [1981], Reisig [1985, 1992], Starke [1990] for introductions to Petri-nets that include statistical analysis methods, and Bause and Kritzinger [1996] for an in depth treatment of generalized stochastic Petri-nets. For simplicity, throughout this section we assume that all Connectors are binary and bidirectional.

9.6.1. Size metrics

Many software development processes require precise measures of the size and complexity of a program, like function-points or Lines-of-code (LOC) etc. To our knowledge, no such measures have been suggested for design or analysis documents yet. Using the UML, the “size” of a document (i.e. diagram) could be measured in the number of ModelElements used, abbreviated to NOM. This measure directly parallels LOC for programs. First consider StateMachines: all States and Transitions count as one.

$$\begin{aligned}
 \text{NOM}_1 : \llbracket \text{sm} : \text{S transition } T^* \dots \rrbracket_{\text{NOM}} &:= 1 + \llbracket S \rrbracket_{\text{NOM}} + \llbracket T^* \rrbracket_{\text{NOM}} \\
 \text{NOM}_2 : \llbracket \text{transition} : \dots \rrbracket_{\text{NOM}} &:= 1 \\
 \text{NOM}_3 : \llbracket \text{simple} : \dots \rrbracket_{\text{NOM}} &:= 1 \\
 \text{NOM}_4 : \llbracket \text{final} : \dots \rrbracket_{\text{NOM}} &:= 1 \\
 \text{NOM}_5 : \llbracket \text{synch} : \dots \rrbracket_{\text{NOM}} &:= 1 \\
 \text{NOM}_6 : \llbracket \text{composite} : \dots \text{subvertex } S^* \dots \rrbracket_{\text{NOM}} &:= 1 + \llbracket S^* \rrbracket_{\text{NOM}} \\
 \text{NOM}_7 : \llbracket \text{capsule} : \dots \text{view } \text{View}^* \dots \rrbracket_{\text{NOM}} &:= 1 + \llbracket \text{View}^* \rrbracket_{\text{NOM}} + \llbracket \text{SUPER} \rrbracket_{\text{NOM}}
 \end{aligned}$$

Capsules are counted by their Views and the number of ModelElements they contain. Importing a ModelElement counts as a unit, and NameSpaces count by the sum of all their ownedElements. A ProtocolRole counts by its Signals and its behavior.

```

capsule: ...
package: importedElement
ns: ownedElement
  own package: importedElement
    ns: ownedElement
      own capsule:
        package: importedElement
        ns: ownedElement
          own component: file file1
            me: name B e: id id3 e.
          visibility +
          own component: file file2
            me: name C e: id id5 e.
          visibility +
          own component: file file3
            me: name D e: id id4 e.
          visibility +
          own usage: supplier id5 client id3 ...
          visibility +
          own usage: supplier id8 client id3 ...
          visibility +
          own usage: supplier id3 client id2 ...
          ...
          visibility +
        me: name A e: id id2 e.
      visibility +
    own capsule:
      package: importedElement
      ns: ownedElement
        own capsule:...
          package: importedElement
          ns: ownedElement
            own capsule: ...
              me: name G e: id id9 e.
            visibility +
            own capsule: ...
              me: name H e: id id10 e.
            visibility +
          me: name F e: id id7 e.
        visibility +
        own capsule:
          package: importedElement
          ns: ownedElement
            me: name I e: id id8 e.
          visibility +
        me: name E e: id id6 e.
      visibility +
    visibility +
  me: name example e: id id1 e.

```

Figure 9.18.: Abstract syntax representation of the Capsule shown in Figure 9.17, left.

NOM ₈ :	$\llbracket \text{view} : \dots \rrbracket_{\text{NOM}}$	$:= 1$
NOM ₉ :	$\llbracket \text{Configuration} \rrbracket_{\text{NOM}}$	$:= 1$
NOM ₁₀ :	$\llbracket \text{ns : ownedElement} \quad \text{ElementOwnership}^* \text{ ModelElement} \rrbracket_{\text{NOM}}$	$:= \llbracket \text{ElementOwnership}^* \rrbracket_{\text{NOM}}$
NOM ₁₁ :	$\llbracket \text{package : importedElement} \quad \text{ElementImport}^* \text{ Namespace} \rrbracket_{\text{NOM}}$	$:= \llbracket \text{ElementImport}^* \rrbracket_{\text{NOM}} + \llbracket \text{Namespace} \rrbracket_{\text{NOM}}$
NOM ₁₂ :	$\llbracket \text{own ME} \rrbracket_{\text{NOM}}$	$:= \llbracket \text{ME} \rrbracket_{\text{NOM}}$
NOM ₁₃ :	$\llbracket \text{import} \dots \rrbracket_{\text{NOM}}$	$:= 1$
NOM ₁₄ :	$\llbracket \text{port : ProtocolRole} \dots \rrbracket_{\text{NOM}}$	$:= 1 + \llbracket \text{ProtocolRole} \rrbracket_{\text{NOM}}$
NOM ₁₅ :	$\llbracket \text{pr : in Signal}_1^* \text{ out Signal}_2^* \quad \text{behavior StateMachine} \dots \rrbracket_{\text{NOM}}$	$:= 1 + \llbracket \text{StateMachine} \rrbracket_{\text{NOM}} + \llbracket \text{Signal}_1^* \rrbracket_{\text{NOM}} + \llbracket \text{Signal}_2^* \rrbracket_{\text{NOM}}$
NOM ₁₆ :	$\llbracket \text{collaboration : interaction} \quad \text{Interaction}^* \dots \rrbracket_{\text{NOM}}$	$:= 1 + \llbracket \text{Interaction}^* \rrbracket_{\text{NOM}}$
NOM ₁₇ :	$\llbracket \text{ia : message Message}^* \dots \rrbracket_{\text{NOM}}$	$:= 1 + \llbracket \text{Message}^* \rrbracket_{\text{NOM}}$
NOM ₁₈ :	$\llbracket \text{Message} \dots \rrbracket_{\text{NOM}}$	$:= 1$

We omit rules accounting for optional arguments, as they are immaterial for this mapping. Finally, we need rules to treat the plural-cases.

NOM ₁₉ :	$\llbracket \text{Transition Transition}^* \rrbracket_{\text{NOM}}$	$:= 1 + \llbracket \text{Transition}^* \rrbracket_{\text{NOM}}$
NOM ₂₀ :	$\llbracket \text{S S}^* \rrbracket_{\text{NOM}}$	$:= \llbracket \text{S} \rrbracket_{\text{NOM}} + \llbracket \text{S}^* \rrbracket_{\text{NOM}}$
NOM ₂₁ :	$\llbracket \text{Port Port}^* \rrbracket_{\text{NOM}}$	$:= \llbracket \text{Port} \rrbracket_{\text{NOM}} + \llbracket \text{Port}^* \rrbracket_{\text{NOM}}$
NOM ₂₂ :	$\llbracket \text{Connector Connector}^* \rrbracket_{\text{NOM}}$	$:= 1 + \llbracket \text{Connector}^* \rrbracket_{\text{NOM}}$
NOM ₂₃ :	$\llbracket \text{Capsule Capsule}^* \rrbracket_{\text{NOM}}$	$:= \llbracket \text{Capsule} \rrbracket_{\text{NOM}} + \llbracket \text{Capsule}^* \rrbracket_{\text{NOM}}$
NOM ₂₄ :	$\llbracket \text{View View}^* \rrbracket_{\text{NOM}}$	$:= \llbracket \text{View} \rrbracket_{\text{NOM}} + \llbracket \text{View}^* \rrbracket_{\text{NOM}}$
NOM ₂₅ :	$\llbracket \lambda \rrbracket_{\text{NOM}}$	$:= 0$

As a next step, a measure like net-lines of code (i.e. lines of non-comment code) can be computed by leaving aside those ModelElements that are just containers (like Configuration, Port, ProtocolRole and StateMachine—but not CompositeState or Capsule!). This measure might be appropriate when estimating implementation effort. A coarser grained measure, on the other hand, would count only those containers, but not their constituents. Such a measure might be appropriate to forecast the size of the design from the result of the analysis phase.

In yet another step, the complexity of POIs may be computed by the function CP as follows.

$$\begin{aligned}
CP_1 : \llbracket \text{Capsule} \rrbracket_{CP} &:= \llbracket \text{Capsule} \rrbracket_{CP}^{i,pr} \\
CP_2 : \llbracket \text{Capsule} \rrbracket_{CP}^{ids} &:= \llbracket \text{SUPER} \rrbracket_{CP}^{ids} \\
CP_3 : \llbracket \text{package : importedElement} \rrbracket_{CP}^{ids} &:= \llbracket \text{Namespace} \rrbracket_{CP}^{ids} \cup \llbracket \text{ElementImport}^* \rrbracket_{CP}^{ids} \\
CP_4 : \llbracket \text{ns : ownedElement} \rrbracket_{CP}^{ids} &:= \llbracket \text{ElementOwnership}^* \rrbracket_{CP}^{ids} \\
CP_5 : \llbracket \text{own ME} \dots \rrbracket_{CP}^{ids} &:= \begin{cases} \llbracket \text{ME} \rrbracket_{NOM} & \text{if } self \in ids \\ 0 & \text{otherwise} \end{cases} \\
CP_6 : \llbracket \text{import ME} \dots \rrbracket_{CP}^{ids} &:= \begin{cases} \llbracket \text{ME} \rrbracket_{NOM} & \text{if } self \in ids \\ 0 & \text{otherwise} \end{cases} \\
CP_7 : \llbracket \text{pr : in Signal}_1^* \text{ out Signal}_2^* \rrbracket_{CP} &:= \llbracket \text{Signal}_2^* \rrbracket_{CP} + \llbracket \text{Signal}_1^* \rrbracket_{CP} + \llbracket \text{StateMachine} \rrbracket_{BC}
\end{aligned}$$

CP stands for capsule points, and we have chosen the name in analogy to function and object points. Similarly, the original object-points measure could be computed: object-points simply correspond to the number of Features in that Class.

9.6.2. A simple behavioral complexity metrics

In the same way, one can define simple complexity metrics like function-points. In order to define a metric capturing the behavioral complexity of a StateMachine, this might be a weighted NOM, i.e. specific factors for the dynamic aspects of models (or their ModelElements). Being calibration data, they must be gleaned from practical experience. We call this measure BC.

$$\begin{aligned}
BC_1 : \llbracket \text{StateMachine} \rrbracket_{BC} &:= c_{sm} \cdot \llbracket \text{StateMachine} \rrbracket_{NOM} \\
BC_2 : \llbracket \text{Collaboration} \rrbracket_{BC} &:= c_{coll} \cdot \llbracket \text{Collaboration} \rrbracket_{NOM} \\
BC_3 : \llbracket \text{Interaction} \rrbracket_{BC} &:= c_{ia} \cdot \llbracket \text{Interaction} \rrbracket_{NOM}
\end{aligned}$$

Thus, for a Capsule, a straightforward measure for the complexity of its interfaces is the complexity of the ProtocolRoles, i.e. the number of Signals it may send and receive, and the specifications of the behaviors of its Ports.

9.6.3. Stochastic analysis of buffer capacities

One important objective in building architectural models is the ability to analyze systems before they are actually built, just like checking the static of an edifice before erecting it. The probabilistic behavior and buffer capacities is an important aspect of (distributed) software systems, so it is quite suggestive to interpret an architectural design as a queuing network, and thus try and analyze its signal traffic behavior. From this point of view, a Configuration can be seen as a queuing network, every Capsule becomes a stochastic process, and Connectors are buffers.

To keep things simple, we only consider binary Connectors. We assume that a Connector buffers Signals that have been sent, but not yet received.¹⁰ The capacity is independent of type of message. Signals are stored separately for the two directions. We assume a semantical function $\llbracket - \rrbracket_Q$ that transforms the Connector characteristics into a (total) buffering capacity, that may be freely allocated for either direction.

We also assume that the semantical function $\llbracket - \rrbracket_Q$ transforms Port characteristics into a Petri-net that has as an additional inscription on transitions a probability distribution that acts as an activity rate for describing the frequency of dispatching and processing Signals of that transition. We assume the reception and transmission are independent from each other, and have the same characteristics. So there are two stochastic processes for each Port.

In order to stay within the formal frame we have established thus far, we need a slightly enhanced Petri-net model here, where the labeling inscription is replaced by two inscriptions $c : P \rightarrow \mathbb{N}$ and $b : T \rightarrow \lambda$ for the buffer capacity and the stochastic behavior of the system entities represented by the places and transitions, respectively. Queuing networks can be represented and analyzed by such nets quite elegantly (see e.g. Bause and Kritzinger [1996]). Figure 9.19 shows the general schema of the translation: A Port is mapped into a pair of transitions, one for each direction. Both transitions have the same processing rate $\lambda_t = \llbracket \text{Port} \rrbracket_Q$. Analogously, a Connector is mapped into a pair of places, one for each direction. Each has as its maximum capacity $\llbracket \text{Connector} \rrbracket_Q$, but dynamically, they have this capacity *together*. In other words, if p_{to} and p_{fro} are the two places for a Connector with capacity c , in all reachable states we have $p_{to} + p_{fro} \leq c$.



Figure 9.19.: Schema for generating a stochastic model: a Configuration (left); and its simple analysis model (right).

Capsules are translated straightforward according to the schema outlined above. If there is no structural refinement, the Capsule is translated flatly, i.e. only its interfaces view. If the structure view contains Capsules, the sub-Configuration is translated separately, and the two parts are then combined by synchronizing those transitions that are connected by the bindings of the Configuration.

$$Q_1 : \llbracket \text{Capsule} \rrbracket_Q^{id} := \langle P, T; F; \beta \rangle$$

where $id \in \llbracket \text{Capsule} \rrbracket_{\text{Ids}}^{s, \text{connector}}$, and we have

$$\begin{aligned} P &:= \{c_1, c_2\} \\ T &:= \{p_1, p_2, p'_1, p'_2\} \\ F &:= \{\langle p_1, c_1 \rangle, \langle c_1, p'_2 \rangle, \langle p_2, c_2 \rangle, \langle c_2, p'_1 \rangle\} \\ \beta &:= \{p_1 \mapsto \llbracket \text{Capsule} \rrbracket_Q^{id_1}, p'_1 \mapsto \llbracket \text{Capsule} \rrbracket_Q^{id_1}, p_2 \mapsto \llbracket \text{Capsule} \rrbracket_Q^{id_2}, p'_2 \mapsto \llbracket \text{Capsule} \rrbracket_Q^{id_2}\} \end{aligned}$$

¹⁰Recall that two distinct StateMachines of the two Ports have distinct event spaces.

and $\langle id_1, id_2 \rangle = \llbracket \text{Capsule} \rrbracket_{\text{Bdg}}$

$$\begin{aligned}
 Q_2 : \llbracket \text{connector} : \dots \text{CE}^* \rrbracket_Q^{ID} &:= \llbracket \text{CE}^* \rrbracket_{ID} \\
 Q_3 : \llbracket \text{ce} : \dots \text{port } id \dots \rrbracket_Q &:= id \\
 Q_4 : \llbracket \text{capsule} : \dots \text{NameSpace} \rrbracket_Q^{id} &:= \llbracket \text{ns} : \text{OE}^* \rrbracket_Q^{id} \\
 Q_5 : \llbracket \text{own} \dots \text{ME} \rrbracket_Q^{id} &:= \begin{cases} \llbracket \text{ME} \rrbracket_Q & \text{if } \llbracket \text{ME} \rrbracket_{ID} = id \\ \emptyset & \text{otherwise} \end{cases} \\
 Q_6 : \llbracket \text{port} : \dots \text{characteristics } \lambda \dots \rrbracket_Q &:= \lambda
 \end{aligned}$$

The firing probabilities to be attached to transitions, and the capacities of places can be extracted from the characteristics-attributes of Ports and Connectors. We assume that an appropriate semantic function $\llbracket \text{string} \rrbracket_{\text{char}}$ is available.

$$\begin{aligned}
 Q_7 : \llbracket \text{port} : \dots \text{characteristics } c \dots \rrbracket_Q &:= \{ \text{self} \mapsto \llbracket c \rrbracket_{\text{char}} \} \\
 Q_8 : \llbracket \text{connector} : \text{port } id^* \text{ characteristics } c \dots \rrbracket_Q &:= \{ \text{self} \mapsto \llbracket c \rrbracket_{\text{char}} \}
 \end{aligned}$$

Obviously, there are rather tight limits on what can be accomplished with this semantics. The purpose, however, is to illustrate the path, rather than go it to its limits. That, unfortunately, is beyond the scope of this book.

9.7. Chapter summary

In this chapter, we have defined a function from abstract syntax (that is, the physical metamodel) to semantic domains (mostly Petri-nets). We have defined it in a very precise, painstaking way. Though not actually coding it in a program, this could be done almost without thinking. This level of precision, along with the scope of concepts covered are far beyond most other current approaches to UML semantics.

In Parts I and II, we have defined concepts, syntax and semantics of architectural models. In the remaining Part III, we shall define formal consistency conditions of architectural models, a specific process for creating and maintaining them, and we provide an extended example of how this may be done.

Part III.

Pragmatics

10. Consistency conditions

In this chapter we provide semantical mappings for the properties of models. The analysis targets identified in Section 6.4 are reconsidered, and given a precise meaning in such a way, that they fit to the semantical objects created by the mappings for models (previous chapter). Thus, finally, the intuitive properties of several diagram types become formally analyzable (see the sketch in the very beginning of this book, page 7). The applications of these properties will be explained in the example in Chapter 12.

We document only those properties that we found intuitive and useful, no doubt there are further properties definable. This, however, is a question of practical application that is beyond the scope of this book. The properties defined here will be used in the process patterns of Chapter 11, and there will be some examples of their actual usage in Chapter 12. Whenever we start any semantic analysis, we assume all syntactic constraints to be satisfied.

10.1. Expressing intuitive properties formally

Most of the conditions we have mentioned above can be reduced to one of a small number of formal properties, since there are only four recurring patterns:¹

- Given an Interaction and a StateMachine, is the former a sample run of the latter?
- Given two StateMachines, does the one implement the other?
- Given two StateMachines, are they equivalent?
- Given a StateMachine, is it, in a certain sense, well-behaved?

These questions give rise to a set of formal relationships. They come in different degrees of discriminative power, according to the underlying semantical model: sequential, true concurrency or interleaving semantics. There is a fundamental difference between these three, and so there are three different notions of implementation and equivalence (see Section 8.2.4).

Accordingly, there are different notions of run, implementation, and equivalence on the intuitive level, too. Which of them is appropriate, depends *exclusively* on the intended purpose and can not be determined theoretically. For purely sequential applications (that is, single-threaded applications or co-routines), only sequentialized behaviors need be considered, so that the $\mathcal{L}$ -implementation and $\mathcal{L}$ -equivalence are sufficient. This might also be the case in an early stage of the development, where only logical errors are looked for.

For distributed applications (or for the later, more detailed stages of the development of sequential systems) concurrent behavior will have to be considered as well. In some execution models, and under certain circumstances, interleaving concurrency is sufficient. Still later on, or for truly concurrent applications, the true concurrency versions will have to be used.

¹Recall that we use Interactions only as scenarios. In other words, we disrespect the activator relationship.

10.1.1. "Is-run-of" relations

The central observation is the following: a StateMachine specifies a complete behavior (relative to an arbitrary, but fixed context), while an Interaction specifies a sample run, a partial specification of a behavior. That is, in Petri-net terminology: is the net representing the Interaction a run of the net representing the StateMachine?

We say that an Interaction is a x -run ($x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$) of a StateMachine iff

$$\langle \llbracket \text{Interaction} \rrbracket_{\text{Run}}, \llbracket \text{Interaction} \rrbracket_{\text{Initial}} \rangle \overset{x}{S} \supseteq \langle \llbracket \text{StateMachine} \rrbracket_{\text{SM}}, \llbracket \text{StateMachine} \rrbracket_{\text{Initial}} \rangle.$$

Similarly, one can say that the set of runs specified by a Collaboration are indeed runs of a given StateMachine, iff

$$\text{tuple} \llbracket \text{Interaction} \rrbracket_{\text{Run}}^i, \llbracket \text{Interaction} \rrbracket_{\text{Initial}}^i \overset{x}{S} \supseteq \langle \llbracket \text{StateMachine} \rrbracket_{\text{SM}}, \llbracket \text{StateMachine} \rrbracket_{\text{Initial}} \rangle$$

for all $i \in \llbracket \text{Collaboration} \rrbracket_{\text{Ids}}^{\text{ia}}$.

In general, the parameter S is the denotation of the set of Signals shared among the two systems, i.e. in the first case, we would have $S = \llbracket \text{StateMachine}_1 \rrbracket_{\text{Signal}} \cap \llbracket \text{StateMachine}_2 \rrbracket_{\text{Signal}}$, and similar for the other cases.

We have seen before, that under certain circumstances, groups of collaborating StateMachines may be joined together as a concurrent region each (see e.g. page 74). This is the case for Protocols, for example, but also for the free Ports of a Configuration. Analogously, Interactions are composed of individual threads. So, there are two possible relationships between such a StateMachine and its Interactions. On the one hand, every complete Interaction may be a run of the joint StateMachine. On the other, each individual thread may be a run of the respective individual StateMachine (i.e. concurrent region).

10.1.2. Well-behavedness of a ProtocolRole

We consider a Port to be well-behaved, if it can repeat its ProtocolRole over and over again, that is, if from its initial state, it can reach only states that may eventually return to the initial state again. In the Petri-net terminology this is the home-property for the initial state. Formally, this property can be expressed as

$$\llbracket \text{ProtocolRole} \rrbracket_{\text{Terminal}} \subseteq \mathcal{R}_N(\mathcal{R}_N(\llbracket \text{ProtocolRole} \rrbracket_{\text{Initial}}))$$

where $N = \llbracket \text{ProtocolRole} \rrbracket_{\text{SM}}$. Furthermore, one might require that the state space be finite.

In some cases, the state space is neither finite, nor may the initial marking be reached again exactly, but a larger marking. This is the case, for instance, if a variable (i.e. place in the net) is used to count the runs, and a token is added for every run. The above property might be generalized as

$$\forall m \in \mathcal{R}_N(\underline{m}) : \exists m' \in \mathcal{R}_N(m) : \underline{m} \subseteq m'.$$

These properties can be applied to all ProtocolRoles: those of Ports, of ConnectorEnds, or of Protocols of Styles. Almost all analysis techniques are suitable for checking them (e.g. P- and T-invariants, model-checking, state-space traversal).

10.1.3. "Implements" relation

Intuitively, both specification and implementation are valid descriptions of a system. The specification, however, is less specific, that is, it has more degrees of freedom than its implementations. On the other hand, an implementation may not change anything that is determined by the specification. Applied to the three behavioral semantics we considered in Section 8.2.3, this means that a specification must (a) generate all the sequential traces, (b) simulate all choices, or (c) be contained as a subset of the partial words of the implementation, respectively. These notions are formalized by the relations $\overset{x}{\geq}$, where $x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$, and S is a set of Signals shared by both the specification and the implementation, i.e.,

$$S = (\llbracket \text{SM}_i \rrbracket_{\text{Out}} \cap \llbracket \text{SM}_s \rrbracket_{\text{Out}}) \cup (\llbracket \text{SM}_i \rrbracket_{\text{In}} \cap \llbracket \text{SM}_s \rrbracket_{\text{In}}).$$

So, we say that a StateMachine SM_i implements a StateMachine SM_s , exactly if $\langle \llbracket \text{SM}_i \rrbracket_{\text{SM}}, \llbracket \text{SM}_i \rrbracket_{\text{Initial}} \rangle \overset{x}{\geq} \langle \llbracket \text{SM}_s \rrbracket_{\text{SM}}, \llbracket \text{SM}_s \rrbracket_{\text{Initial}} \rangle$.

10.1.4. "Is-equivalent" relation

We say that two StateMachines SM and SM' are equivalent wrt. x , exactly if

$$\langle \llbracket \text{SM} \rrbracket_{\text{SM}}, \llbracket \text{SM} \rrbracket_{\text{Initial}} \rangle \overset{x}{=} \langle \llbracket \text{SM}' \rrbracket_{\text{SM}}, \llbracket \text{SM}' \rrbracket_{\text{Initial}} \rangle,$$

where $x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$ specifies the notion of concurrency. Obviously, this is the case, iff SM and SM' implement each other.

For a ProtocolRole, the implements and is-equivalent relations need to consider the signal-sets of the role, too. Formally, we have that ProtocolRole PR_i implements ProtocolRole PR_s with respect to a behavioral semantics x (written $\text{ProtocolRole}_i \overset{x}{\geq} \text{ProtocolRole}_s$) iff

$$\begin{aligned} & \wedge \quad \begin{array}{l} \llbracket \text{PR}_i \rrbracket_{\text{In}} \subseteq \llbracket \text{PR}_s \rrbracket_{\text{In}} \\ \llbracket \text{PR}_i \rrbracket_{\text{Out}} \subseteq \llbracket \text{PR}_s \rrbracket_{\text{Out}} \end{array} \\ & \wedge \quad \langle \llbracket \text{PR}_i \rrbracket_{\text{SM}}, \llbracket \text{PR}_i \rrbracket_{\text{Initial}} \rangle \overset{x}{\geq} \langle \llbracket \text{PR}_s \rrbracket_{\text{SM}}, \llbracket \text{PR}_s \rrbracket_{\text{Initial}} \rangle \end{aligned}$$

Again, two ProtocolRoles are equivalent, and thus freely interchangeable, if they both implement each other. One could additionally require, that both ProtocolRoles terminate at the same time, that is, whenever ProtocolRole_1 reaches its terminal state after executing some sequence of steps, ProtocolRole_2 must have reached its own terminal state, too. Formally, this means that

$$\forall_{w \in \mathcal{S}^*} : \llbracket \text{ProtocolRole}_i \rrbracket_{\text{Terminal}} \subseteq \mathcal{R}_{N_i}(\llbracket \text{ProtocolRole}_i \rrbracket_{\text{Initial}}, w) \implies \overline{m_s} \in \mathcal{R}_{N_s}(\underline{m_s}, w)$$

where $N_i = \llbracket \text{ProtocolRole}_i \rrbracket_{\text{SM}}$.

10.2. Consistency within views

In this section, we consider properties that can be proved or disproved by considering a group of ModelElements correlated by a single view. Recall that while a diagram usually belongs to a single view, most views consist of more than one diagram. So, intra-view consistency may well be inter-diagram consistency.

There are no semantic consistency conditions for the implementation, deployment, domain viewpoints, since they have no formal meaning in our approach so far. There are formal semantics for the structure and behavior viewpoints.

All of these consistency conditions are intended for use with component-Capsule: some are applicable also for styles, but projects typically do not satisfy any of them.

10.2.1. Interfaces

The interfaces view just consists of a set of Ports, each with its ProtocolRole. Apart from the well-behavedness conditions of these ProtocolRoles in the sense of Section 10.1.2, there are no further consistency conditions.

10.2.2. Functionality

The functionality view consists of context diagrams with Actors, UseCases, their relationships, and possibly interaction diagrams as scenarios of the UseCases. Since there is no formal semantics for (proper) Actors, there are no consistency conditions. If there are Ports instead of Actors, further consistency conditions can be checked (see Section 10.3.5 below).

10.2.3. Behavior

A behavior-view contains a StateMachine and possibly a Collaboration with several Interactions that are supposed to be runs of the StateMachine.

1. If there is only a StateMachine, we require that it terminates, i.e.

$$\llbracket \text{Capsule} \rrbracket_{\text{Terminal}}^{b, \text{sm}} \subseteq \mathcal{R}_{\llbracket \text{Capsule} \rrbracket_{\text{SM}}^{b, \text{sm}}}(\llbracket \text{Capsule} \rrbracket_{\text{Initial}}^{b, \text{sm}}).$$

2. If there is also a Collaboration, its Interactions (if any) are supposed to be runs of the StateMachine. Formally, for all Interactions such that $\llbracket \text{Interaction} \rrbracket_{\text{ID}} \in \llbracket \text{Capsule} \rrbracket_{\text{IDs}}^{b, \text{ia}}$ it must be true that

$$\langle \llbracket \text{Interaction} \rrbracket_{\text{Run}}, \llbracket \text{Interaction} \rrbracket_{\text{Initial}} \rangle \overset{x}{\mathcal{S}} \supseteq \langle \llbracket \text{Capsule} \rrbracket_{\text{SM}}^{b, \text{sm}}, \llbracket \text{Capsule} \rrbracket_{\text{Initial}}^{b, \text{sm}} \rangle.$$

3. The composition of the behaviors of the Capsules, Ports and Connectors of a Configuration must be well-behaved, i.e.

$$\text{well-behaved}(\llbracket \text{Capsule} \rrbracket_{\text{Sim}}, \llbracket \text{Capsule} \rrbracket_{\text{Initial}}^{\text{sim}}).$$

We might strengthen this condition so that the initial state is a home state, requiring that $\llbracket \text{Capsule} \rrbracket_{\text{Terminal}}^{\text{sim}} \subseteq \mathcal{R}_N(\mathcal{R}_N(\llbracket \text{Capsule} \rrbracket_{\text{Initial}}^{\text{sim}}))$, where $N = \llbracket \text{Capsule} \rrbracket_{\text{Sim}}$.

10.2.4. Structure

A structure view consists of a Configuration, a ConfigurationSpace and binding Connectors. We focus on the static aspect, and defer styles till Section 10.4.1. Assume for the following that a set $\{P_1, \dots, P_n\}$ of Ports are connected by a n -ary Connector with ConnectorEnds $\{CE_1, \dots, CE_n\}$ (see Figure 10.1 for an illustration). The following conditions must hold.

3. The ConnectorEnds of the Connector must match each other. That is, on the one hand, every outgoing Signal of some ConnectorEnd must be understood as an incoming Signal by some other ConnectorEnd. On the other hand, the ensemble of the behaviors of the ConnectorEnds must be well-behaved. So, formally, it must be true that

$$\forall 0 < i \leq n : \llbracket CE_i \rrbracket_{\text{Out}} \subseteq \bigcup_{j \in \{1, \dots, n\} \setminus i} \llbracket CE_j \rrbracket_{\text{In}}$$

and $\text{well-behaved}(\bigcup_{i=0}^n \llbracket CE_i \rrbracket_{\text{SM}})$.

4. The ProtocolRoles of the ConnectorEnds must specialize the ProtocolRoles of the Ports to which they are attached, i.e. $\forall_{0 < i \leq n} : \langle \llbracket \text{CE}_i \rrbracket_{\text{SM}}, \llbracket \text{CE}_i \rrbracket_{\text{Initial}} \rangle \overset{x}{\mathcal{S}} \geq \langle \llbracket \text{p}_i \rrbracket_{\text{SM}}, \llbracket \text{p}_i \rrbracket_{\text{Initial}} \rangle$. Again, x is determined by the application. As a rule of thumb, we have $x = \mathcal{B}$ ($x = \mathcal{C}$) only if the characteristics of the Connector indicate a synchronous (asynchronous) connection.

Note that these two properties can not be expressed as a single one, e.g. using the Ports only, as such a condition would not be defined for *short-circuited* Ports. For a binary Connector these properties amount to behavioral equivalence.

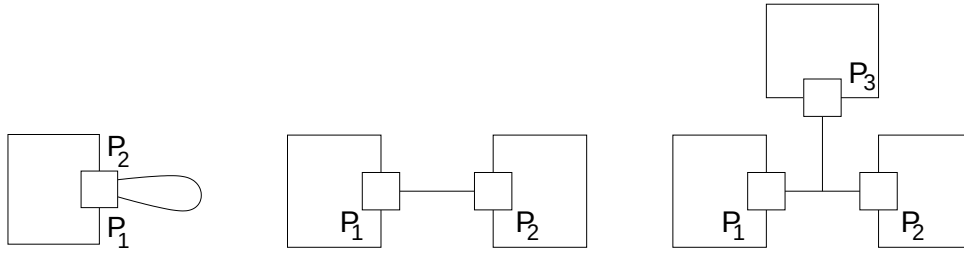


Figure 10.1.: Examples of relationships between ProtocolRoles of ConnectorEnds in a Configuration: a short circuited Connector (left), a simple binary, and a ternary Connectors (middle and right). Note that the labels p_i refer to the ProtocolRoles of the ConnectorEnds, not the Ports.

10.2.5. Protocol

Protocols are not Views, but they resemble the structure View of a Capsule, so that the respective constraints may be applied (with appropriate changes). A Protocol consists of a set of ProtocolRoles $\{\text{PR}_1, \dots, \text{PR}_m\}$, and a Collaboration between them. Assume that $\text{Collaboration.interaction} = \{\text{IA}_1, \dots, \text{IA}_n\}$. Then, the following condition must hold.

5. The Interactions of the Collaboration must be a run of the composition of the StateMachines of the ProtocolRoles. We have:

$$\forall_{i \in \{1, \dots, n\}} : \langle \llbracket \text{IA}_i \rrbracket_{\text{Run}}, \llbracket \text{IA}_i \rrbracket_{\text{Initial}} \rangle \overset{x}{\mathcal{S}} \geq \bigcup_{j=1}^m \langle \llbracket \text{PR}_j \rrbracket_{\text{SM}}, \llbracket \text{PR}_j \rrbracket_{\text{Initial}} \rangle.$$

For a change consider the following example of this property. Assume that in some system, there is a binary SimpleConnector between Ports A and B. The Ports are specified with a ProtocolRole each. Since it the Connector is a SimpleConnector, its actual behavior may be described as a single STD. This situation, together with the semantics of the respective StateMachine, is shown in Figure 10.3.

Additionally, the Connector is specified with a Protocol, one of whose sample runs we select. The diagram is presented with its semantic interpretation in Figure 10.2.

So, we have a StateMachine for the actual behavior and an Interaction for the desired behavior,² and one may wonder whether the latter really is a run of the former.² This is exactly the situation described in greater semantic detail in Section 9.4.5. A pragmatic situation where this problem might arise will be shown in Section 12.2.3.

²Hopefully, the reader actually does *not* really face this question.

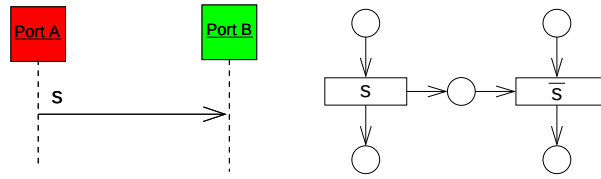


Figure 10.2.: A sample run of the Protocol.

10.3. Consistency among views

10.3.1. Overview

In this section we define consistency conditions among pairs of different viewpoints, see Figure 10.4 for an illustration. The other viewpoints are not (yet) formal, so there are no properties of this level of precision.

A Capsule has up to four different behavioral specifications of increasing level of detail: the functionality, interfaces, behavior, and structure viewpoints. Referring to the same architectural unit, they must be consistent, of course. Recall the contents of these views wrt. behavior. The functionality viewpoint contains sample runs of the Capsule's external behavior. The interfaces viewpoint contains the entire external behavior, and parts of the internal behavior, i.e. it restricts the internal behavior without specifying it completely. The behavior viewpoint, on the other hand, contains the complete specification of the internal behavior together with some sample runs of it. Finally, the structure viewpoint can be used to derive a complete specification of the internal behavior again, as the interfaces of the Capsules of its Configuration are a complete specification of their respective Capsules external behavior.

10.3.2. Interfaces vs. structure

The structure view of a Capsule contains a Configuration. In order to use the Configuration, there must be Connectors between Ports of the Capsule and Ports of the Configuration, i.e. what is called binding in the physical metamodel. For simplicity, we will refer to these as internal and external Ports of a Capsule, respectively. It is also possible, that there are Connectors inside a Capsule, that connect external Ports of that Capsule directly. We will call these *bridges*.

Bridges must be treated just like ordinary Connectors (see Section 10.2.4). For bindings, however, less strict constraints suffice. Consider Figure 10.5. In the first case (left), the bridge connects two Ports that are on equal footing, so they (or rather: their ProtocolRoles) have to be (behaviorally) equivalent wrt. the Signals they share.

In the second case (Figure 10.5, middle), P_1 may restrict P_2 . So, it is sufficient to require that P_1 implements P_2 . The conditions must be formulated differently for the third case (Figure 10.5, right). Here, there are several internal and external Ports involved. Observe, that the binding specifies explicitly, that the Ports belong to the same event space. Thus, the StateMachines of their ProtocolRoles may be put in parallel as concurrent regions of a StateMachine, or, speaking on the semantical level, the respective nets may be simply united.

6. For Connectors used as n -ary bridges ($n > 1$), the same conditions as for ordinary n -ary Connectors must hold, see above. Figure 10.5 (left) shows a binary bridge, for which we require that $\langle \llbracket P_1 \rrbracket_{SM}, \llbracket P_1 \rrbracket_{Initial} \rangle \stackrel{x}{=} \langle \llbracket P_2 \rrbracket_{SM}, \llbracket P_2 \rrbracket_{Initial} \rangle$.

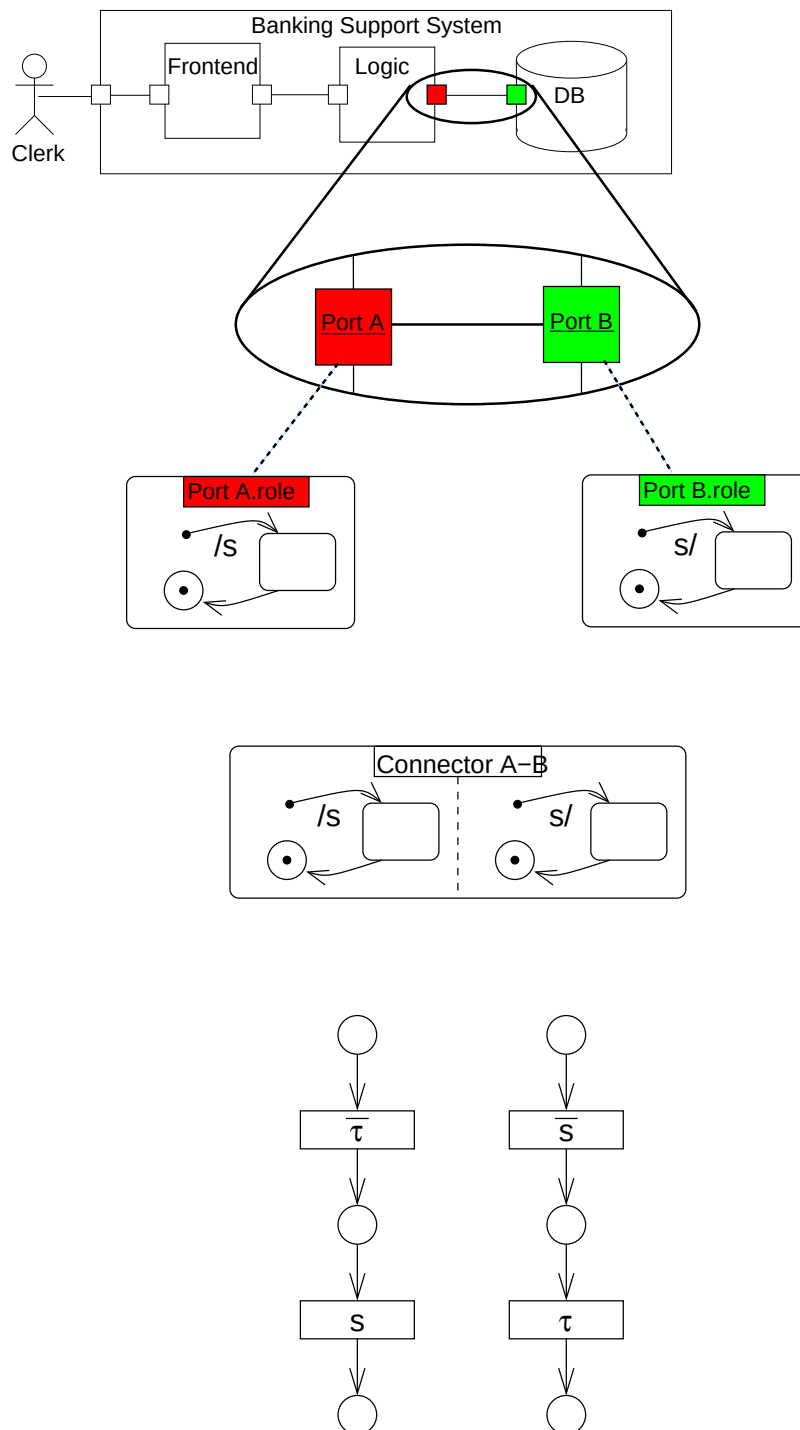


Figure 10.3.: The actual behavior of the Connector. See Section 9.4.5 for a strict interpretation of this model, and Section 12.2.3 for a pragmatic motivation of this case.

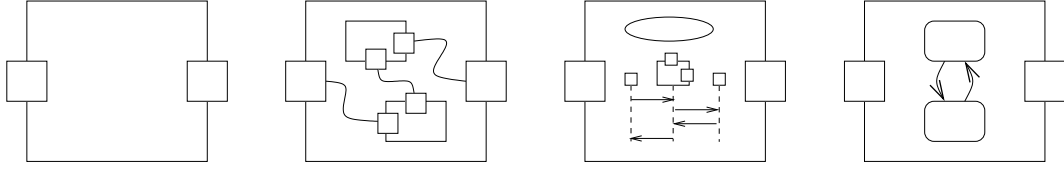


Figure 10.4.: Symbolic representations of the four Views that we have equipped with (partial) formal semantics, so that they are amenable to inter-View consistency checking: interface, structure, functionality and behavior (left to right).

7. For a Connector used as a binary binding, the ProtocolRole of the external Port P_1 may specialize the ProtocolRole of the internal Port P_2 , i.e. $\langle \llbracket P_1 \rrbracket_{SM}, \llbracket P_1 \rrbracket_{Initial} \rangle \overset{x}{S} \geq \langle \llbracket P_2 \rrbracket_{SM}, \llbracket P_2 \rrbracket_{Initial} \rangle$. Figure 10.5 (middle) shows a binary binding.

8. For Connectors used as n -ary bindings, the sets of internal and external Ports may be considered together, by putting them in parallel and applying the same condition as before, i.e.

$$\langle \llbracket P_1 \rrbracket_{SM} \cup \llbracket P_2 \rrbracket_{SM}, \llbracket P_1 \rrbracket_{Initial} + \llbracket P_2 \rrbracket_{Initial} \rangle \overset{x}{S} \geq \langle \llbracket P_3 \rrbracket_{SM} \cup \llbracket P_4 \rrbracket_{SM}, \llbracket P_3 \rrbracket_{Initial} + \llbracket P_4 \rrbracket_{Initial} \rangle.$$

Figure 10.5 (right) shows a n -ary binding. Again, we have

$$S = (\llbracket SM_i \rrbracket_{Out} \cap \llbracket SM_s \rrbracket_{Out}) \cup (\llbracket SM_i \rrbracket_{In} \cap \llbracket SM_s \rrbracket_{In}).$$

and $x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$.

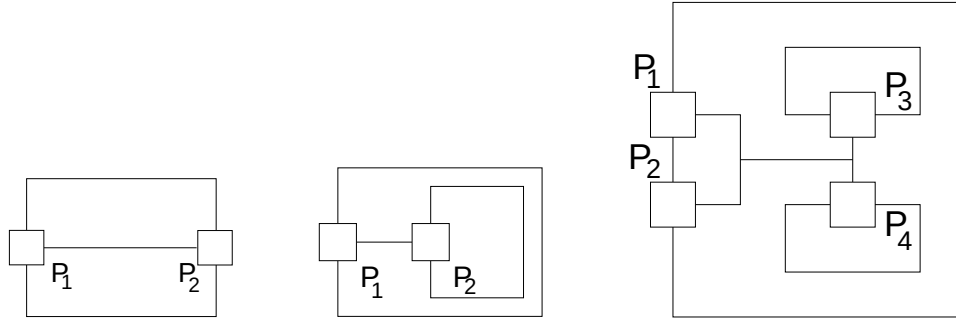


Figure 10.5.: Different kinds of Connectors within a Capsule: a bridge Connector (left), a binary binding (middle), and a n -ary binding (right).

10.3.3. Interfaces vs. behavior

The StateMachine of the behavior view may be embedded into the behavioral environment of the Capsule, yielding a net N with

$$N = \text{Synch}(\llbracket \text{Capsule} \rrbracket_{SM}^{b,sm} \cup \bigcup_{i \in I} \llbracket \text{Port}_i \rrbracket_{SM}, \llbracket \text{Capsule} \rrbracket_{Signals}^{b,sm})$$

and initial and terminal markings $\underline{m}$ and $\overline{M}$:

$$\underline{m} = \sum_{i \in I} \llbracket \text{Port}_i \rrbracket_{\text{Initial}} + \llbracket \text{Capsule} \rrbracket_{\text{Initial}}^{b, \text{sm}}$$

$$\overline{M} = \sum_{i \in I} \llbracket \text{Port}_i \rrbracket_{\text{Terminal}} + \llbracket \text{Capsule} \rrbracket_{\text{Terminal}}^{b, \text{sm}}.$$

where $I = \llbracket \text{Capsule} \rrbracket_{\text{Bdg}}^{\text{internal}}$ identify the Ports of the sub-Configuration that are bound to the Ports of the Capsule.

9. First, the behavior embedded in the environment must terminate under its initial marking, i.e. $\overline{m} \in \mathcal{R}_N(\underline{m})$. For some applications, one might even require that the initial state is always reachable again, i.e. $\underline{m} \in \mathcal{R}_N(\mathcal{R}_N(\underline{m}))$
10. Also, the runs specified in the behavior run must be runs of this system, i.e. for every Interaction I in the behavior view, it must be true that $\langle \llbracket \text{Interaction} \rrbracket_{\text{SM}}, \llbracket \text{Interaction} \rrbracket_{\text{Initial}} \rangle \overset{x}{S} \triangleright \langle N, \underline{m} \rangle$, where $x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$, determined by the application. S is the set of all Signals shared by the ProtocolRoles of the Ports and the StateMachine of the behavior view, i.e. $S = \llbracket \text{Capsule} \rrbracket_{\text{Signals}}^{b, \text{sm}} \cap (\cup \llbracket \text{Capsule} \rrbracket_{\text{In}}^{i, \text{port}} \cup \llbracket \text{Capsule} \rrbracket_{\text{Out}}^{i, \text{port}})$.

10.3.4. Behavior vs. structure

The structure and the behavior view represent two different realizations of a Capsule: while the latter represents the *how*, the former represents the *what*. There is a relationship between them that is quite similar to that between the interfaces view on the one hand, and the structure and behavior view on the other. Thus, the externally visible behavior of the structure view and the behavior view can be related.

11. First of all, the behavior specified by the behavior view (if any) must be equivalent to the behavior exhibited by the sub-Configuration via the Ports of the structure view that are bond to the Ports of the interfaces view of C , i.e.

$$\langle \llbracket C \rrbracket_{\text{SM}}^{b, \text{sm}}, \llbracket C \rrbracket_{\text{Initial}}^{b, \text{sm}} \rangle \overset{x}{S} \triangleright \langle \llbracket P_i \rrbracket_{\text{SM}}, \llbracket P_i \rrbracket_{\text{Initial}} \rangle.$$

where $I = \llbracket \text{Caps} \rrbracket_{\text{Bdg}}^{\text{external}}$ and $x \in \{\mathcal{L}, \mathcal{B}, \mathcal{C}\}$. Here, S is the set of all Signals common the Ports of the Capsule and its Ports, i.e., $S = \llbracket \text{Capsule} \rrbracket_{\text{Signal}}^{b, \text{sm}} \cap \llbracket \text{Capsule} \rrbracket_{\text{Signal}}^{i, \text{port}}$.

12. Similarly, the static structure (if present) may be validated by the scenarios of the behavior view. For all Interactions IA , such that $\llbracket IA \rrbracket_{\text{ID}} \in \llbracket C \rrbracket_{\text{IDs}}^{b, \text{ia}}$, we have that

$$\langle \llbracket IA \rrbracket_{\text{Run}}, \llbracket IA \rrbracket_{\text{Initial}} \rangle \overset{x}{S} \triangleright \bigcup_I \langle \llbracket P_i \rrbracket_{\text{SM}}, \llbracket P_i \rrbracket_{\text{Initial}} \rangle$$

where $\{P_i\}_{i \in I}$ is the family of all those Ports of the structure view of C which are bond to Ports of C , i.e., $\{P_i\}_{i \in I}$ are the Ports whose identifiers are in $\llbracket C \rrbracket_{\text{Bdg}}^{\text{internal}}$. Usually, x will be $\mathcal{C}$, since the free Ports are independent, and thus concurrent, and $S = \llbracket \text{Port} \rrbracket_{\text{In}} \cup \llbracket \text{Port} \rrbracket_{\text{Out}}$.

10.3.5. Functionality vs. interfaces

Recall that Port is a subclass of Actor, so that the Actors of a system may be specified in greater detail in our approach than is customary for Actors (i.e. with ProtocolRoles). If the Actors of a Capsule are not Ports, no semantic analysis can be carried out. Otherwise, the following conditions can be verified.

13. The Ports of Capsule C constitute a behavioral specification of the environment of C as far as C is concerned. Since all Ports are independent, the environment is described concisely by the concurrent composition of the behaviors of the Ports, i.e. $\bigcup_{0 \leq i \leq n} \llbracket \text{Port}_i \rrbracket_{SM}$. Then, the scenarios of the UseCases must be runs of the behavioral environment. So formally, for all Interactions IA with $\llbracket \text{IA} \rrbracket_{ID} \in \llbracket \text{Capsule} \rrbracket_{Ids}^{f,ia}$ it must be true, that

$$\langle \llbracket \text{IA} \rrbracket_{Run}, \llbracket \text{IA} \rrbracket_{Initial} \rangle \overset{x}{S} \supseteq \bigcup_{i \in I} \langle \llbracket P_i \rrbracket_{SM}, \llbracket P_i \rrbracket_{Initial} \rangle.$$

where S is the set of all Signals of the Ports, i.e. $S = \bigcup \llbracket \text{Capsule} \rrbracket_{In}^{i, \text{port}} \cup \llbracket \text{Capsule} \rrbracket_{Out}^{i, \text{port}}$. For non-trivial Capsules, x will be $\mathcal{C}$, for low-level Capsules (see Section 12.3), it might well be the case that the Capsule is sequential, and so, $x = \mathcal{L}$ would be appropriate.

10.4. Architectural refinement

We speak of architectural refinement, if one Capsule is replaced by a more elaborated Capsule. Typically, two distinct Capsules will be involved, but it is also possible, that a later version refines an earlier version of one and the same Capsule. So, pragmatically, we distinguish between three kinds of architectural refinement:

Elaboration: a partial specification of a Capsule may be completed, e.g. by providing additional details in existing or newly added views. For instance, the behavior and the structure view can be seen as specifying the what and the how of a Capsule's behavior, i.e. they address different levels of abstraction: the structure view can be seen as a refinement of the behavior view. When there are consistency conditions for (groups of) development activities, the progress of the development process may be checked for consistency.

Exchange: a Capsule may be replaced by another Capsule which is in some sense more specific, detailed, or in some sense more appropriate. For example, a patched or enhanced version of a Capsule may be substituted for the original one.

Abstraction: a style is used to comment a Capsule, or rather, elements of the Capsule are interpreted as elements of a given style. Conceptually, a style is a kind of template Capsule. So, it may checked for consistency according to the conditions presented above. Then, each time a style is applied, its consistency proofs can be reused, provided the abstraction itself is correct.

Technically, an elaboration is not a refinement, but rather a modification. Abstraction and exchange, on the other hand, are refinements proper: an abstraction refinement specifies a style, i.e. a template for the Capsule under scrutiny, and an Abstraction. An exchange refinement specifies a more detailed refinement for the current Capsule, and an Abstraction. See Figure 10.6 for the three different cases. Within elaboration-refinement, there are again several different possibilities. Depending on which view is elaborated, one may distinguish between functional, behavioral, structural and interface elaboration, where (parts of) the functionality, behavior, structure, and interfaces view are added, respectively. Combinations of these are of course also possible.

10.4.1. Correct Abstraction between Capsules

All the architectural refinements we have discussed above result in an Abstraction between ModelElements, in particular between Capsules. The correctness of the Abstraction can be checked formally.

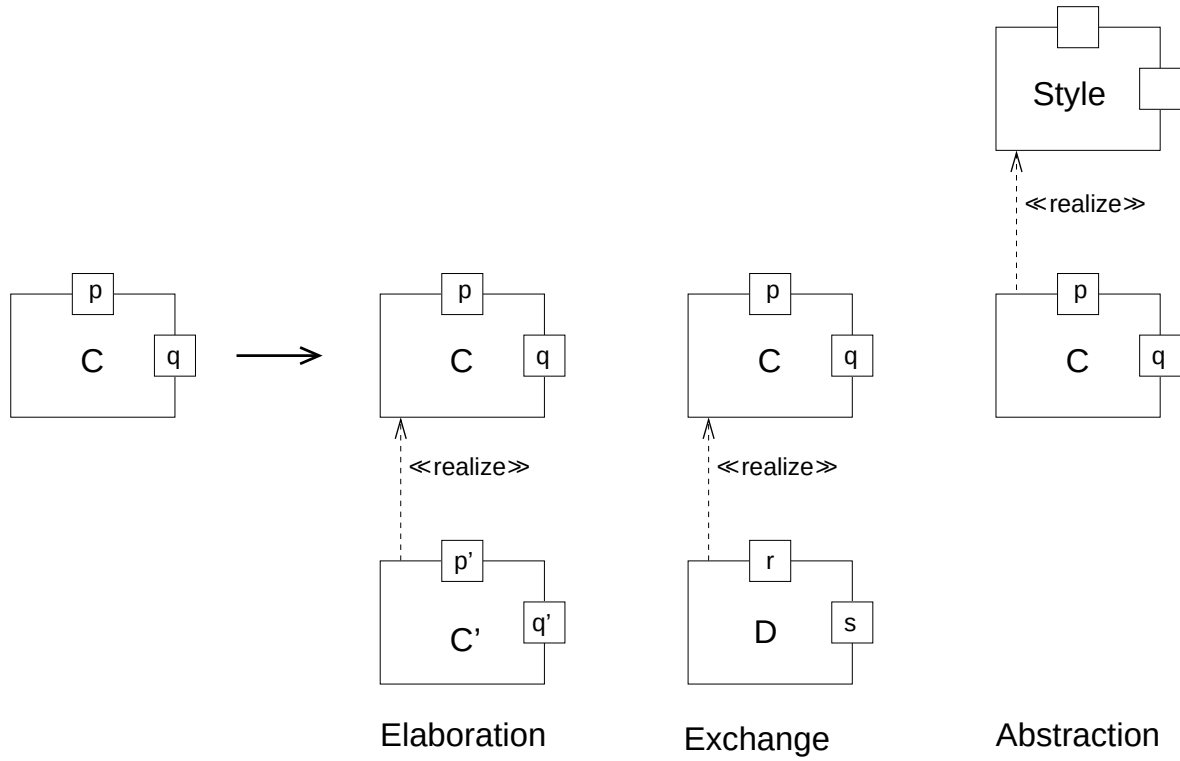


Figure 10.6.: Different usages leading to Abstractions. Starting from a Capsule C, consider three cases (left to right): C' is an elaboration of C; D is a replacement for C with possibly completely different internal structure; and finally, C may itself be a specialization of some Style.

We consider only «realize»-Abstractions between Views, Caspules, and from Views to Caspules, but not between other ModelElements contained by them. Any relationships between, say, StateMachines and Interactions of two related Capsules have to be established by the mapping of the Abstraction. We have seen an example of such a relationship in Figure 5.27 in Section 5.5.5 already. See Sections 11.3, 11.3, 11.3 and 11.3 on when and how such relationships are established and used during development.

The rule R transforms an Abstraction into a triple $\langle s, i, \rho \rangle$, where s and i are the identifiers of the two Capsules playing the role of specification and implementation, respectively. ρ is a partial mapping from ModelElements of the specification to ModelElements of the implementation:

$$\rho : \llbracket \text{Spec} \rrbracket_{\text{Ids}} \rightharpoonup \llbracket \text{Impl} \rrbracket_{\text{Ids}}.$$

In the special case where $\text{dom}(\rho) = \llbracket \text{Spec} \rrbracket_{\text{Ids}}$ we say that the Abstraction is *complete*, and call the specification an *embedding* of the implementation. If $\forall_{i \in \text{dom}(\rho)} : \llbracket \text{Spec} \rrbracket_{\text{KIND}}^i \subseteq \llbracket \text{Impl} \rrbracket_{\text{KIND}}^{\rho(i)}$ we say that ρ respects types, and that the Abstraction is homogenous, otherwise it is heterogeneous. Since the UML does not provide any syntax or semantics for mappingExpression, these properties can not be checked by OCL.

In the remainder of this section, we assume for simplicity, that mappings relate StateMachine and Interactions, but not individual Transitions, Messages or Signals. Such a generalization is of course possible and could prove quite useful, but requires additional technical and methodological effort.

10.4.2. Homogenous Abstraction among Views

A homogenous Abstraction between two Views features a mapping between two Views with the same kind. This implies that the two Views belong to different Capsules, which we call C_s and C_i . The mapping is homogeneous. Abstractions using the following Views can be checked:

14. For the interfaces View, the mapping relates Ports. We require that for all $j \in \text{dom}(\rho)$ we have

$$\langle \llbracket C_i \rrbracket_{SM}^j, \llbracket C_i \rrbracket_{Initial}^j \rangle \overset{x}{S} \geq \langle \llbracket C_s \rrbracket_{SM}^{\rho(j)}, \llbracket C_s \rrbracket_{Initial}^{\rho(j)} \rangle,$$

where $\langle \text{client} \mapsto \text{supplier}, \rho \rangle = \llbracket \text{Abstraction} \rrbracket_R$, and *client* and *supplier* are Views (cf. Section 10.4.6).

15. For the behavior View, the mapping relates StateMachines. Condition 14 applies again.
16. The structure Views may be related, too. Interpreting only the free Ports of sub-Configurations. Condition 14 applies again.

Abstraction between structure Views can be analyzed formally. However, we currently do not deal with structural refinements. There are no further constellations that can be treated at this point.

Homogenous Abstractions among Views arise for instance, when one Capsule is replaced by another, View by View. It also arises, when elaborating a Capsule, i.e. when two versions of the same View are related to each other.

10.4.3. Homogenous Abstraction among Capsules

A homogenous Abstraction between two Capsules features a mapping between the (same kinds of) Views of the Capsules. Condition 14 applies again.

Homogenous Abstractions between two Capsules arise for instance, when one Capsule is replaced by another wholesale, ignoring the Views. It also arises, when elaborating a Capsule, i.e. when two versions of the same Capsule are related to each other. These two cases are used in the "Replace Capsule" and "Realize Capsule"-patterns described in Sections 11.3 and 11.3, respectively.

10.4.4. Heterogenous Abstractions among Views

A heterogenous Abstraction between two Views features a mapping between the Views of the Capsules. Depending on the Views involved, various of the previous consistency conditions apply. The direction of the Dependency is irrelevant here.

17. if the Abstraction relates interface and structure View, the conditions of Section 10.3.2 apply.
18. if the Abstraction relates interface and behavior View, the conditions of Section 10.3.3 apply.
19. if the Abstraction relates functionality and interface View, the conditions of Section 10.3.5 apply.
20. if the Abstraction relates behavior and structure View, the conditions of Section 10.3.4 apply.

Heterogenous Abstraction among Views arise for instance, when a Capsule is elaborated by adding detailed or specific Views to complement coarse or abstract Views, e.g. when adding a behavior View to a functionality View, or when adding a structure View to a behavior view.

10.4.5. Heterogenous Abstraction among Capsules

A heterogenous Abstraction among Capsules features a mapping between the Views of the Capsules. Heterogenous Abstraction among Capsules arise for instance, when one Capsule is replaced by another, more detailed one (e.g. when different Views of the two Capsules are to be related). The same conditions as in Section 10.4.4 apply.

10.4.6. Style vs. Capsule

Conceptually, a style is a role of a Capsule. So, on the one hand it can be analyzed for consistency like a normal Capsule. On the other hand, using a style is exactly the same as replacing one Capsule by another, so the conditions of Section 10.4.3 apply again.

21. A Connector must match its template in the Protocol of the Style (if there is any), that is, each the ProtocolRole of each ConnectorEnd of the style reification must be equivalent to the ProtocolRole of the corresponding ConnectorEnd of the Style. The correspondence is established of course by the mappingExpression of the Abstraction. So, for all $role \in \llbracket \text{Connector} \rrbracket_{\text{Ids}}^{\text{pr}}$ we must have

$$\langle \llbracket \text{Style} \rrbracket_{\text{SM}}^{\text{p(role)}}, \llbracket \text{Style} \rrbracket_{\text{Initial}}^{\text{p(role)}} \rangle \overset{x}{S} \geq \langle \llbracket \text{Connector} \rrbracket_{\text{SM}}^{\text{role}}, \llbracket \text{Connector} \rrbracket_{\text{Initial}}^{\text{role}} \rangle$$

where $\langle id_{\text{impl}} \mapsto id_{\text{spec}}, \rho \rangle = \llbracket \text{Abstraction} \rrbracket_{\text{R}}$ (cf. also Section 14).

22. The respective Connector must implement its Protocol, i.e. the ProtocolRoles of the Ports $port_i$ must be a behavioral subtype of the corresponding ProtocolRoles of the Protocol.
23. The sample runs of the Protocol must be partial runs of the composition of the automata of the respective Ports.

10.5. Adequacy

Similar to the adequacy of the semantical mappings for models, the adequacy of the translation of properties is an important issue. However, which translation of a property is adequate depends to a large degree on the application. We have consistently defined three different variants for sequential, interleaving and true concurrency semantics, and there is no other argument to motivate a choice between them than practical experience—which we still lack. The same is true for the “well-behavedness”. On the other hand, it is obvious that, say, the “is-run-of” relationships really do capture the associated intuition.

One hard and fast reason to decide between the different possibilities is the computational effort involved in determining the formal relationships. This is closely related with the degree of compositionality achieved.

10.6. Compositionality

The semantical objects generated from the behavioral models can become quite large, in particular when considering the strict semantics of StateMachines, i.e., including both an event queue and a context. Recall that the latter are high-level nets, i.e. if the state space needs to be computed, one is faced with possibly very large state spaces. There are different approaches to tackle this problem.

The standard approach to achieve compositionality is to define a custom-made formalism or restrict or modify an existing formalism. We need not augment the massive body of research that has been piled up along these lines in the previous decades.³

The low-level approach to compositionality may result in unnatural models, however, and thus interfere with adequacy. Also, it is insufficient for large systems, that require state spaces well beyond what is technically feasibly today (and tomorrow).

Still, an abstract structure for the state spaces of models is needed. Obviously, introducing such a structure by hand is incompatible with our goal of making the formal methods applied completely transparent (and is expensive, as highly skilled personnel is required). Generating this structure automatically, on the other hand is computationally impossible.

So, we pursue a different approach: we exploit the compositional structure of systems that is provided by user. To this end, we have provided special modeling concepts like Capsule and Port that are geared to achieve maximal independent subsystems, thus enabling compositional operation on them. This way, we only have to deal with small models, within a Capsule or Port. When combining several Capsules to a Configuration, the number of externally visible states is significantly reduced, as the Ports of the Capsule present only *their own* in and out-Signals (and behavior) to the outside, while hiding those of the Configuration. So, large scale models of the *interaction* of several Capsules need not burden themselves with the complexity of the behavior or structure views of Capsules. Thus, analysis methods like those described in Kindler [1995], Schmidt [1996] become applicable. *This is a direct consequence of the way we have defined the concept of architectural unit.*

10.7. Chapter summary

In this chapter, we have defined a range of properties of architectural models like consistency between or within views, correct abstraction/refinement, and correct implementation of an architectural style. We have defined these properties in a mathematically precise way. They can be viewed as a kind of proof obligation. Provided the models are reasonably small, most of them can be discharged of mostly automatically by, say, model checkers or theorem provers. For the few remaining cases, manual work is unavoidable.

In the following Chapter, we shall show how these consistency conditions may be used practically in the construction and maintenance of an architectural model.

³For example, the *Connect*-operator formalizes basically the same notion as [Kindler, 1995, Def. 4.11].

11. The architectural process

We have considered typical scenarios in architectural modeling in Section 3.4. In this chapter, we elaborate these scenarios to fragments of a development process, that is, snapshots of a design flow. We describe these fragments as process patterns, referring to the concepts, notations, semantics, and properties we have presented in the previous chapters. This chapter, thus, may be read as an illustration of the material presented so far.

11.1. Introduction

According to a standard textbook, the “*software (development) process is a set of activities and associated results which produce a software product*” (cf. [Sommerville, 1996, p. 7]). It is generally accepted today, that an orderly development process has a fundamental importance to the quality and administrative controllability of creating and maintaining software. There are several rather elaborated standards for improving the software process (like the *V-Model* and the Capability Maturity Model (CMM), see GMD [1997], Uthke [1997] and CMU SEI CMM, respectively), and industrial methods like Syntropy, Catalysis, Fusion, or UP and RUP (see Coleman et al. [1993], Cook and Daniels [1994], D’Souza and Wills [1999], Jacobson et al. [1999], Rational RUP, respectively).

11.1.1. Characteristics of architectural processes

We will now consider the differences between traditional and truly architecture-centric processes. The following four main issues are genuinely characteristic for component-based and/or architecture centric processes. For further discussion see e.g. Braun [1999], Cooke [1994], Deiters et al. [1994], Foote and Opdyke [1995], Hesse [1997], Lai [1994], Moreton [1990].

Artifacts There are a number of new artifacts in an architecture-centric process, e.g. reusable components, styles, and product lines to be used as a framework or template, but also an explicit specification of the context. These artifacts must be catered for in the development process. It is not sufficient to subsume them under other kinds of artifacts: they play the lead role, so they must be first class citizens. We have described these artifacts in Part I.

Roles, Activities Dually, there are also new activities and associated roles, mainly for these new artifacts, but not exclusively. For instance, a Capsule may be procured from a third party specialized in the creation and maintenance of reusable components. Alternatively, it may be created in-house, or its creation may be handed over to a sub-contractor. Depending on which variant is chosen, the obtained Capsule may need evaluation or adaptation. The whole field of quality assurance of the various kinds of Capsules comes into view. Also, reusable components must be created at some point. Similarly, activities related to the creation and usage of styles and plas have to be defined. We spell out these activities in detail below in Section 11.3.

Organization Obviously, there is a very close relationship between the software process and the organization that is its substrate, so that with different roles, artifacts, and activities, the organization must be adapted, too (cf. e.g. CMU SEI CMM, Goldberg [1998], Katayama [1990], Tully [1988]). In particular in the area of software reuse, quite elaborated, and empirically calibrated process models have been proposed (see e.g. Bayer et al. [1999], Braun [1999], Matsumoto et al. [1991]). In the long run, if architecture-based software construction is widely applied, new professional specializations will emerge similar to those found in building architecture today, as we have outlined in our vision.

Process structure A component-based architecture has, with necessity, a recursive structure: it consists of components that are again made up of components, as it is the case for Capsules. In other words, it exhibits self-similarity on all scales, that is, it is fractal. Obviously, such a structure can not be adequately managed by a straight or iterative process. For instance, developing complex sub-systems is really a development project of its own, that is, to some degree, the software process exhibits the same structure as the software created. So, a development process with a fractal structure is needed (see Bergner et al. [1998], Foote [1993], Foote and Opdyke [1995], Hesse [1996, 1997] and Lehman [1980] where the term recursive is used instead of fractal).

None of the methods and standards mentioned above take these particularities of the architecture level of software into account adequately.

11.1.2. Domains of pattern languages

We have outlined the characteristics of architecture-centric software processes above. Addressing the last issue, we think that such processes are best described using process patterns, since “*patterns can exist at all scales*” Alexander [1979], which means that the same pattern can be applied over and over again, at several levels of abstraction. So, the overall structure becomes simpler and more manageable. Note that the *structure* of the process determines its *temporal* progression only partially: if the structure were a tree, there are several regimes of traversing it, e.g. a waterfall-like regime would correspond to a sequential breadth-first traversal of the structure. In a nutshell, a *pattern* is a proven solution to a recurring practical problem. This definition deliberately makes no restriction as to what phenomena may be described by patterns, and what notation is to be used doing this. In fact, patterns can be postulated for almost every walk of life. A pattern is always part of a system of related patterns: a pattern language. In computer science, there are at least three kinds of pattern languages in use: apart from the usual result patterns Buschmann et al. [1998], Gamma et al. [1995], there are also organizational Coplien [1995], Harrison [1995] and process patterns Aalst et al. [2000], Ambler [1998a,b], Bergner et al. [1998], D’Souza and Wills [1999], Foote and Opdyke [1995], Foote and Yoder [1995], Kerth [1995], Stevens and Pooley [1998], Wells [1995], Whitenack [1995]. See [Coplien and Schmidt, 1995b, pp. xi-xii] for a brief historical overview of patterns in general and in computer science. Irrespective of the application domain, the main benefit of patterns lies in the additional abstraction level they provide: by structuring an artefact using patterns, communication about that pattern is improved for anyone familiar with the patterns employed, and thus the artifact’s understandability and maintainability is enhanced. Concerning the software process in particular, structuring it in patterns allows for problem-oriented, ad-hoc modifications of the process without sacrificing clarity and manageability. Using process patterns leads to modular development processes. Using a pattern-structured development process has the additional advantage that one may choose not to apply some pattern at some point, adding an additional degree of freedom: traditional processes

have often been complained about for stifling creativity. Of course, in the absence of proper process management, this advantage becomes a disadvantage. One defining criterion of a pattern is that it be abstracted from practical applications, i.e., that it is a seasoned, practically, proven solution, to a recurring problem, taken from real life cases. However, to some degree, patterns are bound to the languages they are expressed in. For instance, idioms depend on some programming language (usually Smalltalk, or C++), and most result patterns use some design language (usually OMT or Martin/Odell Martin and Odell [1992] class diagrams, see also Table 11.1). This is of course no problem if the source and target languages are close enough—but that need not be the case. It can be difficult to adapt, say, a C++ idiom exploiting (multiple) inheritance to a language that supports only single inheritance, or no inheritance at all. Similarly, many result patterns are probably not applicable in a, say, Structured Analysis and Design Technique (SADT) project.

11.1.3. Reconciling rigor and flexibility

A software process has to be both rigorous and flexible: rigor is needed to allow measurement, tracing, process improvement etc., but it should not cause excessive inconvenience wrt. efficient scheduling, personal preferences, or pragmatic decisions during development. Loosely specified processes offer flexibility without rigor; detailed (“heavy weight”) processes with a traditional structure offer rigor with too little flexibility, but often stifling creativity in a administrative overhead. In many non-standard situation, the process just does not fit, and causes extra effort in circumventing it. Even when organizationally enforced and tool-supported, clumsy work-arounds and insufficient usability hamper their efficiency. Structuring development processes by patterns offers a solution: patterns may be used both as generators and comments. So, the rigor and tool support of heavy-weight processes is possible, while it is also possible to improvise here and there. The balance between formality and improvisation may be adjusted for each concrete project.

11.2. Description of process patterns

11.2.1. Overall schema

Process patterns can be described by a schema similar to those known from result patterns.¹ Certain adaptations become necessary, however (see also Bergner et al. [1998]). We use the terminology described in [Gamma et al., 1995, p. 6ff]. There, the following aspects of patterns are distinguished.

Title First, each pattern has a name. For process patterns, this is usually identical to the name of the task that is supported by this pattern.

Synonyms Sometimes, there is not a unique name to a patterns, but a number of equally adequate names.

Classification There are several classification schemas for result patterns. Buschmann et al. propose to use the abstraction level and the problem category as dimensions ([Buschmann et al., 1998, p. 362ff]). The Gang-of-four propose to use the purpose and the scope as classification dimensions (see [Gamma et al., 1995, p. 9ff]). These schemas have to be adapted for process patterns. We use the following:

¹See [Gamma et al., 1995, p. 6ff] or [Buschmann et al., 1998, p. 20ff] for two of the most common schemas. A large number of different, usually less elaborated schemas can be gathered from the various articles in e.g. Coplien and Schmidt [1995b].

- Abstraction level: technique, process pattern, or development style;
- Phase: specification, design, realization, or maintenance (cf. “problem category” in Buschmann et al. [1998], see Figure 11.1, top);
- Purpose (within a given project): administration, construction, reuse, or quality assurance (cf. creational, structural, and behavioral result patterns in Gamma et al. [1995], see Figure 11.1, bottom);
- Scope: i.e. the kind of Capsule the pattern may be applied to: project, component, style, pla, or several of these.

Typically, there is no hard and fast dividing line between the values of the respective dimensions. Note that granularity, i.e. the actual or conceptual size of the architectural elements occurring in a process pattern, is not a useful classification criterion for a fractal process. There is no hard and fast discrimination between the different *simple classes* of these facettes. See Section 11.2.3 below for more details. In the remainder, the emphasis will be on the purpose “construction”.

Related Patterns Relationships to other patterns, i.e. alternatives, companions, or prerequisites and discussion of differences.

Since the respective sections already carry the name, this aspect of a pattern is dropped. The classification is shown in Table 11.2, and the relationship of patterns is provided by Figure 11.7. Thus, these three aspects do not occur in the descriptions of the individual patterns.

Intent A intuitive account of the rationale of the pattern, its benefits and application area.

Motivation A scenario that illustrates the applicability conditions and purpose of the pattern.

Consequences Discussion of the advantages and disadvantages of using the pattern, e.g. some quality goal or other end that is achieved by this pattern.

Lacking sufficient practical experience using the pattern fragments we provide, motivation and consequences are integrated into the new aspect “Discussion”.

Participants The actors, roles/resources, techniques, activities, tools and document types involved in the process, and their respective roles.

Applicability The prerequisites for applying a pattern, the context where it may be applied. In the scope Capsule, this may be specified by the required Views and possibly consistency conditions established.

Deliverables Describes what (parts of) documents are created or changed in which way by applying the pattern. In our approach, it is possible to formally define the post-condition of applying a process pattern referring to the created or modified Views that are provided and possibly consistency conditions established (see Chapter 10). See Figure 11.3 for the document-type icons used.

Process The central part of a process pattern is of course the description the process fragment itself, i.e. the individual steps and their dependencies. In result patterns, this aspect corresponds to Structure in Buschmann et al. [1998], Gamma et al. [1995]. See Section 11.2.2 below.

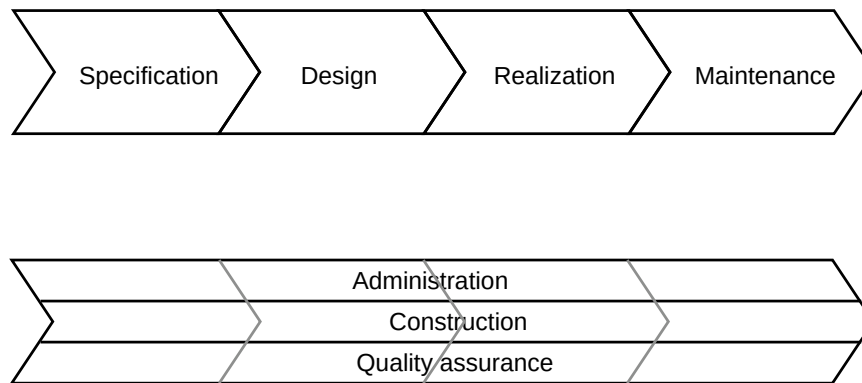


Figure 11.1.: The phases (top) and purposes (bottom) used for classification, presented in the “fish”-notation of [Jacobson et al., 1999, p. 26]. As for all classification facettes, there are no strict boundaries between the various simple classes. In particular, the phases are not intended to imply a strict sequential order.

The schema of Gamma et al. additionally provides synonyms (“Also Known As”) and references of previous applications of the pattern (“Known Uses”). Due to the novelty of our patterns, neither are applicable. The schema of Gamma et al. also includes sample code implementing the pattern (“Realization”). For process patterns, this would be an executable description representation of “Process”. In the sense of generative patterns, a tool implementation could easily operationalize activity diagrams as are used for the description of process patterns and so we omit this aspect.

11.2.2. Process modeling

For software processes, there is currently no generally accepted, adequate notation. Among others, Petri nets, Rules (pre/post conditions), activity trees/transaction graphs, planning systems, and state transition diagrams have been proposed for the description of general workflows, see Keller and Rombach [1990]. Naturally, one would like to use UML to describe design flows. However, the UML is very poor in this respect (cf. Jäger et al. [1998]). The standard proper contains nothing at all, the “UML Standard Profiles” tries to deal with the complex topic on a few pages, see [OMG UML 1.3, part 4]. The description given there is very superficial indeed. The Unified Process (UP) (see Jacobson et al. [1999]) offers a little more in terms of notations and concepts, in fact a variant of activity diagrams, and so we adopt the UP-notation here.

Workflow diagrams

The UP uses a non-standard notation to describe workflows and activities that is vaguely reminiscent of activity diagrams (see e.g. [Jacobson et al., 1999, pp. 25, 145]). The relationship between the two is not explained in the UP, however, and the notation as such is not defined at all. So we define for our purposes a special kind of activity diagrams that we call workflow diagrams (workflow diagram). Notationally, workflow diagrams are very similar to the examples of activity descriptions given in the UP, but with a precise mapping to the UML metamodel. So, the relationship between the activity and workflow diagrams is firmly established. Note that, although the outward appearance is quite different, workflow graphs are really a special case of activity diagrams, similar to activity diagrams being a special case of state transition diagrams. Workflow diagrams have three different kinds of

notational elements to represent documents, activities, and dependencies between them. Following the UP, we use a twin cogwheels-icon for activities and plain arrows for dependencies. The new icon for activities is introduced in order to set them apart from activities that have a lower level of granularity, and less or no interaction. Documents are represented as sheets with a dog's ear (see Figure 11.2 for an overview). Views are also represented as a document, but carry a large letter indicating the type of View. Views may also be presented with (some of) the documents they contain (see Figure 11.3). A “model” in UP-terminology is quite similar to a View in our terminology, and so the package-symbol used in the UP is more or less equivalent to the view-symbol. Workflow diagrams may be decorated with swimlanes to improve the layout. Note that swimlanes have no semantics. There are a number of different kinds of documents that may be presented with individual icons, see Figure 11.3. There, we show a number of different documents (small icons on the left), including a view (last row, left). Views often contain several documents, which can be presented similar to composition relationship of classes, see the large icon in Figure 11.3 on the right. The state of a document (such as the role of a Capsule) may be added in square brackets below the document icon. Views are decorated with the respective initial indicating the viewpoint.

Document types representing an UML diagram type are specified by this fact entirely already. Document types representing views are defined by the structures of the document types they contain. Other kinds of document types may or may not be specified explicitly.

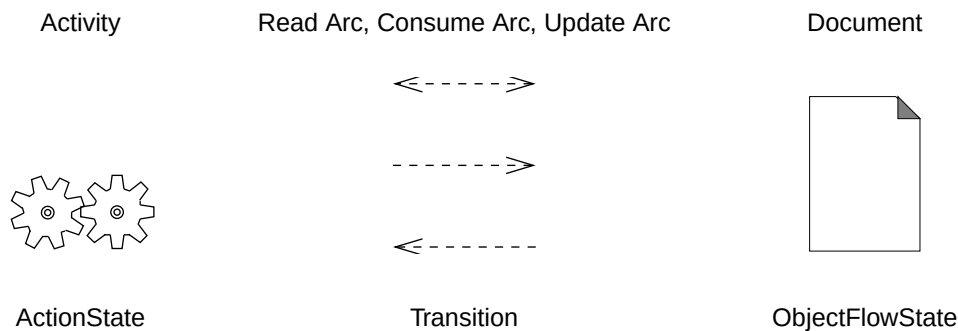


Figure 11.2.: Overview of the notational elements of workflow diagrams and their mapping to the metaclasses associated with ActivityGraph.

Intuitively, simple activity diagram can be transformed into a workflow diagram. PseudoStates and sequences of States without intermediate ObjectFlowStates are not allowed at top-level, but may occur inside of CompositeStates. The transformation simply asks us to (1) replace ControlState-icons by the “twin cog wheels”-icons; (2) replace ObjectFlowState-icons by the “twin cog wheels”-icons; and (3) remove Transitions between ControlStates, see Figure 11.4.

“Small” workflow diagrams

Workflow diagrams can also be used to describe individual activities (the standard case, when looking at the UP). In that case, there is only a single activity with its associated documents in a workflow, plus the workers involved in this activity. We call such diagrams small workflow diagrams (SWD). They are very similar to the anonymous notation in the UP (see e.g. Figure 11.6), only that their semantics is now well-defined. Therefore, it is possible to composed small workflow diagrams into one large workflow diagram by merging documents, see Figure 11.5. This way, larger processes can be created step by step directly from the descriptions of individual activities.

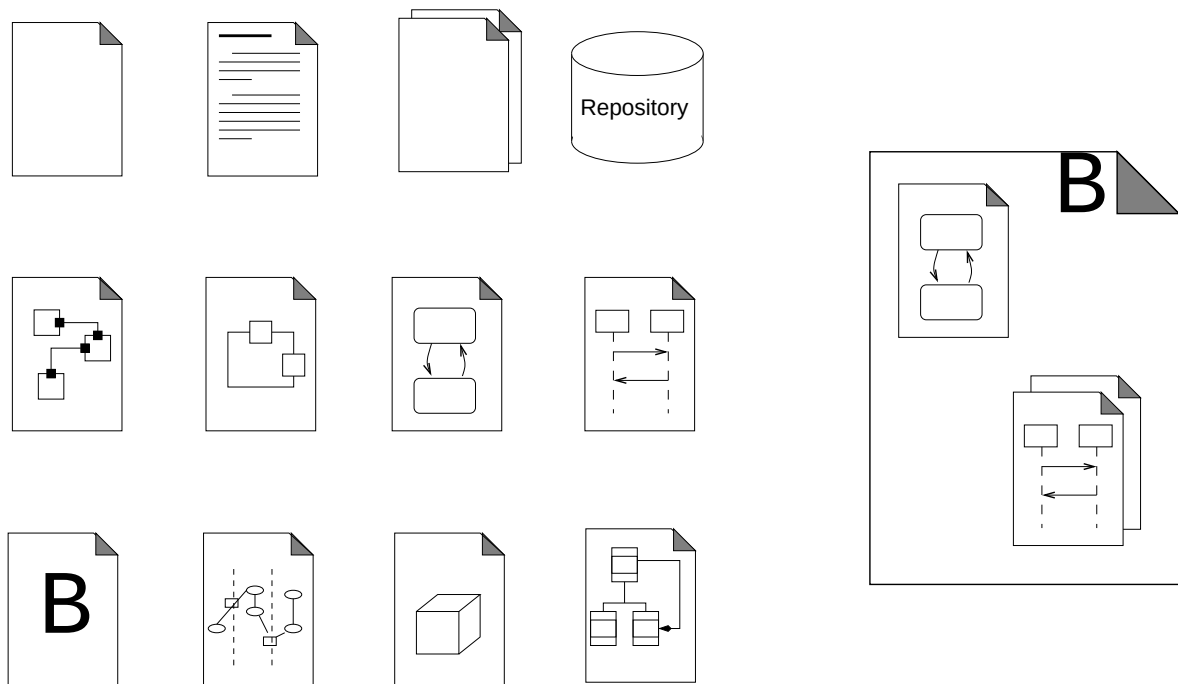


Figure 11.3.: Some types of Documents used in process patterns (normal reading order): arbitrary document, text, set of arbitrary documents, repository, capsule diagram, Capsule playing role component, state transition diagram, sequence diagram, behavior view (opaque), activity diagram, deployment diagram, class diagram. On the right, there is a transparent representation of a behavior view, containing a set of interaction diagrams and a state transition diagram.

Small workflow diagrams always refer to a project as their context, that is, the documents are understood to be elements of some View of the project. They are also decorated with UP-style workers.

11.2.3. Classification of process patterns

The other aspect in the description schema for process patterns that needs adaptation is the “Classification” aspect. We identified four dimensions in this aspect: the abstraction level, the phase, the purpose and the scope. While the latter three are obvious, the first of these is slightly non-standard, and thus needs elaboration. Buschmann et al. [1998] distinguish between three abstraction levels of patterns: idioms, design patterns, and architectural patterns. Similarly, process patterns can be separated into techniques, process patterns, and development styles. See Table 11.1 for their relationship and typical notations.

Techniques apply to individual artifacts like UML diagrams, other design documents, or code. They deal with how the particular notation or format is best created and used. Most of these techniques are folklore. Typical examples are role playing with paper prototypes (e.g. storyboards) for the creation of scenarios, structured interviews for requirements elicitation, and isolating nouns and verbs from such interviews to define analysis level domain models (see e.g. [Robinson, 1992, Chapter 3] or [Pomberger and Blaschek, 1993, pp. 37]). Process patterns are fragments of a development process schema, i.e. templates for coordinating groups of related activities and the documents involved. Process patterns refer to groups of tightly coupled artifacts, such as architectural units. In our approach,

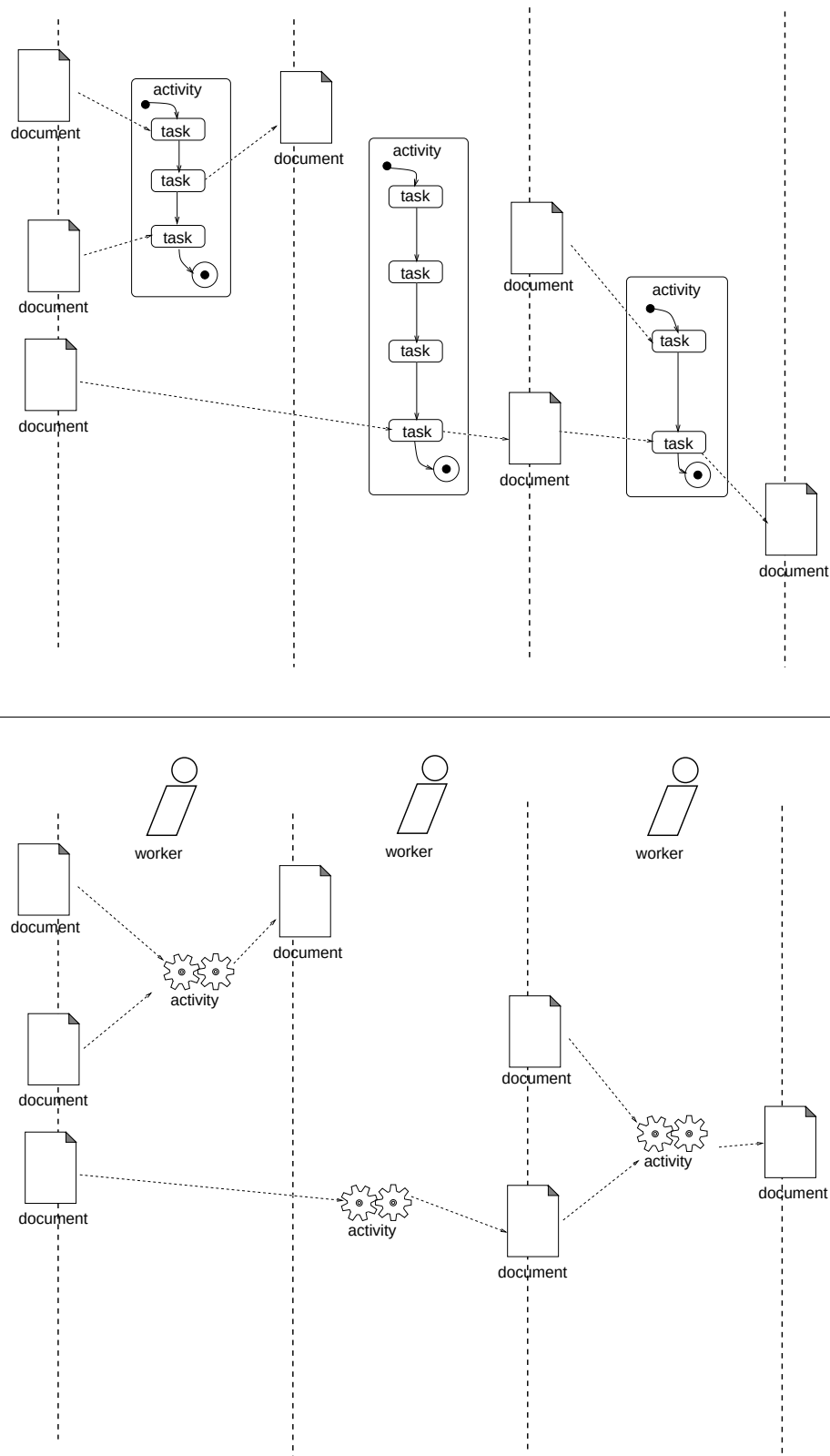


Figure 11.4.: The relationship between activity and workflow diagrams: an activity diagram describing a workflow (top); the same diagram presented as a workflow diagram (bottom).

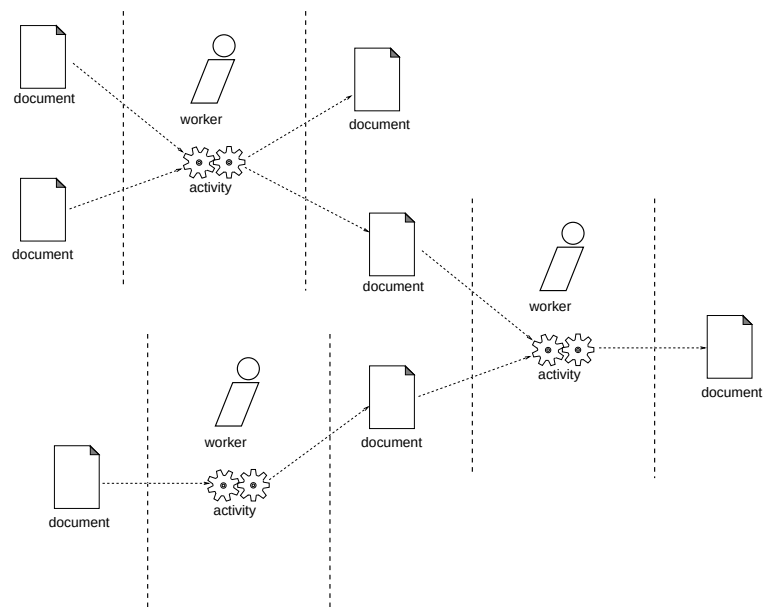
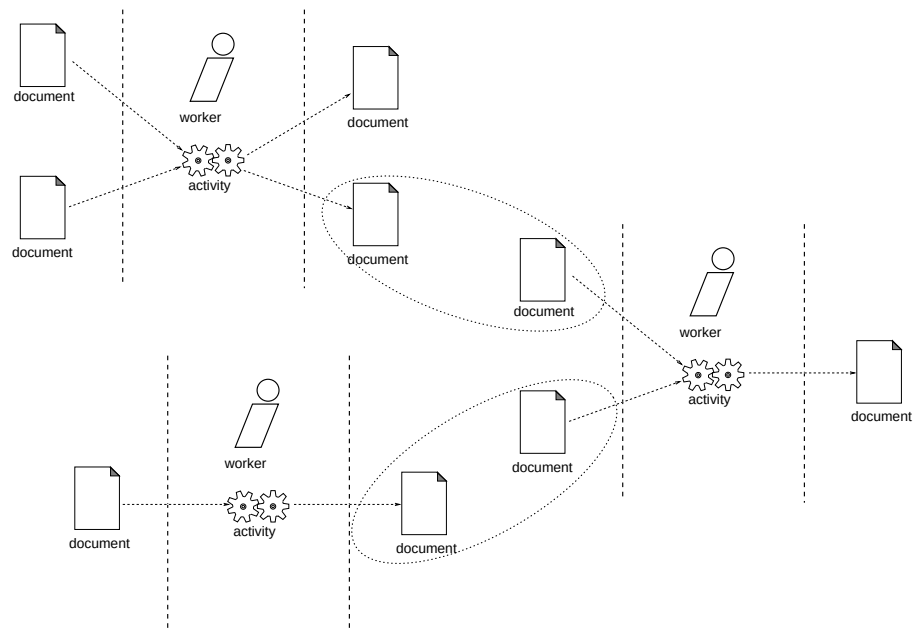


Figure 11.5.: Small workflow diagrams can be composed by merging documents, resulting in a large workflow diagram.

result pattern		process pattern	
level	typical notation	typical notation	level
idiom	programming language (C++, Smalltalk, Java)	development graph (see Hacklinger [2000])	technique
result pattern	OMT class diagrams (MVC, Proxy)	UML activity diagrams (see Foote and Opdyke [1995])	process pattern
style	box-and-line diagrams, natural language (see Section 5.5)	natural language (structured, OO, iterative, fractal)	development style

Table 11.1.: Both result and process patterns can be separated into three levels of abstraction.

a process pattern usually involves one or two Capsules. Development styles affect complete development projects or organizations. Examples are structured and object-oriented development styles, but also the “fractal” styles discussed in Hesse [1997] and Foote [1993], that are particularly well-suited for component-based development. In the remainder, we will not deal with development styles.

See Table 11.1 for a synopsis of the result and process domains of patterns, and their three levels of abstraction.

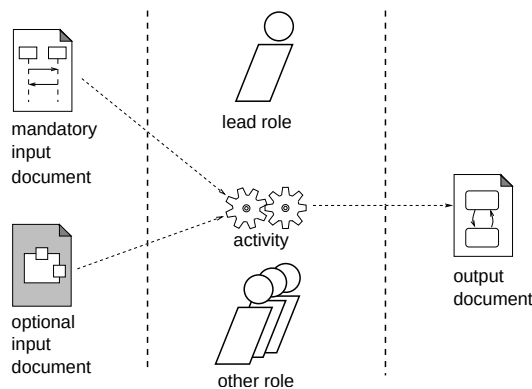


Figure 11.6.: Schematic notation of an activity according to the notational schema proposed in the Unified Process, see [Jacobson et al., 1999, pp. 25, 145]: input- and output documents (left and right), workers (top and bottom).

11.3. An architectural process pattern language

Patterns always come in families, and the patterns we present are no exception. In this section, we explain the relationship between the patterns that are defined in Section 11.3. All patterns are headed by a box giving its name and distinctive classification criteria.

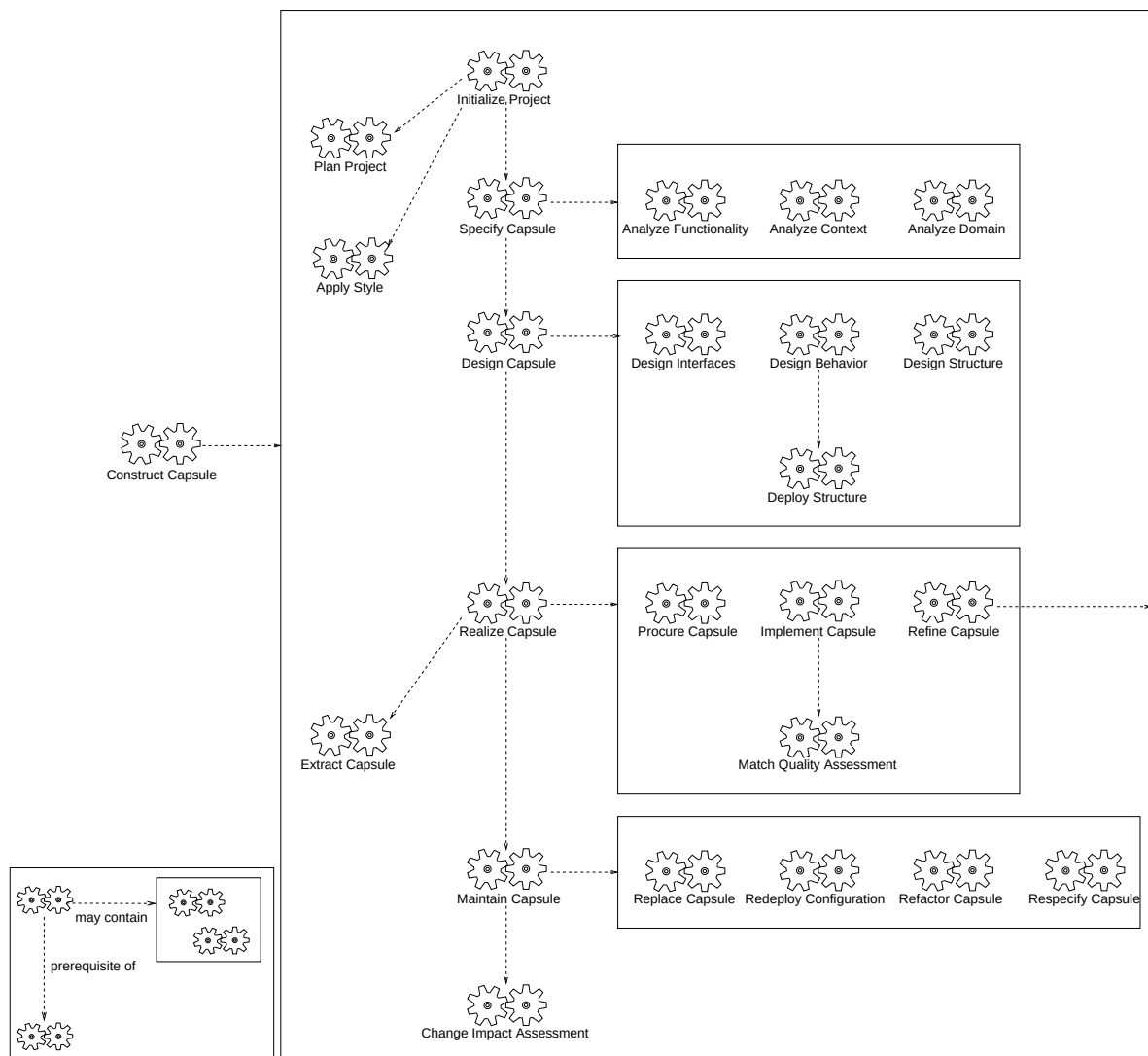


Figure 11.7.: The relationship between the patterns described in this Chapter. Arrows show which process pattern establishes the prerequisites for which other pattern. In the lower left corner, a small legend explains the graph. The arrow from “Refine” to “Construct” indicates the recursion.

Figure 11.7 shows a synopsis of all the process patterns we will present in this chapter. This graph shows the logical dependencies between the patterns. Table 11.2 shows the classification of the patterns. This table also shows which views are used and created during which activities. Every pattern is associated to one of four phases (analysis, design, realization, maintenance, though we do not mean to imply a strict allocation of a pattern to one and only one phase, or a linear sequence of phases).

No. Name	Definition	Phase	Classification Purpose	Scope	Views (required/preferred/created/updated)												
					a	c	f	d	b	i	s	de	im				
1. Initialize project	(11.3, p. 205)	misc	administration	project	c												
2. Plan project	(11.3, p. 205)	misc	administration	project	u	p	p	p	p	p	p	p	p				
3. Construct Capsule	(11.3, p. 206)	misc	construction	project	c	c	c	c	c	c	c	c	c				
4. Extract Capsule	(11.3, p. 207)	misc	reuse	Capsule	u	u	u	u	u	u	u	u	u				
5. Specify Capsule	(11.3, p. 207)	specification	construction	project	u	c	c	c									
6. Analyze context	(11.3, p. 208)	specification	construction	project	u	c											
7. Analyze domain	(11.3, p. 209)	specification	construction	project	u		c										
8. Analyze functionality	(11.3, p. 209)	specification	construction	project	u			c									
9. Design Capsule	(11.3, p. 211)	design	construction	Capsule	u	r	p	p	p	c	c	c	c				
10. Design behavior	(11.3, p. 212)	design	construction	component	u	r	p	p	p	c	p						
11. Design interfaces	(11.3, p. 212)	design	construction	component	u	r	r	p	p	p	p	c					
12. Design structure	(11.3, p. 213)	design	construction	Capsule	u	r	p	p	p	p	p	c					
13. Deploy structure	(11.3, p. 215)	design	construction	Capsule	u	r	r		p	p	r	c					
14. Apply style	(11.3, p. 216)	design	reuse, quality assurance	project	u	r	p	p	p	p	p	p					
15. Realize Capsule	(11.3, p. 217)	realization	construction	Capsule	u	p	r	p	p	r	p		c				
16. Procure Capsule	(11.3, p. 218)	realization	construction, reuse	Capsule	u	r	r	r	p	r			c				
17. Refine Capsule	(11.3, p. 219)	realization	construction	Capsule	u	r	r	p	p	r	p		c				
18. Implement Capsule	(11.3, p. 220)	realization	construction	Capsule	u	r	r	p	p	r	p		c				
19. Match quality assessment	(11.3, p. 220)	realization	quality assurance	project	u	p	r	r	p	r	p						
20. Maintain Capsule	(11.3, p. 221)	maintenance	reuse	project	u	u	u	u	u	u	u	p	p				
21. Refactor Capsule	(11.3, p. 222)	maintenance	reuse	project, component	u	r	r	p	p	u	u	p					
22. Redeploy structure	(11.3, p. 223)	maintenance	reuse	project, component	u	r	r	p	p	p	r	u					
23. Replace Capsule	(11.3, p. 223)	maintenance	reuse	project, component	u	r	r	p	p	r	u	p	p				
24. Respecify Capsule	(11.3, p. 224)	maintenance	reuse	project, component	u	u	u	u	u	p	p	p	p				
25. Change impact assessment	(11.3, p. 225)	maintenance	quality assurance	Capsule	u	r	r	p	p	p	p	p	p				

Table 11.2.: An overview over the process patterns presented in this chapter. The left block of columns contains the name of the pattern and a reference to its definition. The middle block of columns contains the classification; all patterns refer to all levels of abstraction. The right block of columns contains the views used and provided by the patterns. Recall the abbreviations: **administration**, **context**, **functionality**, **domain**, **behavior**, **interfaces**, **structure**, **deployment** and **implementation**. Concerning the views, note that “update” implies “requires”

1.: INITIALIZE project

misc / administration

Intent Initialize a project with libraries of components, plas and styles.

Discussion In the very beginning of a development project, it is usually not entirely clear, what the objective is. Thus, before any other development activity ensues, as many details of the requirements as possible must be specified. On the one hand, this extends to the list of relevant viewpoints, that has to be determined. On the other hand, if that list contains the administration viewpoint (useful for all but the tiniest of projects), it must at least provide a mission statement (the business and/or technical goals). Having made the project rationale explicit is invaluable during client validation.

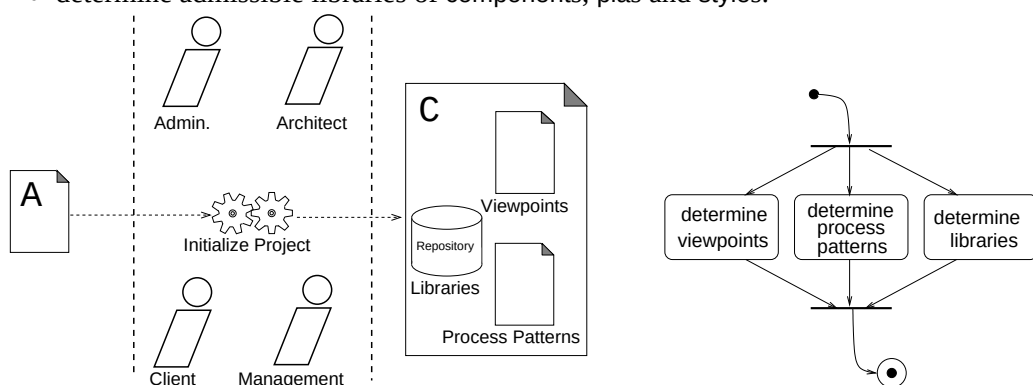
Participants Architect, Management, Development administration, Client.

Applicability An administration view has to be present, but it may be empty (i.e. in its initial state).

Deliverables Set-up project with list of relevant viewpoints; possibly sets of admissible/adapted process patterns and styles.

Process

- Starting from an empty Capsule, as a first step the relevant viewpoints of the Capsule have to be defined, and thus, to some degree, the overall quality criteria of the development project. For the sake of argument we assume that exactly those views have been agreed upon that are listed in Section 4.2.2.
- determine applicable process patterns and other project-wide guidelines
- determine admissible libraries of components, plas and styles.



2.: PLAN project

misc / administration

Intent Fix a frame for time and budget, based on estimates of gains, cost and duration.

Discussion The administration view has to be initialized and updated continuously. It is concerned with what resources are available (people, time, budget, office space, machines, tools), who takes over what roles and responsibilities, and which goals are to be pursued with which priority. It helps keeping track of the schedule.

Participants Client, Management, Development administration.

Applicability At any time. May be re-applied at any point, after changes to the specification, to the mission statement or to the side conditions.

Deliverables Drafts of schedule, and budget; cost estimates, allocation of resources.

Process The details of the planning process are beyond the scope of this book.

3.: CONSTRUCT Capsule

misc / construction

Intent This pattern realizes the lifecycle of a Capsule as given in Figure 11.1 (top). It is structured similar to the standard iterative development process (see e.g. [Pomberger and Blaschek, 1993, p. 25]). It serves to organize the other patterns, and to clarify their relationships.

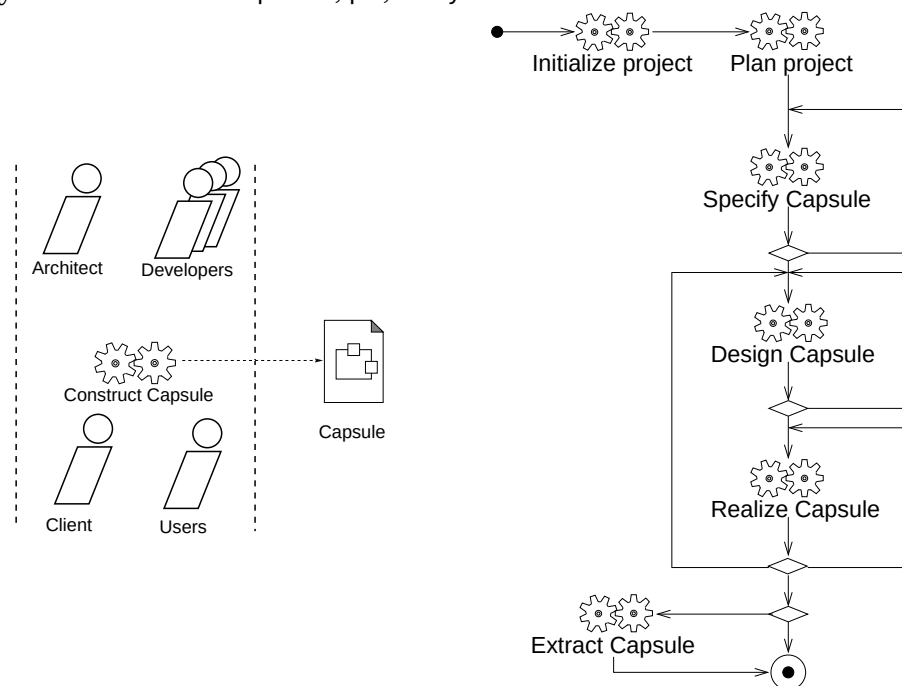
Discussion On first sight, this pattern realizes a waterfall, softened up by some iterations. However, it is recursive, as the realization stage allows for the application of the Refine Capsule pattern, which employs this pattern again.

Participants Architect, Developers, Users, possibly Client.

Applicability Whenever a Capsule is to be constructed, i.e. right after the creation of a project, and similarly when refining a Capsule.

Deliverables A complete Capsule with all relevant views defined and checked for consistency and validity. The Capsule may be declared as a component, pla, or style, possibly satisfying additional constraints.

Process First, a project must be initialized. Then it may be specified until a satisfactory specification has been reached. Similarly, the design and realization sub-processes may be iterated. It is possible to adapt the design after realization and repeat the realization step. Finally, the Capsule may be turned into a component, pla, or style.



When constructing a Capsule as part of a refinement process, the context and domain views usually need not be specified any more. Furthermore, low-level Capsules sometimes can do without any specification, and also parts of the design may be omitted, then.

4.: EXTRACT Capsule

misc / reuse

Intent Extract a new a style, pla or component from a finished project in order to make past experiences reusable.

Discussion The quality of components in a component library is a crucial factor for the effectiveness of reuse. Thus, one may not populate the library with any component—a process of careful selection, improvement, alignment to development standards and classification is required. As a rule of thumb from empirical studies, reuse pays no earlier than the third time a component is reused. This gives an indication of the effort that may be spent during extraction.

Participants Developers, Development administration.

Applicability The project from which the style is to be extracted has to be finished, or at least that portion of it from which the style is to be drawn.

Deliverables A style, pla, or component.

Process Within an old project, create new Capsule and copy to it whatever items are to be part of the abstraction. Strip project specific items and inessential bits, and execute all tests and verification steps for styles. Comment and classify style. The style may now be published, i.e., checked in into the component library.

5.: SPECIFY Capsule

specification / construction

Intent Provide a specification for a Capsule.

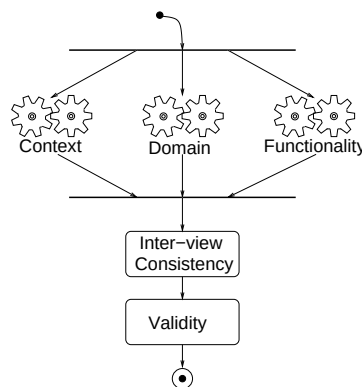
Discussion This pattern subsumes the patterns of the specification phase. It is used as a commentary, for structuring the development process. The common part of the sub-patterns could be integrated into these, of course, but this would result in a less clearly structured overall development process.

Participants Architect, Developers, Client, Users.

Applicability After a project has been initialized.

Deliverables Defined and checked context, domain and functionality views.

Process The context, domain and functionality views are defined. The overall specification is then checked for consistency and validity. In each sub-process, the intra-view consistency is checked, while checking the inter-view consistency until all views are present, that are to be there at all.



6.: ANALYZE CONTEXT

specification / construction

Intent Establish the system boundaries, determine stakeholders and technical side conditions.

Discussion The context of a system is the set of all relevant technical, organizational, business, or functional requirements and other side conditions. Examples are legacies that are to be integrated, legal regulations to be obeyed, business goals to be achieved and so on. It is important to first determine the forces effective in the context before determining their precise nature and specific influences in greater detail.

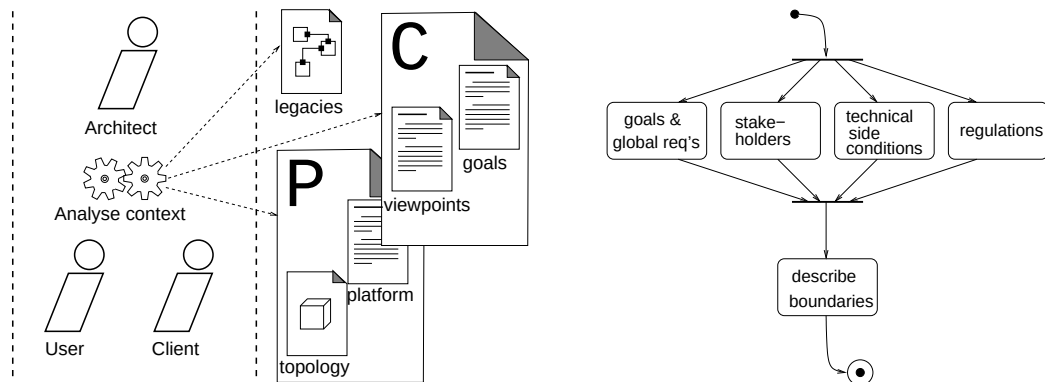
Participants Architect, Developers, Development administration, Client.

Applicability Requires a set-up project demanding a functionality viewpoint.

Deliverables Use case diagram with descriptions of the actors, set of interaction diagrams for the scenarios, initialized administration views, natural language description of business goals, roles/resources/workers, and schedule; texts about legal, technical, and organizational side-conditions and implications.

Process After the necessary information has been gathered, it can be summarized in a context diagram and associated documents.

- state business goals and global, overarching requirements;
- identify all stakeholders;
- list the technical side conditions (available tools and programming languages, target/development platform, available major components like databases, TP-monitors, middleware, etc. at the target platform);
- identify all other side conditions (legal, health or safety regulations that may apply, e.g. such as concerning privacy, security, reliability and so on);
- establish the system boundaries, resulting in a specification of the environment conditions, and a context diagram that identifies the Actors interacting with the Capsule.



7.: ANALYZE DOMAIN

specification / construction

Intent Provide structural and dynamic domain models as (1) a documentation of the capabilities of the Capsule in application domain terminology, and (2) a kind of manual.

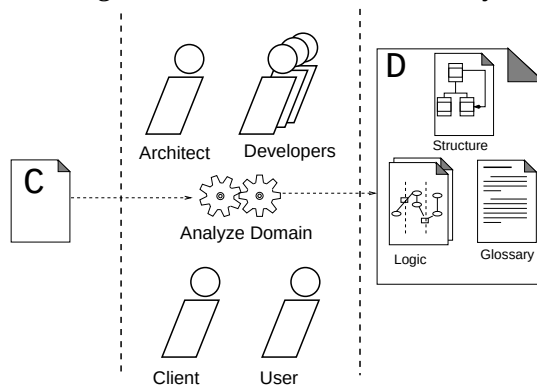
Discussion Only worthwhile if application domain is relevant for understanding of Capsule, i.e. in particular style and plas, and for non-trivial components, but not for low-level Capsules.

Participants Architect, Developers, Users, Client.

Applicability not necessary for throw-away prototypes and for elementary Capsules.

Deliverables The structural domain model is given by class diagrams. An analysis level class diagram introduces the relevant domain terminology, and clearly shows, what notions are expected from the environment, and which are provided by the Capsule itself. Additionally, an activity diagram may present the domain logic and how the Capsule's concepts may be used.

Process The structure and logic of the application domain should be analyzed in the usual ways, resulting in a number of class and activity diagrams in the respective views.



8.: ANALYZE FUNCTIONALITY

specification / construction

Intent Determine actors, use cases, and scenarios, which implies an allocation of responsibilities to resources in a domain workflow.

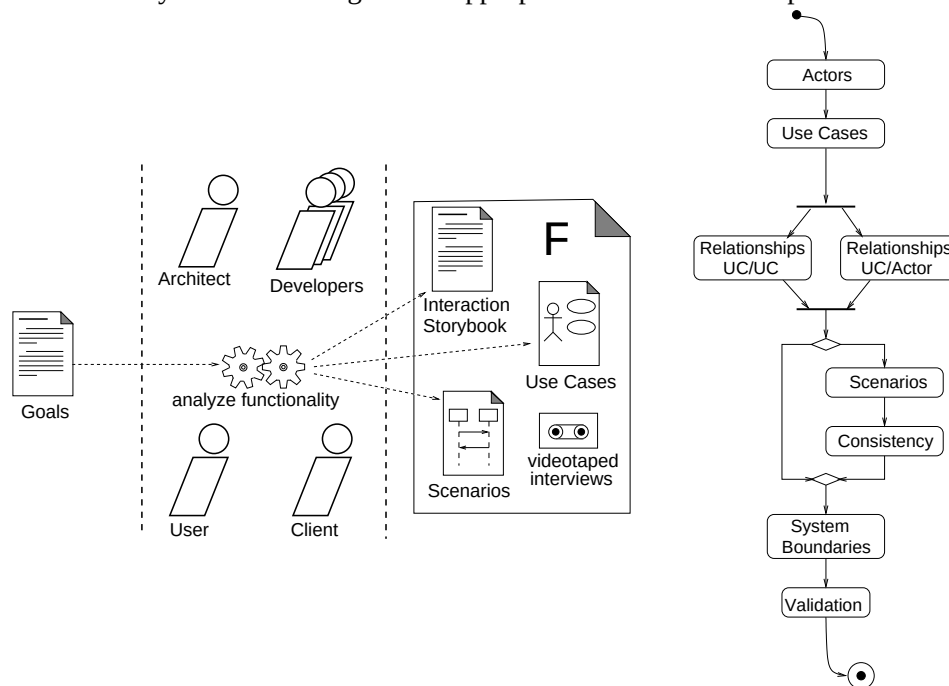
Discussion Defining the functionality of a Capsule is an important step during specification, and may be used both as a validation point and for the later phases of design and realization. Apart from the domain viewpoint, it provides the most important criteria for inclusion in a facette classification.

Participants Architect, Users, Developers, possibly Client.

Applicability Whenever the stakeholders are determined. The system boundary need not be established yet.

Deliverables Complete functionality view, including a storyboard serving as a contract with client.

Process First, the actors of the system have to be determined, and described. At this point, a natural language description will do, that will be made more precise during interface design. Then, the use cases of the systems have to be determined. Any of the standard techniques are viable, e.g. customized checklists for the descriptions of the actor's responsibilities and the use cases (see for instance Balzert [1999], Jacobson et al. [1992, 1995]). Next, the relationships among actors and use cases, respectively, need to be determined. This includes determining the initiator(s) of each use case. The use cases may be refined by usage scenarios expressed in natural language, indicating the initiating actor, non-functional requirements, parameters, pre- and post-conditions and possible exceptions. A further refinement by interaction diagrams is appropriate in all but the simplest cases.



Finally, the consistency of the functionality view has to be established, and the specification must be validated with the end-users, e.g. by role-playing with a paper-prototype, walking through a storyboard or animating the use cases. After a successful validation, the specification documents are a contract. Appropriate documentation (e.g. videotaping) is advisable for complex specifications. Such recordings can be analyzed later on, separately and off-line, also by the client. Determining the use cases implies an allocation of tasks to either the Capsule under development or its actors. As it is known at this point, which actor is a human and which is a machine, this amounts to determining what functionalities are to be provided by people. Thus, business process (re)design and staff training can be initiated at this point.

Intent Provide a design for a Capsule, that enables its subsequent realization.

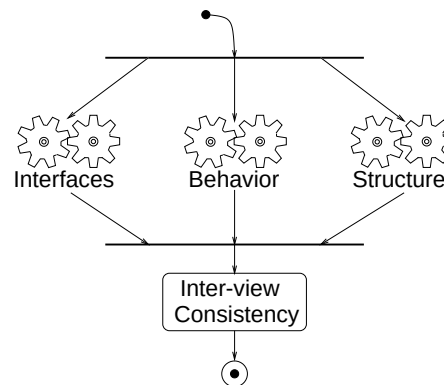
Discussion This pattern subsumes the patterns of the design phase. It is used as a commentary, for structuring the development process. The common part of the sub-patterns could be integrated into these, of course, but this would result in a less clearly structured overall development process. For high-level Capsules, specifying the behavior may be omitted. For low-level Capsules, the structure may be left out.

Participants Architect, Developers.

Applicability Whenever the context, domain, and functionality are established.

Deliverables Behavior, interfaces and structure views.

Process The order in which the views are supplied is not specified. In each sub-process, the intra-view consistency is checked, while checking the inter-view consistency until all views are present, that are to be there at all.



10.: DESIGN BEHAVIOR

design / construction

Intent Design the internal behavior of a capsule, i.e. its states and the transitions between them.

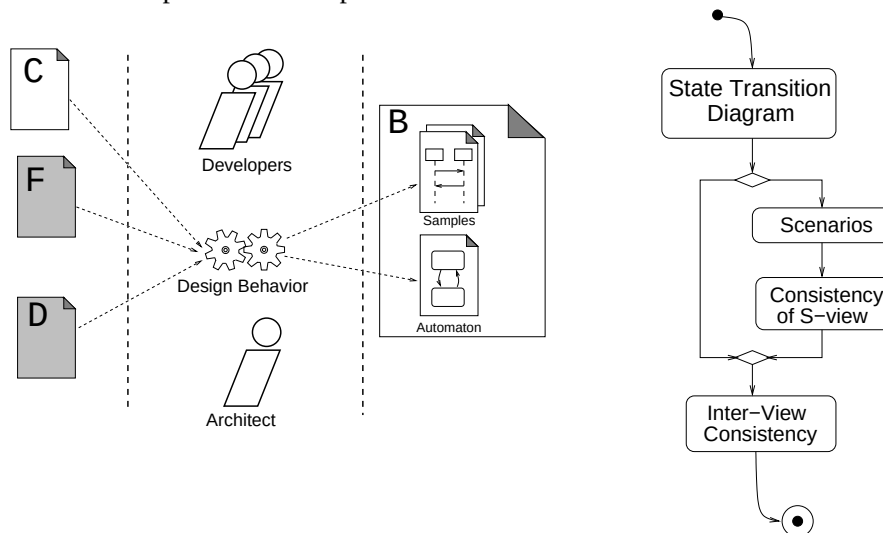
Discussion Designing the behavior of a component explicitly is helpful as additional guidance when realizing complex components. It is also useful during validation.

Participants Developers.

Applicability One may commence designing the behavior as soon as the context and functionality are established. Neither the domain nor any parts of the design are required, though it is helpful if the interfaces view is present already.

Deliverables The behavior view, including a state transition diagram, possibly with a set of interaction diagrams.

Process As a first step in the design, the intended behavior of the Capsule has to be specified. This may be done using a state transition diagram. It is inscribed by Actions and Events that need not be specified at this point.



Then, the behavior may be elaborated by scenarios with concrete values, described by interaction diagrams. These scenarios are elaborations of the scenarios in the functionality view. Finally, establish consistency within the behavior view, and between the behavior view and the functionality view, and, if already present, the interfaces and structure views.

11.: DESIGN INTERFACES

design / construction

Intent Determine the interfaces of an Capsule, i.e. its Ports and their ProtocolRoles. If possible, (specializations of) predefined ProtocolRoles should be used.

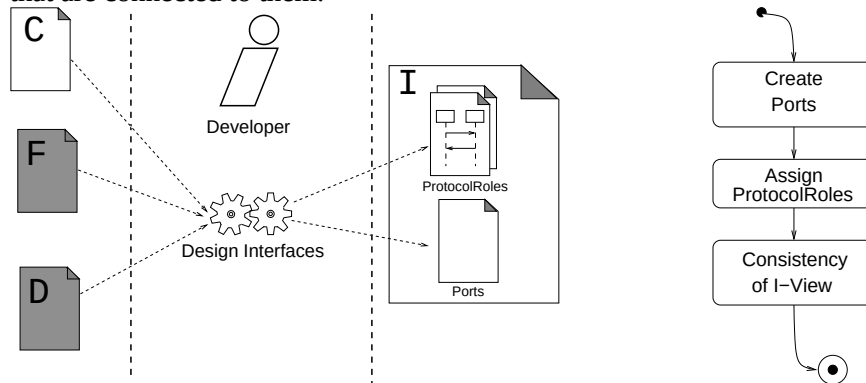
Discussion The interfaces ensure, that the component is indeed self-contained, an essential for compositional analysis (see Sections 11.3 and 11.3), and for smooth collaboration between different development teams.

Participants Developers.

Applicability Any time during design or analysis.

Deliverables The interfaces view with ProtocolRoles for each Port, and possibly interaction diagrams in the behavior view as sample runs.

Process the Ports must be derived from the scenarios. As a rule of thumb, every intersection of an Association between an Actor and a UseCase on the one hand, and the system boundary on the other becomes a Port. The scenarios of the respective UseCase have to be adapted accordingly, i.e. Actors are replaced by Ports. The sets of incoming and outgoing Signals and the behavior of a Port can be simply compiled from the scenarios of all the UseCases that are connected to them.



The intersections between the system boundary on the one hand and the associations between actors and use cases on the other are the points of interaction of the Capsule. They can be bundled according to the scenarios. Each group of points of interaction becomes a port.

- Create ports and assign ProtocolRoles conforming to the interactions described in the behavior and functionality views, and the behavior described in the behavior view.
- Establish consistency within interfaces views (i.e. have OCL-constraints checked), between the interfaces and functionality views, and, as far as already present, the other design-phase views.

12.: DESIGN STRUCTURE

design / construction

Intent Refine a Capsule structurally.

Discussion Only very simple Capsules have no further structural decomposition. Capsules with medium and high complexity, however, require refinement by a Configuration.

Participants Developers.

Applicability Requires the context and functionality views to be established beforehand. Availability of the interfaces and behavior view is useful, but not mandatory.

Deliverables Structure view, i.e. a Configuration and a set of Connectors binding its free Ports to the Ports of the Capsule.

Process Complex Capsules may be refined structurally. There are several decomposition techniques that may be applied, we discuss functional, state-based, and domain-based decomposition.

- In a functional decomposition, first the use cases and scenarios of the functionality view are refined. They are clustered according to these heuristics:

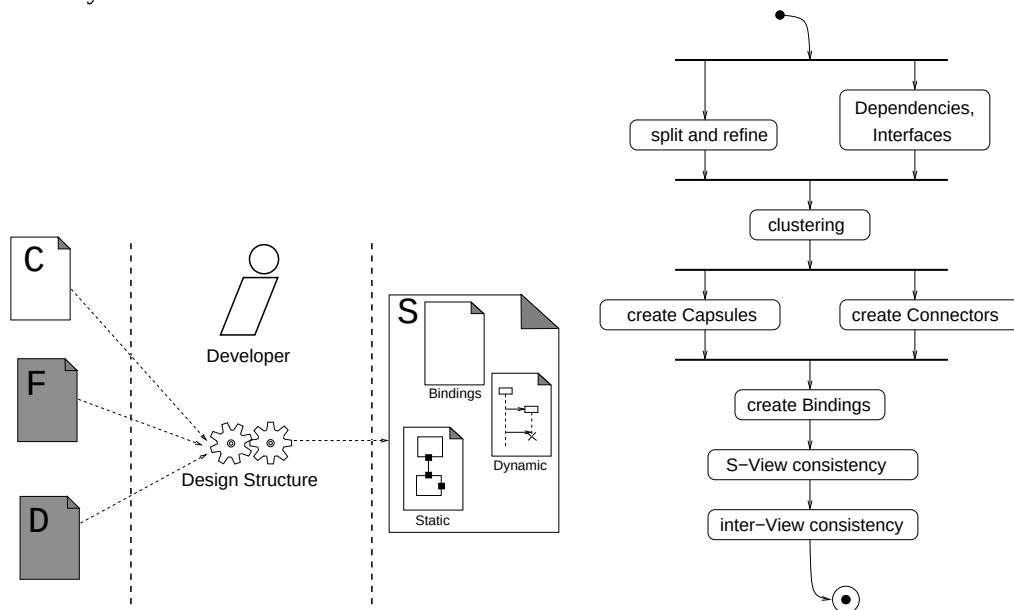
1. use cases with a Generalization between them should belong to the same cluster, if the Generalization is indeed used to express a taxonomic relationship. If it is abused (as frequently seen!) as a form of inheritance, then it really should be a Extends or Includes Relationship, and rule 2 applies;
2. if there is either an Extends or Includes Relationship between two use cases, and there is a lot of traffic between them, they should belong to the same cluster;
3. use cases that are used by several other use cases are candidates for factoring out as an individual Capsule;

Finally, each cluster is turned into an individual Capsule, and the Usages are turned into Connectors. Their ConnectorEnd's ProtocolRoles are defined so as to accommodate the (refined) scenarios.

- Alternatively, the clustering may be guided by the behavior view: the StateMachine is refined with a view to factoring CompositeStates. The regions of concurrent CompositeStates are easily turned into StateMachines of their own, representing the behavior of a potential sub-Capsule. This works similarly for sequential CompositeStates.
- Also, the clustering may be guided by the domain view, i.e., a set of class and activity diagrams. To this end, the class diagrams are split up into clusters of closely related notions and processes. These are then turned into the respective domain models of potential sub-Capsules. For other clustering-techniques see Jacobson et al. [1995], Zendler [1999].

Other influences in clustering are the structure imposed by reused Capsules or styles, or, if there already is a deployment (such as in a later iteration), the topology. Of course, combinations of these techniques are also possible.

Now the clustering has been achieved, and some clusters have been turned into Capsules, and their links into Connectors. After creating some Connectors as bindings, the structure is ready.



Intent Provide deployment view.

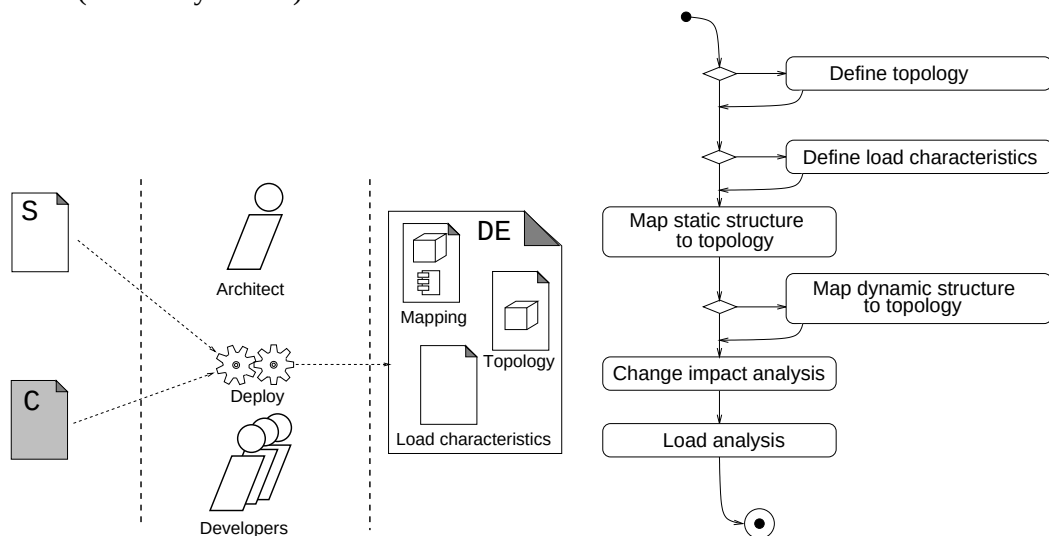
Discussion For simple static structures, the deployment is usually quite trivial, or up to empirical load analysis. For more complex or dynamic structures, it may be rather intricate, however.

Participants Architect, Developers.

Applicability At least the static part of the structure view must be defined. A topology may or may not have been defined in the specification phase.

Deliverables The deployment view, possibly including a load prediction document that may be used as a concrete test case during integration testing.

Process If a topology is not already defined, it has to be determined now. Then, the static structure is mapped to it. If defined, the dynamic structure must also be mapped to the topology. In the Activity load analysis, the mappings are analyzed quantitatively (see Huszerl and Majzik [1999], Kosmidis and Huszerl [2000], Monroe et al. [1997], Spitznagel and Garlan [1998] and Section 9.6.3). As a result, deploying a Configuration may raise new requirements about the static or dynamic structure, i.e. it may instigate the (re-)application of the “Design Structure”-pattern, the “Redeploy Capsule”-pattern or the “Refactor Capsule”-pattern (see below). Note that Capsules and Nodes must be properly contained in each other (either way round!).



14.: APPLY style

design / reuse, quality assurance

Intent Use a predefined Capsule-template to establish an outline of the following development.

Discussion Apart from reusing plas and components, it is also possible to reuse styles. Applying a style is different from applying a pla and component, though: the elements are not used directly (as for component), or with modifications (as for plas), but they are a kind of patterns that are to be filled out. So, an Abstraction is to be established.

Participants Architect, Developers, possibly Development administration, Client

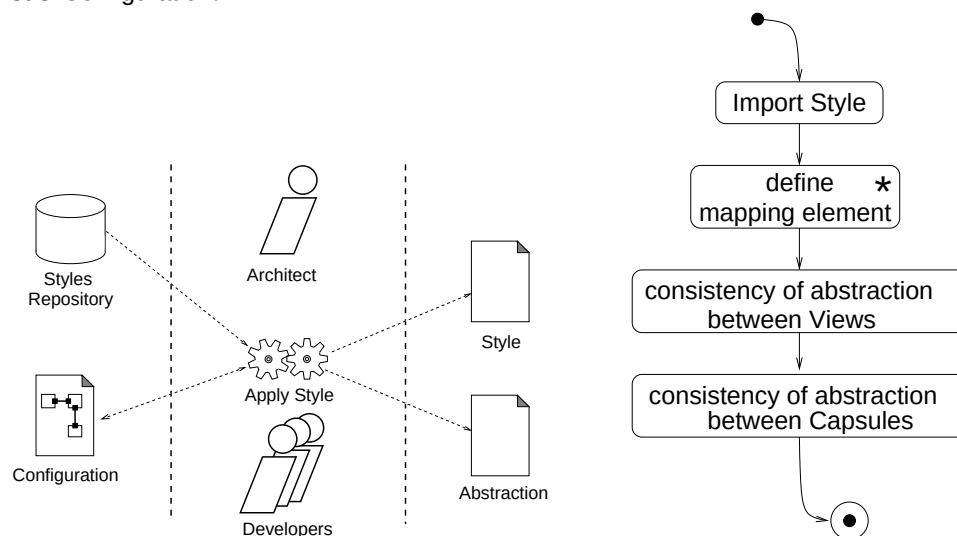
Applicability At all times during a development.

Deliverables Consistency proofs. Apart of that, it depends on the style and the time of application.

Process Assuming that it is known which style is to be used,

- import the appropriate style into the development project;
- connect the style with an Abstraction, providing mappings between the elements of the style and the elements of the current Capsule;
- check the satisfaction of the Abstraction, i.e. Section 10.4 on “Abstraction”.

Note that there is an arbitrary number of independent mappingElements to define, so the respective State is an ActionState with isDynamic set and its dynamicMultiplicity has the value *. The dynamicArgument is of course the list of all Capsules and Connectors of the sub-Configuration.



15.: REALIZE Capsule

realization / construction

Intent Provide a realization for a Capsule that matches the specification and design.

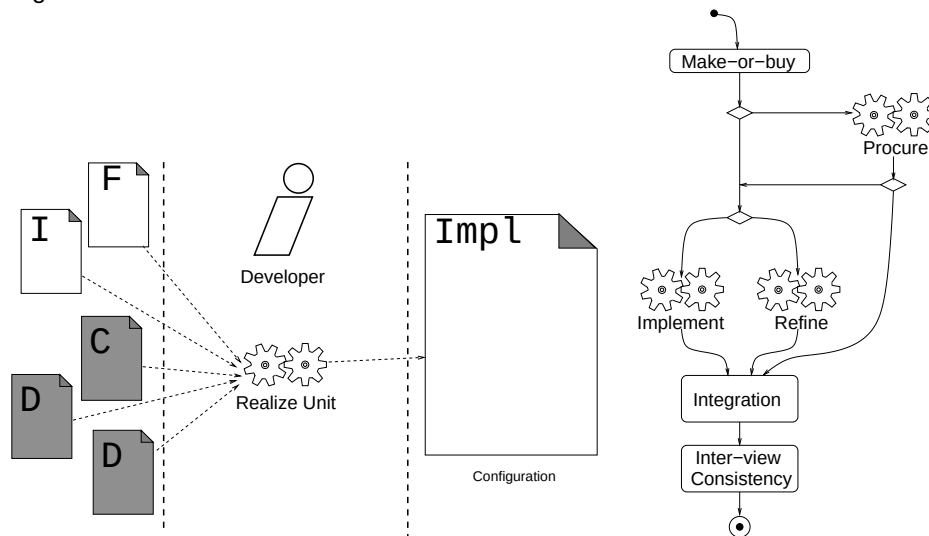
Discussion Once the analysis and abstraction of a component are completed, one has a rather precise specification, which can be used for the realization of architectural components. At this stage, the Capsule can be realized by any of three alternative procedures: procurement, implementation, and refinement. This pattern subsumes the patterns of the design phase. It is used as a commentary, for structuring the development process. The common part of the sub-patterns could be integrated into these, of course, but this would result in a less clearly structured overall development process. For high-level Capsules, specifying the behavior may be omitted. For low-level Capsules, the structure may be left out.

Participants Developer.

Applicability Whenever the specification and design are established.

Deliverables A complete Capsule, with all relevant views defined and checked for consistency.

Process First of all, we need to decide whether the given specification is to be procured or to be realized by ourselves. In the former case, the “Procure Capsule” pattern is applied. The component may be drawn from a component-library or obtained as the result of sub-project, or a proper project of its own. In the former case, and if the procurement fails, we need to decide whether the Capsule can be implemented directly, or requires further refinement. In any case, the resulting Capsule must be checked for consistency and integrated into its Configuration.



16.: PROCURE Capsule

realization / construction

Intent Realize a Capsule matching the specification and design. Procurement may mean either (1) purchasing a Capsule matching the specification from a components catalog; or (2) extracting it from some other system, a pla or a style; or (3) giving the specification to some sub-contractor to supply the Capsule according to the specification. Thus, this pattern could also be called “cannibalize architecture”, “use pla”, or “custom make Capsule”.

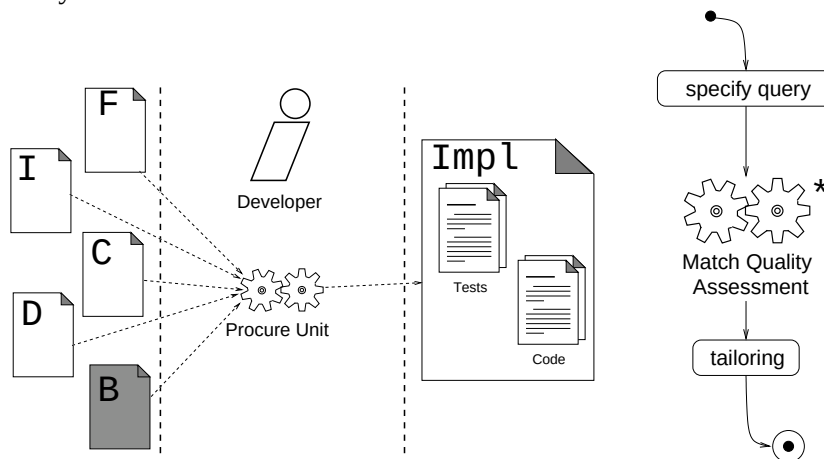
Discussion One form of realization is procurement. Usually, it is the cheapest alternative only if no tailoring of the procured component is required. Sometimes, however, the adaptations necessary may be negligent, or the procured component has some very desirably features justifying the effort. For such cases, an optional tailoring stage is included.

Participants Developers, component-Librarians, Development administration.

Applicability Whenever a structure view needs to be filled with detail.

Deliverables A given set of candidate components, ordered according to the degree of matching. All of the candidates are complete Capsules. The candidate that best matches the specification may be tailored to satisfy the needs specified in the request.

Process Procuring a Capsule means first finding the desired Capsule and then replacing it for the specification. So first specify a request by defining at least the interfaces and functionality views, possibly also (parts of) the domain or the behavior view. Query the component-libraries of the project with it. Apply a match quality assessment for each found Capsule in the result set. If a satisfying Capsule is found, it may need some tailoring to achieve a perfect fit. Possibly, this includes refactoring and/or redeployment, and thus change impact analysis.



Note that there is an arbitrary number of sub-Capsules to be checked for their match quality, that may be carried out in parallel. So, there is an ActionState with isDynamic set and its dynamicMultiplicity has the value *. The dynamicArgument is the list of all candidate components.

17.: REFINE Capsule

realization / construction

Intent Construct the Capsules in the Configuration of the structure view.

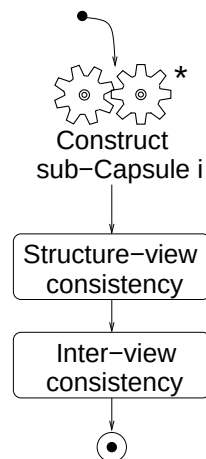
Discussion All but the simplest Capsules are systems in their own right, i.e., they are again composed of other Capsules, arranged in a Configuration of the structure view. These Capsules need to be constructed, and their consistency with each other must be established.

Participants Architect, Developers.

Applicability At least the functionality and interfaces views must be present. The structure view may or may not be available.

Deliverables A set of Capsules realizing the structure view, i.e. both the static and dynamic structure of the Capsule.

Process All the Capsules in the Configuration of the structure view are constructed. After that, the inter-view consistency of all sub-Capsules is checked. Some tests may already be carried out with half-way constructed Capsules.



Note that the sub-Capsules may be constructed concurrently thanks to them being cleanly separated by Ports. Thus, “Construct sub-Capsules” is an ActionState with isDynamic set and its dynamicMultiplicity has the value *. The dynamicArgument is of course the list of all sub-Capsules.

18.: IMPLEMENT Capsule

realization / construction

Intent Provide a Capsule by straight implementation.

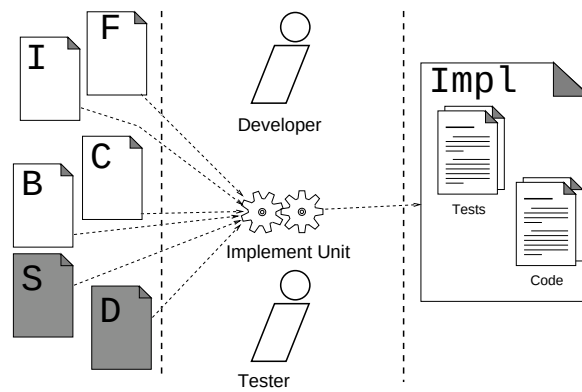
Discussion Simple Capsules may be implemented directly, guided by the design views. Also, more complete Capsules might be implemented prototypically.

Participants Developer, Tester.

Applicability Whenever a structure view needs to be filled with detail.

Deliverables Realization view

Process The specification (either just the functionality view or also the behavior view) is directly implemented by hand. The Capsule must be subjected to unit test.



19.: MATCH QUALITY ASSESSMENT

realization / quality assurance

Intent Assess the degree of matching between two given Capsules, where one (the specification) is incompletely defined.

Discussion When looking for a component to fill some gap in a design, we face two problems. First, a vast number of reusable components has to be sifted for a smallish set of seemingly matching candidates. Then, these candidates have to be further scrutinized for their precise degree of fitness. We assume that the first step is done by using the classification schema and keywords of the domain and functionality views of the potential candidates. The second step, that is, determining the precise degree to which a given component matches the requirements is done by match quality assessment.

Participants Developers.

Applicability Whenever a component is to be fitted into a Configuration, such as in realization or replacement. The pattern is always applied to exactly two Capsules that play the roles implementation and specification.

Deliverables An assessment of the match between the two Capsules it was applied to.

Process In the following, we use the terms candidate and binding site for the Capsule that is to be analyzed and its prospective environment (i.e. a Configuration), respectively.

For all required Ports of the binding site, find a Port of the candidate which has as its ProtocolRole either the same or a behavioral subtype of the ProtocolRole of the respective Port of the binding site.

Provided, this check has succeeded, insert the candidate provisionally, and check constraints of Section 10.4.

Finally, analyze manually the matching of domain and functionality view (e.g. by inspection or walk-through).

20.: MAINTAIN Capsule

maintenance / reuse

Intent Provide a specification for a Capsule.

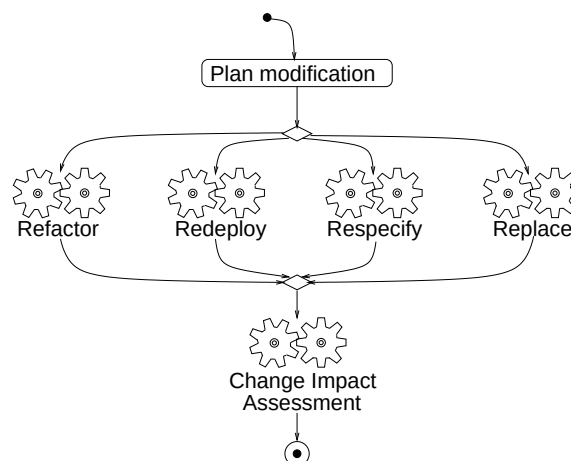
Discussion This pattern subsumes the patterns of the maintenance phase. It is used as a commentary, for structuring the development process. The common part of the sub-patterns could be integrated into these, of course, but this would result in a less clearly structured overall development process.

Participants Developers.

Applicability Whenever a Capsule has gone through each of the prior phases at least once.

Deliverables Modified Capsule and a list of change impacts.

Process A Capsule may be refactored, replaced, respecified, or its structure redeployed. Afterwards, the impact of the change must be assessed.



21.: REFACTOR Capsule

maintenance / reuse

Intent Modify the static structure of a Capsule such that it better suits its purpose, while keeping all its other features. Ensure consistency of this change. Possible applications include: (1) splitting up a Capsule into a Configuration; (2) moving part of a Capsule's static structure Configuration to an adjacent Capsule; and (3) joining several Capsules into one.

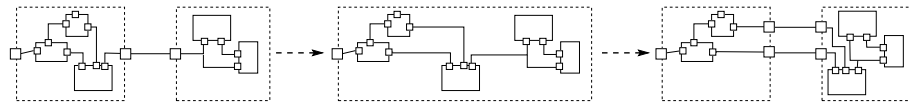
Discussion In the lifecycle of a Capsule, a restructuring may become necessary at some point, e.g. after deployment or changing requirements. While the term refactoring is often understood as referring to classes, more general positions have been stated, too: as two examples, consider [Noack et al., 2000, p. 8] and Stevens and Pooley [1998]. In the former approach, transformational patterns are presented for migrating from a fat-client to thin client architecture. The emphasis is on structure and performance. The latter deals mainly with organizational (and some technical) issues of reengineering legacy systems. It is in this sense of “refactoring” that we use the term in the remainder.

Participants Developers.

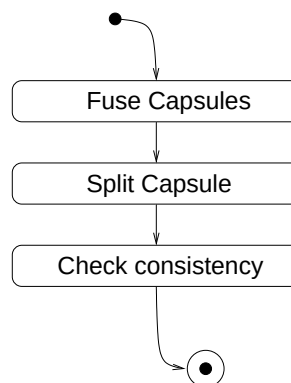
Applicability Structure and implementation views must be present. All other views are not necessary, but may be helpful. Refactoring may not be applicable for procured Capsules—their structure, if any, may not be within the scope of delivery.

Deliverables Changed structure or implementation views.

Process The process of refactoring a Capsule with intent (2) can be illustrated by the following sketch, showing the sequence of intermediate results.



We start from a Configuration, in which we have identified a Connector and its adjacent Capsules as our focus. Without loss of generality, we assume that this Connector is binary. Fuse the two Capsules by creating a new joint Capsule, whose static structure is the union of the two Configurations. The original Connector, the Ports and the binding Connectors adjacent to it, are replaced by a single Connector. In a second phase, the joint Capsule is again split up into as many Capsules as desired. If the Ports that have vanished in fusing carried any transformational or filtering functionality, it has to be reintroduced at this point. The consistency checks for the structure view must be re-verified.



22.: REDEPLOY Capsule

maintenance / reuse

Intent Deploy a distributed system in a different way, e.g. for performance tuning, and detect possible implications of the changes.

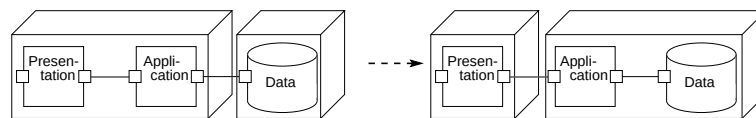
Discussion Sometimes it is necessary to change the deployment of a Capsule, i.e. the distribution of its sub-Capsules to Nodes. Often, this step is either a consequence of or a reason for refactoring that Capsule: a typical motivation is to minimizing network traffic.

Participants Developers, Network and security engineers.

Applicability Whenever both a structure and a deployment view exist, and are consistent and analyzed or validated wrt. load aspects.

Deliverables Updated deployment view.

Process Shift a Capsule along the Connector of one of its Ports across the outline of the enclosing Node (see sketch).



Possibly, the properties of Connectors have to be changed accordingly: if a SimpleConnector connected two Ports within a Node that now reside on different Nodes, the Connector is likely to become an asynchronous ComplexConnector with faults and delay etc.—the same applies vice versa, too, an may cause, say, race conditions. If changes like these are made, all qualitative and quantitative analyses concerning this Connector have to be re-executed.

23.: REPLACE Capsule

maintenance / reuse

Intent Replace an outdated, buggy or otherwise unsatisfying Capsule in the structure of a host-Capsule by another.

Discussion There is a fundamental difference between replacing and reusing (i.e. this pattern vs. e.g. Capsule realization): in the first case, something new is used instead at some place; in the second case, we use the same thing over in some other place. In other words, in the first case, we reuse a Capsule knowing nothing about its environs. In the second case, we reuse its environment, a Configuration, expecting the new Capsule to be similar in a certain sense to its predecessor.

Participants Developers.

Applicability Whenever there is a structure view.

Deliverables Updated structure view.

Process Remove the offending Capsule, and replace it by the new one. Optionally, the Configuration needs to undergo redeployment. Consistency is reestablished by checking again all conditions concerning the interfaces, behavior, and structure views of the hosting Capsule, i.e. the conditions in Section 10.4 concerning “Exchange”.

24.: RESPECIFY Capsule

maintenance / reuse

Intent Adapt the specification of a Capsule to new requirements, functionalities, or context conditions, and help controlling the change impact.

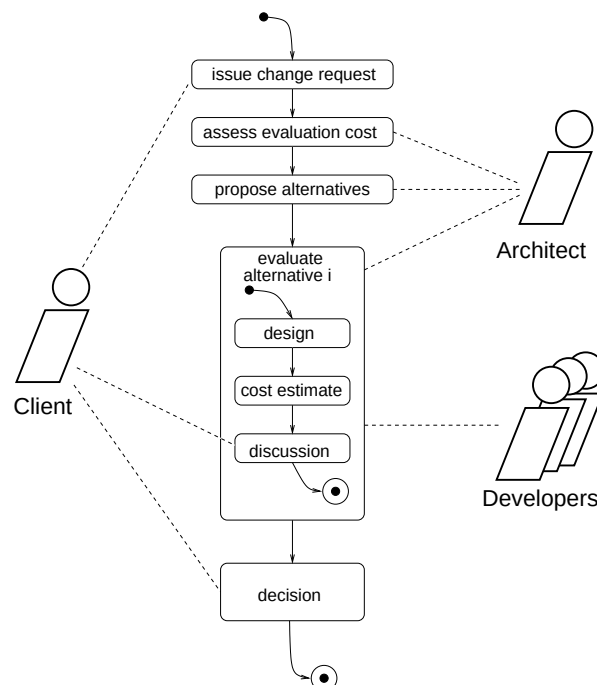
Discussion The intended goal, desired functionality, any context conditions, or indeed the domain of a system may change, instigating change on behalf of the system. While changing the implementation is local only, changing the design may compromise the overall goal, so that a change impact analysis become necessary. Changing the specification, however, almost necessarily implies a change impact analysis. As the effort of going through a respecification may be quite substantial, it is required that the client formally states a change request, and that the cost of evaluation this request is agreed upon before any actual work is done.

Participants Architect, Developers, Users, Client.

Applicability Whenever the client raises an issues worth a change request.

Deliverables A set of possible ways of dealing with the change request issued by the client, one of which may be singled out as the accepted solution.

Process The clients request a change to be made to the functionality, domain or context of a Capsule. The architect first assesses the cost of evaluating the request. The client may choose to abandon the project at this stage or to go ahead with it. Then the architect proposes a number of candidate solutions, and develops these as far as seems necessary, that is, possibly to a complete specification and design. The cost and duration of realizing the alternative solutions is estimated together with the developers. The client is presented with the results, and may decide to realize one of them.



There may be several alternatives to assess, which, in principle, may be done concurrently. So, “evaluate alternative” is an ActionState with isDynamic set and its dynamicMultiplicity has the value *. The dynamicArgument is of course the list of all alternatives.

Intent Reestablish all properties that might have been compromised by a preceeding change.

Discussion In some of the previous patterns considerable change has been instigated. Every time, a set of consistency conditions had to be reestablished afterwards. However, only those implications are detected this way, which have been expressed as formal consistency conditions—but there are many more. It is the purpose of this pattern to assess the overall impact of a change.

Participants Architect, Developers, Users, Client.

Applicability A change impact analysis may be conducted at any time when a change to a Capsule is made. Given the effort involved, however, it is justified only when the vital interests of client might be compromised seriously.

Deliverables List of possible problems, to be added to the context view. This list is delivered and kept even if during change impact analysis it is

Process First of all, all formal consistency conditions must be re-established, provided the views they are defined on are present. Then, the informal conditions need to be looked at, that is, the goals contained in the context view must be reconsidered. For instance, a refactoring may block certain future development paths, a redeployment may compromise safety goals and so on.

determined to abolish the change and go back to the status quo ante—a negative result such as this is equally important and dear as a positive result.

11.4. Chapter summary

In this chapter, we have defined a language of process patterns of the genuinely architectural concerns. The description we have given is much more detailed and precise than any other such pattern languages that we know of (e.g. Catalysis, cf. Section 2.4). As far as exclusively architectural issues are concerned, it is also more comprehensive. Within these patterns, we show which of the properties of the previous chapter are useful under which circumstances.

In the next chapter, we will provide a large example, that ties all of the previous strands together—concepts, syntax, semantics, properties and process patterns.

12. Extended example

In this chapter we present some further examples for the concepts, semantics, notations and process patterns presented above. For this purpose, we chose to elaborate on the banking case study introduced in Section 5.4.1 for the following reasons.

The choice of example is motivated by three reasons. First of all, we already have introduced this case study above. Second, the example is both rich and realistic enough to illustrate our points, as was confirmed independently by several domain experts.

Third, it is clear that the approach we present in this book involves some methodological overhead. We have tried to compensate this by providing design-flow support in the form of process patterns, but even so, the approach presented in this book is not necessarily economic for *all* applications. On the other hand, it is clear that there are indeed applications where the effort required does indeed pay back, e.g.

- for applications with a long overall software life cycle, so that the evolution cost dwarfs the development costs. Here, any effort put into the system architecture can be seen as a kind of quality assurance measure;
- in application domains where there are particularly high requirements concerning e.g. availability, security, safety, or legal constraints.

This profile is typical for online banking systems. See e.g. Dewal and Schnichels [2000] for further motivation. We will now trace an example through the phases shown in Figure 11.1, top.

12.1. Specification

12.1.1. Context

In Section 5.4.1, we have outlined a support system to support business processes occurring in retail banking (see Figure 5.12) with a view to certain future perspectives (see Figure 5.13). We shall call it the Banking Support SystemBSS for want of a better name. So far, we have only sketched some fairly intuitive ideas about the system, such as might be voiced by the customer. In this case, we assume that the customer is the management of the bank, that has created the following elements of the context in a brain-storming: (1) there ought to be a system with certain specified particular functionalities; (2) due to the existing hardware and software, certain components are to be (re)used, in particular a relational database; (3) following prior experience, a certain structure and deployment are suggested; (4) a set of development paths is foreseen; (5) due to a shortage of staff, the user-interface subsystem is to be outsourced¹. We will now pursue this example, stepping into the shoes of the development team.

¹Another reason why one would outsource such a task is the fact that highly formalized processes such as UP are simply to awkward for GUI design.

The client imposes as one requirement that the system to be built is a three-tier client-server system, consisting of separate subsystems for the presentation, the application logic and the data representation (called Frontend, Logic, and Database, respectively in the remainder). These subsystems are to be deployed to a server (Database and Logic) situated at the headquarter's computing center, and to desktop computers (Frontend) at the local branches. Figure 12.1 gives a first impression of formalizing this requirement. Implicitly, the requirements lead to vague descriptions of the use cases of the subsystems, too, though the precise mapping between functionality and structure is yet to be determined.

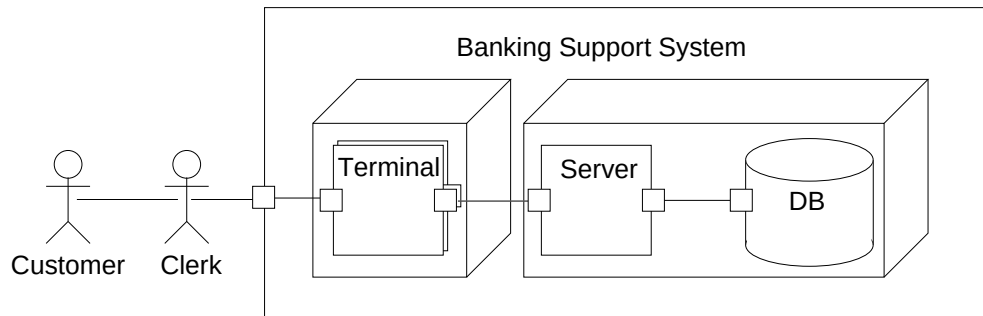


Figure 12.1.: The top-level structure and deployment is determined by the client.

In the case of Database, this is pretty straightforward, as the existing database cannot be changed, and has to be taken as is. There may not even be a conceptual model for the database, so we are down to tables. Similarly, there is exactly one possibility of accessing the database, and that is by issuing SQL-statements against the database via some query-interface. So, we are already able to identify the database tables needed to accomplish the functionality (see Figure 12.2).

CUS					ACC	
NR	FN	LN	OCC	CR	NR	BAL
⋮	⋮	⋮	⋮	⋮	⋮	⋮

Figure 12.2.: The database tables required to accomplish the functionalities of the Banking Support System: customers with columns account number, first name, last name, occupation, and credit (left); and accounts with account number and balance (right).

For the sake of argument, suppose that the frontend is beyond the knowledge, competence, or capacity of the development team, so that it is outsourced to another company in a procurement sub-process. The contractor is given a complete specification, that is, context, domain, functionality, interfaces, and behavior views. The implementation and structure views are provided by the contractor.

As we have mentioned right at the start, there are certain possible development paths that our client wishes to be considered in the development. A good example are extensions of the system to deal with complementary *channels* for the bank's customers. The first task is thus to document these paths (see Figure 12.3). Obvious candidates are ATMs, phone service, and direct online access.

At first sight, these diagrams may seem rather trivial and informal to the software engineer. However, they show different future scenarios quite clearly, and allow to communicate certain issues as-

sociated with them. Besides, note that the Frontend stays (almost) the same in all the documented development paths—this is it say, that the extensions sketched will be realizable by adding appropriate Ports Frontend. A very strong statement indeed!

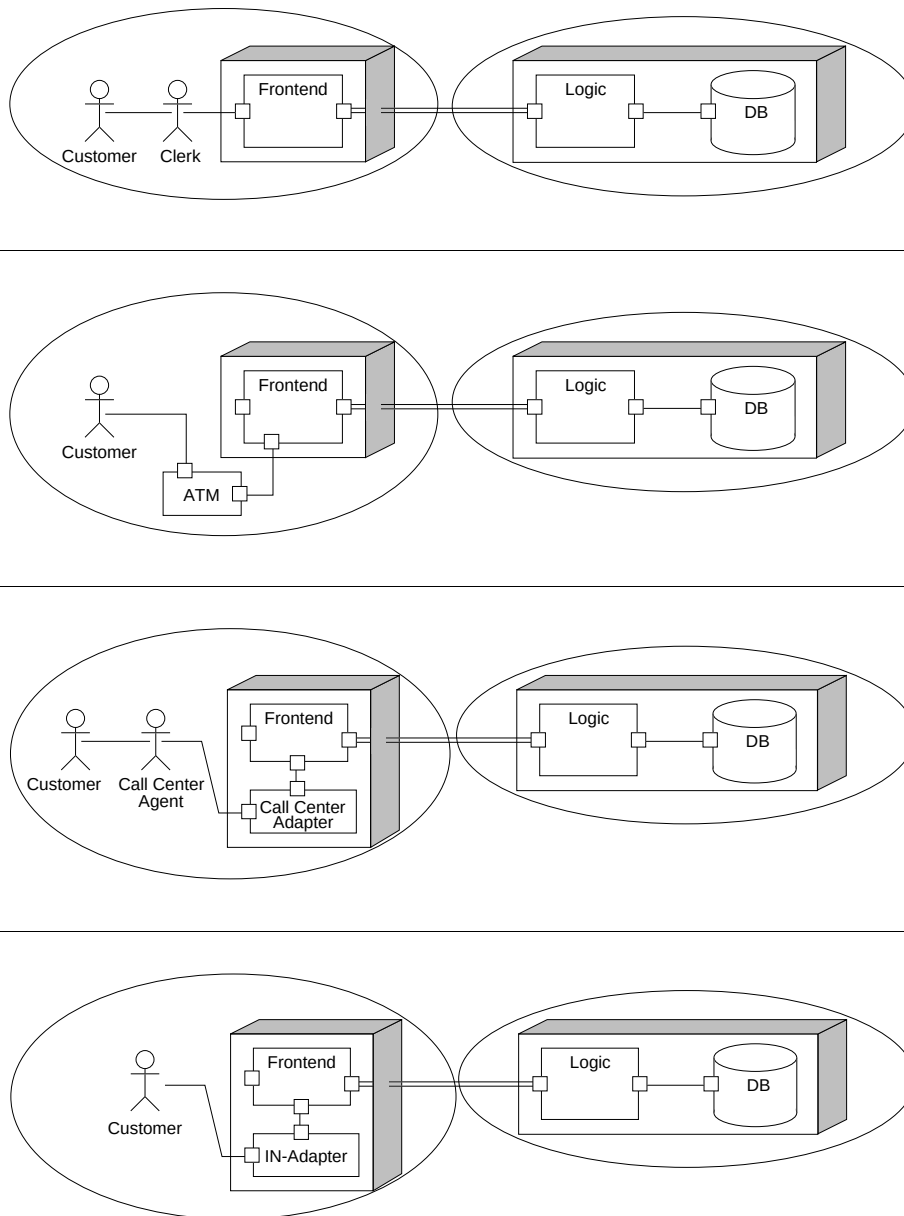


Figure 12.3.: Some development paths foreseen for the BSS, with several complementary channels added consecutively (top to bottom): the initial stage, addition of ATMs at branches, addition of phone service via a call-center, addition of direct internet access by the customer. Geographically close system parts are enclosed by ovals.

In Figure 12.3, we show the respective structures, deployments and geographical distributions. Proceeding from top to bottom, there is first the simplest channel (client at counter). We have here added an ellipse around items that exist at the same physical location. In the second step, ATMs may

be added at the branches to reduce workload for the clerks. In the third step, a phone service might be added, realized in a call center that is technically similar to a branch, but needs more elaborated identification procedures. In a fourth step, the customer may be provided with direct online access via the internet. Clearly, yet new security issues arise. In the scenario presented, there is also a change at the central computing center, where a new server is planned.

Such diagrams help communicate technical issues, and may even be used as a requirements specification in the development process itself (i.e. by the developers). Whether there is promise of more elaborated technical use, such as analysis procedures similar to the consistency conditions defined in Chapter 10 is as yet unclear.

12.1.2. Domain

Having settled on the context, we turn to the application domain. Firstly, we need to analyze the business processes. This has to be done in close collaboration with the domain experts, and so, particular emphasis has to be put on visualization. Secondly, the boundary between the system and the social context has to be established, i.e., it has to be determined which part of the business process is to be automated, and which remains a manual task. The former parts, thirdly, have to be mapped to the architectural structure.

Visualizing business processes

One of the important purposes of representing an architecture explicitly and diagrammatically is the desire to use it as a communications medium both among developers, and between developers and other groups, in particular customers and users. These stakeholders frequently lack technical background. Yet, it is important to have them involved in the development process, as it is them who know about the actual business processes. Thus, the notations used for this level of abstraction must be particularly easily understandable. Now, suppose a development team needs to propose how a certain business process would look like with the new system, so that the users can determine its adequacy. Such a business process would be represented in UML by an activity diagram such as is shown in Figure 12.4.

The purpose of business modeling is to identify and document the business processes of an organization. One important result of this procedure is the identification of the system/organization divide, i.e. what tasks are accomplished by a human, and what can or should be automatized. In the model we have created above, this divide is presented in a clear and unequivocal way. From this divide, a great deal of information can be derived. First, the sequences of messages transmitted across the system boundary are system test cases. Also, these sequences can be used for user documentation such as tutorials. Finally, process improvements can be based on existing process description as follows.

Obviously, the process presented in Figure 12.4 is not optimal with respect to the time the customer and the clerk spend at the counter. First, the customer needs to queue to get a form, then he might need to queue again for handing it in, and has to wait while the clerk checks and types the data. Finally, it might take some time until the central server responds.

This process can be improved to better serve the customer, see Figure 12.5. Here, the clerk types in the data provided by the customer directly. While the form is printed, the request is already being issued to the server and the customer is asked for an identification. The printing being completed, the customer signs the printout, his signature is verified by the clerk and the completed form is filed. In the meantime, the server has had time to process the request and reply with the data needed to complete the transaction. Obviously, this process is faster for the customer. It also slightly reduces the

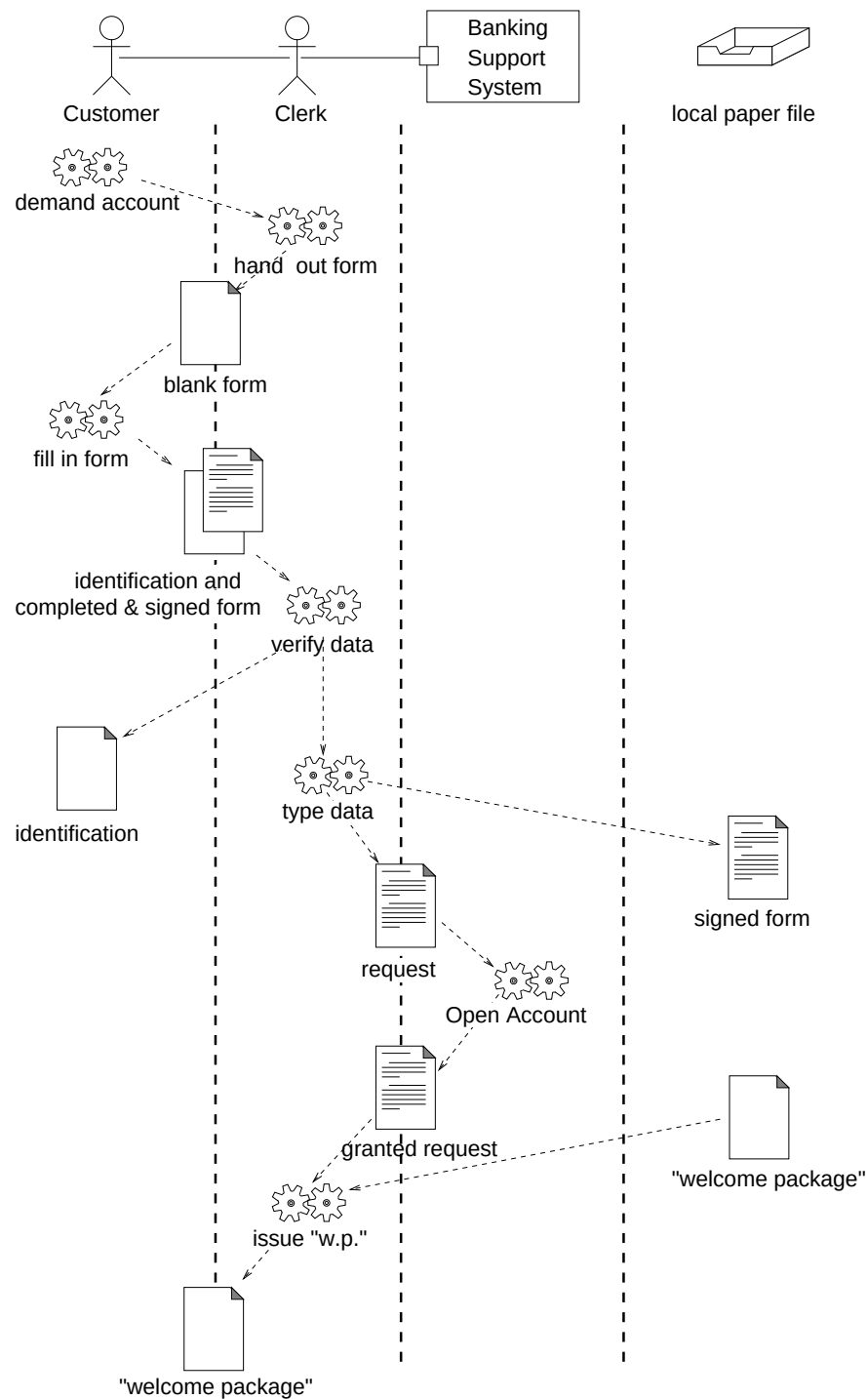


Figure 12.4.: The business case "Open account at counter".

time the clerk is waiting for the processing being done, and adds only a little additional net traffic and futile computation for those cases where opening an account is initiated but aborted in the middle.

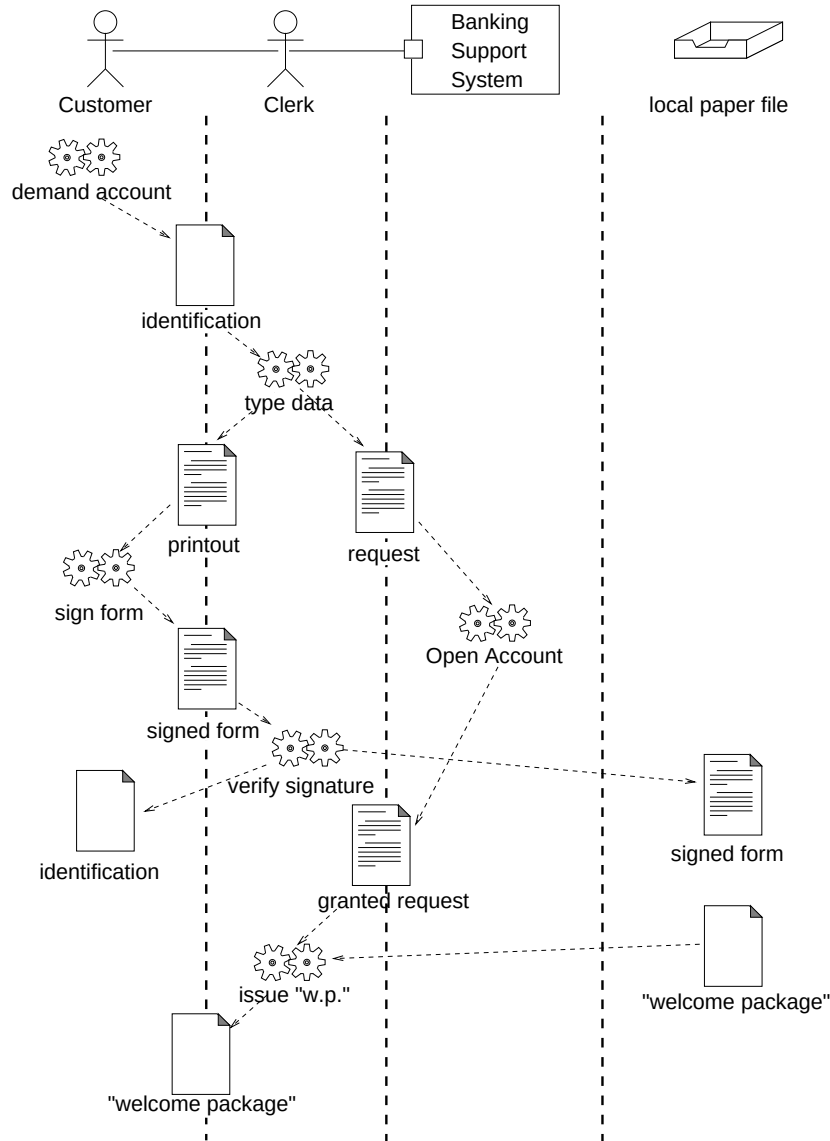


Figure 12.5.: An improved business case “Open account at counter”.

Mapping functionality to structure

In distributed systems, many functionalities are provided by collaborations of different subsystems. In order to understand an existing subsystem, it is often necessary to understand the interrelationship between the various subsystems and their functionalities, for instance to do certain kinds of performance analysis (see Section 9.6.3, or Huszerl and Majzik [1999], Kosmidis and Huszerl [2000]), or for understanding *feature interaction* problems. Here, a mapping between use cases and architectural structure can be very helpful. The use case map (UCM) approach has been used for this purpose with

considerable success (see Buhr [1998], Buhr and Casselman [1996] for an introduction).

There have been attempts to integrate UCM concepts into the UML metamodel (see e.g. Amyot and Mussbacher [2000]), but we think for mere visualization, this is not necessary, let alone justified. Conceptually, a UCM shows the flow of control through various parts of a system. This is quite similar to an activity diagram, but the notation is more like a collaboration diagram. Both activity and collaboration diagrams can not represent the synopsis of structure, functionality and flow of control that have made UCMs so popular. It is possible, however, to capture this combination of aspects in a capsule diagram, however.

12.1.3. Functionality

Assuming, that (1) the application domain is reasonably clear to the developers involved (including those of the contractor); and (2) that the context is set by the customer; and (3) that both domain and context have been described to general acclaim by the developers and accepted by the customer, we now need to achieve a better understanding of the desired functionalities of the subsystems. The functionality analysis pattern of Section 11.3 may be used to do this. A possible result is presented in Figure 12.6.

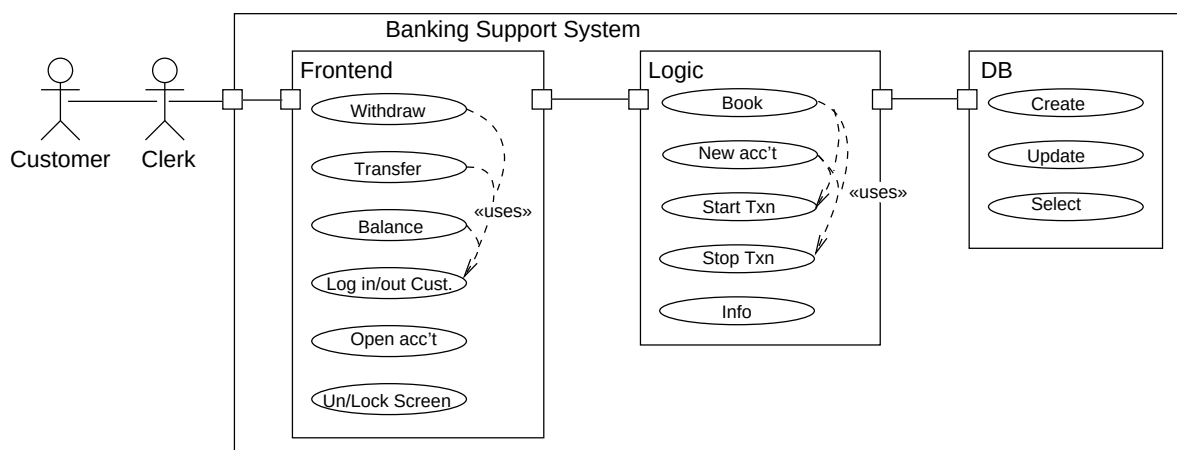


Figure 12.6.: Refining functional requirements: structural decomposition of use cases results in functionality views for the three subsystems.

The functionality of the Frontend consists in the most frequent processes demanded at the counter: asking for the balance, withdrawing and transferring money, and opening a new account. We have added a use case “lock/unlock” to lock the terminal to avoid unauthorized use while the clerk operating the terminal is away for brief periods (like when double checking with a superior). Finally, we have added a use case “log in/out customer”: with this use case factored out, the customer data has to be entered only once, even if the customer has several jobs to do in one go.

Suppose that for the Logic subsystem, we have identified the use cases “book”, “log in/out customer”, and “new account”, according with the overall requirements. Note that there are seemingly identical use cases in Frontend and Logic: these are really quite different. Looking at the “open account” use case, the Frontend’s use case is concerned with handling the user interaction, getting all relevant data and checking them for consistency. The corresponding Logic use case is concerned with translating the user-level requests into Database queries. Note also, that “withdraw” has no parallel in

the Logic subsystem, as it is really a booking from the customer's account to that of the branch. Observe, that any instance of Frontend may serve only one customer at a time, while there are many such instances that must be handled by the Logic subsystem. Thus, there must be some form of "transaction management", which we added as an additional use case.

The functionality of the Database subsystem is fairly obvious. Since integrating this legacy is a must, the use cases are fixed. We chose the use cases "create", "update", "select", in analogy to traditional Structured Query Language (SQL) statements.

12.2. Design

The first phase is now completed: we have established the context, domain, and functionality of the system under construction. So, we can now turn to designing the technical details, that is, the interfaces and the behavior. The structure has already been part of the requirements.

12.2.1. Interfaces

Interfaces determined by context

Let us reconsider, what items of information we have about the interfaces (see Figure 12.7). First, the database is to be reused as an (almost) black box, so its interfaces are already entirely defined. For our application, we use only the query-interface, and its ProtocolRole is exactly the SQL-query sublanguage. A simplification is shown on the right of Figure 12.8 (the other half of the figure is explained later).

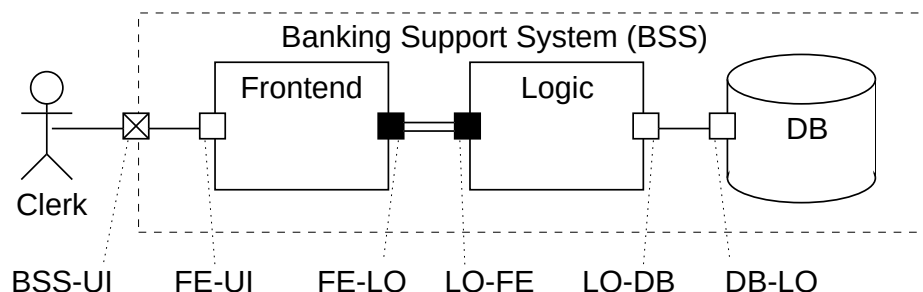


Figure 12.7.: The starting point concerning the interfaces: the (pragmatic) kind of the various interfaces follows already by the requirements (recall that relays, filters, and transformers are presented by crossed, empty, and filled boxes, respectively). The remote connection between Logic and Frontend is realized by a ComplexConnector, the one between Logic and Database is realized by a SimpleConnector.

By the requirements, the BSS is supposed to be a distributed system, and the Frontend subsystem is supposed to be run remotely, i.e. in the respective branches. Thus, Connector between Frontend and Logic must be a ComplexConnector with delay, a certain error probability and security implications. The interfaces adjacent to this Connector will thus be transformers. The Logic and Database are running on the same node, so that we may assume a SimpleConnector between them, and LO-DB will be a filter playing a subrole of the complement of LO-DB's role. The user interface of the whole BSS is just a virtual interface (a relay), that is, a conceptual entity, realized by the UI-Port of the Frontend subsystem.

With Figure 12.7, we introduce naming conventions and abbreviations that we will use throughout this example.

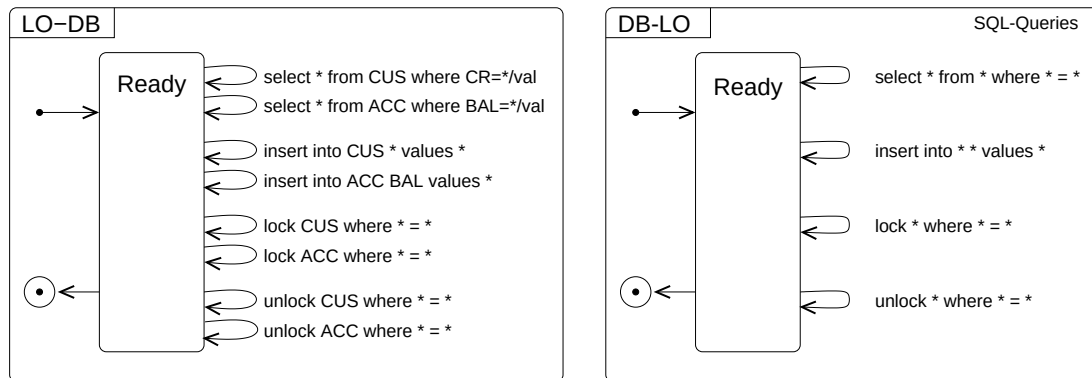


Figure 12.8.: The behaviors of the (ProtocolRoles of the) Ports between Logic and Database: the Logic side (left); the Database side (right). Observe the difference between the argument ranges on the two sides.

User interface

Since BSS-UI is a relay Port, the ProtocolRole of FE-UI is effectively the user interface. We design it by exploring user-level scenarios and storyboards.

We start with a simple scenario (see Figure 12.9). There, we present a withdrawal process: the clerk identifies the customer by his/her name and enters the name into the system, that finds the account number, balance and customer details. The customer asks to withdraw a certain amount, which the clerk forwards to the system. When the approval of the system is transmitted, a receipt is printed automatically, and handed to the customer, along with the money.

Scenarios can be described by sequence diagrams, and each such diagram is a partial specification of the behavior of the ProtocolRole of FE-UI. Many end users find sequence diagrams difficult to understand, and so, it might be helpful to present them in the form of a *paper prototype* (see below), i.e., as a set of drawings showing made-up screenshots of interesting intermediate states in the scenario. They can then be flashed to the user in response to the actions he triggers. Practical experience has shown, that storyboards are a very good medium for user validation.

We call this dialect window event diagram (WED). It is read as follows: a State is presented by a window or menu layout (a fabricated screen shot), a Transition is presented as arrows as usual, but its trigger is given as a button-like icon plus an inscription like “left”, “CR” or “double right” at its source, indicating a focus and the respective kind of Event. Clicking the left mouse button is the default Event, so it may be omitted. Additional constraints are given as comments. Features like history states, join and fork states and syntactic sugar like choice states applies. Using this notation, the user interface of the frontend subsystem may be designed (see Figure 12.11). The storyboards may be directly transformed and refined to a proper state transition diagram (see Figure 12.12).

After collecting a sufficient number of such scenarios, they may be summarized to a *storyboard*, that is, the paper prototypes of the scenarios may be merged into a single, state-transition-like notation, where the fake screenshots become States connected by user actions (see Figure 12.10 for a schematic example).

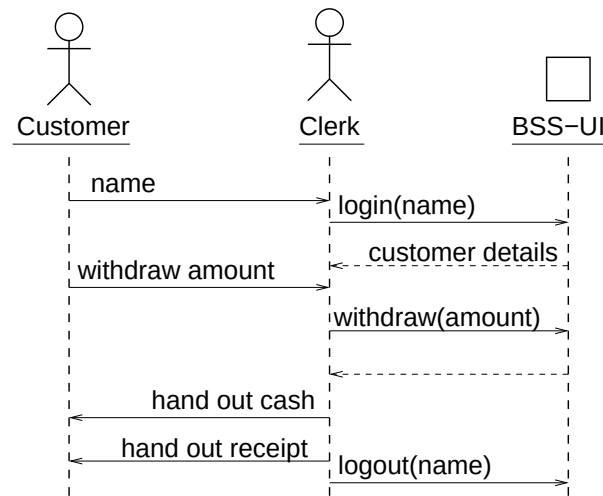


Figure 12.9.: A usage scenario including both the interaction between customer and clerk, and clerk and terminal. Here we present a withdrawal process. For more detailed descriptions of business processes, see Section 12.1.2.

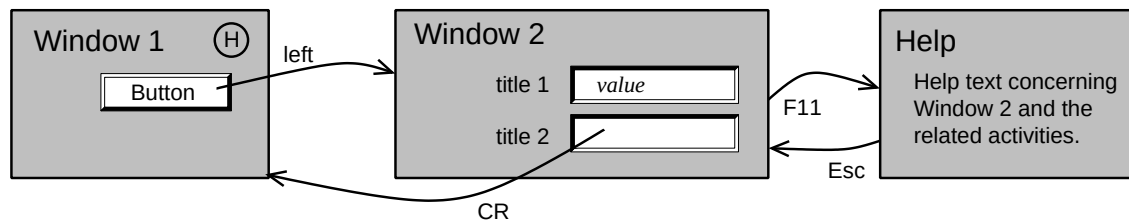


Figure 12.10.: Schematic syntax of window event diagrams: States are represented as window layouts, transitions are represented as arcs, their triggers are composed of the source of the arc indicating a focus and an inscription indicating an event, such as clicking the left mouse button. The latter is the default and may be omitted. Additional constraints are given as comments. Data fields are presented as lowered regions. Computed values are indicated by italics.

This procedure is quite helpful in gaining overview, and it has also the advantage of ensuring a smooth transition from partial to total specifications, and on to plain UML STDs. The resulting storyboard for FE-UI might look like Figure 12.11. WEDs may also be used as the basis for generating user documentation such as manuals and tutorials—in fact, Figure 12.11 is almost self-explanatory, even for the technically challenged.

The interfaces between Logic and Frontend

Let us now proceed to the other Port of Frontend, the interface to Logic. It has to be designed together with the opposing Port of Logic, LO-FE. This pair of Ports realizes the remote connection, so their most prominent job is the translation from local calls to remote messages. Inserting a layer of abstraction from the concrete representation at this point is essential for later reusability.

The ProtocolRoles of FE-LO and LO-FE depend on the distribution of functionality to structure, the exact boundaries of the various use cases, which is as yet unclear. In fact, it is one of the most

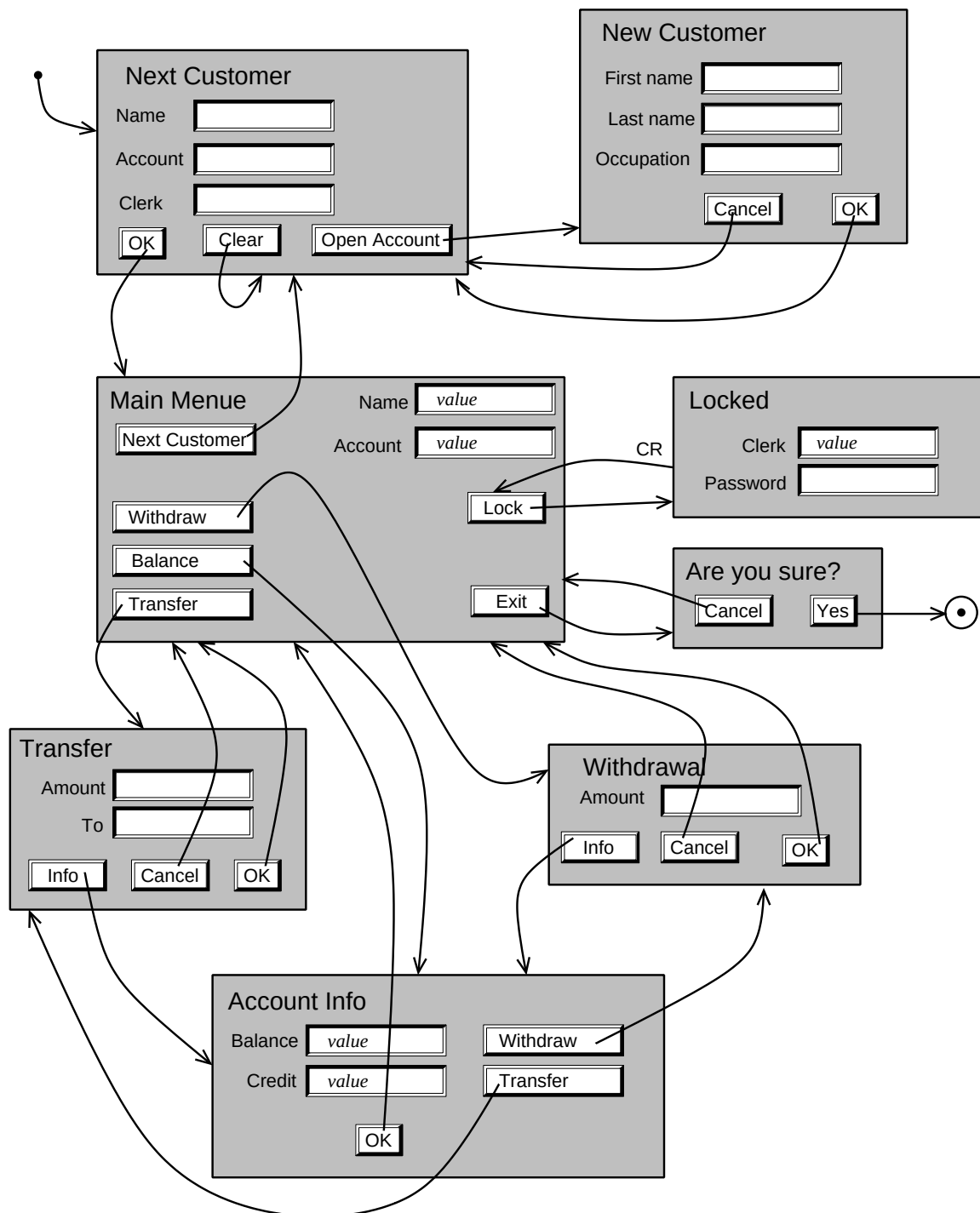


Figure 12.11.: A storyboard for the system, presented as a kind of state transition diagram. Here, states are given by window layouts, and arc triggers are given by button icons from which an arc emanates.

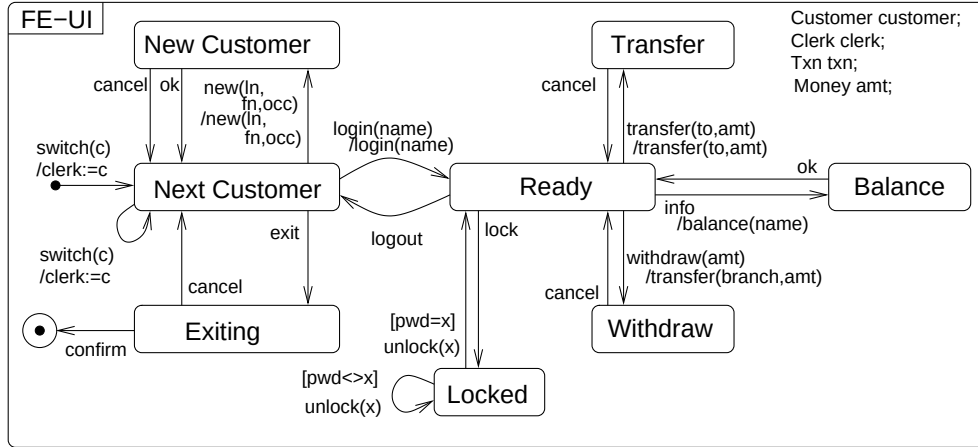


Figure 12.12.: For programming purposes, the storyboard may be transformed into a proper STD in the standard syntax.

important outcomes of the whole project, and thus requires a good amount of discretion on behalf of the architect and developers. Let us assume, that we have the Protocol between FE-LO and LO-FE as shown in Figure 12.13.

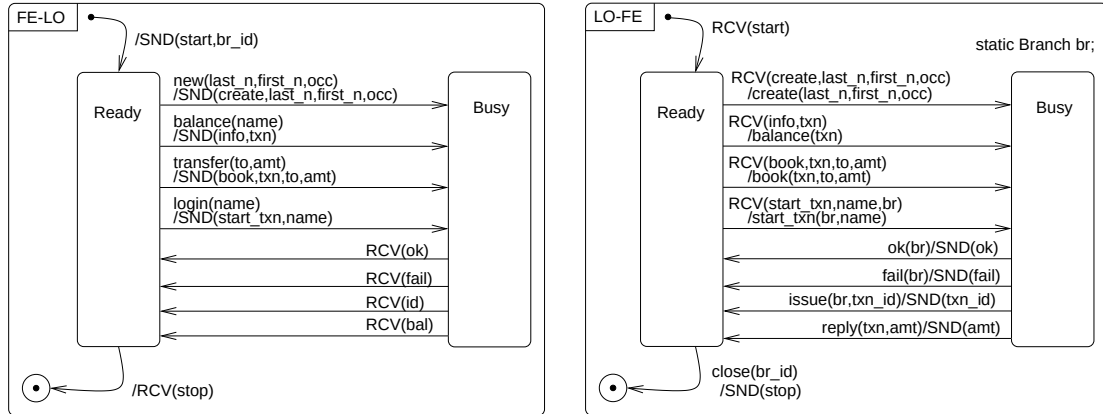


Figure 12.13.: The behaviors of the (ProtocolRoles of the) Ports between the Frontend and the Logic subsystems: the Frontend side (left); the Logic side (right).

Since the data base existed before, its usage interface is fixed beforehand. We assume it to look like shown in Figure 12.8. A data base would probably also have an administration interface, which we disregard for this example, however. Note how we restricted the possible values of queries on the logic side. Thus we have $\llbracket \text{DB-LO} \rrbracket_{\text{Out}} \subset \llbracket \text{LO-DB} \rrbracket_{\text{In}}$, while we hold on to $\llbracket \text{DB-LO} \rrbracket_{\text{In}} = \llbracket \text{LO-DB} \rrbracket_{\text{Out}}$, that is, for

$$S = (\llbracket \text{DB-LO} \rrbracket_{\text{Out}} \cap \llbracket \text{LO-DB} \rrbracket_{\text{In}}) \cup (\llbracket \text{DB-LO} \rrbracket_{\text{In}} \cap \llbracket \text{LO-DB} \rrbracket_{\text{Out}}),$$

we have that $\llbracket \text{LO-DB} \rrbracket_{\text{SM}} \stackrel{x}{=} \llbracket \text{DB-LO} \rrbracket_{\text{SM}}$.

In the given example, the UI-Port of the overall system is a relay Port, i.e. it has no implementation at all. The FE-UI-Port may be a SubSystem, the FE-LO-Port may be realized as an $\llcorner \text{active} \rceil$ Class.

12.2.2. Behavior

After the functionality and interfaces have been specified, we can now design the behavior of the three subsystems. One of the central context conditions has been to reuse the existing database, so we just have to accept whatever is defined there and make the best of it. So, we are done designing the behavior of Database. For design debugging purposes, we assume that the behavior model of Database is specified as a STD, and that it abstracts from the data contents, but counts the number of locks held on any table.

Frontend

Concerning the Frontend, its behavior can easily be derived from the storyboard of Figure 12.11. The resulting STD is shown in Figure 12.14.

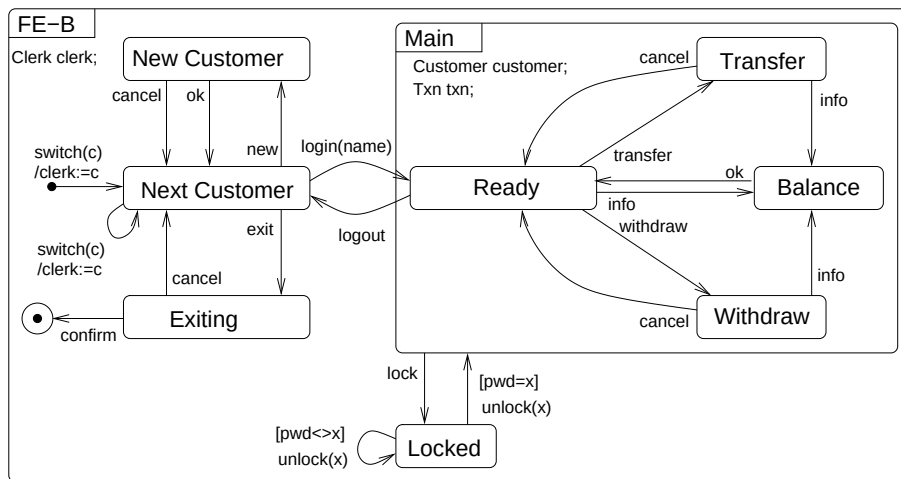


Figure 12.14.: The behavior of the Frontend-component. It may be derived from the FE-UI ProtocolRole (see Figure 12.12), but may also introduce changes for technical reasons, e.g. simplifications and refinements.

This design model is then given to the sub-contractor, who is responsible for providing the Frontend exactly matching this specification, or else. Upon delivery, the resulting component can be tested against the specification, by checking first the consistency of the delivered Capsule, and then the consistency of the BSS Capsule. If any inconsistencies are discovered during this process, these have either been present already in the original specification, i.e. they are our own fault. Or they are due to an inconsistent implementation on the side of the contractor. So, errors may be clearly attributed to someone.

Logic

The application logic, on the other hand, formalizes the business process knowledge accumulated over the years. It is the key to the whole architecture, so we specify, design and implement it ourselves.

Consider again the sunshine-scenario for the use case “Withdraw” in Figure 12.9. This scenario is given at user level, that is, we have so far left open how the function is realized by the system. On the other hand, we have already defined the functionalities of Logic and Frontend, and the interfaces

between them, FE-LO and LO-FE (see Figure 12.13). So we may refine the user level scenario of Figure 12.9 by the system scenario shown in Figure 12.15. Note the roles BSS-UI of Figure 12.9 and Frontend of Figure 12.15 match.

Observe, that there are use cases of similar name in the frontend and logic subsystems. How they are related precisely is the issue addressed by the logic subsystem's behavior view, see Figure 12.16.

There are several techniques for getting to this behavior model, typically, a mixture will be used. On the one hand, as the interfaces to the frontend and the database are already fixed, one may simply try and make ends meet. Additional inspiration might be drawn from considering the scenarios of the functionality view. On the other hand, we may choose to first define the behavior, and then derive the ProtocolRoles of FE-LO and LO-FE.²

12.2.3. Structure

Again, we need not bother with the Frontend and Database subsystems. We have seen another example of structural refinement in the very beginning of this chapter, namely the refinement of BSS as given in Figure 12.1. Refining Logic is quite similar to the process defined in this chapter, and it should be—Capsules have a recursive structure, after all. Again, the various views need definition, one by one.

At this point, reconsider Sections 10.2.5 and 9.4.5. There, we have seen an example for the strict interpretation of a StateMachine, and the property where such an interpretation might be used, respectively. The situation we have described there arises exactly at this point: here, it is useful to check whether the actual behavior specified for Ports really matches the behavior specified for the Protocol of the Connector between these Ports (cf. Figure 10.3 on page 185).

The functionality view of Logic has been given in Figure 12.6 already, the context and domain views are the same as for BSS as such. As there is no more deployment within Logic, there are only SimpleConnectors and relay Ports. Defining the structure is, probably, again influenced by the goal to reuse components from some library. For simplicity, we assume that the structure view of Logic is defined like in the left half of Figure 12.17 (the other half of the figure is explained in Section 12.3 below). Apart from the Configuration, the structure view of Logic may also contain test cases, component dependencies (e.g. as a component diagram), and an execution model (e.g. as object diagrams). Having seen this whole refinement process with BSS before, we stop at this point and do not go into further detail.

In a real world application, the process of structural refinement would probably go on for several levels of abstraction. For this example, however, assume that the team decides that the present specification is detailed enough for implementation.

12.3. Implementation

12.3.1. Logic

Depending on the abstraction level of a Capsule, different procedures may be appropriate for the implementation of a Capsule. For high-level Capsules, the implementation is derived from its structure: the implementations of the respective sub-Capsules are connected according to the structure of the Configuration. So, the structure provides the make- and link information, and no additional code needs be provided in the implementation view. The implementation view, then, contains only information

²An interesting and fun approach is role-playing, where (teams of) developers take on the role of Capsules or Ports, and information is passed on sheets of paper.

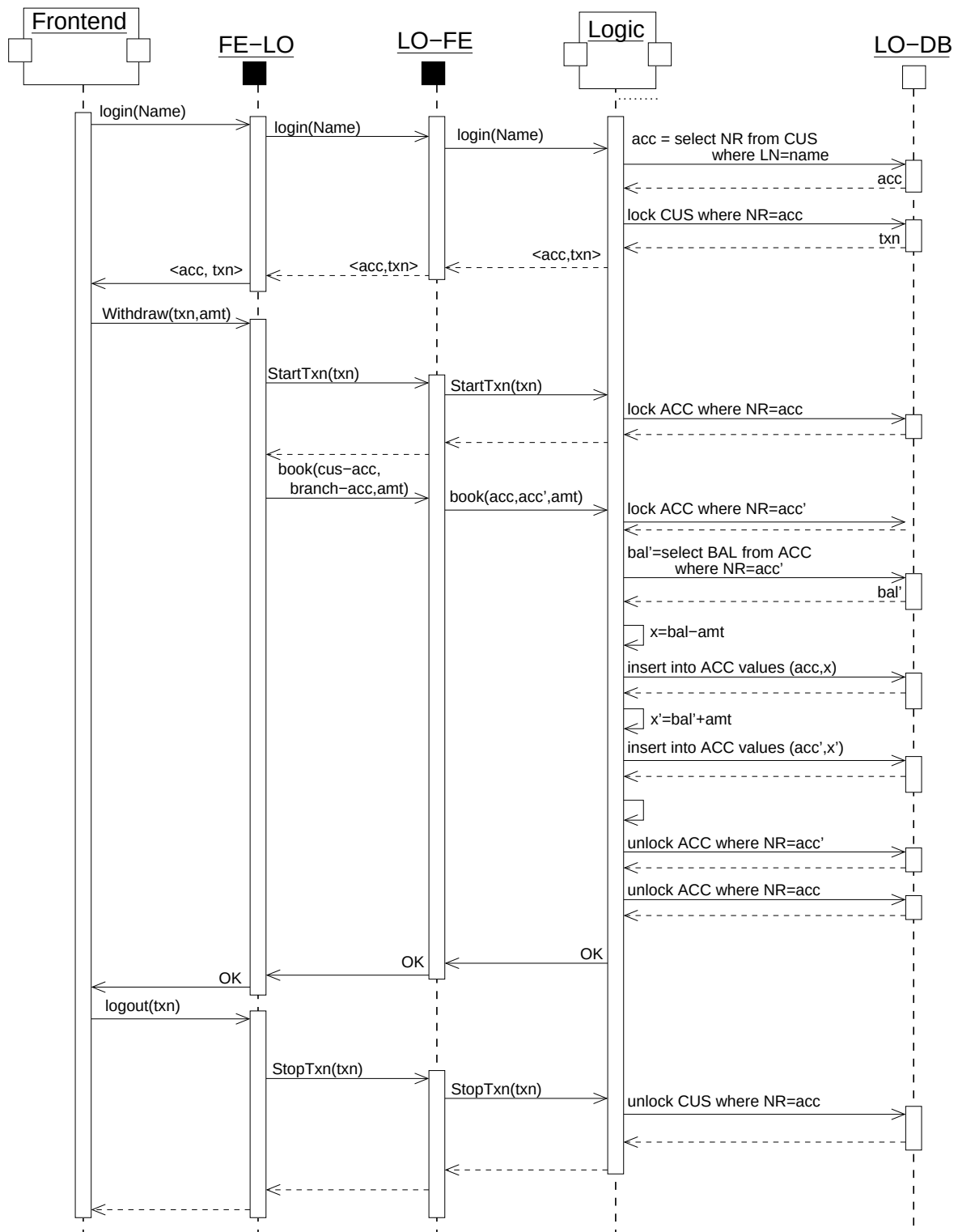


Figure 12.15.: The Withdraw scenario in greater detail. We omit the Messages transferred between Capsules and their respectivePorts. We omitted FE-UI and DB, as these are irrelevant for this scenario.

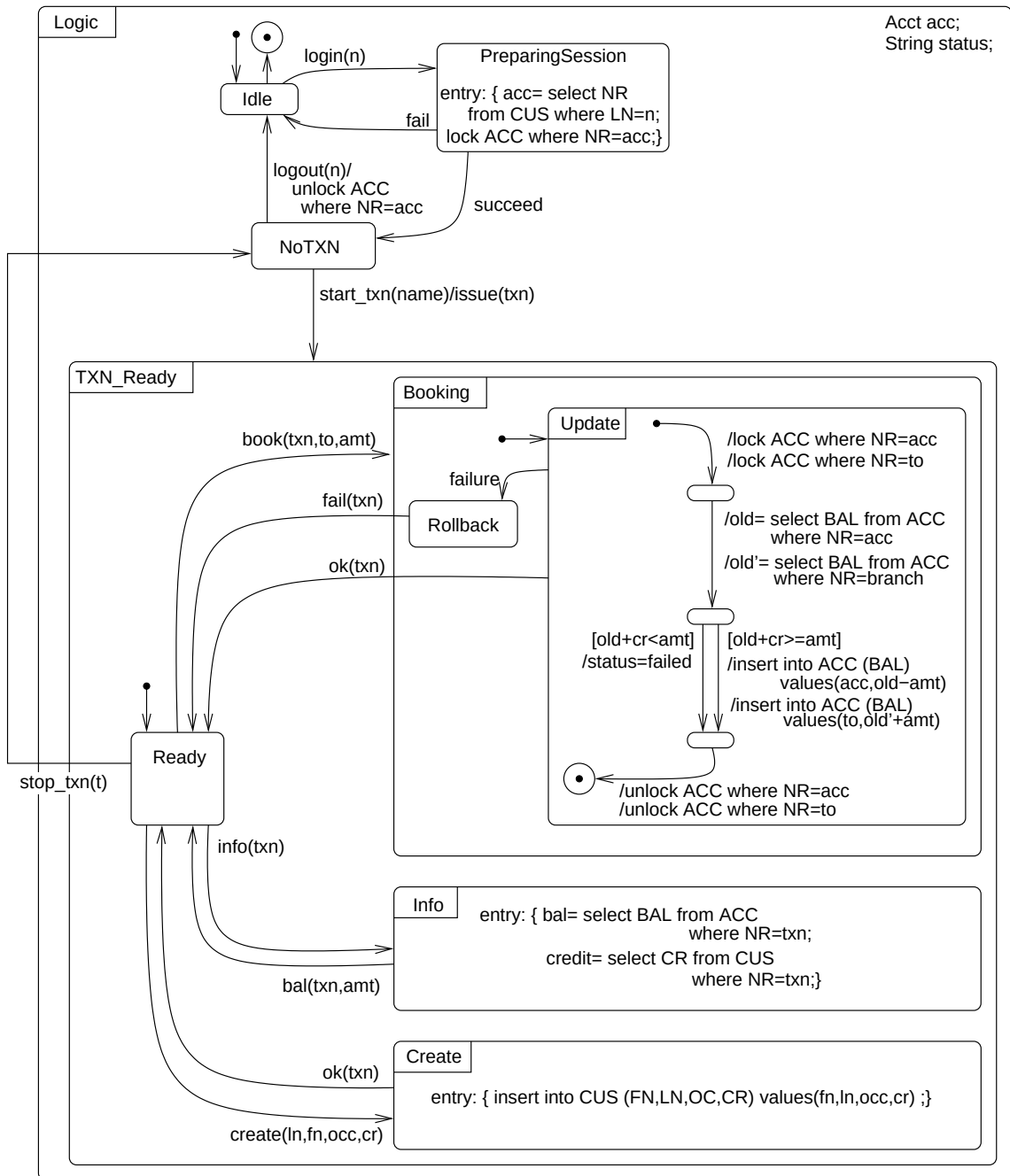


Figure 12.16.: The behavior of the Logic subsystem: while the overall structure is (necessarily) reminiscent of the behavior of the frontend-interface, the States for the core use cases show how the user requests are mapped into data base queries.

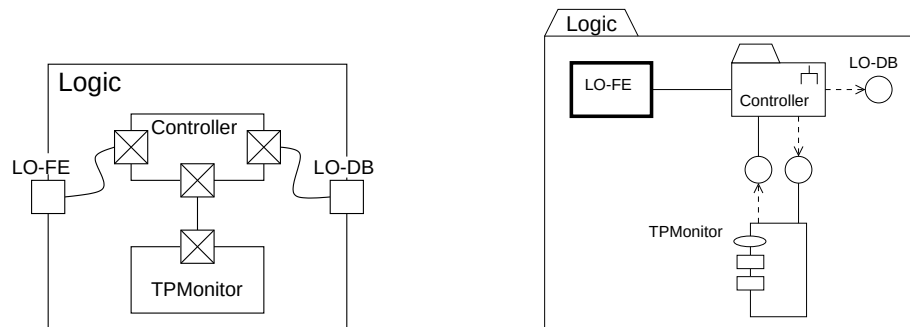


Figure 12.17.: Realizing Logic: the structure view (left) and implementation view (right).

such as execution models or integration test data, and possibly test-code, but not the implementation proper.

Medium-level Capsules may provide both a structure and an implementation view with real code, capturing the logical and the physical structure in different views. Both the structure view of the Capsule and the implementation views of its sub-Capsules may or may not be used (we will see an example below). For low-level Capsules, it might be unreasonable to define a separate logical structure in the structure view, since the functionality is so simple that it can be coded directly and without problems. So, there is no structure view at all.

Going back to the example, we have decided that Logic is a medium-level Capsule, so that we can go for a straight implementation. There are now four entities for which we need to provide an implementation, Controller, the TP-Monitor, and the two Ports of Logic, LO-FE and LO-DB. The LO-FE Port is realized as an `«active» Class` using library elements to handle the remote connection and all issues related to data transmission. Realizing its `ProtocolRole`, it provides and requires an interface for the outgoing and incoming Signals, respectively. The LO-DB Port becomes simply an Interface of type `SQL-Query` required from Database for the LO-DB is really just a relay to a simple Connector. It might be decorated with some OCL pre- and postconditions. The Controller becomes a `SubSystem` and require further development effort, though more along the lines of traditional programming rather than architecture-level design. For the TP-Monitor, let's assume that we find an appropriate Capsule in the project repository. So, the implementation view would look like in Figure 12.17 (right), omitting test data etc. for now.

12.3.2. Low level Capsules

Frequently, we encounter legacies, i.e. architectural units without (known) internal structure. Examples include (1) small units like modules or classes that are to be used as they are (e.g. elements of a library or framework); and (2) wrappers for legacy applications or the legacy application proper. Any realization of architectural units should be sufficiently flexible to cover such units, too. As a *pars pro toto*, we now show how Java Beans can be represented by Capsules.

Java Beans

A *Java Bean* (or bean, for short) is a Java `class` satisfying certain syntactic conventions. It is supposed to be “a reusable software component that can be manipulated visually in a builder tool” (cf. [Flanagan, 1997, p. 178]). This definition is strongly reminiscent of UML Components, so we can actually

use beans as an example for programming components. Beans may be of varying size, ranging from individual AWT classes, say, to whole spreadsheets hiding behind a single class.

A bean has *properties*, *events*, and *methods*. Properties are private attributes that may be accessed by setter and getter-methods: for private every attribute `Attribute` of type `T`, there are two methods `public void setAttribute(T value)` to update its value and `public T getAttribute()` to retrieve it. All other public methods are just called methods. A bean defines an event if it provides methods for adding and removing event listener objects from subscribers of that event. A bean has no public attributes, except static attributes, i.e. constants. For a more thorough introduction to Java Beans, confer to either of Englander [1997], Flanagan [1997], Voss [2000].

Since a bean is a plain Java class, it may be compiled for and executed on an appropriate target machine, or a Java Virtual Machine (JVM). A bean usually relies on the standard Java libraries as its runtime environment. In our terminology, both beans and the runtime environment are architectural units.

Beans vs. Capsules

Similar to architectural units, beans may exist on all levels of granularity. Beans, however, are implementation-level constructs, lacking support for design-time manipulations. So, it is of course possible to realize them as Capsules. This makes sense e.g. when there are legacies or other run-time entities that are to be integrated.

The opposite however, realizing Capsules as beans, is possible only for very low-level architectural units, and only by replacing all other views by a piece of natural language comment, or omitting them altogether. Since one would give away exactly the benefit of Capsules over implementation level components, this is sensible only in rare cases.

Representing a bean as a Capsule

Programming components such as beans have certain disadvantages over design components such as Capsules. First, there are no well-defined POIs: anybody may access all (public) methods in every sequence. While possibly dispensable and even bothering for low-granularity components, such POIs are vital for larger subsystems, as these inevitably have a state that determines their reactions to a good degree.

Second, a bean consists only of code (plus natural language comments), but lacks design information. There is no abstract specification of its behavior, functionality or underlying domain model. The administrative aspect, that is, e.g. its development status and history, is not contained in a bean. Consider the sample bean in Figure 12.20.

So, when striving for reuse in a systematic development process, it is desirable to use design rather than implementation components, even for low level entities. Basically, a bean can be turned into a Capsule by the following steps.

- An interfaces view with a single Port is added to represent the methods provided by the bean. The ProtocolRole of the Port has the methods with their parameters as incoming Signals. The types of the return values are the outgoing Signals. The behavior is a StateMachine with an initial state, a terminal state, and a single proper state that allows all incoming Signals to be taken as triggers, and return the result values of the respective method-call as effects. Preconditions of methods are mapped directly to the guards of the respective Transition. Postconditions can be encoded in the entry-action of the state reached by that transition. The behavior of the

ProtocolRole associated to the provided interface would look like the STD in the schematic example in Figure 12.18.

- More Ports may be added to allow for dependencies to other Capsules, such as representing other beans. The behavior of the required interfaces ProtocolRole is a similar StateMachine, but with triggers and effects, and incoming and outgoing Signals reversed.
- Possibly, it may be useful to add a specific “classification”-viewpoint for bean-Capsules, that can represent the characteristics of the bean, if these are provided by the bean already (this should be the case e.g. for frameworks).
- Additional views may be added as appropriate to represent the aspects behavior, functionality, administration and so on.

By these steps, a faithful representation of a bean as a Capsule has been achieved. See Figure 12.18 for a sketch of the schematic translation. With this schematic translation, little advantage has been taken of the concepts specific to Capsules – this is due to the immanent restrictions of beans. Beans whose interactions depend on their state are not adequately captured by a STD as given in Figure 12.18.

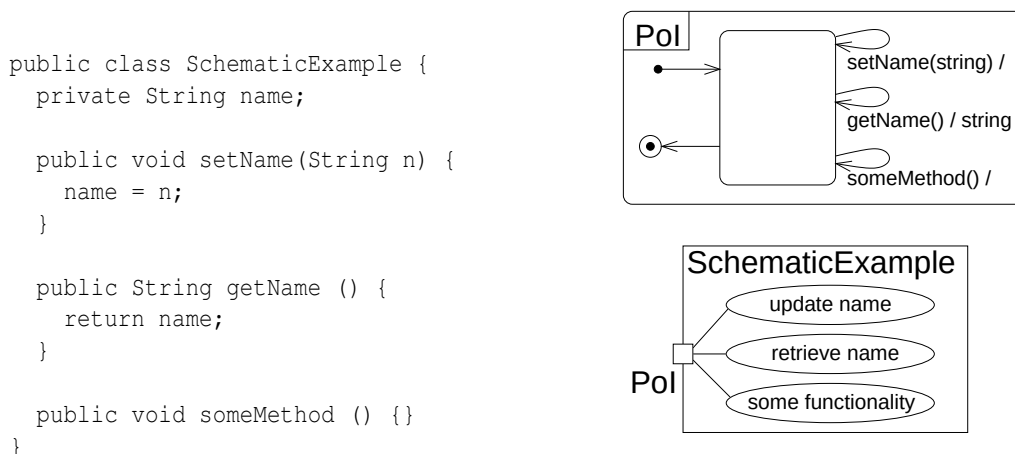


Figure 12.18.: Schematic translation of beans into Capsules: bean fragment (left); STD of the POI of the corresponding Capsule (right top); functionality view (right bottom).

Example

As an example, consider the TP-Monitor of Logic. Suppose we have a bean that realizes a very simple transaction processing monitor, and we want to be able to use it as an architectural unit. The functionality is simple: jobs can be submitted to the monitor as long as it has space in its queue, and the current job can be terminated. In Figure 12.20, there is a code skeleton to implement such a bean. Figure 12.19 shows the behavior view and part of the interfaces view, namely the behavior of the ProtocolRole of the Port of the Capsule representing this bean.

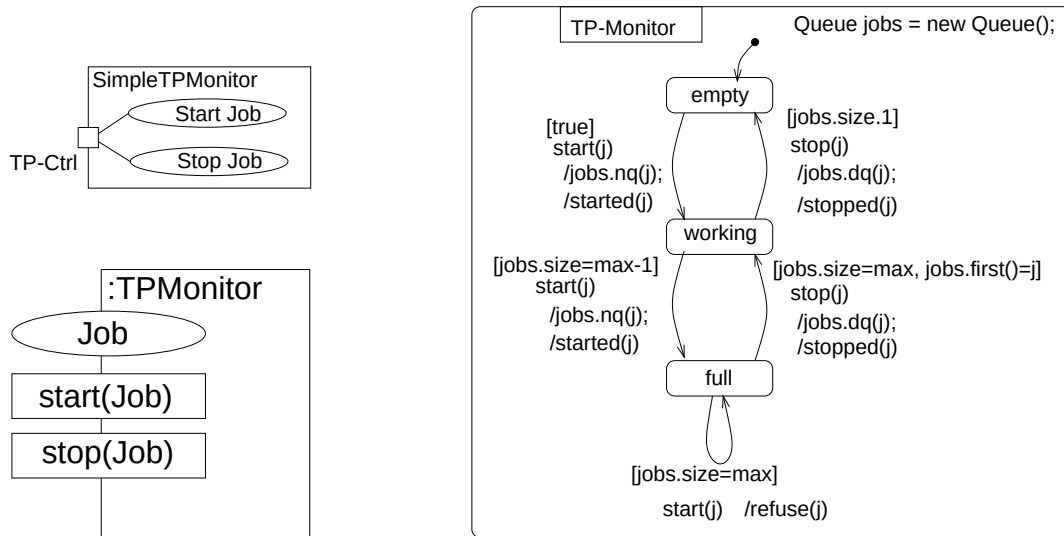


Figure 12.19.: The Capsule TP-Monitor: functionality view (left top); behavior view (right); and implementation view (left bottom).

12.4. Process patterns

So far, we have described the development process in intuitive terms only. In this section, we will present it again, structured by the process patterns of Chapter 11. Consider first Table 12.1, which maps of the Figures presented above to their function in the development process.

view	BSS	Frontend	Logic	Database	TP-Monitor
context	5.12 (a)				
domain	12.4, 12.5				
functionality	5.12 (b), 12.9	12.6			12.19 (left top)
interfaces	12.11	12.11, 12.12 12.13 (left)	12.8 (left), 12.13 (right)	12.8 (right) 12.8 (right)	†
behavior	12.9	12.14	12.16	†	12.19 (left bottom)
structure	5.12 (c), 12.7, 12.15	†	12.17 (left) 12.17 (left)	†	—
deployment	5.12 (d), 5.13, 12.1	—	—	†	—
implementation	—	—	12.17 (right)	12.2	12.19 (right), 12.20

Table 12.1.: Mapping of Figures in this section to views of the subsystems. † indicates a model which is present, but which is provided by other sources, and which we do not care about.

The overall construction process is shown in Figure 12.21. It calls on the realization sub-process, which is shown in the middle. This process calls the refinement sub-process in turn, which is shown on the right. The refinement again calls three sub-processes, one for the construction of each of the three Capsules in the Configuration described in the static structure view of BSS.

We present the construction processes for the subsystems Logic and Frontend in Figure 12.22.

```
package Test;

public class Job {
    private String name = ``default_name";
    private String status = ``idle";
    public String getStatus() {return status;}
    public Job(String theName) {
        name = theName;
        status = ``idle";}
}

public class SimpleTPMonitor {
    // A constant indicating the maximum number of jobs controllable.
    // This value may be changed to adapt to actual conditions.
    public static final byte MAX = 5;

    // The queue is implemented as an array,
    // held in a private variable called q.
    private Job[] jobs = new Job[MAX];

    // The variables first and last point to the first and last items
    // in the queue.
    private byte first = 0;
    private byte last = 0;

    // Enqueue a new job
    public void nq(Job newJob) throws TP_Error {
        if (first != last+1)
            {jobs[last] = newJob;
             (byte) last = last++ % MAX;
            }
        else throw new TP_Error();
    }

    // Dequeue the first (ie oldest) job, if its finished,
    // and if it is the job specified for termination.
    public void dq() {
        (byte) first = --first % MAX;
    }

    // Determine the first item
    public int getCurrentJob() {
        return jobs[first];
    }
}
```

Figure 12.20.: The skeleton if a Java implementing a simplistic transaction processing monitor (see text for a description). The (trivial) exception `TP_Error` is omitted.

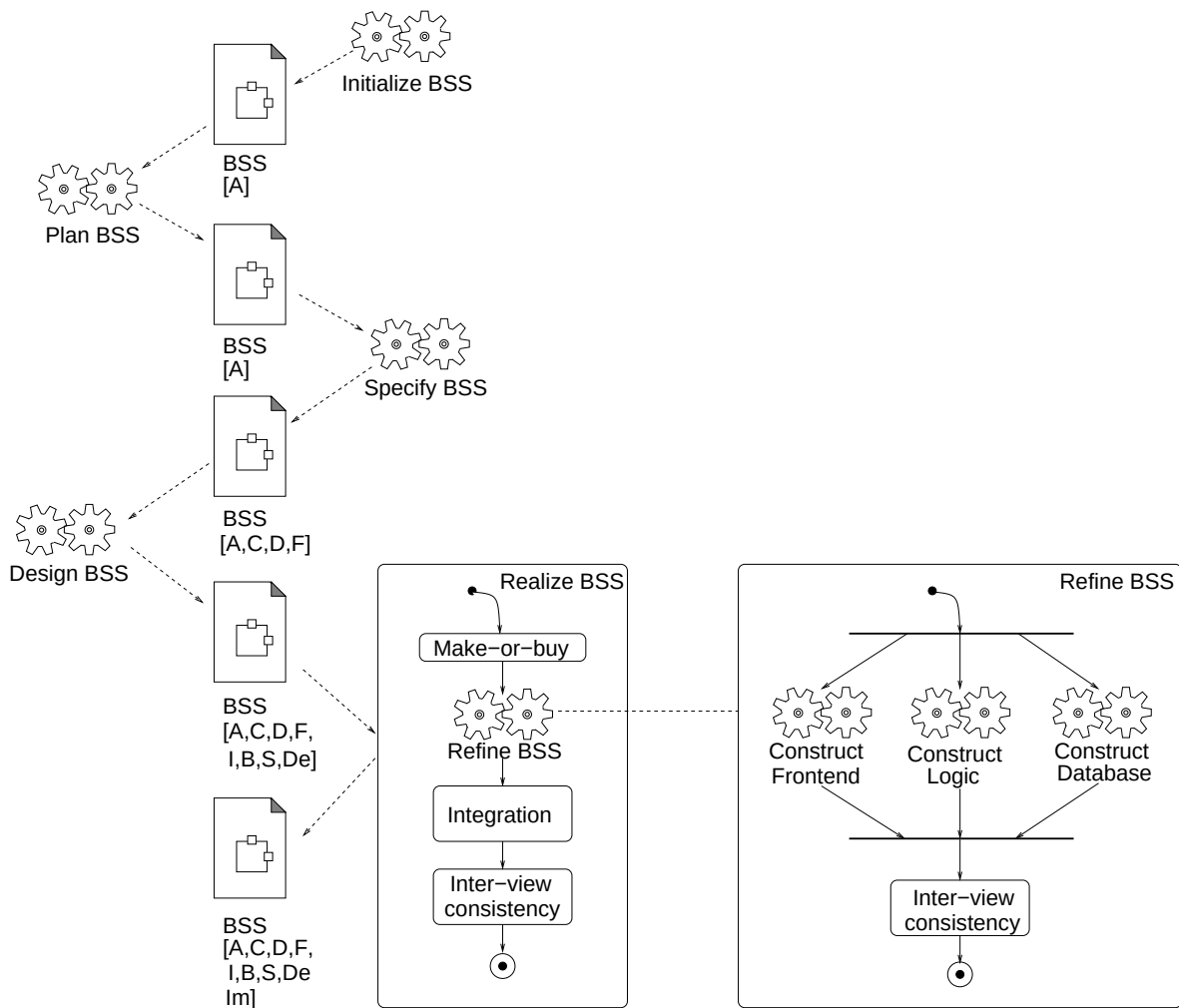


Figure 12.21.: The development process for constructing BSS. The sub-processes for constructing the subsystems Logic and Frontend are shown in Figure 12.22.

Again, their realization phases are shown in greater detail. Realizing Logic uses the refinement pattern and thus calls on the construction pattern recursively. Realizing Frontend uses the procurement pattern. The supplier may use a similar process than we do, so that he, too would use the construction pattern again recursively. He may also choose not to do so, and use some proprietary process model. Either way, the deliverables are clearly defined, and can be analyzed for consistency, both by the supplier and the principal.

12.5. Design debugging and maintenance

Up to now, we have created a large model. Its consistency may be enforced using process patterns the way shown in the previous section. However, creating a consistent model is not the hardest problem, anyway. It is much more difficult to change an existing model in a consistent way, particularly, if we have to change a model we have not created ourselves, or which we have created a long enough time ago to have forgotten about the details. We will now show how this may lead to subtle, but significant

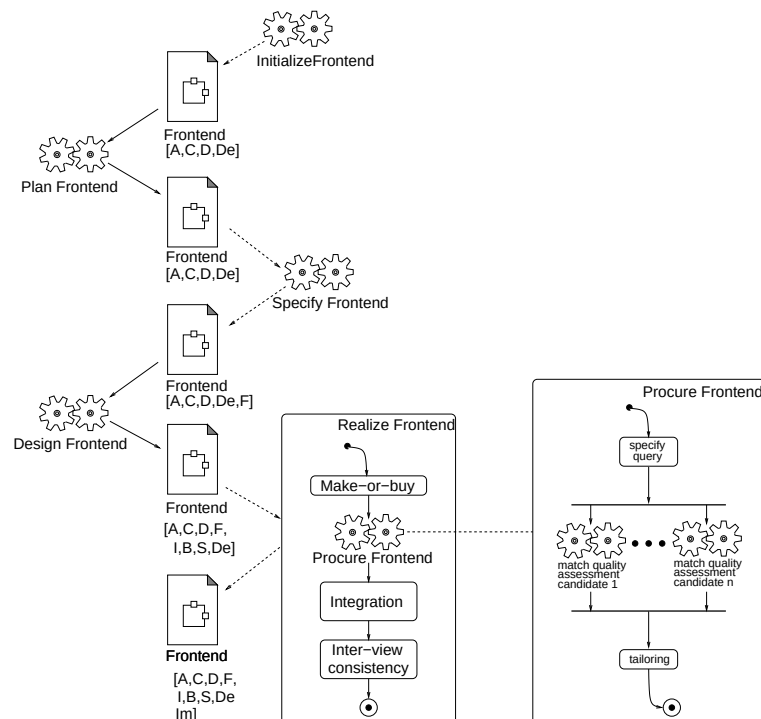
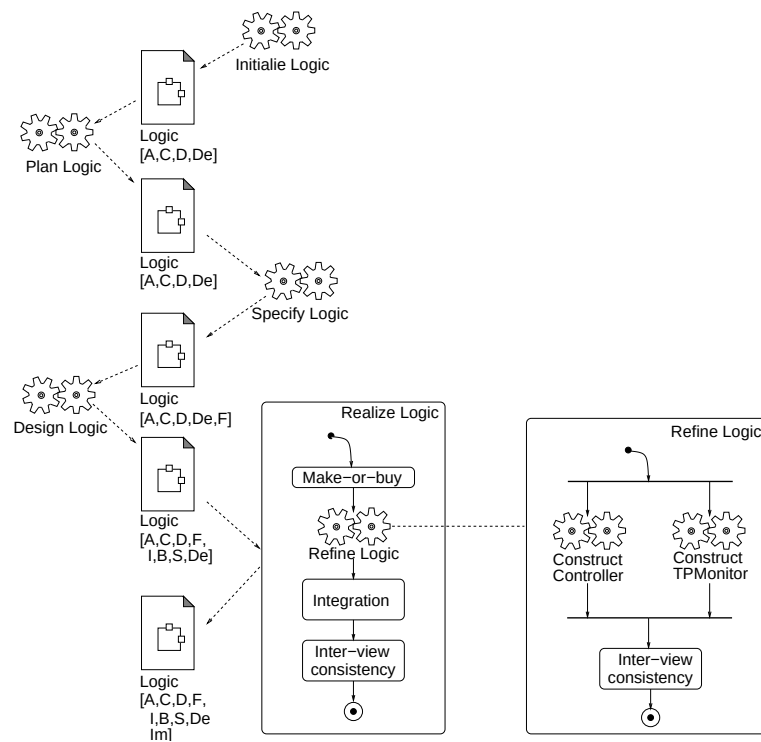


Figure 12.22.: The development processes for constructing the subsystems Logic (top) and Frontend (bottom). The processes for realizing BSS, which calls upon these two subprocesses is shown in Figure 12.21.

errors.

12.5.1. Tuning

Suppose that the system we have constructed is indeed consistent, but exhibits inadequate performance. Assume further, that the reason for this can be tracked down to a large number of transactions that are started, but not completed, or lie idle for a long time. Even such suspended transactions occupy memory and thus decrease performance. The designers of the Logic-component decide to introduce a time-out to get rid of such deadwood transactions. Thus, they change the behavior view of their subsystem by adding an arc from TXN_Ready to the terminal state—this is the only difference to the STD of Figure 12.16.

This change may seem quite trivial and local, but has quite severe implications on the overall system. For instance, *all* sub-States of TXN_Ready may be exited, including booking. This amounts to aborting a transaction without rollback, and so, the database may end up in an inconsistent state. This error can be detected, as the locks set within TXN_Ready.Booking.Update would not be released on a time-out. An error trace may be discovered by checking the following property:

Considering the composition of the behavioral models for Logic, Database, and the Ports between them, is their combined initial state a home state, i.e. may it always be reached again? The combined initial state is Logic.Idle, LO-DB.Ready, DB-LO.Ready, and DB.⟨locks = 0⟩.

This error obviously may be detected with the techniques available to us, an error-trace is given as a sequence diagram in Figure 12.23.

This error may be solved by replacing the arc we have added by a similar arc which goes from Logic.TXN_Ready.Ready to Logic.Idle. This solution, however, still does not solve the problem: the lock acquired in Logic.PreparingSession is still not released. This error may be found by asking the same question again. The shortest error trace is like the one presented in Figure 12.23 until the first lock is acquired, and then ends with a time-out. This problem can be fixed by releasing the last lock explicitly by the time-out arc, i.e. by copying the effect of the arc from Logic.NoTXN to Logic.Idle.

12.5.2. Rearchitecting

Suppose now, that tuning has somewhat improved the performance of BSS, but not sufficiently. Further investigation reveals, that the Connector between Frontend and Logic is a bottleneck: the amount of data that needs to be transferred can not be handled with the available bandwidth. Thus, the problem may be due to a wrong architectural decision, and so it might be solved by reallocating the sub-Capsules in a different way.³ With this perspective, a study of an alternative architecture is launched, complementary to the development paths already documented in the context view (see Figure 12.3).

The overall process of rearchitecting is managed by the modify pattern. We then apply the refactoring and change impact assessment patterns. Figure 12.24 gives an impression of the process of rearchitecting. Note that the Connector between Frontend and Logic is a ComplexConnector, modeling a remote connection. By rearchitecting, this Connector is now between two Capsules within the new Logic subsystem. On the other hand, a SimpleConnector is turned into a ComplexConnector. Inevitably, this will lead to major implications, that we discover using the change impact assessment pattern.

³A similar example is found in Noack et al. [2000], where the migration from thin client-architectures thick client-architectures is described, including quantitative guidelines when to prefer which solutions.

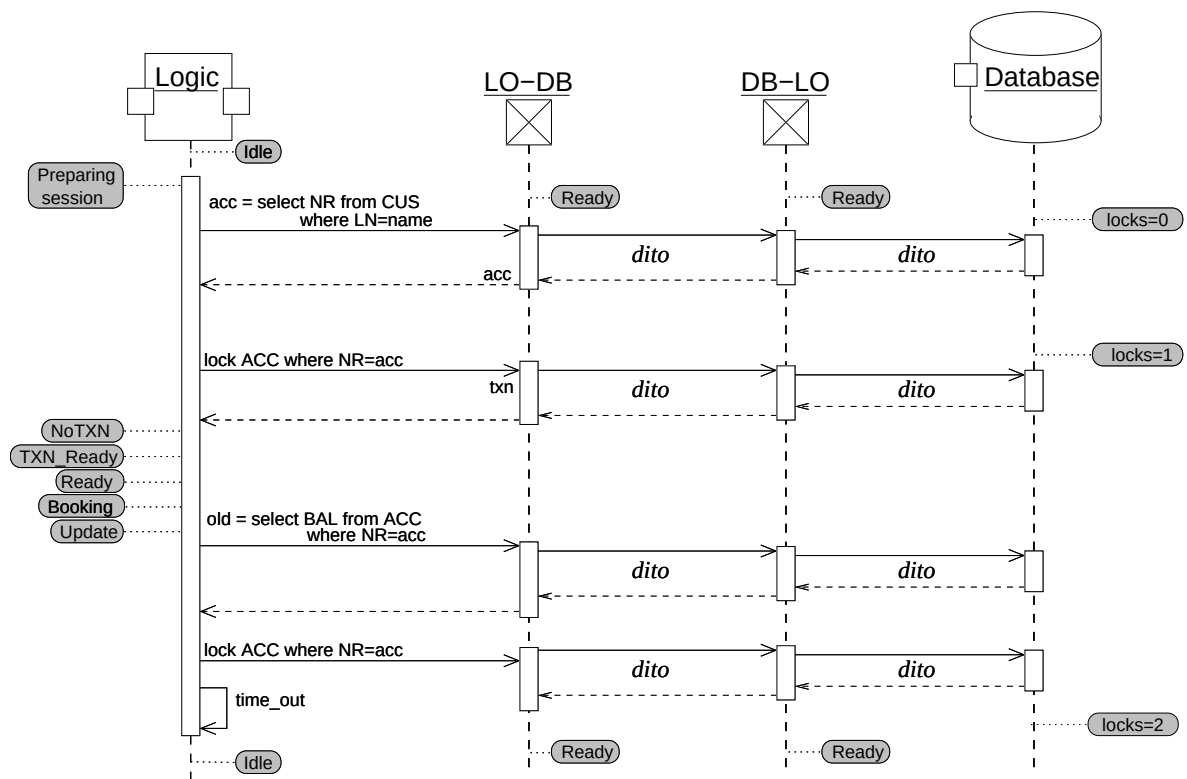


Figure 12.23.: An error trace for the first attempt to provide a time-out. For added clarity, we abbreviated the messages between LO-DB, DB-LO, and Database as *dito*, as they are identical to the messages emitted from Logic. The local states are shown in shaded, State-like boxes.

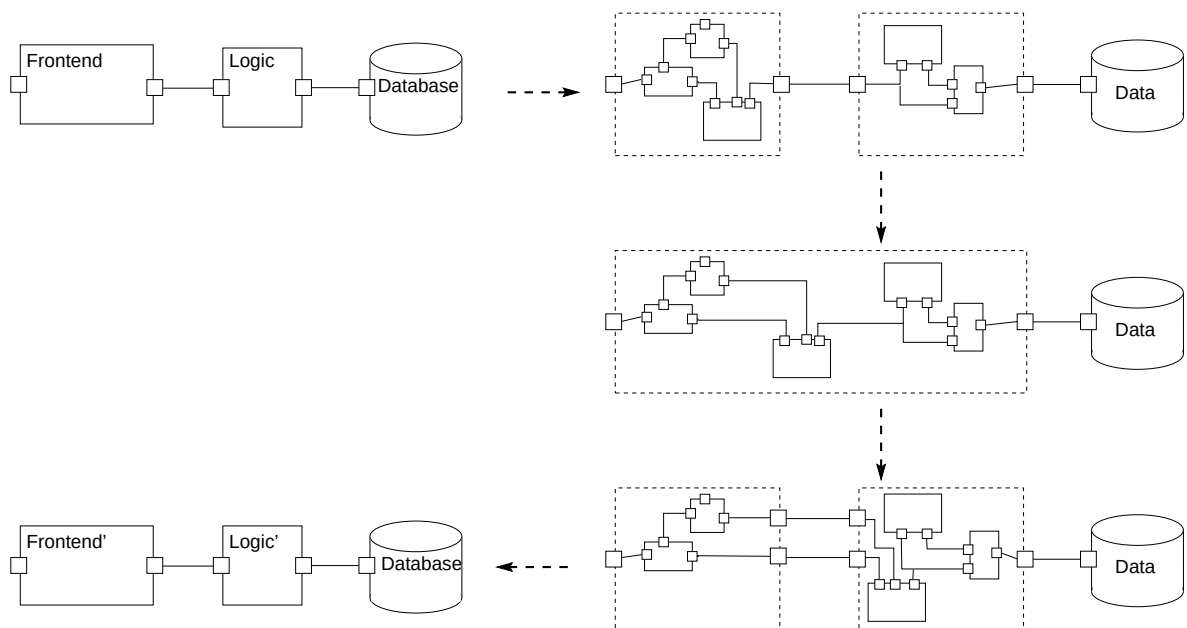


Figure 12.24.: Rearchitecting BSS.

12.6. Chapter summary

In this chapter we have provided an example to illustrate the concepts, notations, semantics, properties and process patterns that we have defined above. Of course, this example is by no means complete—our approach proved to be rather far reaching. Just how far it does carry, is as yet not clear. In the final Part of this book, we will go back to the initial requirements, and test them against what we have done.

Part IV.

Final Remarks

13. Results

In the following sections, we first sum up the results of our work. Then, we compare them to the initial goals and discuss their practical relevance. Finally, we go back to our initial vision, and reconsider it in the light of our work.

13.1. Summary

First of all, we define our approach in detail. Starting, on the one hand, from our vision of software architecture as a discipline comparable to building architecture, and, on the other hand, from some background knowledge of existing approaches to software architecture, the UML and formal methods, we first survey the state of the art in greater detail. The criteria for this survey are derived directly from our vision.

Our survey exhibits a number of deficiencies in existing approaches, concerning in particular the system context, views and viewpoints, and the interplay of reuse and architecture. So, we go about analyzing the stakeholders of software architecture and their stakes. This leads us to a set of requirements, culminating in the novel concept of architectural unit. Around it, we gather a set of concepts compiled from various sources. Their ensemble is crafted minutely to support our vision of software architecture.

We realize this conceptual framework as an extension of the logical metamodel of the UML. Great care is taken to embed our concepts in such a way, that the original structure and character of the UML is maintained. This way, all the benefits of the UML are immediately applicable to our concepts as well—notations, methods, and, to a slightly reduced degree, tools. As one true extension to the UML, we add the SARA-syntax, which is close to the traditional box-and-line-style of high-level structure diagrams, and which is also used in ROOM and UML-RT. Figure 13.1 gives an overview of this approach.

Next, we define a transparent formal semantics for a significant part of the UML metamodel, namely, the metaclasses Collaboration, StateMachine, and Capsule, and all the related concepts. Our semantics is extremely detailed, and, although it does not cover all features of the UML, it is much more complete than most others. Its definition is entirely formal. As the mapping from concrete syntax to the abstract syntax (i.e. the metamodel) is already defined in the standard, it is now possible to translate concrete UML models into their semantics completely mechanically. As the formality may be used, but does not show up, we call it transparent formality. We also define a couple of incidental semantics, like complexity measures of designs, an elementary quantitative model, and a simple model for code-dependencies. Figure 13.2 gives an overview of this approach.

To demonstrate the practical value of our approach, we have also defined a set of consistency conditions. Together with the semantics, they may be exploited to formally check the consistency of architectural models. Also, the meaning of architectural refinement and change impact can now be formalized. We provide a language of process patterns for the working on the architecture level. We also provide an extensive example to demonstrate the usage of the concepts, notations, properties and

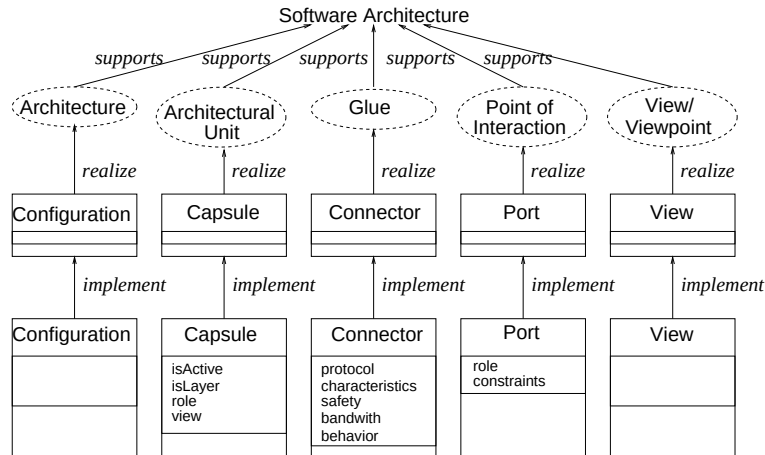


Figure 13.1.: A schematic overview of the most important concepts used in this book, and their relationships: The overall vision of software architecture is supported by a number of concepts, realized as classes in the (extension of the) UML logical metamodel, which are implemented as classes in a physical metamodel.

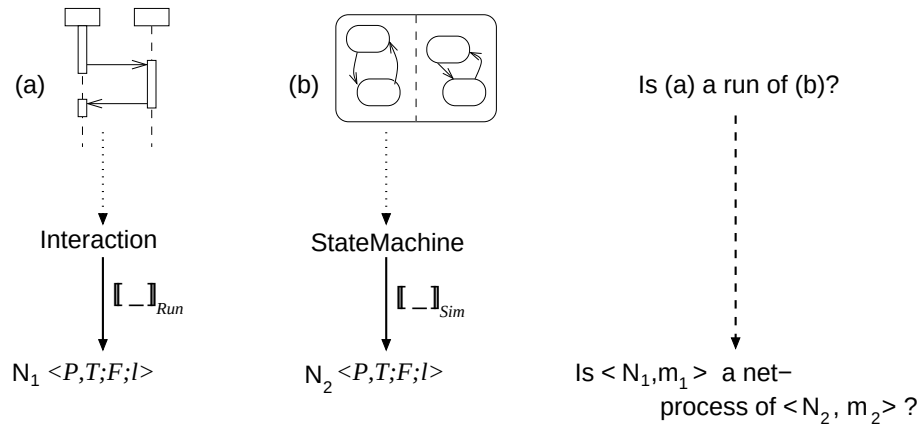


Figure 13.2.: A schematic overview of the semantics defined in this book, and their usage: the concrete syntax is mapped to items of the metamodel by the standard (dotted arrows). Our semantics allow mechanical translation from there into the semantic domain of Petri-nets (solid arrows). Thus, user level properties can be given a formal semantics (dashed arrow). The initial markings are denoted m_1 and m_2 , respectively.

process patterns practically.

13.2. Discussion

In this section, we first compare the initial research questions with the results we have yielded. Then we try to determine whether our results are of any significance to the rest of the world. Finally, we explore the limits of our approach to see how far further it might carry us when pursuing the roads of enquiry it has opened before us.

13.2.1. Results vs. goals—are the research questions answered?

First recall the goals we have put up in Sections 1.1.3 and 1.2.4 and check whether we have indeed reached them. We wanted:

a clear notion of software architecture, who uses it for what purpose and in which phase. ✓
We have elaborated this issue quite systematically in Chapters 1 and 3, so this is a clear yes.

We also demanded concepts and notations to support this notion of software architecture, ✓
and we did this in a particularly elegant matter, so this is a resounding yes, too.

Finally, we asked how these concepts and notations might be supported pragmatically, i.e., ✓
by tools and a development processes. We have provided an array of formal and analyzable properties, a language of process patterns, and a large example for this purpose, so again, we check this issue.

Going into greater detail now, we resume the survey started in Chapter 2, and compare our results to the best one or two contenders (see pages 18, 21, and 25).

We started with a requirements analysis that led us to a set of concepts, in particular a notion of architectural unit. We realize it by Capsule, whose closest competitor is UML's SubSystem. In direct comparison, we find that Capsules are much more self-contained and precise.

Approach	encapsulation and self-containedness	architecture level abstraction	phase coverage	precision	proximity to UML
UML SubSystems	(✓)	✓	✓	(×)	✓
our approach (Capsules)	✓	✓	✓	✓	✓

Next, we considered languages for architectural modeling. Here, we distinguished mainly between ADLs and general-purpose software modeling languages. The two closest competitors in these fields are Wright and HOOD. Our approach, shares the precision and breadth of concepts of Wright, and the coverage of all phases of HOOD.

Approach	concepts				expressive notations		early phases	precise semantics	process support
	view	style	POI	context	structure	behavior			
Wright	×	✓	✓	×	s/–	✓	×	✓	×
HOOD	(×)	×	(✓)	(✓)	s/–	✓	(✓)	(✓)	(✓)
our approach	✓	✓	✓	✓	s/d	✓	✓	✓	✓

Then, we turned towards formal semantics of UML. The two closest competitors here are the works of Knapp and of Lilius and Paltor. While the former is a landmark in precision, it is restricted to a small portion of UML, and is not well suited for practical purposes. The latter, on the other hand, leaves out exactly the tricky aspects and features of the formalization.

Approach	UML based	Metamodel as domain	completeness	precision	scope	codomain
Lilius and Paltor [1999b]	✓	×	(×)	(✓)	–/d	PROMELA
Knapp [2000]	✓	✓	(×)	✓	s/d	Maude, LTL, event spaces
our approach	✓	✓	(✓)	✓	s/d	Petri-nets

Finally, we considered the pragmatism support available for working on the architecture level. The two closest competitors are the Unified Process and HOOD: the former is geared towards UML and explicitly introduces architectural artefacts and the role of architect, but fails to address reuse and evolution adequately. The former does address reuse to some extent, and has less technology bias, but does not cater for architecture very well. Among other things, there are no architects in HOOD. In this last comparison, proc. and maint. stand for procurement and maintenance.

Approach	precision	proc.	reuse	extent maint.	a	s	p	all stake- holders	recursive product	recursive process	tech. neutral
UP	✓	×	(×)	(×)	✓	(✓)	×	✓	(×)	×	×
HOOD	(✓)	×	(✓)	(×)	✓	×	×	(×)	✓	✓	(×)
our approach	✓	(✓)	✓	✓	✓	✓	✓	✓	✓	✓	✓

So, judging by the criteria elicited as requirements right at the beginning of this book, we see our work significantly ahead of the competition in many respects. More important than each of individual contribution, however, is the ensemble, the approach as such and the road to further enquiry it opens up.

13.2.2. Relevance—do the answers really matter?

Having indeed reached the research objective—does it really matter? That is to say, is the result in any way relevant? So far, we have appealed to the reader's intuition that the target of our efforts and the approach taken are indeed the right ones. But we can do better than that: there is empirical evidence in favor of our position, too.

According to many different sources, about 60-70% of the overall cost of software is spent on maintenance, see e.g. Boehm [1981], Brooks [1975], Ghezzi et al. [1991], Lehman [1980], Weinberg [1978]. During maintenance, software engineers are busy understanding the current status of a system, the change request and the impact it would have, see Figure 13.3. Speeding up this process by adequate documentation and tool support obviously can cut development costs and time-to-market significantly.

47%	understand status and request	that is:	18%	study request
			6%	study documentation
			23%	study code
25%	implement change		6%	update documentation
			19%	implement
28%	test implementation of change			

Figure 13.3.: Time spent on activities during maintenance; figures after Fjelstadt and Hamlen [1979].

So, the target of architectural evolution is indeed promising, but can it be attained by the path we pursue, by debugging architectural designs? It has long since been known, that the most important factor in software engineering productivity is personal capability. Boehm reports variations in individual productivity of up to 1:4.18 (see [Boehm, 1981, p. 642]). Clearly, there is very little that any notation, semantics, tool or method can do to improve personal capability.

Thus, Boehm names two major controllable factors in software productivity, reuse and code generation (see [Boehm, 1981, p. 646]). Concerning software reuse, we have gone to considerable lengths

to ensure that architectural units meet all requirements, in particular self-containedness and support for automatic cataloging.

Concerning code generation, one has to consider the fact, that the number of lines-of-code produced is independent of the programming language (see e.g. Brooks [1975]). That is, using a higher-level programming language will result in more machine instructions per man-month being produced. Moving from programming languages to design languages like the UML can thus exhibit an altogether new degree of productivity. This is not to say that we expect the UML to entirely replace traditional programming languages. We are well aware that simply using UML instead of programming language will never work, as the developer would be swamped with detail. However, introducing several layers of abstraction has proved to be practical in industrial applications—and this is exactly what software architectures and architectural units are all about. Without them, the semantical models generated from designs would grow too large too quickly, and the analysis procedures of our transparent semantics would grind to a halt.

But simply moving to a higher level of abstraction is not good enough either. Fisher reports, that almost two third of all errors are introduced during analysis and design, but only less than one third of these errors are detected before user validation: according to [Fisher, 1991, p. 5], 64% (36%) of all errors are introduced during analysis and design (coding); of these, 30% (75%) are detected before user validation, i.e. during debugging. In total, this accounts for 44.8% of all errors (70% of 64%). So, between 45 and 65% (see [Boehm, 1981, p. 381-384]) of all errors in software development are introduced during analysis and design, but not detected then and there. This is the class of error we address with our approach. Since design errors proliferate through the whole system, early errors are dearest, i.e. these are also the most expensive errors: design errors spotted during implementation or deployment may be as much as a hundred times as expensive to fix than those detected during analysis or design already (see [Boehm, 1981, p. 39ff]).

Assuming that these figures are representative, and assuming furthermore that correcting errors accounts for approximately half the total development costs, every percent of design errors found by design debugging is worth at least half a percent of the total development budget.

13.2.3. Limits—how far does the approach carry?

Our work answers some questions, but raises even more new issues. Among the most pressing are extensions of our semantics, real-life case studies, and a tool implementation. We will now consider some of these issues in greater detail.

Semantics

First, there are certain limits on the semantics we have defined. On the one hand, there are some weak spots—not all cases are treated satisfactorily, e.g. NameSpaces like the contexts of a StateMachines, or the event queues. Then, there are some empty spaces in our semantics that we would like to see closed, e.g. some kinds of Action, guards, entry, do, and exit-activities in StateMachine. Beyond these, UML offers a baroque wealth of syntactic sugar (like some of the PseudoStates), which, in principle would need to be covered, too, but which provides little new insight or expressive powers.

Finally, we come to a more serious point. Transparent formality is restricted to theoretically simple properties: automatic analysis only works for special cases and comparatively small state spaces. This surely includes a large number of practically relevant properties, but excludes even more. For the time being, verifying such properties is a proof obligation that has to be discharged off by informal techniques, dedicated formal methods experts, or plain hand-waving.

Field application

Our approach is purely academic so far—it has not been field proven, but it is realistic. It has been used occasionally with students but not to the degree necessary to evaluate the contribution of this book in earnest. We have also not succeeded in obtaining real-life industrial application architectures as case studies. The reason is that companies would not disclose any such artifacts, however outdated or mutilated. Also, as a prerequisite for conducting big case studies, we would need tool support. There are prototype implementations (the PUSH-system and the CWB), but we have not reached presentable results here.

Also, practical experience is required to assess and calibrate the equivalence relations (Chapter 8) and the consistency properties (Chapter 10). Only practical experience will tell. Possibly, it will also turn out that additional viewpoints will be needed, maybe requiring additional semantical mappings.

Comprehensiveness

Our approach is restricted by the kinds of architectures it may be applied to. Recall the Perry and Wolf-slogan: “*Architectural Style = {Elements, Form, Rationale}*”. Following this definition, we have restricted ourselves to architectures that may be expressed by architectural units with views and POIs, connected by glue, and, as the rationale, addressing the concerns of the stakeholders identified and scrutinized in Chapter 3. This restriction is further tightened by the way we implement these concepts, i.e. by Capsules with Views and Ports, connected into Configurations by Connectors.

So, one could say our approach is restricted to a particular kind of architectures, to a particular style, just like C2/SADL. There are a number of possibilities of regaining some of the freedom, however. One example is the introduction of virtual Capsules, a concept that arose out of one of the few real practical applications. It is our hope that the restrictions we create with our approach will be more than justified by added value—we may not be able to capture all kinds of architectures, but for those we do, we can offer significant help, from concepts to CASE-tools. Also, we would hope that most of those architectures we are not capable of expressing are not architectures one would want to see put into action anyway. Just like using, say, Java does prevent some programs being written.

13.3. Computer Aided Software Architecture—a vision revisited

In the very beginning of this book, we used an analogy to building architecture to illustrate our vision of software architecture. Now, at the very end, let us reconsider this analogy. Obviously, building architecture has reached a professional level far beyond that found in software architecture today. Why is this so?

First, building architecture has several millenia of a head start. Even as a proper science and profession, there are still a good two centuries of experience in erecting buildings. This has, as one effect, lead to a diversified spectrum of professional specializations, ranging from architects, civil engineers, stress analysts, interior architects, via logistics experts and construction supervision, down to craftsmen like masons, carpenters, electricians, and plumbers. In software engineering, there is barely an established professional distinction between designers and programmers.

Also, buildings are part of everybody’s daily life. They are mostly static, and they are three-dimensional, restricted by the laws of physics. Program structures, on the other hand, are known only to few. Programs have behavior, and even dynamic structure. Programs are restricted only by the laws of logic, and there is a virtually unlimited number of degrees of freedom. So, the problems and

products of building architecture have immediate intuitive appeal absent from software architecture—no one demands another subterranean garage for a completed skyscraper.

The work on formal ADLs yielded plenty of insight, but has not had a lasting mark on software construction in practice. We believe this is due to insufficient consideration of practical concerns, especially with respect to user-friendly notations, process support, and design debugging techniques. We have reconsidered the requirements of architectural models and tried to address these shortcomings by joining together ADL concepts and the UML.

Looking back to our starting point, we have made a significant step ahead. Still, many questions remain unanswered. In fact, many new questions have been added to our list: it seems that UML is not yet ready for formalization—how can we achieve that? Is there a single right formalism? If so, which? What tool support can we give to software architects to leverage their work? And, actually, what exactly should be the role of the software architect? How can performance prediction be supported? Can quantitative measures known from programming be applied to designing as well? In fact, can we move from programming to a higher level of abstraction, namely designing, similar to the transition from Assembler languages to structured and modular languages as suggested by Selic [2000]? Or, to put it in the words of Stevens [2000], are we at the brink of “extreme modeling”? So, again, there is work to be done.

A. The Unified Modeling Language

A.1. Introduction

A.1.1. Historical overview

The UML is a merger of the three largest pre-UML object-oriented design languages. Its first draft version, 0.8, appeared in 1995. It was based on the two OO-design languages, namely OMT and the Booch-method. The OOSE-approach was incorporated in version 0.9, released in 1996. Version 1.3 of the UML was adopted as a standard by the Object Management Group (OMG) in mid-1999. An edited version labeled as “September 2000” has been published some time ago.¹ The one and only authoritative document on the current version of the UML standard is OMG UML 1.3. When referring to “the standard”, it is this document that we think of.² Other publication might provide interesting interpretations or comments, but they present subjective views only. In this appendix, we provide an overview over the UML, in particular its metamodel, that any reader unfamiliar with the standard might wish to consult. Also, there are some drafts and unofficial documents on future versions of UML: considerable changes are to be expected from the 2.0 version due sometime in 2001 (see Section A.4). However, judging by past experience, these preliminary documents do not represent a good approximation of the final result.

A.1.2. Structural overview

The UML has a metamodel, that is organized into packages (see Appendix A for an overview of UML). Underlying the UML is the 4+1-viewpoints³ model with a prominent role for use cases (see Kruchten [1995] and [Booch et al., 1999, p. 31]). A viewpoint can be instantiated to a view (“model”), which is expressed by one diagram.⁴ There are nine different diagram types: use case-, class-, package-, component-, deployment-, sequence-, collaboration-, state transition-, and activity-diagram, though the syntax is not really very strictly defined in many places—there are guidelines, but few strict requirements. The UML has its own extension mechanisms and a textual constraint language.

The UML pursues a meta-modeling approach. It defines four abstraction levels (see Table A.1):

¹Actually, the UML is currently not really standardized: the OMG is not a body capable of issuing standards, it merely “adopts technology”. However, the function, general perception, and de-facto significance is very similar to that of a standard, so it would be appropriate to speak of a quasi- or de-facto-standard rather than just standard. This distinction, however, is dropped in the remainder in favor of the simpler term standard. In the fullness of time, the OMG-specification will become an ISO-standard anyway.

²To be precise: the UML, marked with release date “Version 1.3, June 1999”, actually published in the internet on the 5th of July, 1999. In the meantime, at least one different version has been published, with the same version number. . .

³Kruchten uses the term view instead of viewpoint, but to avoid confusion, we adopt the IEEE AWG/P1471 terminology here.

⁴It is not entirely clear, whether a model is described by exactly one diagram, and thus whether every diagram type precisely represents a viewpoint. It might be the case, that some viewpoints are represented by sets of diagrams.

level M_0 (instance) Entities of this level describe actual objects and states of actual systems (applications). The entities on this level are objects and links, i.e. instances of the concepts of level 1.

level M_1 (model) This level provides the concepts of actual systems. Models of this level abstract from actual occurrences and present families of similar states and entities of actual systems. The entities of this level are classes and associations, i.e. instances of the concepts of level 2.

level M_2 (metamodel) This level provides the concepts that may be used for modeling actual systems. The entities of this level are metaclasses and meta-associations, i.e. instances of the concepts of level 3. Level M_2 is subdivided into the logical and the physical metamodel, each of which is structured by inheritance.

level M_3 (meta-metamodel) This level describes the concepts used to describe the metamodel, that is metaclasses and meta-associations. There is no level above level 3: this level is defined in terms of itself. The UML uses the OMG's Meta Object Facility (MOF) (see Crawley et al. [2000], OMG MOF 1.3), shared with the other OMG standards like CORBA and BOCA.⁵

This is strongly reminiscent of programming language, where there are also four levels of description: memory snapshots, symbolic traces, programs, and the programming language's EBNF (see also Cook [2000]). In the remainder, we will deal only with levels 2 to 0.

Level		Example	Representation
M_3	meta-metaclass	CDIF, MOF	class diagram and OCL
M_2	metaclass	UML logical metamodel, UML physical metamodel Common Warehouse Model (CWM)	class diagram and OCL
M_1	class		class diagram and OCL
M_0	object	system in main memory	object diagram

Table A.1.: Relationships among the four metamodel levels, and examples for them.

A.2. Level 2: metamodel

A.2.1. Overview

The UML logical metamodel is a rather complex artifact. It contains 128 metaclasses and a large number of stereotypes (see below) organized into 11 packages (see Figure A.1). The inheritance hierarchy of the UML has been compiled in Figure A.2. It presents an overview of the most important parts of the UML metamodel.

⁵This is not stated directly in the UML standard documents, but can be inferred from the latest MOF specification: “the MOF meta-metamodel is the meta-metamodel for the UML metamodel [...] the UML metamodel may be considered an instance-of the MOF meta-metamodel” (cf. [OMG MOF 1.3, p. xii]).

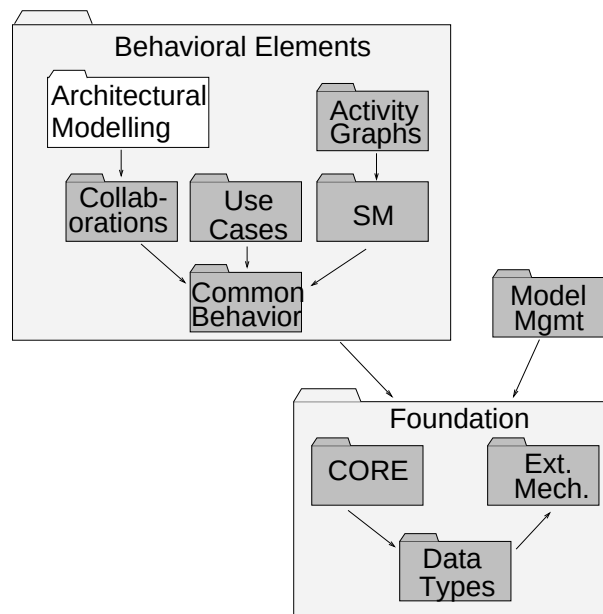


Figure A.1.: The package-structure of the UML metamodel. The shaded Packages are plain UML. The extensions we provide are summarized in the Architectural Modeling-package.

Note, that in Figure A.2, there are two metaclasses, *ElementOwnership* and *ElementImport*, which play a prominent role in the metamodel, but are not part of the inheritance hierarchy.

The physical metamodel is used to accommodate the structure of the logical metamodel to technical constraints. It comes in two representations, as a static structure diagram [OMG UML 1.3, p. 6-1ff], and as an XML-text [OMG UML 1.3, p. 6-23ff]. The latter is claimed to have been generated automatically from the former, but there is circumstantial evidence of manual post-processing. Also, it is not clear whether and if so, how the OCL-constraints of the logical metamodel can be applied to the physical metamodel as well. The relationship between the meta-models and their relationships is with necessity informal. See Figure A.3 for an overview.

A.2.2. An example: model management

As an example for how the metamodel works, consider the model-management facilities of the UML see Figure A.4. The UML provides *Namespace* and *Package* (and its subclasses *Model* and *Subsystem*) for model management.

Namespace has a 1-n-association ownedElement to *ModelElement*. It is decorated with the association class *ElementOwnership*, that features the attribute *visibility*.⁶ So, every *Namespace* can access a set of *ModelElements* under ownedElement, each of which has a *visibility* (valid outside the *Namespace*). Note that the association is a composition, that is, the ownedElements belong only to the respective *Namespace*, and that *Namespace* cannot exist without them.

Namespace is an abstract class (indicated by setting its name in italics), that is, it can not be instantiated. But all its subclasses inherit ownedElement, for instance *Package*. *Package* also has a *weak* aggregation association to *ModelElement*, under the name *importedElement*.

⁶We omit the attribute *isSpecification* for the time being.

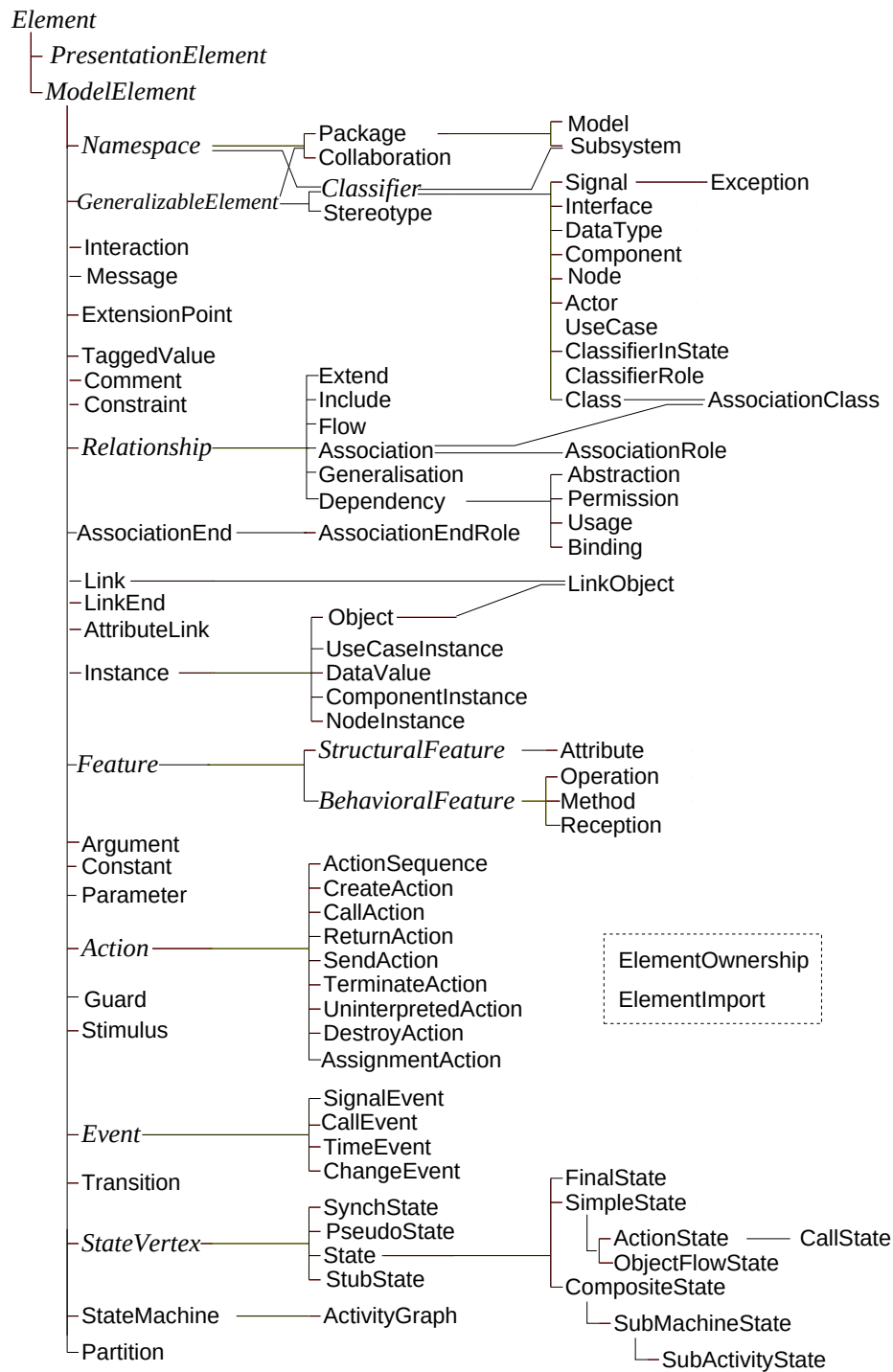


Figure A.2.: Inheritance of metaclasses in the UML. Names set in italic represent abstract classes. The three classes on the right are association classes, and are not placed anywhere in the inheritance hierarchy.

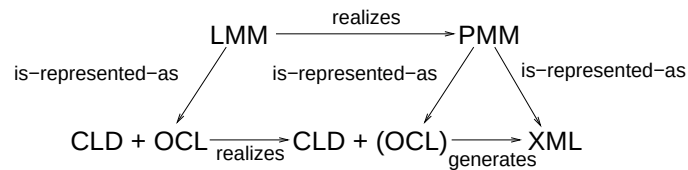


Figure A.3.: Relationship between the logical and physical meta-models of the UML.

It serves to allow shared ownership of ModelElements, i.e. importing. Imported ModelElements can be renamed and endowed with a new visibility by the attributes alias and visibility of the association class ElementImport.

So, all ModelElements that are NameSpaces but not Packages (i.e. Classifier and Collaboration) may consist of other ModelElements, but only Packages can import, and thus share them.

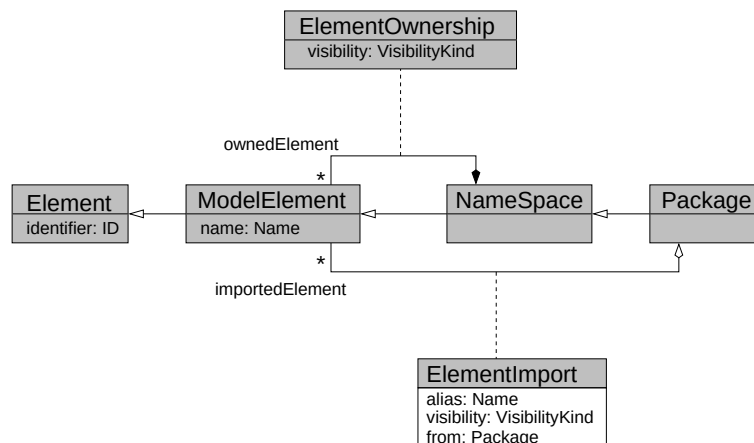


Figure A.4.: In the UML, Namespace and Package can be used for model management.

Package is also a GeneralizableElement, i.e. there can be generalization-relationships between Packages, though their semantics is largely undefined. We shall not use it at all.

A.2.3. OCL

The Object Constraint Language (OCL) (see [OMG UML 1.3, part 7]) is a formal language for annotating UML-models with additional constraints such as pre- and post-conditions, that can not at all or not easily be expressed diagrammatically. It is based on first order predicate calculus.

A.3. Levels 0 and 1: models

Underlying the UML is the 4+1-views model (see e.g. Kruchten [1995] and [Booch et al., 1999, p. 31]). View, model and diagram are used as synonyms. In our terminology, we omit the word model completely, and replace view by viewpoint (aligned with the IEEE AWG/P1471-terminology). A viewpoint may be instantiated to a view. Similarly, we distinguish between diagram types and diagrams. There is a n-m-relationship between diagram types and viewpoints. We use the following abbreviations (see Figure A.5).

dynamics

IAD	interaction diagram
SQD	sequence diagram
CBD	collaboration diagram
STD	state transition diagram
ACD	activity diagram
WFD	workflow diagram
SWD	“small” workflow diagram
WED	window event diagram

functionality

UCD	use case diagram
-----	------------------

structure

SSD	static structure diagram
CLD	class diagram
OBD	object diagram
PKD	package diagram
ASD	architectural structure diagram (aka. capsule diagram)

implementation

CPD	component diagram
DPD	deployment diagram

Figure A.5.: Abbreviations for diagram types.

A.3.1. Interaction diagrams

The UML offers two kinds of interaction diagrams: sequence and collaboration diagrams. They are semantically equivalent: both are Interactions. Interactions are the dynamic aspect of Collaborations, that can be considered to represent a set of interaction diagrams. The former originate with the Booch method, the later come down to us from OMT.

However, they focus on different aspects: interaction diagrams explicitly represent the time axis, and are thus suitable to display long and concurrent interactions of a small number of participants. Collaboration diagrams on the other hand, can be used to describe short interactions of a larger number of participants. It is possible to express concurrency in collaboration diagrams, though not visually.

Collaboration diagrams can be used to describe interactions of aggregated Classifiers more easily (see [OMG UML 1.3, fig. 3-58, p. 3-123]). Similarly, the distribution aspect (i.e. the physical deployment) may be included by allowing the elements of deployment diagrams in collaboration diagrams. Both is also possible in sequence diagrams, but in a less appealing style.

A.3.2. Activity and state transition diagrams

The UML includes a variant of state charts and object charts (see Coleman et al. [1992], Harel [1987], respectively). Confusingly, the UML standard uses the term state chart diagram. In order to avoid confusion, we use the traditional (and more generic) term “state transition diagram” instead.

The UML also offers activity diagrams, that can be seen as a descendant of the classic data- and control-flow diagrams known from structured methods. They also bear considerable similarity with Petri-nets and extended Event-Process-chains (eEPCs) (see e.g. Allweyer and Loos [1998] for the latter). Activity diagrams serve both to model business workflows and data- and control flows. They do neither particularly well: on the one hand, they are much too complicated for most of the users involved in business modeling (typically not engineers or computer scientists). Also, they are not precise enough to allow for automatic enactment (i.e. execution) of workflows, omitting important aspects like role, responsibility, resources etc. On the other hand, expressiveness of activity diagrams is too small to satisfy technical users.

Semantically, a state transition diagram is a `StateMachine`, and an activity diagram is an `ActivityGraph`, which is a subclass of `StateMachine`. Intuitively, an activity diagram is a state transition

diagram, where swimlanes correspond to concurrent regions. Synchronization between the different swimlanes is done explicitly via data-flow ("ObjectFlowState"), and not implicitly by the inscriptions, as is usual in state transition diagrams.

Similar to the two kinds of interaction diagram, state transition and activity diagrams have a different focus: state transition diagrams are mainly used to describe the behavior of an individual Classifier, while activity diagrams are better suited to present the flow of control and data among different Classifiers. The respective other aspect is neglected. This results in a couple of pragmatic differences. For instance, while transitions from one concurrent region to another are legal, if silly in state transition diagrams, they are essential in activity diagrams (e.g for synchronization in a workflow).

We also use a slight extension of activity diagrams (so called workflow diagrams), and an extension of state transition diagrams (window event diagrams), see Chapters 11 and 12, respectively.

A.3.3. Use case diagrams

Use case diagrams in the UML are a stripped version of the use case notation and method invented by Jacobson et al (see Jacobson et al. [1992, 1995]). Note that in use case diagrams, people frequently use a box to represent the system under development. This syntax is included in the UML, but not provided with a semantics (or a even a metaclass). Also, note that it seems to be the case, that there be only one such box in any use case diagram. Compare this to the context diagrams we provide in Section 5.2.3.

A.3.4. Static structure diagrams

The UML features a successor to OMT's class diagrams, called "static structure diagram". We stick to the traditional name. The UML distinguishes between two special forms of static structure diagrams, object and package diagrams. An object diagram is a class diagram that contains only instances (i.e. Links⁷, LinkEnds and Objects). Similarly, a package diagram is a static structure diagram in which there are only Packages, Interfaces and their Relationships.⁸

In practice, class and object diagrams are frequently mixed together, so that both Classes and Objects appear in the same diagram. Note that the items of static structure diagrams are not roles, so that having both a class and an object in the same static structure diagram makes no sense at all. Syntactically, class and object diagrams only differ in the inscriptions.

A.3.5. Architectural structure diagram

We add another kind of structural diagram which we dub architectural structure diagram for want of a better name. It also goes under the title capsule diagram. It is more like the static part of a collaboration diagram than like a class diagram, but still, it presents a genuinely structural aspect. It contains only Capsules and Connectors. However, within these Capsules, any of its views may be presented ("deep views"), see Section 5.2.1.

⁷The UML currently has no concepts for the instances of other Relationships, in particular not for Dependencies.

⁸To be precise, according to the standard, there is no separate diagram type for package diagrams. Rather, its concepts are used freely in class and implementation diagrams, without being explicitly mentioned in the notation guide, see [OMG UML 1.3, p. 2-171, p. 3-17-20].

A.3.6. Implementation diagrams

The UML specifies only two kinds of implementation diagrams, namely, component and deployment diagrams. Component diagrams may contain Components and Relationships only. Deployment diagrams may only contain instances thereof (i.e. ComponentInstance and Link), and NodeInstance. It is unclear, where Nodes are to be used.⁹

The third kind, package diagrams and their derivatives, are not mentioned as a diagram type of their own, but they are used extensively throughout the standard.

A.3.7. Inscriptions and OCL

The UML does not fix an inscription language. However, it defines the OCL. We consider the OCL as the canonical inscription language for UML models, although it is currently unclear, how various quantitative aspects like time, capacity, performance etc. might be expressed.

A.4. UML 1.3 versus UML 1.4 and 2.0

The current standard of UML is still version 1.3. Version 1.4 had been announced for some time earlier this year, but has so far not appeared. A beta version is distributed clandestinely in interested circles, but has not been made accessible to the general public. It contains editorial changes only, it is rumored.

The UML 2.0 is supposed to contain major advances, see UML Revision Task Force [2000]. Taking into account the duration of the individual steps of the OMG adoption process as seen in the past, and given the OMG's schedule, UML 2.0 is not to be expected earlier than late 2002. An interesting suggestion as to the future structure of the UML is found in Clark et al. [2000], but it remains to be seen how much of that will make its way into the standard.

⁹The examples and the textual description in the standard are contradictory, see [OMG UML 1.3, pp. 3-165].

B. Glossary

For an extensive list of object oriented terminology, see Schneider [2000b]. For further abbreviations, cf. Kind and Kind [2000]. If applicable, the items in this glossary also contain pointers to recommended reading.

Abstraction UML metaclass that represents the same entity on different levels of abstraction (see Section 7.3).

Action UML metaclass that represents e.g. effects of transitions (see Section 9.4).

active class UML slang for a class that has its own thread of control, and can send out messages without being triggered by stimuli from other classes.

activity According to the Workflow Management Coalition (WfMC), an activity is “*a logical step of description of a piece of work within a process definition that contributes towards the achievement of the process*” (see WfMC). Activities consist of ↑tasks, they are assigned to some performer, either a person or a machine/application. Activities are the building blocks of ↑processes.

ActivityGraph UML metaclass to represent ↑activity diagrams.

activity diagram Abbreviated to ACD. Notation to represent workflows or scenarios. Semantically, activity diagrams are ActivityGraphs, a subclass of StateMachine, which is the semantics of ↑state transition diagrams. Shares concept (and notational elements) with ↑Petri-nets and ↑eEPCs.

actor (1) In ROOM, an actor is a what is called a ↑Capsule here, i.e., a complete (sub-)system. (2) In UML, an actor is an external system that interacts with the system under scrutiny (see Section 4.2.3). It is represented by an ↑Actor.

Actor UML metaclass representing an ↑actor.

analysis (1) The phase in the process of engineering a software system concerned with understanding the problem domain, specifying it in a precise enough way and drafting an initial architecture and design. During analysis, the what dominates over the how. (2) In the title of this book, analysis is taken to have its general meaning, i.e. the analysis of a model with some (formal) method.

architecture ↑software architecture

architectural unit An architectural unit (AU) is a building block of architectures. It may be represented by a ↑Capsule. Inferior representations include: active classes (as in “UML for real-time”), UML SubSystems, and ROOM actors (see Section 2.2 for a discussion). AUs communicate with their environment only via ↑points of interaction.

architectural style An architectural style is a recurring pattern of architectural elements. It may occur in any viewpoint. It is made up of an architectural vocabulary, that is, (1) classes of $\uparrow$ architectural units, (2) constraints on their relationships; and the underlying motivation, purpose and area of application, that is, (3) context information similar to that found in $\uparrow$ patterns. The classic slogan is “*Style = {Elements, Form, Rationale}*” (cf. [Perry and Wolf, 1992, p. 45]). The seminal paper by Perry and Wolf introducing this definition is still highly recommendable.

AWG (Architecture Working Group) The AWG was set up by the IEEE to explore the meaning of “Software Architecture”. It produced a draft of a *Recommended Practice for Architectural Design*, i.e. one kind of IEEE standard, see IEEE Architecture Working Group [1998, 1999a,b].

Capsule Metaclass in the extension of the UML metamodel representing an $\uparrow$ architectural unit (see Section 4.2.1).

CapsuleInstance metaclass representing an instance of a $\uparrow$ CapsuleInstance (see Section 4.3).

CapsuleRole enumeration type of the kinds of $\uparrow$ roles a Capsule may play, namely $\uparrow$ component, $\uparrow$ style, $\uparrow$ project, and $\uparrow$ pla (see Section 4.2.1).

CASE (Computer Aided Software Engineering) Sometimes, upper- and lower-CASE are distinguished, i.e. high-level tools for modeling, simulation and checking of designs on the one hand, and low-level tools for versioning, configuration management, and other programming-related activities. As an introduction, we recommend Fisher [1991], Simon [1993].

Classifier UML metaclass that represents entities like $\uparrow$ Class, $\uparrow$ Component, but not $\uparrow$ Package.

collaboration diagram Abbreviated to CBD. One of two kinds of $\uparrow$ interaction diagram, focusing on the participating roles rather than on the sequence of exchanged signals.

Collaboration UML metaclass representing a set of interaction diagrams (logical: see Section 4.2.6, physical: see Section 7.4.1), see also $\uparrow$ role modeling.

component (1) The notion of components has been first used wrt. software in conjunction by McIlroy at the Garmisch conference, see [McIlroy, 1969, p. 138]. There, components were meant to mean “*routines*”, i.e. very small units of granularity by today’s standards, but quite large entities by the understanding of that time; (2) in UML: “*A component represents a physical piece of implementation of a system [...]*” (cf. [OMG UML 1.3, p. 2-29]). Brown [1996] makes good introductory reading. (3) in this book, the term component is not further overloaded, to avoid confusion. See $\uparrow$ Capsule for our understanding of a software engineering component.

component One possible role that a Capsule can play, see Section 4.2.1.

Component UML metaclass representing “*a physical replaceable part of a system that packages implementation and provides the realization of a set of interfaces. It represents a physical piece of implementation of a system*” (cf. [OMG UML 1.3, p. 2-29]), i.e. in particular executable code (see Section 4.2.1)

compositionality Property of the semantics of a language to describe systems, such that properties of the system as a whole can be easily computed from constituents of the system, i.e. if A consists of A_1 and A_2 ($A = A_1 \circ A_2$), then there should be efficiently computable functions g_1 , g_2 and f such that $\phi(A_1 \circ A_2) \iff f(\phi(g_1(A_1)), \phi(g_2(A_2)))$, where ϕ is a property on systems.

Configuration metaclass (logical: see Section 4.2.4, physical: see Section 7.4.2).

Connector metaclass representing some architectural $\uparrow$ glue (logical: see Section 4.2.5, physical: see Section 7.3).

ConnectorEnd metaclass representing where a $\uparrow$ Connector and a $\uparrow$ Port touch (logical: see Section 4.2.5, physical: see Section 7.3).

ConnectorEndInstance metaclass representing an instance of a $\uparrow$ ConnectorEnd (see Section 4.3).

ConnectorEndRole metaclass representing a $\uparrow$ ConnectorEnd in a $\uparrow$ role model (see Section 4.2.5).

ConnectorInstance metaclass representing an instance of a $\uparrow$ Connector (see Section 4.3).

ConnectorRole metaclass representing a $\uparrow$ Connector in a $\uparrow$ role model (see Section 4.2.5).

ConfigurationSpace metaclass representing the dynamic structure of a $\uparrow$ Configuration (see Section 4.2.6)

construction We prefer the term software construction over the more common term software development, as the latter has a connotation of quasi-natural, organic growth which we find misleading for software production.

context The outside of a system. See Hammer [1997] for a discussion and enumeration of the relevant factors.

deployment (1) The phase during which a (software) system is put into service, i.e. distributed and installed over a net of computers. (2) The activity of assigning functional and/or physical units to physical nodes (i.e. machines). (3) The result of either (1) or (2).

design (1) The phase in the process of engineering a software system concerned with transforming the insights (and artifacts) resulting from the analysis into a detailed design, taking into account technical side-conditions as well. During design, the how dominates over the what. (2) The result of the design phase, i.e. a design-level $\uparrow$ model.

design debugging Finding and correcting errors in a $\uparrow$ design, in particular inconsistencies within or among $\uparrow$ views, thus, design debugging is also referred to as $\uparrow$ view integration. We distinguish between syntactic and semantic design errors, i.e. such that require formal semantics to be defined, and such that may be expressed using OCL.

Diagram metaclass representing a diagram (logical: see Section 4.2.2, physical: see Section 7.2)

DTD (Document Type Definition) an $\uparrow$ XML document that declares a grammar for a document type. A DTD is processed by an XML-interpreter to generate an interpreter for a customized document type.

Edge New metaclass representing an edge in a $\uparrow$ Diagram (see Section 7.2)

eEPC extended Event Process Chain, a semiformal notation to express workflows. It is widely used in implementations of standard software such as SAP R/3. It is based on Petri-nets (see Chen and Scheer [1994]), but its precise formal semantics is as yet unclear. It has been mentioned together with the UML (see e.g. Allweyer and Loos [1998]), and seems to play a role in the definition of a business modeling profile for the UML.

Element UML metaclass, root of the inheritance hierarchy if the UML metamodel. Its immediate subclasses are $\uparrow$ ModelElement, and $\uparrow$ PresentationElement.

Event UML metaclass that represents e.g. triggers of transitionss.

facette A facette is an aspect of an entity that might have as its value one or several $\uparrow$ simple classes (see [Prieto-Diaz, 1989, p. 102f]). Facette (classification) has originally been invented to classify books in libraries and specimen in biological taxonomies. It has later been adapted to represent code items, too (see Prieto-Diaz [1991]). Today it is frequently used to classify $\uparrow$ patterns. The classification schemas for patterns (e.g. [Gamma et al., 1995, p. 6ff], [Buschmann et al., 1998, p. 20ff]) present examples of facettes.

framework Frameworks have been introduced by Johnson and Foote [1988]. A framework can be seen as a class library, structured by $\uparrow$ patterns (1) “A *framework is an architectural $\uparrow$ pattern that provides an extensible template for applications within a domain.*” (cf. [Booch et al., 1999, pp. 385]) (2) “*Libraries which define interactions for objects are called frameworks.*” (cf. [Pflister and Szyperski, 1998, p. 2]).

glue The substance connecting $\uparrow$ architectural units via their $\uparrow$ points of interaction. In this approach, we use $\uparrow$ Connectors to realize glue.

implementation (1) As a formal relationship, see $\uparrow$ specification; as a kind of (2) *view*, see $\uparrow$ Views. (3) The phase after the design phase has been completed. (4) The result of the implementation phase, i.e. code and test data.

Interaction UML metaclass which represents the dynamic part of a $\uparrow$ Collaboration.

interaction diagram Abbreviated to IAD. Either of $\uparrow$ sequence diagram or $\uparrow$ collaboration diagram. Both types of diagrams are semantically represented in the same way, by the concept Interaction.

life cycle The sequence of $\uparrow$ specification, $\uparrow$ design, $\uparrow$ implementation, and $\uparrow$ maintenance as periods in the life of a system.

low-level Petri-net see $\uparrow$ Petri-net

maintenance The last, and usually longest phase in the system life cycle. During maintenance, a system may undergo $\uparrow$ changes of various kinds.

Message UML metaclass representing an instance of the exchange of information between two $\uparrow$ ClassifierRoles in $\uparrow$ role modeling.

model Derives from Latin *modulus*, scale, measure; “**1. a.** *a representation, usually on a smaller scale, of a device, structure, etc.[...] 9. a.* *a simplified representation or description of a system or complex entity, esp. one designed to facilitate calculations and predictions.*” (cf. [Makins, 1991, p. 967])

ModelElement UML metaclass for all modeling concepts. Its only attribute is name. It is a subclass of $\uparrow$ Element, and peer to $\uparrow$ PresentationElement.

Namespace UML metaclass representing a namespace, that is, a set of names referring to values (i.e. arbitrary ModelElements). NameSpaces have nothing but ownedElements: only Packages, a subclass of Namespace, may have importedElements.

Node metaclass representing a node in a Diagram (see Section 7.2)

Package UML metaclass representing a namespace with imports, see also `Namespace`.

pattern (1) The notion of patterns originates in architecture (see e.g. Alexander [1979], Alexander et al. [1987]), where it is used for “*a practically proven solution to a recurring problem*”. It has subsequently been adopted by computer science (see e.g. Buschmann et al. [1998], Fowler [1997], Gamma et al. [1995], Pree [1995]). (2) In the UML, the concept of “pattern” does not occur as a first class citizen, that is, as a metaclass. It is declared to be a “*template collaboration*” (cf. [OMG UML 1.3, p. 115])—without much further explanation.

Petri-net (also **PN**, low-level net, net) Petri-nets are a formalism first described by Carl Adam Petri in Petri [1962]. A widely available though rather demanding introduction is Reisig [1985], popular alternatives are Peterson [1981] and Baumgarten [1996]. While the former is somewhat restricted in scope and seriously outdated, the latter is currently available in German only. The most up to date source of information on Petri-nets is Christensen and Mortensen [1997].

point of interaction structural feature of an $\uparrow$ architectural unit that bundles incoming and outgoing stimuli. Architectural units and their environments communicate only via points of interaction. In this book, they are realized by $\uparrow$ Ports.

Port metaclass representing a $\uparrow$ point of interaction of a $\uparrow$ Capsule (logical: see Section 4.2.1, physical: see Section 7.4.2)

PortInstance metaclass representing an instance of a $\uparrow$ PortInstance (see Section 4.3)

PresentationElement UML metaclass (see Section 7.2)

process (1) software (development) process: a set of $\uparrow$ activities and associated $\uparrow$ results which produce a software product. (2) Petri-net process, i.e. a concurrent run of a Petri-net given an initial marking.

product family A product family is a group of closely related products, that share large parts of their specification and realization, but differ also in some relevant aspects. Typical examples are mobile phones or the platform approach in the car industry.

product line $\uparrow$ product family.

product line architecture A description of the common ground of a $\uparrow$ product line, similar, but more concrete than a $\uparrow$ style.

pla role of Capsule to represent a $\uparrow$ product line architecture, see Section 4.2.1.

product The result of an $\uparrow$ activity in some $\uparrow$ workflow (such as a software $\uparrow$ process). Products may be expressed e.g. by UML diagrams, but also by any other adequate notation.

project role of Capsule, see Section 4.2.1.

Protocol metaclass representing a set of collaborating $\uparrow$ ProtocolRoles (see Section 4.4.2)

ProtocolRole metaclass representing an individual role in a $\uparrow$ Protocol (logical: see Section 4.4.1, physical: see Section 7.4.1). A ProtocolRole has incoming and outgoing Signals, a StateMachine to describe its behavior, and possibly some OCL-constraints.

resource A machine, program or human capable of performing a characteristic set of ↑tasks in a ↑software process.

responsibility a set of ↑tasks or ↑activities assigned to some ↑resource.

result ↑product

role (1) design: an entity of ↑role modeling; (2) workflow: a specification of the capabilities, rights and skills required of a ↑resource to take over a certain ↑responsibility, and finally to perform a certain ↑task.

role modeling A role is neither a class nor an object, but something in between, a kind of paradigmatic object. Roles can be used to describe collaborations of an ensemble of objects, each of which may play any number of roles. An object can play several roles consecutively. Note that this is not (easily) possible with classes.

In the UML, there is the notion of Collaboration, which is a role model. Roles are realized by ClassifierRoles, AssociationRoles and AssociationEndRoles. As the name suggests, not only Classes can be represented by ClassifierRoles, but all kinds of Classifiers: “[it] specifies a restricted view of a classifier” (cf. [OMG UML 1.3, p. 2-105]). Although it is not said in the standard, it must be the case that ClassifierRoles can also specify views of Instances.

Role modeling is an approach first described by Trygve Reenskaug in Reenskaug et al. [1995]. The key article is Kristensen and Osterbye [1996]. A more formal account of role modeling is found in [Andersen, 1997, pp. 30-42], containing also a large number of pointers and further references. Also, Riehle [2000] and [Marshall, 1999, p. 92ff] are interesting sources.

run to completion step A run to completion step is an uninterruptable cycle of activities to process a set of ↑Events in a ↑StateMachine, see [OMG UML 1.3, p. 2-150ff]. The notion of run to completion step is also found in ROOM, see [Selic et al., 1994, p. 218ff].

sequence diagram Abbreviated to SQD. One of the two ↑interaction diagrams, focusing on the exchange of ↑Messages among ↑Classifiers. Syntactically, sequence diagrams are direct descendants of message sequence charts.

SEC ↑software engineering component

simple class An elementary value of a ↑facette, e.g. for the facette “granularity”, the simple classes “architectural”, “design”, or “implementation” might be defined, or for the facette “problem category” the simple classes “adaptable systems”, “interactive systems”, or “distributed systems” might be defined (both examples taken from [Buschmann et al., 1998, p. 363f])

software architecture see also ↑style and ↑Capsule. (1) (loosely) a high-level description of a system’s static or dynamic structure, or general ubiquitous properties or policies of that system, cf. Clements and Northrop [1996], CMU SEI [2000], Donohoe [1999], Perry and Wolf [1992]. The reader is strongly discouraged from Shaw and Garlan [1996], however.

software process see ↑process.

software engineering component An ↑architectural unit that exists during all phases of a system’s life-cycle, i.e. that is a self-contained unit during analysis as well as during implementation, and in between. Compare this to ↑Component in the UML sense.

specification (1) The first phase in the life of a system: here, the system's characteristics are outlined, the what dominated over the how. (2) The result of the specification phase, i.e. an analysis-level model. Such a description of a system might be $\uparrow$ implemented (subsequently). The specification is less specific than all its implementations, that is, it has more degrees of freedom. On the other hand, an implementation may not change anything that is determined by the specification. (3) Any kind of template that may be filled out in a multitude of different ways (e.g. a $\uparrow$ style).

stakeholder A person or party affected by a system, that is, anybody involved in the system's operation or non-operation. See [D'Souza and Wills, 1999, p. 486] for a crisp definition.

state space explosion problem Composing systems from subsystems can lead to an exponential growth of the number of states of the system (i.e. the size of the state spaces) in the number and complexity of the subsystems, thus disabling all algorithms. This problem can be avoided by $\uparrow$ compositionality of the subsystems.

state transition diagram Abbreviated to STD. A notation to describe the behavior of Classifiers, in particular ($\ll$ active $\gg$) Classes. UML's state transition diagrams are direct descendants of ROOM's ObjectCharts, and share concepts with Harel's state charts Harel [1987]. In the UML standard, STDs are called state charts.

style One of the roles a $\uparrow$ Capsule may play, see Section 4.2.1.

style $\uparrow$ architectural style

SubSystem The UML metaclass representing a subsystem. It is a subclass of Package and Classifier. It is structured into three "compartments" without semantics, but lacks e.g. $\uparrow$ views, $\uparrow$ Ports and several other features of $\uparrow$ Capsules. It seems to be rather close to $\uparrow$ software engineering components, however.

task A task is an element of an $\uparrow$ activity, either an atomic entity, or an activity again. For instance, editing, linking, and compiling are tasks of the activity "programming".

UML (Unified Modeling Language) The UML is the de-facto standard notation for expressing object-oriented models (cf. OMG UML 1.3). We would rather discourage the reader from the "Amigo"-books (i.e. Booch et al. [1999], Rumbaugh et al. [1999]). For the beginner, there is a plethora of superficial treatments of the syntactic aspects of the UML. Two of better ones are Kappel and Hitz [1999], Oesterreich [1998]. For the practitioner, Larman [1998], Stevens and Pooley [2000] are accessible introductions, very practically minded, and with a healthy common sense approach towards the UML. For more information, the CETUS archive is a good starting point (look at Schneider [2000b], in particular Schneider [2000a]).

use case (1) Originally, a use case denoted a prototypical interaction between a system and its environment, exhibiting a self-contained piece of functionality of the system. In this sense, the term scenario is synonymous. UML use cases are direct descendants of Jacobson's use cases. The best introductions around are still the original monographs on use cases, see Jacobson et al. [1992, 1995]. (2) More recently, the UML was equipped with the Metaclass UseCase, denoting a piece of functionality quite similar to sense (1), though discontinuing the synonym $\uparrow$ scenario. The prototypical interaction may be omitted, or specified either by plain text or as an $\uparrow$ interaction diagram.

use case diagram Abbreviated to UCD. A notation to represent the static aspect of ↑use cases. The dynamic aspect is captured using (structured) natural language or ↑sequence diagrams.

view Instance of a ↑viewpoint.

view integration see ↑design debugging.

View metaclass to represent a ↑view of a concrete system, i.e., a ↑Capsule (logical: see Section 4.2.2).

viewpoint A viewpoint is a way of looking at a system, a “*a specification of the conventions for constructing and using a view.*” (cf. [IEEE Architecture Working Group, 1999a, Def. 3.1.10]). It is specified by the notations that may be used. It may be instantiated to a ↑view. Hammer [1997] discusses the purpose and possibilities extensively. Other accounts of viewpoints of system design are found in Davis and Williams [1997], ISO RM-ODP [a], Jacobson et al. [1999], Kruchten [1995] or [Booch et al., 1999, p. 31]. All of these approaches fix a particular set of viewpoints, but none give sufficient justification of their choice. See Appendix D for a complete list of all viewpoints, their elements, and syntax used in this book.

virtual unit role of Capsule to represent a logical entity, that does not exist as a physical entity (but rather as parts of other entities, say), see Section 4.2.1.

worker A kind of ↑resource, namely a person.

XML The Extensible Markup Language, a subset of SGML. It is a language for defining (context-free) grammars (so called ↑DTDs) that describe document types.

C. Rules

In this appendix, we present a synopsis of all semantical rules. In the following table, the first column shows the name of the semantical rule, and, if applicable, the index. In the second column, there is the metaclass on which the rule is defined, i.e., the syntactical category. The third column shows an additional argument used as a modifier, the fourth column shows the semantical object resulting from an application of that rule. The last column shows the page on which the rule is defined.

Rule	Metaclass	argument	result	Definition
asc		<i>id</i>	id set	149
BC ₁	StateMachine		int	173
BC ₂	Collaboration		int	173
BC ₂	Interaction		int	173
Bdg ₃	Capsule		(id set) sequence	167
Con ₁	Association		id set	166
Con ₂	AssociationEnd		id set	166
Con ₃	Connector		id set	166
Con ₄	ConnectorEnd		id set	166
Con ₇	Configuration	<i>binding</i>	(id set) sequence	166
CP ₁	Capsule		int	172
CP ₂	Capsule	<i>id set</i>	int	172
CP ₃	Package	<i>id set</i>	int	172
CP ₄	Namespace		int	172
CP ₇	ProtocolRole		int	172
D ₁	Capsule		id graph	169
D ₂	Capsule	<i>“im”</i>	id→filename	169
D ₃	Capsule	<i>“s”</i>	id→filename	169
D ₄	Package		id→filename	169
D ₅	Namespace		id→filename	169
D ₇	Component		id→filename	169
esc		<i>id</i>	id set	150
EQ	StateMachine	<i>n</i>	net	156
H	State		id→history-kind	148
I	State		id set	149
ID	Element		id	138
Ids ₁	Capsule	<i>view</i>	id set	163
Ids ₂	View	<i>view</i>	id set	163
Ids ₃	Capsule	<i>type</i>	id set	163
Ids ₄	Package	<i>type</i>	id set	163
Ids ₇	Capsule	<i>type,view</i>	id set	163

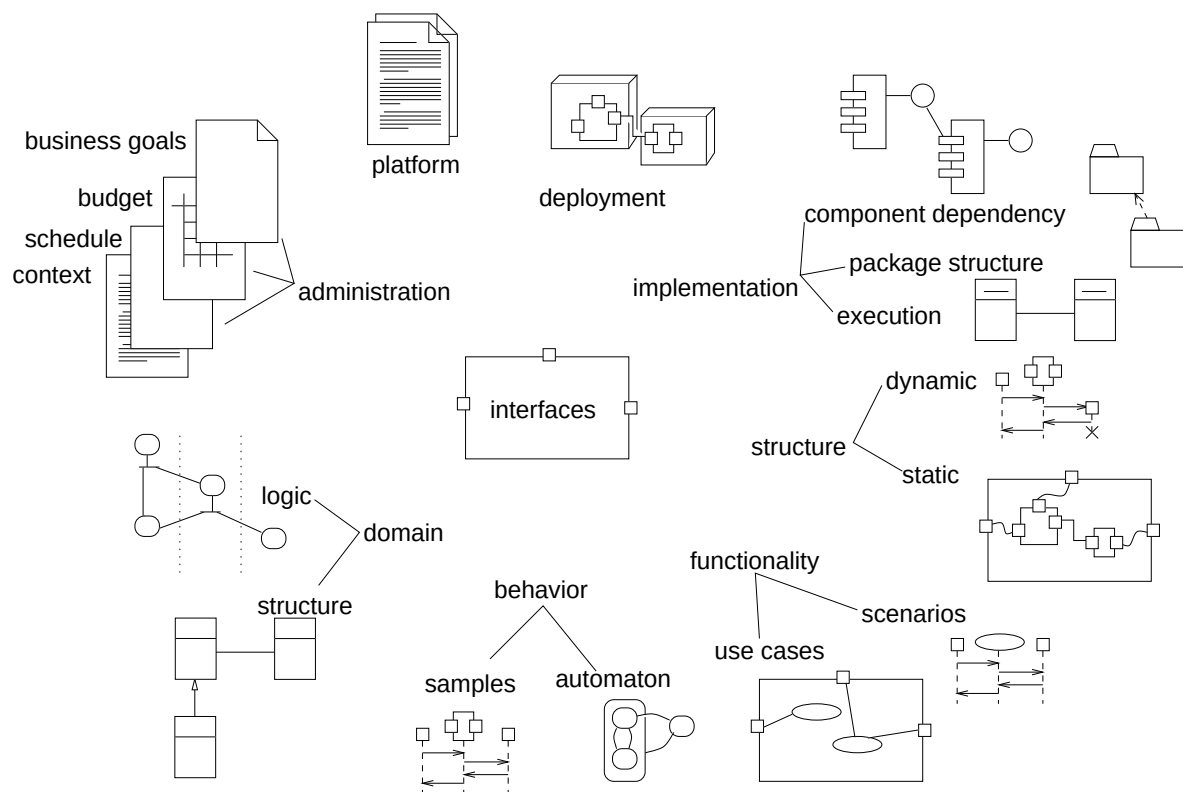
Ids ₈	Protocol	<i>type</i>	id set	163
Ids ₉	Collaboration		id set	163
Ids ₁₀	Connector		id set	163
Ids ₁₁	Association		id set	163
In ₁	Transition		string	152
In ₂	StateMachine		string set	152
In ₃	ProtocolRole		string set	164
In ₄	ConnectorEnd		string set	164
In ₅	Port		string set	164
In ₆	Capsule	<i>view,type</i>	string set	165
In ₇	Capsule	<i>id set</i>	string set	165
In ₈	Package	<i>id set</i>	string set	165
In ₉	Namespace	<i>id set</i>	string set	165
Initial ₁	Interaction		marking	142
Initial ₂	StateMachine		marking	162
Initial ₃	ProtocolRole		marking	164
Initial ₄	ConnectorEnd		marking	164
Initial ₅	Port		marking	164
Initial ₆	Capsule	<i>view, type</i>	marking	165
Initial ₇	Capsule	<i>id set</i>	marking	165
Initial ₈	Package	<i>id set</i>	marking	165
Initial ₉	Namespace	<i>id set</i>	marking	165
Initial ₁₂	Configuration		marking	167
Initial ₁₃	Capsule		marking	167
Initial ₁₄	Capsule	<i>“sub”</i>	marking	167
Initial ₁₅	ComplexConnector		marking	167
Initial ₁₆	SimpleConnector		marking	167
isc		<i>id</i>	id set	150
K	State		id→state-kind	149
KIND ₁	Element	<i>id</i>	type set	138
KIND ₂	Element		type set	138
Msg	Message	<i>role</i>	id×id set×string seq	142
NAME	ModelElement		string	138
Names ₁	Capsule		string set	163
Names ₂	Capsule	<i>type</i>	string set	163
Names ₃	Capsule	<i>type,view</i>	string set	163
Names ₄	Capsule	<i>id set</i>	string set	163
Names ₅	Package	<i>id set</i>	string set	163
Names ₇	Namespace	<i>id set</i>	string set	163
Names ₉	Protocol	<i>type</i>	id set	163
NOM ₁	StateMachine		int	170
NOM ₇	Capsule		int	170
NOM ₉	Configuration		int	170
NOM ₁₀	Namespace		int	170
NOM ₁₃	Package		int	170
NOM ₁₄	Port		int	170

NOM ₁₆	Collaboration		int	170
NOM ₁₇	Interaction		int	170
Out ₁	Transition		string set	152
Out ₂	StateMachine		string set	152
Out ₃	ProtocolRole		string set	164
Out ₄	ConnectorEnd		string set	164
Out ₄	Port	<i>id</i>	string set	164
Out ₅	Capsule	<i>view,type</i>	string set	165
Out ₆	Capsule	<i>id set</i>	string set	165
Out ₇	Package	<i>id set</i>	string set	165
Out ₈	NameSpace	<i>id set</i>	string set	165
Q	Capsule		net	174
R	Dependency	<i>id→id</i>	<i>id→id×id→id</i>)	168
R	Abstraction		<i>id→id</i>	168
R	Usage		<i>id→id</i>	168
Roles ₁	Interaction		id set	141
Roles ₂	Message		id set	141
Run ₁	Collaboration	<i>id</i>	net	141
Run ₄	Interaction	<i>id</i>	net	141
Run ₅	Interaction		net	141
Run ₆	Interaction	<i>role</i>	net	142
SigIds ₁	Capsule	<i>view,type</i>	string set	139
SigIds ₂	Capsule	<i>id</i>	string set	139
SigIds ₃	Package	<i>id</i>	string set	140
SigIds ₄	NameSpace	<i>id</i>	string set	140
SigIds ₇	Interaction	<i>role</i>	string set	140
SigIds ₈	Message	<i>role</i>	string set	140
Signal ₁	Message		string seq	140
Signal ₂	SendAction		string seq	140
Signal ₃	ActionSequence		string seq	140
Signal ₆	SignalEvent		string seq	140
Signal ₇	Interaction		string set	140
Signal ₈	Capsule		string set	140
Signal ₉	Connector		string set	140
Sim ₁	Capsule		net	166
Sim ₂	Configuration		net	166
Sim ₃	ComplexConnector		net	166
Sim ₄	SimpleConnector		net	166
SM ₁	StateMachine		net	151
SM ₃	ProtocolRole		net	164
SM ₄	ConnectorEnd	<i>id</i>	net	164
SM ₅	Port	<i>id</i>	net	164
SM ₆	Capsule	<i>view,type</i>	net	165
SM ₇	Package	<i>id set</i>	net	165
SM ₈	NameSpace	<i>id set</i>	net	165
SMs	Element	<i>arg</i>	result	157

SMstrict	Element	<i>int</i>	net	155
Sub	State		id set $\times$ id set	149
K	State		id $\rightarrow$ type	149
T	Transition		transition set	152
TE	Transition		transition $\rightarrow$ string	152
Terminal ₂	StateMachine		marking set	162
Terminal ₃	ProtocolRole		marking set	164
Terminal ₄	ConnectorEnd		marking set	164
Terminal ₅	Port		marking set	164
Terminal ₆	Capsule	<i>view, type</i>	marking set	165
Terminal ₇	Capsule	<i>id set</i>	marking set	165
Terminal ₈	Package	<i>id set</i>	marking	165
Terminal ₉	NameSpace	<i>id set</i>	marking	165
Terminal ₁₂	Configuration		marking	167
Terminal ₁₃	Capsule		marking	167
Terminal ₁₄	Capsule	<i>“sub”</i>	marking	167
Terminal ₁₅	ComplexConnector		marking	167
Terminal ₁₆	SimpleConnector		marking	167
TEs	Element	<i>arg</i>	result	158
Thread	Interaction	<i>role</i>	net	141
Ts	Element	<i>arg</i>	result	157
Ts'	Element	<i>arg</i>	result	157
TYPE	Element		type	138

D. Viewpoints

In this Appendix, we provide a complete list of all the viewpoints that are used throughout this book, complete with their elements, and customary realizations. Note again, that this is to be seen as an example only (albeit a sensible one). Any project or style may contain arbitrary viewpoints, including redefinitions of those given here. A synopsis of this table is given in the following sketch.



Abbrev.	Viewpoint	Elements	Realization	
			Document types	Multiplicity
a b s r	Administration	Budget	Spreadsheet	+
		Schedule	Text, Pert & Gantt-charts	+
		Resources	Text	1
b a s	Behavior	Automaton	STD	1
		Samples	IAD	*
c g v l	Context	Goals	Text	1
		Viewpoints	Text	1
		Legacies	any	*
d s l g	Domain	Structure	CLD	+
		Logics	STD, IAD, AD, WFD	*
		Glossary	Text	0..1
de s d t l	Deployment	Static	DPD	0..1
		Dynamic	IAD, DPD	*
		Topology	DPD	1
		Load characteristics	Text	*
f u s i	Functionality	Use cases	UCD	+
		Scenarios	IAD, AD	+
		Interaction Storybooks	WED	*
i p r	Interfaces	Ports	STD, Text	+
		Protocol roles	IAD, STD, Text	*
im x p d c t	Implementation	Execution	OBD	*
		Package structure	PKD	*
		Component dependency	CPD	*
		Code	Text	*
		Tests	Text	*
p	Platform	—	Text	1
s s d	Structure	Static	ASD	1
		Dynamic	IAD	*

Bibliography

- Will van der Aalst, Bartek Kiepuszewski, Arthur ter Hofstede, and Alistair Barros. Workflow Patterns WWW Resource, 2000. Available at <http://tmitwww.tm.tue.nl/research/patterns/>.
- Gregory D. Abowd, Robert J. Allen, and David Garlan. Formalizing Style to Understand Descriptions of Software Architecture. *ACM TOSEM*, 4(4):319–364, October 1995.
- Christopher Alexander. *The Timeless way of Building*. Oxford University Press, 1979. cited after Buschmann et al. [1998].
- Christopher Alexander, Hajo Neis, Artemis Anninou, and Ingrid King. *A new Theory of Urban Design*. Oxford University Press, 1987.
- Robert J. Allen. *A Formal Approach to Software Architecture*. PhD thesis, Carnegie Mellon, School of Computer Science, May 1997. Issued as CMU Technical Report CMU-CS-97-144.
- Robert J. Allen, Rémi Douence, and David Garlan. Specifying and Analyzing Dynamic Software Architectures. In Egidio Astesiano, editor, *Fundamental approaches to software engineering (FASE)*, number 1382 in LNCS. Springer Verlag, March 1998.
- Robert J. Allen and David Garlan. A Formal Basis for Architectural Connection. *ACM TOSEM*, 6(3): 213–249, 1997.
- Thomas Allweyer and Peter Loos. Process Orientation in UML through Integration of Event-Driven Process Chains. In Muller and Bézivin [1998], pages 183–193.
- Scott W. Ambler. *More Process Patterns: Delivering Large-Scale Systems Using Object Technology*. Cambridge University Press, 1998a.
- Scott W. Ambler. *Process Patterns: Building Large-Scale Systems Using Object Technology*. Cambridge University Press, 1998b.
- Daniel Amyot and Gunter Mussbacher. On the Extension of UML with Use Case Maps Concepts. In Selic et al. [2000], pages 16–31.
- Egil P. Andersen. *Conceptual Modeling of Objects*. PhD thesis, Department of Informatics, University of Oslo, 1997.
- Jim Arlow, Wolfgang Emmerich, and John Quinn. Literate Modeling—Capturing Business Knowledge with the UML. In Muller and Bézivin [1998], pages 165–172.
- Colin Atkinson, Christian Bunse, Stefan Jungmayr, Erik Kamsties, and Oliver Laitenberger. Systematic Object-Oriented Development. *Rundbrief des GI-Fachausschusses 5.1*, 4(7):4–14, 1998.

- Jos C.M. Baeten and W. Peter Weijland. *Process Algebra*. Number 18 in Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 1990.
- Heide Balzert. *Lehrbuch der Objektmodellierung*. Spektrum Akademischer Verlag, 1999.
- Alistair Barros, Keith Duddy, Michael Lawley, Zoran Milosevic, Kerry Raymond, and Andrew Wood. Processes, Roles and Events: UML Concepts for Enterprise Architecture. In Selic et al. [2000], pages 62–77.
- Bernd Baumgarten. *Petri-Netze. Grundlagen und Anwendungen*. Spektrum Akademischer Verlag, Heidelberg, 1996. 2. Aufl.
- Falko Bause and Peter Kritzinger. *Stochastic Petri Nets*. Vieweg, 1996.
- Joachim Bayer, Oliver Flege, Peter Knauber, Roland Laqua, Dirk Muthig, Klaus Schmid, Tanya Widen, and Jean-Marc DeBaud. PuLSE: A Methodology to Develop Software Product Lines. In Jazayeri [1999], pages 122–131. Available via <http://csalpha.ist.unomaha.edu/~ssr99/>.
- Klaus Bergner, Andreas Rausch, Marc Sihling, and Alexander Vilbig. A Componentware Development Methodology based on Process Patterns. In Joseph Yoder, editor, *Proc. 5th Annual Conf. on the Pattern Languages of Programs (PLOP)*, 1998. Available at www4.informatik.tu-muenchen.de/~rausch/publications/.
- Ted Biggerstaff and Charles Richter. Reusability Framework, Assessment, and Directions. *IEEE Software*, pages 41–49, March 1987.
- Barry W. Boehm. *Software Engineering Economics*. Prentice Hall, 1981.
- Barry W. Boehm and Frank Belz. Applying Process Programming to the Spiral Model. In Tully [1988], pages 46–56.
- Shawn A. Bohnert and Robert S. Arnold, editors. *Software Change Impact Analysis*. IEEE Computer Society Press, 1996.
- Grady Booch, James Rumbaugh, and Ivar Jacobson. *The Unified Modelling Language User Guide*. Addison-Wesley, 1999.
- Jan Bosch, Görel Hedin, Kai Koskimies, and Bent Bruun Kristensen, editors. *Proc. 1st Nordic Workshop on Software Architecture (NOSA98)*, 1998. Available at www.ide.hk-r.se.
- Christine L. Braun. A Lifecycle Process for the Effective Reuse of Commercial Off-the-Shelf (COTS) Software. In Jazayeri [1999], pages 29–36. Available via <http://csalpha.ist.unomaha.edu/~ssr99/>.
- Francois Bronsard, Douglas Bryan, W. Voytek Kozaczynski, Edy S. Liongosari, Jim Q. Ning, Asgeir Olafsson, and John W. Wetterstrand. Toward software plug-and-play. In Harandi [1997]. Also appeared as no. 3, vol. 22 of ACM SE Notes.
- Frederick P. Brooks. *The Mythical Man-Month*. Addison-Wesley, 1975.
- Alan W. Brown, editor. *Component Based Software Engineering. Selected Papers from the SEI*. IEEE Computer Society Press, 1996.

- Ray J. A. Buhr. Use Case Maps as Architectural Entities for Complex Systems. *IEEE Transactions on Software Engineering*, 24(12):1131–1155, December 1998.
- Ray J. A. Buhr and R.S. Casselman. *Use Case Maps for Object-Oriented Systems*. Prentice Hall, 1996.
- Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. *Pattern-Oriented Software Architecture. A System of Patterns*. John Wiley & Sons Ltd., 1998. Also published in German by Addison-Weseley, 1998, under the title "Pattern-Orientierte Software-Architektur", to which all citations refer.
- Fabiano Cattaneo. An evaluation of architecture description languages. Technical Report 96.152, Dipartimento di Elettronica e Informazione, Politecnico di Milano, October 1996. Available at www.cefriel.it/Se/People/Researchers/Cattaneo/Papers.
- R. Chen and August-Wilhelm Scheer. Modellierung von Prozeßketten mittels Petri-Netz-Theorie. Technical Report IWi-Heft 107, Institut für Wirtschaftsinformatik, Universität Saarbrücken, 1994.
- Sören Christensen and Kjeld H. Mortensen. Petri Nets Tools Database, 1997. Available at www.daimi.aau.dk/~petrinet/tools/quick.html.
- Tony Clark, Andy Evans, Stuart Kent, Steve Brodsky, and Steve Cook. A Feasibility Study in Rearchitecting UML as a Family of Languages using a Precise OO Meta-Modeling Approach. Technical report, pUML-group and IBM, September 2000. Version 1.0, available at www.puml.org.
- Paul C. Clements, Len Bass, Rick Kazman, and Gregory Abowd. Predicting software quality by architecture-level evaluation. In Brown [1996], pages 19–25.
- Paul C. Clements and Linda N. Northrop. Software Architecture: An Executive Overview. In Brown [1996], pages 55–68.
- CMU SEI. How Do You Define Software Architecture?, 2000. Web site maintained by Carnegie Mellon University, Software Engineering Institute. Available at www.sei.cmu.edu/architecture/definitions.html.
- CMU SEI CMM. Software, Systems Engineering, and Product Development Capability Maturity Models, August 2000. Available at www.sei.cmu.edu/cmm/cmms/transition.html.
- Derek Coleman, Stephanie Bodoff, and Patrick Arnold, editors. *Object-Oriented Development: The Fusion Method*. Prentice Hall, 1993.
- Derek Coleman, Frank Hayes, and Steven Bear. Introducing Objectcharts or How to Use Statecharts in Object-Oriented Design. *IEEE Transactions on Software Engineering*, 18(1):9–18, 1992.
- David Conelly. Best practices in business software integration and the plug and play software. *Wirtschaftsinformatik*, 41(6):497–505, Dezember 1999. Available as a whitepaper at www.openapplications.org.
- Steve Cook. The UML Family: Profiles, Prefaces, and Packages. In Selic et al. [2000], pages 255–264.
- Steve Cook and David Daniels. *Designing Object Systems: Object-Oriented Modelling with Syntropy*. Prentice Hall, 1994.

- Daniel E. Cooke, editor. *The Impact of CASE Technology on Software Process*. World Scientific, 1994.
- James O. Coplien. A Generative Development-Process Pattern. In Coplien and Schmidt [1995b], pages 183–238. Based on the proceedings of PLoP'94. Available also at www1.bell-labs.com/user/cope/Patterns/Process/index.html.
- James O. Coplien and Douglas C. Schmidt, editors. *Draft Proc. 2nd Conf. Pattern Languages of Programs (PLoP)*, Monticello, Illinois, 1995a. Available via <http://st-www.cs.uiuc.edu/~plop/>.
- James O. Coplien and Douglas C. Schmidt, editors. *Pattern Languages of Program Design*. Addison-Wesley, 1995b. Based on the proceedings of PLoP'94. Available also at www1.bell-labs.com/user/cope/Patterns/Process/index.html.
- Jacques M. Couvreur, Serge Haddad, and Jean F. Peyre. Generative families of positive invariants in coloured nets subclasses. In G. Rozenberg, editor, *Proceedings Annual European Workshop: Applications and Theory of Petri Nets '91*, number 674 in LNCS, pages 51–64. Springer Verlag, 1993.
- Brad Cox. *Object-Oriented Programming: An Evolutionary Approach*. Addison-Wesley, 1986.
- Stephen Crawley, Simon McBride, and Kerry Raymond. Meta-Object Facility Tutorial, July 2000. Available at www.dstc.edu.au/~Research/~Projects/~MOF/~Tutorial.html.
- Margret J. Davis and Roger B. Williams. Software Architecture Characterization. In Harandi [1997]. Also appeared as no. 3, vol. 22 of ACM SE Notes.
- Wolfgang Deiters, Volker Gruhn, and Herbert Weber. Software Process Evolution in MELMAC. In Cooke [1994], pages 301–323.
- Sanjay Dewal and Ludger Schnichels. Bank 2010: Eine fachliche und technische Vision. *Softwaretechnik Trends*, 20(3):9–14, September 2000.
- Patrick Donohoe, editor. Kluwer Academic Publishers, 1999.
- Desmond Francis D'Souza and Alan Cameron Wills. *Objects, Components and Frameworks with UML. The Catalysis Approach*. Addison-Wesley, 1999.
- Alexander Egyed and Nenad Medvidovic. Extending Architectural Representation in UML with View Integration. In France and Rumpe [1999], pages 2–16.
- Robert Englander. *Developing Java Beans*. O'Reilly, 1997.
- Gerald Estrin, Robert S. Fenchel, and Mary K. Razouk, Rami R. Vernon. SARA (System Architect's Apprentice): Modelling, Analysis and Simulation Support for Design of Concurrent Systems. *IEEE Transactions on Software Engineering*, 12(2):293–311, 1986.
- Clive Fencott. *Formal Methods for Concurrency*. International Thomson Computer Press, London, 1996.
- Thomas Firley, Michaela Huhn, Karsten Diethers, Thomas Gehrke, and Ursula Goltz. Timed Sequence Diagrams and Tool-Based Analysis—A Case Study. In France and Rumpe [1999], pages 645–660.

- Alan S. Fisher. *CASE. Using Software Development Tools*. John Wiley & Sons, 2nd edition edition, 1991.
- Ivar Fjelstadt and Knut Hamlen. Application Program Maintenance Study—Report to Our Respondents. Report, IBM Corp. DP Marketing Group, 1979. Cited after [McClure, 1992, p. 22].
- David Flanagan. *Java in a Nutshell*. O'Reilly, second edition, 1997.
- Rony G. Flatscher. BODTF ("Business Object Domain Task Force") und BO-Initiativen. In Klaus Turowski, editor, *Proc 1st Ws. Komponentenorientierte betriebliche Anwendungssysteme (WKBA 1)*, pages 17–22. Otto-von-Guericke-Universität Magdeburg, 1999. In German, available at www-wi.cs.uni-magdeburg.de/workshops/komponenten/tagungsband.html.
- Brian Foote. A Fractal Model of the Lifecycle of Reusable Objects. In James O. Coplien, Russel Winder, and Susan Hutz, editors, *OOPLSA'93 Workshop on Process Standards and Iteration*, 1993. Available at <http://www.laputan.org/frameworks/fractal.html>.
- Brian Foote and William F. Opdyke. Lifecycle and Refactoring Patterns That Support Evolution and Reuse. In Coplien and Schmidt [1995b], pages 239–258. Available at www.laputan.org/lifecycle/Lifecycle.html.
- Brian Foote and Joseph Yoder. Evolution, Architecture, and Metamorphosis. In Coplien and Schmidt [1995a]. Available via <http://st-www.cs.uiuc.edu/~plop/>.
- Martin Fowler. *Analysis Patterns: Reusable Object Models*. Addison-Wesley, 1997.
- Robert France and Bernhard Rumpe, editors. *Proc. 2nd Intl. Conf. on the Unified Modeling Language (UML) 1999. Beyond the Standard*. Number 1723 in LNCS. Springer Verlag, 1999.
- Ulrich Frank and Sören Halter. Delegation: Eine sinnvolle Ergänzung gängiger objektorientierter Modellierungskonzepte. *Rundbrief des GI-FA 5.2*, 3(1):16–19, September 1996.
- Peter Freeman. Reusable Software Engineering: Concepts and Research Directions. In Alan Perlis, editor, *Proc. Ws. Reusability in Programming*, pages 2–16. ITT Programming, September 1983. Cited after [Prieto-Diaz, 1994, p. 1].
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
- David Garlan, Robert J. Allen, and John Ockerbloom. Exploiting Style in Architectural Design. In *Proc. SIGSOFT'94: Foundations of Software Engineering*, pages 175–188. ACM, 1994.
- David Garlan and Andrew J. Kompanek. Reconciling the Needs of Architectural Description with Object-Modeling Notations. In Selic et al. [2000], pages 498–512.
- Thomas Gehrke, Ursula Goltz, and Heike Wehrheim. The Dynamic Models of UML: Towards a Semantics and its Application in the Development Process. Technical Report 11/98, Institut für Informatik, Universität Hildesheim, 1998.
- Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, 1991.

- Rob J. H. van Glabbeek. What is branching time semantics and why to use it? Report STAN-CS-93-1486, Stanford University, 1993.
- Rob J. H. van Glabbeek. *Comparative Concurrency Semantics and Refinement of Actions*. Number 109 in CWI Tracts. CWI, Amsterdam, 1996. 2nd Ed.
- GMD. Der V-Modell Browser, 1997. Available at <http://www.scope.gmd.de/vmodell/>.
- Patrice Godefroid. *Partial-Order Methods for the Verification of Concurrent Systems*. Number 1032 in LNCS. Springer Verlag, 1996.
- Adele Goldberg. A reuse business model. *Software Concepts & Tools*, 19(1):11–13, 1998.
- Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics*. Addison-Wesley, 1989.
- Martin L. Griss, John Favaro, and Paul Walton. *Managerial and organizational issues—starting and running a software reuse program*, chapter 3, pages 51–78. In , Schäfer et al. [1994], 1994. Proc. 1st Intl. Conf. Software Reusability, 1991.
- Florian Hacklinger. Erstellung eines Front-Ends für das PUSH-System. Master's thesis, Institut für Informatik, Universität München, August 2000.
- Serge Haddad. A reduction theory for coloured nets. In G. Rozenberg, editor, *Proceedings Annual European Workshop: Applications and Theory of Petri Nets '88*, number 424 in LNCS. Springer Verlag, 1990.
- Dieter K. Hammer. The many Aspects of an IT-Architecture. In *IEEE Symposium and Workshop on Engineering of Computer-based Systems, ECBS97*, pages 304–311. IEEE Computer Press, 1997.
- Mehdi T. Harandi, editor. *Proc. 1997 Symp. on Software Reusability (SSR'97)*, 1997. ACM SIGSOFT. Also appeared as no. 3, vol. 22 of ACM SE Notes.
- David Harel. State Charts: a Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3):231–274, 1987.
- Neil B. Harrison. Organizational Patterns for Teams. In Coplien and Schmidt [1995a]. Available via <http://st-www.cs.uiuc.edu/~plop/>.
- Wolfgang Hesse. Theory and Practice of the Software Process - a Field Study and its Implications for Project Management. In *European Workshop Software Process Technology*, number 1149 in LNCS, pages 241–256, 1996.
- Wolfgang Hesse. From WOON to EOS: New development methods require a new software process model. In A. Smolyaninov and A. Shestiatynow, editors, *Proc. 1st and 2nd Intl. Ws. on OO Technology (WOON'96/WOON'97)*, pages 88–101, 1997.
- Christine Hofmeister, Robert L. Nord, and Dilip Soni. Describing Software Architecture in UML. In Donohoe [1999], pages 145–159.
- Erika Horn, Mario Kupries, and Thomas Reinke. Architekturtypen für Agentenanwendungssysteme. Technical report, Universität Potsdam, 1999.

- Heinrich Hußmann, Birgit Demuth, and Frank Finger. Modular Architecture for a Toolset Supporting OCL. In Selic et al. [2000], pages 278–293.
- Gábor Huszerl and István Majzik. Quantitative Analysis of Dependability Critical Systems Based on UML Statechart Models. In *Proc. 4th IEEE Intl. Symp. High Assurance Systems Engineering (HASE'99)*, 1999.
- The IEEE Architecture Working Group. *Frequently Asked Questions*, 17. October 1998. Version 1.0, available at www.pitecanthropus.com/~awg.
- The IEEE Architecture Working Group. Draft Recommended Practice for Architectural Description, IEEE P1471/D5.1. Technical report, IEEE, October 1999a. Available at www.pitecanthropus.com/~awg.
- The IEEE Architecture Working Group. Slides for a talk on P1471, 30. March 1999b. Available at www.pitecanthropus.com/~awg.
- ISO RM-ODP. *Reference Model of Open Distributed Processing (ISO/IEC 10746-1): Overview*. International Standardization Organization (ISO), 1995a. Same document as ITU-T Rec. X.901.
- ISO RM-ODP. *Reference Model of Open Distributed Processing (ISO/IEC 10746-2): Foundations*. International Standardization Organization (ISO), 1995b. Same document as ITU-T Rec. X.902.
- ISO RM-ODP. *Reference Model of Open Distributed Processing (ISO/IEC 10746-3): Architecture*. International Standardization Organization (ISO), 1995c. Same document as ITU-T Rec. X.903.
- ISO RM-ODP. *Reference Model of Open Distributed Processing (ISO/IEC 10746-4): Architectural semantics*. International Standardization Organization (ISO), 1995d. Same document as ITU-T Rec. X.904.
- SDL 2000. *Recommenration Z.100. Specification and Description Language, SDL*. International Telecommunication Union, 2000.
- Ivar Jacobson, Grady Booch, and James Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- Ivar Jacobson, Magnus Christerson, Patrik Jonsson, and Gunnar Övergaard. *Object-Oriented Software Engineering—A Use Case Driven Approach*. Addison-Wesley, 1992.
- Ivar Jacobson, Maria Ericsson, and Agneta Jacobson. *The Object Advantage. Business Process Reengineering with Object Technology*. Addison-Wesley, 1995.
- Dirk Jäger, Ansgar Schleicher, and Bernhard Westfechtel. Using UML for Software Process Modeling. In Nierstrasz [1998], pages 91–108.
- Matthias Jantzen. Eigenschaften von Petrinetzsprachen. Technical Report IFI-HH-B-64, FB Informatik, Universität Hamburg, November 1979a.
- Matthias Jantzen. On the hierarchy of petri net languages. *R.A.I.R.O. Theoretical Informatics*, 13(1): 19–30, 1979b.

- Mehdi Jazayeri, editor. *Proc. 5th Symp. on Software Reusability: Bridging the Gap Between Research and Practice (SSR'99)*, May 1999. ACM Press. Available via <http://csalpha.ist.unomaha.edu/~ssr99/>.
- Kurt Jensen. Condensed state spaces for symmetrical coloured petri nets. *Formal Methods in System Design*, 9:7–40, 1996.
- Kurt Jensen and Grzegorz Rozenberg. *High-Level Petri Nets. Theory and Application*. Springer Verlag, 1991.
- Eike Jessen and Rüdiger Valk. *Rechensysteme. Grundlagen der Modellbildung*. Springer Verlag, 1987.
- Ralph Johnson and Brian Foote. Designing Reusable Classes. *J. OO Programming*, 1(2):22–35, June/July 1988.
- Mohamed Manconá Kandé and Alfred Strohmeier. Towards a UML Profile for Software Architecture Descriptions. In Selic et al. [2000], pages 513–527.
- Gerti Kappel and Martin Hitz. *UML @ Work. Von der Analyse zur Realisierung*. dpunkt Verlag, 1999.
- Takuya Katayama, editor. *Proc. 6th Intl. Software Process Ws. "Support for the Software Process"*, Hakodate, Japan, October 1990. IEEE Computer Society Press.
- Marc I. Keller and H. Dieter Rombach. Comparison of Software Process Descriptions. In Katayama [1990], pages 7–18.
- Norman L. Kerth. Caterpillar's Fate: A Pattern Language for the Transformation from Analysis to Design. In Coplien and Schmidt [1995b], pages 293–320. Based on the proceedings of PLoP'94. Available also at www1.bell-labs.com/user/cope/Patterns/Process/index.html.
- Astrid Kiehn. *A Structuring Mechanism for Petri Nets*. Dissertation, TU München, 1989.
- Irving Kind and Richard Kind. BABEL: A Glossary of Computer Oriented Abbreviations and Acronyms, 2000. Available at www.geocities.com/ikind_babel/babel/babel.html.
- Ekkart Kindler. *Modularer Entwurf verteilter Systeme mit Petrinetzen*. Number 1 in Edition Versal. Dieter Bertz Verlag, Berlin, 1995.
- Ekkart Kindler. Database Theory—Petri Net Theory—Workflow Theory. *Petri Net Newsletter*, 5(5): 14–18, October 1998.
- Michael Kishinevsky, Jordi Cortadella, Alex Kondratyev, Luciano Lavagno, Alexander Taubin, and Alex Yakovlev. Coupling Asynchrony and Interrupts: Place Chart Nets. In Piére Azéma and Gianfranco Balbo, editors, *Intl. Conf. Applications and Theory of Petri Nets '97*, number 1248 in LNCS. Springer Verlag, 1997.
- Alexander Knapp. A Formal Semantics for UML Interactions. In France and Rumpe [1999], pages 116–130.
- Alexander Knapp. *A Formal Approach to Object-Oriented Software Engineering*. PhD thesis, Ludwig-Maximilians-Universität München, Institut für Informatik, May 2000.

- Konstantinos Kosmidis and Gábor Huszerl. UML Extensions for Quantitative Analysis. In Reggio et al. [2000], pages 70–75. Also appeared as Technical Report No. 0006 of the Ludwig-Maximilians-Universität, München, Fakultät für Informatik, October 2000.
- Bent Bruun Kristensen and Kasper Osterbye. Roles: Conceptual Abstraction Theory and Practical Language Issues. *Theory and Practice of Object Systems*, 2(3):143–160, 1996.
- Philippe B. Kruchten. The 4+1-view model of architecture. *IEEE Software*, November 1995. Available at www.rational.com/uml/resources/whitepapers/index.jhtml.
- Philippe B. Kruchten. The Software Architect—and the Software Architecture Team. In Donohoe [1999], pages 565–583.
- Robert Chi Tau Lai. A Process Modeling Approach and Notation. In Cooke [1994], pages 271–299.
- Kevin Lano. *Formal Object-Oriented Software Development*. Springer Verlag, 1995.
- Craig Larman. *Applying UML And Patterns. An Introduction to Object-Oriented Analysis and Design*. Prentice Hall, 1. edition, 1998.
- Meir M. Lehman. Programs, life cycles, and laws of software evolution. *IEEE Transactions on Software Engineering*, 68(9), September 1980.
- Johan Lilius and Ivàn Porres Paltor. Formalising UML State Machines for Model Checking. In France and Rumpe [1999], pages 430–445. Short version of Lilius and Paltor [1999b].
- Johan Lilius and Ivàn Porres Paltor. The Semantics of UML State Machines. Technical Report TUCS Technical Report No 273, Turku Centre for Computer Science, May 1999b.
- Frank J. van der Linden and Jürgen K. Müller. Creating Architectures with Building Blocks. *IEEE Software*, pages 51–60, November 1995.
- Barbara Liskov and Jeannette M. Wing. A Behavioral Notion of Subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, November 1994.
- Kenneth C. Loudon. *Programming Languages. Principles and Practice*. PWS-Kent, 1993.
- David C. Luckham, John J. Kenney, Larry M. Augustin, James Vera, Doug Bryan, and Walter Mann. Specification and Analysis of System Architecture using Rapide. *IEEE Transactions on Software Engineering*, 21(4):336–355, April 1995.
- Andrew Lyons. UML for Real-Time (Overview). Technical report, ObjecTime Inc., see www.objecttime.com, 1998.
- Jeff Magee, Naranker Dulay, and Jeff Kramer. Regis: A Constructive Development Environment for Distributed Programs. *Distributed Systems Engineering Journal*, 1(5):304–312, 1995.
- Jeff Magee and Jeff Kramer. Dynamic Structure in Software Architectures. In *Proc. Conf. Foundations of Software Engineering*. ACM, 1996.
- Marian Makins, editor. *Collins Softback English Dictionary*. HarperCollins Publishers, 3rd edition, 1991.

- Chris Marshall. *Enterprise Modeling with UML. Designing Successful Software through Business Analysis*. Addison-Wesley, 1999.
- James Martin and James J. Odell. *Object-Oriented Analysis and Design*. Prentice Hall, 1992.
- Masao Matsumoto, Satoshi Hayano, Takahiro Kudo, Hideo Yoshida, Shunzo Imai, and Kiyoshi Ohshima. Specifications Reuse Process Modelling and Case Study-Based Evaluations. In *Proc. 15th Annual Intl. Computer Software and Applications Conf. (COMPSAC'91)*, 1991.
- Friedemann Mattern, Gerd Meister, Marco Gruteser, and Thomas M. Schoch. CTI-Ansteuerung einer TK-Anlage über CORBA und Java Telephony API am Beispiel eines Least-Cost-Routers. Technical report, Technische Universität Darmstadt, FB Informatik, FG Verteilte Systeme, July 1998. In German, available at www.informatik.tu-darmstadt.de/VS.
- Carma McClure. *The Three Rs of Software Automation. Re-engineering, Repository, Reusability*. Prentice Hall, 1992.
- M.D. McIlroy. *Mass Produced Software Components*, pages 88–98. In , Naur and Randell [1969], 1969. Paper presented at the 1968 NATO conference in Garmisch.
- Nenad Medvidovic. *Architecture-Based Specification-Time Software-Evolution*. PhD thesis, University of California, Irvine, 1999.
- Nenad Medvidovic and Richard N. Taylor. A framework for classifying and comparing architecture description languages. In Mehdi Jazayeri and Helmut Schauer, editors, *Proc. 6th Eur. Software Engineering Conference, Workshop on Formal Methods in Software Engineering*, number 1301 in LNCS. Springer Verlag, 1997. also appeared in SE Notes, Vol. 22, No. 6, pages 60-75.
- John A. Mills. A Pragmatic View of the System Architect. *Communications of the ACM*, 28(7): 708–717, July 1985.
- Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- Daniel Moldt. *Systemspezifikation mit Petrinetzen*. Dissertation, FB Informatik, Universität Hamburg, 1996.
- Robert T. Monroe, Andrew Kompanek, Ralph Melton, and David Garlan. Architectural Styles, Design Patterns, and Objects. *IEEE Software*, 14(1):43–52, January 1997.
- Robert Moreton. A process model for software maintenance. *J. Information Technology*, 5:100–104, 1990. Reprinted in [Bohner and Arnold, 1996, p. 29-33].
- Pierre-Alain Muller and Jean Bézuvin, editors. *International Workshop «UML»'98: Beyond the Notation*. Ecole Supérieure des Sciences Appliquées pour l'Ingénieur—Mulhouse, Université de Haute-Alsace, 1998.
- National Institute of Standards and Technologies. Integration Definition for Function Modeling. Technical report, Computer Systems Laboratory, NIST, 1993. Available at www.omg.org/techprocess/sigs.html.
- Peter Naur and Brian Randell, editors. *Report on a conference sponsored by the Nato Science Committee, Garmisch, Germany, 7.-11. Oct. 1968*. NATO, 1969. 231 pages.

- Oscar Nierstrasz. The Next 700 Concurrent Object-Oriented Languages—Reflections on the Future of Object-Based Concurrency. In Dennis Tsichritzis, editor, *Object Composition*. Centre Universitaire d'Informatique, Université de Genève, 1991.
- Oscar Nierstrasz, editor. *Proc. Joint 6th Eur. Software Engineering Conf. and 6th ACM SIGSOFT Intl. Symp. Foundations of Software Engineering (ESEC/FSE'99)*, number 1687 in LNCS, 1998.
- Jörg Noack, Hamarz Mehmanesh, Homayoun Mehmanèche, and Andreas Zender. Architekturen für Network Computing. *Wirtschaftsinformatik*, 42(1):5–14, Februar 2000.
- Dietmar Ochs, Michael A. and Pfahl, Gunther Chrobok-Diening, and Beate Nothelfer-Kolb. A COTS Acquisition Process. Definition and Application Experience. IESE-Report 008.00/E, Fraunhofer Institute for Experimental Software Engineering, 2000. Available at <http://docserver.fhg.de/iese/2000/reports/008.pdf>.
- Bernd Oesterreich. *Objektorientierte Softwareentwicklung*. Oldenbourg, 1998. 4. aktualisierte Auflage, in German. Available in english under the title "Developing Software with UML: Object-Oriented Analysis and Design in Practice".
- OMG BODTF. Business Object Component Architecture. Technical report, OMG, 1998. Revised Proposal, Revision 1.2 of July, 6th, 1998. Available at www.omg.org/pub/docs/bom/98-07-01.pdf.
- OMG Issues DB. Unresolved Issues database, 2000. published under <http://cgi.omg.org/issues/uml-rtf.html>.
- OMG MOF 1.3. OMG Meta Object Facility MOF (version 1.3). Technical report, Object Management Group, March 2000. Available at www.omg.org.
- OMG UML 1.3. OMG Unified Modeling Language Specification (version 1.3). Technical report, Object Management Group, June 1998. Available at uml.shl.com.
- Gunnar Övergaard and Karin Palmkvist. Interacting Subsystems in UML. In Selic et al. [2000], pages 359–368.
- Jens Palsberg and Michael Schwartzbach. Three Discussions on Object-Oriented Typing. *ACM SIGPLAN OOPS Messenger*, 3(2):31–38, 1999.
- D. M. R. Park. Concurrency and automata on infinite sequences. In Peter Deussen, editor, *Proc. 5th GI Conference on Theoretical Computer Science*, number 104 in LNCS. Springer Verlag, 1980.
- Dwayne E. Perry and Alexander L Wolf. Foundations for the Study of Software Architecture. *SE Notes*, 17(4):40–52, October 1992.
- John L. Peterson. *Petri Net Theory and the Modeling of Systems*. Prentice Hall Inc., Englewood Cliffs NJ, 1981.
- Carl Adam Petri. *Kommunikation mit Automaten*. Schriften des Institutes für instrumentelle Mathematik, 1962.
- Cuno Pfister and Clemens Szyperski. Why Objects Are Not Enough. In Thomas Jell, editor, *Proc. 1st Intl. Component User Conference (CUC'96)*, Munich. Cambridge University Press, July 1998.

- Gustav Pomberger and Günther Blaschek. *Grundlagen des Software Engineering. Prototyping und objektorientierte Software-Entwicklung*. Hanser, 1993.
- Dick Pountain and Clemens Szyperski. Extensible Software Systems. *BYTE*, (5):87–62, May 1994.
- Wolfgang Pree. *Design Patterns for Object-Oriented Software Development*. Addison-Wesley, 1995.
- Ruben Prieto-Diaz. *Classification of Reusable Modules*, volume I - Concepts and Models, pages 99–124. ACM Press, 1989.
- Ruben Prieto-Diaz. Implementing Faceted Classification for Software Reuse. *Communications of the ACM*, 34(5), May 1991.
- Ruben Prieto-Diaz. *Historical overview*, chapter 1, pages 1–16. In , Schäfer et al. [1994], 1994. Proc. 1st Intl. Conf. Software Reusability, 1991.
- Rational RUP. The Rational Unified Software Development Process, 1999. Version 5.5 (Build 12).
- Eberhard Rechting. *System Architecting: Creating and Building Complex Systems*. Prentice Hall, 1991.
- Trygve Reenskaug, P. Wald, and O.A. Lehne. *Working with Objects*. Prentice Hall, 1995.
- Gianna Reggio, Alexander Knapp, Bernhard Rumpe, Bran Selic, and Roel Wieringa, editors. *Dynamic Behavior in UML Models: Semantic Questions. Workshop Proceedings*, Oktober 2000. Also appeared as Technical Report No. 0006 of the Ludwig-Maximilians-Universität, München, Fakultät für Informatik, October 2000.
- Björn Regnell, Michael Andersson, and Johan Bergstrand. A Hierarchical Use Case Model with Graphical Representation. In IEEE, editor, *IEEE Symp. and Ws. on Engineering of Computer-based Systems, ECBS96*, pages 270–277. IEEE Computer Press, 1997.
- Wolfgang Reisig. *Petri-Nets: an Introduction*, Springer Verlag. Springer Verlag, 1985.
- Wolfgang Reisig. *A Primer in Petri Net Design*. Springer Verlag, 1992.
- Mark Richters and Martin Gogolla. Validating UML Models and OCL Constraints. In Selic et al. [2000].
- Dirk Riehle. *Framework Design. A Role Modeling Approach*. PhD thesis, ETH Zürich, 2000. Available at www.riehle.org.
- Peter J. Robinson. *Hierarchical Object-Oriented Design*. Prentice Hall, 1992.
- James Rumbaugh, Ivar Jacobson, and Grady Booch. *The Unified Modeling Language Reference Manual*. Addison-Wesley, 1999.
- Bernhard Rumpe. *Formale Methodik des Entwurfes verteilter objektorientierter Systeme*. PhD thesis, TU München, Institut für Informatik, 1996.
- Wilhelm Schäfer, Ruben Prieto-Diaz, and Masao Matsumoto, editors. *Software Reuse*. Ellis Horwood, 1994. Proc. 1st Intl. Conf. Software Reusability, 1991.

- Karsten Schmidt. *Symbolische Analysemethoden für algebraische Petrinetze*. Dieter Bertz Verlag, 1996. Edition Versal, Nr. 4.
- Manfred Schneider. Architecture and Design: Unified Modeling Language (UML), June 2000a. Available at www.cetus-links.de/oo_uml.html.
- Manfred Schneider. CETUS Links, 2000b. Available at www.cetus-links.com.
- Bran Selic. Turning Clockwise: Using UML in the Real-Time Domain. *Communications of the ACM*, 42(10):46–54, October 1999.
- Bran Selic, October 2000. Speech delivered at the conference banquet of UML'00.
- Bran Selic, Garth Gullekson, and Paul T. Ward. *Real Time Object Oriented Modeling*. John Wiley & Sons, 1994.
- Bran Selic, Stuart Kent, and Andy Evans, editors. *Proc. 3rd Intl. Conf. «UML» 2000—Advancing the Standard*, number 1939 in LNCS, October 2000. Springer Verlag.
- Bran Selic and James Rumbaugh. Using UML for Modeling Complex Real-Time Systems. Technical report, Rational Software Corp. & ObjecTime Ltd., 1998.
- Mary Shaw. Comparing Architectural Design Styles. *IEEE Software*, (11):27–41, 1995a.
- Mary Shaw. Patterns for Software Architectures. In Coplien and Schmidt [1995b], pages 453–466. Based on the proceedings of PLoP'94. Available also at www1.bell-labs.com/user/cope/Patterns/Process/index.html.
- Mary Shaw. Some Patterns for Software Architecture. In Coplien and Schmidt [1995a]. Available via <http://st-www.cs.uiuc.edu/~plop/>.
- Mary Shaw and Paul C. Clements. A Field Guide to Boxology. Preliminary Classification of Architectural Styles for Software Systems. Technical report, Carnegie Mellon University, Software Engineering Institute, April 1996.
- Mary Shaw, Robert DeLine, Daniel V. Klein, Theodore L. Ross, David M. Young, and Gregory Zelenik. Abstractions for Software Architecture and Tools to Support Them. *IEEE Transactions on Software Engineering*, 21(4):314–335, April 1995.
- Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996.
- Alan R. Simon. *The Integrated CASE-Tools Handbook*. Van Nostrand Reinhold, 1993.
- H.R. Simpson. Real-Time Network Architecture. In *Integrated Design and Process Technology, IDPT*, 1998.
- Oliver Sims. *Business Objects—Delivering Cooperative Objects for Client-Server*. McGraw-Hill, 1994.
- Ian Sommerville. *Software Engineering*. Addison-Wesley, 1996. 5th ed.
- Bridget Spitznagel and David Garlan. Architecture-Based Performance Analysis. In Lofti A. Zadeh, editor, *Proc. 1998 Conf. Software Engineering and Knowledge Engineering (SEKE'98)*, June 1998.

- Hans-Peter Starke. *Analyse von Petri-Netz-Modellen*. Leitfäden und Monographien der Informatik. Teubner, Stuttgart, 1990.
- Perdita Stevens. Advanced Tools for UML: now and in the future. slides of a tutorial given at UML'00, October 2000. Available at www.dcs.ed.ac.uk/home/pxs/talksEtc.html.
- Perdita Stevens and Rob Pooley. Systems Reengineering Patterns. In Nierstrasz [1998]. Available at www.dcs.ed.ac.uk/home/pxs/talksEtc.html.
- Perdita Stevens and Rob Pooley. *Using UML: Software Engineering with Objects and Components*. Addison-Wesley, 2000. 2nd edition.
- Harald Störrle. An Evaluation of High-End Tools for Petri-Nets. Technical Report 9802, Ludwig-Maximilians-Universität München, Institut für Informatik, 1998.
- Harald Störrle. A different notion of components. In Andreas Schürr and Peter Hofmann, editors, *Object-Oriented Modelling of Real-Time Systems (OMER'99)*. Universität der Bundeswehr, München, Fakultät für Informatik, Bericht Nr. 1999-01, 1999a.
- Harald Störrle. A Petri-Net Semantics for Sequence Diagrams. In Katharina Spies and Bernhard Schätz, editors, 9. *GI/ITG Fachgespräch Formale Beschreibungstechniken für verteilte Systeme (FBT'99)*, June 1999b.
- Harald Störrle. Architectural modeling with the UML. In Jan Bosch, editor, *Nordic Workshop on Software Architectures (NOSA'99)*, 1999c.
- Wolfgang Thomas. Automata on infinite objects. In Jan van Leeuwen, editor, *Handbook of Theoretical Computer Science*, volume 2, pages 133–192. Elsevier, 1990.
- Colin Tully, editor. *Proc. 4th Intl. Software Process Ws.: Representing and Enabling the Software Process*, Moretonhampstead, Devon, UK, May 1988. ACM Press.
- Josée Turgeon and Nazim H. Madhavji. View-Based vs. Traditional Modeling Approaches: Which is Better? Number 1780 in LNCS, pages 131–137. Springer Verlag, 2000.
- Jon Udell. Component Software. *BYTE*, (5), 1994.
- UML Revision Task Force. UML 2.0 Working Group Home Page, September 2000. Available at www.celigent.com/omg/adptf/wgs/uml2wg.htm.
- Ernst Uthke. Das V-Modell. Allgemeiner Umdruck Nr. 250: Vorgehensmodell Planung und Durchführung von IT-Vorhaben (Entwicklungsstandard für IT-Systeme des Bundes), 1997. Available at <http://www.v-modell.iabg.de/>.
- Robert Valette. Analysis of Petri Nets by Stepwise Refinement. *Journal of Computer and Systems Science*, 18:35–46, 1979.
- Anti Valmari. State of the art report: Stubborn sets. *Petri Net Newsletter*, 4(6), 1993.
- Walter Vogler. *Modular construction and partial order semantics of Petri Nets*. Number 625 in LNCS. Springer Verlag, 1992.

- Greg Voss. Java Beans, 2000. Available at <http://developer.java.sun.com/developer/onlineTraining/Beans/Beans1/index.html>.
- Thomas Weigert, Cris Kobryn, Bran Selic, Birger Møller-Pedersen, John Knapman, and James Carter. Modeling of Architectures with UML. In Selic et al. [2000], pages 556–570.
- Victor Weinberg. *Structured Analysis*. Prentice Hall, 1978.
- Timothy D. Wells. A Pattern Language for Dynamic Project Management. In Coplien and Schmidt [1995a]. Available via <http://st-www.cs.uiuc.edu/~plop/>.
- WfMC. WfMC Web page, 2000. Available at www.aai.org/wfmc.
- David A. Wheeler, Bill Brykczynski, and Reginald N. Jr. Meeson, editors. *Software Inspection. An Industry Best Practice*. IEEE Computer Society Press, 1996.
- Bruce Whitenack. RAPPeL: A Requirements-Analysis-Process Pattern Language. In Coplien and Schmidt [1995b], pages 259–292. Based on the proceedings of PLoP’94. Available also at www1.bell-labs.com/user/cope/Patterns/Process/index.html.
- Niklaus Wirth. *Programming in Modula-2*. Springer, 2nd edition edition, 1983.
- Martin Wittmann. Erstellung eines OCL-Interpreters. Master’s thesis, Institut für Informatik, Universität München, July 2000.
- WWISA. Charter: Role of the Architect. Technical report, World Wide Institute of Software Architects (WWISA), 1997. See www.wisa.org.
- Andreas Zendler. *Konzepte, Erfahrungen und Werkzeuge zur Software-Wiederverwendung*. Tectum Verlag, Reihe Softwaretechnik, Nr. 1, 1997. Schriftenreihe des FAST.
- Andreas Zendler. Foundation of the Taxonomic Object System. *Information and System Sciences*, 40: 475–492, 1998.
- Andreas Zendler. Komponentenorientierte Entwicklung im Software Life Cycle. Talk given on the 6.12.1999 to Munich local chapter of the Gesellschaft für Informatik, December 1999.

Index

- van der Aalst et al., 194, 285
- Abowd et al., 46, 285
- Abstraction
 - «derive», **68**
 - heterogeneous, **189**
 - homogenous, **189**
 - «realize», **68**
 - «refine», **68**
 - «trace», **68**
- Abstraction, **68**, 97, 116
- ACD (activity diagram), **267**
- Action, **118**
- action, **138**
- ActionState, **198**
- active class, **20**, 60, 238
- Activity, **198**
- activity, **193**
- ActivityGraph, **268**
- ADL (Architecture Description Language), **2**
- Alexander, 194, 275, 285
- Alexander et al., 275, 285
- Allen, 19, 21, 44, 285
- Allen and Garlan, 14, 19, 285
- Allen et al., 19, 285
- Allweyer and Loos, 77, 268, 273, 285
- Ambler, 194, 285
- AML (architectural modeling language), **18**
- Amyot and Mussbacher, 233, 285
- Andersen, 23, 25, 276, 285
- architectural instance diagram, **75**, 269
- architectural modeling language, **18**
- architectural pattern, **199**
- architectural structure diagram, **75**, 269
- architectural style, **46**
- architectural unit, **5**, **44**, 45
- Architecture, **66**
- Architecture Description Language, **2**
- Architecture Working Group, **2**
- Arlow et al., 77, 285
- ASD (architectural structure diagram), **267**
- Association, **63**
- AssociationEnd, **63**
- AssociationEndRole, **276**
- AssociationRole, **276**
- Atkinson et al., 25–27, 285
- ATM (automated teller machine), **83**, 228
- automated teller machine, **83**
- AWG (Architecture Working Group), **2**
- Baeten and Weijland, 132, 285
- Balzert, 210, 286
- bandwidth, **63**
- Banking Support System, **227**
- Barros et al., 57, 286
- Baumgarten, 126, 275, 286
- Bause and Kritzing, 170, 174, 286
- Bayer et al., 25–27, 194, 286
- behavioral type, **67**
- Bergner et al., 194, 195, 286
- Biggerstaff and Richter, 5, 42, 286
- binding, **57**, 78, 214
- BO (Business Object), **14**
- BOCA (Business Object Component Architecture), **15**
- Boehm, 40, 258, 259, 286
- Boehm and Belz, 33, 286
- Bohner and Arnold, 41, 286, 294
- Booch et al., 43, 263, 267, 274, 277, 278, 286
- Bosch et al., 32, 286
- branching, **129**
- Braun, 2, 3, 5, 25–27, 193, 194, 286
- Bronsard et al., 13, 286
- Brooks, 39, 52, 105, 258, 259, 286
- Brown, 272, 286, 287
- Buhr, 36, 57, 72, 76, 77, 233, 286
- Buhr and Casselman, 72, 233, 287
- Buschmann et al., 16, 26, 46, 89, 91, 94, 194–196, 199, 274–276, 285, 287
- Business Object, **14**
- Business Object Component Architecture, **15**

- C2 (Components-and-Connectors), **19**
- C2/SADL, **260**
- Capability Maturity Model, **193**, 194, 287
- Capsule, **52**, 117
- CapsuleInstance, **66**, 116, 117
- CapsuleRole, **54**
- CASE (Computer Aided Software Engineering), **4**
- Cattaneo, 13, 20, 21, 287
- CBD (collaboration diagram), **267**
- CBO (Cooperative Business Object), **15**
- CDL (Component Description Language), **15**
- channel, **228**
- characteristics, **61**, 63, 120
- Chen and Scheer, 273, 287
- Christensen and Mortensen, 275, 287
- Clark et al., 270, 287
- class diagram, **75**
- ClassifierRole, **276**
- CLD (class diagram), **267**
- Clements and Northrop, 13, 276, 287
- Clements et al., 31, 287
- client/server-architecture, **82**
- CMM (Capability Maturity Model), **193**, 194, 287
- CMU SEI, 13, 276, 287
- CMU SEI CMM, 193, 194, 287
- Coleman et al., 193, 268, 287
- Collaboration, **276**
- colored Petri-nets, **22**
- COM, **5**, 15, 35
- Commercial off-the-shelf, **5**
- Common Object Request Broker Architecture, **89**
- Common Warehouse Model, **264**
- complete, **189**
- ComplexConnector, **62**
- component, 205
- Component Description Language, **15**
- component diagram, **240**
- Components-and-Connectors, **19**
- CompositeState, **118**, 198
- compound transition, **121**
- Computer Aided Software Engineering, **4**
- concurrent engineering, **6**
- Conelly, 15, 287
- Configuration, **61**
- ConfigurationSpace, **63**, 116
- Connector, **62**
 - binding, **182**
 - bridge, **184**
- Connector, **117**
- ConnectorEnd, **62**, 117
- ConnectorEndInstance, **66**
- ConnectorEndRole, **64**, 117
- ConnectorInstance, **66**
- ConnectorRole, **117**
- context, 33, **36**, 39, 41, **44**, 44, 45
- contract, **60**
- ControlState, **198**
- Cook, 264, 287
- Cook and Daniels, 193, 287
- Cooke, 193, 287, 288, 293
- Cooperative Business Object, **15**
- Coplien, 194, 288
- Coplien and Schmidt, 194, 195, 288–290, 292, 297, 299
- CORBA (Common Object Request Broker Architecture), **89**
- COTS (Commercial off-the-shelf), **5**
- Couvreux et al., 129, 288
- Cox, 39, 288
- CPD (component diagram), **267**
- Crawley et al., 264, 288
- customers, **32**
- CWM (Common Warehouse Model), **264**
- D'Souza and Wills, 3, 13, 14, 24, 31, 34, 35, 193, 194, 277, 288
- Davis and Williams, 43, 278, 288
- deferred, **158**
- Deiters et al., 193, 288
- Dependency, **68**, 168
- design debugging, **40**
- design pattern, **199**
- Design/CPN, **158**, 159, 162
- developers, **31**
- development process, **193**
- Dewal and Schnichels, 227, 288
- Diagram, **58**
- diagram, **267**
 - activity, **267**
 - capsule, **73**
 - class, **267**

- collaboration, **267**
- component, **267**
- deployment, **267**
- interaction, **267**
- package, **267**
- sequence, **267**
- state transition, **267**
- state chart, **267**
- static structure, **267**
- use case, **267**
- workflow, **267**
- doActivity, **118**
- document, 198
- Document type definition, **50**
- Donohoe, 276, 288, 290, 293
- DPD (deployment diagram), **267**
- DTD (Document type definition), **50**
- EBNF (Extended Backus Naur Form), **111**
- eEPCs (extended Event-Process-chains), **268**
- Egyed and Medvidovic, 20, 21, 288
- embedding, **189**
- Englander, 244, 288
- ESA (European Space Agency), **20**
- Estrin et al., 18, 72, 75, 130, 288
- European Space Agency, **20**
- event, **244**
- event space, **147**, 184
- Extended Backus Naur Form, **111**
- extended Event-Process-chains, **268**
- Extensible markup language, **50**
- facette, **42**, 196
- feature interaction, **232**
- Fencott, 132, 288
- filter, **234**
- FinalState, **118**
- Firley et al., 23, 25, 288
- Fisher, 259, 272, 288
- Fjeldstad and Hamlen, 258, 289
- Flanagan, 243, 244, 289
- Flatscher, 15, 289
- Foote, 194, 202, 289
- Foote and Opdyke, 3, 193, 194, 202, 289
- Foote and Yoder, 194, 289
- Fowler, 275, 289
- France and Rumpe, 288, 289, 292, 293
- Frank and Halter, 16, 289
- Freeman, 5, 289
- Gamma et al., 26, 194–196, 274, 275, 289
- Garlan and Kompanek, 20, 289
- Garlan et al., 19, 44, 289
- Gehrke et al., 23, 25, 289
- Ghezzi et al., 258, 289
- Glabbeek, 129, 133, 289, 290
- glue, **45**
- GMD, 193, 290
- goal, **36**, 79, 83, 208
- Godefroid, 129, 290
- Goldberg, 31, 34, 194, 290
- Graham et al., 156, 290
- Griss et al., 33, 34, 290
- group transitions, **146**
- Hacklinger, 202, 290
- Haddad, 129, 290
- Hammer, 13, 35, 42, 273, 278, 290
- Harandi, 286, 288, 290
- Harel, 268, 277, 290
- Harrison, 194, 290
- Hesse, 3, 25–27, 193, 194, 202, 290
- Hierarchical OO Design (HOOD), **20**
- hierarchical use case, **77**
- high-level Petri-net, **156**, 158
- high-level transition, **146**, 151
- Hofmeister et al., 20, 21, 290
- home marking, **127**
- HOOD, 2, **20**
- Horn et al., 35, 290
- hotspot, **47**
- Huszerl and Majzik, 215, 232, 291
- Hußmann et al., 144, 290
- IAD (interaction diagram), **267**
- ICC, **85**
- IDEF (Integration Definition for Function Modeling), **26**
- idiom, 199
- IEEE Architecture Working Group, **14**
- IEEE Architecture Working Group, 4, 14, 31, 35, 36, 42, 50, 56, 57, 272, 278, 291
- IEEE AWG (IEEE Architecture Working Group), **14**
- inspection, **40**

- Instance, **66**
- Integration Definition for Function Modeling, **26**
- Interface, **243**
- interleaving, **129**
- ISO RM-ODP, 19, 43, 278, 291
- Jacobson et al., 3, 24, 26, 43, 79, 193, 197, 202, 210, 214, 269, 277, 278, 291
- Jantzen, 128, 291
- Java
 - Beans, 5, 15, 35, 243
 - Java Telephony API, **89**
 - Virtual Machine, **244**
- Jazayeri, 286, 291
- Jensen, 129, 292
- Jensen and Rozenberg, 126, 158, 292
- Jessen and Valk, 129, 292
- Johnson and Foote, 274, 292
- join, **122**
- JTAPI, **89**
- JVM, **244**
- Jäger et al., 197, 291
- Kandé and Strohmeier, 20, 21, 292
- Kappel and Hitz, 277, 292
- Katayama, 194, 292
- Keller and Rombach, 197, 292
- Kerth, 194, 292
- Kiehn, 126, 128, 129, 133, 292
- Kind and Kind, 271, 292
- Kindler, 110, 192, 292
- Kishinevsky et al., 149, 292
- Knapp, 23–25, 53, 257, 292
- Kosmidis and Huszerl, 215, 232, 292
- Kristensen and Osterbye, 276, 293
- Kruchten, 1, 43, 263, 267, 278, 293
- Lai, 193, 293
- Lano, 6, 22, 25, 293
- Larman, 4, 14, 277, 293
- layers architecture, **94**
- LCR (Least cost router), **83**
- Least cost router, **83**
- Lehman, 194, 258, 293
- life-cycle, **52**
- Lilius and Paltor, 23, 25, 257, 293
- van der Linden and Müller, 17, 293
- Lines-of-code, **170**
- Liskov and Wing, 17, 67, 108, 293
- LOC (Lines-of-code), **170**
- Louden, 15, 293
- low-level, **206**
- Luckham et al., 13, 14, 19, 72, 130, 293
- Lyons, 16, 20, 72, 293
- Magee and Kramer, 72, 293
- Magee et al., 19, 293
- Makins, 274, 293
- managers, **31**
- Mapping, **69**
- mappingExpression, **69**
- Marshall, 31, 36, 57, 276, 293
- Martin and Odell, 195, 294
- Matsumoto et al., 5, 16, 36, 194, 294
- Mattern et al., 83–85, 87, 89, 93, 294
- McClure, 5, 34, 289, 294
- McIllroy, 5, 14, 272, 294
- Medvidovic, 16, 19–21, 46, 294
- Medvidovic and Taylor, 18, 294
- Message, **139**
- Message Sequence Chart, **19**
- Meta Object Facility, **264**
- method, **244**
- Mills, 1, 294
- Milner, 132, 133, 294
- MOF, **112**, 264
- MOF (Meta Object Facility), **264**
- Moldt, 22, 25, 294
- Monroe et al., 215, 294
- Moreton, 3, 193, 294
- MSC (Message Sequence Chart), **19**
- Muller and Bézivin, 285, 294
- multiplicity, **55**, 61
- Namespace, **53**, 116
- National Institute of Standards and Technologies, 26, 294
- Naur and Randell, 294
- Nierstrasz, 17, 291, 294, 295, 298
- Noack et al., 222, 250, 295
- OAG (Open Applications Group), **15**
- OBD (object diagram), **267**
- Object Constraint Language, **267**
- object diagram, 75, 240

- Object Management Group, **263**
- ObjectFlowState, **198**, 269
- Ochs et al., 25, 26, 295
- OCL (Object Constraint Language), **267**
- Oesterreich, 277, 295
- Övergaard and Palmkvist, 16, 17, 295
- OMG (Object Management Group), **263**
- OMG BODTF, 15, 17, 295
- OMG Issues DB, 139, 295
- OMG MOF 1.3, 264, 295
- OMG UML 1.3, 10, 15, 16, 21, 46, 50, 53, 64, 65, 68, 69, 71, 72, 74, 76, 103, 109, 115, 118, 121, 122, 139, 141, 144–147, 154, 156, 159, 169, 197, 263, 265, 267–270, 272, 275–277, 295
- Open Applications Group, **15**
- P1471, 2, 3, 20, **42**
- Package, **53**, 55
- package diagram, **75**
- Palsberg and Schwartzbach, 108, 295
- paper prototype, **235**
- Park, 132, 295
- pattern, **46**, 194
 - functionality analysis, 233
- PBX (private branch exchange), **84**
- Perry and Wolf, 4, 13, 46, 52, 272, 276, 295
- Peterson, 170, 275, 295
- Petri, 275, 295
- Pfister and Szyperski, 16, 43, 274, 295
- phase, **4**
- PKD (package diagram), **267**
- pla, **205**
- PLA (Product Line Architecture), **47**
- POI (point-of-interaction), **45**
- point of sale, **82**
- point-of-interaction, **45**
- Pomberger and Blaschek, 199, 206, 295
- Port, **60**, 116
 - active, **60**, 71, 71
 - binary, **62**
 - control, **60**, 71
 - document-type, **60**, 71
 - filter, **60**, 71, 234
 - free, **60**, 62
 - passive, 60, **71**, 71
 - relay, **60**, 60, 71, 234, 235, 238, 240
 - symmetric, **62**
 - transformer, **60**, 71, 234
 - unidirectional, **60**, 62, 71
- PortInstance, **66**, 116
- PortKind, **61**
- POS (point of sale), **82**
- Pountain and Szyperski, 16, 296
- Pree, 275, 296
- PresentationElement, **58**, 115
- Prieto-Diaz, 5, 42, 274, 289, 296
- private branch exchange, **84**
- process, **193**
- process pattern, **24**
- product line, **1**, 37
- Product Line Architecture, **47**
- programming component, 5, 35, 243
- project, 205
- property, **244**
- Protocol, **67**, 116
- ProtocolRole, **67**
- PseudoState
 - fork, **122**
 - choice, **121**
 - deepHistory, **121**
 - initial, **121**
 - shallowHistory, **121**
- quality of service, **62**
- Rational RUP, **3**, 193, 296
- rationale, **45**, 77
- Real Time Object Oriented Modeling, **15**
- Rechting, 35, 296
- Reenskaug et al., 276, 296
- Reference Model of Open Distributed Processing, **19**
- Reggio et al., 23, 293, 296
- regions, **146**, 214
- Regnell et al., 36, 57, 77, 296
- Reisig, 126, 128, 129, 170, 275, 296
- Relationship, 116, **169**
- Remote Method Invocation, **62**
- Remote Procedure Call, **62**
- result, **193**
- reuse, 31, 33, **36**
- Richters and Gogolla, 144, 296
- Riehle, 276, 296

- RM-ODP (Reference Model of Open Distributed Processing), **19**
- RMI (Remote Method Invocation), **62**
- Robinson, 16, 20, 26, 199, 296
- role, **55**, **61**
 - component, **52**, **55**
 - pla, **52**, **55**
 - project, **52**, **55**
 - style, **52**, **55**
 - virtual, **52**, **55**
- role model, **46**
- ROOM (Real Time Object Oriented Modeling), **15**
- RPC (Remote Procedure Call), **62**
- RTC-step, **155**
- Rumbaugh et al., 277, 296
- Rumpe, 23, 25, 296
- run place, **159**
- run-to-completion, **153**
-
- S (communicated Signals), **152**
- SADL (Simple ADL), **19**
- SADT (Structured Analysis and Design Technique), **195**
- safety, **63**
- SARA (System Architect's Apprentice), **18**
- Schmidt, 129, 192, 296
- Schneider, 13, 20, 271, 277, 297
- Schäfer et al., 5, 42, 290, 296
- SDL (Specification and Description Language), **18**
- SDL 2000, 19, 291
- SEI (Software Engineering Institute), **13**
- Selic, 20, 261, 297
- Selic and Rumbaugh, 16, 20, 21, 72, 297
- Selic et al., 3, 4, 15, 26, 276, 285–287, 289, 291, 292, 295–297, 299
- Shaw, 46, 89, 297
- Shaw and Clements, 46, 52, 297
- Shaw and Garlan, 276, 297
- Shaw et al., 72, 297
- short-circuit, **183**
- SignalEvent, **138**
- Simon, 8, 272, 297
- Simple ADL, **19**
- simple class, **196**
- Simpson, 72, 297
-
- Sims, 15, 297
- small workflow diagram, **198**
- software architect, **31**
- software architecture, **45**
- software crisis, **v**
- software engineering components, **5**
- Software Engineering Institute, **13**
- Sommerville, 193, 297
- source, **118**
- Specification and Description Language, **18**
- Spitznagel and Garlan, 215, 297
- SQD (sequence diagram), **267**
- SQL (Structured Query Language), **234**
- SSD (static structure diagram), **267**
- stakeholder, 4, 9, **31**, **38**
- Starke, 170, 297
- State, **118**
 - Composite, **118**
 - Final, **118**
 - Pseudo, **113**, **114**
 - Simple, **118**
 - Stub, **118**
 - top, **118**, **145**
 - Vertex, **118**, **146**
- state-space explosion problem, **191**
- StateMachine, 118, **144**, **268**
- static structure diagram, **75**, **269**
- STD (state transition diagram), **267**
- Stevens, 261, 298
- Stevens and Pooley, 194, 222, 277, 298
- Stimulus, **139**
- storyboard, 199, 210, **235**, **235**, **239**
- Structured Analysis and Design Technique, **195**
- Structured Query Language, **234**
- StubState, **120**
- style, **205**
- Störrle, 5, 110, 298
- SubSystem, **60**, **238**, **243**
- SWD (small workflow diagram), **198**, **267**
- System Architect's Apprentice, **18**
-
- target, **118**
- Telephony Services API, **88**
- Thomas, 132, 298
- transformer, **234**
- Transition, **118**, **198**

- transparent formality, **6**, 255
- trigger, **118**, 138, 139
- true concurrency, **129**
- TSAPI (Telephony Services API), **88**
- Tully, 194, 286, 298
- Turgeon and Madhavji, 43, 298
- UCD (use case diagram), **267**
- UCM (use case map), **232**
- Udell, 16, 298
- UML, **2**
 - 1.4, **263**
 - 2.0, **263**
 - UML-RT, **20**
- UML Revision Task Force, 270, 298
- Unified Process, **197**
- UP (Unified Process), **197**
- usage, **169**
- use case map, **232**
- users, **32**
- Uthke, 193, 298
- V-Model, **193**
- Valette, 129, 298
- Valmari, 129, 298
- View, **55**, 56
- view, 35, **42**, 56
 - administration, 42, **57**
 - behavior, 42, **57**, 79
 - budget, **81**
 - context, **83**
 - deployment, **57**, 79
 - domain, **57**
 - environment, 42, **58**, 81
 - execution, **57**
 - functionality, 42, **57**, 83
 - goals, **57**
 - implementation, 42, **57**, 79
 - interfaces, 42, **57**, 79
 - schedule, **81**
 - structure, 42, **57**, 79, 83, 94, 97
- viewpoint, 35, **42**, 56
- virtual unit, **234**
- Vogler, 129, 132, 298
- Voss, 244, 298
- walk-through, **40**
- WED (window event diagram), **235**, **267**
- Weigert et al., 299
- Weinberg, 258, 299
- Wells, 194, 299
- WFD, **197**
- WFD (workflow diagram), **267**
- WfMC, 271, 299
- WfMC (Workflow Management Coalition), **271**
- Wheeler et al., 104, 299
- Whitenack, 194, 299
- window event diagram, **235**
- Wirth, 15, 299
- Wittmann, 144, 299
- workflow diagram (WFD), **197**
- Workflow Management Coalition, **271**
- WWISA, 1, 2, 299
- XML, **265**
- XML (Extensible markup language), **50**
- Zendler, 5, 17, 36, 37, 42, 214, 299

Curriculum vitae

10.5.69	Born in Hamburg
1975-1979	Elementary school (Grundschule Glashütte, Norderstedt, Schleswig-Holstein)
1979-1988	Grammar school (Gymnasium im Schulzentrum Süd, Norderstedt, Schleswig-Holstein)
1990-1997	First degree (Dipl. Inform.) in Computer Science (Informatik) at the Universität Hamburg
1997-2000	PhD. student at the Ludwig-Maximilians-Universität München